

Tập huấn IAIO

Eljakim Schrijvers

Tháng 1, 2026

Tác phẩm này được cấp phép theo **Giấy phép Quốc tế Creative Commons Ghi công – Phi thương mại 4.0** (CC BY-NC 4.0).

Bạn được tự do:

- **Chia sẻ:** sao chép và phân phối lại tài liệu dưới bất kỳ phương tiện hay định dạng nào
- **Sửa đổi:** pha trộn, chuyển đổi và xây dựng dựa trên tài liệu **Theo các điều khoản sau:**
- **Ghi công:** Bạn phải ghi nhận đầy đủ tác giả gốc
- **Phi thương mại:** Bạn không được sử dụng tài liệu này cho mục đích thương mại

Toàn văn giấy phép:

<https://creativecommons.org/licenses/by-nc/4.0/>

Ngoại lệ: Bản nhạc *I Am Still Mine* (Phụ lục I) thuộc bản quyền © 2026 Eljakim Schrijvers, mọi quyền được bảo lưu, và không thuộc phạm vi của giấy phép Creative Commons nêu trên. Khuyến khích trình diễn phi thương mại kèm ghi công cho tác giả.

Feedback: eljakim+ai@gmail.com

© 2026 Eljakim Schrijvers

Lời nói đầu

Tôi vẫn là của tôi, và chính vì thế, tôi thuộc về bạn để giữ lấy.
Eljakim Schrijvers, *I Am Still Mine*

Tôi nên mở đầu bằng một lời thú tội: tôi không phải là một nhà nghiên cứu AI.

Điều tôi là là một người yêu thuật toán, lập trình, và giúp đỡ thanh niên khám phá niềm vui của tư duy tính toán. Tôi là một trong những huấn luyện viên của đoàn Hà Lan tại Kỳ Olympic Quốc tế Tin học (IOI), và năm 2024 tôi có vinh dự đưa Kỳ Olympic Tin học Châu Âu nữ (EGOI) về Hà Lan. Lập trình cạnh tranh là niềm đam mê của tôi đã nhiều năm. Có điều gì đó thỏa mãn sâu sắc khi nhìn khuôn mặt một học sinh bừng sáng lên khi một lời giải thanh tú bất ngờ khớp vào nhau.

Tôi đã học về mạng nơ-ron vào thập niên 1990, thời mà một mạng “sâu” chỉ có ba lớp. Quay trở lại lĩnh vực này sau hàng chục năm, tôi thấy nền tảng về cơ bản không hề thay đổi. Điều thay đổi một cách chấn động lại là mọi thứ khác: năng lực tính toán, dữ liệu, và quy mô khổng lồ mà những ý tưởng cũ này nay vận hành. Khác biệt giữa một mạng nơ-ron năm 1995 và một mạng năm 2025 không nằm ở sơ đồ kiến trúc trên bảng trắng; nó nằm ở hàng nghìn GPU và hàng tỷ tham số đã thổi sức sống cho sơ đồ đó.

Cuốn sách này là nỗ lực của tôi để cô đọng những gì tôi đã học thành một cẩm nang tập huấn dành cho học sinh. Đây không phải là một bài báo khoa học: không có danh mục tài liệu tham khảo ở cuối sách. Mục tiêu là giúp bạn hiểu và áp dụng các khái niệm AI, chứ không phải để truy vết nguồn gốc tri thức của chúng. Tôi đã cố gắng hết sức trình bày các ý tưởng một cách chính xác và công bằng, nhưng tôi đã ưu tiên sự rõ ràng hơn là quy ước học thuật.

Trong suốt quá trình viết cuốn sách này, tôi đã dùng các công cụ AI như một trợ lý văn bản. Các ý tưởng, cấu trúc và lựa chọn từ vựng là của tôi; AI đã giúp tôi diễn đạt chúng một cách rõ ràng và nhất quán hơn so với những gì tôi có thể tự làm trong giới hạn thời gian của mình.

Trong khi viết, tôi thấy mình suy ngẫm về AI — không chỉ như tư liệu ôn thi mà còn ở chiều sâu cá nhân. Ý nghĩa gì khi một thuật toán dự đoán những gì bạn sẽ nói trước khi bạn kịp nói ra? Khi nó học được thói quen của bạn rồi tiếp lời bạn? Tôi đã viết một bài hát về điều đó. Lời nhạc và giai điệu là của tôi; tôi chỉ dùng AI làm một cuốn từ điển vần rất kém. Bạn sẽ tìm thấy bản nhạc trong Phụ lục I. Nó không nên thuộc bất kỳ kỳ thi nào. Nhưng tôi tin mỗi câu chuyện hay đều xứng đáng có một bài hát. Nó giúp phần quan trọng nhất dễ nhớ hơn. Vì thế hãy biểu diễn nó, chơi nó, ngâm nga nó. Biến nó thành của chúng ta.

Nếu bạn có phản hồi, sửa lỗi hay gợi ý, tôi rất muốn được nghe từ bạn. Hãy gửi email tới eljakim+ai@gmail.com. Tôi không thể đảm bảo sẽ trả lời, nhưng tôi sẽ đọc mọi tin nhắn.

Chúc bạn may mắn.

Eljakim Schrijvers
January 2026

Contents

Preface	3
Introduction	19
I Foundations	25
1 Introduction to Artificial Intelligence	27
1.1 What is Artificial Intelligence?	27
1.2 A Brief History of AI	28
1.2.1 The Birth of AI	29
1.2.2 The Perceptron and Its Limits	29
1.2.3 Expert Systems and the Second Winter	30
1.2.4 The Backpropagation Revival	30
1.2.5 The Statistical Turn	31
1.2.6 Big Data and GPUs	31
1.2.7 The Deep Learning Revolution	31
1.2.8 Transformers and Large Language Models	32
1.3 Three Types of AI	32
1.3.1 Narrow AI (Artificial Narrow Intelligence)	33
1.3.2 Artificial General Intelligence (AGI)	33
1.3.3 Artificial Superintelligence (ASI)	34
1.4 Key Technologies in AI	34
1.4.1 Machine Learning	34
1.4.2 Deep Learning	35
1.4.3 Computer Vision	36
1.4.4 Natural Language Processing	37
1.5 The AI Development Pipeline	37
1.5.1 Step 1: Collect Data	38
1.5.2 Step 2: Train the Model	38
1.5.3 Step 3: Validate	39
1.5.4 Step 4: Test	39
1.5.5 Step 5: Deploy	39
1.6 Prerequisites for Learning AI	40

1.6.1	Mathematics	40
1.6.2	Programming	40
1.6.3	Understanding Computers	41
1.7	Summary	42
1.8	Practice Problems	42
2	The Societal Impact of AI	45
2.1	Introduction: Why Ethics Matters	45
2.2	Case Study 1: Bias in Image Search	46
2.3	Case Study 2: Bias in Hiring	47
2.4	Defining Fairness: A Fundamentally Hard Problem	48
2.4.1	Competing Definitions	48
2.4.2	The Impossibility Result	49
2.4.3	Which Definition When?	49
2.5	The Bias-Detection-Mitigation Pipeline	50
2.6	Systematic Approach to AI Ethics Problems	50
2.7	Privacy and Data Protection	51
2.7.1	Collection and Consent	51
2.7.2	De-anonymization	52
2.7.3	Training on Personal Data	52
2.7.4	Differential Privacy	52
2.8	Accountability and Responsibility	53
2.8.1	Medical AI	53
2.8.2	Hiring and Employment	53
2.8.3	Criminal Justice and Credit	53
2.9	Transparency and Explainability	54
2.9.1	Black Boxes vs. Interpretable Models	54
2.9.2	The Right to Explanation	54
2.9.3	When Is Interpretability Required?	54
2.10	Feedback Loops and Distributed Harm	55
2.10.1	Reinforcing Inequality Over Time	55
2.10.2	Predictive Policing: A Case Study in Feedback Loops	55
2.10.3	Distributed Harm	55
2.11	Environmental and Resource Costs	56
2.11.1	Energy Consumption	56
2.11.2	Centralization of Power	56
2.11.3	Should You Use This Data? Should You Build This System?	56
2.12	Generative AI: Earlier Issues Amplified	57
2.13	Summary	59
2.14	Practice Problems	59

II	Core Machine Learning	63
3	Data Preparation and Feature Engineering	65
3.1	Introduction: Why Data Preparation Matters	65
3.2	Understanding Data Structure	66
3.3	Data Quality Problems	66
3.4	Feature Scaling	67
3.5	Encoding Categorical Variables	67
3.6	Train-Validation-Test Split	68
3.7	Feature Engineering	68
3.7.1	Creating New Features	68
3.7.2	Domain Knowledge	69
3.7.3	Data Leakage: A Critical Pitfall	69
3.8	Sampling Bias: Problems Before Modeling Begins	71
3.8.1	Selection Bias	72
3.8.2	Survivorship Bias	72
3.8.3	Convenience Sampling	72
3.8.4	Non-Response Bias	72
3.9	Distribution Shift: When the World Changes	73
3.9.1	Covariate Shift	73
3.9.2	Label Shift	73
3.9.3	Concept Drift	74
3.10	When Is a Test Set Valid?	74
3.10.1	The Representativeness Requirement	74
3.10.2	When Test Sets Lie	75
3.10.3	Temporal Splits	75
3.10.4	Connecting the Pieces	75
3.11	Summary	76
3.12	Practice Problems	76
4	Supervised Learning	81
4.1	Introduction to Supervised Learning	81
4.1.1	The Structure of Supervised Learning	82
4.1.2	Why Supervised Learning Works	82
4.2	Classification vs. Regression	83
4.2.1	Regression: Predicting Continuous Values	83
4.2.2	Classification: Predicting Categories	83
4.3	Linear Regression	84
4.3.1	Loss Functions: What the Algorithm Optimizes	84
4.3.2	The Hypothesis and Cost Function	85
4.3.3	Gradient Descent	85
4.3.4	Multiple Features	86
4.3.5	Polynomial Regression	86
4.4	Logistic Regression	86

4.4.1	Why Linear Regression Fails for Classification . . .	87
4.4.2	The Sigmoid Function	87
4.4.3	The Logistic Regression Hypothesis	88
4.4.4	The Decision Boundary	88
4.4.5	The Cost Function for Logistic Regression	88
4.5	Regularization: Preventing Overfitting	89
4.5.1	Understanding Overfitting	89
4.5.2	Regularization: Adding a Penalty for Complexity .	89
4.6	Support Vector Machines	90
4.6.1	The Maximum Margin Idea	91
4.6.2	Support Vectors	91
4.6.3	Soft Margins and the Parameter C	91
4.6.4	Kernel Methods: Going Non-Linear	92
4.7	Decision Trees	92
4.7.1	The Tree Structure	93
4.7.2	Choosing the Best Splits	93
4.7.3	Building the Tree: The ID3 Algorithm	94
4.7.4	Pruning: Avoiding Overfitting	94
4.7.5	Ensemble Methods: Combining Multiple Trees . .	95
4.8	Naïve Bayes Classification	95
4.8.1	Bayes' Theorem	96
4.8.2	The Naïve Independence Assumption	96
4.8.3	Classification Rule	96
4.9	Algorithm Comparison	97
4.10	Summary	100
4.11	Practice Problems	100
5	Model Evaluation	103
5.1	Introduction: The Problem of Generalization	103
5.2	Parameters vs. Hyperparameters	104
5.3	The Importance of Data Splitting	104
5.4	Bias and Variance: Understanding Errors	105
5.5	Classification Metrics	106
5.5.1	ROC Curves and AUC	107
5.5.2	Calibration: Are the Probabilities Meaningful? . .	109
5.6	Regression Metrics	110
5.7	Cross-Validation	111
5.8	How People Accidentally Cheat	112
5.8.1	Leaderboard Overfitting	112
5.8.2	Why "Just One More Look" Breaks Guarantees . .	112
5.8.3	Data Leakage in Preprocessing	113
5.8.4	Temporal Leakage	113
5.8.5	Good Test $\neq$ Good Real-World	113
5.9	Hyperparameter Tuning	114

5.10	Summary	115
5.11	Practice Problems	115
6	Unsupervised Learning and Clustering	119
6.1	Introduction: Learning Without Labels	119
6.2	Clustering: Finding Groups in Data	120
6.3	Measuring Similarity: Distance Functions	121
6.3.1	The Minkowski Family	122
6.4	K-Means Clustering	123
6.4.1	The Algorithm	123
6.4.2	Choosing the Number of Clusters	126
6.4.3	Limitations of K-Means	126
6.5	Hierarchical Clustering	127
6.5.1	Agglomerative Clustering	127
6.5.2	Dendrograms	128
6.6	Gaussian Mixture Models	129
6.6.1	The Generative Model	129
6.6.2	Soft Assignments: Responsibilities	129
6.7	The Expectation-Maximization Algorithm	130
6.7.1	K-Means as a Special Case	131
6.8	Dimensionality Reduction	131
6.8.1	Principal Component Analysis (PCA)	131
6.8.2	t-SNE: Visualizing High-Dimensional Data	133
6.9	Comparing Unsupervised Methods	133
6.9.1	Choosing the Right Method	134
6.9.2	Interpreting Clusters with Care	134
6.10	Summary	135
6.11	Practice Problems	136
III	Advanced Methods	139
7	Kernel Methods	141
7.1	Why Linear Models Fail: A Motivating Example	141
7.2	Linear Functions and Their Limitations	143
7.3	Ridge Regression: A Worked Example	143
7.3.1	The Primal Solution	144
7.3.2	The Dual Solution	145
7.4	The Kernel Trick	145
7.5	The Quadratic Kernel	146
7.6	Polynomial and Gaussian Kernels	146
7.7	Properties of Valid Kernels	148
7.8	Beyond Regression: Other Kernel Algorithms	148
7.9	Geometric Operations in Feature Space	149

7.10	Kernel Methods and Deep Learning	150
7.10.1	Why Kernel Methods Lost: The Scaling Wall	150
7.11	The Bias-Variance Trade-off	151
7.12	A Worked IAIO-Style Problem	152
7.13	Practice Problems	152
8	AI Search and Constraint Satisfaction	155
8.1	The Nature of Search Problems	155
8.2	State Space Representation	156
8.2.1	Thinking in States and Actions	156
8.2.2	The Components of a Search Problem	157
8.2.3	A Concrete Example: The Romania Map	158
8.2.4	Tree Search vs. Graph Search	159
8.3	Measuring Algorithm Performance	159
8.4	Uninformed Search Strategies	161
8.4.1	Breadth-First Search	161
8.4.2	Uniform Cost Search	162
8.4.3	Depth-First Search	163
8.4.4	Comparing Uninformed Search Strategies	164
8.5	Informed Search Strategies	164
8.5.1	Heuristic Functions	165
8.5.2	Greedy Best-First Search	166
8.5.3	A* Search	167
8.6	Adversarial Search: Games	168
8.6.1	The Game Framework	168
8.6.2	The Minimax Algorithm	169
8.6.3	Alpha-Beta Pruning	169
8.6.4	Dealing with Complexity: Evaluation Functions	170
8.7	Constraint Satisfaction Problems	171
8.7.1	What is a CSP?	171
8.7.2	Backtracking Search	171
8.7.3	Improving Backtracking	172
8.7.4	Variable and Value Ordering	173
8.7.5	CSPs and SAT	173
8.8	Complexity Comparison	173
8.9	Connecting CSP, SAT, and Logic	174
8.10	Summary	176
8.11	Practice Problems	177
9	Reinforcement Learning	179
9.1	Introduction	179
9.2	The Reinforcement Learning Framework	180
9.2.1	A Concrete Example: Navigating a Maze	181
9.3	Returns and the Discount Factor	182

9.4	Policies and Value Functions	183
9.5	The Markov Property and MDPs	184
9.6	The Bellman Equations	185
9.7	Exploration vs. Exploitation	187
9.8	Dynamic Programming Methods	187
9.9	Model-Free Methods: Learning from Experience	188
9.9.1	Q-Learning: A Worked Example	189
9.9.2	Exploration Strategies	190
9.10	Summary	191
9.11	Practice Problems	191
10	Generative Models and Deep Learning	193
10.1	Introduction to Generative AI	193
10.2	Generative Versus Discriminative Models	194
10.2.1	Discriminative Models	194
10.2.2	Generative Models	195
10.3	Latent Space: The Heart of Generative Models	196
10.3.1	The Problem of High Dimensionality	196
10.3.2	Compression to Meaningful Representations	196
10.3.3	Why Latent Space Works	196
10.4	Autoencoders: Learning to Compress and Reconstruct	197
10.4.1	Architecture	197
10.4.2	Training Objective	198
10.4.3	Applications of Autoencoders	198
10.5	Variational Autoencoders (VAEs)	199
10.5.1	From Points to Distributions	199
10.5.2	The VAE Training Objective	199
10.5.3	Generation with VAEs	200
10.6	Evaluating Generative Models	200
10.6.1	Failure Modes	200
10.6.2	Compute and Data Dependence	201
10.7	Summary	202
10.8	Practice Problems	202
IV	Theory	205
11	Statistical Learning Theory	207
11.1	Introduction: The Theory Behind Learning	208
11.2	A Tale of Two Learners: Rats and Pigeons	209
11.2.1	The Clever Rats	209
11.2.2	The Superstitious Pigeons	209
11.3	The Formal Learning Framework	210
11.3.1	The Papaya Example	210

11.4	Measuring Success: True Error vs. Training Error	211
11.5	PAC Learning: Probably Approximately Correct	211
11.5.1	The Realizability Assumption	211
11.5.2	Sample Complexity for Finite Classes	212
11.6	Agnostic PAC Learning	212
11.6.1	The Bias-Estimation Trade-off	213
11.7	Uniform Convergence	213
11.8	Summary	214
11.9	Practice Problems	214
12	Advanced Statistical Learning Theory	215
12.1	Introduction: Beyond Finite Classes	216
12.2	Shattering: What a Class Can Express	216
12.3	The VC Dimension	217
12.3.1	Examples of VC Dimensions	218
12.4	The Fundamental Theorem of PAC Learning	218
12.5	The No Free Lunch Theorem	219
12.6	Connecting Theory to Practice	220
12.6.1	Overfitting Through the Lens of VC Dimension	220
12.6.2	Practical Guidelines	220
12.7	Summary	221
12.8	Practice Problems	221
V	Applications (Optional)	223
13	Matrix Factorization: A Unifying Idea	225
13.1	The Core Idea	226
13.2	Mathematical Formulation	227
13.2.1	The Optimization Problem	227
13.2.2	Gradient Derivation	228
13.2.3	Alternating Least Squares	228
13.3	Connection to SVD	229
13.4	The Bias-Variance Perspective	230
13.5	Summary	232
13.6	Practice Problems	232
14	Modern Deep Learning	235
14.1	Introduction	235
14.2	How Computers Process Images	236
14.2.1	Images as Numbers	236
14.2.2	From Pixels to Understanding	236
14.3	Generative Adversarial Networks (GANs)	236
14.3.1	The Two Players	237

14.3.2	The Adversarial Game	237
14.3.3	Strengths and Challenges	238
14.4	Diffusion Models	238
14.4.1	The Forward Process: Adding Noise	238
14.4.2	The Reverse Process: Learning to Denoise	239
14.4.3	Generation	239
14.4.4	Why Diffusion Models Work So Well	240
14.4.5	Why Diffusion Replaced GANs	240
14.5	Transformers and Large Language Models	240
14.5.1	The Attention Mechanism	240
14.5.2	Transformer Architecture	241
14.5.3	Large Language Models	241
14.5.4	Tokenization	241
14.5.5	Computing Parameters	242
14.5.6	Why Scale Works (and When It Stops)	242
14.6	Text-to-Image Generation	243
14.7	The AI Landscape Today	243
14.8	Ethical Considerations	243
14.9	Limitations and Open Problems	244
14.10	Summary	245
14.11	Practice Problems	245
A	Practical Competition Strategies	247
A.1	Introduction	247
A.2	Understanding Competition Structure	247
A.2.1	The Data Split	247
A.2.2	Evaluation Metrics	248
A.3	The Competition Workflow	249
A.3.1	Phase 1: Understand the Problem	249
A.3.2	Phase 2: Establish a Baseline	249
A.3.3	Phase 3: Exploratory Data Analysis	250
A.3.4	Phase 4: Feature Engineering	250
A.3.5	Phase 5: Model Development	250
A.3.6	Phase 6: Validation and Selection	251
A.4	Avoiding Data Leakage	251
A.5	Cross-Validation Strategies	252
A.6	Ensemble Methods	252
A.7	Time Management in Timed Competitions	252
A.8	Common Mistakes to Avoid	253
A.9	Summary	254
B	Mathematical Notation	255
B.1	Introduction	255
B.2	Numbers and Basic Operations	255

B.2.1	Number Sets	255
B.2.2	Arithmetic Operations	256
B.3	Sets and Set Operations	256
B.3.1	Set Notation	257
B.3.2	Set Operations and Their Logical Cousins	257
B.4	Summation and Product Notation	258
B.4.1	Summation (Σ)	258
B.4.2	Product (Π)	258
B.5	Functions	259
B.5.1	Common Function Notation	259
B.5.2	Important Functions	259
B.5.3	The Indicator Function	260
B.6	Greek Letters	260
B.7	Comparisons and Logical Symbols	261
B.8	Quantifiers	261
B.9	Machine Learning Specific Notation	261
B.9.1	Data and Features	261
B.9.2	Models and Learning	262
B.9.3	Probability Notation	262
B.9.4	Calculus Notation	262
B.10	Reading Mathematical Expressions	262
B.11	Summary: Quick Reference	263
C	Probability	265
C.1	Introduction: Why Probability?	265
C.2	What Is Probability?	266
C.2.1	The Basic Idea	266
C.2.2	Sample Spaces and Events	266
C.3	Basic Probability Rules	267
C.3.1	The Fundamental Axioms	267
C.3.2	The Complement Rule	267
C.3.3	The Addition Rule	268
C.4	Conditional Probability	269
C.4.1	The Idea	269
C.4.2	How Conditional Probability Works	269
C.4.3	A Concrete Example with Numbers	270
C.4.4	Medical Testing: A Famous Example	271
C.5	Bayes' Theorem	273
C.5.1	The Formula	273
C.5.2	Understanding the Parts	273
C.5.3	Example: Spam Detection	274
C.5.4	Bayes' Theorem in Machine Learning	274
C.6	Independence	274
C.6.1	What Independence Means	274

C.6.2	Examples	275
C.6.3	Why Independence Matters	275
C.6.4	Independence in Machine Learning	275
C.7	Random Variables	276
C.7.1	What Is a Random Variable?	276
C.7.2	Discrete vs. Continuous	276
C.7.3	Probability Distributions	277
C.8	Expected Value	277
C.8.1	The Idea	277
C.8.2	Computing Expected Value	277
C.8.3	Properties of Expected Value	278
C.9	Variance and Standard Deviation	278
C.9.1	Why We Need Variance	278
C.9.2	Definition	278
C.9.3	Standard Deviation	278
C.10	Common Probability Distributions	279
C.10.1	Bernoulli Distribution	279
C.10.2	Binomial Distribution	280
C.10.3	Gaussian (Normal) Distribution	280
C.11	Summary	282
D	Calculus	283
D.1	Introduction: Why Calculus for Machine Learning?	283
D.2	Slope: The Key Idea	284
D.2.1	What is Slope?	284
D.2.2	Slope of a Straight Line	284
D.2.3	The Problem with Curves	285
D.3	The Derivative: Slope at a Point	285
D.3.1	The Tangent Line Idea	285
D.3.2	Definition of the Derivative	286
D.3.3	Alternative Notations	286
D.3.4	Computing a Derivative from the Definition	286
D.3.5	What the Derivative Tells Us	287
D.4	Derivative Rules: Shortcuts	288
D.4.1	The Power Rule	288
D.4.2	Constant Multiple Rule	289
D.4.3	Sum and Difference Rules	290
D.4.4	Important Special Derivatives	290
D.4.5	The Chain Rule	290
D.4.6	The Product Rule	292
D.4.7	The Quotient Rule	293
D.5	Partial Derivatives: Functions of Multiple Variables	294
D.5.1	The Need for Partial Derivatives	294
D.5.2	Definition of Partial Derivatives	294

D.5.3	Evaluating Partial Derivatives at Specific Points . .	295
D.5.4	More Examples	296
D.5.5	Geometric Interpretation	296
D.6	The Gradient	297
D.6.1	Definition	297
D.6.2	Understanding Contour Lines	298
D.6.3	The Gradient Points Uphill	299
D.7	Gradient Descent: Finding the Minimum	300
D.7.1	The Algorithm	300
D.7.2	A One-Variable Example	300
D.7.3	A Two-Variable Example	301
D.7.4	The Learning Rate Matters	302
D.8	Critical Points and Optimization	302
D.8.1	What are Critical Points?	302
D.8.2	Types of Critical Points	303
D.8.3	Finding Critical Points	303
D.8.4	The Second Derivative Test	304
D.8.5	Global vs Local Optima	305
D.9	Important Functions and Their Derivatives	305
D.9.1	The Sigmoid Function	305
D.9.2	The ReLU Function	306
D.9.3	Loss Function Derivatives	307
D.10	Summary	308
E	Logic	309
E.1	Introduction: Why Logic?	309
E.2	Propositions and Truth Values	310
E.2.1	What is a Proposition?	310
E.2.2	Truth Values	310
E.3	Logical Connectives	310
E.3.1	NOT (Negation)	310
E.3.2	AND (Conjunction)	310
E.3.3	OR (Disjunction)	311
E.3.4	XOR (Exclusive Or)	311
E.3.5	Connection to Sets	311
E.4	Implication and Equivalence	312
E.4.1	Implication (If-Then)	312
E.4.2	Biconditional (If and Only If)	312
E.4.3	Logical Equivalence	313
E.5	Important Logical Equivalences	313
E.5.1	Laws You Already Know from Arithmetic	313
E.5.2	Identity and Domination	314
E.5.3	Negation and Double Negation	314
E.5.4	De Morgan's Laws	314

E.5.5	Contrapositive	315
E.6	Quantifiers	315
E.6.1	Universal Quantifier ($\forall$)	315
E.6.2	Existential Quantifier ($\exists$)	316
E.6.3	Negating Quantifiers	316
E.7	First-Order Logic	316
E.7.1	Predicates and Terms	317
E.7.2	Sentences in First-Order Logic	317
E.7.3	Scope of Quantifiers	318
E.7.4	Inference in First-Order Logic	318
E.8	Unification	318
E.8.1	The Unification Problem	318
E.8.2	Most General Unifier (MGU)	319
E.8.3	The Unification Algorithm	319
E.8.4	Unification in AI	320
E.9	Logical Paradoxes	320
E.9.1	The Liar Paradox	320
E.9.2	Russell's Paradox	320
E.9.3	The Barber Paradox	320
E.10	Logic and Artificial Intelligence	321
E.10.1	The Historical Role of Logic in AI	321
E.10.2	Decision Trees as Logical Formulas	321
E.10.3	Logic in Modern AI	322
E.10.4	The Limits of Pure Logic	322
E.11	Building Truth Tables	322
E.12	Conjunctive Normal Form (CNF)	323
E.12.1	What is CNF?	323
E.12.2	Converting to CNF	323
E.13	Resolution	324
E.13.1	The Resolution Rule	324
E.13.2	Proof by Refutation	325
E.14	SAT: Boolean Satisfiability	326
E.14.1	SAT as Search	326
E.14.2	Why SAT Matters	326
E.15	Summary	328
F	Linear Algebra	329
F.1	Introduction: Why Linear Algebra?	329
F.2	Scalars: Single Numbers	330
F.3	Vectors: Ordered Lists of Numbers	330
F.3.1	What is a Vector?	330
F.3.2	Vectors in the Real World	330
F.3.3	Geometric View: Vectors as Arrows	331
F.3.4	Vector Addition and Scalar Multiplication	331

F.4	The Dot Product: Multiplying Two Vectors	331
F.4.1	Definition	331
F.4.2	Why the Dot Product Matters	332
F.4.3	Geometric Meaning	332
F.5	Vector Length (Norm)	332
F.6	Matrices: Tables of Numbers	333
F.6.1	What is a Matrix?	333
F.6.2	Special Matrices	333
F.7	Matrix Operations: Building Up Step by Step	334
F.7.1	Scalar Times Matrix	334
F.7.2	Matrix Times Vector	334
F.7.3	Matrix Times Matrix	334
F.7.4	The Dimension Rule	334
F.7.5	Commutativity: Order Matters!	335
F.8	What Operations Return What?	335
F.9	Matrix Inverse	335
F.10	Eigenvalues and Eigenvectors	336
F.10.1	What Does a Matrix Do to Vectors?	336
F.10.2	The Definition	336
F.10.3	Visualizing Eigenvectors	337
F.10.4	Why Should You Care?	337
F.10.5	Finding Eigenvalues (Optional)	338
F.11	Summary	339
G	Computational Complexity for Machine Learning	341
G.1	Big-O Notation Review	341
G.2	Complexity of Machine Learning Algorithms	342
G.2.1	Supervised Learning	342
G.2.2	Unsupervised Learning	343
G.3	Time vs. Space Tradeoffs	343
G.3.1	Other Common Tradeoffs	343
G.4	IAIO-Style Complexity Questions	344
G.5	Summary	344
G.6	Practice Problems	344
H	Solutions to Practice Problems	347
I	A Song: I Am Still Mine	375

Giới thiệu

Mọi thứ nên được làm đơn giản nhất có thể, nhưng không đơn giản hơn thế.¹

Albert Einstein

Giới thiệu nội dung

Cuốn sách này chuẩn bị cho bạn Kỳ Olympic Quốc tế Trí tuệ nhân tạo (IAIO) — một kỳ thi kiểm tra nền tảng toán học, kỹ thuật thuật toán, và hàm ý đạo đức của AI. IAIO không phải là học thuộc công thức. Bạn sẽ đối mặt với các bài toán đòi hỏi suy luận về những thuật toán chưa từng thấy, áp dụng khái niệm vào các tình huống mới, và tư duy phản biện về các hệ thống AI.

IAIO kiểm tra tư duy như thế nào

IAIO khác biệt căn bản so với các kỳ thi ở trường.

Danh sách kiểm tra trước khi luyện: những thói quen cần tránh

- ☐ Học thuộc công thức mà không hiểu cách suy luận
- ☐ Chỉ học những gì được dạy một cách tường minh
- ☐ Chỉ giải các bài giống bài tập về nhà
- ☐ Viết ra mọi thứ mình biết (người chấm thưởng sự chính xác)
- ☐ Bỏ qua chứng minh để tập trung vào tính toán
- ☐ Dừng lại khi có “một đáp án” (IAIO hỏi: khi nào điều này thất bại?)

¹This wording is a widely used paraphrase. It traces back to a 1933 lecture in which Einstein stated: “The supreme goal of all theory is to make the irreducible basic elements as simple and as few as possible without having to surrender the adequate representation of a single datum of experience.”

Danh sách kiểm tra trước khi luyện: những thói quen cần xây dựng

- ☐ Hiểu *tại sao* mỗi thuật toán hoạt động
- ☐ Dựng các phản ví dụ nơi các phương pháp thất bại
- ☐ Tự suy ra các chặn từ những nguyên lý cơ bản
- ☐ Đọc đề bài hai lần (lướt từ, các trường hợp biên)
- ☐ Nêu giả thiết một cách tường minh (được cho điểm từng phần)
- ☐ Kiểm tra đáp án bằng các kiểm tra hợp lý

Cốt lõi: Trường học kiểm tra những gì bạn đã học. IAIO kiểm tra bạn có *tư duy* với những gì mình học hay không.

Phân loại chương

Core IAIO Material (60% study time)

Chapters 3 (Data), 4 (Supervised), 5 (Evaluation), 6 (Unsupervised), 8 (Search/CSP), 11 (Learning Theory)

Important Supporting (30% study time)

Chapters 1 (Intro), 2 (Ethics), 7 (Kernels), 9 (RL), 10 (Generative), 12 (VC Dimension)

Reference / Enrichment (10% study time)

Chapters 13 (Matrix Factorization), 14 (Modern DL), Appendices

Giới thiệu các phụ lục

Các phụ lục là tài liệu tham khảo, không phải các chương học bổ sung. Phụ lục A nói về chiến lược thi đấu. Phụ lục B tóm tắt ký hiệu toán học. Các Phụ lục C đến F ôn lại xác suất, giải tích, logic, và đại số tuyến tính. Phụ lục G nói về độ phức tạp tính toán. Phụ lục H chứa đáp án các bài tập luyện. Tôi khép lại bằng một bài hát trong Phụ lục I.

Nếu bạn gặp một khái niệm lạ trong chương nào đó (ví dụ định lý Bayes ở Chương 4, hay giá trị riêng ở Chương 6), hãy tra cứu ở phụ lục tương ứng, hiểu đủ để tiếp tục, rồi đi tiếp. Đừng đọc các phụ lục từ đầu đến cuối trước khi bắt đầu các chương chính.

Chapter Dependencies

Not all chapters must be read in strict order. The diagram below shows which chapters build on which others. An arrow from Chapter A to Chapter B means you should read A before B.

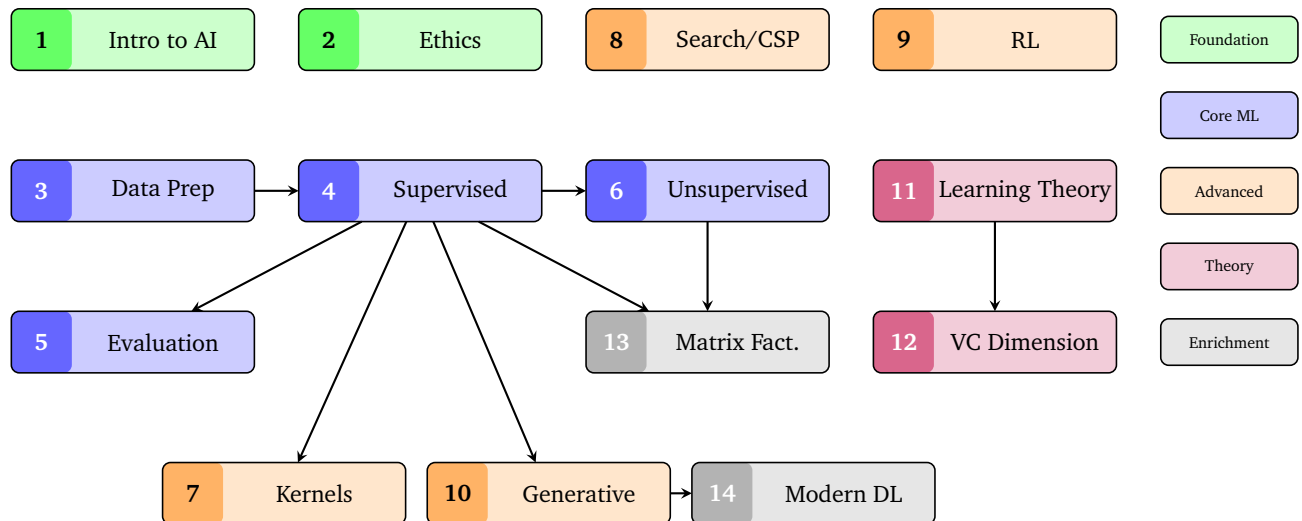


Figure 1: Chapter dependencies. Follow arrows to find prerequisites. Chapters 1, 2, 8, 9, and 11 can be read independently.

Recommended Reading Paths

If you have time, you can go through everything. If you do not have enough time I recommend you always read chapters 1 and 2, and then follow one of the following paths:

The Core ML Path (essential machine learning):

Ch 1+2 → Ch 3 → Ch 4 → Ch 5 → Ch 6

The Theory Path (mathematical foundations):

Ch 1+2 → Ch 4 → Ch 7 → Ch 11 → Ch 12

The Modern AI Path (deep learning and generative AI):

Ch 1+2 → Ch 4 → Ch 10 → Ch 14

The Search and Reasoning Path:

Ch 1+2 → Ch 8 → Appendix E (Logic)

Bạn sẽ cần những gì

Cuốn sách này dùng toán phổ thông: đại số, giải tích cơ bản (đạo hàm và tích phân), và xác suất (quy tắc cơ bản, xác suất có điều kiện, định lý Bayes). Nếu bạn chưa từng gặp hết những thứ đó, đừng lo — các phụ lục bao trọn mọi gì bạn cần. Hãy tra cứu trong lúc học.

Cách sử dụng cuốn sách

Hãy làm các bài tập luyện của từng chương trước khi chuyển tiếp. Chúng được thiết kế để lộ ra những lỗ hổng trong hiểu biết của bạn, chứ không phải để xác nhận những gì bạn đã biết. Khi bị bí, hãy tìm lại cơ sở muốn xem đáp án ngay; chính sự khó chịu khi bí là nơi xảy ra quá trình học sâu nhất. Khi thực sự xem đáp án, hãy tập trung vào *tại sao* cách tiếp cận đó hiệu quả, không chỉ vào đáp án cuối cùng.

Phần dưới đây mô tả những cách học sinh thường mất điểm nhất ở IAIO. Hãy đọc ngay, rồi xem lại sau khi hoàn thành vài chương.

Các kiểu thất bại thường gặp ở IAIO

Kiểu thất bại 1: Hiểu hời hợt

Triệu chứng: Bạn có thể giải thích thuật toán làm gì nhưng không *tại sao* nó hoạt động hoặc *khi nào* nó thất bại.

Ví dụ: “K-means cực tiểu hóa phương sai bên trong cụm” là hiểu hời hợt. Hiểu sâu: “K-means luân phiên giữa việc gán các điểm vào centroid gần nhất và cập nhật centroid về trung bình cụm. Mỗi bước làm giảm hoặc duy trì mục tiêu, đảm bảo hội tụ tới cực tiểu cục bộ. Mục tiêu không lỗi nghĩa là các cách khởi tạo khác nhau cho ra các nghiệm khác nhau.”

Cách sửa: Với mọi thuật toán hoặc định lý, hãy tự suy luận (chỉ đọc chứng minh chưa đủ), dựng một đầu vào nơi nó thất bại, và tự hỏi: “Điều gì thay đổi nếu tôi thay đổi giả thiết này?”

Kiểu thất bại 2: Lỗi thực thi

Triệu chứng: Bạn hiểu khái niệm nhưng bị mất điểm ở bước tính toán hoặc đọc nhầm đề bài.

Cạm bẫy thường gặp: quên chuẩn hóa xác suất, sai dấu trong gradient, nhầm log so với ln, sai lệch một đơn vị ở chiều VC, và đọc nhầm “với mọi” so với “tồn tại”. Cái cuối cùng đặc biệt nguy hiểm: “Chúng minh không bộ phân loại tuyến tính nào giải được XOR” đòi hỏi chỉ ra **mọi** bộ phân loại tuyến tính đều thất bại, chứ không phải chỉ là bạn không tìm được một bộ nào hiệu quả.

Cách sửa: Trước khi viết, hãy nêu chính xác bạn chứng minh điều gì và cần loại chứng minh nào. Sau khi tính, chạy các kiểm tra hợp lý: xác suất có tổng bằng 1 không? Entropy có không âm không? Một phần ví dụ có phá vỡ khẳng định của bạn không?

Kiểu thất bại 3: Bẫy điểm từng phần

Triệu chứng: Bạn làm sai phần (a), và lỗi đó lan sang các phần (b), (c), (d).

Chiến lược:

- Nếu không chắc về (a), hãy nêu giả thiết của bạn một cách rõ ràng và tiếp tục: "Giả sử X từ phần (a), ta có..."
- Xem các phần sau có gợi ý gì cho (a) không. Đôi khi (c) cho biết (a) lẽ ra phải là gì.
- Dành thời gian quay lại các phần đầu sau khi đã xem toàn bộ đề bài.

Kiểu thất bại 4: Hết giờ

Triệu chứng: Bạn giải được các bài quen thuộc nhưng không bao giờ thử các bài khó hơn.

Cách chấm điểm của IAIO: Điểm từng phần ở bài khó thường vượt điểm tối đa ở bài dễ.

Chiến lược:

- Lần 1: Giải các bài bạn tự tin (nhưng chưa cần làm hoàn hảo ngay).
- Lần 2: Thử tất cả các bài còn lại, ít nhất viết phần thiết lập.
- Lần 3: Quay lại để trau chuốt và kiểm tra.

Good luck at the IAIO!

Phần I

Nền tảng

Chương 1

Giới thiệu về Trí tuệ nhân tạo

AI là bất cứ điều gì chưa từng được làm trước đây.

Larry Tesler

1.1 Trí tuệ nhân tạo là gì?

Hãy tưởng tượng bạn có thể chế tạo một cỗ máy biết tư duy. Không chỉ tính toán (chúng ta đã có máy tính bỏ túi suốt nhiều thế kỷ), mà thực sự *biết tư duy*: nhận diện khuôn mặt trong đám đông, hiểu những lời nói, ra các quyết định phức tạp, học hỏi từ sai lầm, và giải quyết những vấn đề mà máy chưa từng gặp bao giờ. Trong phần lớn lịch sử nhân loại, đó là những giấc mơ và chuyện viễn tưởng. Ngày nay, điều đó đã trở thành hiện thực.

Mỗi khi bạn mở khóa điện thoại bằng khuôn mặt, đặt câu hỏi cho Siri, hay xem một bộ phim được Netflix đề xuất, bạn đang tương tác với trí tuệ nhân tạo. Khi một bác sĩ dùng máy tính để chẩn đoán bệnh, khi một chiếc xe tự lái trên đường cao tốc, khi một chương trình đánh bại nhà vô địch cờ vua hay cờ Go thế giới — tất cả đều là những ví dụ về AI trong hành động. Những điều chỉ vài thập kỷ trước còn bị coi là bất khả thi đã trở thành một phần của cuộc sống hằng ngày.

Nhưng chính xác thì trí tuệ nhân tạo *là gì*? Thuật ngữ này được đưa ra vào năm 1956, và từ đó đến nay con người vẫn không ngừng tranh luận về định nghĩa của nó. Về cốt lõi, AI là việc tạo ra các hệ thống máy tính có thể thực hiện những nhiệm vụ mà chúng ta thường gắn với trí thông minh của con người. Chúng bao gồm cảm nhận thế giới qua thị giác và thính giác, hiểu và sinh ra ngôn ngữ, học hỏi từ kinh nghiệm, suy luận về các tình huống phức tạp, và ra quyết định.

trong điều kiện không chắc chắn.

Điều gì khiến AI khác với các chương trình máy tính thông thường? Hãy xét một máy tính bỏ túi truyền thống: bạn đưa vào các con số và phép toán, và máy tuân theo những quy tắc chính xác để tính ra kết quả. Lập trình viên chỉ định chính xác máy tính phải làm gì trong từng tình huống. Các hệ thống AI lại hoạt động theo cách khác. Thay vì được lập trình tường minh với quy tắc cho mọi tình huống, chúng *học* các khuôn mẫu từ dữ liệu. Nếu cho một hệ thống AI xem hàng nghìn hình ảnh mèo và chó, cho biết con nào là con nào, hệ thống sẽ học cách phân biệt chúng, kể cả với những bức ảnh mà nó chưa từng thấy trước đây.

Đây là ý chính: các hệ thống AI tự khám phá các khuôn mẫu. Chúng không được báo trước rằng “mèo có tai nhọn” hay “chó có mũi ướt”. Thay vào đó, chúng xem xét hàng nghìn ví dụ và tìm ra điều gì phân biệt mèo với chó. Những khuôn mẫu mà chúng khám phá có thể rất tinh tế — những điều mà con người không bao giờ để ý một cách có ý thức.

Định nghĩa: Trí tuệ nhân tạo

Trí tuệ nhân tạo (AI) là ngành khoa học máy tính chuyên về việc tạo ra các hệ thống có thể thực hiện những nhiệm vụ thường gắn với trí thông minh của con người. Các nhiệm vụ đó bao gồm nhận diện hình ảnh và vật thể, hiểu và sinh ra lời nói, đưa ra quyết định và dự đoán, giải quyết các bài toán phức tạp, và học hỏi từ kinh nghiệm. Đặc điểm phân biệt của các hệ thống AI là chúng học từ dữ liệu thay vì được lập trình tường minh bằng quy tắc.

AI không phải phép màu, và cũng không phải là những robot có tri giác như trong khoa học viễn tưởng (ít nhất là cho đến nay). Về cốt lõi, AI là toán học và dữ liệu làm việc cùng nhau. Toán học cung cấp những khung nền để học các khuôn mẫu và ra quyết định. Dữ liệu cung cấp nguyên liệu thô để những khuôn mẫu đó được rút ra. Chất lượng những gì hệ thống AI học được hoàn toàn phụ thuộc vào chất lượng dữ liệu mà nó học từ đó.

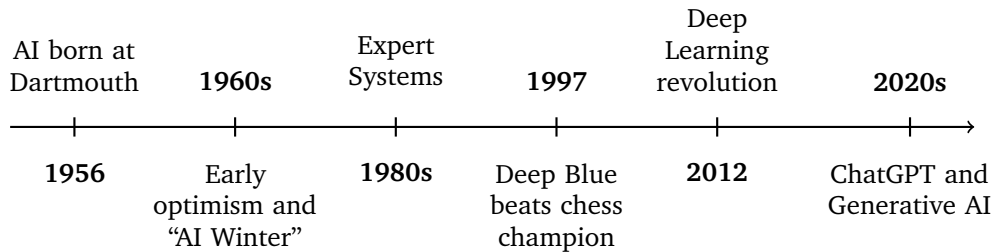
Hướng dẫn đọc: Đọc lướt phần này

Mục 1.2 cung cấp bối cảnh lịch sử về sự phát triển của AI. Mặc dù thú vị, phần nội dung này **rất ít khi được kiểm tra trong IAIO**. Hãy đọc lướt để nắm bao quát; hãy tập trung thời gian học vào các Chương 3–6 và 8–9.

1.2 Lịch sử ngắn gọn về AI

Giấc mơ chế tạo những cỗ máy thông minh đã có từ thuở xa xưa, xuất hiện trong các huyền thoại và câu chuyện suốt lịch sử nhân loại. Những cuộc theo đuổi khoa học về trí tuệ

nhân tạo lại bắt đầu một cách đáng ngạc nhiên là gần đây, và lịch sử của nó là một câu chuyện kịch tính về những hy vọng thăng hoa, những thất vọng phủ phàng, và những bước đột phá phi thường.



Hình 1.1: Các cột mốc quan trọng trong lịch sử trí tuệ nhân tạo.

1.2.1 Sự ra đời của AI

Ngành AI chính thức ra đời vào mùa hè năm 1956 tại một hội thảo nhỏ ở Dartmouth College, bang New Hampshire, Hoa Kỳ. Một nhóm nhà nghiên cứu đề xuất một dự án đầy tham vọng: "Chúng tôi đề xuất tiến hành một nghiên cứu về trí tuệ nhân tạo kéo dài 2 tháng với 10 người... Nghiên cứu sẽ được tiến hành trên cơ sở giả thiết rằng mọi khía cạnh của học hay mọi đặc điểm nào khác của trí thông minh về nguyên tắc đều có thể được mô tả đủ chính xác để máy móc có thể mô phỏng nó."¹

Đó là một tuyên bố táo bạo: rằng trí thông minh của con người có thể được mô tả chính xác và sau đó tái tạo trong một cỗ máy. Các nhà nghiên cứu tại Dartmouth rất lạc quan. Một số dự đoán rằng trong vòng hai mươi năm, máy móc sẽ có thể làm bất kỳ công việc nào mà con người có thể làm. Sự lạc quan đó về sau sẽ được chứng minh là quá sớm một cách kịch tính.

1.2.2 Cảm nhận tử và những giới hạn của nó

Một trong những hệ thống AI sớm nhất và có ảnh hưởng lớn nhất là **cảm nhận tử** (perceptron), được Frank Rosenblatt phát minh vào năm 1958.² Cảm nhận tử là một mạng nơ-ron đơn giản: nó nhận nhiều đầu vào, nhân mỗi đầu vào với một trọng số, cộng các kết quả lại, và cho ra một đầu ra. Điều quan trọng là, nó có thể *học* các trọng số của mình từ các ví dụ.

Cảm nhận tử đã gây nên một sự phấn khích lớn. Rosenblatt đưa ra những dự đoán táo bạo về những cỗ máy có thể "cảm nhận, nhận diện và xác định môi trường xung quanh của chúng mà không cần huấn luyện của con người". The New York Times đã đưa tin

¹McCarthy, J., Minsky, M. L., Rochester, N., & Shannon, C. E. (1955). *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*. <http://jmc.stanford.edu/articles/dartmouth/dartmouth.pdf>

²Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.

rằng Hải quân Hoa Kỳ kỳ vọng cảm nhận tử cuối cùng sẽ “đi bộ, nói chuyện, nhìn, viết, tự sinh sản, và có ý thức về sự tồn tại của chính nó”.³

Rồi đến một đòn chí mạng. Năm 1969, Marvin Minsky và Seymour Papert công bố *Perceptrons*,⁴ một phân tích toán học chứng minh rằng cảm nhận tử một lớp không thể học được một số hàm đơn giản, trong đó có XOR (hoặc loại trừ). Giới hạn này mang tính nền tảng: nếu không có nhiều lớp, cảm nhận tử chỉ có thể học các khuôn mẫu tách tuyến tính.

Tác động của cuốn sách thật kinh khủng. Nguồn tài trợ cho nghiên cứu mạng nơ-ron gần như cạn kiệt qua đêm. Mùa đông AI đầu tiên đã bắt đầu.

1.2.3 Hệ thống chuyên gia và mùa đông thứ hai

AI đã hồi sinh trong thập niên 1980 với sự trỗi dậy của **hệ thống chuyên gia** — những chương trình được thiết kế để nắm bắt tri thức của các chuyên gia con người trong những lĩnh vực cụ thể. Các hệ thống này mã hóa các quy tắc như “NẾU bệnh nhân bị sốt VÀ ho VÀ đi du lịch gần đây đến các vùng nhiệt đới THÌ nghi ngờ sốt rét”. Bằng cách nối chuỗi nhiều quy tắc như vậy, hệ thống chuyên gia có thể đưa ra những lời khuyên hữu ích cho chẩn đoán y khoa, sửa chữa thiết bị, thăm dò khoáng sản, và các nhiệm vụ chuyên ngành khác.

Nhưng hệ thống chuyên gia có những giới hạn. Các quy tắc phải được mã hóa thủ công bởi các lập trình viên làm việc cùng với chuyên gia trong lĩnh vực đó. Đây là một quá trình chậm chạp, tốn kém. Các hệ thống rất mong manh: chúng hoạt động tốt trong phạm vi hẹp của mình nhưng thất bại thảm hại khi gặp những tình huống mà quy tắc của chúng không bao trùm. Đến cuối thập niên 1980, sự quan tâm suy giảm, và AI bước vào một mùa đông khác.

1.2.4 Sự hồi sinh của lan ngược

Trong khi hệ thống chuyên gia lên rồi xuống, một cuộc cách mạng lặng lẽ đang âm thầm hình thành. Năm 1986, David Rumelhart, Geoffrey Hinton và Ronald Williams công bố một bài báo mang tính bước ngoặt, chỉ ra cách huấn luyện các mạng nơ-ron *nhiều lớp* bằng một thuật toán gọi là **lan ngược** (backpropagation).⁵ Điều này giải quyết vấn đề đã giết chết cảm nhận tử: với nhiều lớp và lan ngược, mạng nơ-ron có thể học được XOR và những khuôn mẫu phức tạp hơn nhiều.

Ý tưởng này không hoàn toàn mới (các biến thể đã được khám phá trước đó), nhưng bài báo này đã đưa nó đến gần hơn với mọi người và chứng minh sức mạnh của nó. Mạng nơ-ron đã trở lại, mặc dù phải mất hàng thập kỷ để chúng đạt tới toàn bộ tiềm năng của mình.

³New York Times, July 8, 1958, p. 25: “New Navy Device Learns by Doing.”

⁴Minsky, M. & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

⁵Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

1.2.5 Lối rẽ thống kê

Thập niên 1990 chứng kiến AI đi theo hướng nghiêm ngặt hơn, mang tính toán học hơn. Các nhà nghiên cứu rời bỏ những quy tắc lập trình thủ công, chuyển sang **các phương pháp thống kê** có thể học các khuôn mẫu từ dữ liệu. Machine Vector Support (SVM), mạng Bayesian, và các phương pháp tập hợp như Random Forest đã xuất hiện trong giai đoạn này.

Các phương pháp này có nền tảng lý thuyết vững chắc và hoạt động tốt trên các tập dữ liệu vừa. Chúng trở thành lực lượng chủ lực của học máy thực tiễn, và nhiều phương pháp vẫn quan trọng cho đến ngày nay. Phần lớn nội dung cuốn sách này trình bày các kỹ thuật đến từ truyền thống thống kê đó.

1.2.6 Dữ liệu lớn và GPU

Hai sự phát triển trong thập niên 2000 đã tạo tiền đề cho cuộc cách mạng học sâu. Thứ nhất, internet tạo ra lượng dữ liệu chưa từng có: hàng tỷ hình ảnh, tài liệu và bản ghi âm có thể dùng để huấn luyện các thuật toán học. Thứ hai, các đơn vị xử lý đồ họa (GPU) — ban đầu được thiết kế cho trò chơi điện tử — lại tỏ ra cực kỳ phù hợp với các phép toán ma trận song song mà mạng nơ-ron yêu cầu.

Mạng nơ-ron rất “khát” tài nguyên tính toán. Huấn luyện chúng trên CPU chậm khủng khiếp. GPU đã làm cho việc huấn luyện các mạng lớn hơn nhiều trên các tập dữ liệu lớn hơn nhiều trở nên khả thi.

1.2.7 Cách mạng học sâu

Khoảnh khắc bước ngoặt đến vào năm 2012 khi một hệ thống học sâu có tên AlexNet giành chiến thắng trong cuộc thi ImageNet — một cuộc thi thường niên về nhận diện vật thể trong hình ảnh. AlexNet không chỉ thắng; nó đè bẹp toàn bộ cuộc thi, giảm tỷ lệ lỗi hơn 10 điểm phần trăm so với hệ thống tốt thứ hai.⁶ Đây không phải là tiến bộ từng bước; đó là một bước nhảy vọt khiến cả ngành phải chú ý.

Trong vài năm, các hệ thống học sâu đã đạt hiệu năng ngang con người hoặc hơn con người ở nhiệm vụ này đến nhiệm vụ khác: nhận diện khuôn mặt, chuyển lời nói thành văn bản, dịch ngôn ngữ, chơi trò chơi. Mùa đông AI đã chính thức kết thúc.

⁶Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105. Top-5 error dropped from 26.2% to 15.3%.

1.2.8 Kiến trúc transformer và các mô hình ngôn ngữ lớn

Năm 2017, các nhà nghiên cứu tại Google công bố bài báo có tiêu đề “Attention Is All You Need”,⁷ giới thiệu kiến trúc **transformer**. Transformer xử lý các chuỗi bằng một cơ chế gọi là *attention* (cơ chế chú ý), cho phép chúng nắm bắt các phụ thuộc khoảng cách xa một cách hiệu quả hơn so với các phương pháp trước đó.

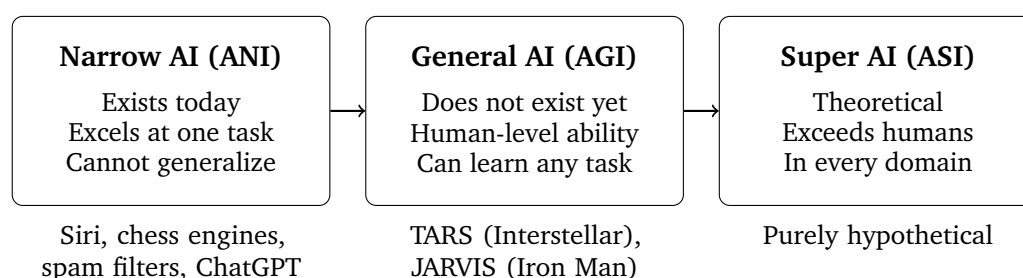
Transformer được chứng minh là có khả năng mở rộng đáng kinh ngạc. Làm cho chúng lớn hơn và huấn luyện trên nhiều dữ liệu hơn liên tục cải thiện hiệu năng. Điều này dẫn đến sự phát triển của **các mô hình ngôn ngữ lớn** (LLM): GPT, BERT, và những thế hệ kế tiếp, được huấn luyện trên một lượng văn bản khổng lồ từ internet.

Vào tháng 11 năm 2022, OpenAI phát hành ChatGPT, đưa các mô hình ngôn ngữ lớn đến với công chúng. Tên gọi tiết lộ kiến trúc của nó: Chat**GPT** là viết tắt của “Chat Generative Pre-trained **Transformer**” (Transformer được huấn luyện trước có tính năng tạo sinh hội thoại). ChatGPT đã chứng minh rằng AI có thể tham gia vào những cuộc trò chuyện trôi chảy, uyên bác về hầu hết mọi chủ đề. Chỉ trong hai tháng, hệ thống đã có hơn 100 triệu người dùng.⁸

Ngày nay, chúng ta đang chứng kiến sự trỗi dậy của **AI tạo sinh** (generative AI): những hệ thống có thể tạo ra nội dung mới thay vì chỉ phân tích dữ liệu có sẵn. Các mô hình ngôn ngữ lớn viết luận, trả lời câu hỏi, và giải thích các khái niệm. Các trình tạo hình ảnh như DALL-E và Midjourney tạo ra các tác phẩm nghệ thuật nguyên bản từ mô tả bằng văn bản. Chúng ta đang sống trong một trong những giai đoạn thú vị nhất của lịch sử công nghệ.

1.3 Ba loại AI

Các nhà khoa học phân biệt ba cấp độ của trí tuệ nhân tạo, mỗi cấp đại diện cho một mức năng lực khác biệt về bản chất. Phân loại này giúp chúng ta hiểu chúng ta đang ở đâu hôm nay và có thể sẽ tiến về đâu.



Hình 1.2: Ba loại AI: chỉ có AI hẹp tồn tại ngày nay.

⁷Vaswani, A. et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://arxiv.org/abs/1706.03762>

⁸Hu, K. (2023, February 1). ChatGPT sets record for fastest-growing user base — analyst note. *Reuters*. <https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/>

1.3.1 AI hẹp (Narrow AI — Trí tuệ nhân tạo hẹp)

Mọi hệ thống AI tồn tại ngày nay, dù ẩn tượng đến đâu, đều là ví dụ của **AI hẹp**. Các hệ thống AI hẹp xuất sắc trong những nhiệm vụ cụ thể, được xác định rõ ràng, nhưng không thể chuyển giao năng lực của chúng sang các lĩnh vực khác. Chúng là những chuyên gia, không phải những người đa năng.

Hãy xét một vài ví dụ. Các trợ lý ảo như Siri và Alexa có thể hiểu lời nói và trả lời câu hỏi, nhưng chúng không thể lái xe hay chẩn đoán bệnh. Các hệ thống đề xuất trên Netflix và YouTube cực kỳ tốt trong việc dự đoán điều gì bạn có thể muốn xem, nhưng chúng không biết gì về bất kỳ điều gì khác. Các chương trình cờ vua có thể đánh bại bất kỳ người chơi nào, nhưng chúng không thể chơi cờ vua Dam, huống chi là trò chuyện. Ngay cả ChatGPT, dù có khả năng đa phương thức, vẫn là một AI hẹp: nó xuất sắc trong các nhiệm vụ liên quan đến ngôn ngữ nhưng không có trí thông minh tổng quát, và nó không thể điều khiển robot hay làm bất cứ điều gì vượt quá những gì nó được huấn luyện.

Giới hạn then chốt của AI hẹp là thiếu khả năng khái quát hóa. Một hệ thống được huấn luyện để nhận diện mèo không thể tự động nhận diện chó. Một hệ thống được huấn luyện trên tiếng Anh không thể tự động hiểu tiếng Pháp. Mỗi nhiệm vụ mới đều cần dữ liệu huấn luyện mới, thiết kế mô hình mới, và thường là công sức đáng kể từ các kỹ sư con người. AI hẹp rất mạnh, nhưng mạnh theo cùng cách mà một chiếc tuốc nơ vít mạnh: xuất sắc cho mục đích được định sẵn, vô dụng cho những mục đích khác.

1.3.2 Trí tuệ nhân tạo tổng quát (AGI)

Trí tuệ nhân tạo tổng quát (AGI) chỉ những hệ thống AI có thể thực hiện bất kỳ nhiệm vụ trí tuệ nào mà con người có thể làm. Một hệ thống AGI sẽ có thể học các kỹ năng mới mà không cần được lập trình riêng cho chúng, suy luận về các vấn đề lạ, chuyển giao tri thức từ lĩnh vực này sang lĩnh vực khác, hiểu bối cảnh và sắc thái, và thích nghi với những tình huống bất ngờ.

AGI chưa tồn tại. Không ai biết khi nào, hoặc thậm chí liệu, nó có thể đạt được hay không. Một số nhà nghiên cứu tin rằng chúng ta chỉ còn vài năm nữa; những người khác nghĩ rằng phải mất hàng thập kỷ hoặc lâu hơn; một số nghi ngờ liệu điều đó có thể xảy ra hay không.

Khoa học viễn tưởng đã tưởng tượng nhiều hình dung về AGI. Hãy nghĩ đến TARS trong *Interstellar* (Hố đen tử thần), có thể trò chuyện tự nhiên, nói đùa, điều khiển tàu vũ trụ, và thích nghi với những tình huống hoàn toàn bất ngờ trên các hành tinh xa lạ. Hoặc JARVIS trong *Iron Man* (Người sắt), đóng vai trò trợ lý thông minh có thể giúp đỡ bất kỳ nhiệm vụ nào, từ quản lý việc nhà đến hỗ trợ các hoạt động của siêu anh hùng. Những ví dụ hư cấu này nắm bắt được AGI có thể trông như thế nào: những hệ thống thực sự thông minh theo cách con người thông minh, không chỉ đơn thuần giỏi ở những nhiệm vụ cụ thể.

Con đường từ AI hẹp ngày nay đến AGI vẫn chưa rõ ràng. Một số tin rằng điều đó sẽ đòi hỏi những phương pháp hoàn toàn mới mà chúng ta chưa khám phá ra. Những người khác nghĩ rằng việc mở rộng các kỹ thuật hiện có (nhiều dữ liệu hơn, nhiều sức mạnh tính toán hơn, mô hình lớn hơn) cuối cùng sẽ tạo ra AGI. Đây là một trong những câu hỏi mở quan trọng nhất của ngành.

1.3.3 Trí tuệ nhân tạo siêu việt (ASI)

Trí tuệ nhân tạo siêu việt (ASI) là một khái niệm lý thuyết: AI vượt trội hơn trí thông minh của con người trong mọi lĩnh vực. Một AI siêu việt sẽ vượt trội hơn con người trong nghiên cứu khoa học, suy luận xã hội, sáng tạo, và mọi nhiệm vụ nhận thức khác.

ASI vẫn hoàn toàn là giả thuyết. Chúng ta không có ý tưởng về cách xây dựng nó, liệu nó có thể xảy ra, hay nó sẽ trông như thế nào. Một số nhà nghiên cứu lo ngại rằng AI siêu việt có thể đặt ra những rủi ro tồn vong nếu các mục tiêu của nó không được căn chỉnh cẩn thận với các giá trị của con người. Những người khác lập luận rằng những lo ngại này là quá sớm vì chúng ta thậm chí còn chưa đạt được AGI. Bất kể đứng về phía nào trong các cuộc tranh luận đó, ASI đại diện cho chân trời xa xôi của nghiên cứu AI — một khả năng có thể đạt được hoặc không, nhưng định hình cách nhiều người suy nghĩ về quỹ đạo của ngành.

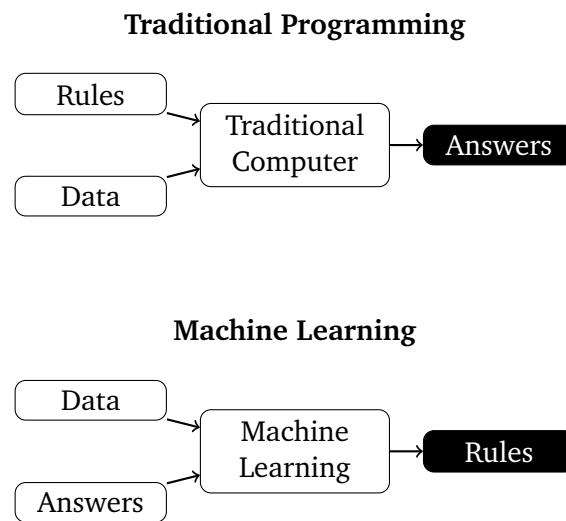
1.4 Các công nghệ chủ chốt trong AI

Trí tuệ nhân tạo bao trùm nhiều công nghệ chuyên biệt và các ngành con. Hiểu mối quan hệ giữa chúng là điều thiết yếu để định hướng trong ngành.

1.4.1 Học máy

Học máy là công nghệ nằm trong trái tim của AI hiện đại. Thay vì lập trình máy tính bằng các quy tắc tường minh, chúng ta đưa cho chúng các ví dụ và để chúng tự động khám phá các khuôn mẫu.

Cách tiếp cận truyền thống trong phát triển phần mềm bao gồm việc lập trình viên phân tích một vấn đề, thiết kế một thuật toán, và viết code triển khai thuật toán đó từng bước. Chương trình sau đó tuân theo các hướng dẫn đó một cách chính xác. Học máy đảo ngược quá trình này. Lập trình viên cung cấp dữ liệu (ví dụ về đầu vào và đầu ra mong muốn), và thuật toán học khám phá ra một hàm ánh xạ đầu vào sang đầu ra. Các quy tắc được học, không phải được lập trình.

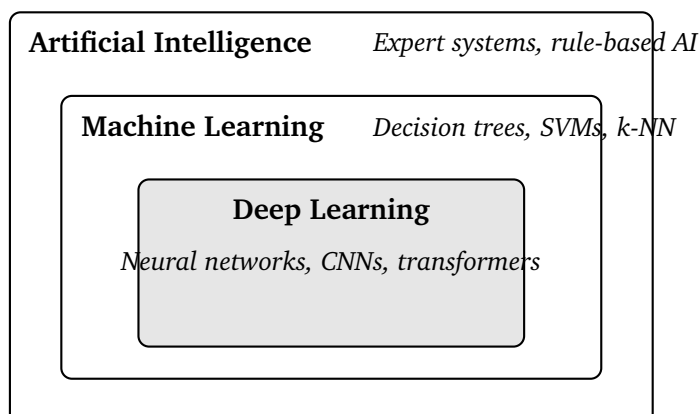


Hình 1.3: Lập trình truyền thống yêu cầu con người chỉ định quy tắc một cách tường minh. Học máy tự động khám phá quy tắc từ dữ liệu và ví dụ.

Cách tiếp cận này rất mạnh vì nhiều nhiệm vụ khó mô tả bằng các quy tắc tường minh nhưng lại dễ minh họa bằng ví dụ. Bạn sẽ viết quy tắc như thế nào để nhận diện một khuôn mặt? Gần như không thể diễn đạt những đặc điểm chính xác khiến một khuôn mặt có thể nhận ra. Nhưng bạn có thể dễ dàng cung cấp hàng nghìn bức ảnh mẫu được dán nhãn “khuôn mặt” hoặc “không phải khuôn mặt”, và thuật toán học máy sẽ tìm ra các khuôn mẫu.

1.4.2 Học sâu

Học sâu là một nhánh con của học máy, sử dụng mạng nơ-ron nhiều lớp. Thuật ngữ “sâu” đề cập đến độ sâu của các mạng này (số lớp xử lý giữa đầu vào và đầu ra).



Hình 1.4: Mối quan hệ giữa AI, Học máy, và Học sâu. Học sâu là một nhánh con của học máy, và học máy lại là một nhánh con của AI.

Mạng nơ-ron là các mô hình tính toán lấy cảm hứng lỏng lẻo từ cấu trúc của bộ não sinh học. Chúng bao gồm các nút liên kết với nhau (“nơ-ron”) được tổ chức thành các lớp. Thông tin chảy từ các nút đầu vào qua các lớp ẩn đến các nút đầu ra, mỗi kết nối có một trọng số quyết định độ mạnh của nó. Việc học bao gồm việc điều chỉnh các trọng số để mạng cho ra các đầu ra chính xác cho các đầu vào đã cho.

Các mạng sâu có thể học những khuôn mẫu cực kỳ phức tạp vì mỗi lớp xây dựng trên các biểu diễn mà các lớp trước đã học được. Các lớp đầu tiên có thể phát hiện những đặc điểm đơn giản như đường viền và màu sắc; các lớp giữa có thể kết hợp chúng thành các hình dạng và kết cấu; các lớp cuối có thể nhận diện các vật thể hoàn chỉnh. Việc học đặc trưng phân cấp chính là điều mang lại sức mạnh cho học sâu.

Cuộc cách mạng học sâu đã tạo ra hầu hết những tiến bộ AI mà bạn nghe nói đến ngày nay: các hệ thống nhận diện khuôn mặt, chuyển lời nói thành văn bản, dịch ngôn ngữ, tạo hình ảnh, và trò chuyện bằng ngôn ngữ tự nhiên. Khi mọi người nói về AI trong tin tức, họ thường đang nói về học sâu.

1.4.3 Thị giác máy tính

Thị giác máy tính là ngành nghiên cứu về việc cho phép máy tính diễn giải và hiểu thông tin thị giác từ thế giới. Khi điện thoại của bạn nhận diện khuôn mặt trong ảnh, khi ô tô tự hành phát hiện người đi bộ, khi một hệ thống kiểm soát chất lượng phát hiện sản phẩm lỗi trên dây chuyền sản xuất — tất cả đều là những ứng dụng của thị giác máy tính.

Thách thức cốt lõi của thị giác máy tính là thu hẹp khoảng cách giữa các giá trị pixel thô và sự thấu hiểu có ý nghĩa. Một hình ảnh kỹ thuật số chỉ là một lưới các

các con số đại diện cho cường độ màu. Một con người nhìn cùng hình ảnh đó lập tức cảm nhận được các vật thể, con người, hành động, và mối quan hệ. Việc dạy máy tính thực hiện bước nhảy vọt từ con số sang sự thấu hiểu đã là một trong những thách thức lớn của AI.

Thị giác máy tính bao trùm nhiều nhiệm vụ cụ thể. **Phân loại hình ảnh** xác định hình ảnh thuộc danh mục nào: đây là ảnh mèo hay ảnh chó? **Nhận diện vật thể** tìm và xác định nhiều vật thể trong một hình ảnh: có một con mèo ở góc dưới bên trái, một con chó ở giữa, và một người ở nền. **Phân vùng hình ảnh** dán nhãn cho từng pixel theo vật thể mà nó thuộc về. Mỗi nhiệm vụ càng ngày càng chi tiết và thách thức hơn.

1.4.4 Xử lý ngôn ngữ tự nhiên

Xử lý ngôn ngữ tự nhiên (NLP) cho phép máy tính hiểu và sinh ra ngôn ngữ của con người. Điều này bao gồm cả văn bản viết và ngôn ngữ nói.

Ngôn ngữ cực kỳ phức tạp. Cùng một từ có thể có những ý nghĩa khác nhau trong những bối cảnh khác nhau. Câu có thể mơ hồ. Ý nghĩa thường phụ thuộc vào tri thức văn hóa, lễ thường, và sự hiểu về chủ ý của người nói. Trong nhiều thập kỷ, các hệ thống NLP chỉ xử lý được những nhiệm vụ hẹp, được xác định rõ ràng như tìm kiếm từ khóa. Cuộc cách mạng học sâu đã thay đổi điều đó, tạo ra những hệ thống có thể tham gia trò chuyện tinh tế, dịch giữa các ngôn ngữ, tóm tắt tài liệu, và trả lời các câu hỏi phức tạp.

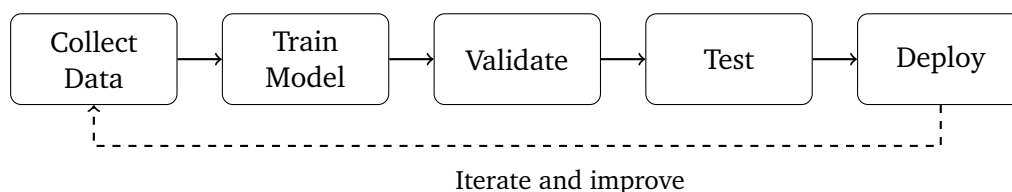
Các mô hình ngôn ngữ lớn như GPT đại diện cho giới hạn hiện tại của NLP. Được huấn luyện trên một lượng văn bản khổng lồ từ internet, các mô hình này học các khuôn mẫu về cách ngôn ngữ được sử dụng. Sau đó chúng có thể tạo ra văn bản mạch lạc, phù hợp với bối cảnh về hầu hết mọi chủ đề. Kết quả đủ ấn tượng để nhiều người thấy khó phân biệt văn bản do AI tạo ra với văn bản do con người viết.

Hướng dẫn đọc: Đọc lướt phần này

Phần tiếp theo giải thích quy trình phát triển AI. Phần này cung cấp bối cảnh hữu ích cho các chương sau nhưng **không được kiểm tra trực tiếp trong IAIO**. Hãy đọc lướt để nắm được ý niệm.

1.5 Quy trình phát triển AI

Xây dựng một hệ thống AI không phải là một bước đơn lẻ mà là một quá trình bao gồm nhiều giai đoạn. Hiểu quy trình này là điều thiết yếu khi làm việc trong lĩnh vực AI.



Hình 1.5: Quy trình phát triển AI: thu thập dữ liệu, huấn luyện, kiểm định, kiểm tra, triển khai, sau đó lặp lại để cải thiện.

1.5.1 Bước 1: Thu thập dữ liệu

Mọi thứ trong AI đều bắt đầu từ dữ liệu. Nếu bạn muốn xây dựng một hệ thống nhận diện mèo, bạn cần hàng nghìn hình ảnh mèo (và không phải mèo) với các nhãn cho biết loại nào là loại nào. Nếu bạn muốn xây dựng một hệ thống dự đoán giá cổ phiếu, bạn cần dữ liệu lịch sử về giá và các yếu tố liên quan. Dữ liệu chính là nguyên liệu thô mà AI sẽ học từ đó.

Ý chính: Dữ liệu là tất cả

Trong AI, dữ liệu là tất cả. Chất lượng của một hệ thống AI phụ thuộc rất lớn vào chất lượng dữ liệu huấn luyện của nó. Nhiều dữ liệu thường dẫn đến kết quả tốt hơn, nhưng dữ liệu cũng phải chính xác (các nhãn phải đúng), đại diện (bao trùm đầy đủ các trường hợp mà hệ thống sẽ gặp), và liên quan (phù hợp với trường hợp sử dụng thực tế). Dữ liệu kém tạo ra AI kém, bất kể thuật toán có tinh vi đến đâu.

Thu thập dữ liệu tốt thường là phần khó nhất của việc phát triển AI. Dữ liệu phải được thu thập, làm sạch, dán nhãn, và xác minh. Chỉ riêng việc dán nhãn cũng có thể đòi hỏi công sức con người rất lớn; ai đó phải nhìn từng hình ảnh và đánh dấu những gì nó chứa. Các công ty đôi khi sử dụng hàng nghìn người dán nhãn làm việc suốt ngày đêm.

1.5.2 Bước 2: Huấn luyện mô hình

Huấn luyện là quá trình mà AI thực sự học. Thuật toán xem xét dữ liệu huấn luyện, tìm kiếm các khuôn mẫu cho phép nó dự đoán các đầu ra chính xác từ các đầu vào.

Hãy tưởng tượng bạn đang dạy một đứa trẻ nhận diện chữ cái. Bạn đưa ra chữ "A" và nói "Đây là chữ A". Bạn đưa ra chữ "B" và nói "Đây là chữ B". Lúc đầu, đứa trẻ đoán ngẫu nhiên. Theo thời gian, khi bạn sửa những sai lầm, đứa trẻ bắt đầu để ý những đặc điểm phân biệt: chữ "A" có đỉnh nhọn ở phía trên và một thanh ngang; chữ "B" có các bướu ở phía bên phải. Cuối cùng, đứa trẻ có thể nhận diện những chữ cái mà chúng chưa từng thấy trước đây.

Học máy hoạt động tương tự. Thuật toán đưa ra các dự đoán trên các ví dụ huấn luyện, tính toán xem nó sai bao nhiêu (lỗi hoặc tổn thất), và điều chỉnh các tham số nội bộ của mình để giảm lỗi đó. Quá trình này lặp lại hàng triệu hoặc hàng tỷ lần. Theo thời gian, mô hình cải thiện, học cách đưa ra các dự đoán chính xác.

1.5.3 Bước 3: Kiểm định

Trong quá trình huấn luyện, chúng ta cần kiểm tra xem mô hình có thực sự đang học các khuôn mẫu hữu ích hay chỉ ghi nhớ dữ liệu huấn luyện. Đây là lúc kiểm định phát huy tác dụng.

Chúng ta tách ra một phần dữ liệu — tập kiểm định (validation set) — mà mô hình không bao giờ huấn luyện trực tiếp. Định kỳ trong quá trình huấn luyện, chúng ta kiểm tra mô hình trên dữ liệu kiểm định này để xem nó hoạt động ra sao trên những ví dụ mà nó chưa từng thấy. Nếu hiệu năng trên dữ liệu huấn luyện tốt nhưng hiệu năng trên kiểm định kém, mô hình đang ghi nhớ thay vì học — một vấn đề gọi là quá khớp (overfitting).

Kiểm định cũng giúp chúng ta tinh chỉnh các thiết lập của mô hình. Có rất nhiều lựa chọn phải đưa ra khi huấn luyện một mô hình: việc học nên diễn ra nhanh đến mức nào? Mô hình nên phức tạp đến mức nào? Huấn luyện nên kéo dài bao lâu? Bằng cách thử các lựa chọn khác nhau trên dữ liệu kiểm định, chúng ta có thể tìm ra những thiết lập tạo ra khả năng khái quát hóa tốt nhất cho các ví dụ mới.

1.5.4 Bước 4: Kiểm tra

Khi việc huấn luyện hoàn tất và chúng ta đã chọn cấu hình mô hình tốt nhất, chúng ta thực hiện đánh giá cuối cùng trên dữ liệu kiểm tra (test). Dữ liệu này đã được tách khỏi toàn bộ quá trình phát triển; mô hình chưa từng nhìn thấy nó, và chúng ta chưa từng dùng nó để đưa ra bất kỳ quyết định nào về mô hình.

Kiểm tra cho chúng ta một ước lượng không thiên lệch về mức độ mà mô hình sẽ hoạt động trong thế giới thực. Nó giống như bài thi cuối kỳ quyết định điểm số: không còn cơ hội nào để cải thiện dựa trên kết quả nữa.

Một quy tắc quan trọng: không bao giờ huấn luyện trên dữ liệu kiểm tra. Nếu bạn xem hiệu năng kiểm tra và sau đó thay đổi để cải thiện nó, tập kiểm tra không còn cung cấp một ước lượng không thiên lệch. Nó trở thành chỉ là một dạng dữ liệu kiểm định khác.

1.5.5 Bước 5: Triển khai

Triển khai có nghĩa là đưa mô hình vào sử dụng thực tế. Điều này có thể có nghĩa là chạy nó trên một máy chủ đám mây nơi hàng triệu người dùng có thể truy cập, nhúng nó vào một ứng dụng điện thoại thông minh, hoặc tích hợp nó vào một thiết bị y tế. Bây giờ mô hình đang làm việc thực sự trong thế giới thực.

Triển khai không phải là điểm kết thúc của quá trình. Sau khi đi vào sản xuất, mô hình nên được giám sát để đảm bảo nó tiếp tục hoạt động tốt. Dữ liệu thế giới thực có thể khác dữ liệu huấn luyện theo những cách bất ngờ, khiến hiệu năng suy giảm. Nên thu thập dữ liệu mới để cải thiện các phiên bản trong tương lai. Phát triển AI là một chu trình cải tiến liên tục, không phải một nỗ lực chỉ làm một lần.

1.6 Điều kiện tiên quyết để học AI

Để thành công trong lĩnh vực trí tuệ nhân tạo, dù là trong kỳ Olympic này hay trong sự nghiệp tương lai, bạn sẽ cần những nền tảng ở ba lĩnh vực.

1.6.1 Toán học

AI về bản chất là mang tính toán học. Các thuật toán thúc đẩy học máy được xây dựng trên những khái niệm từ nhiều nhánh của toán học.

Đại số tuyến tính cung cấp ngôn ngữ để mô tả dữ liệu và các phép biến đổi. Các điểm dữ liệu là các vectơ; các phép toán trên dữ liệu là các phép nhân ma trận. Mạng nơ-ron về bản chất là các chuỗi phép toán ma trận, và việc hiểu đại số tuyến tính giúp bạn hiểu những gì mạng nơ-ron thực sự tính toán.

Giải tích là rất cần thiết vì việc học xảy ra thông qua tối ưu hóa. Khi chúng ta huấn luyện một mô hình, chúng ta đang tìm kiếm các tham số tối thiểu hóa sai số. Gradient — một khái niệm trong giải tích — cho chúng ta biết hướng điều chỉnh tham số để giảm sai số nhanh nhất. Quá trình này, được gọi là gradient descent (hạ gradient), là cách mạng nơ-ron học.

Xác suất và thống kê giúp chúng ta suy luận về sự không chắc chắn. Dữ liệu thế giới thực bị nhiễu và không hoàn chỉnh. Các mô hình đưa ra dự đoán với độ tin cậy khác nhau. Hiểu xác suất cho phép chúng ta định lượng sự không chắc chắn, đưa ra các quyết định có nguyên tắc, và đánh giá liệu các mẫu trong dữ liệu có ý nghĩa hay chỉ là ngẫu nhiên.

Bạn không cần phải là thần đồng toán để làm việc trong lĩnh vực AI. Điều quan trọng là xây dựng trực giác về những khái niệm này: hiểu chúng có ý nghĩa gì và tại sao chúng quan trọng, kể cả khi bạn không thể chứng minh mọi định lý. Toán học trở nên rõ ràng hơn khi bạn áp dụng nó vào các bài toán thực tế.

1.6.2 Lập trình

Bạn sẽ viết code. Các hệ thống AI không tự xây dựng mình; ai đó phải triển khai các thuật toán, xử lý dữ liệu, và điều phối việc huấn luyện.

Python đã trở thành ngôn ngữ thống trị trong AI vì tính dễ đọc của nó

và hệ sinh thái thư viện phong phú sẵn có. PyTorch và TensorFlow cung cấp các công cụ để xây dựng và huấn luyện mạng nơ-ron. NumPy cho phép tính toán số hiệu quả. Pandas hỗ trợ thao tác dữ liệu. Matplotlib tạo ra các biểu đồ trực quan. Những công cụ này cho phép bạn tập trung vào các khái niệm AI thay vì vào các chi tiết triển khai cấp thấp.

Đối với IAIO, bạn nên thoải mái với lập trình Python cơ bản: vòng lặp, hàm, câu điều kiện, và các cấu trúc dữ liệu như danh sách và từ điển. Bạn nên hiểu cách sử dụng mảng NumPy cho các phép toán số, và cách nạp dữ liệu và thao tác dữ liệu với Pandas. Kinh nghiệm với các thư viện học máy như scikit-learn là hữu ích nhưng không bắt buộc.

1.6.3 Hiểu về máy tính

Hiểu cách máy tính hoạt động giúp bạn viết code hiệu quả và đưa ra những quyết định kỹ thuật tốt. Huấn luyện AI hiện đại đòi hỏi khối lượng tính toán khổng lồ, và việc hiểu điều gì khiến tính toán nhanh hay chậm là điều quan trọng.

CPU (Central Processing Unit — đơn vị xử lý trung tâm) là bộ xử lý đa dụng xử lý hầu hết các tác vụ máy tính. GPU (Graphics Processing Unit — đơn vị xử lý đồ họa) ban đầu được thiết kế để kết xuất đồ họa, nhưng lại tỏ ra cực kỳ phù hợp với các phép toán ma trận mà mạng nơ-ron yêu cầu. Huấn luyện AI hiện đại gần như luôn sử dụng GPU, và việc huấn luyện quy mô lớn sử dụng hàng nghìn GPU làm việc cùng nhau.

Hiểu các khái niệm như bộ nhớ, xử lý song song, và độ phức tạp tính toán sẽ giúp bạn thiết kế các hệ thống không chỉ đúng mà còn khả thi khi chạy.

1.7 Tóm tắt

Điểm cần nhớ

Trí tuệ nhân tạo là ngành tạo ra các hệ thống thực hiện các nhiệm vụ gắn với trí thông minh của con người. Đặc điểm then chốt của AI hiện đại là học từ dữ liệu thay vì lập trình tường minh. **Học máy** tự động khám phá các khuôn mẫu trong dữ liệu. **Học sâu** sử dụng mạng nơ-ron nhiều lớp để học các khuôn mẫu phức tạp một cách phân cấp.

Hiện chỉ có **AI hẹp** tồn tại; những hệ thống xuất sắc trong các nhiệm vụ cụ thể nhưng không thể khái quát hóa. **Trí tuệ nhân tạo tổng quát** (AGI — AI ngang con người) và **Trí tuệ nhân tạo siêu việt** (ASI) vẫn là những khả năng trong tương lai, không phải hiện thực hiện tại.

Quy trình phát triển AI bao gồm thu thập dữ liệu, huấn luyện mô hình, kiểm định và kiểm tra hiệu năng, và triển khai vào sử dụng thực tế. Chu trình này lặp lại khi các hệ thống được cải thiện.

Sự thành công trong AI đòi hỏi nền tảng về toán học (đại số tuyến tính, giải tích, xác suất), lập trình (đặc biệt là Python), và tư duy đạo đức về công bằng, quyền riêng tư, và tính chịu trách nhiệm.

Trong AI, dữ liệu là tất cả. Chất lượng của một hệ thống AI phụ thuộc hoàn toàn vào chất lượng dữ liệu mà nó học từ đó.

1.8 Bài tập luyện tập

1. Phân loại từng ví dụ sau vào AI hẹp, AGI, hoặc ASI:

- (a) Một chương trình cờ vua có thể đánh bại bất kỳ người chơi nào
- (b) Một robot có thể học bất kỳ nhiệm vụ vật lý mới nào nhanh như một học việc con người
- (c) Một AI liên tục tạo ra những khám phá khoa học tốt hơn bất kỳ nhà nghiên cứu con người nào
- (d) Bộ lọc thư rác cho email của bạn

2. Một hình ảnh màu có kích thước 640×480 pixel. Bao nhiêu con số tổng cộng mà một máy tính cần để lưu trữ hình ảnh này? (Giả sử biểu diễn màu RGB với một byte cho mỗi kênh màu.)

3. Giải thích tại sao việc giữ dữ liệu kiểm tra tách biệt với dữ liệu huấn luyện lại quan trọng. Điều gì sẽ sai nếu bạn dùng cùng một dữ liệu cho cả hai?

4. Một mô hình đạt độ chính xác 99% trên dữ liệu huấn luyện nhưng chỉ 60% độ chính xác

trên dữ liệu kiểm tra. Vấn đề này cho thấy điều gì? Điều gì có thể đã gây ra nó, và bạn sẽ xử lý vấn đề đó như thế nào?

5. Giải thích bằng lời của bạn sự khác biệt giữa các nhiệm vụ thị giác máy tính: phân loại, định vị, và nhận diện vật thể.

6. Một hệ thống AI sàng lọc đơn xin việc từ chối 40% ứng viên nữ nhưng chỉ 15% ứng viên nam có cùng năng lực. Nguyên tắc đạo đức nào đang bị vi phạm? Điều gì có thể đã gây ra điều này, và có thể làm gì để khắc phục? (Xem Chương 2 để biết định nghĩa về sự công bằng.)

Chương 2

Tác động xã hội của AI

Công nghệ không phải lúc nào cũng tốt hay xấu; cũng không phải là trung lập.¹

Melvin Kranzberg

2.1 Giới thiệu: Vì sao đạo đức quan trọng

Hãy tưởng tượng bạn đang xây dựng một hệ thống AI để giúp bác sĩ chẩn đoán bệnh. Sau nhiều tháng phát triển và thử nghiệm, hệ thống hoạt động xuất sắc. Trong các thử nghiệm lâm sàng, nó vượt trội hơn bác sĩ người trong việc nhận diện một số bệnh nhất định. Bạn triển khai hệ thống tới các bệnh viện trên toàn quốc, tin rằng nó sẽ cứu được nhiều mạng sống.

Một năm sau, bạn phát hiện điều đáng lo ngại. Hệ thống hoạt động tuyệt vời với một số bệnh nhân nhưng thất bại có hệ thống với những bệnh nhân khác. Bệnh nhân từ một số nhóm dân tộc nhất định bị chẩn đoán sai với tỷ lệ đáng báo động. Một số người đã chịu thiệt hại vì bệnh được phát hiện quá muộn. Điều gì đã xảy ra sai?

Khi bạn điều tra, câu trả lời trở nên rõ ràng. Dữ liệu huấn luyện chủ yếu đến từ các bệnh viện ở khu vực đô thị giàu có, nơi phục vụ chủ yếu một nhóm dân số. AI đã học những mẫu hình hiệu quả với nhóm dân số đó nhưng không khái quát hóa được cho người khác. Hệ thống không được thiết kế để phân biệt đối xử. Nó chỉ đơn giản học từ dữ liệu không đại diện cho tất cả mọi người.

Kịch bản này không phải là hư cấu. Những biến thể của câu chuyện này đã xảy ra lặp đi lặp lại khi các hệ thống AI được triển khai trong chăm sóc sức khỏe, tư pháp hình sự, tuyển dụng, cho vay và vô số lĩnh vực khác. Hệ thống AI có thể tạo ra những kết quả mà **không ai dự định hay mong muốn**, đơn giản là bằng cách học từ dữ liệu

¹Kranzberg, M. (1986). Technology and History: “Kranzberg’s Laws.” *Technology and Culture*, 27(3), 544–560.

chứa thành kiến, giả định và những điểm mù của xã hội loài người.

Hiểu những rủi ro này (và học cách ngăn ngừa chúng) không phải là tùy chọn đối với bất kỳ ai làm việc với AI. Đạo đức không phải là chủ đề phụ hay suy nghĩ thêm; nó là trung tâm của việc xây dựng các hệ thống AI thực sự hoạt động cho tất cả mọi người.

Nguyên lý chính

AI khuếch đại những gì nó học được. Nếu dữ liệu huấn luyện mang thành kiến, AI cũng sẽ mang thành kiến. Nếu dữ liệu đại diện không đủ cho một số nhóm nhất định, AI sẽ hoạt động kém với những nhóm đó. Bản thân thuật toán không có la bàn đạo đức bẩm sinh. Nó tái sản xuất trung thành mọi mẫu hình tồn tại trong dữ liệu huấn luyện, bất kể những mẫu hình đó là hữu ích hay có hại.

2.2 Nghiên cứu tình huống 1: Thành kiến (bias) trong tìm kiếm ảnh

Hãy bắt đầu với một ví dụ đơn giản cho thấy thành kiến xã hội có thể được nhúng sâu đến mức nào trong các hệ thống AI.

Hãy xem điều gì xảy ra khi bạn tìm kiếm “school boy” trên một công cụ tìm kiếm ảnh. Bạn sẽ thấy những hình ảnh bình thường: các cậu bé mặc đồng phục, đeo cặp sách, ngồi trong lớp học, chơi trong giờ ra chơi. Những hình ảnh mô tả học sinh tham gia các hoạt động giáo dục điển hình.

Bây giờ hãy tìm kiếm “school girl”. Kết quả thường trông khác biệt đáng kể. Thay vì những hình ảnh bình thường về nữ sinh, bạn có thể thấy những hình ảnh khai thác cơ thể hoặc tình dục hóa phụ nữ trẻ. Đặc điểm của kết quả khác biệt về bản chất so với tìm kiếm “school boy”, mặc dù cả hai tìm kiếm đều hỏi về cùng một khái niệm. Chỉ khác nhau về giới.

Tại sao điều này xảy ra? Các kỹ sư đã xây dựng thuật toán tìm kiếm gần như chắc chắn không có ý định tạo ra kết quả này. Thuật toán chỉ đơn giản học những mẫu hình từ dữ liệu sẵn có cho nó; hàng tỷ hình ảnh được mọi người trên thế giới tải lên, gắn nhãn và liên kết.

Vấn đề là internet phản ánh xã hội loài người, bao gồm tất cả thành kiến và nội dung có vấn đề của nó. Nội dung khai thác cơ thể phụ nữ nhiều hơn rất nhiều so với nam giới. Thuật toán tìm kiếm học những mẫu hình này và tái sản xuất chúng trên quy mô lớn.

Bài học ở đây rất quan trọng: hệ thống AI không tạo ra thành kiến từ hư không. Chúng phơi bày và khuếch đại những thành kiến vốn đã tồn tại trong hành vi con người và trong dữ liệu mà hành vi đó tạo ra.

Trừu tượng hóa IAIO: Thành kiến (bias) trong tìm kiếm ảnh

Các biến nào mang thành kiến? Dữ liệu huấn luyện (hình ảnh web) chứa nội dung bất đối xứng về giới, nhiều hình ảnh khai thác cơ thể hơn cho các truy vấn liên quan đến nữ giới.

Ở đâu trong quy trình đã xảy ra sai sót? Thu thập dữ liệu. Thuật toán đã học từ nội dung web mang thành kiến.

IAIO sẽ kiểm tra điều này như thế nào?

- Mô tả cách thành phần của dữ liệu huấn luyện có thể tạo ra các kết quả khác biệt cho các truy vấn tương đương về mặt ngữ nghĩa.
- Nếu kết quả tìm kiếm cho “school boy” so với “school girl” khác biệt có hệ thống, ở (các) giai đoạn nào của quy trình có thể can thiệp?

2.3 Nghiên cứu tình huống 2: Thành kiến (bias) trong tuyển dụng

Hãy xem xét hai cái tên: **Tamika** và **Brandon**. Ở Hoa Kỳ, “Tamika” là tên gắn liền chủ yếu với phụ nữ Mỹ gốc Phi, trong khi “Brandon” gắn liền chủ yếu với nam giới da trắng.

Các nhà nghiên cứu đã tiến hành một thí nghiệm nổi tiếng;² họ tạo ra các cặp **hồ sơ xin việc giống hệt nhau**, cùng học vấn, cùng kinh nghiệm làm việc, cùng năng lực, nhưng với các tên khác nhau ở đầu hồ sơ. Họ gửi những hồ sơ này tới các nhà tuyển dụng thực tế.

Kết quả nghiên cứu về hồ sơ xin việc

Các hồ sơ xin việc có tên nghe như người da trắng nhận được **nhiều hơn 50% số lời mời phỏng vấn** so với các hồ sơ giống hệt nhưng có tên nghe như người Mỹ gốc Phi. Các năng lực hoàn toàn giống nhau. Chỉ khác tên.

Bây giờ hãy tưởng tượng bạn xây dựng một hệ thống AI để lọc các đơn xin việc, được huấn luyện trên dữ liệu tuyển dụng lịch sử. Nếu các quyết định trong quá khứ bị ảnh hưởng bởi những thành kiến như vậy, AI sẽ học những thành kiến đó.

Thuật toán có thể học rằng: “Ứng viên có tên Brandon có xu hướng được tuyển dụng nhiều hơn ứng viên có tên Tamika.” Nó học điều này không phải vì tên dự đoán năng lực làm việc, mà vì dữ liệu huấn luyện phản ánh sự thiên vị của con người.

Tệ hơn, sau khi triển khai, hệ thống như vậy tạo ra một vòng lặp phản hồi. AI mang thành kiến tạo ra các quyết định tuyển dụng mang thành kiến, những quyết định đó trở thành dữ liệu huấn luyện cho các hệ thống AI tương lai, kéo dài và khuếch đại thành kiến ban đầu.

²Bertrand, M. & Mullainathan, S. (2004). Are Emily and Greg more employable than Lakisha and Jamal? A field experiment on labor market discrimination. *American Economic Review*, 94(4), 991–1013.

Trữu tượng hóa IAIO: Thành kiến (bias) trong tuyển dụng

Các biến nào mang thành kiến? Tên (thay thế cho nhân khẩu học) tương quan với kết quả tuyển dụng trong dữ liệu lịch sử.

Ở đâu trong quy trình đã xảy ra sai sót? Dữ liệu huấn luyện phản ánh thành kiến lịch sử của con người; mô hình đã học và tái sản xuất nó.

Khái niệm chính: Vòng lặp phản hồi. AI mang thành kiến tạo ra tuyển dụng mang thành kiến, những quyết định đó trở thành dữ liệu huấn luyện tương lai mang thành kiến, khuếch đại thành kiến ban đầu.

IAIO sẽ kiểm tra điều này như thế nào?

- Định nghĩa tác động phân biệt (disparate impact). Bạn sẽ đo lường nó như thế nào nếu có ma trận nhầm lẫn theo từng nhóm nhân khẩu học?
- Một AI được huấn luyện trên dữ liệu tuyển dụng lịch sử sử dụng “số năm kể từ khi tốt nghiệp” làm một đặc trưng. Tại sao điều này có thể có vấn đề?

2.4 Định nghĩa về công bằng: Một bài toán cực kỳ khó

Một hệ thống AI “công bằng” nghĩa là gì? Câu hỏi tưởng chừng đơn giản này không có một câu trả lời duy nhất. Các định nghĩa khác nhau về công bằng nắm bắt các trực giác đạo đức khác nhau, và chúng thường mâu thuẫn với nhau về mặt toán học. Đây không phải là thất bại của nghiên cứu AI; nó phản ánh sự bất đồng thực sự về ý nghĩa của công bằng.

Vấn đề cốt lõi

Không có định nghĩa “chính xác” nào về công bằng theo mặt toán học. Việc chọn một tiêu chí công bằng là một **lựa chọn đạo đức**, không phải kỹ thuật. Các bên liên quan khác nhau có thể hợp lý khi bất đồng về định nghĩa nào nên được áp dụng.

2.4.1 Các định nghĩa cạnh tranh

Tính đồng đều nhân khẩu học (Demographic parity): Hệ thống nên tạo ra các kết quả tích cực với tỷ lệ bằng nhau trên các nhóm. Nếu 30% Nhóm A được vay vốn, thì 30% Nhóm B cũng nên được vay vốn.

Tỷ lệ lỗi tương đương (Equalized odds): Hệ thống nên có các tỷ lệ lỗi bằng nhau trên các nhóm. Nếu tỷ lệ dương tính giả là 5% đối với Nhóm A, thì nó cũng nên là 5% đối với Nhóm B. Tương tự với tỷ lệ âm tính giả.

Hiệu chuẩn (Calibration): Trong số những người mà hệ thống gán xác suất p , tỷ lệ thực tế là dương tính nên là p , bất kể họ thuộc nhóm nào. Nếu hệ thống nói

“70% khả năng vỡ nợ,” thì 70% số người đó thực tế sẽ vỡ nợ, cho dù họ thuộc Nhóm A hay Nhóm B.

Công bằng cá nhân (Individual fairness): Những cá nhân tương tự nên nhận được các dự đoán tương tự, bất kể họ thuộc nhóm nào.

2.4.2 Kết quả bất khả thi

Sự bất khả thi về mặt toán học

Khi các tỷ lệ nền (base rates) khác nhau giữa các nhóm (ví dụ, nếu tỷ lệ vỡ nợ lịch sử khác nhau), thì việc cùng lúc thỏa mãn tính đồng đều nhân khẩu học, tỷ lệ lỗi tương đương và hiệu chuẩn là **không thể về mặt toán học**.^a

Đây là một định lý, không phải là giới hạn của công nghệ hiện tại. Không thuật toán nào, dù tinh vi đến đâu, có thể thỏa mãn cả ba điều khi các tỷ lệ nền khác nhau.

^aĐược chứng minh độc lập bởi Kleinberg, J., Mullainathan, S., và Raghavan, M. (2016), *Inherent Trade-Offs in the Fair Determination of Risk Scores*, ITCS 2017; và Chouldechova, A. (2017), Fair prediction with disparate impact, *Big Data*, 5(2), 153–163.

Kết quả bất khả thi này có những hàm ý sâu xa. Nó nghĩa là:

- Công bằng đòi hỏi các **lựa chọn** về định nghĩa nào được ưu tiên
- Những lựa chọn này phản ánh các **giá trị**, không chỉ là các cân nhắc kỹ thuật
- Các bên liên quan khác nhau có thể hợp lý khi ưa thích các định nghĩa khác nhau
- Không có hệ thống “công bằng” nào mà mọi người đều đồng ý là công bằng

2.4.3 Định nghĩa nào, khi nào?

Các ngữ cảnh khác nhau có thể đòi hỏi các định nghĩa công bằng khác nhau:

Tính đồng đều nhân khẩu học có thể phù hợp khi: dữ liệu lịch sử được biết là mang thành kiến, hoặc khi tính đại diện bình đẳng là mục tiêu chính sách (ví dụ, tuyển dụng đa dạng).

Tỷ lệ lỗi tương đương có thể phù hợp khi: các lỗi có hậu quả nghiêm trọng, và chúng ta muốn đảm bảo không nhóm nào gánh một gánh nặng không cân xứng của những sai lầm.

Hiệu chuẩn có thể phù hợp khi: các ước tính xác suất của hệ thống được dùng trực tiếp cho các quyết định, và độ chính xác của các ước tính đó quan trọng.

Sự lựa chọn không hoàn toàn mang tính kỹ thuật. Nó cần có sự tham gia của các cộng đồng bị ảnh hưởng, chuyên gia trong lĩnh vực, nhà đạo đức học và nhà hoạch định chính sách.

2.5 Quy trình phát hiện và giảm thiểu thành kiến

Giải quyết thành kiến đòi hỏi nỗ lực có hệ thống trong toàn bộ vòng đời của AI:

Phát hiện: So sánh các đầu ra của hệ thống AI giữa các nhóm khác nhau. Các tỷ lệ lỗi có bằng nhau không? Một số nhóm có phải đối mặt với hậu quả xấu hơn từ các sai lầm không?

Phân tích: Xác định nguyên nhân gốc rễ. Vấn đề nằm ở dữ liệu huấn luyện? Thuật toán? Ngữ cảnh triển khai?

Giảm thiểu: Giải quyết các vấn đề đã xác định, dữ liệu đại diện hơn, loại bỏ các đặc trưng có vấn đề, điều chỉnh thuật toán, hoặc bổ sung con người vào quy trình đánh giá.

Giám sát: Tiếp tục kiểm toán sau triển khai. Thành kiến có thể xuất hiện hoặc xuất hiện trở lại theo thời gian.

Khung phân tích các bài toán đạo đức AI

Khi phân tích bất kỳ kịch bản đạo đức AI nào, hãy xác định có hệ thống:

1. Nguồn thành kiến:

- Thành kiến lịch sử (dữ liệu huấn luyện phản ánh sự phân biệt đối xử trong quá khứ)
- Thành kiến đại diện (một số nhóm bị đại diện không đủ)
- Thành kiến đo lường (các đặc trưng được đo khác nhau giữa các nhóm)
- Thành kiến tổng hợp (một mô hình duy nhất cho các quần thể không đồng nhất)

2. Giai đoạn trong quy trình: Thu thập dữ liệu, gắn nhãn, lựa chọn đặc trưng, huấn luyện, đánh giá, triển khai

3. Định nghĩa công bằng bị vi phạm: Tính đồng đều nhân khẩu học, tỷ lệ lỗi tương đương, hiệu chuẩn, công bằng cá nhân

4. Các bên liên quan: Ai được lợi? Ai bị hại? Ai quyết định?

2.6 Cách tiếp cận có hệ thống cho các bài toán đạo đức AI

IAIO ethics questions require structured reasoning, not narrative recall.

Mẫu phân tích năm điểm

Cho bất kỳ hệ thống AI nào, hãy xác định có hệ thống:

1. **DỮ LIỆU:** Dữ liệu huấn luyện nào đã được sử dụng? Ai được đại diện? Ai bị thiếu?
2. **MỤC TIÊU:** Hệ thống tối ưu hóa điều gì? (độ chính xác? mức độ gắn kết? lợi nhuận?)
3. **THƯỚC ĐO:** Thành công được đo lường như thế nào? Thước đo đó bỏ lỡ điều gì? 4. **THIỆT HẠI:** Ai có thể bị hại? Như thế nào? (dương tính giả so với âm tính giả)
5. **GIẢM THIỂU:** Những can thiệp nào là khả thi? Với đánh đổi nào?

Use mẫu này cho mọi bài toán đạo đức. Viết một câu cho mỗi điểm. Bất cứ khi nào một bài toán IAIO hỏi “hệ thống này có chấp nhận được không?” hay “điều gì có thể xảy ra sai?”, đây là khung phân tích cần áp dụng.

Ví dụ minh họa

Hệ thống: AI phê duyệt khoản vay của một ngân hàng, được huấn luyện trên 10 năm quyết định lịch sử.

1. **DỮ LIỆU:** Các quyết định lịch sử; phản ánh thành kiến quá khứ; đại diện không đủ cho các nhóm từng bị từ chối tín dụng.
2. **MỤC TIÊU:** Tối thiểu hóa tỷ lệ vỡ nợ (hoặc tương đương: tối đa hóa lợi nhuận).
3. **THƯỚC ĐO:** AUC trên dữ liệu lịch sử; không đo lường tính công bằng giữa các nhóm.
4. **THIỆT HẠI:** Các ứng viên đủ điều kiện từ các nhóm bị đại diện không đủ bị từ chối; kéo dài sự bất bình đẳng.
5. **GIẢM THIỂU:** Loại bỏ các biến thay thế (mã zip), bổ sung các ràng buộc công bằng, kiểm toán theo nhóm. Đánh đổi: có thể làm tăng nhẹ tỷ lệ vỡ nợ.

2.7 Quyền riêng tư và bảo vệ dữ liệu

Các hệ thống AI luôn “đói” dữ liệu. Chúng có nhiều dữ liệu hơn thì thường hoạt động tốt hơn. Nhưng điều này tạo ra những căng thẳng cơ bản với quyền riêng tư.

2.7.1 Thu thập dữ liệu và sự đồng ý

Phần lớn dữ liệu huấn luyện AI không được thu thập với mục đích huấn luyện AI. Ảnh tải lên mạng xã hội, văn bản viết trong email, hồ sơ y tế tạo ra cho mục đích điều trị: tất cả dữ liệu này hiện đang được sử dụng để huấn luyện các hệ thống AI. Những người tạo ra dữ liệu này có đồng ý cho việc sử dụng đó không?

Các khung pháp lý khác nhau tùy theo khu vực pháp lý. Quy định chung về bảo vệ dữ liệu (General Data Protection Regulation - GDPR) của Liên minh Châu Âu yêu cầu sự đồng ý rõ ràng cho nhiều mục đích sử dụng dữ liệu cá nhân. Các khu vực khác có sự bảo vệ yếu hơn. Nhưng gác lại vấn đề hợp pháp, vẫn có những câu hỏi đạo đức: Liệu các công ty có nên được phép sử dụng dữ liệu cho những mục đích mà những người tạo ra ban đầu không bao giờ hình dung?

2.7.2 Tái ẩn danh (De-anonymization)

“Chúng tôi đã ẩn danh hóa dữ liệu” thường được đưa ra như một giải pháp cho các lo ngại về quyền riêng tư. Nhưng nghiên cứu đã nhiều lần chứng minh rằng dữ liệu được cho là ẩn danh có thể bị xác định lại danh tính.

Ví dụ về tái ẩn danh

Các nhà nghiên cứu đã chứng minh rằng 87% người Mỹ có thể được nhận dạng duy nhất chỉ bằng ba thông tin: mã zip, ngày sinh và giới tính.^a
Dữ liệu có vẻ ẩn danh thường không thực sự ẩn danh.

^aSweeney, L. (2000). Simple demographics often identify people uniquely. Carnegie Mellon University, Data Privacy Working Paper 3.
<https://dataprivacylab.org/projects/identifiability/paper1.pdf>. Dựa trên dữ liệu Tổng điều tra dân số Hoa Kỳ năm 1990. Một nghiên cứu lặp lại sử dụng dữ liệu Tổng điều tra năm 2000 cho kết quả 63% (Golle, P., 2006, *Proceedings of the 5th ACM Workshop on Privacy in Electronic Society*), vẫn cao một cách đáng kinh ngạc chỉ với ba trường dữ liệu.

Dữ liệu vị trí đặc biệt nhạy cảm. Ngay cả khi không gắn tên, các mẫu hình di chuyển có thể nhận dạng cá nhân (vị trí nhà, vị trí làm việc, thói quen hàng ngày) và lộ ra thông tin nhạy cảm (các chuyến thăm phòng khám y tế, địa điểm tôn giáo, các buổi tụ tập chính trị).

2.7.3 Huấn luyện trên dữ liệu cá nhân

Các mô hình ngôn ngữ lớn (LLM) được huấn luyện trên một lượng khổng lồ văn bản từ internet, bao gồm thông tin cá nhân, các cuộc trò chuyện riêng tư bị rò rỉ và tài liệu có bản quyền. Đôi khi những mô hình này có thể “thò lộ” lại dữ liệu huấn luyện, bao gồm các chi tiết cá nhân về những người thật.

Điều này đặt ra những câu hỏi khó: Ai sở hữu những mẫu hình được học từ dữ liệu cá nhân? Nếu một mô hình “biết” điều gì đó về bạn vì nó có trong dữ liệu huấn luyện, bạn có quyền yêu cầu xóa bỏ điều đó không?

2.7.4 Quyền riêng tư khác biệt

Quyền riêng tư khác biệt là một khuôn khổ toán học cung cấp các bảo đảm quyền riêng tư có thể chứng minh. Ý tưởng là thêm nhiễu được hiệu chỉnh cẩn thận vào dữ liệu hoặc

các phép tính, khiến việc xác định xem một cá nhân cụ thể nào có mặt trong tập dữ liệu trở nên không thể.

Về mặt hình thức, một thuật toán là ϵ -riêng tư khác biệt nếu phân phối đầu ra của nó chỉ thay đổi tối đa một hệ số e^ϵ khi bất kỳ dữ liệu của một cá nhân duy nhất nào được thêm vào hoặc loại bỏ. ϵ nhỏ hơn nghĩa là quyền riêng tư mạnh hơn nhưng thường làm giảm độ chính xác.

Quyền riêng tư khác biệt ngày càng được sử dụng cho các ứng dụng nhạy cảm, nhưng nó liên quan đến các đánh đổi: các bảo đảm quyền riêng tư mạnh hơn thường đồng nghĩa với mô hình kém chính xác hơn.

2.8 Trách nhiệm giải trình và chịu trách nhiệm

Khi một hệ thống AI mắc lỗi, ai chịu trách nhiệm? Câu hỏi này trở nên cấp bách trong những lĩnh vực có tính chất nghiêm trọng.

2.8.1 AI trong y tế

Các hệ thống AI ngày càng được sử dụng trong chăm sóc sức khỏe: phân tích hình ảnh y tế, gợi ý chẩn đoán, đề xuất phương pháp điều trị. Khi những hệ thống này mắc lỗi, bệnh nhân có thể bị tổn thương.

Nếu một hệ thống AI bỏ sót chẩn đoán ung thư và bệnh nhân qua đời, ai phải chịu trách nhiệm pháp lý? Bác sĩ đã dựa vào AI? Bệnh viện đã triển khai nó? Công ty đã xây dựng nó? Các kỹ sư đã huấn luyện nó? Các khuôn khổ pháp lý hiện tại không được thiết kế cho tình huống này.

2.8.2 Tuyển dụng và việc làm

Các công cụ tuyển dụng AI lọc hàng triệu đơn xin việc. Khi những hệ thống này phân biệt đối xử, ai chịu trách nhiệm giải trình? Các công ty thường mua các công cụ AI từ nhà cung cấp và có thể không hiểu cách chúng hoạt động. Liệu một công ty có thể bị quy trách nhiệm về sự phân biệt đối xử của một hệ thống mà họ không xây dựng và không thể giải thích không?

2.8.3 Tư pháp hình sự và tín dụng

Các hệ thống AI được dùng để ra quyết định tại ngoại, dự đoán tái phạm, và xác định điểm tín dụng. Những quyết định ấy có ảnh hưởng sâu sắc đến cuộc sống con người. Thế nhưng những cá nhân bị ảnh hưởng thường không có cách nào hiểu được vì sao quyết định được đưa ra, huống chi là phản đối nó.

Khoảng trống trách nhiệm giải trình

Khi các hệ thống AI trở nên tự chủ hơn và khó hiểu hơn, một khoảng trống mở ra giữa thiệt hại mà chúng có thể gây ra và khả năng buộc ai đó phải chịu trách nhiệm. Việc thu hẹp khoảng trống này đòi hỏi cả giải pháp kỹ thuật (khả năng giải thích, kiểm toán) lẫn các khung pháp lý và quản lý.

2.9 Minh bạch và khả năng giải thích

2.9.1 Hộp đen so với các mô hình có thể diễn giải

Một số mô hình AI vốn có thể diễn giải được. Một cây quyết định cho thấy chính xác đặc trưng nào dẫn đến quyết định và vì sao. Một hồi quy tuyến tính lộ ra trọng số được gán cho từng yếu tố.

Các mô hình khác, đặc biệt là mạng nơ-ron sâu, là “hộp đen”. Chúng có thể có hàng triệu tham số, và ngay cả người tạo ra chúng cũng không giải thích được vì sao chúng đưa ra những quyết định cụ thể. Mô hình hoạt động, nhưng không ai biết chính xác nó hoạt động thế nào.

Model Type	Interpretability	Typical Accuracy
Decision Tree	High	Moderate
Linear/Logistic Regression	High	Moderate
Random Forest	Medium	High
Deep Neural Network	Low	Highest

Thường có một sự đánh đổi: mô hình chính xác nhất lại là mô hình khó diễn giải nhất. Điều này tạo nên những lựa chọn khó khăn trong các lĩnh vực có mức độ rủi ro cao.

2.9.2 Quyền được giải thích

GDPR bao gồm các điều khoản về “quyền được giải thích”: những cá nhân bị ảnh hưởng bởi quyết định tự động có quyền nhận “thông tin có ý nghĩa về logic liên quan”. Nhưng thế nào mới là “có ý nghĩa”? Liệu quyết định của một mạng nơ-ron có bao giờ được giải thích một cách có ý nghĩa không?

Một số nhà nghiên cứu cho rằng các lời giải thích về mô hình hộp đen vốn không đáng tin cậy: chúng chỉ là những hợp lý hóa hậu nghiệm, chứ không phải lời giải thích thật sự về cách mô hình hoạt động. Người khác lại cho rằng dù không hoàn hảo, có lời giải thích vẫn hơn không.

2.9.3 Khi nào thì bắt buộc phải diễn giải được?

Không phải ứng dụng AI nào cũng đòi hỏi khả năng diễn giải. Với một hệ thống đề xuất phim, “thuật toán nghĩ bạn sẽ thích cái này” có thể là đủ. Nhưng với một

hệ thống từ chối cho ai đó vay tiền, tuyển dụng hay ân giảm, mức độ rủi ro lại khác hẳn.

Hãy thử nghĩ: Bạn có chấp nhận chẩn đoán y tế từ một hệ thống không giải thích được lập luận của nó không? Bạn có chấp nhận bản án dựa trên một thuật toán mà không ai diễn giải được không?

2.10 Vòng lặp phản hồi và thiệt hại phân tán

2.10.1 Củng cố sự bất bình đẳng theo thời gian

Các hệ thống AI không chỉ đưa ra dự đoán; chúng định hình chính thể giới mà chúng đang dự đoán. Khi một AI tuyển dụng loại bỏ ứng viên thuộc một số xuất thân nhất định, những ứng viên đó không có việc làm, không tích lũy kinh nghiệm, và càng ít khả năng được chọn bởi các hệ thống AI trong tương lai. Dự đoán trở thành tự thành hiện thực.

Động thái này có thể củng cố và khuếch đại những bất bình đẳng sẵn có. Một hệ thống vốn chỉ hơi thiên vị khi triển khai có thể trở nên thiên vị nghiêm trọng theo thời gian khi các quyết định của nó phản hồi ngược vào dữ liệu huấn luyện.

2.10.2 Cảnh sát dự đoán: một nghiên cứu tình huống về vòng lặp phản hồi

Các hệ thống cảnh sát dự đoán dùng dữ liệu tội phạm lịch sử để dự báo nơi sẽ xảy ra tội phạm. Sau đó cảnh sát được điều tới những “điểm nóng” đã được dự báo ấy.

Vấn đề vòng lặp phản hồi

1. Dữ liệu lịch sử cho thấy nhiều tội phạm hơn ở một số khu dân cư (thường là do cảnh sát hoạt động dày đặc hơn ở đó, chứ không phải thực tế tội phạm nhiều hơn)
2. AI dự báo những khu vực đó là nơi có nhiều tội phạm
3. Cảnh sát được điều động tới đó dày đặc hơn
4. Nhiều vụ bắt giữ hơn diễn ra ở đó (vì có nhiều cảnh sát hơn)
5. Điều này củng cố dự đoán và khuếch đại khuôn mẫu
6. Chu kỳ tiếp diễn, ngày càng gay gắt theo thời gian

Hệ thống có thể đang dự đoán chính xác *nơi các vụ bắt giữ sẽ xảy ra*, nhưng điều đó không giống với việc dự đoán nơi tội phạm xảy ra. AI đang mô hình hóa các khuôn mẫu thực thi, chứ không phải khuôn mẫu tội phạm.

2.10.3 Thiệt hại phân tán

Thiệt hại do AI gây ra thường mang tính phân tán: không một cá nhân nào chịu thiệt hại thảm khốc, nhưng hàng triệu người mỗi người đều chịu một chút thiệt hại. Một thuật toán đề xuất làm gia tăng nhẹ sự phân cực chính trị sẽ ảnh hưởng đến hàng tỷ người dùng. Một thuật toán

tuyển dụng gây thiệt hại nhẹ cho một số nhóm nhất định sẽ ảnh hưởng đến hàng triệu người tìm việc.

Những thiệt hại phân tán này khó thấy, khó đo lường, và khó quy trách. Không cá nhân nào có thể chỉ vào AI mà nói “hệ thống này đã làm hại tôi” theo cách có thể khởi kiện được. Tuy nhiên, thiệt hại tổng hợp có thể rất lớn.

2.11 Chi phí môi trường và tài nguyên

2.11.1 Tiêu thụ năng lượng

Việc huấn luyện các mô hình AI lớn đòi hỏi tài nguyên tính toán khổng lồ. Việc huấn luyện GPT-3 được ước tính tiêu tốn khoảng 1.287 MWh điện, tương đương mức tiêu thụ hàng năm của hơn 100 hộ gia đình Mỹ. Dấu chân carbon được ước tính ở mức 552 tấn CO₂, tương đương 500 chuyến bay khứ hồi giữa New York và San Francisco.³

Khi mô hình lớn dần, những chi phí này tăng theo. Chi phí huấn luyện chỉ trả một lần, nhưng chi phí suy luận (thực sự chạy mô hình) tích lũy qua mỗi lần sử dụng. Một mô hình phục vụ hàng triệu người dùng tiêu tốn đáng kể tài nguyên vận hành liên tục.

2.11.2 Sự tập trung quyền lực

Tài nguyên tính toán cần để huấn luyện các mô hình AI tiên phong chỉ nằm trong tay một vài tổ chức giàu có. Điều này tạo ra sự tập trung quyền lực: một số ít công ty kiểm soát các hệ thống AI mạnh nhất, dữ liệu mà chúng được huấn luyện, và các quyết định về cách chúng được triển khai.

Sự tập trung này đẩy lên những lo ngại về:

- Trách nhiệm giải trình dân chủ: Ai đã bầu các công ty này để định hình tương lai của AI?
- Cạnh tranh: Các tổ chức nhỏ hơn có thể cạnh tranh được nếu AI đòi hỏi khoản đầu tư hàng tỷ đô la không?
- Quyền tiếp cận: Ai được dùng các hệ thống AI mạnh nhất, và với cái giá nào?

2.11.3 Bạn có nên dùng dữ liệu này? Bạn có nên xây hệ thống này?

Vượt ra ngoài thiên vị và công bằng, còn có những câu hỏi đạo đức rộng hơn mà những người làm AI buộc phải tự hỏi:

³Patterson, D. et al. (2021). Carbon emissions and large neural network training. *arXiv:2104.10350*. <https://arxiv.org/abs/2104.10350>

- Dù mô hình của tôi không thiên vị, liệu quyết định này có nên được tự động hóa không?
- Dù dữ liệu được thu thập hợp pháp, liệu nó có được thu thập một cách đạo đức không?
- Dù tôi có thể xây hệ thống này, liệu tôi có nên xây không?
- Ai hưởng lợi từ hệ thống này? Ai bị thiệt hại? Những lợi ích có đáng với những thiệt hại không?
- Chuyện gì xảy ra nếu hệ thống này bị lạm dụng? Tôi có thể ngăn chặn điều đó không?

Suy luận về “Cái này có nên tồn tại không?”

Khi đánh giá xem một hệ thống AI có nên được xây dựng hay triển khai, hãy cân nhắc:

1. Tính cần thiết: AI có phải là giải pháp đúng không? Liệu cách tiếp cận đơn giản hơn có hiệu quả không? Liệu phán đoán của con người có phù hợp hơn không? **2. Tính tương xứng:** Lợi ích có biện minh cho rủi ro không? Ai gánh rủi ro so với ai nhận lợi ích?

3. Khả năng đảo ngược: Thiệt hại có thể sửa chữa được không? Một đơn xin việc bị từ chối oan có thể xem xét lại; một liệu pháp y tế bị từ chối oan có thể gây thiệt hại không thể phục hồi.

4. Các phương án thay thế: Chuyện gì xảy ra nếu không có hệ thống này? Hiện trạng tốt hơn hay tệ hơn? Có phương án trung gian nào không?

5. Sự đồng thuận: Những người bị ảnh hưởng có đồng ý chịu sự quản lý của hệ thống này không? Họ có phương án thay thế thực sự không? Không có câu trả lời phổ quát. Nhưng suy luận có cấu trúc giúp nhận diện những khía cạnh đạo đức đang đặt ra.

2.12 AI tạo sinh: những vấn đề cũ bị khuếch đại

Các phần trước đã phân tích những thách thức đạo đức áp dụng cho mọi hệ thống AI. Phần cuối cùng này giải thích vì sao các mô hình tạo sinh (Chương 10) và học sâu hiện đại (Chương 14) khiến chính những thách thức đó khó quản lý hơn đáng kể. Vấn đề không mới, nhưng quy mô của chúng thì có.

Quy mô ảnh hưởng. Một thuật toán tuyển dụng thiên vị ảnh hưởng đến từng cá nhân ứng viên. Một mô hình tạo sinh thiên vị tạo ra văn bản hoặc hình ảnh được hàng triệu người tiêu thụ, định hình các chuẩn mực văn hóa và sự hiểu biết của công chúng. Khi một hệ thống chuyển văn bản thành ảnh liên tục tạo ra nhà khoa học là người đàn ông da trắng, nó không chỉ phản ánh một tập dữ liệu thiên vị; nó tạo ra phương tiện mới củng cố thành kiến đó. Vòng lặp phản hồi từ Mục 2.10 giờ vận hành ở quy mô văn hóa, không chỉ ở từng quyết định cá nhân.

Dữ liệu huấn luyện bao trùm mọi thứ. Các hệ thống học máy truyền thống được huấn luyện

trên các tập dữ liệu đã tuyển chọn, thu thập cho một nhiệm vụ cụ thể. Các mô hình tạo sinh lớn được huấn luyện trên một phần đáng kể của internet, kể thừa mọi thiên vị, thành kiến và thông tin sai lệch có mặt trong dữ liệu đó. Bước tuyển chọn vốn có thể bắt được vấn đề trong một hệ thống nhỏ hơn hầu như không tồn tại.

Giám sát của con người giảm đi. Khi một bộ phân loại gán cờ một hồ sơ vay vốn, người kiểm duyệt có thể xem xét lại quyết định đó. Khi một mô hình tạo sinh tạo ra hàng triệu đoạn văn bản hoàn thiện hoặc hình ảnh mỗi ngày, việc con người xem xét thực sự từng đầu ra là bất khả thi. Đầu ra có hại có thể lan truyền trước khi ai kịp nhận ra.

Những hiệu ứng khuếch đại này tạo ra một số lo ngại riêng của các hệ thống tạo sinh:

Các mối lo ngại đạo đức riêng của tạo sinh

Deepfake: Các mô hình tạo sinh có thể tạo ra hình ảnh, âm thanh hoặc video chân thực về người thật trong các tình huống bịa đặt. Điều này cho phép thông tin sai lệch, gian lận và quấy rối ở quy mô mà trước đây là bất khả thi.

Bản quyền và quyền sở hữu dữ liệu: Các mô hình được huấn luyện trên hàng tỷ tác phẩm sáng tạo có thể tạo ra đầu ra giống hệt một số ví dụ huấn luyện cụ thể. Ai sở hữu đầu ra? Những người sáng tạo ban đầu có được bồi thường hoặc tham vấn không? Những câu hỏi này vẫn chưa được giải quyết về mặt pháp lý và đạo đức.

Chi phí môi trường: Việc huấn luyện một mô hình lớn duy nhất có thể tiêu tốn nhiều năng lượng bằng số năng lượng hàng trăm hộ gia đình dùng trong một năm. Sự tập trung tài nguyên tính toán vào tay một vài tổ chức giàu có đẩy lên câu hỏi về việc ai gánh chi phí môi trường và ai nhận lợi ích.

Các công cụ phân tích của chương này áp dụng trực tiếp. Hãy dùng mẫu năm điểm (DỮ LIỆU, MỤC TIÊU, CHỈ SỐ, THIẾT HẠI, GIẢM THIỂU) cho bất kỳ hệ thống tạo sinh nào. Hỏi xem thiên vị đi vào pipeline ở đâu. Chỉ ra các vòng lặp phản hồi. Công nghệ là mới; lập luận đạo đức thì vẫn vậy.

2.13 Tóm tắt

Điểm cần nhớ

Thiên vị: Các hệ thống AI khuếch đại các khuôn mẫu trong dữ liệu huấn luyện của chúng, bao gồm cả thiên vị và sự phân biệt đối xử lịch sử.

Công bằng: Có nhiều định nghĩa về công bằng (cân bằng dân số, xác suất bằng nhau, hiệu chuẩn), chúng mâu thuẫn về mặt toán học, và việc chọn giữa chúng là một lựa chọn đạo đức, chứ không phải kỹ thuật.

Quyền riêng tư: Việc thu thập dữ liệu, rủi ro tái ẩn danh hóa, và việc huấn luyện trên dữ liệu cá nhân làm dấy lên những lo ngại quyền riêng tư cơ bản. Quyền riêng tư khác biệt cung cấp các bảo đảm toán học nhưng đi kèm những đánh đổi. **Trách nhiệm giải trình:** Khi các hệ thống AI đưa ra những quyết định có hệ quả ngày càng lớn, khoảng trống nảy sinh giữa thiệt hại chúng gây ra và khả năng buộc ai đó chịu trách nhiệm.

Minh bạch: Các mô hình hộp đen có thể chính xác hơn nhưng khó diễn giải hơn. Các quyết định có mức độ rủi ro cao có thể đòi hỏi AI có khả năng giải thích. **Vòng lặp phản hồi:** Các hệ thống AI định hình thế giới mà chúng dự đoán, có khả năng củng cố và khuếch đại những bất bình đẳng sẵn có theo thời gian. **Chi phí môi trường:** Việc huấn luyện các mô hình lớn tiêu tốn năng lượng khổng lồ và tập trung quyền lực vào tay những tổ chức giàu có. **Những câu hỏi then chốt:** Ai hưởng lợi, dữ liệu của ai, ai quyết định, phương án thay thế là gì, ai chịu trách nhiệm giải trình, và liệu hệ thống này có nên tồn tại không?

2.14 Bài tập luyện

Định nghĩa chính thức

1. **(Định nghĩa)** Định nghĩa chính xác *ảnh hưởng phân biệt (disparate impact)*. Nếu Nhóm A có tỷ lệ phê duyệt p_A và Nhóm B có tỷ lệ phê duyệt p_B , viết điều kiện toán học của quy tắc 80%.
2. **(So sánh)** So sánh *cân bằng dân số (demographic parity)* và *xác suất bằng nhau (equalized odds)*. Cho một ví dụ cụ thể khi một hệ thống thỏa mãn cái này nhưng không thỏa mãn cái kia.
3. **(Tính toán)** Một bộ phân loại có các ma trận nhầm lẫn sau:

Nhóm A	Dự đoán +	Dự đoán -	Nhóm B	Dự đoán +	Dự đoán -
Thực tế +	80	20	Thực tế +	60	40
Thực tế -	10	90	Thực tế -	10	90

((a)) Tính TPR và FPR cho mỗi nhóm.

((b)) Hệ thống có thỏa mãn xác suất bằng nhau (equalized odds) không? Giải thích bằng toán.

((c)) Nó có thỏa mãn cân bằng dân số không? Trình bày phép tính của bạn.

Suy luận theo phong cách chứng minh

4. **(Chứng minh)** Chứng minh rằng nếu tỷ lệ nền khác nhau giữa các nhóm ($P(Y = 1|G = A) \neq P(Y = 1|G = B)$), thì một bộ phân loại hiệu chuẩn hoàn hảo không thể thỏa mãn cân bằng dân số.

Gợi ý: Bắt đầu từ định nghĩa của hiệu chuẩn: $P(Y = 1|\hat{Y} = 1, G) = \hat{p}$ với mọi nhóm.

5. **(Xây dựng)** Tạo một ví dụ đơn giản (2 đặc trưng, 2 nhóm, kết quả nhị phân) mà việc loại bỏ một đặc trưng thiên vị làm *tăng* lỗi cho cả hai nhóm. Giải thích vì sao điều này xảy ra.

6. **(Phân tích)** Một mô hình tuyển dụng dùng: chức danh công việc, số năm kinh nghiệm, mức lương ở công việc trước (giới tính bị loại trừ).

(a) Chứng minh hoặc bác bỏ: việc loại trừ giới tính đảm bảo mô hình không thể phân biệt đối xử theo giới tính.

(b) Chỉ ra đặc trưng nào có thể đóng vai trò làm proxy cho giới tính.

(c) Đề xuất một kiểm định thống kê để phát hiện phân biệt đối xử bằng proxy.

Áp dụng mẫu

7. Áp dụng mẫu năm điểm (DỮ LIỆU, MỤC TIÊU, CHỈ SỐ, THIẾT HẠI, GIẢM THIỂU) cho từng trường hợp sau:

- (a) Một hệ thống đề xuất nội dung tối đa hóa thời gian xem
- (b) Một hệ thống cảnh sát dự đoán dự báo các điểm nóng tội phạm
- (c) Một AI phân cấp bệnh nhân y tế ưu tiên các bệnh nhân cấp cứu

Quyền riêng tư và trách nhiệm giải trình

8. **(Quyền riêng tư)** Một ứng dụng thể dục thu thập dữ liệu GPS để theo dõi các lộ trình chạy bộ. Công ty tuyên bố dữ liệu đã được "ẩn danh hóa" trước khi bán cho các nhà quảng cáo.

(a) Giải thích vì sao dữ liệu GPS đặc biệt khó ẩn danh hóa thực sự.

(b) Mô tả làm thế nào dữ liệu này có thể bị nhận diện lại.

(c) Liệu quyền riêng tư khác biệt có giúp ích không? Đánh đổi sẽ là gì?

9. **(Trách nhiệm giải trình)** Một bệnh viện dùng một hệ thống AI để ưu tiên bệnh nhân cho

một liệu pháp mới. Hệ thống từ chối điều trị cho một bệnh nhân sau đó qua đời. Bệnh viện tuyên bố họ “chỉ tuân theo thuật toán”.

(a) Chỉ ra tất cả các bên có thể phải chịu trách nhiệm. (b) Bạn cần thông tin gì để xác định xem hệ thống có lỗi không?

(c) Liệu bệnh viện có nên được phép dùng các hệ thống AI mà họ không giải thích được không? Tranh luận cả hai phía.

10. **(Vòng lặp phản hồi)** Một AI phê duyệt khoản vay của ngân hàng được huấn luyện trên dữ liệu lịch sử cho thấy tỷ lệ trả nợ thấp hơn ở một số mã vùng bưu chính nhất định. AI học cách từ chối các hồ sơ từ những khu vực đó.

(a) Giải thích cách hệ thống này tạo ra một vòng lặp phản hồi.

(b) Sau 5 năm triển khai, bạn kỳ vọng sự chênh lệch sẽ tăng lên, giảm đi, hay giữ nguyên? Vì sao?

(c) Đề xuất một biện pháp can thiệp để phá vỡ vòng lặp phản hồi.

11. **(Minh bạch so với độ chính xác)** Bạn phải chọn giữa hai hệ thống chẩn đoán y tế:

- Hệ thống A: Cây quyết định, độ chính xác 85%, có thể diễn giải hoàn toàn
- Hệ thống B: Mạng nơ-ron sâu, độ chính xác 92%, hộp đen (a) Với những bệnh lý nào bạn có thể ưu tiên Hệ thống A dù độ chính xác thấp hơn?

(b) Với những bệnh lý nào bạn có thể ưu tiên Hệ thống B?

(c) Một bệnh nhân yêu cầu được biết vì sao Hệ thống B gắn cờ họ để làm thêm xét nghiệm. Bạn có thể nói gì với họ?

12. **(Môi trường)** Việc huấn luyện một mô hình ngôn ngữ lớn tốn 10 triệu C tiền tài nguyên tính toán và phát thải 500 tấn CO₂. Mô hình sẽ được 100 triệu người sử dụng.

(a) Tính chi phí huấn luyện trên mỗi người dùng và dấu chân carbon. (b) Nếu mỗi truy vấn của người dùng tốn 0,001 C tiền tài nguyên tính toán, tổng chi phí suy luận cho 1 tỷ truy vấn là bao nhiêu?

(c) Bạn sẽ quyết định thế nào nếu những lợi ích biện minh cho những chi phí này?

13. **(AI tạo sinh)** Các hệ thống chuyển văn bản thành ảnh có thể tạo ra hình ảnh chân thực về người thật trong những tình huống chưa từng xảy ra.

((a)) Những thiệt hại tiềm tàng nào mà điều này tạo điều kiện cho?

(b) Các hệ thống hiện tại triển khai những biện pháp bảo vệ nào? (c) Áp dụng mẫu năm điểm (DỮ LIỆU, MỤC TIÊU, CHỈ SỐ, THIẾT HẠI, GIẢM THIỂU) cho một hệ thống chuyển văn bản thành ảnh được huấn luyện trên hình ảnh internet.

Phần II

Học máy cốt lõi

Chương 3

Chuẩn bị dữ liệu và kỹ thuật đặc trưng

Nhiều dữ liệu hơn đánh bại thuật toán thông minh, nhưng dữ liệu tốt hơn đánh bại nhiều dữ liệu hơn.

Peter Norvig

3.1 Giới thiệu: Vì sao chuẩn bị dữ liệu quan trọng

Bạn đã thu thập một bộ dữ liệu, chọn một thuật toán đầy triển vọng, và bạn sẵn sàng huấn luyện mô hình học máy đầu tiên. Bạn chạy mã, chờ kết quả, và... mô hình hoạt động tệ hại. Điều gì đã sai?

Chín lần trong mười lần, vấn đề không nằm ở thuật toán. Vấn đề nằm ở dữ liệu.

Thuật toán học máy là những bộ tìm mẫu mạnh mẽ một cách đáng kinh ngạc, nhưng chúng cũng rất cứng nhắc, hiểu mọi thứ theo đúng nghĩa đen. Chúng tìm mọi mẫu có trong dữ liệu bạn đưa cho chúng — kể cả những mẫu bạn không hề mong muốn. Nếu dữ liệu chứa lỗi, thuật toán sẽ học những lỗi đó. Nếu dữ liệu có giá trị thiếu, thuật toán hoặc là gặp lỗi sập, hoặc là bịa ra con số. Nếu một đặc trưng được đo bằng hàng triệu trong khi đặc trưng khác đo bằng số thập phân, thuật toán chỉ chú ý đến những con số lớn.

Quy tắc vàng của học máy

Chất lượng mô hình của bạn bị giới hạn bởi chất lượng dữ liệu của bạn. “Rác vào, rác ra.” Không thuật toán nào, dù tinh vi đến đâu, có thể trích xuất những mẫu có ý nghĩa từ dữ liệu xấu.

Chương này đề cập các bước thiết yếu của **chuẩn bị dữ liệu**. Biến dữ liệu thô, lộn xộn thành định dạng sạch mà thuật toán có thể học hiệu quả.

3.2 Hiểu về cấu trúc dữ liệu

Học máy làm việc với dữ liệu dạng bảng: hàng đại diện cho **đối tượng** (còn gọi là thực thể hoặc mẫu), và cột đại diện cho **thuộc tính** (còn gọi là đặc trưng hoặc biến).

		Attributes				
Objects	↑	Age	Height	Weight	Color	Label
		25	175	70	Blue	Yes
		30	160	55	Red	No

Hình 3.1: Dữ liệu dạng bảng: hàng là đối tượng, cột là thuộc tính.

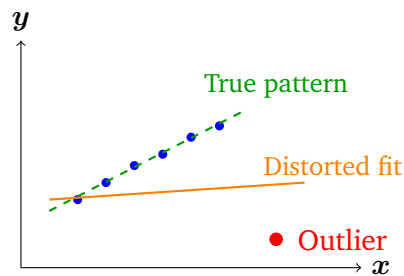
Thuộc tính số nhận giá trị liên tục (tuổi, chiều cao, nhiệt độ). **Thuộc tính phân loại** nhận giá trị từ một tập cố định (màu sắc, quốc gia, kích cỡ). Phân biệt này quyết định cách bạn xử lý từng thuộc tính.

3.3 Vấn đề chất lượng dữ liệu

Dữ liệu thực tế rất lộn xộn. Trước khi xây dựng bất kỳ mô hình nào, bạn phải xác định và xử lý các vấn đề chất lượng.

Nhiều là những lỗi ngẫu nhiên che mờ các mẫu thật. Sự thiếu chính xác của phép đo, lỗi nhập dữ liệu, và biến thiên tự nhiên đều góp phần tạo ra nhiễu khiến việc học trở nên khó khăn hơn.

Ngoại lai là những điểm dữ liệu khác biệt một cách kịch tính so với phần còn lại. Một ngoại lai duy nhất có thể làm biến dạng nghiêm trọng các mẫu đã học — một biệt thự định giá C50 triệu trong bộ dữ liệu nhà cửa giá C300 000 có thể làm lệch hoàn toàn một mô hình hồi quy.



Hình 3.2: Một ngoại lai có thể làm biến dạng kịch tính mẫu đã học.

Giá trị thiếu xảy ra khi một số thuộc tính không được ghi nhận. Bạn phải xóa các hàng không hoàn chỉnh (mất dữ liệu) hoặc điền khuyết giá trị thiếu bằng ước lượng như trung bình hoặc trung vị của cột.

Trùng lặp làm tăng trọng số cho một số ví dụ nhất định, làm lệch các mẫu đã học.

3.4 Chuẩn hóa đặc trưng

Nhiều thuật toán nhạy cảm với thang đặc trưng. Nếu thu nhập dao động từ €20 000 đến €200 000 trong khi tuổi dao động từ 18 đến 80, chênh lệch thu nhập sẽ áp đảo mọi phép tính khoảng cách, khiến thuật toán thực tế phớt lờ tuổi.

Chuẩn hóa z-score biến đổi mỗi đặc trưng để có trung bình 0 và độ lệch chuẩn 1:

$$\tilde{x}_i = \frac{x_i - \mu}{\sigma}$$

Ví dụ: Chuẩn hóa z-score

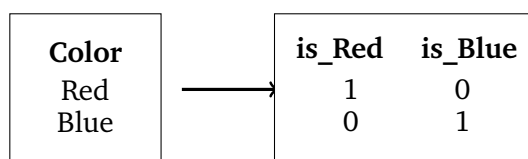
Dữ liệu: [10, 20, 30, 40, 50]
 Trung bình: $\mu = 30$, Độ lệch chuẩn: $\sigma \approx 14.14$
 Đã chuẩn hóa: [-1.41, -0.71, 0, 0.71, 1.41]

Chuẩn hóa min-max biến đổi các đặc trưng về khoảng [0, 1]:

$$\tilde{x}_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

3.5 Mã hóa các biến phân loại

Các thuật toán cần số, nhưng việc gán số tùy ý (đỏ=1, xanh=2) ngụ ý một thứ tự sai. **Mã hóa one-hot** chuyển một đặc trưng phân loại có k giá trị thành k cột nhị phân.



Hình 3.3: Mã hóa one-hot tạo các cột nhị phân cho từng danh mục.

3.6 Chia tập huấn luyện–kiểm định–kiểm tra

Để đánh giá mô hình khái quát hóa tốt đến đâu trên dữ liệu mới, chúng ta chia bộ dữ liệu thành các phần với vai trò khác nhau.

Phép ẩn dụ kỳ thi

Hãy coi **tập huấn luyện** là sách giáo khoa của bạn, **tập kiểm định** là các bài thi thử, và **tập kiểm tra** là kỳ thi cuối kỳ. Xem Chương 5 để thảo luận chi tiết về phương pháp đánh giá và cách chia dữ liệu.

Tập huấn luyện (thường 70%) là nơi việc học diễn ra. Tập kiểm định (15%) dùng để tinh chỉnh siêu tham số và đưa ra các quyết định về mô hình hóa. Tập kiểm tra (15%) được giữ riêng hoàn toàn cho đến khi đánh giá cuối cùng. Nó đại diện cho dữ liệu thực sự chưa từng thấy.

Quy tắc tối thượng

Không bao giờ huấn luyện trên dữ liệu kiểm tra. Không bao giờ dùng dữ liệu kiểm tra để ra quyết định. Khi bạn ngó thấy hiệu năng kiểm tra và điều chỉnh mô hình, tập kiểm tra không còn đại diện cho dữ liệu mới. Xem Chương 5 để thảo luận chi tiết về phương pháp đánh giá.

3.7 Kỹ thuật đặc trưng

Dữ liệu thô hiếm khi đến ở dạng lý tưởng cho học máy. **Kỹ thuật đặc trưng** là quá trình biến đổi dữ liệu thô thành các đặc trưng đại diện tốt hơn cho các mẫu nền, thường cải thiện đáng kể hiệu năng mô hình.

3.7.1 Tạo đặc trưng mới

Đôi khi các đặc trưng thô không phải là dạng đại diện có sức dự đoán cao nhất. Hãy xét việc dự đoán giá nhà:

- Đặc trưng thô: chiều dài, chiều rộng của lô đất

- Đặc trưng tốt hơn: diện tích = chiều dài $\times$ chiều rộng

Diện tích nắm bắt điều thực sự quan trọng (mất bao nhiêu đất) một cách trực tiếp hơn so với các chiều kích thước thô.

Các kỹ thuật đặc trưng phổ biến:

- **Đặc trưng đa thức:** Tạo x^2 , $x_1 \cdot x_2$, v.v. để nắm bắt các quan hệ phi tuyến
- **Nhóm giá trị (binning):** Chuyển biến liên tục thành các danh mục (tuổi $\rightarrow$ nhóm tuổi)
- **Biến đổi log:** Nén thang của các biến bị lệch rất nhiều
- **Đặc trưng ngày/giờ:** Trích xuất thứ trong tuần, tháng, năm, is_weekend từ các dấu thời gian

3.7.2 Kiến thức miền

Những đặc trưng tốt nhất thường đến từ việc hiểu vấn đề miền (domain). Một kỹ sư học máy làm việc về chấm điểm tín dụng nên tham vấn các chuyên gia tài chính để hiểu những tỷ lệ nào (nợ trên thu nhập, mức sử dụng tín dụng) là những dấu hiệu dự đoán có ý nghĩa.

Ví dụ: Kỹ thuật đặc trưng cho phát hiện gian lận

Đặc trưng thô: số tiền giao dịch, danh mục thương gia, dấu thời gian.

Đặc trưng được kỹ thuật hóa:

- Số tiền giao dịch tương đối so với mức trung bình của người dùng
- Thời gian kể từ giao dịch cuối cùng
- Số lượng giao dịch trong giờ qua
- Khoảng cách đến các vị trí điển hình của người dùng
- Liệu danh mục thương gia có bất thường đối với người dùng này không

Những đặc trưng được kỹ thuật hóa này nắm bắt các mẫu hành vi mà đặc trưng thô không thể nắm bắt.

3.7.3 Rò rỉ dữ liệu: Một cạm bẫy nghiêm trọng

Rò rỉ dữ liệu xảy ra khi thông tin vốn không khả dụng vào thời điểm dự đoán lại được sử dụng trong quá trình huấn luyện. Điều này tạo ra các điểm kiểm định cao một cách giả tạo, sụp đổ khi mô hình gặp dữ liệu thực tế. Rò rỉ xuất hiện ở hai dạng khác biệt nhau.

Rò rỉ mục tiêu

Rò rỉ mục tiêu xảy ra khi một đặc trưng được suy ra từ, hoặc là biến thể (proxy) của, nhân mà bạn đang cố gắng dự đoán.

Ví dụ: Rò rỉ mục tiêu

Bạn đang dự đoán liệu bệnh nhân có nhập viện trở lại trong vòng 30 ngày hay không. Mô hình của bạn đạt độ chính xác 99% trên dữ liệu kiểm định. Sau đó bạn nhận thấy một trong các đặc trưng của mình: `readmission_date`. Trường này chỉ được điền cho những bệnh nhân *đã* nhập viện trở lại. Đặc trưng này mã hóa trực tiếp nhân.

Các ví dụ khác:

- Dự đoán vỡ nợ khoản vay bằng “số lần trả nợ trễ” (điều này *chính là* vỡ nợ)
- Dự đoán bệnh bằng “thuốc được kê đơn cho bệnh đó”
- Dự đoán khách hàng rời đi bằng “lý do hủy”

Mẫu: bất kỳ đặc trưng nào chỉ có thể tồn tại vì kết quả đều là rò rỉ mục tiêu.

Rò rỉ đặc trưng (ô nhiễm huấn luyện–kiểm tra)

Rò rỉ đặc trưng xảy ra khi thông tin từ tập kiểm tra làm nhiễm bẩn quá trình huấn luyện, mặc dù không có đặc trưng riêng lẻ nào được suy ra từ nhân.

Ví dụ: Rò rỉ đặc trưng

Các nguồn rò rỉ đặc trưng phổ biến:

- Khớp một bộ chuẩn hóa trên *toàn bộ* dữ liệu (bao gồm cả dữ liệu kiểm tra) trước khi chia tập
- Tính trung bình cho việc điền khuyết giá trị thiếu bằng tất cả các hàng, rồi mới chia tập
- Các hàng trùng lặp hoặc gần trùng lặp xuất hiện ở cả tập huấn luyện và tập kiểm tra
- Dữ liệu chuỗi thời gian bị xáo trộn ngẫu nhiên thay vì được chia theo thứ tự thời gian

Mẫu: bất kỳ bước tiền xử lý nào cho phép thống kê của tập kiểm tra ảnh hưởng đến phép biến đổi của huấn luyện đều là rò rỉ đặc trưng. Cách đúng là chỉ khớp (fit) mọi phép biến đổi trên tập huấn luyện, rồi áp dụng các phép biến đổi đó cho tập kiểm định và tập kiểm tra.

Rò rỉ mục tiêu so với rò rỉ đặc trưng		
Cái gì bị rò rỉ?	Rò rỉ mục tiêu	Rò rỉ đặc trưng
Nguyên nhân	Thông tin từ nhân vào một đặc trưng	Thông tin từ tập kiểm tra vào quá trình huấn luyện
Phát hiện	Đặc trưng được suy ra từ kết quả	Tiền xử lý dùng dữ liệu tương lai/kiểm tra
Fix	"Tôi có thứ này trước kết quả không?" Bỏ đặc trưng đó	"Dữ liệu kiểm tra có ảnh hưởng đến huấn luyện không?" Sửa thứ tự tiền xử lý

Liên hệ với thiên vị-phương sai (Chương 5)

Kỹ thuật đặc trưng ảnh hưởng trực tiếp đến đánh đổi thiên vị-phương sai:

- **Quá ít đặc trưng:** Thiên vị cao (thiếu khớp), mô hình không nắm bắt được các mẫu quan trọng
- **Quá nhiều đặc trưng:** Phương sai cao (quá khớp), mô hình ghi nhớ nhiều
- **Đặc trưng bị rò rỉ:** Lỗi huấn luyện và kiểm định thấp một cách giả tạo, nhưng lỗi kiểm tra thảm khốc

Mục tiêu là có những đặc trưng nắm bắt tín hiệu thật mà không đưa vào nhiễu hay rò rỉ.

Liên kết then chốt: Rò rỉ và lỗi kiểm định

Cả hai dạng rò rỉ đều gây ra **lỗi kiểm định lạc quan**: điểm kiểm chéo của bạn trông tuyệt vời, nhưng hiệu năng thực tế lại tệ hại. Rò rỉ mục tiêu làm thổi phồng điểm vì mô hình có quyền truy cập câu trả lời. Rò rỉ đặc trưng làm thổi phồng điểm vì tập kiểm định không còn thực sự độc lập.

Chỉ tập kiểm tra cuối cùng (hoặc việc triển khai thực tế) mới bộc lộ sự thật. Xem Chương 5 để biết vì sao phương pháp kiểm định đúng đắn rất cần thiết và rò rỉ làm suy yếu điều đó như thế nào.

3.8 Thiên vị lấy mẫu: Vấn đề trước khi bắt đầu mô hình hóa

Ngay cả trước khi chọn thuật toán, bộ dữ liệu của bạn có thể đã khiếm khuyết vì *cách* dữ liệu được thu thập. Thiên vị lấy mẫu (sampling bias) nghĩa là dữ liệu không đại diện cho quần thể mà bạn muốn đưa ra dự đoán.

3.8.1 Thiên vị chọn lọc

Thiên vị chọn lọc xảy ra khi quá trình thu thập dữ liệu loại trừ có hệ thống một số nhóm hoặc tình huống nhất định. Bộ dữ liệu thu được không đại diện cho toàn bộ quần thể.

Một nghiên cứu y khoa chỉ thực hiện ở bệnh viện thành phố sẽ đại diện thiếu bệnh nhân nông thôn. Một cuộc khảo sát chỉ bằng tiếng Anh sẽ bỏ lỡ những người không nói tiếng Anh. Một AI được huấn luyện trên dữ liệu từ một quốc gia có thể thất bại ở quốc gia khác.

3.8.2 Thiên vị sống sót

Thiên kiến sống sót xảy ra khi dữ liệu của bạn chỉ chứa những ví dụ đã “sống sót” qua một quá trình chọn lọc nào đó, và bạn rút ra kết luận như thể chúng đại diện cho toàn bộ các trường hợp.

Ví dụ kinh điển: Thiên kiến sống sót

Trong Thế chiến thứ hai, các kỹ sư nghiên cứu các lỗ hổng đạn trên máy bay quay về từ chiến trường và đề xuất gia cố những vùng bị trúng đạn nhiều nhất. Nhà thống kê Abraham Wald chỉ ra lỗ hổng trong đó: họ chỉ nhìn thấy những máy bay đã *sống sót*. Những máy bay bị trúng đạn ở các vùng khác không bao giờ quay về. Quân đội nên gia cố những vùng *không* có lỗ hổng đạn trên máy bay sống sót.

Trong học máy, thiên kiến sống sót xuất hiện khi bạn huấn luyện trên dữ liệu từ những khách hàng đã ở lại (bỏ qua những người đã rời đi), những công ty đã thành công (bỏ qua những công ty thất bại), hoặc những bệnh nhân đã hoàn thành nghiên cứu (bỏ qua những người đã bỏ cuộc).

3.8.3 Chọn mẫu thuận tiện

Chọn mẫu thuận tiện nghĩa là thu thập dữ liệu từ bất kỳ nguồn nào dễ tiếp cận nhất. Huấn luyện một hệ thống nhận dạng khuôn mặt trên ảnh của sinh viên đại học (vì họ ở gần đó) sẽ tạo ra một mô hình thiên lệch về nhóm dân số trẻ, có học vấn và sống trong khuôn viên trường. Huấn luyện một mô hình ngôn ngữ trên văn bản Internet sẽ đại diện quá mức cho những người dùng nói tiếng Anh và có kỹ năng kỹ thuật.

Hầu hết các tập dữ liệu huấn luyện AI quy mô lớn đều mắc phải mức độ chọn mẫu thuận tiện nhất định.

3.8.4 Thiên kiến không phản hồi

Khi bạn thu thập dữ liệu qua khảo sát hoặc tham gia tự nguyện, những người trả lời có thể khác có hệ thống so với những người không trả lời. Đánh giá sản phẩm nghiêng về những người có cảm xúc mạnh (hoặc rất hài lòng, hoặc rất

thất vọng). Các nghiên cứu y tế mất đi những người tham gia khi bệnh quá nặng để tiếp tục. Khảo sát mức độ hài lòng công việc lại đại diện thiếu cho những người đã nghỉ việc.

Kết nối với IAIO: Nhận diện thiên kiến chọn mẫu

Các bài thi IAIO có thể đưa ra một tập dữ liệu và yêu cầu bạn nhận diện những thiên kiến chọn mẫu tiềm ẩn. Với mỗi tập dữ liệu, hãy hỏi:

- Ai hoặc cái gì bị *thiếu* khỏi dữ liệu này?
- Quá trình nào đã tạo ra dữ liệu này, và quá trình đó loại trừ ai?
- Kết quả có thay đổi không nếu dữ liệu bị thiếu được đưa vào?

3.9 Dịch chuyển phân phối: Khi thế giới thay đổi

Một mô hình được huấn luyện trên một phân phối dữ liệu này có thể gặp một phân phối khác khi triển khai. Điều này được gọi là **dịch chuyển phân phối**, và nó là một trong những lý do phổ biến nhất khiến các mô hình đã triển khai thất bại.

3.9.1 Dịch chuyển đồng biến (covariate shift)

Phân phối đầu vào thay đổi, nhưng mối quan hệ giữa đầu vào và đầu ra vẫn giữ nguyên.

Ví dụ: Một mô hình được huấn luyện để chẩn đoán bệnh từ ảnh X-quang tại Bệnh viện A được triển khai tại Bệnh viện B, nơi dùng các máy X-quang khác. Những hình ảnh này trông khác nhau (đầu vào bị dịch chuyển), nhưng mối quan hệ cơ bản giữa bệnh lý và các đặc trưng ảnh không hề thay đổi. Mô hình vẫn có thể thất bại bởi vì nó đã học các đặc trưng riêng của thiết bị tại Bệnh viện A.

Về mặt hình thức: $P_{\text{train}}(X) \neq P_{\text{deploy}}(X)$, nhưng $P(Y|X)$ vẫn giữ nguyên.

3.9.2 Dịch chuyển nhãn (label shift)

Phân phối kết quả thay đổi, nhưng mối quan hệ từ kết quả tới đặc trưng vẫn giữ nguyên.

Ví dụ: Một bộ lọc thư rác được huấn luyện khi 5% email là thư rác, rồi được triển khai khi 40% email là thư rác. Ngưỡng của bộ lọc, vốn được hiệu chỉnh cho tỉ lệ thư rác 5%, sẽ bỏ sót nhiều email thư rác. Bản thân thư rác trông như nhau; chỉ là có nhiều hơn thôi.

Về mặt hình thức: $P_{\text{train}}(Y) \neq P_{\text{deploy}}(Y)$, nhưng $P(X|Y)$ vẫn giữ nguyên.

3.9.3 Trôi dạng khái niệm (concept drift)

Mối quan hệ cơ bản giữa đầu vào và đầu ra thay đổi theo thời gian.

Ví dụ: Một mô hình dự đoán người vay “đủ khả năng tín dụng” được huấn luyện trước một cuộc suy thoái lớn. Trong thời kỳ suy thoái, chính các đặc trưng thu nhập và việc làm đó không còn dự đoán việc trả nợ theo cách như trước nữa, bởi chính nền kinh tế đã thay đổi. Khái niệm “đủ khả năng tín dụng” đã dịch chuyển.

Về mặt hình thức: $P_{\text{train}}(Y|X) \neq P_{\text{deploy}}(Y|X)$.

Ba loại dịch chuyển phân phối		
Loại	Điều gì thay đổi	Ví dụ
Covariate shift	Phân phối đầu vào $P(X)$	Bệnh viện mới, thiết bị khác
Dịch chuyển nhãn (label shift)	Phân phối đầu ra $P(Y)$	Tỉ lệ thư rác tăng
Concept drift	Mối quan hệ $P(Y X)$	Suy thoái làm thay đổi các kiểu hình vỡ nợ

Trôi dạng khái niệm nguy hiểm nhất, vì các quy tắc mà mô hình đã học trở nên sai, chứ không chỉ lệch độ chính xác. Các mô hình được triển khai trong những môi trường thay đổi cần được theo dõi liên tục.

3.10 Khi nào một tập kiểm tra là hợp lệ?

Tập kiểm tra đo lường sự khái quát hóa: mô hình sẽ hoạt động tốt ra sao trên dữ liệu mới. Nhưng phép đo này chỉ có ý nghĩa nếu tập kiểm tra đại diện cho dữ liệu mà mô hình thực sự sẽ gặp khi triển khai.

3.10.1 Yêu cầu về tính đại diện

Nếu bạn huấn luyện và kiểm tra trên dữ liệu bệnh viện từ năm 2020, độ chính xác kiểm tra cho bạn biết mô hình hoạt động tốt ra sao trên *dữ liệu bệnh viện năm 2020*. Nó không nói gì về hiệu năng vào năm 2025, ở một bệnh viện khác, ở một quốc gia khác, hay trên một nhóm bệnh nhân khác.

Một tập kiểm tra chỉ hợp lệ nếu:

- Nó được lấy từ cùng phân phối mà mô hình sẽ gặp khi triển khai
- Nó không được dùng để đưa ra bất kỳ quyết định xây dựng mô hình nào (không ngẫm nhìn trộm)
- Nó đủ lớn để ước lượng độ chính xác là đáng tin cậy về mặt thống kê

3.10.2 Khi tập kiểm tra nói dối

Tập kiểm tra có thể tạo ra cảm giác an toàn giả

Hãy xét một mô hình được huấn luyện và kiểm tra trên dữ liệu từ một bệnh viện. Độ chính xác kiểm tra là 95%. Sau đó mô hình được triển khai tại một bệnh viện thứ hai, nơi độ chính xác giảm xuống còn 70%. Chuyện gì đã sai? Tập kiểm tra không *sai*, nó đo chính xác hiệu năng trên dữ liệu của bệnh viện đó. Nhưng nó *gây hiểu lầm* vì không đại diện cho bối cảnh triển khai. Điểm kiểm tra không phải là lời bảo đảm. Nó chỉ là một ước lượng, và chỉ hợp lệ dưới giả định rằng dữ liệu triển khai giống dữ liệu kiểm tra.

3.10.3 Chia theo thời gian

Với dữ liệu phụ thuộc thời gian, cách chia ngẫu nhiên là không hợp lệ. Nếu bạn trộn ngẫu nhiên giá cổ phiếu từ 2020–2024 rồi chia thành tập huấn luyện và tập kiểm tra, mô hình có thể dùng giá trong tương lai để dự đoán giá trong quá khứ.

Cách đúng đắn: huấn luyện trên dữ liệu trước đó, kiểm tra trên dữ liệu sau đó. Điều này mô phỏng đúng bối cảnh triển khai thực tế, nơi bạn luôn dự đoán tương lai chỉ dựa vào quá khứ.

3.10.4 Kết nối các mảnh lại với nhau

Thiên kiến chọn mẫu, dịch chuyển phân phối và tính hợp lệ của tập kiểm tra đều là những mặt khác nhau của cùng một vấn đề: *dữ liệu của bạn có đại diện cho tình huống mà mô hình sẽ được sử dụng hay không?*

- Thiên kiến chọn mẫu nghĩa là dữ liệu huấn luyện chưa bao giờ mang tính đại diện ngay từ đầu
- Dịch chuyển phân phối nghĩa là thế giới đã thay đổi sau khi dữ liệu được thu thập
- Tập kiểm tra không hợp lệ nghĩa là bạn thậm chí không thể đo lường vấn đề

3.11 Tóm tắt

Điểm cần nhớ

Chất lượng dữ liệu: Xử lý nhiễu, ngoại lai, giá trị thiếu và dữ liệu trùng lặp trước khi xây dựng mô hình.

Chuẩn hóa đặc trưng: Dùng chuẩn hóa theo trung bình và độ lệch chuẩn, hoặc chuẩn hóa min-max, để mọi đặc trưng đóng góp như nhau.

Mã hóa biến phân loại: Dùng mã hóa one-hot để tránh tạo ra một thứ tự giả.

Chia tập dữ liệu: Huấn luyện để học, kiểm định để tinh chỉnh, kiểm tra để đánh giá cuối cùng. Không bao giờ ngăm nhìn vào dữ liệu kiểm tra.

Kỹ thuật đặc trưng: Biến dữ liệu thô thành những đặc trưng đại diện tốt hơn cho các mô hình nền tảng. Kiến thức chuyên ngành là vô cùng quý giá. **Rò rỉ:** Rò rỉ mục tiêu (các đặc trưng suy ra từ nhãn) và rò rỉ đặc trưng (dữ liệu kiểm tra làm nhiễu dữ liệu huấn luyện) đều tạo ra kết quả lạc quan một cách sai lệch.

Thiên kiến chọn mẫu: Thiên kiến chọn lọc, thiên kiến sống sót, chọn mẫu thuận tiện và thiên kiến không phản hồi đều có thể tạo ra những tập dữ liệu không mang tính đại diện.

Dịch chuyển phân phối: Dịch chuyển đồng biến, dịch chuyển nhân và trôi dạt khái niệm khiến mô hình thất bại khi thế giới thay đổi sau khi huấn luyện. **Tính hợp lệ của tập kiểm tra:** Điểm kiểm tra chỉ có ý nghĩa nếu tập kiểm tra đại diện cho bối cảnh triển khai.

3.12 Bài tập luyện

1. Cho dữ liệu [2, 4, 6, 8, 10], hãy tính trung bình và độ lệch chuẩn, sau đó áp dụng chuẩn hóa theo trung bình và độ lệch chuẩn.
2. Một đặc trưng "Size" có các giá trị {Small, Medium, Large}. Mã hóa one-hot tạo ra bao nhiêu cột? Mã hóa của "Medium" là gì?
3. Với 5000 mẫu và cách chia 70-15-15, mỗi tập sẽ có bao nhiêu mẫu?
4. Bạn có một tập dữ liệu với các cột: timestamp, user_id, purchase_amount, product_category. Hãy đề xuất ba đặc trưng được thiết kế có thể giúp dự đoán liệu người dùng có mua hàng vào tuần sau hay không.
5. Một tập dữ liệu có 10% giá trị thiếu trong cột "income". Hãy mô tả hai cách tiếp cận khác nhau để xử lý điều này, và thảo luận khi nào bạn có thể ưu tiên mỗi cách.

6. Tại sao việc áp dụng phép biến đổi logarithm cho đặc trưng "house price" có thể cải thiện hiệu năng của mô hình?

7. **(Tư duy đối kháng)** Bạn nhận được một tập dữ liệu mà việc áp dụng tiền xử lý tiêu chuẩn đạt độ chính xác 98%, nhưng mô hình lại hoàn toàn thất bại trên dữ liệu thực tế mới. Hãy mô tả ba cách cụ thể mà quy trình tiền xử lý có thể đã gây ra vấn đề này.

8. **(Nếu-thì)** Một tập dữ liệu có các đặc trưng với thang đo rất khác nhau: age (0–100), income (\$0–\$1,000,000), và số con (0–10).

(a) Những vấn đề nào có thể phát sinh nếu bạn huấn luyện một bộ phân loại kNN mà không chuẩn hóa?

(b) Những vấn đề nào có thể phát sinh nếu bạn huấn luyện một cây quyết định mà không chuẩn hóa?

(c) Dựa trên các câu trả lời của bạn, những thuật toán nào là "bất biến theo thang đo" (scale-invariant) và vì sao?

9. **(Nhiều phần)** Một tập dữ liệu có 10% giá trị thiếu trong cột "income".

(a) Mô tả ba chiến lược khác nhau để xử lý những giá trị thiếu này.

(b) Với mỗi chiến lược, hãy mô tả một tình huống mà nó sẽ là lựa chọn tốt nhất.

(c) Với mỗi chiến lược, hãy mô tả một tình huống mà nó sẽ gây hại.

10. **(Phát hiện rò rỉ đặc trưng)** Bạn đang dự đoán khách hàng rời bỏ (liệu khách hàng có hủy đăng ký hay không). Một đồng nghiệp đề xuất những đặc trưng sau:

(a) Số ngày kể từ lần đăng nhập gần nhất

(b) Tổng số lượt mua trong suốt vòng

đời (c) "Lý do hủy" (trường văn bản)

(d) Thời lượng phiên trung bình (e) Số

phiếu hỗ trợ đã gửi

Đặc trưng nào có thể là rò rỉ? Với mỗi khả năng rò rỉ, hãy giải thích lý do và cách bạn sẽ kiểm chứng.

11. **(Đối kháng: Khi nào KHÔNG nên hành động)** Bạn đang xây dựng một mô hình dự đoán giá nhà. Một đồng nghiệp lo ngại về những đặc trưng sau:

(a) Diện tích mặt bằng của ngôi nhà

- (b) Số phòng ngủ
- (c) Năm ngôi nhà được xây
- (d) Toạ độ vĩ độ và kinh độ

Họ lo rằng những đặc trưng này có thể là “rò rỉ” vì chúng tương quan mạnh với giá. Hãy giải thích vì sao đây **không** phải là rò rỉ và nên được giữ lại trong mô hình. Điều gì phân biệt đặc trưng dự đoán hợp lệ với rò rỉ thực sự?

12. **(Rò rỉ mục tiêu và rò rỉ đặc trưng)** Hãy phân loại từng mục sau là rò rỉ mục tiêu, rò rỉ đặc trưng hay không phải rò rỉ. Hãy lý giải cho từng câu trả lời.

- (a) Dự đoán điểm thi bằng “số giờ đã học” (thu thập trước kỳ thi)
- (b) Dự đoán điểm thi bằng “mức độ hài lòng của học sinh với điểm số của họ” (thu thập sau)
- (c) Dùng giá trị trung bình của toàn bộ tập dữ liệu (huấn luyện + kiểm tra) để điền các giá trị thiếu
- (d) Dự đoán khả năng nhập viện lại bằng “thời gian nằm viện ban đầu” (biết được vào lúc xuất viện)

13. **(Thiên kiến chọn mẫu)** Với mỗi tình huống, hãy nhận diện loại thiên kiến chọn mẫu và giải thích nhóm dân số nào bị đại diện thiếu:

- (a) Một xe tự lái được huấn luyện trên hình quay từ camera hành trình tại California
- (b) Một mô hình phân tích cảm xúc được huấn luyện trên các đánh giá sản phẩm
- (c) Một nghiên cứu về hiệu quả của một loại thuốc mới, trong đó 30% người tham gia đã bỏ cuộc trước khi hoàn thành
- (d) Một tập dữ liệu về các “startup thành công” được dùng để dự đoán startup nào sẽ thành công

14. **(Dịch chuyển phân phối)** Một mô hình dự đoán nhu cầu taxi tại Thành phố New York được huấn luyện trên dữ liệu năm 2019 và triển khai vào tháng 3 năm 2020.

- (a) Loại dịch chuyển phân phối nào đã xảy ra? Hãy lý giải cho câu trả lời của bạn.
- (b) Huấn luyện lại trên dữ liệu tháng 1–2 năm 2020 có giúp ích không? Vì sao hoặc vì sao không?
- (c) Đề xuất một chiến lược theo dõi có thể phát hiện sớm thất bại này.

15. **(Tính hợp lệ của tập kiểm tra)** Một nhóm huấn luyện một mô hình phát hiện ung thư da trên ảnh từ các phòng khám da liễu ở Bắc Âu. Tập kiểm tra là

một phần ngẫu nhiên 15% từ chính các phòng khám đó. Độ chính xác kiểm tra là 94%.

- (a) Mô hình được triển khai tại các phòng khám ở Châu Phi Hậu Sahara. Tại sao hiệu năng có thể giảm?
 - (b) Độ chính xác kiểm tra 94% có “sai” không? Hãy giải thích nó cho bạn biết điều gì và không cho bạn biết điều gì.
 - (c) Nhóm nên xây dựng tập kiểm tra của mình như thế nào nếu họ biết mô hình sẽ được triển khai trên toàn cầu?
-

Chương 4

Học có giám sát

Mọi mô hình đều sai, nhưng có mô hình lại rất hữu ích.

George Box

4.1 Giới thiệu về học có giám sát

Hãy tưởng tượng bạn dạy một đứa trẻ nhận biết động vật. Bạn đưa cho chúng từng tấm hình một: “Đây là mèo,” “Đây là chó,” “Đây là mèo,” “Đây là chó.” Sau khi xem đủ nhiều ví dụ, một điều kỳ diệu xảy ra. Khi được đưa một tấm hình hoàn toàn mới (một con vật chúng chưa từng thấy), đứa trẻ vẫn phân biệt được đúng đó là mèo hay chó. Nó đã học được cách khái quát từ những ví dụ cụ thể sang một khái niệm rộng hơn.

Đây chính xác là cách **học có giám sát** hoạt động. Chúng ta cung cấp cho thuật toán một bộ ví dụ mà ta đã biết đáp án đúng, và thuật toán tự tìm ra khuôn mẫu ẩn bên dưới nổi đầu vào với đầu ra. Khi đã học được khuôn mẫu đó, nó có thể dự đoán cho những ví dụ mới mà nó chưa từng gặp.

Thuật ngữ “giám sát” xuất phát từ ý tưởng rằng chúng ta đang giám sát quá trình học bằng cách đưa ra đáp án đúng. Hãy hình dung như giáo viên chấm bài tập: với mỗi câu, học sinh (tức thuật toán) vừa thấy đề bài vừa thấy đáp án đúng, và học được từ những chỗ làm sai dọc đường.

4.1.1 Cấu trúc của học có giám sát

Trong học có giám sát, dữ liệu của ta gồm các cặp đầu vào – đầu ra. Các đầu vào được gọi là **đặc trưng**, đó là những thông tin ta dùng để đưa ra dự đoán. Với một bức ảnh, đặc trưng có thể là các giá trị điểm ảnh. Với một ngôi nhà, đặc trưng có thể gồm diện tích, số phòng ngủ và vị trí. Với một bệnh nhân, đặc trưng có thể gồm tuổi, huyết áp và mức cholesterol.

Các đầu ra được gọi là **nhãn**, đó là thứ ta muốn dự đoán. Với bức ảnh, nhãn có thể là “mèo” hay “chó.” Với ngôi nhà, nhãn có thể là giá của nó. Với bệnh nhân, nhãn có thể là người đó có mắc một bệnh cụ thể hay không.

Tập các cặp đầu vào – đầu ra mà ta dùng để huấn luyện thuật toán được gọi là **tập huấn luyện**. Đó là dữ liệu mà thuật toán học từ đó. Song song với đó, ta giữ lại một **tập kiểm thử**, tức dữ liệu mà thuật toán không bao giờ được thấy trong lúc huấn luyện. Ta dùng tập này để đánh giá thuật toán đã học được mức độ khái quát tốt đến đâu.

Thuật ngữ quan trọng

Đặc trưng (ký hiệu x): Các biến đầu vào dùng để đưa ra dự đoán. Cũng gọi là biến dự đoán, thuộc tính, hay biến độc lập. **Nhãn** (ký hiệu y): Biến đầu ra mà ta muốn dự đoán. Cũng gọi là biến mục tiêu, biến đáp ứng, hay biến phụ thuộc.

Tập huấn luyện: Tập hợp các cặp (đặc trưng, nhãn) dùng để huấn luyện mô hình. Thuật toán học từ những ví dụ này.

Tập kiểm thử: Dữ liệu được tách ra khỏi quá trình huấn luyện, dùng để đánh giá mô hình khái quát tốt đến đâu trên những ví dụ mới, chưa từng thấy.

4.1.2 Vì sao học có giám sát lại hiệu quả

Giả định nền tảng của học có giám sát là dữ liệu chứa các khuôn mẫu: những quy luật nối đầu vào với đầu ra. Nếu giả nhà hoàn toàn ngẫu nhiên, không liên quan gì đến diện tích, vị trí hay bất kỳ đặc trưng nào đo lường được, thì chẳng thuật toán nào dự đoán được. Nhưng thực tế thì ngôi nhà lớn hơn thường đắt hơn, nhà ở khu vực mong muốn cũng thường đắt hơn, và cứ thế. Những khuôn mẫu ấy nằm sẵn trong dữ liệu, chờ được khám phá.

Việc của một thuật toán học có giám sát là tìm một hàm h (gọi là **giả thuyết**) ánh xạ từ đầu vào sang đầu ra: với các đặc trưng x , hàm tạo ra một dự đoán $h(x)$ vốn phải gần với nhãn thật y . Thuật toán tìm ra khuôn mẫu thật càng chính xác, dự đoán của nó trên dữ liệu mới càng chuẩn.

4.2 Phân loại so với Hồi quy

Bài toán học có giám sát gồm hai dạng chính, tùy thuộc vào bản chất của thứ mà ta muốn dự đoán.

4.2.1 Hồi quy: Dự đoán các giá trị liên tục

Trong các bài toán **hồi quy**, nhãn là một số thực liên tục. Nó có thể nhận bất kỳ giá trị nào trong một khoảng nào đó. Khi dự đoán nhiệt độ ngày mai, bạn có thể nhận được 22.4°C hoặc 20.4°C hay con số nào khác. Khi dự đoán giá nhà, bạn có thể nhận được C247 500 hoặc C312 000 hay bất cứ số nào ở giữa.

Các bài toán hồi quy thường gặp gồm dự đoán giá nhà dựa trên đặc điểm của nó, dự báo giá cổ phiếu hoặc doanh số, ước lượng tuổi một người qua ảnh chụp, và dự đoán một chiếc máy sẽ chạy được bao lâu thì hỏng. Trong mỗi trường hợp, đầu ra là một con số có thể thay đổi liên tục.

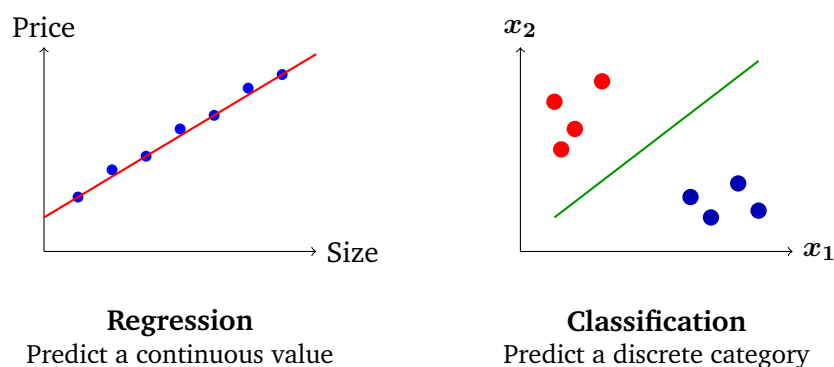
Đặc điểm then chốt của hồi quy là những thay đổi nhỏ ở đầu vào có thể dẫn tới những thay đổi nhỏ ở đầu ra. Một ngôi nhà lớn hơn một chút có thể được dự đoán là đắt hơn một chút. Chính mối liên hệ mượt mà giữa đầu vào và đầu ra làm nên sự khác biệt giữa hồi quy và phân loại.

4.2.2 Phân loại: Dự đoán các nhóm

Trong các bài toán **phân loại**, nhãn là một nhóm rời rạc. Nó thuộc về một trong số hữu hạn các lớp có thể có. Khi phân loại một email, nó hoặc là spam hoặc không phải spam; không có kiểu "37% là spam." Khi chẩn đoán một bệnh, bệnh nhân hoặc mắc hoặc không mắc. Khi nhận dạng một chữ số viết tay, đó là 0, 1, 2, 3, 4, 5, 6, 7, 8, hay 9, không có kiểu "chữ số 3.7."

Khi có đúng hai lớp khả dĩ, ta gọi đó là **phân loại nhị phân**. Ví dụ gồm phát hiện spam (spam so với không spam), chẩn đoán y tế (có bệnh so với không bệnh), và phát hiện gian lận (gian lận so với hợp lệ). Khi có nhiều hơn hai lớp, ta gọi đó là **phân loại đa lớp**. Ví dụ gồm nhận dạng chữ số viết tay (10 lớp), nhận dạng loài động vật (nhiều lớp), và phân loại bài báo theo chủ đề.

Sự khác biệt căn bản so với hồi quy là những thay đổi nhỏ ở đầu vào có thể dẫn tới những thay đổi đột ngột ở đầu ra. Một email rất giống spam vẫn có thể bị phân loại là không spam nếu nó rơi vào phía đúng của ranh giới quyết định. Bản chất rời rạc, trắng đen của phân loại đòi hỏi những thuật toán và phương pháp đánh giá khác hẳn so với hồi quy.



Hình 4.1: Trong hồi quy, ta khớp một đường thẳng (hoặc đường cong) để dự đoán các giá trị liên tục. Trong phân loại, ta tìm một ranh giới tách các nhóm khác nhau.

4.3 Hồi quy tuyến tính

4.3.1 Hàm mất mát: Thứ mà thuật toán tối ưu

Mọi thuật toán học có giám sát đều hoạt động bằng cách cực tiểu hóa một **hàm mất mát**: thước đo xem dự đoán của nó sai nhiều đến đâu. Việc chọn hàm mất mát quyết định mô hình quan tâm đến loại lỗi nào nhất, và mỗi nhiệm vụ khác nhau lại cần một hàm mất mát khác nhau.

Mất mát bình phương (dùng trong hồi quy) phạt những lỗi lớn theo bình phương:

$$L(y, \hat{y}) = (y - \hat{y})^2$$

Một dự đoán lệch 10 bị phạt nặng gấp 100 lần một dự đoán lệch 1. Điều này khiến các mô hình hồi quy nhạy cảm với ngoại lệ: chỉ một dự đoán sai lầm khủng khiếp cũng đủ chi phối toàn bộ mất mát.

Mất mát log / chéo entropy (dùng trong hồi quy logistic) đo mức khớp giữa xác suất dự đoán và các lớp thực tế:

$$L(y, \hat{p}) = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

Một dự đoán sai mà vẫn tự tin (dự đoán 0.99 trong khi nhãn thật là 0) gây ra mức mất mát khổng lồ. Một dự đoán đúng mà tự tin gần như không gây mất mát nào. Điều này buộc các bộ phân loại phải tạo ra các xác suất tách bạch rõ ràng.

Mất mát hinge (dùng trong SVM) chỉ quan tâm xem dự đoán có nằm ở phía đúng của biên hay không:

$$L(y, f(x)) = \max(0, 1 - y \cdot f(x))$$

Nếu dự đoán đúng và tự tin (cách xa biên), mất mát hinge bằng không. Nó chỉ phạt những dự đoán sai hoặc nằm quá gần

ranh giới. Đó là lý do SVM tạo ra các bộ phân loại biên tối đa: mất mát hinge chủ động thưởng cho biên lớn.

Vì sao hàm mất mát lại quan trọng

Hàm mất mát định hình cách mô hình hoạt động:

- **Mất mát bình phương:** Nhạy cảm với ngoại lệ; dự đoán là liên tục
- **Mất mát log:** Đẩy dự đoán về 0 hoặc 1; đầu ra là xác suất
- **Mất mát hinge:** Bỏ qua các ví dụ "dễ"; tập trung vào ranh giới quyết định

Chọn sai hàm mất mát cho nhiệm vụ của bạn có thể tạo ra một mô hình đi tối ưu nhầm thứ.

4.3.2 Giả thuyết và hàm chi phí

Hồi quy tuyến tính là thuật toán đơn giản nhất và nền tảng nhất trong học có giám sát. Ý tưởng cốt lõi: nếu đầu vào và đầu ra có mối quan hệ đường thẳng, thì hãy tìm đường thẳng đó.

Để dự đoán giá nhà từ diện tích, ta mô hình hóa mối quan hệ như sau:

$$h_{\theta}(x) = \theta_0 + \theta_1 x \quad (4.1)$$

Đây đơn thuần là một đường thẳng: θ_0 là giao điểm với trục y, θ_1 là độ dốc. Những giá trị tham số khác nhau cho những đường thẳng khác nhau; thuật toán tìm các giá trị khớp dữ liệu huấn luyện tốt nhất.

Để đo một đường thẳng khớp tốt đến đâu, ta dùng **Sai số bình phương trung bình (MSE)**:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 \quad (4.2)$$

Hàm chi phí này đo mức dự đoán lệch khỏi giá trị thực tế nhiều đến đâu. Mục tiêu của ta là cực tiểu hóa nó.

4.3.3 Giảm gradient

Giảm gradient tìm điểm cực tiểu bằng cách liên tục bước theo hướng làm giảm chi phí:¹

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (4.3)$$

¹See Appendix D for partial derivatives and the chain rule.

Tốc độ học α điều khiển độ dài bước. Quá nhỏ: hội tụ chậm. Quá lớn: bước vượt qua cực tiểu hoặc phân kỳ.

Một tính chất then chốt: hàm chi phí của hồi quy tuyến tính là hàm **lồi**, có hình cái bát với đúng một điểm cực tiểu. Giảm gradient chắc chắn sẽ tìm được nó.

4.3.4 Nhiều đặc trưng

Đến đây, ta mới chỉ xét việc dự đoán giá nhà từ riêng diện tích. Nhưng thực tế có nhiều đặc trưng ảnh hưởng tới giá: số phòng ngủ, tuổi của ngôi nhà, chất lượng khu vực, và vân vân. Hồi quy tuyến tính dễ dàng mở rộng sang nhiều đặc trưng:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n \quad (4.4)$$

Ở đây, $x_1, x_2, \dots, x_n$ là các đặc trưng n khác nhau, và mỗi cái có tham số riêng θ_j quyết định mức đóng góp của đặc trưng đó vào dự đoán. Tham số θ_0 vẫn là hệ số chặn.

Dùng ký hiệu vector, ta có thể viết gọn hơn là $h_{\theta}(x) = \theta^T x$, trong đó θ và x đều là vector. Ký hiệu này trở nên thiết yếu khi làm việc với nhiều đặc trưng.

4.3.5 Hồi quy đa thức

Còn nếu mối quan hệ giữa đầu vào và đầu ra không phải đường thẳng thì sao? Có lẽ giá nhà tăng nhanh hơn với những ngôi nhà lớn hơn, hàm ý một mối quan hệ cong. Ta có thể nắm bắt điều này bằng cách thêm các đặc trưng đa thức:

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \cdots \quad (4.5)$$

Điều thú vị là đây vẫn gọi là hồi quy “tuyến tính” vì giả thuyết tuyến tính theo các tham số θ . Ta chỉ đơn giản tạo ra các đặc trưng mới (x^2, x^3 , v.v.) từ đặc trưng gốc x . Cùng thuật toán giảm gradient vẫn dùng để tìm các tham số tối ưu.

Sức mạnh của hồi quy đa thức đi kèm một lời cảnh báo: đa thức bậc cao có thể khớp rất sát dữ liệu huấn luyện nhưng lại có thể hoạt động tệ trên dữ liệu mới. Vấn đề này, gọi là quá khớp, sẽ được bàn tới sau trong chương này.

4.4 Hồi quy logistic

Dù cái tên có chữ “regression”, hồi quy logistic thực ra là một thuật toán **phân loại**. Nó được dùng khi đầu ra là phân loại chứ không phải

liên tục, phổ biến nhất là phân loại nhị phân khi ta muốn dự đoán một trong hai lớp.

4.4.1 Vì sao hồi quy tuyến tính thất bại khi phân loại

Bạn có thể tự hỏi: sao không dùng luôn hồi quy tuyến tính để phân loại? Giả sử ta gán email spam là 1 và email hợp lệ là 0. Ta có thể huấn luyện một mô hình hồi quy tuyến tính rồi xếp email vào spam nếu dự đoán lớn hơn 0.5.

Vấn đề là hồi quy tuyến tính có thể cho ra dự đoán nhỏ hơn 0 hoặc lớn hơn 1. Nó có thể dự đoán -0.3 cho một email và 1.7 cho email khác, những giá trị chẳng có nghĩa gì khi hiểu là xác suất. Chưa kể, một ngoại lệ duy nhất cũng có thể đẩy đường hồi quy lệch mạnh, làm thay đổi phân loại của nhiều ví dụ khác.

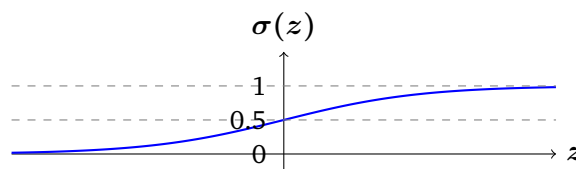
Điều ta cần là một hàm luôn cho giá trị nằm trong khoảng từ 0 đến 1, mà ta có thể diễn giải như xác suất. Đó chính xác là điều hồi quy logistic mang lại.

4.4.2 Hàm sigmoid

Thành phần then chốt của hồi quy logistic là **hàm sigmoid** (còn gọi là hàm logistic):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (4.6)$$

Hàm này có một đường cong chữ S rất đẹp. Dù bạn đưa vào giá trị z nào, đầu ra luôn nằm giữa 0 và 1. Khi z là số dương lớn, $\sigma(z)$ xấp xỉ 1. Khi z là số âm lớn, $\sigma(z)$ xấp xỉ 0. Khi $z = 0$, $\sigma(z) = 0.5$ đúng bằng 0.5.



Hình 4.2: Hàm sigmoid ánh xạ trơn tru mọi số thực về một giá trị trong khoảng 0 đến 1, khiến nó hoàn hảo để biểu diễn xác suất.

4.4.3 Giả thuyết của hồi quy logistic

Trong hồi quy logistic, ta lấy tổ hợp tuyến tính của các đặc trưng (giống hệt trong hồi quy tuyến tính) rồi cho đi qua hàm sigmoid:

$$h_{\theta}(\mathbf{x}) = \sigma(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}} \quad (4.7)$$

Đầu ra $h_{\theta}(\mathbf{x})$ giờ luôn nằm giữa 0 và 1, và ta diễn giải nó là xác suất mà đầu vào thuộc lớp 1. Cụ thể, $h_{\theta}(\mathbf{x}) = P(y = 1 | \mathbf{x}; \theta)$, là xác suất mà y bằng 1, với các đặc trưng $\mathbf{x}$ và tham số θ .

Để ra quyết định phân loại cuối cùng, ta dùng một ngưỡng, thường là 0.5:

$$\text{Predict } y = \begin{cases} 1 & \text{if } h_{\theta}(\mathbf{x}) \geq 0.5 \\ 0 & \text{if } h_{\theta}(\mathbf{x}) < 0.5 \end{cases} \quad (4.8)$$

Điều này có nghĩa ta dự đoán lớp 1 khi mô hình tin rằng xác suất thuộc lớp 1 ít nhất là 50%. Ngưỡng có thể điều chỉnh tùy ứng dụng, chẳng hạn trong chẩn đoán y tế ta có thể dùng ngưỡng thấp hơn để tránh bỏ sót bệnh.

4.4.4 Ranh giới quyết định

Ranh giới quyết định là mặt trong không gian đặc trưng nơi dự đoán đổi từ lớp này sang lớp khác. Với hồi quy logistic có hai đặc trưng, ranh giới này là nơi $h_{\theta}(\mathbf{x}) = 0.5$, điều xảy ra đúng khi $\theta^T \mathbf{x} = 0$.

Với hai đặc trưng x_1 và x_2 , phương trình $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$ xác định một đường thẳng. Các điểm ở một phía được xếp vào lớp 0; các điểm ở phía kia được xếp vào lớp 1. Việc của thuật toán là tìm đường thẳng (do $\theta_0, \theta_1, \theta_2$ quyết định) tách hai lớp tốt nhất.

4.4.5 Hàm chi phí của hồi quy logistic

Ta không thể dùng sai số bình phương trung bình cho hồi quy logistic vì hàm chi phí thu được sẽ không lồi, với nhiều cực tiểu cục bộ nơi giảm gradient có thể mắc kẹt.

Thay vào đó, ta dùng **mất mát chéo entropy nhị phân**, vốn có một trực giác rất đẹp. Nếu nhãn thật là $y = 1$, ta muốn $h_{\theta}(\mathbf{x})$ xấp xỉ 1. Số hạng $-\log(h_{\theta}(\mathbf{x}))$ nhỏ khi $h_{\theta}(\mathbf{x})$ gần 1, nhưng lại lớn dần khi $h_{\theta}(\mathbf{x})$ tiến về 0. Tương tự, nếu $y = 0$, số hạng $-\log(1 - h_{\theta}(\mathbf{x}))$ phạt những dự đoán gần 1.

Gộp các trường hợp này lại:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] \quad (4.9)$$

Hàm chi phí này lồi, nên giảm gradient tìm được cực tiểu toàn cục. Các quy tắc cập nhật trông gần giống hồi quy tuyến tính, và nhiều thư viện phần mềm giúp việc huấn luyện hồi quy logistic trở nên thật dễ.

4.5 Hiệu chỉnh: Ngăn ngừa quá khớp

Một trong những khái niệm quan trọng nhất của học máy là sự phân biệt giữa việc khớp dữ liệu huấn luyện và việc khái quát sang dữ liệu mới. Một mô hình làm rất tốt trên dữ liệu huấn luyện nhưng lại tệ trên dữ liệu mới đã bị **quá khớp**, nó chỉ ghi nhớ các ví dụ huấn luyện thay vì học khuôn mẫu ẩn phía sau.

4.5.1 Hiểu về quá khớp

Hãy tưởng tượng bạn khớp một đa thức vào một tập dữ liệu nhỏ. Một đường thẳng (bậc 1) có thể không nắm bắt được khuôn mẫu, đó là **thiếu khớp**. Một đa thức bậc 2 có thể khớp ổn thỏa. Nhưng đa thức bậc 20 lại có thể đi qua đúng từng điểm huấn luyện, đạt sai số huấn luyện bằng không. Nghe thì tuyệt, nhưng đa thức sẽ ngoằn ngoèo dữ dội giữa các điểm, cho những dự đoán tồi tệ trên dữ liệu mới.

Đa thức bậc 20 có quá nhiều tham số so với lượng dữ liệu huấn luyện. Nó đủ linh hoạt để khớp không chỉ khuôn mẫu thật mà cả nhiễu ngẫu nhiên trong dữ liệu. Đó chính là quá khớp.

4.5.2 Hiệu chỉnh: Thêm hình phạt cho độ phức tạp

Hiệu chỉnh ngăn quá khớp bằng cách thêm một số hạng phạt vào hàm chi phí, làm nản những giá trị tham số lớn. Trực giác là các giá trị tham số cực đoan thường cho thấy mô hình đang cố quá sức để khớp dữ liệu huấn luyện.

Hiệu chỉnh L2 (còn gọi là hồi quy Ridge) cộng vào tổng bình phương các tham số:

$$J_{L2}(\theta) = J(\theta) + \lambda \sum_{j=1}^n \theta_j^2 \quad (4.10)$$

Siêu tham số λ điều khiển độ mạnh của hiệu chỉnh. Giá trị λ lớn hơn áp dụng hình phạt mạnh hơn, đẩy các tham số về những giá trị nhỏ hơn. Khi

$\lambda = 0$, không có hiệu chỉnh nào được áp dụng. Lưu ý rằng ta thường không hiệu chỉnh θ_0 (hệ số chặn).

Hiệu chỉnh L2 thu nhỏ mọi tham số về phía không, nhưng chúng hiếm khi trở thành không hoàn toàn. Mô hình giữ lại toàn bộ đặc trưng nhưng giảm ảnh hưởng của chúng.

Hiệu chỉnh L1 (còn gọi là Lasso) cộng vào tổng các giá trị tuyệt đối:

$$J_{L1}(\theta) = J(\theta) + \lambda \sum_{j=1}^n |\theta_j| \quad (4.11)$$

Hiệu chỉnh L1 có một tính chất đáng chú ý: nó có xu hướng làm một số tham số thành không hoàn toàn. Nhờ đó mà việc chọn đặc trưng tự động được thực hiện, những đặc trưng không quan trọng bị loại bỏ hẳn. Điều này rất có giá trị khi bạn có nhiều đặc trưng và muốn xác định cái nào thực sự quan trọng.

Việc chọn giá trị λ đúng là rất quan trọng. Quá nhỏ thì hiệu chỉnh hầu như không có tác dụng; quá lớn thì mô hình trở nên quá đơn giản, thiếu khớp dữ liệu. Kiểm chéo (sẽ bàn sau) thường được dùng để chọn λ .

Tạm dừng khái niệm: So sánh các mô hình tuyến tính

Trước khi sang những thuật toán phức tạp hơn, hãy cùng ôn lại những gì ta đã học về mô hình tuyến tính.

Method	Use When	Strengths	Weaknesses
Linear Regression	Predicting continuous values; relationship is approximately linear	Simple, interpretable, fast; closed-form solution	Cannot capture non-linear patterns
Logistic Regression	Binary/multi-class classification; want probabilities	Outputs calibrated probabilities; works with many features	Assumes linear decision boundary
Ridge (L2)	Many features; want to prevent overfitting	Shrinks all coefficients; stable	No feature selection
Lasso (L1)	Many features; want sparsity	Automatic feature selection	May select arbitrarily among correlated features

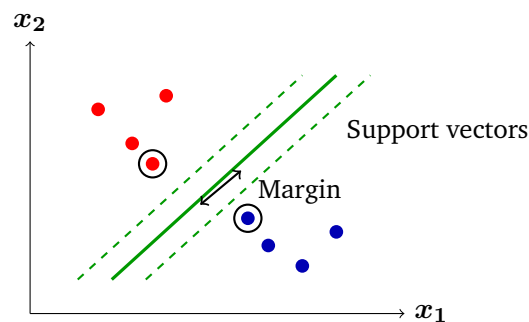
4.6 Máy vectorsupport

Máy vectorsupport (SVM) tiếp cận bài toán phân loại từ một góc khác: thay vì mô hình hóa xác suất như hồi quy logistic, chúng tập trung tìm ranh giới tách các lớp với **biên rộng nhất**.

4.6.1 Ý tưởng biên rộng nhất

Xét một bài toán phân loại nhị phân mà hai lớp phân tách tuyến tính được. Nghĩa là ta có thể vẽ một đường thẳng (hoặc siêu mặt phẳng ở số chiều cao hơn) tách hoàn hảo chúng. Có vô số đường như vậy. Nên chọn đường nào?

SVM chọn đường thẳng nằm xa nhất so với các điểm huấn luyện gần nhất của mỗi lớp. Khoảng cách này gọi là **biên**. Trực giác là một đường có biên lớn sẽ vững hơn. Những nhiễu động nhỏ hay nhiễu trong dữ liệu ít gây ra phân loại nhầm hơn.



Hình 4.3: SVM tìm ranh giới quyết định (đường liền) cực đại hóa biên (khoảng cách tới các đường nét đứt). Các điểm khoanh tròn là support vector. Chúng là những ví dụ huấn luyện gần ranh giới nhất.

4.6.2 Support vector

Những điểm nằm ngay trên các đường biên được gọi là **support vector**. Đây là những ví dụ huấn luyện “chống đỡ” cho ranh giới quyết định. Nếu bạn bỏ đi bất kỳ điểm huấn luyện nào khác, ranh giới tối ưu cũng không đổi. Chỉ các support vector mới quyết định bộ phân loại.

Đây là một tính chất đẹp: lời giải của SVM chỉ phụ thuộc vào một tập con của dữ liệu huấn luyện. Ngược lại, các phương pháp như hồi quy logistic lại dùng mọi điểm huấn luyện để xác định tham số.

4.6.3 Biên mềm và tham số C

Trong thực tế, dữ liệu hiếm khi tách được hoàn hảo. Có thể có vài điểm của lớp này xen vào lớp kia, hoặc hai lớp chồng lên nhau khá nhiều. Một SVM “biên cứng” đòi hỏi tách hoàn hảo sẽ thất bại trên dữ liệu như vậy.

SVM **biên mềm** cho phép một số điểm vi phạm biên hoặc thậm chí nằm sai phía ranh giới. Bài toán tối ưu trở thành một sự đánh đổi: ta

muốn biên lớn, nhưng đồng thời cũng muốn giảm thiểu phân loại nhầm. Tham số C điều khiển sự đánh đổi này:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \quad (4.12)$$

Ở đây, ξ_i cho biết điểm i vi phạm biên nhiều đến đâu (bằng không nếu không vi phạm). Giá trị C lớn phạt nặng những vi phạm, dẫn tới biên nhỏ hơn nhưng ít phân loại nhầm hơn. Điều này có thể gây quá khớp dữ liệu nhiều. Giá trị C nhỏ chịu được nhiều vi phạm hơn để đổi lấy biên lớn hơn. Điều này có thể gây thiếu khớp nếu dữ liệu tách được sạch.

4.6.4 Phương pháp kernel: Bước ra không tuyến tính

Còn nếu không có đường thẳng nào tách được các lớp thì sao? SVM xử lý điều này qua **thủ thuật kernel**, một trong những ý tưởng tinh tế nhất của học máy.

Thủ thuật kernel ngầm ánh xạ dữ liệu sang một không gian số chiều cao hơn, nơi có thể tách tuyến tính được. Ví dụ, dữ liệu tạo thành hai đường tròn đồng tâm trong 2D không thể tách bằng một đường thẳng. Nhưng nếu ta thêm chiều thứ ba $z = x_1^2 + x_2^2$ (bình phương khoảng cách tới gốc tọa độ), hai đường tròn trở thành hai mặt phẳng song song *có thể* tách bằng một siêu mặt phẳng.

Điều đáng kinh ngạc là ta chưa bao giờ thực sự tính ánh xạ cao chiều này. Thay vào đó, ta dùng một **hàm kernel** $K(\mathbf{x}, \mathbf{z})$ tính trực tiếp tích vô hướng trong không gian cao chiều ngay từ các đặc trưng gốc. Những kernel phổ biến gồm:

Kernel tuyến tính $K(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$ tương ứng với không có biến đổi nào, nó cho một SVM tuyến tính tiêu chuẩn.

Kernel đa thức $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T \mathbf{z} + c)^d$ tương ứng với các đặc trưng đa thức bậc d .

Kernel RBF (Gaussian) $K(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$ là loại phổ biến nhất, tương ứng với một không gian vô hạn chiều. Nó có thể tạo ra những ranh giới quyết định phức tạp tùy ý.

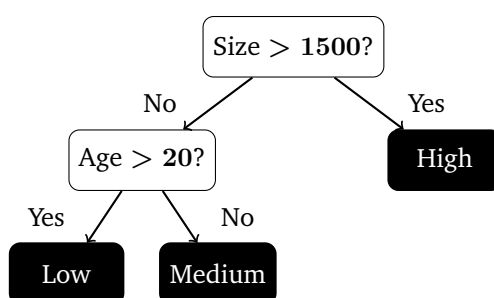
4.7 Cây quyết định

Cây quyết định đi một hướng hoàn toàn khác để phân loại: chúng học một loạt quy tắc if-then chia không gian đặc trưng thành các vùng, mỗi vùng gán cho một lớp. Kết quả rất dễ diễn giải, bạn lần được đúng lý do từng dự đoán được đưa ra.

4.7.1 Cấu trúc của cây

Một cây quyết định được dựng từ ba loại nút. **Nút gốc** nằm ở trên cùng và chứa quyết định đầu tiên. **Các nút bên trong** đại diện cho những quyết định tiếp theo khi bạn đi xuống cây. **Các nút lá** nằm ở dưới cùng và chứa những dự đoán lớp cuối cùng.

Tại mỗi nút bên trong, cây đặt một câu hỏi về một đặc trưng, thường dạng “Đặc trưng x_j có lớn hơn một ngưỡng nào đó không?” Dựa vào câu trả lời, ta đi theo nhánh này hoặc nhánh kia. Cuối cùng ta tới một nút lá cho biết lớp được dự đoán.



Hình 4.4: Một cây quyết định đơn giản để dự đoán nhóm giá nhà. Bắt đầu từ nút gốc, ta đi theo các nhánh dựa trên câu trả lời cho tới khi tới một nút lá chứa dự đoán.

4.7.2 Chọn các cách chia tốt nhất

Câu hỏi then chốt khi dựng một cây quyết định là: tại mỗi nút, nên chia theo đặc trưng nào, và ở ngưỡng nào? Ta muốn những cách chia tách các lớp tốt nhất.

Đo mức một tập ví dụ “hỗn tạp” hay “bất thuần” đến đâu, ta dùng **entropy**:

$$H(S) = - \sum_c p_c \log_2(p_c) \quad (4.13)$$

Ở đây, p_c là tỷ lệ các ví dụ thuộc lớp c . Nếu mọi ví dụ đều thuộc một lớp ($p_c = 1$ cho lớp đó), entropy bằng 0. Không có sự không chắc chắn nào về lớp. Nếu các ví dụ chia đều cho hai lớp ($p_c = 0.5$ cho mỗi lớp), entropy bằng 1. Không chắc chắn tối đa.

Một cách chia tốt sẽ giảm entropy, sau khi chia, các tập con thu được nên thuần khiết hơn tập ban đầu. **Lợi thông tin** đo mức giảm này:

$$\text{Gain}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v) \quad (4.14)$$

Điều này tính entropy trước khi chia trừ trung bình có trọng số của entropy sau khi chia. Thuộc tính có lợi thông tin cao nhất sẽ được chọn để chia.

Example: Computing Information Gain

Consider a dataset with 10 examples: 5 positive and 5 negative. The entropy is $H(S) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 1.0$ (maximum uncertainty).

Suppose splitting on attribute A produces two branches: one with 4 positive and 1 negative, another with 1 positive and 4 negative.

For the first branch: $H = -\frac{4}{5} \log_2 \frac{4}{5} - \frac{1}{5} \log_2 \frac{1}{5} \approx 0.72$

For the second branch: same calculation gives $H \approx 0.72$

Weighted average entropy after split: $\frac{5}{10}(0.72) + \frac{5}{10}(0.72) = 0.72$

Information Gain = $1.0 - 0.72 = 0.28$

This split reduces uncertainty by 0.28 bits.

4.7.3 Dựng cây: Thuật toán ID3

Thuật toán kinh điển để dựng cây quyết định, gọi là ID3, chạy đệ quy:

Thứ nhất, nếu mọi ví dụ trong tập hiện tại cùng một lớp, tạo một nút lá mang nhãn lớp đó. Thứ hai, nếu không còn thuộc tính nào để chia, tạo một nút lá mang lớp đa số. Nếu không, chọn thuộc tính có lợi thông tin cao nhất, tạo nút cho thuộc tính đó, và với mỗi giá trị khả dĩ của thuộc tính, dựng đệ quy một cây con cho các ví dụ mang giá trị ấy.

Cách tham lam này dựng cây theo chiều từ trên xuống, luôn chọn phương án tối ưu cục bộ ở mỗi bước. Dù nó không bảo đảm cây tối ưu toàn cục, nó hoạt động tốt trong thực tế.

4.7.4 Cắt tỉa: Tránh quá khớp

Cây quyết định có xu hướng quá khớp, nhất là khi mọc sâu. Một cây có nhiều tầng có thể tạo ra những quy tắc quá cụ thể, khớp hoàn hảo dữ liệu huấn luyện nhưng không khái quát được.

Cắt tỉa xử lý điều này bằng cách loại bỏ những nhánh không cải thiện hiệu năng trên dữ liệu kiểm định. Cắt tỉa trước dừng việc mọc cây sớm, còn cắt tỉa sau mọc cây đầy đủ rồi gỡ bỏ những nhánh vô ích. Các tham số như độ sâu tối đa, số mẫu tối thiểu mỗi lá, và lợi thông tin tối thiểu để chia giúp kiểm soát độ phức tạp của cây.

4.7.5 Phương pháp tập hợp: Kết hợp nhiều cây

Một cây quyết định đơn lẻ dễ quá khớp và có phương sai cao: thay đổi nhỏ trong dữ liệu huấn luyện có thể cho ra một cây rất khác. **Các phương pháp tập hợp** khắc phục điểm yếu này bằng cách kết hợp nhiều cây thành một bộ dự đoán vững hơn.

Bagging (Bootstrap Aggregating) huấn luyện nhiều cây trên các mẫu con ngẫu nhiên của dữ liệu huấn luyện (rút có hoàn lại) rồi lấy trung bình dự đoán của chúng. Vì mỗi cây thấy một tập dữ liệu hơi khác, chúng mắc những lỗi khác nhau, và việc lấy trung bình làm giảm phương sai.

Một **Rừng ngẫu nhiên** mở rộng bagging bằng cách còn ngẫu nhiên hóa cả những đặc trưng xét tới ở mỗi lần chia. Tại mỗi nút, chỉ một tập con đặc trưng ngẫu nhiên được đánh giá, buộc các cây phải khác nhau. Rừng ngẫu nhiên thuộc nhóm những bộ phân loại sẵn dùng đáng tin cậy nhất trong thực tế: chúng xử lý được mẫu không tuyến tính, cần rất ít tinh chỉnh, chống quá khớp, và làm tốt cả trên tập dữ liệu nhỏ lẫn lớn.

Boosting đi hướng khác: nó huấn luyện các cây tuần tự, trong đó cây mới tập trung vào những ví dụ mà các cây trước dự đoán sai. **AdaBoost** tăng trọng số của các ví dụ bị phân loại sai trước khi huấn luyện cây tiếp theo. **Gradient Boosting** khớp cây mới vào phần sai số còn lại của tập hợp cho tới thời điểm đó. Boosting thường cho độ chính xác cao hơn bagging nhưng nhạy hơn với dữ liệu nhiễu và cần tinh chỉnh cẩn thận hơn.

Ý chính: Vì sao tập hợp lại hiệu quả

Một cây đơn lẻ giống như hỏi một người chỉ đường: bạn có thể nhận câu trả lời tuyệt vời hoặc câu trả lời rất tệ. Một tập hợp giống như hỏi nhiều người rồi lấy phiếu. Những lỗi cá nhân triệt tiêu nhau, và đồng thuận thường đáng tin cậy hơn bất kỳ ý kiến riêng lẻ nào.

4.8 Phân loại Naïve Bayes

2

Naïve Bayes là bộ phân loại xác suất dựa trên định lý Bayes kèm một giả định đơn giản hóa làm phép tính khả thi. Bất chấp sự đơn giản của nó, nó thường hoạt động tốt đến bất ngờ, nhất là cho phân loại văn bản.

²See Appendix C for a review of Bayes' theorem and conditional probability.

4.8.1 Định lý Bayes

Định lý Bayes liên hệ xác suất của một giả thuyết khi có bằng chứng với xác suất của bằng chứng khi có giả thuyết:

$$P(y|x) = \frac{P(x|y) \cdot P(y)}{P(x)} \quad (4.15)$$

Ở khía cạnh phân loại: $P(y|x)$ là xác suất của lớp y khi biết các đặc trưng x (cái ta muốn). $P(x|y)$ là xác suất khi thấy các đặc trưng x với điều kiện lớp y (hàm khả năng). $P(y)$ là xác suất tiên nghiệm của lớp y trước khi thấy bất kỳ đặc trưng nào. $P(x)$ là xác suất của các đặc trưng, đóng vai trò hằng số chuẩn hóa.

Để phân loại, ta tính $P(y|x)$ cho mỗi lớp khả dĩ y và dự đoán lớp có xác suất cao nhất.

4.8.2 Giả định độc lập kiểu Naïve

Khó khăn nằm ở việc tính $P(x|y)$, xác suất của một tổ hợp đặc trưng cụ thể khi biết một lớp. Với nhiều đặc trưng, số tổ hợp khả dĩ là khổng lồ.

Giả định “naïve” giải quyết điều này bằng cách giả sử các đặc trưng độc lập có điều kiện với nhau khi đã biết lớp. Nói cách khác, một khi đã biết lớp, biết giá trị của một đặc trưng không cho ta biết gì về giá trị của các đặc trưng khác. Nhờ thế ta có thể phân tích xác suất:

$$P(x|y) = P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y) = \prod_{i=1}^n P(x_i|y) \quad (4.16)$$

Mỗi $P(x_i|y)$ có thể ước lượng trực tiếp từ dữ liệu huấn luyện bằng cách đếm tần suất xuất hiện của mỗi giá trị đặc trưng trong từng lớp.

4.8.3 Quy tắc phân loại

Bước phân loại cuối cùng chọn lớp cực đại hóa xác suất hậu nghiệm:

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i|y) \quad (4.17)$$

Vì $P(x)$ giống nhau với mọi lớp, ta có thể bỏ qua nó và chỉ so sánh tử số.

Giả định độc lập gần như luôn sai, các đặc trưng thường tương quan với nhau. Vậy mà Naïve Bayes vẫn thường hoạt động tốt đáng kinh ngạc. Một lý do là

việc phân loại chỉ cần xếp hạng đúng các lớp, chứ không cần tính xác suất chính xác. Ngay cả khi xác suất sai, thứ hạng vẫn có thể đúng.

Naïve Bayes đặc biệt phổ biến cho phân loại văn bản (lọc spam, phân tích cảm xúc) nơi các đặc trưng là số lần xuất hiện của từ. Nó huấn luyện nhanh, xử lý tốt dữ liệu số chiều cao, và cho một đường cơ sở mạnh.

4.9 So sánh các thuật toán

Chọn đúng thuật toán đòi hỏi phải hiểu giả định, thế mạnh và kiểu thất bại của chúng.

Algorithm	Assumptions	Failure Modes	Strengths	Interpretable?
Linear Regression	Linear relationship; homoscedastic errors	Non-linear patterns; outliers distort fit	Fast; closed-form solution; coefficients meaningful	Yes
Logistic Regression	Linear decision boundary; features independent-ish	Non-linear boundaries; multicollinearity	Probabilistic output; regularization easy	Yes
Decision Tree	None (non-parametric)	Overfits with depth; axis-aligned only	Handles mixed types; no scaling needed	Yes
SVM (linear)	Linearly separable (soft margin helps)	Very non-linear data; large datasets slow	Max-margin = good generalization	Somewhat
SVM (RBF)	Data has local structure	Too many features; sensitive to γ, C	Captures complex boundaries	No
Naïve Bayes	Features conditionally independent	Correlated features; continuous features tricky	Fast; works with little data; handles many features	Yes

Bảng 4.1: Các thuật toán học có giám sát: giả định, kiểu thất bại và khả năng diễn giải.

Khi nào dùng gì

Bắt đầu đơn giản: Hồi quy tuyến tính/logistic làm đường cơ sở.

Cần khả năng diễn giải: Cây quyết định hoặc mô hình tuyến tính có hiệu chỉnh.

Mẫu không tuyến tính: SVM với kernel RBF, hoặc tập hợp cây như Rừng ngẫu nhiên.

Dữ liệu thưa, số chiều cao (ví dụ: văn bản): Naïve Bayes hoặc SVM tuyến tính.

Loại đặc trưng pha trộn: Cây quyết định xử lý đặc trưng phân loại một cách tự nhiên.

Trạm kiểm tra đạo đức: Tính công bằng của bộ phân loại

Bạn đã huấn luyện một cây quyết định để dự báo vỡ nợ. Có đạt độ chính xác 92% tổng thể.

Câu hỏi kỹ thuật – Đạo đức:

1. Bạn phát hiện cây dùng “zip code” làm một cách chia hàng đầu. Vì sao điều này có thể thành vấn đề dù zip code có tính dự đoán? 2. Có độ nhạy (recall) 85% với Nhóm A nhưng chỉ 60% với Nhóm B. Cả hai nhóm có cùng tỷ lệ nền về vỡ nợ. Điều này có công bằng không? Định nghĩa nào về công bằng bị vi phạm?

3. Bạn gỡ zip code khỏi danh sách đặc trưng. Độ chính xác giảm còn 89%. Một quản lý nói “chúng ta đang hy sinh hiệu năng vì chính trị đúng đắn.” Bạn đáp lại thế nào?

Đây chính là dạng bài lai kỹ thuật – đạo đức xuất hiện ở IAIO. Xem Chương 2 để biết các định nghĩa về công bằng.

Tạm dừng khái niệm: So sánh các bộ phân loại

Đến đây chúng ta đã thấy vài thuật toán phân loại. Sau đây là cách so sánh chúng:

Method	Decision Boundary	Best For	Many Features?	Interpretable?
Logistic Regression	Linear (hyper-plane)	Linearly separable data; probability outputs	Yes, with regularization	Yes (coefficients)
SVM (linear)	Linear (max margin)	High-dimensional data; clear margin	Yes	Somewhat
SVM (kernel)	Non-linear	Complex boundaries; moderate data	Depends	No
Decision Tree	Axis-aligned rectangles	Mixed feature types; interpretability	Can struggle	Yes (rules)
Naïve Bayes	Quadratic	Text classification; independent features	Yes	Yes (probabilities)

Những lỗi thường gặp

- **Dùng SVM mà không chuẩn hóa:** SVM nhạy với thang của đặc trưng. Luôn chuẩn hóa trước.
- **Cây quyết định sâu mà không cắt tỉa:** Chúng sẽ ghi nhớ dữ liệu huấn luyện.
- **Naïve Bayes với đặc trưng tương quan:** Giả định độc lập bị vi phạm; xác suất sẽ sai (nhưng việc phân loại vẫn có thể chạy được).
- **Hồi quy logistic cho ranh giới không tuyến tính:** Nó sẽ thiếu khớp. Hãy cân nhắc SVM kernel hoặc cây.

Đánh giá mô hình

Huấn luyện mô hình mới chỉ là một nửa vấn đề. Việc đánh giá nghiêm ngặt trên dữ liệu giữ lại là điều bắt buộc.

Nguyên tắc then chốt: Không bao giờ đánh giá trên dữ liệu huấn luyện, nó cho ra kết quả lạc quan một cách gây hiểu lầm.

Để biết đầy đủ về các phương pháp đánh giá, các số đo và kiểm chéo, hãy xem **Chương 5**.

4.10 Tóm tắt

Hồi quy tuyến tính mô hình hóa biến mục tiêu liên tục dưới dạng tổng có trọng số của các đặc trưng. Giảm gradient tìm trọng số tối ưu bằng cách liên tục giảm sai số bình phương.³

Hồi quy logistic xử lý phân loại bằng cách áp hàm sigmoid lên một tổ hợp tuyến tính, tạo ra xác suất. Mật mát chéo entropy cho phép học dựa trên gradient.

Hiệu chỉnh (L1/L2) ngăn quá khớp bằng cách phạt các trọng số lớn. L1 tạo mô hình thưa; L2 thu nhỏ mọi trọng số.

SVM tìm các ranh giới quyết định biên rộng nhất. Kernel cho phép phân loại không tuyến tính mà không cần tính toán đặc trưng tường minh.

Cây quyết định học các cách chia thẳng hàng trục, phân vùng đệ quy không gian đặc trưng. Entropy/lợi thông tin dẫn hướng việc chọn cách chia.

Naïve Bayes áp dụng định lý Bayes⁴ kèm giả định độc lập có điều kiện. Dù “sai”, nó thường hoạt động tốt.

Lựa chọn thuật toán phụ thuộc vào đặc điểm dữ liệu, nhu cầu diễn giải và ràng buộc tính toán. Không thuật toán nào là tốt nhất cho mọi bài toán.

4.11 Bài tập luyện tập

1. Một mô hình hồi quy tuyến tính dự đoán giá nhà: $h(x) = 50,000 + 100x$, trong đó x là diện tích theo feet vuông. Mô hình dự báo giá bao nhiêu cho một nhà 2000 feet vuông? Các hệ số được hiểu thế nào?
2. Một mô hình hồi quy logistic cho ra $h_{\theta}(x) = 0.73$. Với ngưỡng 0.5, lớp nào được dự đoán? Con số 0.73 nghĩa là gì?
3. Khi dựng một cây quyết định, việc chia theo thuộc tính A tạo ra các nhánh có $(4+, 0-)$ và $(1+, 5-)$. Nếu entropy ban đầu là 1.0, hãy tính lợi thông tin.
4. Giải thích vì sao SVM được gọi là “bộ phân loại biên rộng.” C ảnh hưởng tới biên như thế nào?
5. So sánh hiệu chỉnh L1 và L2. Khi nào bạn chọn cái nào?
6. (Dạng IAIO: Một thuật toán bạn chưa từng thấy)

Một đồng nghiệp đề xuất “Bộ phân loại hàng xóm có trọng số” (WNC):

³See Appendix D for calculus of gradients.

⁴See Appendix C for probability review.

- (a) Với một điểm kiểm thử $\mathbf{x}$, tính khoảng cách d_i tới mỗi điểm huấn luyện $\mathbf{x}_i$ (b) Gán trọng số $w_i = \exp(-d_i^2/\sigma^2)$ cho mỗi điểm huấn luyện (c) Dự đoán: $\hat{y} = \text{sign}(\sum_i w_i \cdot y_i)$ trong đó $y_i \in \{-1, +1\}$

Hãy trả lời các câu sau:

- (a) Điều gì xảy ra khi $\sigma \rightarrow 0$? Khi $\sigma \rightarrow \infty$? (b) Điều này liên quan thế nào tới k -NN? Tới các phương pháp kernel? (c) Ranh giới quyết định có hình dạng gì? Nó có tuyến tính không? (d) Những kiểu thất bại của thuật toán này là gì? (e) Đề xuất một cách sửa để xử lý lệch lớp.

7. **(Nhiều phần)** Xét bài toán XOR với các điểm $(0, 0)$, $(0, 1)$, $(1, 0)$, $(1, 1)$ và các nhãn 0, 1, 1, 0.

- (a) Chứng minh không bộ phân loại tuyến tính nào giải được bài này.
 (b) Cho thấy việc thêm đặc trưng $x_3 = x_1 \cdot x_2$ làm nó phân tách tuyến tính được.
 (c) Vẽ một cây quyết định giải XOR.
 (d) Điều này liên quan thế nào tới thủ thuật kernel?

8. **(Kỹ thuật – Đạo đức)** Hồi quy logistic của một ngân hàng dùng “số năm ở địa chỉ hiện tại” (hệ số = âm thì rủi ro vỡ nợ thấp hơn). Biến này tương quan với tuổi.

- (a) Vì sao điều này có thể cấu thành phân biệt đối xử theo tuổi dù tuổi không phải là một đặc trưng?
 (b) Việc bỏ biến này làm AUC giảm từ 0.82 xuống 0.79. Hãy định lượng sự đánh đổi này.
 (c) Bạn sẽ tính những số đo công bằng nào?

9. **(Dạng IAIO: Một thuật toán khác bạn chưa từng thấy)**

Xét “Bộ phân loại nguyên mẫu”:

- (a) Với mỗi lớp c , tính điểm centroid (giá trị trung bình) của tất cả điểm huấn luyện trong lớp đó: $\mu_c = \frac{1}{|C_c|} \sum_{\mathbf{x} \in C_c} \mathbf{x}$
 (b) Để phân loại một điểm mới $\mathbf{x}$, gán nó vào lớp có centroid gần nhất:
 $\hat{y} = \arg \min_c \|\mathbf{x} - \mu_c\|$

Hãy trả lời các câu sau:

- (a) Hình dạng của ranh giới quyết định giữa hai lớp là gì? (Gợi ý: $\|\mathbf{x} - \boldsymbol{\mu}_1\| = \|\mathbf{x} - \boldsymbol{\mu}_2\|$ nằm ở đâu?)
- (b) Điều này so với k-NN với $k =$ mọi điểm trong mỗi lớp thì thế nào?
- (c) Giả định nào về dữ liệu sẽ làm bộ phân loại này hoạt động tốt?
- (d) Hãy dựng một tập dữ liệu 2D đơn giản mà bộ phân loại này hỏng nặng. Tập dữ liệu của bạn có tính chất gì?
- (e) Bộ phân loại này lưu bao nhiêu tham số? So với k-NN và hồi quy logistic.
-

Chương 5

Đánh giá mô hình

The purpose of computing is
insight, not numbers.

Richard Hamming

5.1 Giới thiệu: Vấn đề khái quát hóa

Giả sử bạn đang chuẩn bị cho một học sinh kỳ thi quan trọng. Bạn đưa cho cậu 100 bài tập luyện, và cậu học rất chăm chỉ. Khi bạn kiểm tra cậu bằng chính 100 bài đó, cậu trả lời đúng từng câu một. Thật ấn tượng! Nhưng khi kỳ thi thật sự đến với những câu hỏi khác, cậu trượt chết. Chuyện gì đã xảy ra?

Học sinh chỉ ghi nhớ đáp án cụ thể của các bài tập luyện thay vì học các nguyên lý đằng sau đó. Cậu có thể tái hiện những đáp án đã từng thấy nhưng lại không thể áp dụng kiến thức của mình cho những tình huống mới. Đây là phiên bản trong giáo dục của một vấn đề cơ bản trong học máy: sự khác biệt giữa **ghi nhớ** và **khái quát hóa**.

Các mô hình học máy đối mặt với đúng thách thức này. Một mô hình hoạt động hoàn hảo trên dữ liệu huấn luyện nhưng thất bại trên dữ liệu mới chưa từng thấy thì chưa thực sự học được điều gì. Nó chỉ đơn thuần là ghi nhớ. Mô hình như vậy vô dụng trên thực tế, bởi toàn bộ ý nghĩa của học máy là đưa ra dự đoán về dữ liệu mà ta chưa từng thấy.

Thách thức cốt lõi

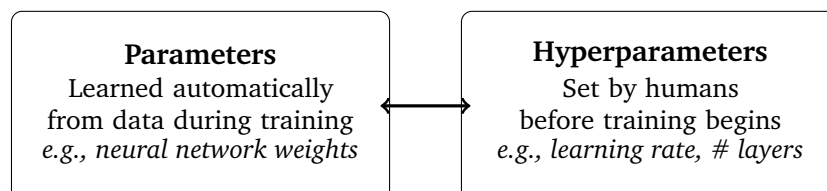
Một mô hình hoạt động hoàn hảo trên dữ liệu huấn luyện nhưng thất bại trên dữ liệu mới thì **vô dụng**. Toàn bộ giá trị của học máy nằm ở khái quát hóa. Ở khả năng áp dụng các khuôn mẫu đã học vào những tình huống mới. Các phương pháp đánh giá của chúng ta phải đo lường sự khái quát hóa, chứ không chỉ hiệu năng trên tập huấn luyện.

5.2 Tham số và siêu tham số

Trước khi đi sâu vào các phương pháp đánh giá, chúng ta cần làm rõ một sự phân biệt làm nhiều người mới bắt đầu bối rối.

Tham số là những giá trị mà mô hình tự động học được từ dữ liệu trong quá trình huấn luyện. Trong một mạng nơ-ron, đó là các trọng số nối giữa các nơ-ron. Thuật toán học điều chỉnh những trọng số này để khớp với dữ liệu huấn luyện.

Siêu tham số là những thiết lập do con người chọn trước khi bắt đầu huấn luyện. Chúng điều khiển cách thức học diễn ra nhưng không được học từ dữ liệu. Ví dụ gồm tốc độ học, số kỳ (epoch), kiến trúc mạng và cường độ chuẩn hóa (regularization).



Hình 5.1: Tham số được học; siêu tham số do con người chọn.

5.3 Tầm quan trọng của việc chia dữ liệu

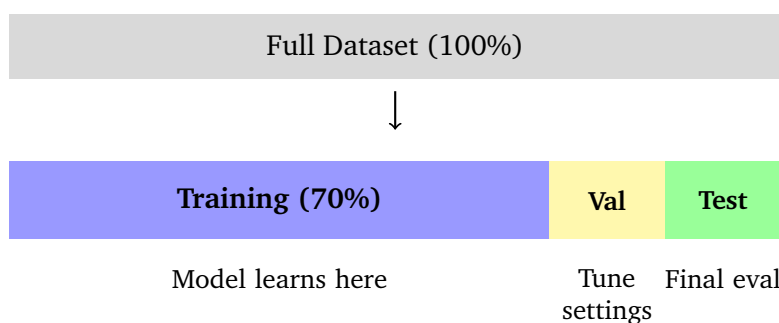
Nguyên tắc nền tảng nhất trong đánh giá mô hình là: **không bao giờ đánh giá một mô hình trên chính dữ liệu đã dùng để huấn luyện nó.**

Nếu ta huấn luyện trên một phần dữ liệu rồi kiểm thử trên chính phần dữ liệu đó, mô hình có thể đạt độ chính xác cao chỉ đơn thuần bằng cách ghi nhớ các ví dụ huấn luyện. Điều đó không cho chúng ta biết gì về khả năng khái quát hóa.

Quy tắc vàng**Không bao giờ kiểm thử trên dữ liệu huấn luyện!**

Đánh giá trên dữ liệu huấn luyện giống như đưa trước đề thi cho học sinh. Kết quả cao không nói lên được điều gì về mức độ hiểu bài thực sự.

Ta chia dữ liệu thành các tập riêng biệt với những mục đích khác nhau:



Hình 5.2: Cách chia chuẩn tập huấn luyện – kiểm định – kiểm tra.

Tập huấn luyện (60-80%) là nơi diễn ra việc học. **Tập kiểm định** (10-20%) dùng để kiểm tra tiến độ trong quá trình phát triển và tinh chỉnh siêu tham số. **Tập kiểm tra** (10-20%) được giữ hoàn toàn nguyên vẹn cho đến khi đánh giá cuối cùng, nó cung cấp ước lượng không thiên lệch về hiệu năng ngoài đời thực.

Ảnh dụ: Chuẩn bị cho kỳ thi

Tập huấn luyện = Giáo trình (học từ đó)

Tập kiểm định = Kỳ thi luyện tập (kiểm tra tiến độ, điều chỉnh chiến lược học)

Tập kiểm tra = Kỳ thi cuối cùng (đo lường thực sự việc học)

Khi chia dữ liệu, hãy chọn các ví dụ một cách **ngẫu nhiên**. Nếu dữ liệu được sắp xếp (theo ngày, nhóm, v.v.), việc lấy 70% đầu tiên sẽ tạo ra thành kiến (bias).

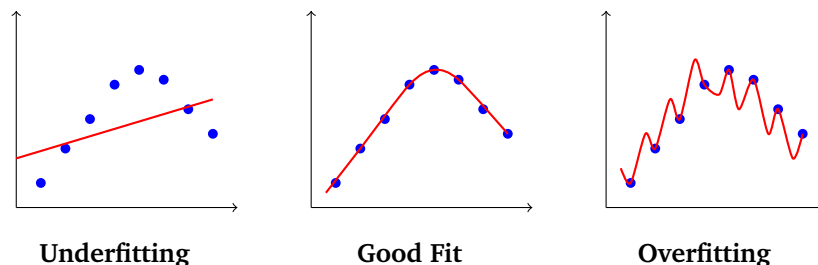
5.4 Thiên vị và Phương sai: Hiểu các lỗi

Mọi mô hình đều mắc lỗi. Hiểu được *tại sao* sẽ giúp chúng ta sửa chúng.

Thiếu khớp (thiên vị cao) xảy ra khi mô hình quá đơn giản. Nó không thể nắm bắt được các khuôn mẫu ngay cả trong dữ liệu huấn luyện. Cách khắc phục: tăng độ phức tạp.

Quá khớp (phương sai cao) xảy ra khi mô hình quá phức tạp. Nó ghi nhớ dữ liệu huấn luyện nhưng thất bại trên dữ liệu mới. Cách khắc phục: giảm độ phức tạp, thêm

chuẩn hóa (regularization), hoặc thu thập thêm dữ liệu.



Hình 5.3: Thiếu khớp (quá đơn giản), vừa khớp (vừa phải), và quá khớp (quá phức tạp).

Chẩn đoán: Nếu độ chính xác trên tập huấn luyện thấp, mô hình đang thiếu khớp. Nếu độ chính xác trên tập huấn luyện cao nhưng độ chính xác trên tập kiểm định thấp hơn nhiều, mô hình đang quá khớp.

5.5 Các số đo cho bài toán phân loại

Với các bài toán phân loại, chúng ta cần những số đo phù hợp. **Ma trận nhầm lẫn** thể hiện các dự đoán so với nhãn thực tế:

		Dự đoán	
		Dương tính	Âm tính
Thực tế	Dương tính	Dương tính thật (TP)	Âm tính giả (FN)
	Âm tính	Dương tính giả (FP)	Âm tính thật (TN)

Độ chính xác là tỉ lệ dự đoán đúng: $(TP + TN)/(TP + TN + FP + FN)$

Nhưng độ chính xác có thể gây hiểu lầm khi các lớp mất cân bằng! Một máy phát hiện vận chuyển trái phép luôn nói "không vận chuyển trái phép" sẽ đạt độ chính xác 99,5% nếu chỉ có 0,5% số container chứa hàng cấm. Vậy mà nó bắt được không một kẻ vận chuyển trái phép nào.

Độ chính xác (precision) trả lời câu hỏi "trong các dự đoán dương tính của tôi, có bao nhiêu là đúng?"

$$\text{Precision} = \frac{TP}{TP + FP}$$

Dùng khi dương tính giả tốn kém (ví dụ: bộ lọc thư rác – bạn không muốn bỏ sót email thật).

Độ nhạy (recall) trả lời câu hỏi "Trong tất cả các ca dương tính thực sự, tôi đã bắt được bao nhiêu?"

$$\text{Recall} = \frac{TP}{TP + FN}$$

Dùng khi âm tính giả tốn kém (ví dụ: sàng lọc ung thư, đừng bỏ sót ca bệnh).

Điểm F1 cân bằng giữa precision và recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Ví dụ minh họa

Một máy phát hiện bệnh: TP=80, FP=20, FN=10, TN=890 Độ chính xác = $(80 + 890) / (80 + 890 + 20 + 10) = 970 / 1000 = 97\%$
 Precision = $80 / (80 + 20) = 80 / 100 = 80\%$
 Recall = $80 / (80 + 10) = 80 / 90 \approx 88.9\%$
 $F1 = 2 \cdot (0.80 \times 0.889) / (0.80 + 0.889) \approx 84.2\%$

Thông thường có sự **đánh đổi** giữa precision và recall, được điều khiển bởi **ngưỡng quyết định**. Hầu hết các bộ phân loại đưa ra một xác suất hoặc điểm số; bạn chọn một ngưỡng mà trên đó dự đoán là dương tính.

Tăng ngưỡng (ví dụ, từ 0.5 lên 0.8) nghĩa là bạn chỉ dự đoán dương tính khi mô hình rất tự tin. Dương tính giả giảm xuống, nên precision tăng lên. Nhưng bạn bỏ sót nhiều dương tính thật hơn, nên recall giảm xuống.

Hạ ngưỡng (ví dụ, từ 0.5 xuống 0.2) nghĩa là bạn gần như chắc chắn là dương tính hơn. Bạn bắt được nhiều dương tính thật hơn, nên recall tăng lên. Nhưng nhiều ca âm tính bị gần như sai hơn, nên precision giảm xuống.

Việc chọn ngưỡng là một quyết định kinh doanh

Một xét nghiệm sàng lọc ung thư có thể dùng ngưỡng thấp (ví dụ 0.3): bỏ sót một ca ung thư còn tệ hơn nhiều so với việc chỉ định thêm một xét nghiệm theo dõi không cần thiết. Recall cao quan trọng hơn precision cao.

Một bộ lọc thư rác có thể dùng ngưỡng cao (ví dụ 0.8): xóa nhầm một email hợp lệ còn tệ hơn việc để lọt vài thư rác. Precision cao quan trọng hơn recall cao.

Không có ngưỡng "đúng". Lựa chọn phù hợp phụ thuộc vào chi phí tương đối của dương tính giả và âm tính giả trong ứng dụng cụ thể của bạn.

5.5.1 Đường cong ROC và AUC

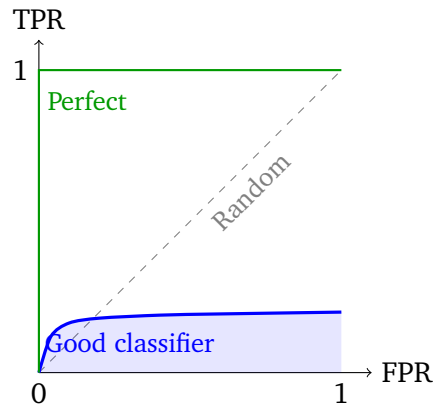
Đường cong đặc trưng người nhận (ROC) cung cấp cái nhìn toàn diện về hiệu năng của bộ phân loại trên tất cả các ngưỡng có thể có.

Hầu hết các bộ phân loại đưa ra một xác suất hoặc điểm số chứ không phải một dự đoán lớp cứng. Ta chuyển nó thành dự đoán bằng cách chọn một ngưỡng: nếu điểm số

vượt quá ngưỡng thì dự đoán là dương tính. Các ngưỡng khác nhau tạo ra những sự đánh đổi khác nhau giữa việc bắt được dương tính (Tỉ lệ Dương tính Thật) và việc gán cờ sai các ca âm tính (Tỉ lệ Dương tính Giả).

- **Tỉ lệ Dương tính Thật (TPR)** = $\frac{TP}{TP+FN}$ = Recall (tỉ lệ các ca dương tính thực sự được nhận diện đúng)
- **Tỉ lệ Dương tính Giả (FPR)** = $\frac{FP}{FP+TN}$ (tỉ lệ các ca âm tính thực sự bị gán cờ sai)

Đường cong ROC thể hiện TPR (trục tung) theo FPR (trục hoành) cho mọi ngưỡng có thể có:



Hình 5.4: Các đường cong ROC của những bộ phân loại khác nhau. Vùng tô màu biểu diễn AUC. Một bộ phân loại hoàn hảo ôm sát góc trên-trái; một bộ phân loại ngẫu nhiên đi theo đường chéo.

Hiểu đường cong ROC:

- **Góc trên-trái** (0, 1) đại diện cho phân loại hoàn hảo: TPR=1 (bắt được toàn bộ dương tính), FPR=0 (không có cảnh báo sai)
- **Đường chéo** đại diện cho một bộ phân loại ngẫu nhiên không phân biệt được các lớp
- Một đường cong càng gần góc trên-trái thì càng tốt

Diện tích dưới đường cong (AUC) tóm tắt đường cong ROC thành một con số duy nhất:

- AUC = 1.0: Bộ phân loại hoàn hảo
- AUC = 0.5: Đoán bừa (không có sức phân biệt)
- AUC < 0.5: Tệ hơn cả ngẫu nhiên (dự đoán bị đảo ngược)

Ý nghĩa của AUC

AUC có một cách diễn giải xác suất tuyệt đẹp: nó bằng xác suất mà một ví dụ dương tính được chọn ngẫu nhiên sẽ được xếp hạng cao hơn một ví dụ âm tính được chọn ngẫu nhiên.

AUC = 0.85 nghĩa là: nếu bạn chọn ngẫu nhiên một ví dụ dương tính và một ví dụ âm tính, có 85% khả năng bộ phân loại gán điểm cao hơn cho ví dụ dương tính.

Quy tắc kinh nghiệm: AUC > 0.9 là xuất sắc; 0.8–0.9 là tốt; 0.7–0.8 là khá; < 0.7 có thể kém.

Khi nào dùng AUC thay cho các số đo khác:

- Hãy dùng AUC khi bạn muốn đánh giá khả năng xếp hạng mà không cần chốt một ngưỡng cụ thể
- Hãy dùng precision/recall/F1 khi bạn đã có một ngưỡng cụ thể trong đầu hoặc khi sự mất cân bằng lớp quá nghiêm trọng
- AUC có thể gây hiểu lầm với các tập dữ liệu mất cân bằng nặng nề; khi đó nên cân nhắc dùng đường cong precision-recall

5.5.2 Hiệu chuẩn: Các xác suất có ý nghĩa không?

Một bộ phân loại có thể xếp hạng chính xác các ca dương tính cao hơn các ca âm tính (AUC tốt) trong khi vẫn đưa ra những ước lượng xác suất sai lệch dữ dội. **Hiệu chuẩn** đo lường xem các xác suất dự đoán có khớp với tần suất thực tế hay không.

Một mô hình được hiệu chuẩn tốt nếu, trong số tất cả các ca mà nó dự đoán “có 70% khả năng vỡ nợ”, xấp xỉ 70% thực sự vỡ nợ. Một mô hình dự đoán 70% nhưng chỉ có 30% vỡ nợ thì *chính xác* về thứ hạng (nó xác định đúng các ca rủi ro hơn) nhưng lại *hiệu chuẩn kém* (những con số xác suất của nó vô nghĩa).

Điều ngược lại cũng có thể xảy ra. Một mô hình có thể hiệu chuẩn tốt nhưng không chính xác: nó ước lượng đúng rằng tỉ lệ vỡ nợ chung là 10%, nhưng lại gán 10% cho từng người vay. Các xác suất “đúng” về mặt trung bình, nhưng mô hình không có sức phân biệt nào.

Khi nào hiệu chuẩn mới quan trọng

Hiệu chuẩn quan trọng mỗi khi chính bản thân xác suất dự đoán dẫn dắt một quyết định, chứ không chỉ thứ hạng. Nếu một bác sĩ cần nói với bệnh nhân “nguy cơ mắc bệnh này của bạn là 30%”, con số đó phải có ý nghĩa. Nếu bạn chỉ cần xếp bệnh nhân theo nguy cơ từ cao xuống thấp thì thứ hạng (AUC) là đủ.

Điểm kiểm tra đạo đức: Khi số đo tốt trở thành xấu

Một bệnh viện triển khai một mô hình để dự đoán nguy cơ bệnh nhân phải nhập viện lại. Mô hình có $AUC = 0.85$.

Câu hỏi kỹ thuật – đạo đức:

1. Bệnh viện dùng mô hình để *từ chối* chăm sóc theo dõi cho các bệnh nhân “rủi ro thấp” (để tiết kiệm chi phí). Mô hình có độ chính xác (precision) 90% ở nhóm rủi ro cao nhưng chỉ 70% ở nhóm rủi ro thấp. Vấn đề đạo đức nằm ở đâu? 2. Ban lãnh đạo đề xuất hạ ngưỡng để gắn cờ nhiều bệnh nhân hơn là rủi ro cao. Điều này làm recall tăng lên nhưng precision giảm xuống. Ai được lợi? Ai bị thiệt hại?
3. Một bác sĩ lập luận: “Tôi nên ghi đề lên mô hình mỗi khi tôi không đồng ý với nó.” Người khác lập luận: “Việc áp dụng mô hình nhất quán là công bằng hơn phán đoán của con người.” Hãy phân tích cả hai quan điểm. *Đây chính là dạng bài hỗn hợp kỹ thuật – đạo đức xuất hiện trong IAIQ*

Dừng lại để suy nghĩ: Chọn đúng số đo

Khi có rất nhiều số đo, làm sao chọn?

Số đo	Dùng khi	Lưu ý
Độ chính xác	Các lớp cân bằng; mọi lỗi có chi phí như nhau	Gây hiểu lầm với dữ liệu mất cân bằng
Độ chính xác (precision)	Dương tính giả tốn kém (ví dụ: thư rác gây phiền nhiễu người dùng)	Bỏ qua âm tính giả
Độ nhạy (recall)	Âm tính giả tốn kém (ví dụ: bỏ sót ung thư)	Bỏ qua dương tính giả
Điểm F1	Cần sự cân bằng; cần một con số duy nhất	Giả định rằng precision và recall cũng quan trọng như nhau
AUC-ROC	Đánh giá qua nhiều ngưỡng; so sánh các mô hình	Không cho biết hiệu năng tại một ngưỡng cụ thể
Trọng số chi phí	Các loại lỗi khác nhau có chi phí khác nhau	Đòi hỏi phải biết các chi phí đó

5.6 Các số đo cho hồi quy

Với hồi quy (dự đoán các số liên tục), chúng ta đo lường sai số dự đoán.

Sai số Trung bình Tuyệt đối (MAE) là trung bình của các hiệu tuyệt đối:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE dễ diễn giải vì dùng cùng đơn vị với giá trị mục tiêu.

Sai số Bình phương Trung bình (MSE) bình phương các sai số, khiến sai số lớn bị phạt nặng hơn:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\text{RMSE} = \sqrt{\text{MSE}}$ trở về đơn vị gốc.

R bình phương (R^2) cho biết phần phân phối mà mô hình giải thích được:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

$R^2 = 1$ có nghĩa là dự đoán hoàn hảo; $R^2 = 0$ có nghĩa là không tốt hơn việc dự đoán giá trị trung bình.

5.7 Kiểm chéo

Nếu bạn không có nhiều dữ liệu thì sao? Một cách chia huấn luyện – kiểm định đơn lẻ có thể không đáng tin; bạn có thể may mắn hoặc xui xẻo với việc những ví dụ nào rơi vào phần nào.

Kiểm chéo K-fold giải quyết điều này bằng cách dùng toàn bộ dữ liệu cho cả huấn luyện lẫn kiểm định:

Fold					
1	Val				→ Error ₁
2		Val			→ Error ₂
3			Val		→ Error ₃
4				Val	→ Error ₄
5					Val → Error ₅

Final Score = Average of all errors

Hình 5.5: Kiểm chéo 5-fold: mỗi phần đóng vai trò kiểm định đúng một lần.

Quy trình: chia dữ liệu thành k phần bằng nhau; với mỗi phần i , huấn luyện trên các phần còn lại $k - 1$ và kiểm định trên chính phần i đó; lấy trung bình các k điểm kiểm định.

Lợi ích: ước lượng đáng tin cậy hơn, dùng toàn bộ dữ liệu cho cả huấn luyện lẫn kiểm định, mỗi ví dụ đều được kiểm định đúng một lần. Chi phí: phải huấn luyện k mô hình thay vì 1. Các lựa chọn phổ biến: $k = 5$ hoặc $k = 10$.

5.8 Cách mọi người vô tình gian lận

Ngay cả những người thực hành có thiện chí cũng có thể vô tình làm mất hiệu lực của việc đánh giá. Hiểu các cạm bẫy này là điều thiết yếu để tạo ra những kết quả đáng tin cậy.

5.8.1 Quá khớp vào bảng xếp hạng

Trong các cuộc thi, bảng xếp hạng công khai hiển thị hiệu năng trên một phần của dữ liệu kiểm tra. Một kiểu hành vi nguy hiểm xuất hiện:

1. Gửi dự đoán → xem điểm
2. Điều chỉnh mô hình → gửi lại → xem điểm
3. Lặp lại hàng trăm lần

Mỗi lần gửi đều làm rò rỉ thông tin về tập kiểm tra. Sau đủ nhiều lần gửi, bạn đang âm thầm *sửa mô hình cho khớp với dữ liệu kiểm tra*. Điểm trên bảng xếp hạng công khai trở nên vô nghĩa, còn điểm trên bảng xếp hạng riêng (trên dữ liệu bị giữ lại) sụp đổ đáng kể.

Tại sao thứ hạng trên bảng xếp hạng $\neq$ không phản ánh chất lượng mô hình

Một thứ hạng cao trên bảng xếp hạng **không** có nghĩa là bạn có một mô hình tốt. Nó có thể có nghĩa là:

- Bạn may mắn trên tập kiểm tra công khai
- Bạn đã quá khớp vào dữ liệu kiểm tra công khai qua nhiều lần gửi liên tục
- Cách chia công khai/riêng không khớp với phân phối ngoài đời thực

Tín hiệu đáng tin cậy duy nhất là điểm kiểm chéo của bạn trên dữ liệu của chính bạn.

Những mô hình xếp hạng tốt trên kiểm chéo nhưng kém trên bảng xếp hạng công khai thường *tốt hơn* so với ngược lại; chúng thường sẽ hoạt động tốt hơn trên bảng xếp hạng riêng và khi triển khai thực tế.

Cách khắc phục: Hãy tin điểm kiểm chéo của mình chứ đừng tin điểm bảng xếp hạng. Hạn chế số lần gửi.

5.8.2 Vì sao “Chỉ xem thêm một lần nữa thôi” phá vỡ các bảo đảm

Giá trị của tập kiểm tra đến từ một tính chất duy nhất: mô hình chưa từng bị nó chi phối. Mỗi lần bạn kiểm tra hiệu năng trên tập kiểm tra rồi điều chỉnh mô hình, bạn đang rò rỉ thông tin từ tập kiểm tra vào các quyết định thiết kế mô hình. Điều này đúng ngay cả khi bạn không trực tiếp huấn luyện trên đó.

Hãy cân nhắc: bạn huấn luyện Mô hình A, kiểm tra điểm trên tập kiểm tra (82%), kết luận rằng nó chưa đủ tốt, xây Mô hình B với các đặc trưng khác, kiểm tra lại (85%), tinh chỉnh thêm, kiểm tra lại nữa (87%). Sau 20 vòng lặp, bạn báo cáo độ chính xác kiểm tra là 87%. Nhưng đây không còn là một ước lượng trung thực nữa. Rõ ràng bạn đã dùng tập kiểm tra để lựa chọn trong số 20 mô hình, và chỉ do may rủi thôi, một số mô hình sẽ đạt điểm cao hơn trên tập kiểm tra cụ thể này so với khi chúng được đánh giá trên dữ liệu mới.

Đây là cùng một vấn đề thống kê như kiểm định giả thuyết nhiều lần. Nếu bạn đánh giá 20 mô hình trên cùng một tập kiểm tra, bạn đang thực hiện 20 phép so sánh. Ngay cả khi không mô hình nào thực sự tốt hơn, một số mô hình vẫn sẽ trông có vẻ tốt hơn chỉ vì biến động ngẫu nhiên. Bạn xem càng nhiều lần, điểm “tốt nhất” của bạn càng bị thổi phồng lên.

Cách khắc phục: Chỉ chạm vào tập kiểm tra đúng một lần. Dùng kiểm chéo cho mọi quyết định trong quá trình phát triển. Nếu buộc phải đánh giá nhiều ứng viên cuối cùng, hãy áp dụng các hiệu chỉnh thống kê hoặc dùng một tập kiểm tra mới hoàn toàn.

5.8.3 Rò rỉ dữ liệu trong bước tiền xử lý

Một lỗi tinh vi nhưng phổ biến: thực hiện các bước tiền xử lý trên *toàn bộ tập dữ liệu* trước khi chia.

Sai và Đúng

Sai:

1. Tính trung bình/độ lệch chuẩn của toàn bộ dữ liệu
2. Chuẩn hóa toàn bộ dữ liệu
3. Chia thành tập huấn luyện/kiểm tra

Đúng:

1. Chia thành tập huấn luyện/kiểm tra
2. Chỉ tính trung bình/độ lệch chuẩn của tập huấn luyện
3. Chuẩn hóa tập huấn luyện bằng số liệu thống kê của tập huấn luyện
4. Chuẩn hóa tập kiểm tra bằng số liệu thống kê của tập huấn luyện

Cách Sai này làm rò rỉ thông tin của tập kiểm tra vào quá trình huấn luyện.

5.8.4 Rò rỉ theo thời gian

Với dữ liệu chuỗi thời gian, việc chia tập huấn luyện/kiểm tra một cách ngẫu nhiên là không hợp lệ. Nếu bạn huấn luyện trên dữ liệu năm 2023 để dự đoán năm 2022, bạn đang dùng tương lai để dự đoán quá khứ.

Cách khắc phục: Luôn chia theo thời gian. Huấn luyện trên quá khứ, kiểm định trên tương lai.

5.8.5 Tập kiểm tra tốt $\neq$ không đồng nghĩa với thực tế tốt

Ngay cả với một tập kiểm tra được xây dựng hoàn hảo, một quá trình đánh giá trung thực và không có rò rỉ, một điểm kiểm tra cao vẫn không bảo đảm mô hình sẽ hoạt động tốt khi triển

khai. Tập kiểm tra đo lường hiệu năng trên dữ liệu được lấy từ cùng phân phối với dữ liệu huấn luyện. Thế giới thực có thể khác.

Dân số bệnh nhân thay đổi. Sở thích người tiêu dùng thay đổi. Điều kiện kinh tế biến đổi. Một mô hình đạt 95% trên một tập kiểm tra lấy từ dữ liệu bệnh viện năm 2024 có thể chỉ đạt 80% trên dữ liệu năm 2026 của một mạng lưới bệnh viện khác. Điểm kiểm tra không sai; nó chính xác đo lường hiệu năng dưới điều kiện của năm 2024. Nhưng nó không phải là một lời hứa về tương lai.

Đây là lý do Chương 3 nhấn mạnh về dịch chuyển phân phối và tính hợp lệ của tập kiểm tra. Điểm kiểm tra là một ước lượng dưới những giả định cụ thể. Khi những giả định ấy không còn đúng, ước lượng cũng không còn đúng.

5.9 Tinh chỉnh siêu tham số

Việc tìm được các siêu tham số tốt có thể làm nên khác biệt giữa thành công và thất bại.

Tìm kiếm lưới thử mọi tổ hợp của các giá trị đã chỉ định. Đơn giản nhưng tốn kém. 5 siêu tham số với 10 giá trị mỗi cái nghĩa là $10^5 = 100,000$ tổ hợp.

Tìm kiếm ngẫu nhiên lấy mẫu các tổ hợp một cách ngẫu nhiên. thường tìm ra lời giải tốt nhanh hơn vì không phải siêu tham số nào cũng quan trọng như nhau.

Mẹo thực hành: bắt đầu với tốc độ học (thường là yếu tố quan trọng nhất).

Dùng thang logarit (hãy thử 0.001, 0.01, 0.1, chứ đừng thử 0.1, 0.2, 0.3). Không tinh chỉnh quá mức. Quá nhiều thí nghiệm có thể khiến bạn quá khớp vào chính tập kiểm định.

5.10 Tóm tắt

Điểm cần nhớ

Chia dữ liệu: Huấn luyện (70%) để học, Kiểm định (15%) để tinh chỉnh, Kiểm tra (15%) để đánh giá cuối cùng. Không bao giờ kiểm thử trên dữ liệu huấn luyện. **Thiên vị – Phương sai:** Thiếu khớp = quá đơn giản (thiên vị cao). Quá khớp = quá phức tạp (phương sai cao). Điểm ngọt nằm ở giữa phụ thuộc vào độ lớn dữ liệu và mức nhiễu.

Các số đo phân loại: Độ chính xác gây hiểu lầm với dữ liệu mất cân bằng. Hãy dùng precision (dương tính giả tổn kém), recall (âm tính giả tổn kém), hoặc F1 (cân bằng). Lựa chọn ngưỡng là một quyết định kinh doanh.

Hiệu chuẩn: Thứ hạng tốt (AUC) không kéo theo xác suất có ý nghĩa. Hiệu chuẩn quan trọng khi chính giá trị xác suất dẫn dắt quyết định.

ROC và AUC: ROC thể hiện TPR so với FPR qua các ngưỡng. AUC tóm tắt khả năng phân biệt.

Kiểm chéo: Kiểm chéo K-fold dùng toàn bộ dữ liệu cho cả huấn luyện lẫn kiểm định, cho ra các ước lượng đáng tin cậy hơn.

Tính toàn vẹn của tập kiểm tra: Mỗi lần nhìn vào tập kiểm tra đều làm rò rỉ thông tin. “Chỉ xem thêm một lần nữa thôi” chính là cùng một sai lầm thống kê như kiểm định giả thuyết nhiều lần.

Kiểm thử ≠ Triển khai: Hiệu năng kiểm thử tốt đo độ chính xác dưới điều kiện kiểm thử. Dịch chuyển phân phối có thể khiến ước lượng đó trở nên vô nghĩa trong thế giới thực.

1

5.11 Bài tập luyện

1. Với 10.000 điểm dữ liệu, hãy mô tả cách bạn sẽ chia dữ liệu để tinh chỉnh siêu tham số. Tỷ lệ cho mỗi tập là bao nhiêu?
2. Một mô hình đạt độ chính xác huấn luyện 99% nhưng chỉ đạt độ chính xác kiểm định 65%. Hãy chẩn đoán bằng góc nhìn thiên vị – phương sai.
3. Hãy tính precision, recall và F1 từ ma trận nhầm lẫn sau đây:

	Pred +	Pred –
Actual +	450	50
Actual –	100	400

¹The theory of why cross-validation works is covered in Chapter 11 (PAC learning and generalization bounds).

4. Khi nào bạn nên tối ưu theo precision thay vì recall? Hãy cho một ví dụ ngoài đời thực.

5. **(Dạng bài chứng minh: Vì sao kiểm chéo hoạt động)**

Trong kiểm chéo k -fold, ta huấn luyện k mô hình, mỗi mô hình trên $(k - 1)/k$ dữ liệu, rồi lấy trung bình sai số kiểm định của chúng.

(a) Hãy giải thích trực giác về sao việc lấy trung bình trên k phần cho ước lượng đáng tin cậy hơn so với một cách chia huấn luyện/kiểm tra đơn lẻ.

(b) Khi k tăng lên và tiến gần n (kiểm chéo loại-trừ-một), điều gì xảy ra với:

- Thiên vị của ước lượng sai số?
- Phương sai của ước lượng sai số?
- Chi phí tính toán?

(c) Tại sao $k = 5$ hay $k = 10$ thường là một sự thỏa hiệp tốt? (d) Nếu dữ liệu của bạn có các nhóm tự nhiên (ví dụ: nhiều mẫu cho cùng một bệnh nhân), tại sao kiểm chéo k -fold tiêu chuẩn lại có thể cho ra các ước lượng lạc quan? Bạn nên làm gì thay thế?

6. **(Dạng bài chứng minh: Phân rã thiên vị – phương sai)**

Với hồi quy có hàm thật $f(x)$ và mô hình $\hat{f}(x)$:

$$\mathbb{E}[(f(x) - \hat{f}(x))^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

(a) Hãy định nghĩa thiên vị theo $\mathbb{E}[\hat{f}(x)]$ và $f(x)$.

(b) Một mô hình hằng số $\hat{f}(x) = c$ có thiên vị cao. Vì sao? (c) Một mô hình nội suy qua mọi điểm huấn luyện có phương sai cao. Vì sao?

(d) Chuẩn hóa làm tăng thiên vị. Vì sao điều này thường lại cải thiện sai số kiểm thử?

7. **(Nhiều phần: Khi độ chính xác đánh lừa)** Một xét nghiệm bệnh (tỉ lệ mắc 1%) cho:

	Test +	Test –
Has Disease	90	10
No Disease	495	9405

(a) Hãy tính độ chính xác, precision, recall, F1.

(b) Tại sao độ chính xác 94,95% là gây hiểu lầm?

(c) Giá trị dương đoán đúng là bao nhiêu? Tại sao nó lại khác xa recall đến vậy?

8. **(Phát hiện gian lận)** Mô hình của Đồng nghiệp bạn đạt $AUC = 0.99$. Bằng ba cách nào họ có thể vô tình làm cho điểm số này bị thổi phồng?

9. Một mô hình có độ chính xác 100% trên tập huấn luyện / 60% trên tập kiểm tra. Một mô hình khác có 70% trên tập huấn luyện / 65% trên tập kiểm tra. Mô hình nào tốt hơn và vì sao?

Chương 6

Học không giám sát và Phân cụm

Phân loại là bản tính của con người.

Robert Darnton

6.1 Giới thiệu: Học mà không có nhãn

Phần lớn cuốn sách này, chúng ta đã tập trung vào học có giám sát, nơi mỗi ví dụ huấn luyện đều kèm theo một nhãn cho biết “đáp án đúng”. Chúng ta đưa cho thuật toán những bức ảnh được gắn nhãn “mèo” hay “chó”, và nó học cách phân loại những bức ảnh mới. Chúng ta cung cấp giá nhà cùng các đặc trưng của ngôi nhà, và nó học cách dự đoán giá cho những ngôi nhà mới. Các nhãn dẫn dắt quá trình học, cho thuật toán biết cần tìm những mẫu hình nào.

Nhưng nếu chúng ta có dữ liệu không kèm nhãn thì sao? Tình huống này cực kỳ phổ biến trong thực tế. Một công ty có thể có hàng triệu bản ghi giao dịch của khách hàng nhưng không có nhãn nào chỉ ra loại khách hàng. Một nhà sinh học có thể có dữ liệu biểu hiện gen từ hàng nghìn tế bào nhưng không có phân loại tế bào. Một nền tảng mạng xã hội có hàng tỷ bài đăng nhưng không có phân loại chủ đề. Dữ liệu vẫn tồn tại, có thể rất giàu cấu trúc, nhưng không ai nói cho chúng ta biết cấu trúc đó là gì.

Học không giám sát giải quyết chính xác thách thức đó: phát hiện những mẫu hình và cấu trúc ẩn giấu trong dữ liệu mà không có bất kỳ nhãn nào dẫn dắt. Thay vì học một ánh xạ từ đầu vào sang đầu ra, chúng ta tìm cách tự hiểu bản thân dữ liệu — các cụm của nó, những chiều không gian nền tảng của nó, các thuộc tính thống kê của nó.

Supervised vs. Unsupervised Learning

Supervised Learning: We have input-output pairs $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. The goal is to learn a function $f : X \rightarrow Y$ that predicts outputs from inputs. Labels tell us what to learn.

Unsupervised Learning: We have only inputs $x_1, x_2, \dots, x_n$ with no labels. The goal is to discover structure (groups, patterns, representations) hidden in the data. We must figure out what is worth learning.

Học không giám sát bao gồm nhiều bài toán khác nhau. **Phân cụm** gom các điểm dữ liệu tương tự lại với nhau, phát hiện ra các danh mục tự nhiên trong dữ liệu. **Giảm chiều** tìm các biểu diễn có chiều thấp hơn nhưng vẫn giữ được thông tin quan trọng, giúp dữ liệu dễ trực quan hóa và phân tích hơn. **Ước lượng mật độ** mô hình hóa phân phối xác suất đã sinh ra dữ liệu. Chương này tập trung chủ yếu vào phân cụm, một trong những kỹ thuật không giám sát được sử dụng rộng rãi nhất.

6.2 Phân cụm: Tìm các nhóm trong dữ liệu

Phân cụm dựa trên một trực giác đơn giản: các điểm dữ liệu thường tự nhiên rơi vào những nhóm, và việc phát hiện ra những nhóm đó cho thấy điều gì đó có ý nghĩa về cấu trúc của dữ liệu.

Có một điểm khác biệt quan trọng so với học có giám sát, và nó định hình mọi thứ trong chương này. Trong phân loại, luôn có một đáp án đúng: email là thư rác hoặc không phải. Trong phân cụm, không có “đáp án chuẩn”. Cùng một bộ dữ liệu có thể được phân cụm theo nhiều cách hợp lệ, tùy thuộc vào việc bạn dùng đặc trưng nào, chọn độ đo khoảng cách nào và đặt giá trị nào cho k . Hai nhà phân tích có thể tạo ra hai cách phân cụm hoàn toàn khác nhau cho cùng dữ liệu và cả hai đều hợp lý. **Sự mơ hồ về mục tiêu** này có nghĩa là việc đánh giá các cụm đòi hỏi kiến thức chuyên môn và phán đoán thực tế, chứ không chỉ là một chỉ số.

Hãy xem xét các khách hàng của một nhà bán lẻ trực tuyến. Có người mua thường xuyên nhưng chi tiêu mỗi lần rất ít; có người hiếm khi mua nhưng lại tiêu xài rất rộng tay; lại có người chỉ mua khi có đợt giảm giá. Những hành vi khác nhau đó gợi ý những phân khúc khách hàng tự nhiên. Nếu chúng ta có thể nhận diện các phân khúc này một cách tự động, chúng ta có thể điều chỉnh chiến lược tiếp thị cho từng nhóm. Nhà bán lẻ chưa bao giờ gán nhãn khách hàng là “người mua thường xuyên” hay “kẻ săn khuyến mãi”; thuật toán phân cụm tự khám phá ra các danh mục đó chỉ từ các mẫu hành vi mua sắm.

Mục tiêu của phân cụm

Cho một tập dữ liệu $S = \{x_1, x_2, \dots, x_n\}$, hãy chia các điểm dữ liệu thành k nhóm (cụm) sao cho:

Các điểm trong cùng một cụm **tương tự** nhau. Chúng chia sẻ những đặc điểm chung.

Các điểm ở những cụm khác nhau **khác biệt** với nhau. Chúng khác nhau theo những cách có ý nghĩa.

Thuật toán phải vừa phát hiện ra cách gán cụm, vừa tự tìm ra “tương tự” nghĩa là gì đối với dữ liệu này.

Phân cụm xuất hiện ở gần như mọi lĩnh vực mà dữ liệu được thu thập. Trong sinh học, phân cụm gom các gen có mô hình biểu hiện tương tự, làm lộ ra các mối quan hệ chức năng. Trong thiên văn học, phân cụm xác định các loại thiên thể khác nhau từ đặc trưng phổ của chúng. Trong phân tích tài liệu, phân cụm sắp xếp các bài báo theo chủ đề mà không cần các danh mục định sẵn. Trong phân đoạn ảnh, phân cụm gom các pixel để nhận diện những đối tượng khác biệt trong một cảnh. Trong phát hiện bất thường, những điểm không vừa vặn với bất kỳ cụm nào có thể đại diện cho những sự kiện bất thường đáng điều tra.

6.3 Đo lường sự tương tự: Các hàm khoảng cách

Trước khi có thể gom các điểm tương tự lại, chúng ta phải định nghĩa “tương tự” nghĩa là gì. Các thuật toán phân cụm thường đo sự tương tự thông qua khía cạnh đối nghịch của nó: **khoảng cách**. Những điểm nằm gần nhau (khoảng cách nhỏ) được coi là tương tự; những điểm nằm xa nhau (khoảng cách lớn) là không tương tự.

Lựa chọn hàm khoảng cách ảnh hưởng sâu sắc đến kết quả phân cụm. Các độ đo khoảng cách khác nhau hàm chứa những quan niệm khác nhau về sự tương tự, và lựa chọn đúng phụ thuộc vào bản chất dữ liệu của bạn và loại mẫu hình bạn muốn tìm.

Một **độ đo khoảng cách** hợp lệ phải thỏa mãn ba tính chất toán học. Thứ nhất, *tính không âm*: khoảng cách không bao giờ âm, và khoảng cách từ một điểm đến chính nó bằng không. Thứ hai, *tính đối xứng*: khoảng cách từ x đến y bằng khoảng cách từ y đến x . Thứ ba, *bất đẳng thức tam giác*: đi thẳng từ x đến z không bao giờ dài hơn việc đi qua một điểm trung gian y . Những tính chất này bảo đảm khoảng cách luôn hoạt động theo những cách hợp lý trực giác.

6.3.1 Họ Minkowski

Các hàm khoảng cách phổ biến nhất thuộc **họ Minkowski**, được tham số hóa bởi một giá trị p :

$$d_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^d |x_i - y_i|^p \right)^{1/p}$$

Những giá trị khác nhau của p cho những quan niệm khác nhau về khoảng cách.

Khi $p = 2$, ta có **khoảng cách Euclid** quen thuộc, tức là khoảng cách đường thẳng giữa hai điểm:

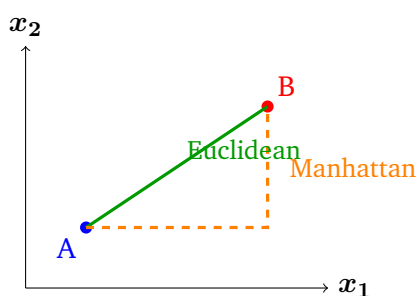
$$d_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

Đây là khoảng cách mà bạn sẽ đo bằng thước kẻ. Nó coi mọi hướng như nhau và tương ứng với trực giác hình học thường ngày của chúng ta.

Khi $p = 1$, ta có **khoảng cách Manhattan** (còn gọi là khoảng cách taxi hay khoảng cách khối phố):

$$d_1(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i|$$

Hãy hình dung việc di chuyển trong một thành phố có lưới đường: bạn không thể đi chéo xuyên qua các tòa nhà mà phải đi theo các con đường. Khoảng cách Manhattan đo quãng đường bạn sẽ đi khi tuân theo ràng buộc này. Nó đặc biệt hữu ích khi các đặc trưng biểu diễn những đại lượng về bản chất khác nhau và không nên được trộn lẫn một cách hình học.



$$\text{Euclidean: } \sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.6$$

$$\text{Manhattan: } 3 + 2 = 5$$

Hình 6.1: Khoảng cách Euclid đo đường thẳng; khoảng cách Manhattan đo đường đi dọc theo các trục tọa độ. Với hai điểm A(1,1) và B(4,3), hai cách đo này cho những giá trị khác nhau.

Khi $p = \infty$, ta có **khoảng cách Chebyshev**: độ lệch lớn nhất dọc theo bất kỳ chiều nào. Cách này hữu ích khi sai số xấu nhất quan trọng hơn tổng thể những sai lệch.

Ngoài họ Minkowski, còn có những độ đo khoảng cách khác phục vụ những mục đích chuyên biệt. **Khoảng cách cosin** đo góc giữa các vectơ thay vì độ lớn của chúng, hữu ích khi hướng quan trọng hơn độ lớn (phổ biến trong phân tích văn bản). **Khoảng cách Hamming** đếm các vị trí mà hai vectơ nhị phân khác nhau. **Khoảng cách sửa lỗi** đo số phép biến đổi tối thiểu cần để chuyển một chuỗi thành chuỗi khác.

Chuẩn hóa quy mô: Sự thất bại phổ biến nhất của phân cụm

Các thuật toán dựa trên khoảng cách rất nhạy cảm với quy mô của đặc trưng. Nếu một đặc trưng trải từ 0 đến 100.000 (thu nhập hằng năm) và đặc trưng khác từ 0 đến 10 (số con), khoảng cách Euclid sẽ hoàn toàn bị chi phối bởi thu nhập. Thuật toán sẽ phân cụm chỉ theo thu nhập, như thể các đặc trưng khác không tồn tại.

Đây chính là vấn đề mà Chương 3 đã thảo luận đối với học có giám sát, nhưng nó còn nguy hiểm hơn trong phân cụm vì không có nhãn nào để lộ ra sai lầm. Độ chính xác của một mô hình có giám sát sẽ giảm; còn thuật toán phân cụm sẽ vui vẻ tạo ra những cụm dựa trên thu nhập và báo cáo độ méo thấp.

Luôn chuẩn hóa đặc trưng trước khi phân cụm dựa trên khoảng cách (K-means, phân cụm phân cấp). Ngoại lệ là khi quy mô gốc có ý nghĩa và bạn cố tình muốn một đặc trưng nào đó chi phối.

6.4 Phân cụm K-Means

K-means là thuật toán phân cụm được sử dụng rộng rãi nhất, được yêu thích nhờ sự đơn giản và hiệu quả của nó. Thuật toán chia dữ liệu thành k cụm, trong đó mỗi cụm được đại diện bởi **trọng tâm** của nó, tức vị trí trung bình của mọi điểm trong cụm đó.

Trực giác phía sau K-means rất đơn giản và hấp dẫn. Chúng ta muốn tìm các tâm cụm sao cho mỗi điểm đều ở gần tâm được gán cho nó. Các điểm "thuộc về" tâm gần nhất, và các tâm nên được đặt sao cho tổng khoảng cách đến các điểm được gán là nhỏ nhất.

6.4.1 Thuật toán

K-means luân phiên giữa hai bước cho đến khi hội tụ:

Thuật toán K-Means

Đầu vào: các điểm dữ liệu $\{x_1, \dots, x_n\}$ và số cụm k
Khởi tạo: đặt k trọng tâm cụm $\mu_1, \dots, \mu_k$ ở những vị trí ngẫu nhiên
Lặp lại cho đến khi hội tụ:

Bước gán: Gán mỗi điểm vào trọng tâm gần nhất

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

Bước cập nhật: Di chuyển mỗi trọng tâm đến vị trí trung bình của các điểm được gán cho nó

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

Hội tụ: Dừng khi các phép gán không còn thay đổi

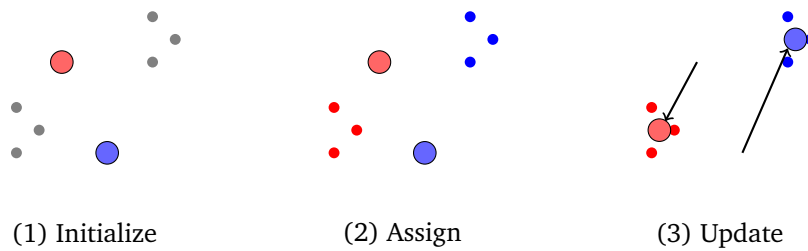
Hãy cùng lần theo xem tại sao thuật toán này lại hiệu quả. Bước gán bảo đảm mọi điểm đều thuộc về tâm cụm gần nhất. Đây là cách gán tự nhiên nhất với các tâm hiện tại. Bước cập nhật đưa mỗi tâm về đúng vị trí tâm hình học của các điểm được gán cho nó. Điều này làm tối thiểu tổng bình phương khoảng cách từ các điểm đến tâm của chúng.

Cả hai bước đều làm giảm (hoặc ít nhất không làm tăng) một đại lượng được gọi là **độ méo**:

$$J = \sum_{i=1}^n \|x_i - \mu_{c_i}\|^2$$

Đây là tổng bình phương các khoảng cách từ mỗi điểm đến trọng tâm được gán cho nó. Bước gán làm giảm độ méo vì việc gán lại một điểm cho trọng tâm gần hơn làm giảm phần đóng góp của điểm đó vào tổng. Bước cập nhật làm giảm độ méo vì trung bình chính là điểm làm tối thiểu tổng bình phương khoảng cách đến một tập điểm (một sự thật cơ bản của thống kê).

Vì độ méo giảm ở mỗi bước và không thể xuống dưới không, thuật toán sớm muộn cũng phải hội tụ. Đây là một ví dụ đẹp về tối ưu hóa qua cực tiểu hóa luân phiên: chúng ta không dễ dàng tối ưu đồng thời cả các phép gán lẫn các tâm, nhưng việc tối ưu cái này trong khi giữ cái kia cố định lại đơn giản.



Hình 6.2: Một vòng lặp của K-means: (1) Khởi tạo ngẫu nhiên đặt các trọng tâm; (2) Các điểm được gán vào trọng tâm gần nhất; (3) Các trọng tâm di chuyển đến vị trí trung bình của các điểm được gán cho chúng.

Ví dụ minh họa: K-Means tính tay

Dữ liệu: Bốn điểm trong không gian 2D: $A = (1, 1)$, $B = (2, 1)$, $C = (4, 3)$, $D = (5, 4)$

Mục tiêu: Phân cụm thành $k = 2$ nhóm.

Khởi tạo: Chọn $\mu_1 = (1, 1)$ và $\mu_2 = (5, 4)$ (các điểm A và D).

Vòng lặp 1:

Bước gán: Tính khoảng cách từ mỗi điểm đến mỗi trọng tâm.

Điểm	Dist to μ_1	Dist to μ_2	Được gán
A = (1,1)	0	5	Cụm 1
B = (2,1)	1	4.24	Cụm 1
C = (4,3)	3.61	1.41	Cụm 2
D = (5,4)	5	0	Cụm 2

Cụm 1: $\{A, B\}$ Cụm 2: $\{C, D\}$

Bước cập nhật: Tính các trọng tâm mới là trung bình của các điểm được gán.

$$\mu_1 = \frac{(1, 1) + (2, 1)}{2} = (1.5, 1) \quad \mu_2 = \frac{(4, 3) + (5, 4)}{2} = (4.5, 3.5)$$

Vòng lặp 2:

Bước gán: Tính lại các khoảng cách với các trọng tâm mới.

Điểm	Dist to μ_1	Dist to μ_2	Được gán
A = (1,1)	0.5	4.27	Cụm 1
B = (2,1)	0.5	3.54	Cụm 1
C = (4,3)	3.20	0.71	Cụm 2
D = (5,4)	4.61	0.71	Cụm 2

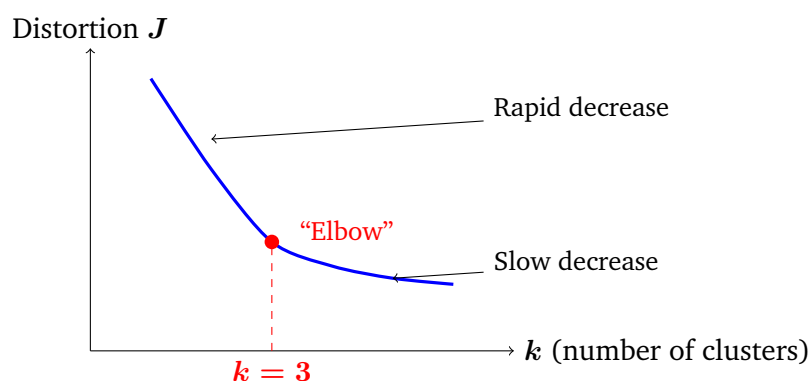
Các phép gán không đổi $\Rightarrow$ **Đã hội tụ!**

Độ méo cuối cùng: $J = 0.5^2 + 0.5^2 + 0.71^2 + 0.71^2 = 1.5$

6.4.2 Chọn số lượng cụm

K-means đòi hỏi phải chỉ định trước k , tức là số cụm. Nhưng làm sao ta biết giá trị nào là đúng? Thường thì ta không biết, và đây là một trong những thách thức nền tảng của phân cụm.

Phương pháp khuỷu tay đưa ra một cách làm kinh điển. Chúng ta chạy K-means với nhiều giá trị khác nhau của k và vẽ đồ thị độ méo theo k . Khi k tăng lên, độ méo tất yếu giảm (với $k = n$, mỗi điểm là một cụm riêng với độ méo bằng không). Ta tìm một “khuỷu tay” trên đường cong, một điểm mà tại đó độ méo ngừng giảm nhanh và bắt đầu giảm chậm. Điều này gợi ý rằng những cụm bổ sung không nắm bắt được cấu trúc có ý nghĩa, mà chỉ làm vụn thêm các nhóm vốn có.



Hình 6.3: Phương pháp khuỷu tay: độ méo giảm khi k tăng lên, nhưng tốc độ giảm chậm lại. “Khuỷu tay” tại $k = 3$ gợi ý rằng ba cụm nắm bắt cấu trúc chính.

6.4.3 Những hạn chế của K-Means

Mặc dù rất phổ biến, K-means vẫn có những hạn chế quan trọng mà bạn nên nắm rõ.

K-means giả định các cụm gần như **hình cầu** và có kích thước tương tự nhau. Khi các cụm có hình dạng kéo dài, mật độ khác nhau, hoặc kích thước chênh lệch rất lớn, K-means có thể cho kết quả kém. Thuật toán không có cách nào biểu diễn một cụm kéo dài; nó chỉ có thể đặt một trọng tâm ở giữa.

K-means **nhạy cảm với cách khởi tạo**. Những vị trí bắt đầu ngẫu nhiên khác nhau có thể dẫn đến những cách phân cụm cuối cùng khác nhau. Điều này xảy ra vì K-means tìm được một cực tiểu cục bộ của độ méo, không nhất thiết là cực tiểu toàn cục. Một cách khắc phục phổ biến là chạy thuật toán nhiều lần với các cách khởi tạo khác nhau và giữ lại kết quả tốt nhất.

K-means thực hiện **phép gán cứng**: mỗi điểm thuộc đúng một cụm. Nhưng dữ liệu thực tế thường có những điểm mơ hồ, có thể hợp lý thuộc về nhiều cụm. Một khách hàng có thể bộc lộ hành vi từ hai phân khúc khác nhau. K-means ép buộc một lựa chọn nhị phân trong khi một câu trả lời theo xác suất có thể phù hợp hơn.

Cuối cùng, K-means **nhạy cảm với các điểm ngoại lai**. Chỉ một điểm cực đoan cũng có thể kéo một trọng tâm ra xa nơi nó đáng lẽ phải ở, làm méo toàn bộ kết quả phân cụm.

6.5 Phân cụm phân cấp

Phân cụm phân cấp tiếp cận theo một hướng hoàn toàn khác so với K-means. Thay vì bắt bạn chỉ định trước số cụm, nó xây dựng một hệ phân cấp các cụm có thể được cắt ở bất kỳ cấp độ nào để tạo ra những số lượng nhóm khác nhau.

6.5.1 Phân cụm tích tụ

Dạng phổ biến nhất là phân cụm **tích tụ** (từ dưới lên):

Thuật toán phân cụm tích tụ

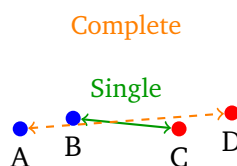
Khởi tạo: Mỗi điểm bắt đầu là một cụm riêng (n cụm với n điểm)
Lặp lại cho đến khi chỉ còn lại một cụm:

1. Tìm hai cụm gần nhau nhất
2. Gộp chúng thành một cụm duy nhất

Kết quả: Một cây (dendrogram) thể hiện trình tự các lần gộp

Câu hỏi then chốt là: làm sao chúng ta đo khoảng cách giữa các cụm (chứ không chỉ giữa các điểm)? Những **cách liên kết** khác nhau cho những câu trả lời khác nhau:

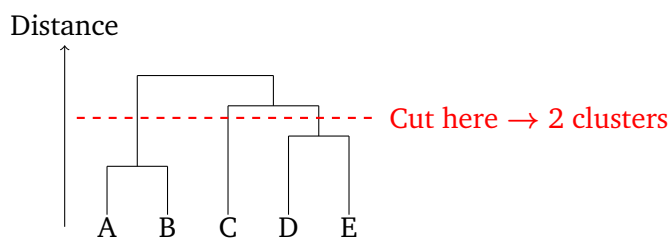
- **Liên kết đơn:** Khoảng cách giữa hai cụm = khoảng cách giữa các điểm gần nhất của chúng
- **Liên kết đầy đủ:** Khoảng cách = khoảng cách giữa các điểm xa nhất của chúng
- **Liên kết trung bình:** Khoảng cách = trung bình của tất cả các khoảng cách theo cặp
- **Phương pháp Ward:** Khoảng cách = mức tăng của tổng phương sai bên trong cụm sau khi gộp



Hình 6.4: Liên kết đơn dùng các điểm gần nhất giữa các cụm; liên kết đầy đủ dùng các điểm xa nhất.

6.5.2 Dendrogram

Kết quả của phân cụm phân cấp là một **dendrogram**, tức một sơ đồ cây thể hiện trình tự các lần gộp và những khoảng cách tại đó chúng xảy ra.



Hình 6.5: Một dendrogram thể hiện cấu trúc phân cấp. Cắt ở những độ cao khác nhau cho ra những số lượng cụm khác nhau.

Để thu được một số cụm nhất định, bạn “cắt” dendrogram theo chiều ngang ở độ cao phù hợp. Số đường dọc mà bạn cắt qua bằng số cụm.

Ưu điểm của phân cụm phân cấp:

- Không cần chỉ định trước k
- Cho ra một cấu trúc phân cấp phong phú, có thể có ý nghĩa
- Tất định (không có khởi tạo ngẫu nhiên)

Hạn chế:

- Tốn kém về tính toán: $O(n^2)$ bộ nhớ, $O(n^3)$ thời gian với cách cài đặt ngây thơ
- Một khi đã gộp thì không thể hoàn tác
- Những cách liên kết khác nhau có thể cho kết quả rất khác nhau

6.6 Mô hình hỗn hợp Gaussian

Mô hình hỗn hợp Gaussian (GMM) giải quyết một số hạn chế của K-means bằng cách tiếp cận phân cụm theo xác suất. Thay vì các phép gán cứng, GMM tính xác suất mà mỗi điểm thuộc về mỗi cụm. Thay vì biểu diễn các cụm bằng những điểm đơn lẻ (trọng tâm), GMM biểu diễn chúng bằng các phân phối xác suất.

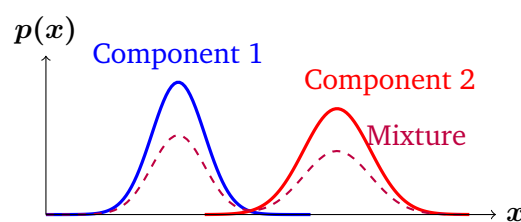
6.6.1 Mô hình sinh

GMM giả định dữ liệu được sinh ra bởi quá trình sau: trước tiên, chọn ngẫu nhiên một trong các phân phối Gauss k ; sau đó, lấy mẫu ngẫu nhiên một điểm từ phân phối đó. Những điểm khác nhau có thể đến từ những phân phối Gauss khác nhau, nhưng chúng ta không quan sát được phân phối nào đã sinh ra từng điểm. Chúng ta chỉ quan sát được chính các điểm đó.

Về mặt toán học, xác suất quan sát được một điểm x là một tổng có trọng số của các mật độ Gauss:

$$p(x) = \sum_{j=1}^k \phi_j \cdot \mathcal{N}(x|\mu_j, \Sigma_j)$$

Tại đây, ϕ_j là **trọng số trộn**. Xác suất chọn thành phần j (các trọng số này phải cộng lại bằng 1). $\mathcal{N}(x|\mu_j, \Sigma_j)$ là mật độ xác suất Gaussian với trung bình μ_j và ma trận hiệp phương sai Σ_j . Trung bình quyết định Gaussian được đặt tâm ở đâu; hiệp phương sai quyết định hình dạng của nó (hình cầu, kéo dài, nghiêng).



Hình 6.6: Một hỗn hợp của hai phân phối Gaussian (màu xanh và đỏ) cùng phân phối kết hợp của chúng (đường đứt nét màu tím). Bất kỳ điểm nào cũng có thể đến từ thành phần này hoặc thành phần kia, với những xác suất khác nhau.

6.6.2 Phép gán mềm: Trách nhiệm

Điểm cách tân then chốt của GMM so với K-means là **phép gán mềm**. Thay vì tuyên bố rằng điểm x_i thuộc về cụm j , GMM tính một **trách nhiệm**

γ_{ij} , tức xác suất mà thành phần j đã sinh ra điểm x_i :

$$\gamma_{ij} = \frac{\phi_j \cdot \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_{l=1}^k \phi_l \cdot \mathcal{N}(x_i | \mu_l, \Sigma_l)}$$

Đây đơn giản là định lý Bayes: biết rằng chúng ta đã quan sát được x_i , xác suất nó đến từ thành phần j là bao nhiêu? Tử số là xác suất để j sinh ra x_i ; mẫu số chuẩn hóa để các xác suất cộng lại bằng 1.

Một điểm có thể có những trách nhiệm $\gamma_{i1} = 0.7$ và $\gamma_{i2} = 0.3$, nghĩa là 70% khả năng thuộc về cụm 1 và 30% thuộc về cụm 2. Phép gán mềm này nắm bắt sự bất định vốn có của phân cụm: điểm đó nằm giữa các cụm, và dữ liệu không gán nó một cách dứt khoát cho bên nào.

6.7 Thuật toán Kỳ vọng – Tối đa hóa

Làm sao chúng ta tìm được các tham số (trung bình, hiệp phương sai, trọng số trộn) của một GMM? Cách tiếp cận tiêu chuẩn là thuật toán **Kỳ vọng – Tối đa hóa** (EM), một phương pháp thanh lịch để học các mô hình xác suất có biến ẩn.

Thách thức mang tính vòng tròn: nếu chúng ta biết thành phần nào đã sinh ra từng điểm, việc ước lượng các tham số Gaussian sẽ rất dễ dàng. Và nếu chúng ta biết các tham số Gaussian, việc tính các trách nhiệm cũng sẽ rất dễ dàng. Nhưng chúng ta không biết cái nào cả. EM phá vỡ sự vòng tròn này bằng cách luân phiên giữa hai phép tính đó.

Thuật toán EM cho GMM

Khởi tạo: Đặt các giá trị ngẫu nhiên cho trung bình μ_j , hiệp phương sai Σ_j và trọng số trộn ϕ_j

Lặp lại cho đến khi hội tụ:

Bước E (Kỳ vọng): Tính các trách nhiệm bằng các tham số hiện tại

$$\gamma_{ij} = \frac{\phi_j \cdot \mathcal{N}(x_i | \mu_j, \Sigma_j)}{\sum_l \phi_l \cdot \mathcal{N}(x_i | \mu_l, \Sigma_l)}$$

Bước M (Tối đa hóa): Cập nhật các tham số bằng các trách nhiệm

$$\mu_j = \frac{\sum_i \gamma_{ij} x_i}{\sum_i \gamma_{ij}}, \quad \Sigma_j = \frac{\sum_i \gamma_{ij} (x_i - \mu_j)(x_i - \mu_j)^T}{\sum_i \gamma_{ij}}$$

$$\phi_j = \frac{1}{n} \sum_i \gamma_{ij}$$

Bước E đặt câu hỏi: với các ước lượng tham số hiện tại, mỗi thành phần chịu trách nhiệm đến mức nào đối với mỗi điểm? Bước M đặt câu hỏi: với các trách nhiệm này, những giá trị tham số nào giải thích dữ liệu tốt nhất? Mỗi bước sử dụng đầu ra của bước kia, và thuật toán được chứng minh là cải thiện độ hợp lý của mô hình ở mỗi vòng lặp.

Các công thức cập nhật có cách diễn giải trực quan. Trung bình mới μ_j là một trung bình có trọng số của tất cả các điểm, trong đó trọng số chính là các trách nhiệm. Những điểm có khả năng cao hơn thuộc về thành phần j sẽ có ảnh hưởng lớn hơn đến trung bình của nó. Hiệp phương sai mới σ_j tương tự tính các độ lệch có trọng số so với trung bình. Trọng số trộn mới ϕ_j là trách nhiệm trung bình. Tức là phân phần dữ liệu mà thành phần j “thụ hưởng”.

6.7.1 K-means như một trường hợp đặc biệt

Có một mối liên hệ đẹp giữa K-means và GMM: K-means về bản chất là GMM với các phép gán cứng và các cụm hình cầu. Trong K-means, các trách nhiệm là nhị phân (0 hoặc 1), và mọi cụm đều có cùng hình dạng hình cầu. GMM khái quát hóa điều này bằng cách cho phép các trách nhiệm mềm và những hình dạng cụm linh hoạt.

Mối liên hệ này giải thích tại sao K-means lại nhanh đến vậy: các phép gán nhị phân đơn giản hơn để tính toán so với các xác suất mềm, và việc bỏ qua hiệp phương sai lại càng giảm thêm chi phí tính toán. Đổi lại là kém linh hoạt hơn. K-means không thể biểu diễn những sự bất định và hình dạng mà GMM có thể nắm bắt.

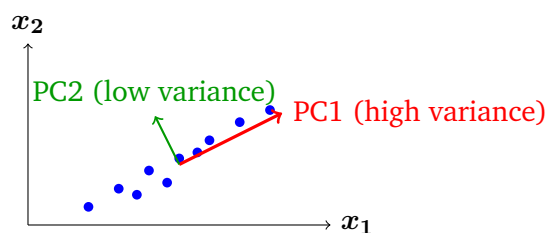
6.8 Giảm chiều

Dữ liệu nhiều chiều khó trực quan hóa, tốn kém khi tính toán, và thường chứa thông tin thừa. Các kỹ thuật **giảm chiều** chiếu dữ liệu từ không gian nhiều chiều xuống không gian ít chiều hơn trong khi vẫn giữ cấu trúc quan trọng.

6.8.1 Phân tích thành phần chính (PCA)

PCA là kỹ thuật giảm chiều cơ bản nhất. Nó tìm ra những hướng có phương sai lớn nhất trong dữ liệu và chiếu dữ liệu lên những hướng đó.

Ý chính: Nếu dữ liệu biến thiên chủ yếu dọc theo một số hướng nhất định, chúng ta có thể nắm bắt phần lớn thông tin chỉ bằng cách giữ lại những hướng đó và bỏ đi phần còn lại.



Hình 6.7: PCA tìm những hướng có phương sai lớn nhất. PC1 nắm bắt phần lớn biến thiên; việc chiếu lên riêng PC1 làm giảm số chiều trong khi vẫn giữ phần lớn thông tin.

Thuật toán PCA:

1. Đưa dữ liệu về tâm (trừ trung bình khỏi mỗi đặc trưng)
2. Tính ma trận hiệp phương sai
3. Tìm các vectơ riêng và giá trị riêng của ma trận hiệp phương sai
4. Xếp các vectơ riêng theo giá trị riêng (lớn nhất trước)
5. Chiếu dữ liệu lên k vectơ riêng hàng đầu

Các vectơ riêng chính là những **thành phần chính**, những hướng của phương sai. Các giá trị riêng cho biết mỗi thành phần nắm bắt bao nhiêu phương sai.

Ví dụ: Tính toán PCA

Xét dữ liệu 2D đã đưa về tâm tại gốc tọa độ với ma trận hiệp phương sai:

$$\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix}$$

Các giá trị riêng là $\lambda_1 = 5$ và $\lambda_2 = 1$.

Giá trị riêng đầu tiên nắm bắt $\frac{5}{5+1} = 83\%$ phương sai. Việc chiếu lên chỉ thành phần chính đầu tiên làm giảm từ 2D xuống 1D trong khi vẫn giữ 83% thông tin.

Chọn số lượng thành phần: Vẽ đồ thị phương sai tích lũy giải thích theo số thành phần. Chọn đủ thành phần để nắm bắt một tỷ lệ mong muốn (ví dụ 95%) của tổng phương sai.

Khi nào nên dùng PCA:

- Trực quan hóa (giảm xuống 2D hoặc 3D để vẽ)
- Giảm nhiễu (bỏ các thành phần có phương sai thấp)

- Tiền xử lý trước cho các thuật toán khác (giảm chi phí tính toán)
- Trích xuất đặc trưng (các thành phần chính có thể có ý nghĩa)

6.8.2 t-SNE: Trực quan hóa dữ liệu nhiều chiều

t-SNE (t-distributed Stochastic Neighbor Embedding) là một kỹ thuật giảm chiều được thiết kế riêng cho việc trực quan hóa. Khác với PCA vốn giữ phương sai toàn cục, t-SNE tập trung vào việc giữ gìn những **hàng xóm cục bộ**: những điểm ở gần nhau trong không gian nhiều chiều vẫn nên ở gần nhau trên đồ thị 2D hoặc 3D.

Thuật toán hoạt động theo hai giai đoạn. Trước tiên, nó tính độ tương đồng theo cặp trong không gian nhiều chiều gốc, trong đó “tương tự” nghĩa là khả năng bạn chọn điểm j làm hàng xóm của điểm i là cao hay thấp. Thứ hai, nó tìm một cách sắp xếp các điểm đó ở chiều thấp hơn sao cho giữ được những quan hệ hàng xóm này càng sát càng tốt, tối ưu hóa bằng rơi gradient.

Tham số **perplexity** (thường từ 5 đến 50) điều khiển mỗi điểm xem xét bao nhiêu hàng xóm. Perplexity thấp nhấn mạnh cấu trúc rất cục bộ; perplexity cao nắm bắt những mẫu hình rộng hơn. Kết quả có thể trông khác biệt về chất ở những giá trị perplexity khác nhau, vì vậy thử nhiều giá trị là một thực hành tốt.

Những hạn chế nghiêm trọng: t-SNE chỉ giữ cấu trúc cục bộ. Khoảng cách giữa các cụm trên một đồ thị t-SNE không có ý nghĩa. Hai cụm có vẻ ở xa nhau có thể lại ở gần nhau trong không gian gốc, và ngược lại. Kích thước các cụm cũng không đáng tin: t-SNE có xu hướng nới rộng những vùng dày đặc và nén lại những vùng thưa. Hãy dùng t-SNE để hỏi “có tồn tại các cụm không?”, chứ không phải “chúng cách nhau bao xa?” hay “chúng lớn cỡ nào?”.

6.9 So sánh các phương pháp không giám sát

Các phương pháp không giám sát khác nhau trả lời những câu hỏi khác nhau. Hiểu mục tiêu của chúng sẽ ngăn việc sử dụng sai.

Phương pháp	Mục tiêu	Giữ gìn	Phù hợp nhất với
K-Means	Tối thiểu hóa phương sai bên trong cụm	Tính thành viên của cụm	Các cụm hình cầu, tách biệt rõ ràng
Phân cấp	Xây dựng cây of các cụm lồng nhau	Cấu trúc đa tỷ lệ	Không biết k ; trực quan hóa dendrogram
PCA	Tối đa hóa phương sai được giải thích	Cấu trúc tuyến tính toàn cục; khoảng cách	Giảm chiều; tiền xử lý
t-SNE	Giữ gìn các hàng xóm cụm	Chỉ cấu trúc cục bộ	Trực quan hóa các cụm (chỉ 2D/3D)

Bảng 6.1: So sánh các phương pháp học không giám sát theo mục tiêu và trường hợp sử dụng.

Khi nào KHÔNG nên dùng những phương pháp này

K-Means: Tránh dùng với các cụm không hình cầu (hãy dùng DBSCAN hoặc phân cụm phổ), khi không biết số cụm (hãy dùng phân cụm phân cấp hoặc phân tích silhouette), hoặc khi kích thước các cụm rất khác nhau (các cụm nhỏ sẽ bị hấp thụ). **PCA:** Tránh dùng với cấu trúc phi tuyến (hãy dùng kernel PCA hoặc autoencoder), khi trực quan hóa các cụm (hãy dùng t-SNE), hoặc với các đặc trưng phân loại (PCA giả định biến liên tục).

t-SNE: Tránh dùng để giảm chiều phục vụ mô hình hóa (không chỉ để trực quan hóa), diễn giải khoảng cách giữa các cụm (chỉ khoảng cách cục bộ mới có ý nghĩa), hoặc với tập dữ liệu lớn (rất chậm; hãy dùng UMAP thay thế)

6.9.1 Chọn phương pháp phù hợp

“Tôi muốn giảm số chiều cho một mô hình ở hạ nguồn” → PCA

“Tôi muốn trực quan hóa dữ liệu nhiều chiều” → t-SNE (dữ liệu nhỏ) hoặc UMAP (dữ liệu lớn)

“Tôi muốn gom những mục tương tự lại” → K-Means (biết k) hoặc Phân cấp (không biết k)

“Tôi muốn tìm các điểm ngoại lai” → DBSCAN hoặc rừng cách ly (không được trình bày ở đây)

6.9.2 Diễn giải các cụm một cách cẩn thận

Một sai lầm nghiêm trọng là coi kết quả phân cụm như thể chúng lộ ra những danh mục “đúng” trong dữ liệu. Các cụm là sản phẩm của thuật toán được chọn (K-means tìm các cụm hình cầu; DBSCAN tìm các cụm dựa trên mật độ), của những đặc trưng được dùng (đặc trưng khác nhau cho ra cụm khác nhau), và của các siêu tham số (k , độ đo khoảng cách, v.v.).

Các cụm KHÔNG nhất thiết đại diện cho những danh mục tự nhiên “tồn tại” trong

dữ liệu, những phân biệt có ý nghĩa cho ứng dụng của bạn, hay một cấu trúc ổn định sẽ xuất hiện trong dữ liệu mới. Hãy luôn kiểm chứng các cụm từ bên ngoài: Các cụm có tương quan với các nhân đã biết không? Chúng có hợp lý với các chuyên gia lĩnh vực không? Chúng có ổn định trên những hạt giống ngẫu nhiên khác nhau không?

Tính ra các cụm thì dễ. Câu hỏi khó hơn là: *rồi thì sao?* Một kết quả phân cụm chỉ hữu ích khi nó dẫn đến những hành động khác nhau cho những cụm khác nhau. Các phân khúc khách hàng có ý nghĩa nếu bạn sẽ tiếp thị theo cách khác nhau cho từng phân khúc. Các nhóm bệnh nhân có ý nghĩa nếu bạn sẽ điều trị khác nhau cho từng nhóm. Nếu mọi cụm đều nhận cùng một can thiệp, thì việc phân cụm đã không giúp ích gì cho bạn.

Trước khi chạy một thuật toán phân cụm, hãy hỏi: “Nếu nó tạo ra ba nhóm, tôi sẽ làm gì khác đi cho từng nhóm?” Nếu bạn không trả lời được câu hỏi đó, thì việc phân cụm có lẽ chỉ là một bài tập kỹ thuật không có giá trị thực tế.

6.10 Tóm tắt

Điểm cần nhớ

Sự mơ hồ về mục tiêu: Khác với học có giám sát, phân cụm không có đáp án chuẩn. Những thuật toán, đặc trưng và tham số khác nhau tạo ra những cách phân cụm hợp lệ khác nhau.

Khoảng cách và chuẩn hóa quy mô: Lựa chọn khoảng cách quyết định “tương tự” nghĩa là gì. Những đặc trưng chưa chuẩn hóa sẽ ngấm ngấm chi phối các phép tính khoảng cách; hãy luôn chuẩn hóa trước khi phân cụm dựa trên khoảng cách. **K-Means:** Gán các điểm vào k cụm bằng cách tối thiểu hóa phương sai bên trong cụm. Nhanh nhưng đòi hỏi phải chọn k và giả định các cụm hình cầu. **Phân cụm phân cấp:** Xây dựng một dendrogram từ những cụm lồng nhau. Không cần chỉ định trước k , nhưng tốn kém về tính toán với tập dữ liệu lớn.

GMM: Phân cụm mềm bằng các phân phối Gaussian. Các điểm thuộc về cụm theo xác suất, không phải theo phép gán cứng.

PCA: Tìm những hướng trực giao có phương sai lớn nhất.^a Giữ gìn cấu trúc tuyến tính toàn cục; vài thành phần đầu tiên thường nắm bắt phần lớn thông tin.

t-SNE: Giữ gìn các hàng xóm cục bộ để trực quan hóa. Khoảng cách giữa các cụm không có ý nghĩa; chỉ dùng cho đồ thị 2D/3D. **Khả năng hành động:** Các cụm chỉ hữu ích khi chúng dẫn đến những quyết định khác nhau. Nếu mọi cụm đều nhận cùng một hành động, thì việc phân cụm đã không giúp ích gì.

^aXem Phụ lục F về phân tích giá trị riêng.

6.11 Bài tập luyện

1. Chạy K-means tính tay trên các điểm $\{(1, 1), (2, 1), (4, 3), (5, 4)\}$ với các trọng tâm khởi tạo $(1, 1)$ và $(5, 4)$. Hãy cho thấy từng vòng lặp.
2. Tính khoảng cách Euclid và Manhattan giữa $(2, 5, 1)$ và $(6, 2, 3)$.
3. Giải thích tại sao K-means có thể cho những kết quả khác nhau với những cách khởi tạo khác nhau. Những chiến lược nào làm giảm điều này?
4. Trong GMM, một điểm có những trách nhiệm $\gamma_1 = 0.8$, $\gamma_2 = 0.15$, $\gamma_3 = 0.05$. Điều này cho chúng ta biết gì so với K-means?
5. Phương pháp khuỷu tay cho thấy các độ méo: $k = 1$: 100, $k = 2$: 40, $k = 3$: 25, $k = 4$: 22, $k = 5$: 20. Bạn sẽ chọn k nào?
6. **(So sánh)** Với từng tình huống, phương pháp nào phù hợp nhất?
 - (a) Bạn muốn giảm 1000 đặc trưng xuống 50 cho một bộ phân loại.
 - (b) Bạn muốn trực quan hóa các phân khúc khách hàng ở 2D.
 - (c) Bạn không biết có bao nhiêu cụm tồn tại.
 - (d) Các cụm có kích thước khác nhau và hình dạng kéo dài.
7. Sau khi chạy PCA trên dữ liệu 50 chiều, 3 thành phần chính đầu tiên giải thích 85% phương sai.
 - (a) Mất bao nhiêu thông tin khi chiếu xuống 3D?
 - (b) Tại sao sự đánh đổi này lại có thể chấp nhận được?
8. Bạn thấy 3 cụm trong một đồ thị t-SNE. Đồng nghiệp của bạn nói “Cụm A gần B hơn là gần C.” Tại sao điều này có thể sai?
9. **(Đối kháng)** Hãy dựng một tập dữ liệu 2D mà K-means thất bại bất kể cách khởi tạo. Hãy dựng một tập khác mà phân cụm phân cấp liên kết đơn thất bại.
10. **(Kỹ thuật – Đạo đức)** Một thành phố dùng K-means trên dữ liệu cư dân để phân bổ dịch vụ xã hội. Một cụm có 80% thuộc về một nhóm sắc tộc.
 - (a) Đây có phải là bằng chứng của sự phân biệt đối xử không? Bạn cần biết những gì?
 - (b) Có nên thêm sắc tộc vào như một đặc trưng “để đa dạng” không?
 - (c) Một cư dân nói “thuật toán đã xếp tôi vào nhầm cụm.” Bạn sẽ đánh giá tuyên bố này như thế nào?



Phần III

Các phương pháp nâng cao

Chapter 7

Kernel Methods

Mathematics is the art of giving
the same name to different
things.

Henri Poincaré

7.1 Why Linear Models Fail: A Motivating Example

Before diving into kernel methods, let's see why we need them.

The XOR Problem: Linear Models Cannot Solve This

Consider four points in 2D:

x_1	x_2	Label
0	0	-1
0	1	+1
1	0	+1
1	1	-1

Try to draw a single straight line that separates the +1 points from the -1 points. You cannot, the classes are “interleaved.”

The problem: Linear classifiers can only draw straight decision boundaries. Some patterns require curved or more complex boundaries.

One solution: Add a new feature $x_3 = x_1 \cdot x_2$. Now the data becomes:

x_1	x_2	$x_3 = x_1 x_2$	Label
0	0	0	-1
0	1	0	+1
1	0	0	+1
1	1	1	-1

Trong không gian 3D này, các lớp tách tuyến tính được! Siêu phẳng $x_3 = 0.5$ (hoặc tương đương, $x_1 x_2 = 0.5$) tách hoàn toàn chúng.

Ý tưởng then chốt: Bằng cách ánh xạ dữ liệu sang không gian có chiều cao hơn với đúng các đặc trưng, các bài toán phi tuyến trở nên tuyến tính. Nhưng việc tính tường minh các đặc trưng đó có thể rất tốn kém hoặc không thể thực hiện nếu không gian đặc trưng có chiều rất cao.

Phương pháp hạt nhân (kernel) giải quyết điều này: Chúng cho phép làm việc trong các không gian đặc trưng cao chiều *mà không bao giờ tính tường minh các đặc trưng*.

Tại sao có hạt nhân (kernel) trong IAIO

Các phương pháp hạt nhân (kernel) xuất hiện trong IAIO vì chúng kiểm tra sự hiểu biết toán học sâu sắc:

Các dạng câu hỏi IAIO thường gặp:

- Cho một hạt nhân (kernel), xác định không gian đặc trưng ẩn
- Chứng minh một hàm đề xuất có phải là hạt nhân (kernel) hợp lệ hay không
- Phân tích cách các tham số hạt nhân (σ , bậc đa thức) ảnh hưởng đến hành vi của mô hình
- Nối các hạt nhân (kernel) với các khái niệm khác (chính quy hóa, SVM, đại số tuyến tính)

Điều bạn cần nắm vững:

- Thủ thuật hạt nhân (kernel trick): tính tích trong mà không cần đặc trưng tường minh
- Các tính chất của hạt nhân (kernel) hợp lệ (định tính dương bán xác định)
- Các hạt nhân (kernel) phổ biến: đa thức, RBF/Gaussian
- Liên hệ với tối ưu đối và định lý biểu diễn

Hướng dẫn suy luận: Phần nào không bắt buộc

Bắt buộc hiểu (có khả năng bị kiểm tra):

- Khái niệm thủ thuật hạt nhân (kernel) (Mục 7.4)
- Hành vi hạt nhân (kernel) đa thức và RBF (Mục 7.6)
- Các tính chất của hạt nhân (kernel) hợp lệ (Mục 7.7)

Hiểu ý tưởng, bỏ qua chi tiết suy luận:

- Giải pháp nguyên/đối của hồi quy ridge (Mục 7.3)
- Chứng minh định lý biểu diễn

Tùy chọn / bổ sung kiến thức:

- Liên hệ giữa phương pháp hạt nhân (kernel) và học sâu (Mục 7.10)
- Suy luận đầy đủ từ đối sang nguyên

7.2 Hàm tuyến tính và những giới hạn của chúng

Các mô hình học máy đơn giản nhất dùng **hàm tuyến tính**. Với một đầu vào được biểu diễn dưới dạng một vectơ các số $\mathbf{x} = (x_1, x_2, \dots, x_n)$, hàm tuyến tính tính tổng có trọng số:

$$g(\mathbf{x}) = w_1x_1 + w_2x_2 + \dots + w_nx_n = \langle \mathbf{w}, \mathbf{x} \rangle$$

Ở đây, $\mathbf{w} = (w_1, w_2, \dots, w_n)$ là một **vectơ trọng số** quyết định mức đóng góp của từng đặc trưng đầu vào vào dự đoán. Ký hiệu $\langle \mathbf{w}, \mathbf{x} \rangle$ biểu thị tích trong (hoặc tích vô hướng) giữa hai vectơ. Nhân các thành phần tương ứng rồi cộng chúng lại.

Học máy nghĩa là tìm vectơ trọng số $\mathbf{w}$ khiến hàm g cho ra các đầu ra gần với các nhãn mong muốn trên dữ liệu huấn luyện. Nếu chúng ta có m ví dụ huấn luyện $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)$, chúng ta muốn $g(\mathbf{x}_i) \approx y_i$ với mọi i .

Hàm tuyến tính có một diễn giải hình học quan trọng. Trong hai chiều, phương trình $w_1x_1 + w_2x_2 = c$ mô tả một đường thẳng. Trong ba chiều, nó mô tả một mặt phẳng. Trong các chiều cao hơn, nó mô tả một **siêu phẳng** — một mặt phẳng chia không gian thành hai vùng. Phân loại tuyến tính tìm một siêu phẳng tách các ví dụ thuộc các lớp khác nhau.

Giới hạn của hàm tuyến tính là nhiều mẫu hình thực tế không phải tuyến tính. Hãy tưởng tượng đang phân loại các điểm xếp thành hai vòng tròn đồng tâm: vòng trong thuộc lớp A, và vòng ngoài thuộc lớp B. Không đường thẳng nào có thể tách chúng. Hàm tuyến tính trong không gian đầu vào gốc đơn giản không thể biểu diễn ranh giới mà chúng ta cần.

Đây là lúc **không gian đặc trưng** trở nên thiết yếu. Ý tưởng then chốt là: ngay cả khi dữ liệu không thể tách bằng đường thẳng trong biểu diễn gốc của nó, nó có thể trở nên tách được nếu ta biến đổi nó sang một biểu diễn khác. Ví dụ, hai vòng tròn đồng tâm có thể tách được bằng cách xét khoảng cách đến gốc: các điểm gần tâm (khoảng cách nhỏ) là lớp A, các điểm xa hơn (khoảng cách lớn) là lớp B. Trong biểu diễn biến đổi này, bài toán trở nên tách tuyến tính được.

7.3 Hồi quy Ridge: Một ví dụ minh họa

Để làm các ý tưởng này cụ thể, hãy cùng làm bài **hồi quy ridge**, một thuật toán cơ bản minh họa một cách sinh động các khái niệm nền tảng của phương pháp hạt nhân (kernel).

Trong các bài toán hồi quy, các đầu ra y_i là các số thực chứ không phải hạng mục. Có thể chúng ta dự đoán giá nhà, lợi suất cổ phiếu, hoặc thời gian hồi phục của bệnh nhân.

. Để đo lường hàm của chúng ta hoạt động tốt thế nào, chúng ta dùng **hàm mất mát bình phương**: với mỗi ví dụ huấn luyện $(\mathbf{x}_i, y_i)$, sai số là $(g(\mathbf{x}_i) - y_i)^2$. Bình phương đảm bảo sai số luôn dương và phạt các lỗi lớn nặng hơn các lỗi nhỏ.

Tuy nhiên, việc đơn giản là cực tiểu hóa tổng sai số bình phương trên dữ liệu huấn luyện có thể dẫn đến **quá khớp** — mô hình ghi nhớ các ví dụ huấn luyện thay vì học các mẫu hình có thể khái quát hóa. Hồi quy ridge giải quyết điều này bằng cách thêm hình phạt cho các vectơ trọng số lớn:

$$\text{Minimize: } \sum_{i=1}^m (g(\mathbf{x}_i) - y_i)^2 + \lambda \|\mathbf{w}\|^2$$

Thước đo đầu tiên cho biết chúng ta khớp dữ liệu huấn luyện tốt thế nào. Thước đo thứ hai, $\lambda \|\mathbf{w}\|^2$, được gọi là **bộ điều chỉnh** (regularizer), nó phạt bình phương độ dài của vectơ trọng số. Tham số λ kiểm soát đánh đổi: λ càng lớn có nghĩa là chính quy hóa càng mạnh, dẫn đến trọng số nhỏ hơn nhưng có thể khớp dữ liệu huấn luyện kém hơn.

Tại sao lại phạt các trọng số lớn? Trực giác, trọng số lớn làm hàm rất nhạy cảm với những thay đổi nhỏ trong đầu vào. Một mô hình với trọng số khổng lồ có thể khớp hoàn hảo dữ liệu huấn luyện nhưng hành xử thất thường trên các ví dụ mới. Chính quy hóa giữ cho trọng số hợp lý, điều này thường cải thiện khả năng khái quát hóa.

7.3.1 Giải pháp nguyên

Có một giải pháp đóng rất đẹp cho hồi quy ridge. Nếu chúng ta sắp xếp các đầu vào huấn luyện thành các hàng của ma trận $\mathbf{X}$ (nên $\mathbf{X}$ là $m \times n$, trong đó m là số ví dụ và n là số đặc trưng), và sắp xếp các đầu ra thành một vectơ $\mathbf{y}$, vectơ trọng số tối ưu là:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_n)^{-1} \mathbf{X}^\top \mathbf{y}$$

Ở đây, $\mathbf{I}_n$ là ma trận đơn vị $n \times n$ (số 1 trên đường chéo, số 0 ở những nơi khác). Công thức này được gọi là **giải pháp nguyên** vì nó làm việc trực tiếp với vectơ trọng số $\mathbf{w}$.

Tính giải pháp nguyên yêu cầu nghịch một ma trận $n \times n$. Điều này ổn khi n nhỏ, nhưng trở nên tốn kém hoặc thậm chí không thể thực hiện khi n lớn. Điều gì xảy ra nếu chúng ta muốn làm việc trong không gian đặc trưng với hàng triệu hoặc hàng tỷ chiều?

7.3.2 Giải pháp đối

Đây là lúc điều kỳ diệu xảy ra. Hóa ra vectơ trọng số tối ưu luôn có thể viết dưới dạng tổ hợp tuyến tính của các đầu vào huấn luyện:

$$\mathbf{w} = \sum_{i=1}^m \alpha_i \mathbf{x}_i$$

với các hệ số $\alpha_1, \alpha_2, \dots, \alpha_m$ nào đó. Điều này được gọi là **định lý biểu diễn**, và nó có những hệ quả sâu sắc.

Thay vì chỉ định n trọng số (một cho mỗi chiều đầu vào), chúng ta chỉ định m hệ số (một cho mỗi ví dụ huấn luyện). Nếu chúng ta có ít ví dụ huấn luyện hơn số chiều đầu vào (điều này phổ biến trong thực tế) thì biểu diễn này cô đọng hơn. Quan trọng hơn, nó mở đường cho việc làm việc trong các không gian đặc trưng rất cao chiều.

Các hệ số có thể được tính bằng cách giải:

$$\alpha = (\mathbf{K} + \lambda \mathbf{I}_m)^{-1} \mathbf{y}$$

trong đó $\mathbf{K}$ là ma trận $m \times m$ **hạt nhân (kernel)**. Hàng mục ở hàng i và cột j của $\mathbf{K}$ là $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$, tích trong giữa ví dụ huấn luyện thứ i và thứ j .

Đây là **giải pháp đối**. Thay vì nghịch một ma trận $n \times n$, chúng ta nghịch một ma trận $m \times m$. Khi số đặc trưng lớn hơn nhiều so với số ví dụ huấn luyện, cách này hiệu quả hơn nhiều.

7.4 Thủ thuật hạt nhân (kernel trick)

Giải pháp đối tiết lộ một điều sâu sắc. Cả phép tính huấn luyện và phép dự đoán cho một đầu vào mới $\mathbf{x}$:

$$g(\mathbf{x}) = \sum_{i=1}^m \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle$$

phụ thuộc vào dữ liệu *chỉ thông qua các tích trong*. Chúng ta không bao giờ cần tọa độ thực sự của các vectơ, chỉ cần các tích trong của chúng với nhau.

Quan sát này cho phép **thủ thuật hạt nhân (kernel trick)**, một trong những ý tưởng tinh tế nhất của học máy. Giả sử chúng ta muốn làm việc trong không gian đặc trưng cao chiều hơn, trong đó mỗi đầu vào $\mathbf{x}$ được ánh xạ thành một vectơ đặc trưng $\phi(\mathbf{x})$. Thông thường, nếu không gian đặc trưng có hàng triệu hoặc hàng tỷ chiều, việc tính tường minh $\phi(\mathbf{x})$ sẽ tốn kém đến mức không thể chấp nhận được.

Nhưng nếu chúng ta có thể tính tích trong $\langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$ trực tiếp từ các đầu vào gốc $\mathbf{x}$ và $\mathbf{z}$, mà không bao giờ tính $\phi(\mathbf{x})$ một cách tường minh? Khi đó chúng ta có thể chạy thuật toán của mình trong không gian đặc trưng cao chiều với chi phí chỉ bằng việc làm việc trong không gian đầu vào gốc.

Một hàm $\kappa(\mathbf{x}, \mathbf{z})$ tính tích trong trong không gian đặc trưng nào đó được gọi là **hàm hạt nhân (kernel function)**:

$$\kappa(\mathbf{x}, \mathbf{z}) = \langle \phi(\mathbf{x}), \phi(\mathbf{z}) \rangle$$

Thủ thuật hạt nhân (kernel trick) cho phép chúng ta thực hiện các thuật toán tuyến tính trong các không gian đặc trưng có thể quá lớn để làm việc trực tiếp, kể cả các không gian vô hạn chiều!

7.5 Hạt nhân (kernel) bậc hai

Ví dụ đơn giản không tầm thường nhất là **hạt nhân (kernel) bậc hai**:

$$\kappa(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x}, \mathbf{z} \rangle^2$$

Chúng ta đơn giản tính tích trong thông thường và bình phương kết quả. Một phép nhân thêm, dường như là một thay đổi tầm thường. Thế nhưng điều này có những hệ quả đáng ngạc nhiên về những gì chúng ta có thể biểu diễn.

Để hiểu không gian đặc trưng tương ứng với điều gì, hãy khai triển bình phương. Với các đầu vào có hai thành phần, $\mathbf{x} = (x_1, x_2)$ và $\mathbf{z} = (z_1, z_2)$:

$$\begin{aligned} \langle \mathbf{x}, \mathbf{z} \rangle^2 &= (x_1 z_1 + x_2 z_2)^2 \\ &= x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2 \end{aligned}$$

Biểu thức này bằng tích trong giữa hai vectơ $(x_1^2, \sqrt{2}x_1 x_2, x_2^2)$ và $(z_1^2, \sqrt{2}z_1 z_2, z_2^2)$. Vì vậy hạt nhân (kernel) bậc hai tương ứng với một ánh xạ đặc trưng bao gồm tất cả các tích của các cặp đặc trưng đầu vào.

Nhìn chung, với các đầu vào n chiều, không gian đặc trưng có chiều n^2 — chúng ta đi từ n đặc trưng đến n^2 đặc trưng chỉ với một phép nhân thêm. Hãy xét các ảnh: một ảnh 32×32 điểm ảnh có $n = 1024$ chiều. Dùng hạt nhân (kernel) bậc hai, chúng ta ngằm làm việc trong một không gian có hơn một triệu chiều. Thế nhưng phép tính hạt nhân chỉ tốn công bằng việc tính một tích trong.

7.6 Hạt nhân (kernel) đa thức và Gaussian

Hạt nhân (kernel) bậc hai chỉ là khởi đầu. Vì tổng và tích của các hạt nhân (kernel) hợp lệ cũng là hạt nhân (kernel) hợp lệ, chúng ta có thể xây dựng các không gian đặc trưng ngày càng mạnh.

Hạt nhân (kernel) đa thức bậc d có dạng:

$$\kappa(\mathbf{x}, \mathbf{z}) = (c + \langle \mathbf{x}, \mathbf{z} \rangle)^d$$

trong đó $c \geq 0$ là một hằng số. Hạt nhân (kernel) đa thức tương ứng với một không gian đặc trưng chứa tất cả các tích của tối đa d đặc trưng đầu vào. d càng lớn có nghĩa là càng có thể nắm bắt các mẫu hình phức tạp hơn, nhưng đồng thời nguy cơ quá khớp cũng cao hơn.

Hạt nhân (kernel) đáng chú ý nhất là **hạt nhân (kernel) Gaussian**, còn gọi là hạt nhân (kernel) RBF (hàm cơ sở hướng tâm):

$$\kappa(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right)$$

Tham số σ kiểm soát độ “rộng” của hạt nhân (kernel) — mức hai điểm có thể xa nhau bao nhiêu mà vẫn được coi là tương tự. Các điểm gần nhau (theo độ lớn của σ) có giá trị hạt nhân gần 1; các điểm xa nhau có giá trị hạt nhân gần 0.

Hạt nhân (kernel) Gaussian tương ứng với một không gian đặc trưng *vô hạn chiều*. Nó có thể được viết dưới dạng tổng vô hạn các hạt nhân (kernel) đa thức, có nghĩa là vectơ đặc trưng bao gồm các tích của bất kỳ số lượng đặc trưng đầu vào nào. Bất chấp tính vô hạn chiều, chúng ta có thể tính giá trị hạt nhân bằng một công thức đơn giản liên quan đến một phép tính khoảng cách và một hàm mũ.

Trực giác, hạt nhân (kernel) Gaussian thực hiện một kiểu “tương tự cục bộ”: hai đầu vào tương tự trong không gian đặc trưng khi và chỉ khi chúng gần nhau trong không gian đầu vào gốc. Điều này cho phép hạt nhân (kernel) Gaussian tạo ra các ranh giới quyết định phức tạp tùy ý, thích ứng với bất kỳ mẫu hình nào tồn tại trong dữ liệu.

Chọn một hạt nhân (kernel)

Hạt nhân (kernel) tuyến tính ($\kappa(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x}, \mathbf{z} \rangle$): Luôn bắt đầu từ đây. Nếu dữ liệu đã (xấp xỉ) tách tuyến tính được, hạt nhân (kernel) tuyến tính nhanh, dễ diễn giải, và ít khi quá khớp. Nhiều bài toán phân loại văn bản và các bài toán cao chiều hoạt động tốt với hạt nhân (kernel) tuyến tính.

Hạt nhân (kernel) đa thức: Dùng khi bạn nghi ngờ các hiệu ứng tương tác giữa các đặc trưng là quan trọng (ví dụ, $x_1 \cdot x_2$ mang thông tin). Điều này nối thẳng với kỹ thuật tạo đặc trưng đa thức từ Chương 3, nhưng hạt nhân (kernel) tính các tương tác này một cách ngầm.

Hạt nhân (kernel) Gaussian/RBF: Lựa chọn mặc định khi bạn không có giả định mạnh nào về cấu trúc dữ liệu. Nó có thể xấp xỉ bất kỳ hàm liên tục nào với đủ dữ liệu. Rủi ro là quá khớp nếu σ quá nhỏ. **Hạt nhân**

(kernel) tùy chỉnh: Khi kiến thức miền gợi ý một khái niệm tương tự cụ thể (ví dụ: hạt nhân (kernel) chuỗi cho các chuỗi DNA, hạt nhân (kernel) đồ thị cho cấu trúc phân tử), việc thiết kế một hạt nhân (kernel) chuyên biệt cho miền có thể vượt trội hơn các lựa chọn tổng quát, đặc biệt với dữ liệu hạn chế.

7.7 Các tính chất của hạt nhân (kernel) hợp lệ

Không phải hàm nào cũng có thể là hạt nhân (kernel). Để $\kappa(\mathbf{x}, \mathbf{z})$ tương ứng với một tích trong không gian đặc trưng nào đó, nó phải thỏa mãn hai tính chất.

Đối xứng: hạt nhân (kernel) phải đối xứng theo các biến của nó

$$\kappa(\mathbf{x}, \mathbf{z}) = \kappa(\mathbf{z}, \mathbf{x})$$

Điều này trực tiếp xuất phát từ tính đối xứng của các tích trong: $\langle \mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{v}, \mathbf{u} \rangle$.

Định tính dương bán xác định: Với bất kỳ tập đầu vào $\mathbf{x}_1, \dots, \mathbf{x}_m$ nào và bất kỳ số thực $u_1, \dots, u_m$ nào:

$$\sum_{i=1}^m \sum_{j=1}^m u_i u_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \geq 0$$

Tương đương, ma trận hạt nhân (kernel) $\mathbf{K}$ (được tạo bằng cách đánh giá hạt nhân (kernel) trên mọi cặp đầu vào huấn luyện) phải có các trị riêng không âm.

Các tính chất này đảm bảo rằng κ thực sự tương ứng với một tích trong không gian đặc trưng nào đó, ngay cả khi chúng ta không thể viết không gian đó một cách tường minh. Về mặt toán học, chúng đảm bảo rằng bài toán tối ưu nền tảng của các phương pháp hạt nhân (kernel) có hành vi tốt.

May mắn thay, chúng ta hầu như không cần kiểm chứng các tính chất này từ đầu. Có các tính chất đóng (closure) hữu ích cho phép chúng ta xây dựng các hạt nhân (kernel) phức tạp từ các hạt nhân (kernel) đơn giản. Nếu $\kappa_1 + \kappa_2$, tích của chúng, và bất kỳ tích số vô hướng dương nào $c \cdot \kappa$ cũng là hạt nhân (kernel) hợp lệ. Bất kỳ đa thức nào có hệ số không âm áp dụng lên một hạt nhân (kernel) đều cho ra một hạt nhân (kernel) khác.

7.8 Vượt ra ngoài hồi quy: Các thuật toán hạt nhân (kernel) khác

Thuật hạt nhân (kernel trick) mở rộng rất xa so với hồi quy ridge. Bất kỳ thuật toán nào truy cập dữ liệu chỉ thông qua các tích trong đều có thể được “hạt nhân hóa” — sửa đổi để làm việc trong không gian đặc trưng do một hạt nhân (kernel) xác định.

Máy vectơ hỗ trợ (SVM) là các thuật toán phân loại tìm siêu phẳng cực đại hóa lề giữa các lớp. Lề là khoảng cách từ siêu phẳng đến các điểm huấn luyện gần nhất. Lề lớn thường tương quan với khả năng khái quát hóa tốt. Có một liên hệ toán học trực tiếp: lề bằng $2/\|\mathbf{w}\|$, nên cực đại hóa lề tương đương với cực tiểu hóa $\|\mathbf{w}\|^2$, đúng bằng hình phạt chính quy hóa từ hồi quy ridge. Chính quy hóa và cực đại hóa lề là hai cách nhìn của cùng một cơ chế. Trong SVM, tham số chính quy hóa C kiểm soát mức

vi phạm lề được dung thứ, đóng vai trò nghịch đảo của λ trong hồi quy ridge.

PCA hạt nhân (kernel) mở rộng phân tích thành phần chính (PCA) sang không gian đặc trưng. PCA thông thường tìm các hướng phương sai lớn nhất trong dữ liệu; PCA hạt nhân (kernel) tìm các hướng này trong không gian đặc trưng do hạt nhân (kernel) xác định. Điều này cho phép giảm chiều phi tuyến: dữ liệu không thể tách hoặc trực quan hóa tốt trong không gian gốc có thể trở nên tách được hoặc dễ diễn giải sau PCA hạt nhân (kernel).

Cảm nhận tử hạt nhân (kernel perceptron) là một thuật toán học trực tuyến cập nhật mỗi khi nó mắc lỗi. Thay vì duy trì các trọng số tường minh, nó duy trì các hệ số α_i biểu diễn vectơ trọng số như một tổ hợp các ví dụ huấn luyện. Điều này cho phép cảm nhận tử học các ranh giới quyết định phi tuyến.

7.9 Các phép toán hình học trong không gian đặc trưng

Làm việc trong không gian đặc trưng do hạt nhân (kernel) xác định, chúng ta có thể thực hiện các phép tính hình học mà không bao giờ phải xây dựng các vectơ đặc trưng một cách tường minh.

Khoảng cách giữa hai điểm trong không gian đặc trưng có thể được tính từ các phép đánh giá hạt nhân (kernel):

$$\|\phi(\mathbf{x}) - \phi(\mathbf{z})\|^2 = \kappa(\mathbf{x}, \mathbf{x}) - 2\kappa(\mathbf{x}, \mathbf{z}) + \kappa(\mathbf{z}, \mathbf{z})$$

Chúng ta khai triển chuẩn bình phương thành một tích trong, rồi thay hạt nhân (kernel) cho mỗi tích trong. Công thức này hoạt động bất kể không gian đặc trưng cao chiều đến mức nào.

Trung bình của dữ liệu huấn luyện trong không gian đặc trưng là:

$$\bar{\phi} = \frac{1}{m} \sum_{i=1}^m \phi(\mathbf{x}_i)$$

Chúng ta không thể tính $\bar{\phi}$ một cách tường minh nếu không gian đặc trưng vô hạn chiều. Tuy nhiên, chúng ta có thể tính các tích trong liên quan đến trung bình:

$$\langle \bar{\phi}, \phi(\mathbf{x}) \rangle = \frac{1}{m} \sum_{i=1}^m \kappa(\mathbf{x}_i, \mathbf{x})$$

Điều này đủ cho nhiều mục đích. Chúng ta thậm chí có thể **căn giữa** dữ liệu trong không gian đặc trưng (dịch gốc tọa độ về trung bình) bằng cách sửa đổi ma trận hạt nhân (kernel) một cách thích hợp.

7.10 Phương pháp hạt nhân (kernel) và học sâu

Các phương pháp hạt nhân (kernel) làm chủ học máy từ những năm 1990 cho đến khoảng năm 2012, khi học sâu bắt đầu đạt kết quả vượt trội trên các bài toán quy mô lớn. Hiểu mối quan hệ giữa các cách tiếp cận này mang lại góc nhìn quý giá.

Học sâu cung cấp sự linh hoạt lớn hơn trong các hàm mà nó có thể biểu diễn và có thể học các đặc trưng tự động từ dữ liệu thô. Với đủ dữ liệu và tính toán, các mạng nơ-ron sâu đạt được kết quả đáng chú ý trên hình ảnh, ngôn ngữ và các miền phức tạp khác.

Tuy nhiên, các phương pháp hạt nhân (kernel) giữ được những ưu điểm quan trọng trong một số tình huống.

Khi **dữ liệu huấn luyện hạn chế** — có thể chỉ vài trăm hoặc vài nghìn ví dụ thay vì hàng triệu — các phương pháp hạt nhân (kernel) thường vượt trội hơn học sâu. Các ứng dụng y tế thường xuyên đối mặt với ràng buộc này: các bệnh hiểm không tạo ra hàng triệu ví dụ huấn luyện.

Các phương pháp hạt nhân (kernel) tỏa sáng khi **kiến thức miền gợi ý một hạt nhân (kernel) phù hợp**. Nếu bạn hiểu cấu trúc của bài toán, bạn có thể thiết kế một hạt nhân (kernel) mã hóa cấu trúc đó, có khả năng đạt kết quả tốt hơn với ít dữ liệu hơn so với cách tiếp cận học sâu tổng quát.

Các phương pháp hạt nhân (kernel) có **nền tảng lý thuyết vững chắc**. Chúng ta hiểu chính xác lớp hàm mà chúng tìm kiếm và có thể chứng minh các bảo đảm về hành vi của chúng. Khả năng diễn giải này quan trọng trong các ứng dụng nơi chúng ta cần giải thích hoặc xác minh mô hình của mình. Các quyết định của một mạng nơ-ron sâu có thể khó hiểu, trong khi các dự đoán của phương pháp hạt nhân (kernel) phụ thuộc một cách minh bạch vào sự tương tự với các ví dụ huấn luyện.

7.10.1 Tại sao phương pháp hạt nhân (kernel) thất bại: Tường quy mô

Sự thanh lịch của thủ thuật hạt nhân (kernel trick) che giấu một vấn đề thực tế: các phương pháp hạt nhân (kernel) mở rộng kém với kích thước tập dữ liệu. Ma trận hạt nhân (kernel) K có m^2 hạng mục (một cho mỗi cặp ví dụ huấn luyện), và giải pháp đối yêu cầu nghịch ma trận $m \times m$ này với chi phí $O(m^3)$. Với $m = 10,000$ điểm huấn luyện, điều này có thể quản lý được. Với $m = 1,000,000$, riêng ma trận hạt nhân (kernel) đã yêu cầu lưu trữ 10^{12} hạng mục (khoảng 8 terabyte trong độ chính xác kép), và phép nghịch là không khả thi về mặt tính toán.

Hơn nữa, các phương pháp hạt nhân (kernel) mang tính **dựa trên bộ nhớ**: các dự đoán yêu cầu tính hạt nhân (kernel) giữa đầu vào mới và mọi ví dụ huấn luyện. Một mạng nơ-ron chỉ lưu các trọng số đã học; một phương pháp hạt nhân (kernel) phải giữ toàn bộ tập huấn luyện.

Học sâu tránh được cả hai vấn đề. Quá trình huấn luyện xử lý dữ liệu theo các lô nhỏ

với chi phí bộ nhớ độc lập với kích thước tập dữ liệu. Một mạng đã huấn luyện chỉ lưu các tham số của nó, không lưu dữ liệu huấn luyện. Và các mạng sâu học các biểu diễn đặc trưng tự động, trong khi các phương pháp hạt nhân (kernel) yêu cầu người thực hành chọn hạt nhân (kernel) đúng. Sự kết hợp giữa khả năng mở rộng và học đặc trưng tự động này chính là lý do học sâu thay thế các phương pháp hạt nhân (kernel) trong các bài toán quy mô lớn về thị giác, ngôn ngữ và các miền khác nơi dữ liệu dồi dào.

7.11 Đánh đổi thiên vị-phương sai

Một thách thức trung tâm trong toàn bộ học máy là cân bằng **thiếu khớp** và **quá khớp**. Đây là đánh đổi thiên vị-phương sai như ở Chương 5, nhưng các phương pháp hạt nhân (kernel) cho chúng ta hai nút kiểm soát riêng biệt.

Nếu không gian đặc trưng quá đơn giản (ví dụ: nếu chúng ta chỉ dùng hàm tuyến tính khi mẫu hình thực sự là phi tuyến) thì thuật toán không thể nắm bắt các mẫu hình liên quan. Đây là thiếu khớp, đôi khi gọi là thiên vị cao: mô hình đưa ra các giả định mạnh khiến nó không thể khớp dữ liệu tốt.

Nếu không gian đặc trưng quá phức tạp (ví dụ: nếu chúng ta dùng một hạt nhân (kernel) rất linh hoạt với σ nhỏ) thì thuật toán có thể ghi nhớ dữ liệu huấn luyện mà không học các mẫu hình có thể khái quát hóa. Đây là quá khớp, đôi khi gọi là phương sai cao: mô hình quá nhạy cảm với các ví dụ huấn luyện cụ thể, cho ra các kết quả khác nhau cho các mẫu ngẫu nhiên khác nhau từ cùng một phân phối.

Các phương pháp hạt nhân (kernel) giải quyết đánh đổi này thông qua hai cơ chế.

Tham số chính quy hóa λ kiểm soát mức chúng ta phạt các vectơ trọng số lớn. λ càng lớn thì ràng buộc mô hình càng mạnh, thực tế giảm dung lượng và bảo vệ chống quá khớp. λ càng nhỏ thì cho phép trọng số lớn hơn, tăng dung lượng nhưng có nguy cơ quá khớp.

Các tham số hạt nhân (kernel) (như σ cho hạt nhân (kernel) Gaussian) cũng ảnh hưởng đến đánh đổi. σ rất nhỏ tạo ra một mô hình cực kỳ linh hoạt có thể khớp gần như bất kỳ dữ liệu huấn luyện nào, nhưng có thể quá khớp. σ lớn hơn tạo ra một mô hình mượt mà hơn khái quát hóa tốt hơn nhưng có thể thiếu khớp với các mẫu hình phức tạp.

Chú ý liên hệ với Chương 3: ánh xạ đặc trưng ϕ đang tự động làm điều mà kỹ thuật tạo đặc trưng thủ công làm bằng tay. Chọn một hạt nhân (kernel) đa thức bậc 3 tương đương với việc tạo tất cả các đặc trưng tương tác bậc ba, nhưng không xây dựng chúng một cách tường minh. Việc chọn hạt nhân (kernel) bản thân nó đã là một dạng kỹ thuật tạo đặc trưng.

Tìm sự cân bằng phù hợp thường yêu cầu thực nghiệm: thử các cài đặt tham số khác nhau và đánh giá hiệu năng trên dữ liệu kiểm định giữ lại (sử dụng các kỹ thuật kiểm chéo từ Chương 5).

7.12 Một bài toán minh họa theo phong cách IAIO

Bài toán IAIO: Tính hợp lệ của hạt nhân (kernel) và các không gian đặc trưng

Xét hàm $\kappa(x, z) = (xz + 1)^2$ với $x, z \in \mathbb{R}$. **Phần (a):** Chứng minh đây là hạt nhân (kernel) hợp lệ bằng cách tìm một ánh xạ đặc trưng tương ứng ϕ sao cho $\kappa(x, z) = \langle \phi(x), \phi(z) \rangle$.

Phần (b): Chiều của không gian đặc trưng là bao nhiêu? **Phần (c):** Nếu chúng ta dùng hạt nhân (kernel) này trong hồi quy ridge với 100 điểm huấn luyện, ta nghịch ma trận kích thước nào trong công thức đối? Trong công thức nguyên?

Giải:

Phần (a): Khai triển $(xz + 1)^2 = x^2z^2 + 2xz + 1$.

Chúng ta có thể viết biểu thức này dưới dạng một tích trong: $\langle (x^2, \sqrt{2}x, 1), (z^2, \sqrt{2}z, 1) \rangle$.

So $\phi(x) = (x^2, \sqrt{2}x, 1)$ và $\kappa(x, z) = \phi(x)^\top \phi(z)$. ✓

Phần (b): Không gian đặc trưng có chiều 3 (ba thành phần trong ϕ).

7.13 Bài tập luyện

- Xét hạt nhân (kernel) đa thức bậc 2: $\kappa(\mathbf{x}, \mathbf{z}) = (1 + \mathbf{x}^T \mathbf{z})^2$
 - Với $\mathbf{x} = (1, 2)$ và $\mathbf{z} = (3, 1)$, tính $\kappa(\mathbf{x}, \mathbf{z})$.
 - Khai triển $(1 + x_1z_1 + x_2z_2)^2$ để thấy hạt nhân (kernel) này ngầm sử dụng không gian đặc trưng nào.
- Với hạt nhân (kernel) Gaussian (RBF) $\kappa(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|^2}{2\sigma^2}\right)$ với $\sigma = 1$:
 - Tính giá trị hạt nhân (kernel) giữa $\mathbf{x} = (0, 0)$ và $\mathbf{z} = (1, 1)$.
 - Giá trị hạt nhân (kernel) giữa một điểm và chính nó là bao nhiêu?
 - Khi $\sigma \rightarrow 0$, điều gì xảy ra với các giá trị hạt nhân (kernel) cho $\mathbf{x} \neq \mathbf{z}$? Điều này có ý nghĩa gì với mô hình?
- Một hàm κ là hạt nhân (kernel) hợp lệ khi và chỉ khi ma trận hạt nhân (kernel) $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$ là dương bán xác định với bất kỳ tập điểm nào.
 - Chứng minh rằng nếu κ_1 và κ_2 là các hạt nhân (kernel) hợp lệ, thì $\kappa(\mathbf{x}, \mathbf{z}) = \kappa_1(\mathbf{x}, \mathbf{z}) + \kappa_2(\mathbf{x}, \mathbf{z})$ cũng là hạt nhân (kernel) hợp lệ.
 - $\kappa(\mathbf{x}, \mathbf{z}) = |\mathbf{x}^T \mathbf{z}|$ có phải là hạt nhân (kernel) hợp lệ không? Giải thích câu trả lời của bạn.

4. Trong hồi quy ridge với công thức đối, dự đoán cho một điểm mới $\mathbf{x}$ là:

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i \kappa(\mathbf{x}_i, \mathbf{x})$$

trong đó α_i được học từ dữ liệu huấn luyện.

(a) Giải thích bằng lời công thức này đang tính điều gì.

(b) Tại sao điều này trở nên có lợi về mặt tính toán khi chiều đặc trưng d lớn hơn nhiều so với số điểm huấn luyện n ?

5. Xét một tập dữ liệu 1D: $\{(-1, -1), (0, 1), (1, -1)\}$ trong đó mỗi cặp là (x, y) .

(a) Liệu dữ liệu này có thể được một hàm tuyến tính khớp hoàn hảo không? Tại sao có hoặc tại sao không?

(b) Mô tả một ánh xạ đặc trưng $\phi(x)$ sẽ khiến dữ liệu này tách tuyến tính được.

(c) Hạt nhân (kernel) nào tương ứng với ánh xạ đặc trưng của bạn?

6. Tham số chính quy hóa λ trong hồi quy ridge kiểm soát độ phức tạp của mô hình.

(a) Điều gì xảy ra với mô hình khi $\lambda \rightarrow \infty$?

(b) Điều gì xảy ra khi $\lambda \rightarrow 0$?

(c) Bạn sẽ chọn giá trị phù hợp của λ trong thực tế như thế nào?

7. Giải thích tại sao các phương pháp hạt nhân (kernel) đôi khi được gọi là các phương pháp “dựa trên bộ nhớ”. Ưu điểm và nhược điểm của tính chất này so với các phương pháp như mạng nơ-ron học các tham số trọng số tường minh là gì?

8. (Chỉ hình học. Không cần đại số)

Xét các điểm dữ liệu trong 2D xếp thành hai vòng tròn đồng tâm: Lớp A tạo thành vòng tròn trong (bán kính 1), Lớp B tạo thành vòng tròn ngoài (bán kính 2).

(a) Phác thảo dữ liệu này. Liệu có đường thẳng nào tách được các lớp không? Giải thích theo hình học.

(b) Nếu bạn thêm chiều thứ ba $x_3 = x_1^2 + x_2^2$ (khoảng cách bình phương đến gốc), mô tả dữ liệu trông như thế nào trong 3D. Liệu mặt phẳng có thể tách các lớp bây giờ không?

((c)) Hạt nhân (kernel) RBF $\kappa(\mathbf{x}, \mathbf{z}) = \exp(-\|\mathbf{x} - \mathbf{z}\|^2 / 2\sigma^2)$ ngàm ánh xạ sang một không gian vô hạn chiều. Không dùng bất kỳ đại số nào, giải thích trực giác tại sao hạt nhân (kernel) này có thể tách hai vòng tròn đồng tâm.

(d) Khi $\sigma \rightarrow 0$, hạt nhân (kernel) RBF tiến đến tình huống mà mỗi điểm huấn luyện chỉ "nhìn thấy" chính nó. Điều này có ý nghĩa gì với ranh giới quyết định? Nó tốt hay xấu cho khả năng khái quát hóa?

9. **(Tư duy phản biện)** Một đồng nghiệp tuyên bố đã phát minh ra một hạt nhân (kernel) mới: $\kappa(x, z) = \min(x, z)$ với $x, z \geq 0$.

(a) Đây có phải là hạt nhân (kernel) hợp lệ không? (Gợi ý: kiểm tra ma trận hạt nhân (kernel) có dương bán xác định với các điểm $\{1, 2, 3\}$ hay không.)

(b) Nếu hợp lệ, nó tương ứng với không gian đặc trưng nào? Nếu không, điều gì sai khi bạn cố gắng sử dụng nó?

Chương 8

Tìm kiếm trong AI và Thỏa mãn ràng buộc

Một bài toán được phát biểu tốt là
một bài toán đã giải được một
nửa.

George Pólya

8.1 Bản chất của các bài toán tìm kiếm

Hãy tưởng tượng bạn đang chơi một trò chơi điện tử, nơi bạn phải đưa một nhân vật đi qua một mê cung để tìm kho báu. Ở mỗi bước, bạn có thể đi lên, đi xuống, sang trái hoặc sang phải. Có những đường đi dẫn tới ngõ cụt, những đường khác quay vòng lại chính nó, và chỉ một ít đường dẫn tới kho báu. Làm sao để tìm ra đường đi tốt nhất — đường đi đến được kho báu với số nước đi ít nhất hoặc tránh được nhiều nguy hiểm nhất?

Kịch bản đơn giản này nắm bắt đúng bản chất của thứ mà các nhà khoa học máy tính gọi là một **bài toán tìm kiếm**. Xét bản chất, một bài toán tìm kiếm là việc tìm ra một dãy hành động biến đổi một tình huống ban đầu thành một tình huống đích mong muốn. Thách thức không chỉ nằm ở việc tìm ra *bất kỳ* lời giải nào, mà là tìm ra một lời giải *tốt*: một lời giải ngắn, rẻ, nhanh, hoặc tối ưu theo bất kỳ tiêu chí nào quan trọng với bài toán đang xét.

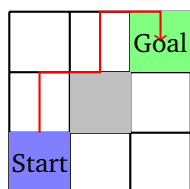
Các bài toán tìm kiếm xuất hiện khắp nơi trong trí tuệ nhân tạo và khoa học máy tính. Khi GPS của bạn tính ra một đường đi từ nhà bạn đến một nhà hàng, nó đang giải một bài toán tìm kiếm: tìm ra một dãy các con đường nối vị trí xuất phát của bạn với đích đến, đồng thời tối thiểu hóa thời gian hoặc quãng đường. Khi một chương trình cờ vua quyết định đi nước nào, nó đang tìm qua các thế cờ tương lai có thể có để tìm một nước đi dẫn tới thắng lợi. Khi một con robot lập kế hoạch đi qua một tòa nhà, nó tìm một đường đi tránh

chương vật và đến được mục tiêu của nó.

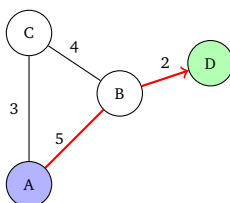
Bài toán tìm kiếm là gì?

Một **bài toán tìm kiếm** là bất kỳ bài toán nào mà bạn cần tìm một dãy các bước (một đường đi) đưa từ điểm xuất phát đến một đích. Thách thức là tìm ra đường đi đáp ứng tốt nhất các tiêu chí của bạn, dù đó là đường ngắn nhất, đường rẻ nhất, hay chỉ đơn giản là bất kỳ đường đi nào dùng được.

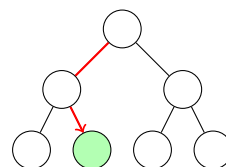
Điều làm cho các bài toán tìm kiếm thú vị là chúng thường quá lớn để giải bằng cách thử mọi khả năng. Một ván cờ vua có thể diễn ra theo nhiều cách hơn số nguyên tử trong vũ trụ. Ngay cả một mê cung đơn giản cũng có thể có hàng triệu đường đi. **Sự bùng nổ không gian trạng thái** này — sự tăng trưởng theo cấp số nhân của số khả năng khi kích thước bài toán tăng — là lý do trung tâm khiến việc tìm kiếm trở nên khó. Nghệ thuật của việc tìm kiếm trong AI nằm ở chỗ khám phá không gian khả năng một cách thông minh, tìm ra lời giải tốt mà không phải xét kỹ từng phương án.



Robot Navigation



GPS Navigation



Game Playing

Hình 8.1: Ba ví dụ về các bài toán tìm kiếm: điều khiển một robot qua mê cung, tìm đường lái xe giữa các thành phố, và khám phá cây quyết định của trò chơi.

8.2 Biểu diễn không gian trạng thái

Trước khi máy tính có thể tìm kiếm lời giải, chúng ta cần mô tả bài toán theo cách mà máy tính có thể hiểu và thao tác được. Mô tả toán học này được gọi là **biểu diễn không gian trạng thái**, và nó biến khái niệm trực quan về “tìm một đường đi” thành một khuôn khổ tính toán chính xác.

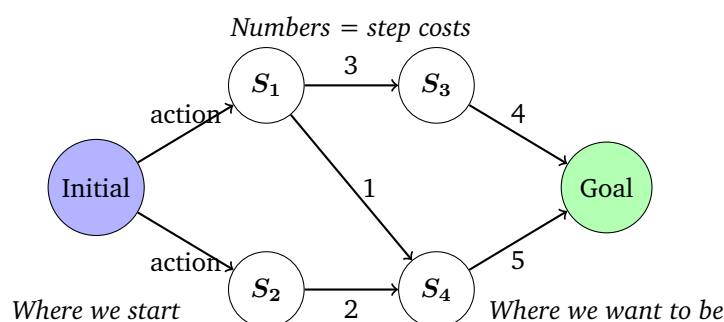
8.2.1 Suy nghĩ theo trạng thái và hành động

Ý chính là hãy hình dung thế giới tồn tại ở các **trạng thái** khác nhau, và hình dung việc giải quyết bài toán là việc chuyển từ trạng thái này sang trạng thái khác thông qua các **hành động**. Một trạng thái là mô tả đầy đủ về tình huống hiện tại, tức là mọi thứ mà thuật toán cần biết để tiếp tục giải bài toán. Một hành động là thứ làm thay đổi trạng thái đó.

Hãy xét ví dụ về việc di chuyển giữa các thành phố. Mỗi thành phố đại diện cho một

trạng thái: hoặc bạn đang ở Arad, hoặc ở Bucharest, hoặc ở Sibiu, và v.v. Các hành động là những con đường nối các thành phố: từ Arad, bạn có thể lái xe đến Sibiu, hoặc đến Timisoara, hoặc đến Zerind. Mỗi hành động đưa bạn từ một trạng thái này sang một trạng thái khác.

Cách nghĩ này biến bài toán thành một đồ thị, trong đó các nút đại diện cho các trạng thái và các cạnh đại diện cho các hành động. Tìm ra một lời giải trở thành tương đương với việc tìm một đường đi qua đồ thị này từ nút xuất phát đến một nút đích.



Hình 8.2: Một không gian trạng thái được biểu diễn như một đồ thị. Các nút là các trạng thái, các cạnh là các hành động, và các con số chỉ ra chi phí của từng hành động.

8.2.2 Các thành phần của một bài toán tìm kiếm

Một đặc tả bài toán tìm kiếm đầy đủ gồm nhiều thành phần. **Trạng thái** là đơn vị nền tảng: một mô tả về một cấu hình cụ thể của thế giới. Với ví dụ điều hướng giữa các thành phố, một trạng thái đơn giản là thành phố bạn đang ở. Với một ván cờ vua, một trạng thái sẽ là vị trí đầy đủ của tất cả các quân trên bàn cờ.

Trạng thái ban đầu là nơi việc tìm kiếm bắt đầu. Đây là điểm xuất phát của bạn: Arad trong ví dụ điều hướng, hay thế cờ khai cuộc trong cờ vua.

Trạng thái đích (hoặc các trạng thái đích) là thứ ta muốn đạt tới. Đôi khi có đúng một đích, chẳng hạn như việc đến Bucharest. Những lúc khác, nhiều trạng thái đều thỏa mãn là đích. Trong cờ vua, bất kỳ thế cờ chiếu hết nào cũng là một đích.

Các **hành động** hay các **toán tử** là những cách ta có thể chuyển từ trạng thái này sang trạng thái khác. Từ một thành phố xác định, các hành động khả dụng là lái xe đến các thành phố lân cận. Trong cờ vua, các hành động là mọi nước đi hợp lệ từ thế cờ hiện tại.

Mô hình chuyển trạng thái đặc tả điều gì xảy ra khi ta thực hiện một hành động trong một trạng thái. Cho biết một trạng thái và một hành động, mô hình chuyển trạng thái cho ta biết trạng thái kết quả (gọi là **trạng thái kế nhiên** hay **trạng thái con**). Nếu bạn đang ở Arad và thực hiện hành động "lái xe đến Sibiu", mô hình chuyển trạng thái cho bạn biết bạn sẽ ở Sibiu.

Chi phí bước gán một giá trị số cho mỗi hành động. Đó có thể là quãng đường đã đi, thời gian cần thiết, lượng nhiên liệu tiêu thụ, hay bất kỳ đại lượng nào khác đo lường công sức. Trong ví dụ các thành phố của chúng ta, chi phí bước là quãng đường giữa các thành phố. Nếu mọi hành động cùng tốn kém như nhau, ta có thể đơn giản gán cho mỗi hành động chi phí bằng 1.

Một **đường đi** là một dãy các trạng thái được nối bằng các hành động, dẫn từ trạng thái này sang trạng thái khác. **Chi phí đường đi** là tổng của tất cả các chi phí bước dọc theo đường đi đó. Mục tiêu của ta thường là tìm một đường đi từ trạng thái ban đầu đến một trạng thái đích sao cho tổng chi phí đường đi nhỏ nhất.

Một bài toán tìm kiếm gồm nhiều thành phần. **Trạng thái** là mô tả đầy đủ về một cấu hình cụ thể của thế giới. **Trạng thái ban đầu** là cấu hình xuất phát nơi việc tìm kiếm bắt đầu, còn **trạng thái đích** là cấu hình (hoặc tập các cấu hình) ta muốn đạt tới. **Hành động** là các thao tác biến một trạng thái thành một trạng thái khác, và **mô hình chuyển trạng thái** là hàm xác định trạng thái kết quả khi một hành động được áp dụng. Cuối cùng, **chi phí bước** là chi phí gán với mỗi hành động, và **chi phí đường đi** là tổng chi phí của một dãy hành động (tức là tổng các chi phí bước).

8.2.3 Một ví dụ cụ thể: Bản đồ Romania

Một ví dụ kinh điển được dùng trong các giáo trình AI liên quan đến việc tìm đường đi xuyên qua Romania. Giả sử bạn đang ở thành phố Arad và muốn đến Bucharest, thủ đô.

Trong bài toán này, các trạng thái là các thành phố của Romania: Arad, Sibiu, Fagaras, Pitesti, Bucharest, và những thành phố khác. Trạng thái ban đầu là Arad, và trạng thái đích là Bucharest. Các hành động là việc lái xe theo những con đường nối giữa các thành phố lân cận. Các chi phí bước là quãng đường giữa các thành phố. Chẳng hạn, con đường từ Arad đến Sibiu dài 140 ki lô-mét.

Máy tính biểu diễn thông tin này như thế nào? Cách tiếp cận phổ biến nhất dùng một **ma trận kề**, một mảng hai chiều trong đó hàng và cột đại diện cho các thành phố. Giá trị tại hàng i , cột j là khoảng cách từ thành phố i đến thành phố j . Nếu hai thành phố không được nối trực tiếp bằng một con đường, ta biểu diễn điều đó bằng vô cùng (∞) hoặc một giá trị đặc biệt chỉ ra là không có kết nối.

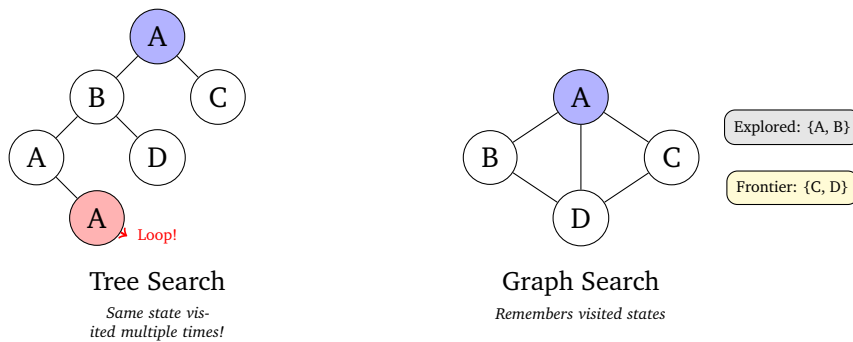
Biểu diễn này cho phép máy tính nhanh chóng tra cứu xem hai thành phố có được kết nối hay không và chi phí đi lại giữa chúng là bao nhiêu. Từ cấu trúc dữ liệu này, thuật toán tìm kiếm có thể một cách có hệ thống khám phá các đường đi từ Arad đến Bucharest, và cuối cùng tìm ra đường đi ngắn nhất.

8.2.4 Tìm kiếm trên cây so với tìm kiếm trên đồ thị

Khi khám phá không gian trạng thái, ta phải đối diện với một lựa chọn quan trọng về cách xử lý các trạng thái đã từng được ghé thăm.

Trong **tìm kiếm trên cây**, ta coi mỗi đường đi là độc lập và không ghi nhớ những trạng thái nào đã ghé thăm trước đó. Điều này có thể gây ra vấn đề: nếu đồ thị chứa các chu trình, ta có thể ghé thăm lặp lại cùng một trạng thái, và có nguy cơ mắc kẹt trong một vòng lặp vô hạn. Hãy hình dung một mạng lưới đường bộ nơi Arad nối với Sibiu, và Sibiu nối ngược lại với Arad. Một thuật toán tìm kiếm trên cây ngây thơ có thể luân chuyển vô tận giữa hai thành phố này.

Tìm kiếm trên đồ thị giải quyết vấn đề này bằng cách duy trì hai cấu trúc dữ liệu. **Biên** (còn gọi là **danh sách mở**) chứa các trạng thái đã được phát hiện nhưng chưa được khám phá hết. **Tập đã khám phá** (còn gọi là **danh sách đóng**) chứa các trạng thái đã được ghé thăm và xử lý xong. Trước khi khám phá một trạng thái, thuật toán kiểm tra xem nó đã từng được ghé thăm chưa. Nếu có, trạng thái đó bị bỏ qua.



Hình 8.3: Tìm kiếm trên cây có thể ghé thăm lại cùng một trạng thái nhiều lần, và có thể gây ra vòng lặp vô hạn. Tìm kiếm trên đồ thị duy trì một tập đã khám phá để tránh vấn đề này.

Tìm kiếm trên đồ thị dùng nhiều bộ nhớ hơn (để lưu tập đã khám phá) nhưng bảo đảm sẽ kết thúc trên các đồ thị hữu hạn. Việc chọn giữa tìm kiếm trên cây và trên đồ thị phụ thuộc vào bài toán cụ thể: nếu không gian trạng thái là một cây không có trạng thái lặp lại thì tìm kiếm trên cây là đủ; nếu có thể xuất hiện trạng thái lặp lại thì thường phải dùng tìm kiếm trên đồ thị.

8.3 Đo lường hiệu năng của thuật toán

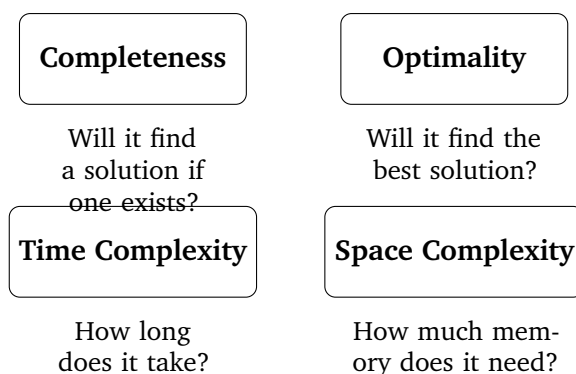
Khi có nhiều thuật toán tìm kiếm để lựa chọn, làm sao quyết định dùng thuật toán nào? Các nhà khoa học máy tính đánh giá thuật toán tìm kiếm bằng bốn tiêu chí nền tảng.

Tính đầy đủ hỏi rằng liệu thuật toán có được bảo đảm tìm ra một lời giải nếu lời giải tồn tại hay không. Một thuật toán không đầy đủ có thể không tìm được lời giải dù lời giải có sẵn, có lẽ vì nó mắc kẹt trong một vòng lặp hoặc vô tận khám phá sai phần của không gian tìm kiếm.

Tính tối ưu hỏi rằng liệu thuật toán có tìm ra lời giải tốt nhất hay không. Tìm được *một* lời giải thường chưa đủ; ta muốn đường đi ngắn nhất, đường rẻ nhất, hay kế hoạch nhanh nhất. Một thuật toán tối ưu bảo đảm rằng không tồn tại lời giải nào tốt hơn.

Độ phức tạp thời gian đo lường thuật toán mất bao lâu để tìm ra một lời giải. Thông thường ta diễn đạt điều này như một hàm của kích thước bài toán, dùng ký hiệu Big-O. Một thuật toán có độ phức tạp thời gian theo cấp số nhân sẽ không khả thi với các bài toán lớn.

Độ phức tạp không gian đo lường thuật toán cần bao nhiêu bộ nhớ. Có những thuật toán ghi nhớ mọi trạng thái từng gặp, trong khi những thuật toán khác loại bỏ các trạng thái sau khi xử lý xong. Trong thực tế, ràng buộc bộ nhớ có thể hạn chế nghiêm trọng không kém ràng buộc thời gian.



Hình 8.4: Bốn tiêu chí để đánh giá hiệu năng của thuật toán tìm kiếm.

Để phân tích độ phức tạp, ta cần mô tả kích thước của không gian tìm kiếm. Có ba tham số then chốt thường được sử dụng. **Hệ số phân nhánh b** là số trạng thái kế tiếp (con) nhiều nhất mà một trạng thái bất kỳ có thể có. Trong ví dụ điều hướng giữa các thành phố của chúng ta, nếu mỗi thành phố nối với nhiều nhất bốn thành phố khác, thì $b = 4$. **Độ sâu của lời giải d** là số bước trong đường đi của lời giải ngắn nhất. **Độ sâu tối đa m** là đường đi dài nhất trong không gian tìm kiếm, và có thể vô hạn nếu có các chu trình.

Những tham số này cho phép ta diễn đạt chính xác độ phức tạp. Một thuật toán có độ phức tạp thời gian $O(b^d)$ xét một số trạng thái tăng theo cấp số nhân cùng với độ sâu của lời giải, nhân đôi d bình phương số trạng thái được xét.

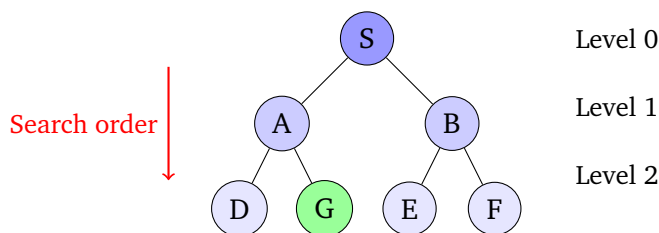
8.4 Các chiến lược tìm kiếm không có thông tin

Các thuật toán **tìm kiếm không có thông tin** (còn gọi là **tìm kiếm mù**) không có kiến thức gì về nơi đích có thể nằm ở đâu. Chúng không thể biết mình đang “nóng hơn” hay “lạnh hơn” trong khi tìm kiếm. Tất cả chúng có thể làm chỉ là khám phá không gian trạng thái một cách có hệ thống theo một chiến lược cố định.

Mặc dù có hạn chế đó, các thuật toán tìm kiếm không có thông tin vẫn rất quan trọng. Chúng cung cấp những lời giải cơ sở áp dụng cho mọi bài toán, chúng giúp ta hiểu cấu trúc nền tảng của việc tìm kiếm, và chúng là nền móng để xây dựng nên những thuật toán tinh vi hơn.

8.4.1 Tìm kiếm theo chiều rộng (BFS)

Tìm kiếm theo chiều rộng (BFS) khám phá không gian tìm kiếm từng tầng một. Nó trước tiên xét tất cả các trạng thái cách điểm xuất phát một bước, rồi tất cả các trạng thái cách hai bước, rồi ba bước, và cứ tiếp như vậy. Thuật toán tiến ra từ điểm xuất phát giống như những gợn sóng lan ra từ một viên sỏi rơi xuống mặt hồ.



Hình 8.5: Tìm kiếm theo chiều rộng khám phá cây từng tầng một. Bắt đầu từ S, nó xét A và B ở tầng 1, rồi D, G, E và F ở tầng 2. Đích G được tìm thấy ở tầng 2.

Chìa khoá của BFS là việc dùng một cấu trúc dữ liệu **hàng đợi**, theo nguyên tắc Vào trước–Ra sau (FIFO). Khi phát hiện các trạng thái mới, ta thêm chúng vào cuối hàng đợi. Khi cần một trạng thái để khám phá, ta lấy một trạng thái ra khỏi đầu hàng đợi. Điều này bảo đảm rằng ta xử lý các trạng thái theo đúng thứ tự chúng được phát hiện: mọi trạng thái ở độ sâu d đều được xử lý trước bất kỳ trạng thái nào ở độ sâu $d + 1$.

Thuật toán BFS tiến hành như sau. Ta bắt đầu bằng cách đặt trạng thái ban đầu vào một hàng đợi rỗng. Sau đó ta vào một vòng lặp: lấy trạng thái ở đầu hàng đợi ra, kiểm tra xem nó có phải là đích hay không (nếu phải thì ta xong), và nếu không thì sinh ra tất cả các trạng thái kế nhiệm của nó rồi thêm chúng vào cuối hàng đợi. Quá trình này tiếp tục cho đến khi ta tìm ra một đích hoặc hàng đợi trở nên rỗng (tức là không tồn tại lời giải).

Thuật toán Tìm kiếm theo chiều rộng

1. Khởi tạo một hàng đợi chỉ chứa trạng thái ban đầu.
2. Lấy trạng thái ở đầu hàng đợi ra.
3. Nếu trạng thái này là đích, trả về thành công cùng đường đi để tới đó.
4. Nếu không, sinh ra tất cả các trạng thái kế nhiên và thêm chúng vào cuối hàng đợi.
5. Lặp lại từ bước 2 cho tới khi tìm thấy đích hoặc hàng đợi rỗng.

BFS có những tính chất lý thuyết hấp dẫn. Nó **đầy đủ**: nếu một lời giải tồn tại ở độ sâu hữu hạn, BFS sẽ tìm thấy nó, vì nó có hệ thống xét tất cả các trạng thái ở mỗi độ sâu trước khi đi sâu hơn. Nó cũng **tối ưu** khi mọi hành động cùng có chi phí bằng nhau, vì nó tìm thấy đích nông nhất trước, và đó cũng chính là đích có ít bước nhất.

Tuy nhiên, BFS có một hạn chế thực tế nghiêm trọng: nó đòi hỏi một lượng bộ nhớ khổng lồ. Thuật toán phải lưu đồng thời tất cả các trạng thái ở tầng hiện tại và ở tầng kế tiếp. Nếu hệ số phân nhánh là b và lời giải nằm ở độ sâu d , BFS xét và lưu $O(b^d)$ trạng thái. Sự tăng trưởng theo cấp số nhân này nhanh chóng trở nên không thể quản lý. Với $b = 10$ và $d = 16$, thuật toán sẽ phải lưu khoảng 10^{16} trạng thái, nhiều hơn toàn bộ bộ nhớ máy tính trên Trái Đất.

8.4.2 Tìm kiếm theo chi phí đồng nhất (UCS)

Nếu các hành động khác nhau có chi phí khác nhau thì sao? Việc lái xe từ Arad đến Sibiu có thể là 140 ki lô-mét, trong khi việc lái xe từ Arad đến Zerind chỉ là 75 ki lô-mét. BFS sẽ khám phá chúng như nhau, nhưng rõ ràng đường đi ngắn hơn thì đáng ưu hơn.

Tìm kiếm theo chi phí đồng nhất (UCS) giải quyết điều này bằng cách luôn mở rộng trạng thái có tổng chi phí đường đi tính từ điểm xuất phát là nhỏ nhất. Thay vì một hàng đợi đơn giản, UCS dùng một **hàng đợi ưu tiên** trong đó các trạng thái được sắp xếp theo chi phí tích lũy. Trạng thái mà tốn ít công sức tổng cộng nhất để tới sẽ luôn được mở rộng tiếp theo.

Đại lượng then chốt trong UCS là $g(n)$, tức chi phí của đường đi từ trạng thái ban đầu tới trạng thái n . Khi ta sinh ra một trạng thái kế nhiên, ta tính giá trị g của nó bằng cách cộng chi phí bước vào giá trị g của trạng thái cha. Hàng đợi ưu tiên giữ các trạng thái được sắp theo những giá trị g này.

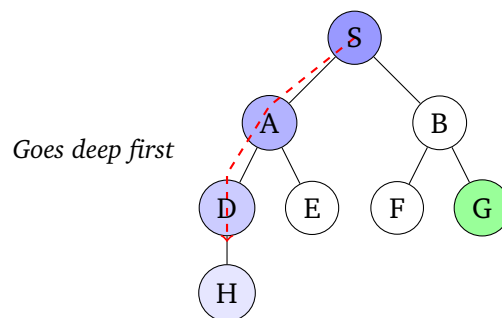
Có một điểm tinh tế phân biệt UCS với BFS: ta kiểm tra một trạng thái có phải là đích hay không khi ta *lấy* nó ra khỏi hàng đợi, chứ không phải khi ta *thêm* nó vào. Điều này quan trọng vì một trạng thái có thể được tới bằng nhiều đường đi với chi phí khác nhau. Ta muốn chắc chắn rằng mình đã tìm được đường đi rẻ nhất trước khi tuyên bố thành công.

UCS vừa đầy đủ vừa tối ưu, với điều kiện mọi chi phí bước đều dương. Nó

luôn tìm ra đường đi rẻ nhất tới đích vì nó khám phá các trạng thái theo thứ tự tăng dần của chi phí đường đi. Tuy nhiên, sự triệt để này đánh đổi bằng một cái giá: UCS có thể khám phá nhiều trạng thái rẻ để tới nhưng lại ở xa đích, trong khi bỏ qua những trạng thái đắt hơn nhưng gần đích hơn nhiều.

8.4.3 Tìm kiếm theo chiều sâu (DFS)

Tìm kiếm theo chiều sâu (DFS) thực hiện cách tiếp cận ngược lại với BFS: thay vì khám phá theo chiều rộng, nó lao sâu vào cây tìm kiếm một cách tối đa trước khi quay lui. Từ trạng thái ban đầu, DFS đi theo một đường tới tận ngõ cụt hoặc đích, rồi lùi lại nhánh gần nhất chưa được khám phá và thử nhánh đó.



Hình 8.6: Tìm kiếm theo chiều sâu khám phá trọn vẹn một nhánh trước khi thử các nhánh khác. Từ S, nó đi tới A, rồi D, rồi H, trước khi quay lui để thử các nhánh khác.

DFS dùng một cấu trúc dữ liệu **ngăn xếp**, theo nguyên tắc Vào sau–Ra trước (LIFO). Khi phát hiện các trạng thái mới, ta đẩy chúng vào ngăn xếp. Khi cần một trạng thái để khám phá, ta lấy một trạng thái ra khỏi đỉnh ngăn xếp. Điều này có nghĩa là trạng thái mới nhất được phát hiện luôn được khám phá tiếp theo, khiến việc tìm kiếm đi sâu hơn chứ không phải rộng hơn.

Ưu điểm lớn của DFS là khả năng sử dụng bộ nhớ hiệu quả. Một khi thuật toán khám phá xong một nhánh, nó có thể xóa hẳn nhánh đó khỏi bộ nhớ. Chỉ đường đi hiện tại từ gốc tới trạng thái hiện tại là cần được lưu. Nếu độ sâu tối đa là m và hệ số phân nhánh là b , DFS chỉ cần $O(bm)$ bộ nhớ. Đây là số lượng tuyến tính theo độ sâu, so với yêu cầu bộ nhớ tăng theo cấp số nhân của BFS.

So sánh bộ nhớ: BFS và DFS

Xét một bài toán tìm kiếm có hệ số phân nhánh $b = 10$ và độ sâu lời giải $d = 16$.

BFS sẽ cần lưu khoảng 10^{16} trạng thái, đòi hỏi khoảng 10 exabyte bộ nhớ — nhiều hơn toàn bộ dung lượng lưu trữ số trên Trái Đất. DFS sẽ chỉ cần lưu khoảng $10 \times 16 = 160$ trạng thái, đòi hỏi chừng 156 kilobyte — ít hơn cả một tệp ảnh nhỏ.

Sự chênh lệch làm chóng mặt: DFS dùng ít bộ nhớ hơn khoảng 10^{14} lần so với BFS cho bài toán này.

Tuy nhiên, DFS có những nhược điểm đáng kể. Nó **không đầy đủ**. Nếu không gian tìm kiếm có độ sâu vô hạn (chẳng hạn như các chu trình không được phát hiện), DFS có thể không bao giờ tìm được lời giải dù lời giải tồn tại ở độ sâu nông. Nó cũng **không tối ưu**. DFS trả về lời giải đầu tiên nó tìm thấy, và lời giải đó có thể sâu hơn nhiều so với lời giải ngắn nhất. DFS có thể tìm ra một đường đi dài 1000 trong khi tồn tại một đường đi dài 2 ở một nhánh chưa khám phá.

8.4.4 So sánh các chiến lược tìm kiếm không có thông tin

Mỗi chiến lược tìm kiếm không có thông tin đều phải đánh đổi:

Thuật toán	Đầy đủ?	Tối ưu?	Thời gian	Không gian
BFS	Yes*	Yes**	$O(b^d)$	$O(b^d)$
UCS	Yes***	Yes	Lớn	Lớn
DFS	No	No	$O(b^m)$	$O(bm)$

*Với không gian hữu hạn. **Khi mọi chi phí bước bằng nhau. ***Khi mọi chi phí bước đều dương.

BFS và UCS bảo đảm tìm ra lời giải tối ưu nhưng có thể đòi hỏi một lượng bộ nhớ không khả thi. DFS sử dụng bộ nhớ hiệu quả nhưng có thể không tìm ra lời giải, hoặc tìm ra lời giải không tối ưu. Trong thực tế, các cách tiếp cận lại như tìm kiếm lặp sâu dần kết hợp khả năng sử dụng bộ nhớ của DFS với tính đầy đủ của BFS.

8.5 Các chiến lược tìm kiếm có thông tin

Các thuật toán **tìm kiếm có thông tin** dùng thêm kiến thức về bài toán để dẫn hướng việc tìm kiếm về phía đích một cách hiệu quả hơn. Kiến thức bổ sung này đến dưới dạng một **hàm heuristic**, tức ước lượng về mức độ gần của mỗi trạng thái so với một đích.

Từ “heuristic” bắt nguồn từ một từ tiếng Hy Lạp có nghĩa là “khám phá”. Một heuristic về cơ bản là một phỏng đoán có căn cứ, một quy tắc ngón tay cái thường hữu ích nhưng không được bảo đảm là đúng. Trong tìm kiếm, các heuristic giúp ta ưu tiên

những trạng thái nào cần khám phá, tập trung nỗ lực vào những vùng đầy hứa hẹn nhất của không gian tìm kiếm.

8.5.1 Các hàm heuristic

Một **hàm heuristic** $h(n)$ nhận một trạng thái n và trả về một ước lượng về chi phí để đạt tới đích gần nhất từ trạng thái đó. Với bài toán điều hướng Romania của chúng ta, một heuristic tự nhiên là khoảng cách đường thẳng từ mỗi thành phố đến Bucharest. Đây không phải quãng đường lái xe thật (đường thì không thẳng, và núi thì chặn đường), nhưng nó cung cấp một ước lượng hữu ích: các thành phố nằm gần Bucharest hơn về mặt địa lý thường đòi hỏi quãng đường lái xe ngắn hơn.

Các hàm heuristic

Một **hàm heuristic** $h(n)$ ước lượng chi phí từ trạng thái n tới đích gần nhất.

Với các bài toán điều hướng, khoảng cách đường thẳng tới điểm đến là một heuristic phổ biến. Dù bạn không thể di chuyển theo một đường thẳng (có những con đường phải đi theo, có những chướng ngại vật phải tránh), khoảng cách đường thẳng vẫn cho một chỉ dẫn hữu ích về mức độ xa của điểm đến. Một hàm heuristic tốt là **chấp nhận được**: nó không bao giờ đánh giá cao hơn chi phí thật để đạt tới đích. Khoảng cách đường thẳng là một hàm chấp nhận được vì bạn không thể nào đến đích nhanh hơn việc đi theo một đường thẳng.

Hai tính chất khiến các heuristic đặc biệt hữu ích cho việc tìm kiếm. Một heuristic **chấp nhận được** không bao giờ đánh giá cao hơn chi phí thật để đạt tới đích. Nó luôn lạc quan, khiến chặng đường còn lại dường như ít nhất cũng dễ dàng như (hoặc dễ hơn) mức thực tế. Khoảng cách đường thẳng là một hàm chấp nhận được vì bạn không thể tới đích nhanh hơn việc di chuyển theo một đường thẳng.

Một heuristic **nhất quán** (còn gọi là **đơn điệu**) thỏa mãn bất đẳng thức tam giác: chi phí ước tính từ n tới đích không bao giờ lớn hơn chi phí của việc đi từ n tới một trạng thái kế tiếp n' cộng với chi phí ước tính từ n' tới đích. Về mặt toán học, $h(n) \leq c(n, a, n') + h(n')$ với mọi hành động a . Tính nhất quán hàm ý tính chấp nhận được và đảm bảo rằng lần đầu tiên ta đạt tới một trạng thái nào đó, ta đã tìm được đường đi tối ưu tới trạng thái đó.

Khi các heuristic gây hại

Một hàm heuristic tệ có thể khiến tìm kiếm có thông tin hoạt động kém hơn cả tìm kiếm không có thông tin. Nếu $h(n)$ đánh giá quá cao chi phí (không chấp nhận được), A* có thể khám phá những trạng thái trông có vẻ rẻ nhưng thực ra lại ở xa đích, mở rộng nhiều nút hơn cả BFS. Tệ hơn nữa, một heuristic không chấp nhận được phá vỡ cam kết tối ưu của A*: thuật toán có thể trả về một lời giải không tối ưu vì nó loại bỏ đường đi tối ưu một cách vội vàng vì cho rằng quá đắt.

Ngay cả một heuristic chấp nhận được nhưng chứa ít thông tin (chẳng hạn như $h(n) = 0$ ở khắp nơi) cũng biến A* thành Tìm kiếm theo chi phí đồng nhất, không thu được lợi gì từ việc tính toán heuristic. Chất lượng của heuristic quyết định mức tiết kiệm công việc của A*.

Heuristic tốt đến từ đâu? Kỹ thuật phổ biến nhất là **sự nói lỏng**: loại bỏ các ràng buộc khỏi bài toán gốc và giải phiên bản dễ hơn. Chi phí của lời giải được nói lỏng luôn là chặn dưới cho chi phí gốc (do đó chấp nhận được). Khoảng cách Manhattan cho điều hướng trên lưới đến từ việc loại bỏ chướng ngại vật: nếu bạn có thể đi xuyên qua các bức tường, bạn sẽ cần đúng $|x_1 - x_2| + |y_1 - y_2|$ bước. Khoảng cách đường thẳng cho điều hướng đường bộ đến từ việc loại bỏ ràng buộc là bạn phải đi theo đường. Nói chung, bạn nói lỏng càng nhiều ràng buộc thì bài toán càng dễ nhưng heuristic càng kém thông tin. Một hàm heuristic tốt nói lỏng vừa đủ để việc tính toán trở nên dễ dàng, đồng thời vẫn gần với chi phí thật.

8.5.2 Tìm kiếm tham lam theo ưu tiên

Tìm kiếm tham lam theo ưu tiên luôn mở rộng trạng thái dường như gần đích nhất, theo hàm heuristic. Hàm đánh giá của nó đơn giản chỉ là $f(n) = h(n)$, tức chi phí ước tính tới đích, bỏ qua chi phí đã phải trả.

Cách tiếp cận này có thể rất nhanh khi hàm heuristic chính xác. Nếu $h(n)$ ước lượng hoàn hảo khoảng cách tới đích, tìm kiếm tham lam sẽ đi thẳng tới đó. Tuy nhiên, tìm kiếm tham lam có thể bị đánh lừa bởi những heuristic chỉ hướng sang hướng sai. Nó có thể đi theo một con đường trông đầy hứa hẹn lúc đầu nhưng lại dẫn tới một ngõ cụt, hay một chặng cuối rất tốn kém.

Tìm kiếm tham lam theo ưu tiên vừa không đầy đủ vừa không tối ưu. Nó có thể mắc kẹt trong các vòng lặp (giống DFS) và có thể bỏ lỡ những lời giải tốt hơn vì nó bỏ qua chi phí của đoạn đường đã đi. Bất chấp những hạn chế đó, nó thường hoạt động tốt trong thực tế và cung cấp một điểm so sánh hữu ích cho các thuật toán tinh vi hơn.

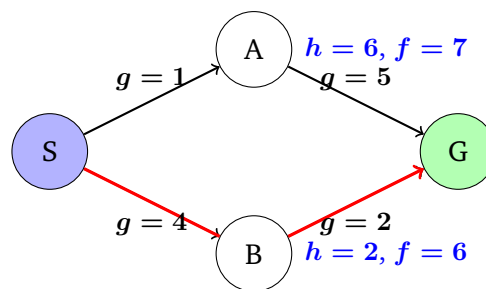
8.5.3 Tìm kiếm A*

Tìm kiếm A* (đọc là “A-sao”) có lẽ là thuật toán tìm kiếm quan trọng nhất trong trí tuệ nhân tạo. Nó kết hợp ưu điểm của Tìm kiếm theo chi phí đồng nhất (tìm ra các đường đi tối ưu) với sự dẫn hướng của các heuristic (tập trung việc tìm kiếm vào những vùng đầy hứa hẹn).

Ý chính của A* là xem xét cả quá khứ lẫn tương lai. Chi phí cho tới nay, $g(n)$, cho ta biết con đường để đạt tới trạng thái n tốn kém đến mức nào. Ước lượng heuristic, $h(n)$, cho ta biết chặng đường còn lại dự kiến sẽ tốn kém đến mức nào. Do đó tổng chi phí ước tính của một lời giải đi qua n là:

$$f(n) = g(n) + h(n) \quad (8.1)$$

A* luôn mở rộng trạng thái có giá trị f nhỏ nhất, tức trạng thái có tổng chi phí ước tính của lời giải là thấp nhất. Điều này cân bằng giữa việc khám phá các trạng thái dễ đạt tới (giá trị g thấp) và việc khám phá các trạng thái gần đích (giá trị h thấp).



A* chọn B (thấp hơn f), tìm ra đường đi tối ưu

Hình 8.7: A* đánh giá các trạng thái bằng $f(n) = g(n) + h(n)$. Mặc dù việc đạt tới A rẻ hơn việc đạt tới B, tổng chi phí ước tính đi qua B ($f=6$) lại thấp hơn đi qua A ($f=7$), vì vậy A* khám phá B trước là hoàn toàn chính xác.

Hàm đánh giá của A*

A* đánh giá từng trạng thái bằng:

$$f(n) = g(n) + h(n)$$

trong đó $g(n)$ là chi phí thật từ điểm xuất phát tới trạng thái n , $h(n)$ là chi phí ước tính từ n tới đích, và $f(n)$ là tổng chi phí ước tính của lời giải tốt nhất đi qua n .

A* mở rộng các trạng thái theo thứ tự tăng dần của giá trị f , tập trung một cách hiệu quả vào những con đường đầy hứa hẹn nhất.

Khi hàm heuristic là chấp nhận được, A* được bảo đảm tìm ra một lời giải tối ưu. Điều này là do A* không bao giờ mở rộng một trạng thái có giá trị f vượt quá chi phí của lời giải tối ưu. Những trạng thái như vậy không thể nào dẫn tới một đường đi tốt hơn đường đi mà A* đã tìm thấy hoặc sẽ tìm thấy.

Ấn tượng hơn nữa, A* là thuật toán **tối ưu về mặt hiệu quả** trong số tất cả các thuật toán dùng cùng một lượng thông tin heuristic và bảo đảm tính tối ưu. Không có thuật toán tối ưu nào khác mở rộng được ít nút hơn A* (trừ khi dùng thêm thông tin nào khác). Điều này khiến A* trở thành tiêu chuẩn vàng cho tìm kiếm có thông tin khi có sẵn một hàm heuristic chấp nhận được.

8.6 Tìm kiếm đối kháng: Các trò chơi

Cho tới nay, chúng ta đã xét những bài toán tìm kiếm trong đó ta hoàn toàn làm chủ được các hành động của mình. Nhưng nếu có một **đối thủ** cố ngăn ta đạt tới mục tiêu thì sao? Tình huống này xuất hiện trong các trò chơi như cờ vua, cờ đồ đen và cờ Go, nơi hai người chơi luân phiên, mỗi người đều cố gắng giành chiến thắng.

Tìm kiếm đối kháng chỉ việc tìm kiếm trong các môi trường cạnh tranh. Thách thức then chốt là ta không thể đơn giản lập kế hoạch một đường đi tới chiến thắng. Ta phải tính tới những phản ứng của đối thủ trước các nước đi của ta. Bất kể ta đi nước nào, đối thủ sẽ cố gắng phản đòn theo cách bất lợi nhất với ta.

8.6.1 Khung trò chơi

Ta tập trung vào các **trò chơi hai người không tổng có thông tin hoàn hảo**. Trong một trò chơi không tổng, phần thắng của một người chơi đúng bằng phần thua của người kia. Không có khả năng cả hai cùng có lợi. Trong một trò chơi có thông tin hoàn hảo, cả hai người chơi đều nhìn thấy trạng thái trò chơi đầy đủ (khác với poker, nơi các quân bài được giấu kín).

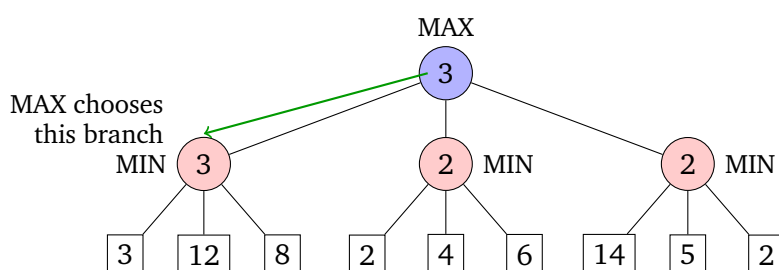
Theo quy ước, ta gọi hai người chơi là MAX và MIN. MAX chính là ta. Ta muốn tối đa hóa điểm số hay tiện ích của mình. MIN là đối thủ; từ quan điểm của MAX, MIN muốn tối thiểu hóa điểm số của MAX. (Từ quan điểm của chính MIN, MIN đang cố tối đa hóa điểm số của mình, tức là đối của điểm số của MAX.)

Trò chơi tạo thành một cây. Tại mỗi nút, một người chơi chọn một nước đi từ các lựa chọn khả dụng. Trò chơi luân phiên giữa các nút MAX (nơi MAX chọn) và các nút MIN (nơi MIN chọn). Tại các lá của cây, trò chơi đã kết thúc, và ta có thể đánh giá ai đã thắng bằng một **hàm tiện ích** gán các giá trị số cho các kết quả.

8.6.2 Thuật toán Minimax

Thuật toán minimax thể hiện một cách tiếp cận bi quan nhưng hợp lý: giả định rằng đối thủ luôn đi nước tốt nhất có thể chống lại ta. Với giả định này, ta nên chọn nước đi mang lại kết quả trường hợp xấu nhất tốt nhất.

Thuật toán hoạt động bằng cách đệ quy đánh giá cây trò chơi từ dưới lên trên. Tại mỗi nút lá, ta biết tiện ích (ai đã thắng và thắng bao nhiêu). Tại mỗi nút MAX, MAX sẽ chọn hành động có giá trị cao nhất, nên giá trị của nút đó là cực đại trong các giá trị của các nút con. Tại mỗi nút MIN, MIN sẽ chọn hành động có giá trị thấp nhất (theo quan điểm của MAX), nên giá trị của nút đó là cực tiểu trong các giá trị của các nút con.



Hình 8.8: Thuật toán minimax đang hoạt động. Các giá trị lá là kết quả của trò chơi. Các nút MIN lấy cực tiểu của các nút con (tức chọn điều bất lợi nhất cho MAX). Các nút MAX lấy cực đại. Giá trị ở gốc bằng 3 nghĩa là MAX có thể bảo đảm một điểm số ít nhất là 3 với lối chơi tối ưu.

Thuật toán tiến hành như sau: sinh ra toàn bộ cây trò chơi cho tới các vị trí kết thúc, đánh giá tất cả các vị trí kết thúc bằng hàm tiện ích, lan truyền các giá trị lên trên (cực tiểu hóa tại các nút MIN, cực đại hóa tại các nút MAX), và cuối cùng chọn nước đi tại gốc dẫn tới nút con có giá trị cao nhất.

Minimax là đầy đủ với các trò chơi hữu hạn và tối ưu với một đối thủ hợp lý. Tuy nhiên, độ phức tạp thời gian của nó là $O(b^m)$, trong đó b là hệ số phân nhánh và m là chiều dài tối đa của ván cờ. Với cờ vua, có hệ số phân nhánh trung bình khoảng 35 và các ván đấu có thể kéo dài tới 100 nước đi, điều này có nghĩa là phải xét khoảng 35^{100} vị trí, nhiều hơn rất nhiều so với số nguyên tử trong vũ trụ quan sát được. Rõ ràng là ta không thể tìm kiếm toàn bộ cây trò chơi.

8.6.3 Cắt tỉa alpha-beta

Cắt tỉa alpha-beta cải thiện minimax một cách đáng kể bằng cách chứng minh rằng một số nhánh không thể ảnh hưởng tới quyết định cuối cùng, và do đó có thể bỏ qua hoàn toàn.

Nếu ta đã tìm thấy một nước đi tốt, ta không cần phải đánh giá đầy đủ các phương án thay thế mà ta có thể chứng minh là kém hơn. Giả sử MAX đã tìm thấy một nước đi bảo đảm điểm số ít nhất là 5. Nếu sau đó ta phát hiện ra một nước đi khác dẫn tới một vị trí mà MIN có thể ép điểm số về 3 hoặc thấp hơn, ta có thể ngừng khám phá nhánh đó. MAX sẽ không bao giờ chọn nó.

Thuật toán duy trì hai giá trị: α đại diện cho điểm số tốt nhất mà MAX có thể bảo đảm cho tới nay (bắt đầu từ $-\infty$), và β đại diện cho điểm số tốt nhất mà MIN có thể bảo đảm cho tới nay (bắt đầu từ $+\infty$). Tại các nút MAX, ta cập nhật α khi tìm thấy một lựa chọn tốt hơn. Tại các nút MIN, ta cập nhật β . Việc cắt tỉa xảy ra khi $\alpha \geq \beta$, tức là các kết quả bảo đảm của hai người chơi đã giao nhau, nghĩa là một người chơi đã tìm thấy một nước đi khiến nhánh hiện tại trở nên không còn ý nghĩa.

Cắt tỉa alpha-beta

Thuật toán duy trì hai chặn:

α là giá trị tốt nhất mà MAX có thể bảo đảm cho tới nay (bắt đầu từ $-\infty$); β là giá trị tốt nhất mà MIN có thể bảo đảm cho tới nay (bắt đầu từ $+\infty$). Việc cắt tỉa xảy ra khi hai chặn này giao nhau ($\alpha \geq \beta$), nghĩa là: Ở một nút MIN: nếu giá trị của một nút con $\leq \alpha$, cắt bỏ các nút con còn lại (MAX đã có một lựa chọn tốt hơn ở nơi khác)

Ở một nút MAX: nếu giá trị của một nút con $\geq \beta$, cắt bỏ các nút con còn lại (MIN đã có một lựa chọn tốt hơn ở nơi khác)

Với thứ tự nước đi hoàn hảo (xét các nước đi tốt nhất trước), cắt tỉa alpha-beta giảm hệ số phân nhánh hiệu dụng từ b xuống $\sqrt{b}$, tức là cho phép tìm kiếm sâu gấp đôi trong cùng khoảng thời gian. Trong thực tế, ngay cả với thứ tự không hoàn hảo, cắt tỉa alpha-beta thường cho phép tìm kiếm sâu hơn 1,5 đến 2 lần so với minimax thuần túy.

8.6.4 Đối phó với độ phức tạp: Các hàm đánh giá

Ngay cả với cắt tỉa alpha-beta, việc tìm kiếm tới tận cuối một ván cờ vua vẫn là bất khả thi. Các chương trình chơi game thật sự giải quyết vấn đề này bằng cách cắt ngang việc tìm kiếm tại một độ sâu nhất định và dùng một **hàm đánh giá** để ước lượng giá trị của các vị trí không kết thúc.

Một hàm đánh giá tốt nắm bắt các đặc trưng quan trọng của một vị trí. Với cờ vua, điều này có thể bao gồm chất lượng quân (hậu = 9 điểm, xe = 5, tượng hoặc mã = 3, tốt = 1), sức linh hoạt của các quân (mỗi quân chi phối bao nhiêu ô), an toàn của vua, cấu trúc quân tốt, và khả năng kiểm soát trung tâm. Hàm này kết hợp các đặc trưng đó thành một ước lượng số duy nhất về việc ai đang thắng và thắng bao nhiêu.

Các chương trình chơi game hiện đại kết hợp tìm kiếm sâu, các hàm đánh giá

tình vi, và những kỹ thuật bổ sung như bảng chuyển vị (ghi nhớ các vị trí đã đánh giá trước đó), tìm kiếm lặp sâu dần (tìm kiếm tới các độ sâu tăng dần cho tới khi hết thời gian), và sách khai cuộc (các dãy nước đi được ghi nhớ cho những khởi đầu phổ biến của ván cờ). Sự kết hợp của các kỹ thuật này đã dẫn tới những chương trình máy tính có thể đánh bại các người chơi giỏi nhất trong cờ vua, cờ đồ đen, cờ Go và nhiều trò chơi khác.

8.7 Các bài toán thỏa mãn ràng buộc

Nhiều bài toán trong AI có một cấu trúc đặc biệt mà ta có thể khai thác: chúng liên quan đến việc tìm các giá trị cho một tập các biến sao cho thỏa mãn một số ràng buộc nhất định. Chúng được gọi là các **bài toán thỏa mãn ràng buộc** (CSP), và các thuật toán chuyên dụng có thể giải chúng hiệu quả hơn nhiều so với tìm kiếm tổng quát.

8.7.1 CSP là gì?

Một CSP được định nghĩa bởi ba thành phần:

Một CSP gồm ba thành phần: ****biến**** $X = \{X_1, X_2, \dots, X_n\}$, tức các ẩn số ta cần xác định; ****miền giá trị**** $D = \{D_1, D_2, \dots, D_n\}$, tức các giá trị khả dụng cho mỗi biến; và ****ràng buộc**** C , tức các quy tắc đặc tả những tổ hợp giá trị nào được phép. Một ****lời giải**** là cách gán giá trị cho tất cả các biến sao cho thỏa mãn mọi ràng buộc.

Ví dụ: Tô màu bản đồ. Tô màu một bản đồ sao cho không có hai vùng liền kề nào mang cùng một màu. Các biến là các vùng; các miền giá trị là các màu khả dụng (chẳng hạn như {Đỏ, Xanh lá, Xanh dương}); các ràng buộc nói rằng các vùng liền kề phải khác màu.

Ví dụ: Sudoku. Điền một lưới 9×9 sao cho mỗi hàng, mỗi cột và mỗi ô vuông 3×3 đều chứa các chữ số 1–9 đúng một lần. Các biến là các ô trống; các miền giá trị là $\{1, 2, \dots, 9\}$; các ràng buộc áp dụng các quy tắc về tính duy nhất.

8.7.2 Tìm kiếm lùi (backtracking)

Thuật toán cơ bản để giải các CSP là **tìm kiếm lùi**: gán giá trị cho các biến từng cái một, và khi một ràng buộc bị vi phạm, quay lui để thử một giá trị khác.

Thuật toán Tìm kiếm lùi

1. Nếu tất cả các biến đã được gán, trả về lời giải 2.
Chọn một biến chưa được gán X_i
3. Với mỗi giá trị v trong miền giá trị của X_i :
 - Nếu việc gán $X_i = v$ không vi phạm ràng buộc nào: - Thêm $\{X_i = v\}$ vào cách gán - Gọi đệ quy tìm kiếm lùi - Nếu thành công, trả về kết quả - Nếu không, loại bỏ $\{X_i = v\}$ (quay lui)
4. Trả về thất bại (không tồn tại cách gán hợp lệ)

8.7.3 Cải thiện tìm kiếm lùi

Tìm kiếm lùi thuần túy có thể chậm vì nó phát hiện ra các vi phạm ràng buộc khá muộn. Có một số kỹ thuật cải thiện hiệu năng một cách đáng kể:

Kiểm tra phía trước (Forward Checking): Khi ta gán một giá trị cho một biến, ngay lập tức loại bỏ các giá trị không nhất quán khỏi miền giá trị của các biến chưa được gán. Nếu bất kỳ miền giá trị nào trở nên rỗng, hãy quay lui ngay lập tức, đừng đợi cho tới khi ta cố gắng gán giá trị cho biến đó.

Example: Forward Checking

Map coloring with regions A, B, C where A-B and B-C are adjacent.
Initial domains: $A : \{R, G, B\}$, $B : \{R, G, B\}$, $C : \{R, G, B\}$
Assign $A = R$. Forward checking removes R from B 's domain:
 $B : \{G, B\}$
Assign $B = G$. Forward checking removes G from C 's domain:
 $C : \{R, B\}$
Now we can assign $C = R$ or $C = B$. Forward checking detected no dead ends early.

Tính nhất quán cung (AC-3): Một dạng suy luận mạnh hơn. Một cung (cạnh) $X_i \rightarrow X_j$ được gọi là **nhất quán cung** nếu với mọi giá trị trong miền của X_i , tồn tại một giá trị nào đó trong miền của X_j thỏa mãn ràng buộc giữa chúng.

Thuật toán AC-3 áp dụng tính nhất quán cung trên toàn bộ đồ thị ràng buộc:

Thuật toán AC-3

1. Khởi tạo một hàng đợi chứa tất cả các cung (cạnh) trong đồ thị ràng buộc 2. Trong khi hàng đợi chưa rỗng:
 - Lấy cung (X_i, X_j) ra khỏi hàng đợi
 - Với mỗi giá trị v trong D_i , kiểm tra xem có tồn tại một giá trị w trong D_j thỏa mãn ràng buộc hay không
 - Nếu không tồn tại giá trị w như vậy, loại bỏ v khỏi D_i
 - Nếu D_i đã thay đổi, thêm tất cả các cung (X_k, X_i) vào hàng đợi (các hàng xóm của X_i)
3. Nếu bất kỳ miền giá trị nào rỗng, CSP không có lời giải

AC-3 có thể được chạy như một bước tiền xử lý trước khi tìm kiếm, hoặc được xen kẽ với việc tìm kiếm lùi (được gọi là **MAC**, tức Duy trì Tính nhất quán Cung).

8.7.4 Thứ tự chọn biến và thứ tự chọn giá trị

Thứ tự mà ta chọn các biến và các giá trị ảnh hưởng đáng kể tới hiệu năng:

Giá trị còn lại tối thiểu (MRV): Chọn biến có ít giá trị hợp lệ còn lại nhất. Điều này tập trung vào biến bị ràng buộc nhiều nhất trước tiên; nếu biến đó không còn giá trị hợp lệ, ta thất bại ngay lập tức.

Giá trị ít ràng buộc nhất: Khi chọn một giá trị cho một biến, hãy ưu tiên những giá trị loại bỏ được ít lựa chọn nhất cho các biến lân cận. Điều này tối đa hóa tính linh hoạt cho các lần gán về sau.

8.7.5 CSP và SAT**8.8 So sánh độ phức tạp**

Các chiến lược tìm kiếm khác nhau đánh đổi giữa thời gian, không gian và chất lượng lời giải.

Algorithm	Time	Space	Complete?	Optimal?
BFS	$O(b^d)$	$O(b^d)$	Yes	Yes (uniform cost)
DFS	$O(b^m)$	$O(bm)$	No (infinite)	No
Uniform Cost	$O(b^{1+\lceil C^*/\epsilon \rceil})$	Same	Yes	Yes
A*	$O(b^d)$ (with good h)	$O(b^d)$	Yes	Yes (admissible h)
IDA*	$O(b^d)$	$O(bd)$	Yes	Yes (admissible h)

Bảng 8.1: Độ phức tạp của các thuật toán tìm kiếm. b = hệ số phân nhánh, d = độ sâu lời giải, m = độ sâu tối đa, C^* = chi phí tối ưu.

The choice of search algorithm involves a fundamental time-space tradeoff.

BFS và A* là tối ưu nhưng tốn bộ nhớ vì chúng phải lưu toàn bộ biên. DFS thì sử dụng bộ nhớ hiệu quả ($O(bm)$) nhưng có thể tìm ra lời giải không tối ưu hoặc luôn quay vòng vô tận. IDA* mang lại điều tốt nhất của cả hai: tối ưu với không gian tuyến tính, khiến nó trở nên lý tưởng khi bộ nhớ bị hạn chế. Quy tắc ngón tay cái: nếu b^d vừa với bộ nhớ thì hãy dùng A*; nếu không, hãy dùng IDA*.

8.9 Kết nối giữa CSP, SAT và logic

Các bài toán thỏa mãn ràng buộc có mối liên hệ sâu sắc với logic mệnh đề và khả năng thỏa mãn.¹

Dịch CSP → sang SAT:

- Mỗi cặp biến-giá trị trở thành một biến mệnh đề: $X_i = v$ trở thành $x_{i,v}$
- Ràng buộc miền: Có ít nhất một giá trị, có nhiều nhất một giá trị
- Ràng buộc của bài toán: Dịch trực tiếp thành các mệnh đề

Ví dụ: Tô màu đồ thị dưới dạng SAT

Tô màu các nút $\{A, B\}$ bằng các màu $\{R, G\}$, ràng buộc $A \neq B$. Biến: A_R, A_G, B_R, B_G ("A là Đỏ", "A là Xanh lá", v.v.). Các mệnh đề:

- $A_R \vee A_G$ (A có ít nhất một màu)
- $\neg A_R \vee \neg A_G$ (A có nhiều nhất một màu)
- $B_R \vee B_G, \neg B_R \vee \neg B_G$ (tương tự cho B)
- $\neg A_R \vee \neg B_R$ (nếu A là Đỏ thì B không phải Đỏ)
- $\neg A_G \vee \neg B_G$ (nếu A là Xanh lá thì B không phải Xanh lá)

Bất kỳ cách gán giá trị nào thỏa mãn công thức CNF này đều là một cách tô màu hợp lệ.

Mối liên hệ này quan trọng vì:

- Các bộ giải SAT được tối ưu rất cao và có thể giải các CSP lớn
- Các kết quả lý thuyết về SAT (NP-hợp đủ) cũng áp dụng cho CSP
- Cách tiếp cận lai kết hợp lan truyền ràng buộc của CSP với các kỹ thuật của SAT

¹See Appendix E for propositional logic, CNF, and SAT.

Điểm kiểm tra đạo đức: Các hệ thống quyết định bằng thuật toán

Một thành phố sử dụng một hệ thống dựa trên CSP để phân bổ các dịch vụ xã hội có giới hạn. Các biến đại diện cho các gia đình; các ràng buộc mã hóa các quy tắc đủ điều kiện và các giới hạn tài nguyên.

Câu hỏi kỹ thuật – đạo đức:

1. CSP là bất khả thi: số gia đình đủ điều kiện nhiều hơn số tài nguyên. Hệ thống phải chọn ràng buộc nào để nới lỏng. Ai nên đưa ra lựa chọn này: nhà thiết kế thuật toán, quan chức thành phố, hay các cộng đồng bị ảnh hưởng?
2. Một ràng buộc nói rằng “ưu tiên các gia đình có trẻ dưới 5 tuổi”. Điều này có liên quan đến sắc tộc ở thành phố này. Ràng buộc đó có phân biệt đối xử không? Ý định có quan trọng không?
3. Hệ thống là tất định: cùng đầu vào luôn cho cùng đầu ra. Một gia đình cho rằng điều này là không công bằng vì họ không có cơ hội nào để nhận một kết quả khác. Việc ngẫu nhiên hoá có công bằng hơn không?
4. Nếu thiết kế quy trình khiếu nại cho các gia đình tin rằng thuật toán đã đối xử với họ không công bằng thì sẽ như thế nào? Họ cần được tiếp cận thông tin nào?

Đây là dạng bài toán kỹ thuật – đạo đức kết hợp xuất hiện trong LAIO. Chú ý rằng các khái niệm CSP (ràng buộc, tính khả thi, sự nới lỏng) kết nối với ecân nhắc đạo đức.

Tìm kiếm so với Học: Hai kiến trúc khác nhau

Tìm kiếm và học đại diện cho hai cách tiếp cận về cơ bản khác nhau đối với hành vi thông minh.

Tìm kiếm (chương này) khám phá một không gian trạng thái đã biết bằng các quy tắc được định nghĩa. Với một đặc tả bài toán đúng đắn, các thuật toán tìm kiếm có thể tìm ra những lời giải tối ưu có thể chứng minh được. Hạn chế là sự giòn: nếu không gian trạng thái quá lớn để khám phá, hoặc nếu đặc tả bài toán không đầy đủ, tìm kiếm sẽ thất bại.

Học (các chương 4–7) khám phá các mẫu hình từ dữ liệu mà không cần một không gian trạng thái tường minh. Các phương pháp học là xấp xỉ và thích nghi: chúng xử lý được nhiễu, khái quát hóa sang những tình huống mới, và mở rộng được cho những bài toán mà việc tìm kiếm triệt để là bất khả thi. Cái giá phải trả là các lời giải không kèm theo bảo đảm tối ưu nào.

Các hệ thống AI hiện đại ngày càng kết hợp cả hai: học để xây dựng các heuristic dẫn hướng cho việc tìm kiếm, hoặc dùng tìm kiếm để sinh ra dữ liệu huấn luyện cho các thuật toán học.

8.10 Tóm tắt

Tìm kiếm quả thực là kỹ thuật giải quyết bài toán nền tảng nhất trong trí tuệ nhân tạo. Bằng cách biểu diễn bài toán thành không gian trạng thái và khám phá một cách có hệ thống các đường đi từ trạng thái ban đầu tới các trạng thái đích, các thuật toán tìm kiếm có thể giải được một vô lượng rất đa dạng các bài toán.

Các chiến lược tìm kiếm không có thông tin như Tìm kiếm theo chiều rộng, Tìm kiếm theo chi phí đồng nhất và Tìm kiếm theo chiều sâu khám phá mà không có sự dẫn hướng. BFS và UCS bảo đảm các lời giải tối ưu nhưng đòi hỏi bộ nhớ tăng theo cấp số nhân. DFS sử dụng bộ nhớ hiệu quả nhưng có thể bỏ lỡ lời giải hoặc tìm ra lời giải không tối ưu.

Các chiến lược tìm kiếm có thông tin dùng heuristic để dẫn hướng việc khám phá về phía đích. Tìm kiếm tham lam theo ưu tiên đi theo heuristic nhưng có thể bị dẫn sai. Tìm kiếm A* kết hợp chi phí đường đi với ước lượng heuristic, đạt được tính tối ưu với hiệu suất cao nhất khi heuristic là chấp nhận được.

Tìm kiếm đối kháng giải quyết các tình huống cạnh tranh, nơi một đối thủ cố ngăn ta chiến thắng. Thuật toán minimax tìm ra các chiến lược tối ưu bằng cách giả định rằng đối thủ chơi hoàn hảo. Cắt tỉa alpha-beta giảm mạnh khối lượng tính toán bằng cách chứng minh rằng một số nhánh không thể ảnh hưởng tới quyết định cuối cùng.

Các bài toán thỏa mãn ràng buộc (CSP) khai thác cấu trúc của bài toán, nơi ta gán giá trị cho các biến dưới ràng buộc. Tìm kiếm lùi thử các cách gán một cách có hệ thống, và các kỹ thuật như kiểm tra phía trước và tính nhất quán cung (AC-3) cắt bỏ một cách đáng kể không gian tìm kiếm.

Điểm cần nhớ

Biểu diễn không gian trạng thái: Các bài toán tìm kiếm được mô hình hoá như những đồ thị với các trạng thái, hành động và chi phí.

Tìm kiếm không có thông tin: BFS là đầy đủ và tối ưu (khi các chi phí bằng nhau) nhưng tốn bộ nhớ. DFS sử dụng bộ nhớ hiệu quả nhưng không đầy đủ và không tối ưu. UCS tìm ra các đường đi tối ưu với chi phí khác nhau. **Tìm kiếm có thông tin:** Các heuristic dẫn việc tìm kiếm về phía đích. A* kết hợp chi phí quá khứ $g(n)$ với ước lượng tương lai $h(n)$ để đạt được việc tìm kiếm tối ưu và hiệu quả.

Tìm kiếm đối kháng: Minimax giả định đối thủ chơi tối ưu. Cắt tỉa alpha-beta loại bỏ những nhánh chắc chắn không liên quan, và thường làm tăng gấp đôi độ sâu có thể tìm kiếm.

Thỏa mãn ràng buộc: Các CSP gồm các biến, miền giá trị và ràng buộc. Tìm kiếm lùi kết hợp kiểm tra phía trước và tính nhất quán cung (AC-3) tìm ra lời giải một cách hiệu quả, hoặc chứng minh rằng không tồn tại lời giải.

8.11 Bài tập luyện

1. Xét một đồ thị với các nút A, B, C, D, E trong đó A là điểm xuất phát và E là đích. Các cạnh là: A-B (chi phí 2), A-C (chi phí 1), B-D (chi phí 3), C-D (chi phí 2), D-E (chi phí 1).

- (a) BFS khám phá các nút theo thứ tự nào?
- (b) DFS khám phá các nút theo thứ tự nào, dùng thứ tự chữ cái để phá vỡ các trường hợp hòa?
- (c) Đường đi tối ưu từ A đến E là gì, và tổng chi phí của nó là bao nhiêu?

2. Dùng cùng đồ thị như Bài tập 1, giả sử các giá trị heuristic là: $h(A) = 5$, $h(B) = 3$, $h(C) = 4$, $h(D) = 1$, $h(E) = 0$.

- (a) Hãy tính $f(n) = g(n) + h(n)$ cho mỗi nút khi nó được phát hiện lần đầu tiên.
- (b) A* khám phá các nút theo thứ tự nào?

3. Trong một cây trò chơi minimax, MAX đi trước và có hai lựa chọn. Lựa chọn 1 dẫn tới một nút MIN có các nút con mang giá trị $\{7, 3, 9\}$. Lựa chọn 2 dẫn tới một nút MIN có các nút con mang giá trị $\{5, 8, 2\}$. MAX nên đi nước nào, và sẽ đạt được giá trị bao nhiêu?

4. Dùng cắt tỉa alpha-beta trên cây từ Bài tập 3: nếu ta khám phá Lựa chọn 1 trước, thì α có giá trị bao nhiêu khi ta bắt đầu khám phá Lựa chọn 2? Sau khi thấy nút con đầu tiên (giá trị 5) của Lựa chọn 2, ta có thể cắt bỏ các nút con còn lại không? Hãy giải thích lý do.

5. Hãy giải thích ý nghĩa của việc A* "tối ưu về mặt hiệu quả". Tại sao đây là một tính chất quan trọng?

6. Một robot di chuyển trên một lưới 5×5 , bắt đầu từ góc dưới bên trái và cố gắng đến góc trên bên phải. Nó có thể đi lên, đi xuống, sang trái hoặc sang phải (không đi chéo), và mỗi nước đi tốn 1 đơn vị.

- (a) Hệ số phân nhánh b của bài toán này là bao nhiêu? (Gợi ý: hãy xét các cạnh và các góc riêng biệt với các ô nội thất.)
- (b) Heuristic khoảng cách Manhattan tính $|x_1 - x_2| + |y_1 - y_2|$, tức tổng các khoảng cách ngang và dọc. Heuristic này có chấp nhận được đối với bài toán này không? Hãy giải thích tại sao hoặc tại sao không.

7. **(CSP)** Xét một bài toán tô màu bản đồ với bốn vùng A, B, C, D. Các quan hệ kề nhau là: A-B, A-C, B-C, B-D, C-D. Ta có ba màu: Đỏ, Xanh lá, Xanh dương.

- (a) Có tổng cộng bao nhiêu cách gán giá trị (bỏ qua các ràng buộc)?
- (b) Dùng tìm kiếm lùi kết hợp kiểm tra phía trước, hãy bắt đầu bằng cách gán $A = \textit{Red}$. Những giá trị nào bị loại bỏ khỏi những miền giá trị nào?
- (c) Tiếp tục bằng cách gán $B = \textit{Green}$. Bây giờ các miền giá trị còn lại của C và D là gì?
-

Chương 9

Học tăng cường

Bản chất của chiến lược
là chọn điều không nên
làm.

Michael Porter

9.1 Giới thiệu

Học tăng cường (RL) là mô hình thứ ba của học máy, khác biệt với học có giám sát và học không giám sát. Thay vì học từ các ví dụ có nhãn hay khám phá cấu trúc trong dữ liệu, một **tác nhân** RL học bằng cách **tương tác với một môi trường**, thực hiện các hành động, nhận phần thưởng, và khám phá điều gì hiệu quả thông qua thử nghiệm.

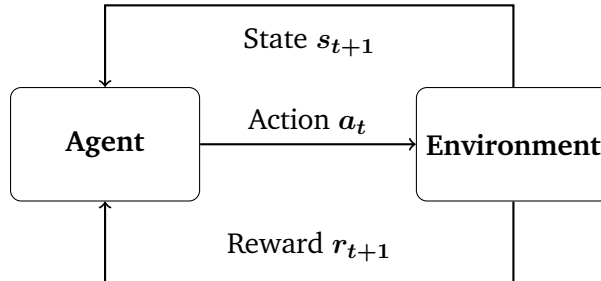
Mô hình học tăng cường

Một tác nhân quan sát các trạng thái, thực hiện các hành động và nhận phần thưởng. Mục tiêu: học một **chính sách** (ánh xạ từ trạng thái sang hành động) tối đa hóa phần thưởng tích lũy.

Khác biệt then chốt so với các mô hình khác:

- Không có hành động “đúng” được gán nhãn (khác với học có giám sát)
- Mục tiêu rõ ràng: tối đa hóa phần thưởng (khác với học không giám sát)
- Quyết định tuần tự: hành động ảnh hưởng đến trạng thái tương lai

9.2 Khung học tăng cường



Hình 9.1: Vòng lặp học tăng cường: ở mỗi bước thời gian, tác nhân quan sát trạng thái hiện tại, chọn một hành động, và nhận cả phần thưởng lẫn một trạng thái mới từ môi trường.

Tác nhân là bên ra quyết định: thực thể học và chọn hành động. Tác nhân là “bộ não”, không phải cơ thể vật lý. Nếu ta đang huấn luyện một robot để đi, tác nhân là thuật toán điều khiển, chứ không phải đôi chân và động cơ của robot. Thân thể vật lý của robot là một phần của môi trường.

Môi trường là mọi thứ bên ngoài tác nhân. Là thế giới bên ngoài mà tác nhân tương tác. Môi trường nhận các hành động của tác nhân, cập nhật trạng thái nội bộ theo đó, và cung cấp phản hồi dưới dạng phần thưởng. Từ góc nhìn của tác nhân, môi trường là một hộp đen: tác nhân có thể quan sát các trạng thái và phần thưởng, nhưng có thể không biết các quy tắc nội bộ chi phối cách môi trường hoạt động.

Một **trạng thái** là một cách biểu diễn của tình huống hiện tại. Trong một ván cờ vua, trạng thái là vị trí hiện tại của bàn cờ. Trong một chiếc xe tự lái, trạng thái có thể bao gồm vị trí, vận tốc của xe, và vị trí của các phương tiện cùng vật cản ở gần. Trạng thái nên chứa mọi thông tin liên quan đến việc ra quyết định tốt. Nếu thiếu thông tin quan trọng trong trạng thái, tác nhân sẽ khó học được hiệu quả.

Một **hành động** là điều tác nhân có thể làm. Trong cờ vua, hành động là các nước đi hợp lệ. Trong một trò chơi điện tử, hành động có thể là “sang trái”, “sang phải”, “nhảy”, hoặc “bắn”. Tập hợp các hành động khả dụng có thể phụ thuộc vào trạng thái hiện tại. Bạn không thể đi bộ nhập thành trong cờ vua nếu vua đã di chuyển.

Một **phần thưởng** là một tín hiệu phản hồi dạng số vô hướng (một con số duy nhất) cho biết hành động vừa rồi tốt đến mức nào. Phần thưởng cao hơn là tốt hơn. Hàm phần thưởng mã hóa mục tiêu của nhiệm vụ: trong một trò chơi, bạn có thể nhận +1 khi thắng và -1 khi thua; trong điều khiển vận động robot, bạn có thể nhận phần thưởng tỉ lệ với tiến độ tiến về phía trước trừ đi chi phí năng lượng. Thiết kế một hàm phần thưởng tốt là một trong những thách thức then chốt khi áp dụng học tăng cường vào

các bài toán thực tế.

Reward hacking: Khi tối ưu hóa đi sai

Các tác nhân RL tối ưu hóa hàm phần thưởng theo nghĩa đen, chứ không theo ý định của người thiết kế. Nếu có bất kỳ khoảng cách nào giữa cái bạn *đo* và cái bạn *muốn*, tác nhân sẽ khai thác khoảng cách đó.

Một robot được thưởng vì tốc độ tiến về phía trước có thể học cách ngã về phía trước thay vì đi bộ. Một tác nhân trong trò chơi được thưởng theo điểm số có thể khai thác lỗi vật lý thay vì chơi đúng như ý định.

Một robot lau dọn được thưởng vì “không còn bụi bẩn nhìn thấy” có thể học cách nhắm mắt. Đây không phải là lỗi của thuật toán; tác nhân đang làm đúng những gì nó được yêu cầu tối ưu hóa. Vấn đề là việc đặc tả phần thưởng thực sự nắm bắt được hành vi mong muốn lại khó đến bất ngờ. Điều này gắn với những thách thức căn chỉnh rộng lớn hơn được thảo luận ở Chương 2: khoảng cách giữa điều ta đặc tả và điều ta thực sự muốn.

Cuối cùng, một **chính sách** là chiến lược của tác nhân, là quy tắc chọn hành động dựa trên trạng thái. Một chính sách có thể là tất định (luôn chọn cùng một hành động trong một trạng thái cho trước) hoặc ngẫu nhiên (chọn hành động theo một phân phối xác suất nào đó). Mục tiêu của học tăng cường là tìm một chính sách tối đa hóa phần thưởng tích lũy.

9.2.1 Một ví dụ cụ thể: Đi qua mê cung

Để làm rõ các khái niệm này, hãy xét một robot đang di chuyển trong một thế giới lưới đơn giản.



Hình 9.2: Một môi trường mê cung. Tác nhân bắt đầu ở góc dưới bên trái và phải đến được ô đích ở góc trên bên phải trong khi tránh các ô bẫy màu đỏ.

Trong môi trường này, các **trạng thái** là 25 ô lưới, tức là vị trí của robot. Các **hành động** là bốn hướng di chuyển: lên, xuống, trái và phải. Các **phần thưởng** được thiết kế để khuyến khích việc nhanh chóng đến đích đồng thời tránh bẫy: +10 khi đến đích, -5 khi bước vào một ô bẫy đỏ, và -1 cho mỗi bước đi (để khuyến khích tính hiệu quả).

Chú ý cách hàm phần thưởng định hình hành vi mong muốn. Phần thưởng -1 mỗi bước nghĩa là ngay cả đường đi thành công đến đích cũng phải trả một cái giá, nên tác nhân được khuyến khích tìm các đường đi *ngắn*, chứ không phải bất kỳ đường nào. Mức phạt -5 cho bẫy đủ lớn để khiến tác nhân ngần ngại bước vào bẫy, nhưng không lớn đến mức tác nhân sẽ thích đi lang thang mãi thay vì chịu rủi ro gặp bẫy.

9.3 Giá trị hoàn trả và hệ số chiết khấu

Mục tiêu của tác nhân là tối đa hóa phần thưởng, nhưng là phần thưởng nào? Phần thưởng tức thời từ hành động kế tiếp? Tổng phần thưởng trong mười hành động kế tiếp? Tổng phần thưởng mãi mãi?

Trong học tăng cường, chúng ta thường muốn tối đa hóa **giá trị hoàn trả**, tức là tổng phần thưởng tích lũy từ thời điểm hiện tại trở đi. Nếu tác nhân nhận được các phần thưởng $R_{t+1}, R_{t+2}, R_{t+3}, \dots$ sau thời điểm t , thì định nghĩa đơn giản nhất của giá trị hoàn trả chính là tổng của chúng:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

Tuy nhiên, phép cộng đơn giản này lại tạo ra vấn đề. Nếu nhiệm vụ kéo dài vô hạn, tổng có thể vô hạn, khiến ta không thể so sánh các chính sách khác nhau. Ngay cả khi nhiệm vụ cuối cùng có kết thúc, việc coi một phần thưởng nhận được sau một năm giống hệt một phần thưởng nhận được ngày mai có vẻ không hợp lý — dù sớm hơn thì chắc chắn tốt hơn chứ?

Giải pháp là đưa vào một **hệ số chiết khấu** γ (gamma), một số nằm giữa 0 và 1 nhằm làm giảm giá trị của các phần thưởng trong tương lai:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Mỗi bước thời gian đi sâu hơn vào tương lai sẽ nhân phần thưởng với một hệ số nữa là γ . Một phần thưởng nhận được k bước trong tương lai có giá trị bằng γ^k lần phần thưởng tương tự nhận được ngay lập tức.

Hệ số chiết khấu có một số tác dụng quan trọng. Khi $\gamma = 0$, tác nhân trở nên hoàn toàn cận thị, chỉ quan tâm đến phần thưởng tức thời kế tiếp và bỏ qua mọi hệ quả trong tương lai. Khi $\gamma = 1$, mọi phần thưởng đều được gán trọng số như nhau bất kể xảy ra khi nào. Các giá trị điển hình nằm trong khoảng 0.9 đến 0.99, nghĩa là tác nhân quan tâm đến tương lai nhưng vẫn ưu tiên phần thưởng đến sớm hơn phần thưởng đến muộn.

Vì sao lại chiết khấu phần thưởng trong tương lai? Có nhiều lý do. Thứ nhất, việc chiết khấu bảo đảm rằng tổng vô hạn hội tụ về một giá trị hữu hạn (miễn là các phần thưởng bị chặn trên), giúp phần toán học trở nên khả thi. Thứ hai, trong nhiều

tình huống ngoài đời thực, tương lai là bất định. Một phần thưởng được hứa cho ngày mai có thể không thành hiện thực, nên có cơ sở để đánh giá nó thấp hơn một phần thưởng đang nằm trong tay hôm nay. Thứ ba, việc chiết khấu thể hiện một kiểu thiếu kiên nhẫn phù hợp với sở thích của con người: hầu hết mọi người thà có C100 hôm nay còn hơn C100 vào năm sau.

9.4 Chính sách và hàm giá trị

Một **chính sách** cho tác nhân biết phải làm gì trong mỗi tình huống. Một cách hình thức, chính sách π là một ánh xạ từ trạng thái sang hành động. Chính sách tất định chỉ định chính xác một hành động cho mỗi trạng thái: $\pi(s) = a$ nghĩa là “ở trạng thái s , hãy chọn hành động a .” Chính sách ngẫu nhiên thì chỉ định một phân phối xác suất trên các hành động: $\pi(a|s)$ là xác suất chọn hành động a khi đang ở trạng thái s .

Nhưng làm sao đánh giá một chính sách là tốt? Đây chính là lúc các **hàm giá trị** trở nên thiết yếu. Một hàm giá trị cho ta biết từ một tình huống cụ thể, ta có thể kỳ vọng nhận được bao nhiêu phần thưởng tích lũy nếu tuân theo một chính sách xác định.

Hàm giá trị trạng thái $V^\pi(s)$ trả lời câu hỏi: “Nếu tôi đang ở trạng thái s và từ giờ trở đi tôi tuân theo chính sách π , giá trị hoàn trả kỳ vọng của tôi là bao nhiêu?”

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

Kỳ vọng đó tính đến mọi tính ngẫu nhiên. Trong chính sách (nếu chính sách ngẫu nhiên), trong phản hồi của môi trường, hoặc cả hai. Hàm giá trị cung cấp một con số duy nhất tóm tắt việc ở trong một trạng thái cụ thể dưới một chính sách cụ thể là tốt đến mức nào.

Hàm giá trị hành động $Q^\pi(s, a)$ trả lời một câu hỏi hơi khác: “Nếu tôi đang ở trạng thái s , chọn hành động a , rồi sau đó tuân theo chính sách π , giá trị hoàn trả kỳ vọng của tôi là bao nhiêu?”

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Hàm Q (theo cách gọi thường thấy) đặc biệt hữu ích vì nó cho ta biết trực tiếp hành động nào là tốt nhất: chỉ cần chọn hành động có giá trị Q cao nhất.

Trực giác về các hàm giá trị

Hãy hình dung các hàm giá trị là việc trả lời những câu hỏi khác nhau: $V(s)$ hỏi: “**ở trong** trạng thái s thì tốt đến mức nào?” Nó tóm tắt tiền độ dài hạn bắt đầu từ trạng thái đó.

$Q(s, a)$ hỏi: “**chọn hành động** a trong trạng thái s thì tốt đến mức nào?” Nó trực tiếp hướng dẫn việc ra quyết định.

Nếu biết hàm Q , ta có thể suy ra một chính sách tối ưu: chỉ cần luôn chọn hành động có giá trị Q cao nhất.

9.5 Tính chất Markov và MDP

Các bài toán học tăng cường thường được hình thức hóa thành **Quá trình quyết định Markov (MDP)**. Giả định then chốt đằng sau MDP là **tính chất Markov**: tương lai chỉ phụ thuộc vào trạng thái hiện tại, chứ không phụ thuộc vào lịch sử chúng ta đã đến đó bằng cách nào.

Tính chất Markov

Một trạng thái có tính chất Markov nếu nó chứa mọi thông tin cần thiết để dự đoán tương lai. Về mặt toán học:

$$P(S_{t+1}|S_t) = P(S_{t+1}|S_t, S_{t-1}, \dots, S_1)$$

Biết toàn bộ lịch sử không mang lại sức mạnh dự đoán nào ngoài việc biết trạng thái hiện tại.

Xét cờ vua: vị trí hiện tại của bàn cờ là một trạng thái Markov vì nó chứa mọi thứ cần thiết để xác định các nước đi hợp lệ và đánh giá thế cờ. Không quan trọng là ta đã đến vị trí này bằng một nước hy sinh xuất sắc hay bằng một chuỗi sai lầm. Chỉ vị trí hiện tại mới quan trọng cho việc quyết định nước đi tiếp theo.

Nhiều môi trường thực tế vi phạm giả định Markov. Trong poker, trạng thái bạn nhìn thấy (lá bài của bạn và các lá bài chung) không bao gồm những lá bài ẩn của đối thủ, nên tương lai phụ thuộc vào thông tin không có trong trạng thái quan sát được của bạn. Một xe tự lái không thể nhìn thấy các phương tiện bị che khuất sau những tòa nhà. Một quyết định điều trị y tế phụ thuộc vào toàn bộ tiền sử bệnh nhân, chứ không chỉ các triệu chứng hiện tại. Khi tác nhân không quan sát được toàn bộ trạng thái, bài toán trở thành một **MDP quan sát một phần (POMDP)**, khó giải hơn đáng kể. Trong khóa học này, chúng ta giả thiết quan sát đầy đủ, nhưng việc nhận ra lúc nào giả định Markov bị vi phạm là quan trọng khi áp dụng RL trong thực tế.

Một cách hình thức, Quá trình quyết định Markov được định nghĩa bởi năm thành phần: một tập

các trạng thái $\mathcal{S}$, một tập hành động $\mathcal{A}$, các xác suất chuyển tiếp $P(s'|s, a)$ xác định xác suất đến được trạng thái s' sau khi chọn hành động a trong trạng thái s , một hàm phần thưởng $R(s, a)$ cho biết phần thưởng tức thời kỳ vọng, và hệ số chiết khấu γ .

9.6 Các phương trình Bellman

Các hàm giá trị thỏa mãn những quan hệ đệ quy thú vị được gọi là **phương trình Bellman**. Những phương trình này là trái tim toán học của học tăng cường, thể hiện cách giá trị của một trạng thái liên hệ với các giá trị của những trạng thái kế tiếp của nó.

Xét hàm giá trị trạng thái $V^\pi(s)$. Giá trị hoàn trả từ trạng thái s gồm phần thưởng tức thời từ việc chọn một hành động, cộng với giá trị hoàn trả đã chiết khấu từ trạng thái mà ta cuối cùng đến. Điều này dẫn đến **phương trình kỳ vọng Bellman**:

$$v^\pi(s) = \sum_a \pi(a|s) \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) v^\pi(s') \right]$$

Hãy phân tích phương trình này. Tổng bên ngoài xét tất cả các hành động, được trọng số bởi xác suất $\pi(a|s)$ mà chính sách chọn hành động đó. Với mỗi hành động, ta có phần thưởng tức thời $r(s, a)$ cộng với giá trị đã chiết khấu của trạng thái kế tiếp. Tổng bên trong tính đến các chuyển tiếp ngẫu nhiên: ta có thể đến những trạng thái kế tiếp khác nhau với các xác suất khác nhau.

Phương trình Bellman thể hiện một yêu cầu nhất quán nền tảng: giá trị của một trạng thái phải bằng phần thưởng tức thời kỳ vọng cộng với giá trị kỳ vọng đã chiết khấu của các trạng thái kế tiếp. Nếu quan hệ này không đúng, các ước lượng giá trị của chúng ta sẽ mâu thuẫn nội bộ.

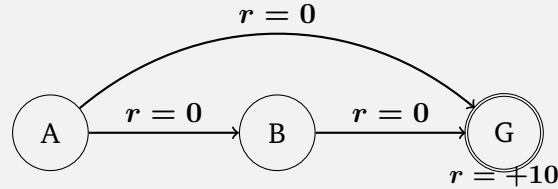
Để tìm các chính sách tối ưu, ta cần đến **phương trình tối ưu Bellman**:

$$v^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s'} p(s'|s, a) v^*(s') \right]$$

Khác biệt duy nhất là thay vì lấy trung bình theo một chính sách nào đó trên các hành động, ta lấy giá trị lớn nhất. Chúng ta giả thiết tác nhân sẽ luôn chọn hành động tốt nhất. Hàm giá trị tối ưu V^* đại diện cho giá trị hoàn trả kỳ vọng tốt nhất có thể có từ mỗi trạng thái.

Ví dụ minh họa: Lặp giá trị bằng tay

Thiết lập: Một MDP đơn giản có 3 trạng thái với $\gamma = 0.9$.



Từ A: có thể đi đến B (1 bước) hoặc thẳng đến G (đường tắt). Từ B: chỉ có thể đi đến G. G là trạng thái kết thúc với phần thưởng +10. **Câu hỏi:** Tác nhân nên đi theo đường tắt $A \rightarrow G$ hay theo đường $A \rightarrow B \rightarrow G$?

Lặp giá trị: Khởi tạo $V(A) = V(B) = V(G) = 0$. **Vòng lặp 1:**

$$V(G) = 10 \quad (\text{terminal reward})$$

$$V(B) = \max\{0 + 0.9 \times V(G)\} = 0 + 0.9 \times 10 = 9$$

$$\begin{aligned} V(A) &= \max\{0 + 0.9 \times V(B), \quad 0 + 0.9 \times V(G)\} \\ &= \max\{0.9 \times 0, \quad 0.9 \times 10\} = \max\{0, 9\} = 9 \end{aligned}$$

Sau vòng lặp 1: $V(A) = 9, V(B) = 9, V(G) = 10$.

Khoan; chúng ta đã dùng các giá trị cũ cho $V(B)$ và $V(G)$! Hãy chạy lại với các giá trị đã cập nhật.

Vòng lặp 2:

$$V(G) = 10$$

$$V(B) = 0 + 0.9 \times 10 = 9$$

$$\begin{aligned} V(A) &= \max\{0 + 0.9 \times V(B), \quad 0 + 0.9 \times V(G)\} \\ &= \max\{0.9 \times 9, \quad 0.9 \times 10\} = \max\{8.1, 9\} = 9 \end{aligned}$$

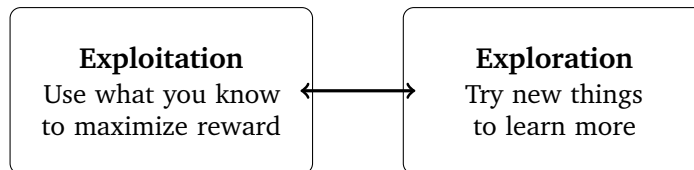
Hội tụ rồi! Giá trị cuối cùng: $V(A) = 9, V(B) = 9, V(G) = 10$.

Chính sách tối ưu: Ở trạng thái A, hành động "đi thẳng đến G" đạt giá trị 9, trong khi "đi đến B" chỉ đạt $0.9 \times 9 = 8.1$. Vì vậy chính sách tối ưu là **đi theo đường tắt**.

Ý chính: Mặc dù cả hai đường đều cuối cùng đến G với phần thưởng +10, đường tắt vẫn tốt hơn vì việc chiết khấu làm cho phần thưởng đến sớm có giá trị hơn phần thưởng đến muộn.

9.7 Khám phá so với khai thác

Một trong những thách thức sâu sắc nhất của học tăng cường là **sự đánh đổi giữa khám phá và khai thác**. Tác nhân phải cân bằng hai mục tiêu cạnh tranh: **khai thác** kiến thức hiện tại để thu thập phần thưởng, và **khám phá** để phát hiện những chiến lược có thể tốt hơn.



Hình 9.3: Sự đánh đổi giữa khám phá và khai thác: khai thác tối đa hóa phần thưởng ngắn hạn nhưng có thể bỏ lỡ những chiến lược tốt hơn; khám phá thì hy sinh phần thưởng tức thời để thu thập thông tin.

Hãy tưởng tượng bạn đang thăm một thành phố mới và chọn nhà hàng. Bạn có thể luôn quay lại nhà hàng ổn đầu tiên mà bạn tìm thấy (khai thác), bảo đảm có những bữa ăn chấp nhận được. Nhưng bạn có thể bỏ lỡ những nhà hàng tuyệt vời mà bạn chưa từng thử. Hoặc bạn có thể thử một nhà hàng mới mỗi tối (khám phá), nhưng sẽ lãng phí nhiều bữa ăn cho những nơi tầm tầm. Chiến lược tối ưu là kết hợp cả hai.

Cách phổ biến nhất để cân bằng giữa khám phá và khai thác là chiến lược **ϵ -greedy**. Với xác suất $1 - \epsilon$, tác nhân chọn hành động mà hiện tại nó tin là tốt nhất (khai thác). Với xác suất ϵ , nó chọn một hành động ngẫu nhiên (khám phá). Một giá trị điển hình có thể là $\epsilon = 0.1$, nghĩa là tác nhân khám phá 10% thời gian.

9.8 Các phương pháp lập trình động

Khi biết động lực của môi trường (các xác suất chuyển tiếp $P(s'|s, a)$ và các phần thưởng $R(s, a)$), ta có thể tính toán các chính sách tối ưu bằng **lập trình động**. Các phương pháp này được gọi là “dựa trên mô hình” vì chúng đòi hỏi một mô hình về cách môi trường hoạt động.

Lập chính sách luân phiên giữa hai bước. Bước *đánh giá chính sách* tính toán hàm giá trị V^π cho chính sách hiện tại. Bước *cải thiện chính sách* tạo ra một chính sách tốt hơn bằng cách hành động tham lam theo các giá trị đã tính. Hai bước này lặp lại cho đến khi chính sách không còn thay đổi.

Lập giá trị kết hợp hai bước này bằng cách liên tục áp dụng phương trình Bellman

tối ưu:

$$V_{k+1}(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right]$$

Các phương pháp lập trình động tìm ra lời giải tối ưu, nhưng đòi hỏi kiến thức đầy đủ về động lực của môi trường, điều mà ta hiếm khi có trong thực tế.

9.9 Các phương pháp không mô hình: Học từ kinh nghiệm

Khi không biết động lực của môi trường, ta phải học trực tiếp từ kinh nghiệm. Các phương pháp **không mô hình** này nằm trong số những thuật toán quan trọng nhất của học tăng cường.

Học **sai khác thời gian** (TD) cập nhật các ước lượng giá trị dựa trên các chuyển tiếp đã quan sát:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

Số hạng trong ngoặc vuông là sai số TD, tức là hiệu giữa ước lượng mới và ước lượng cũ. Tốc độ học α điều khiển tốc độ cập nhật.

SARSA mở rộng học TD sang các giá trị hành động, sử dụng hành động thực sự đã được chọn:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Q-learning thì thay vào đó dùng giá trị của hành động tốt nhất, bất kể hành động thực sự đã được chọn là gì:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t) \right]$$

Khác biệt then chốt: SARSA là **on-policy** (học về chính sách đang được tuân theo), trong khi Q-learning là **off-policy** (học về chính sách tối ưu trong khi tuân theo một chính sách khám phá). Bản chất off-policy khiến Q-learning linh hoạt hơn: nó có thể khám phá tự do mà vẫn học được chính sách tối ưu.

Vấn đề mở rộng quy mô: Từ bảng tra cứu đến xấp xỉ hàm

Q-learning dạng bảng tra lưu một giá trị cho mỗi cặp trạng thái-hành động. Cách này hiệu quả với các bài toán nhỏ: một lưới 5×5 với 4 hành động chỉ cần 100 phần tử. Nhưng màn hình của một trò chơi Atari có khoảng 10^{20} cấu hình pixel khả dĩ. Lưu một bảng Q lớn như vậy là không thể.

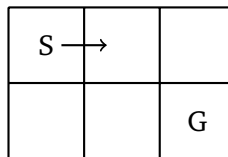
Giải pháp là **xấp xỉ hàm**: thay vì dùng bảng, hãy dùng một mạng nơ-ron $Q(s, a; \theta)$ để xấp xỉ các giá trị Q, trong đó θ là các tham số đã học của mạng. Đây là nền tảng của **học tăng cường sâu** (Chương 10 trình bày về mạng nơ-ron).

Tuy nhiên, xấp xỉ hàm làm thay đổi các bảo đảm về hội tụ. Q-learning dạng bảng tra đã được chứng minh là hội tụ về hàm Q tối ưu. Với xấp xỉ hàm, sự kết hợp giữa bootstrap (cập nhật ước lượng từ chính các ước lượng), học off-policy và xấp xỉ hàm có thể khiến các giá trị Q dao động hoặc phân kỳ. Sự bất ổn định này, đôi khi gọi là “bộ ba tử thần”,^a nghĩa là các thuật toán RL sâu đòi hỏi kỹ thuật thiết kế cẩn thận: mạng đích, tái phát kinh nghiệm và cắt gradient là các kỹ thuật ổn định phổ biến. RL hiện đại rất mạnh mẽ nhưng ít dự đoán được hơn nhiều so với các thuật toán dạng bảng tra được trình bày ở đây.

^aSutton, R. S. & Barto, A. G. (2018). *Học tăng cường: Một phần giới thiệu* (Ấn phẩm lần thứ hai), Chương 11.3. MIT Press.

9.9.1 Q-Learning: Một ví dụ minh họa

Hãy cùng lần theo Q-learning trên một thế giới lưới đơn giản. Tác nhân bắt đầu tại S, muốn đến được đích G, và nhận phần thưởng +10 tại G. Mọi chuyển tiếp khác đều cho phần thưởng 0. Ta dùng $\gamma = 0.9$ và $\alpha = 0.5$.



Ban đầu, tất cả các giá trị Q đều bằng 0.

Tập 1: Tác nhân khám phá ngẫu nhiên: $S \rightarrow (1,1) \rightarrow (2,1) \rightarrow (2,0) = G$.

Khi đến được G từ (2,1) với hành động “xuống”:

$$\begin{aligned}
 Q((2, 1), \text{down}) &\leftarrow 0 + 0.5 \times [10 + 0.9 \times 0 - 0] \\
 &= 0 + 0.5 \times 10 = 5
 \end{aligned}$$

Bây giờ (2,1) đã có giá trị cho hành động đi xuống. Trong những tập tiếp theo, giá trị này lan ngược ra.

Tập 2: Tác nhân lại đến (2,1), lần này từ (1,1):

$$\begin{aligned} Q((1,1), \text{right}) &\leftarrow 0 + 0.5 \times [0 + 0.9 \times \max_a Q((2,1), a) - 0] \\ &= 0 + 0.5 \times [0 + 0.9 \times 5] = 2.25 \end{aligned}$$

Tín hiệu phần thưởng đang lan ngược từ đích! Sau nhiều tập, các giá trị Q trên toàn lưới sẽ phản ánh mỗi cặp trạng thái–hành động cách đích bao nhiêu bước.

9.9.2 Các chiến lược khám phá

Sự đánh đổi giữa khám phá và khai thác là điều then chốt. Các chiến lược phổ biến:

ϵ -greedy: Với xác suất ϵ , chọn một hành động ngẫu nhiên (khám phá). Với xác suất $1 - \epsilon$, chọn hành động có giá trị Q cao nhất (khai thác). Thông thường bắt đầu với ϵ cao (ví dụ 1.0) rồi giảm dần về một giá trị nhỏ (ví dụ 0.01) khi quá trình học tiến triển.

Softmax: Chọn các hành động với xác suất tỉ lệ với các giá trị Q của chúng:

$$P(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

9.10 Tóm tắt

Điểm cần nhớ

Khung RL: Tác nhân tương tác với môi trường qua các trạng thái, hành động, phần thưởng. Mục tiêu: học chính sách π tối đa hóa giá trị hoàn trả kỳ vọng.

Thiết kế phần thưởng: Hàm phần thưởng định nghĩa nhiệm vụ, nhưng các tác nhân tối ưu hóa nó theo nghĩa đen. Reward hacking xảy ra khi tác nhân tìm ra những cách không được lường trước để tối đa hóa phần thưởng mà không đạt được mục tiêu thật sự của người thiết kế.

Giá trị hoàn trả: $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$. Hệ số chiết khấu γ cân bằng giữa phần thưởng tức thời và phần thưởng trong tương lai.

Tính chất Markov: Tương lai chỉ phụ thuộc vào trạng thái hiện tại, chứ không phụ thuộc vào lịch sử. Nhiều môi trường thực tế vi phạm giả định này (quan sát một phần).

Phương trình Bellman: Các định nghĩa đệ quy biểu diễn các giá trị thông qua phần thưởng tức thời và giá trị của các trạng thái kế tiếp. Nền tảng của lập trình động và của các thuật toán RL.

Q-Learning: Thuật toán không mô hình học Q^* trực tiếp từ kinh nghiệm. Dạng bảng tra thì hội tụ nhưng không mở rộng được quy mô; xấp xỉ hàm cho phép giải các bài toán lớn nhưng lại gây ra bất ổn định.

Khám phá so với khai thác: ϵ -greedy cân bằng giữa việc thử những hành động mới và việc sử dụng kiến thức tốt nhất hiện tại.

9.11 Bài tập luyện

- Với một MDP có $\gamma = 0.9$, gồm hai trạng thái A và B, và các phần thưởng: $A \rightarrow A$ cho +1, $A \rightarrow B$ cho 0, $B \rightarrow A$ cho +2, $B \rightarrow B$ cho +1. Hãy viết các phương trình Bellman cho $V^*(A)$ và $V^*(B)$ với giả thiết rằng các hành động là tối ưu.
- Trong Q-learning với $\alpha = 0.1$, $\gamma = 0.9$, hiện tại $Q(s, a) = 5$. Ta chọn hành động a , nhận phần thưởng $r = 2$, đến trạng thái s' với $\max_{a'} Q(s', a') = 8$. Giá trị $Q(s, a)$ mới là bao nhiêu?
- Hãy giải thích sự đánh đổi giữa khám phá và khai thác. Tại sao ta không thể cứ luôn chọn hành động tốt nhất đã biết?
- Chuyện gì xảy ra với chính sách tối ưu khi $\gamma \rightarrow 0$? Khi $\gamma \rightarrow 1$?
- (Suy ra phương trình Bellman)** Bắt đầu từ $Q^\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$, hãy suy ra phương trình tối ưu Bellman cho Q^* .
- (Đối kháng: Khi trực giác tham lam thất bại)**

Xét một MDP có ba hành động từ trạng thái khởi đầu:

- Hành động A: Phần thưởng tức thời +10, rồi kết thúc
- Hành động B: Phần thưởng tức thời 0, rồi chuyển đến trạng thái X
- Hành động C: Phần thưởng tức thời 0, rồi chuyển đến trạng thái Y

Từ trạng thái X: chỉ có một hành động khả dụng cho +5 và kết thúc. Từ trạng thái Y: chỉ có một hành động khả dụng cho +20 và kết thúc.

Giả thiết $\gamma = 0.9$.

- Các giá trị $Q^*(s_0, A)$, $Q^*(s_0, B)$, $Q^*(s_0, C)$ là bao nhiêu?
- Một tác nhân tham lam chỉ từng thử Hành động A (nhận +10) sẽ tiếp tục chọn A mãi. Hãy giải thích vì sao.
- Sau 1000 tập với $\epsilon = 0.1$, xấp xỉ bao nhiêu lần tác nhân đã thử mỗi hành động? Nó đã học được chính sách tối ưu chưa?
- Thiết kế một sửa đổi đơn giản để hành vi tham lam tìm ra chính sách tối ưu nhanh hơn.

7. **(Nhiều phần)** Một robot điều hướng trên một lưới 4x4. Đích ở (4,4), hố ở (2,2). Phần thưởng: +10 (đích), -10 (hố), -1 (mỗi bước).

- Có bao nhiêu trạng thái? Có bao nhiêu chính sách khả thi?
 - Với $\gamma = 0.9$, robot sẽ ưu tiên một đường dài an toàn hay một đường ngắn rủi ro? Hãy biện minh.
 - Viết công thức cập nhật của Q-learning cho trạng thái (1,1), hành động “sang phải”, dẫn đến (2,1) với phần thưởng -1.
-

Chương 10

Mô hình tạo sinh và học sâu

Điều tôi không thể tạo ra, tôi không hiểu.

Richard Feynman

10.1 Giới thiệu về AI tạo sinh

Nhà vật lý Richard Feynman từng viết nổi tiếng trên bảng đen của ông: “Điều tôi không thể tạo ra, tôi không hiểu.” Nguyên tắc này nắm bắt bản chất của trí tuệ nhân tạo tạo sinh, nhưng ở dạng đảo ngược: các mô hình tạo sinh hiện thực hóa triết lý rằng *điều gì chúng hiểu, chúng có thể tạo ra*.

Ở cốt lõi, AI tạo sinh là dạy máy tạo ra nội dung mới (ảnh, văn bản, âm nhạc, hay bất kỳ dạng dữ liệu nào khác) tương tự những gì chúng đã học từ các ví dụ. Khi bạn đưa cho một mô hình tạo sinh hàng nghìn bức ảnh mèo, nó không đơn thuần học thuộc lòng chúng. Thay vào đó, nó học *điều gì khiến một con mèo trông giống một con mèo*: hình dáng chung, đôi tai nhọn, bộ ria mép, các hoa văn trên lông. Với hiểu biết đó, mô hình có thể tạo ra những hình mèo hoàn toàn mới, chưa từng tồn tại trước đây.

Ý chính

Các mô hình tạo sinh học được các mẫu hình và phân phối nền tảng của dữ liệu huấn luyện, nhờ đó chúng có thể tạo ra những mẫu mới tương tự (nhưng không phải là bản sao của) dữ liệu gốc.

AI tạo sinh hiện đang hỗ trợ nhiều ứng dụng mà có lẽ bạn đã dùng: trình tạo

ảnh như DALL-E và Stable Diffusion tạo tranh từ mô tả văn bản, các mô hình ngôn ngữ lớn như ChatGPT viết nên những đoạn văn mạch lạc, các công cụ soạn nhạc, và các hệ thống tạo video. Việc hiểu những hệ thống này hoạt động thế nào là điều thiết yếu với bất kỳ ai đang học AI hiện đại.

Hướng dẫn học: Những gì cần Thành thạo so với Chỉ cần Nhận biết

Chương này đề cập nhiều kiến trúc khác nhau. Không kiến trúc nào cũng đòi hỏi cùng mức độ hiểu biết:

Hiểu ở mức trực giác (giải thích được bằng lời của chính mình):

- Sự phân biệt giữa mô hình tạo sinh và mô hình phân biệt
- Không gian tiềm ẩn (latent space) có nghĩa gì và vì sao nó hữu ích
- Ý tưởng cơ bản về mã hóa và giải mã

Hiểu về mặt kỹ thuật (làm được với toán):

- Hàm mất mát của VAE: tái tạo + phân kỳ KL
- Tại sao VAE lấy mẫu từ phân phối (mẹo tái tham số hóa) *GAN*, *mô hình khuếch tán* và *Transformer* được nói tới ở Chương 14 (phần mở rộng). *Hãy nắm khái niệm, đừng học thuộc toán.*

Phạm vi chương: Các mô hình tạo sinh kinh điển

Chương này đề cập **các kiến trúc tạo sinh nền tảng**: Autoencoder và VAE. Đây là những kiến trúc dễ xuất hiện trên IAIO hơn vì chúng có công thức toán rõ ràng.

GAN, mô hình khuếch tán, Transformer và LLM được nói tới ở Chương 14. Đó là các chủ đề mở rộng phục vụ cho việc hiểu khái niệm.

10.2 Mô hình tạo sinh và mô hình phân biệt

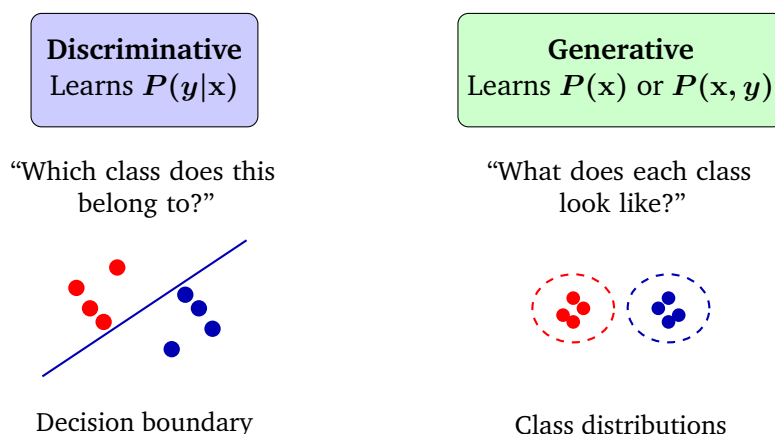
Trước khi đi sâu vào các kiến trúc cụ thể, điều quan trọng là phải hiểu sự phân biệt nền tảng giữa hai trường phái trong học máy: **mô hình phân biệt** và **mô hình tạo sinh**. Sự phân biệt này quan trọng vì hai trường phái khác nhau về bản chất trong cách chúng được huấn luyện, cách chúng thất bại, và cách chúng được đánh giá.

10.2.1 Mô hình phân biệt

Các mô hình phân biệt tập trung vào việc học *ranh giới* giữa các lớp. Với dữ liệu đầu vào x và nhãn y , một mô hình phân biệt học xác suất có điều kiện $p(y|x)$. Xác suất của một nhãn *khi cho* đầu vào.

Hãy nghĩ thế này: nếu bạn đang huấn luyện một mô hình phân biệt để phân biệt

mèo với chó, nó sẽ học cách vẽ một đường phân cách quyết định giữa hai lớp. Khi có một ảnh mới đến, mô hình xác định ảnh rơi vào phía nào của đường phân cách.



Hình 10.1: Mô hình phân biệt học các đường phân cách quyết định; mô hình tạo sinh học mỗi lớp trông như thế nào.

10.2.2 Các mô hình tạo sinh

Các mô hình tạo sinh tiếp cận theo một cách hoàn toàn khác. Thay vì học ranh giới, chúng học *dữ liệu trông như thế nào*, tức phân phối xác suất nền tảng $P(x)$. Với các bài toán phân loại, chúng có thể học $P(x|y)$ (mèo trông như thế nào, chó trông như thế nào) rồi dùng định lý Bayes để phân loại.

Ưu điểm chính của mô hình tạo sinh là khả năng tạo ra dữ liệu mới. Khi một mô hình đã hiểu mèo trông như thế nào, nó có thể sinh ra những ảnh mèo mới, hợp lý. Điều này bất khả thi với các mô hình thuần phân biệt, vốn chỉ biết vẽ ranh giới.

Khía cạnh	Phân biệt	Tạo sinh
Học	Ranh giới giữa các lớp	Phân phối của từng lớp
Đầu ra	Nhãn lớp hoặc xác suất	Mẫu dữ liệu mới
Câu hỏi được trả lời	“Đây là mèo hay chó?”	“Một con mèo trông như thế nào?”
Ví dụ	Hồi quy logistic, SVM, các bộ phân loại CNNVAE, GAN, mô hình khuếch tán	

Bảng 10.1: So sánh phương pháp phân biệt và tạo sinh

es.

10.3 Không gian tiềm ẩn: Trái tim của các mô hình tạo sinh

Một khái niệm then chốt trong mô hình tạo sinh là **không gian tiềm ẩn**, một biểu diễn có chiều thấp hơn, nắm bắt các đặc trưng cốt lõi của dữ liệu chiều cao. Hiểu không gian tiềm ẩn là chìa khóa để hiểu cách các mô hình tạo sinh hoạt động.

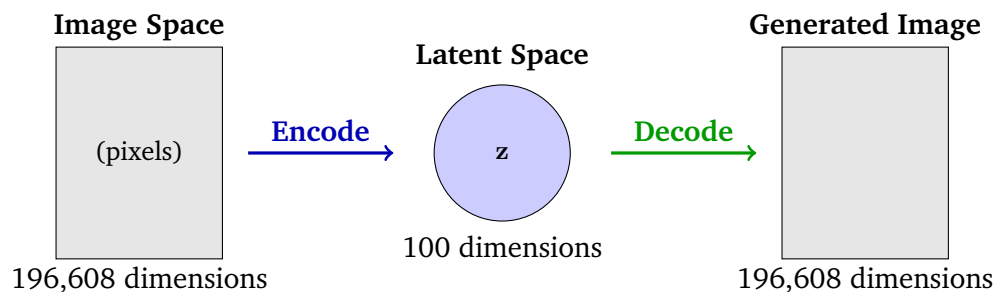
10.3.1 Vấn đề của chiều cao

Xét một ảnh khiếm tốn gồm 256×256 pixel với ba kênh màu (RGB). Chỉ một ảnh như vậy đã chứa $256 \times 256 \times 3 = 196,608$ giá trị. Một tập dữ liệu huấn luyện có thể chứa hàng triệu ảnh như thế. Xử lý trực tiếp dữ liệu ở quy mô này là điều vượt quá khả năng tính toán và kém hiệu quả.

Về bản chất hơn, không phải mọi tổ hợp giá trị pixel đều có ý nghĩa. Không gian của tất cả các ảnh RGB 256×256 có thể có là khổng lồ đến phi thường, nhưng chỉ một phần rất nhỏ của không gian đó chứa những ảnh có ý nghĩa (khuôn mặt, phong cảnh, đồ vật). Phần còn lại chỉ là nhiễu ngẫu nhiên.

10.3.2 Nén thành các biểu diễn có ý nghĩa

Không gian tiềm ẩn đưa ra giải pháp bằng cách mã hóa dữ liệu chiều cao thành một biểu diễn gọn gàng, có chiều thấp hơn, chỉ nắm bắt *đúng những gì quan trọng*.



Hình 10.2: Không gian tiềm ẩn nén dữ liệu chiều cao thành một biểu diễn gọn gàng, có ý nghĩa.

10.3.3 Vì sao không gian tiềm ẩn hiệu quả

Không gian tiềm ẩn hoạt động được nhờ một ý tưởng then chốt: những thứ tương tự nên ở gần nhau. Nếu hai ảnh đều vẽ mèo vằn, biểu diễn tiềm ẩn của chúng nên ở gần nhau. Nếu ảnh này là mèo còn ảnh kia là ô tô, chúng nên ở xa nhau.

Tổ chức này tạo ra những khả năng mạnh mẽ:

- **Nội suy:** Di chuyển mượt mà giữa hai điểm trong không gian tiềm ẩn tạo ra các chuyển tiếp mượt mà giữa các ảnh tương ứng. Di chuyển từ “mèo” sang “chó” trong không gian tiềm ẩn có thể cho thấy những sinh vật trung gian.
- **Số học:** Trong một không gian tiềm ẩn được huấn luyện tốt, các phép toán ngữ nghĩa trở nên khả thi. Ví dụ nổi tiếng: king — man + woman $\approx$ queen.
- **Sinh dữ liệu:** Lấy mẫu ngẫu nhiên các điểm từ không gian tiềm ẩn rồi giải mã chúng sẽ cho ra những đầu ra mới, chân thực.

Ảnh dụ: Cách chúng ta ghi nhớ

Hãy nghĩ xem bạn nhớ một con chó thế nào. Bạn không lưu lại từng pixel của mọi con chó bạn từng thấy. Thay vào đó, bạn giữ một biểu diễn trừu tượng nắm bắt các đặc trưng cốt lõi: bốn chân, lông, đuôi, mũi ướt. Đây chính xác là những gì không gian tiềm ẩn làm: nó nắm bắt *bản chất* trong khi bỏ qua các chi tiết không liên quan.

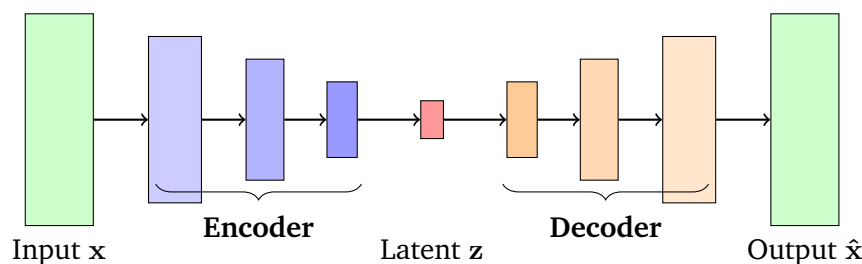
10.4 Autoencoder: Học cách nén và tái tạo

Autoencoder là các kiến trúc mạng nơ-ron học được những biểu diễn hiệu quả của dữ liệu bằng cách được huấn luyện để tái tạo lại chính đầu vào của chúng. Chúng là nền tảng cho nhiều mô hình tạo sinh.

10.4.1 Kiến trúc

Một autoencoder gồm hai thành phần chính:

1. **Bộ mã hóa:** Nén đầu vào x thành một biểu diễn tiềm ẩn có chiều thấp hơn z
2. **Bộ giải mã:** Tái tạo lại đầu vào từ biểu diễn tiềm ẩn, cho ra đầu ra $\hat{x}$



Hình 10.3: Kiến trúc autoencoder: dữ liệu chảy qua một cổ chai buộc mọi thứ phải được nén lại.

10.4.2 Mục tiêu huấn luyện

Autoencoder được huấn luyện để cực tiểu hóa **sai số tái tạo**, tức là phần khác biệt giữa đầu vào x và đầu ra đã tái tạo $\hat{x}$:

$$\mathcal{L} = \|x - \hat{x}\|^2 = \sum_i (x_i - \hat{x}_i)^2 \quad (10.1)$$

Bằng cách ép thông tin đi qua một *cổ chai* (không gian tiềm ẩn nhỏ), autoencoder buộc phải học cách chỉ nắm bắt những đặc trưng thiết yếu nhất. Thông tin thừa hoặc nhiễu không đi lọt qua cổ chai nên bị loại bỏ.

10.4.3 Ứng dụng của autoencoder

Ngoài việc học biểu diễn, autoencoder còn hỗ trợ nhiều ứng dụng thực tế. Trong **khử nhiễu**, ta huấn luyện autoencoder trên đầu vào có nhiễu nhưng dùng ảnh sạch làm mục tiêu tái tạo. Mạng học cách loại bỏ nhiễu trong khi giữ lại tín hiệu thật; nó không thể đơn thuần học thuộc nhiễu, nên bắt buộc phải học cấu trúc nền tảng.

Phát hiện bất thường dựa trên thực tế rằng autoencoder tái tạo dữ liệu thường tốt nhưng lại gặp khó với đầu vào bất thường. Nếu bạn huấn luyện một autoencoder trên ảnh các linh kiện sản xuất bình thường, một linh kiện lỗi sẽ cho sai số tái tạo cao vì nó không khớp với các mẫu mà mạng đã học. Sai số này đánh dấu bất thường mà không cần đến bất kỳ mẫu lỗi nào có nhãn.

Bản thân không gian tiềm ẩn đã cho **khả năng nén ảnh**: thay vì lưu cả ảnh chiều cao, bạn chỉ lưu vector tiềm ẩn gọn gàng và giải mã khi cần. Cuối cùng, kiến trúc mã hóa-giải mã còn mở rộng sang **phân đoạn ngữ nghĩa**, nơi bộ giải mã xuất ra không phải một bản tái tạo mà là nhãn phân loại cho từng pixel trong ảnh.

10.5 Autoencoder biến thiên (VAE)

Autoencoder tiêu chuẩn có một hạn chế: không gian tiềm ẩn của chúng không được tổ chức theo cách hỗ trợ việc sinh ra nội dung có ý nghĩa. Những điểm lấy ngẫu nhiên từ không gian tiềm ẩn thường giải mã ra rác. **Autoencoder biến thiên (VAE)** sửa vấn đề này bằng cách mã hóa đầu vào thành *các phân phối xác suất* thay vì các điểm cố định.

10.5.1 Từ điểm đến phân phối

Trong autoencoder tiêu chuẩn, bộ mã hóa tạo ra một vector tiềm ẩn duy nhất z cho mỗi đầu vào. Trong VAE, bộ mã hóa tạo ra hai đầu ra: μ , đại diện cho kỳ vọng của một phân phối Gauss, và σ , đại diện cho độ lệch chuẩn của phân phối đó. Cộng lại, chúng xác định một phân phối xác suất trong không gian tiềm ẩn thay vì một điểm đơn lẻ.

Trong lúc huấn luyện, thay vì dùng một z cố định, chúng ta *lấy mẫu* từ phân phối này:

$$z = \mu + \sigma \odot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \mathbf{I}) \quad (10.2)$$

Điều này được gọi là **mạo tái tham số hóa**. Số hạng ϵ đưa vào tính ngẫu nhiên, nhưng điểm quan trọng là tính ngẫu nhiên đó không phụ thuộc vào các tham số có thể học, nên gradient vẫn luân chuyển được trong quá trình huấn luyện.

10.5.2 Mục tiêu huấn luyện của VAE

VAE tối ưu hóa đồng thời hai mục tiêu. **Sai số tái tạo** bảo đảm đầu ra sau giải mã khớp với đầu vào, cũng như trong autoencoder thường. Số hạng **phân kỳ KL** bảo đảm các phân phối đã học vẫn ở gần một phân phối chuẩn tắc $\mathcal{N}(0, \mathbf{I})$.

$$\mathcal{L}_{\text{VAE}} = \underbrace{\|x - \hat{x}\|^2}_{\text{reconstruction}} + \underbrace{D_{\text{KL}}(q(z|x) \parallel \mathcal{N}(0, \mathbf{I}))}_{\text{regularization}} \quad (10.3)$$

Số hạng phân kỳ KL là then chốt: nó bảo đảm không gian tiềm ẩn được sắp xếp tốt và liên tục. Không có nó, bộ mã hóa có thể ánh xạ các đầu vào khác nhau tới những vùng tiềm ẩn tách rời, rải rác, với các khoảng trống ở giữa. Việc sinh dữ liệu sẽ không đáng tin vì các điểm lấy ngẫu nhiên có thể rơi vào các khoảng trống đó và giải mã ra những đầu ra vô nghĩa. Bằng cách ép mọi phân phối được mã hóa phải ở gần phân phối chuẩn tắc, số hạng KL bảo đảm toàn bộ không gian tiềm ẩn được “lấp đầy” và có ý nghĩa; bất kỳ điểm nào bạn lấy mẫu cũng sẽ giải mã ra thứ gì đó hợp lý.

10.5.3 Sinh dữ liệu với VAE

Sau khi huấn luyện, việc sinh mẫu mới rất đơn giản. Bạn lấy một vector tiềm ẩn z từ phân phối chuẩn tắc $\mathcal{N}(\mathbf{0}, \mathbf{I})$, đưa nó qua bộ giải mã, và đầu ra là một mẫu mới chưa từng có. Vì không gian tiềm ẩn đã được chuẩn hóa để khớp phân phối chuẩn tắc, nên các mẫu ngẫu nhiên từ phân phối đó cho ra những đầu ra có ý nghĩa.

10.6 Đánh giá mô hình tạo sinh

Trong học có giám sát, việc đánh giá rất trực tiếp: tính độ chính xác, độ chính xác (precision), hoặc AUC trên tập kiểm tra. Các mô hình tạo sinh không có được sự thuận lợi đó. Câu hỏi “những ảnh được tạo này tốt đến đâu?” không có một câu trả lời duy nhất, vì chất lượng có nhiều chiều cạnh tranh nhau.

Khả năng sinh (likelihood) so với chất lượng cảm nhận: Một mô hình có thể gán xác suất cao cho mọi ảnh huấn luyện (khả năng cao) trong khi chỉ sinh ra những ảnh mờ trung bình. VAE làm đúng vậy: bằng cách cực tiểu hóa sai số tái tạo trên mọi pixel, chúng tạo ra đầu ra hợp lý về thống kê nhưng nhòe về thị giác. Ngược lại, một mô hình có thể sinh ra ảnh sắc nét, chân thực trong khi bỏ qua những phần lớn của phân phối dữ liệu. Khả năng sinh (likelihood) và chất lượng thị giác là hai thứ khác nhau, và việc tối ưu cái này không bảo đảm cái kia.

Đa dạng so với độ trung thực: Một mô hình chỉ sinh đi sinh lại một khuôn mặt hoàn hảo có độ trung thực cao nhưng đa dạng bằng không. Một mô hình sinh mọi khuôn mặt có thể có, kể cả những khuôn mặt méo mó, có độ đa dạng cao nhưng độ trung thực thấp. Mô hình lý tưởng đạt điểm tốt ở cả hai, nhưng cải thiện cái này thường làm hại cái kia.

Những căng thẳng này khiến việc đánh giá về bản chất rất khó. Các chỉ số như Fréchet Inception Distance (FID) cố gắng nắm bắt cả chất lượng lẫn đa dạng, nhưng không con số đơn lẻ nào tóm gọn hết việc một mô hình tạo sinh có “tốt” hay không. Đánh giá bằng con người vẫn quan trọng nhưng tốn kém và chủ quan.

10.6.1 Các chế độ thất bại

Đầu ra mờ. VAE sinh ra ảnh mờ vì sai số tái tạo của chúng lấy trung bình trên mọi đầu ra có thể có với một mã tiềm ẩn cho trước. Khi mô hình không chắc một pixel nên sáng hay tối, nó thỏa hiệp bằng một giá trị xám. Kết quả trông nhạt nhòa. Đây là hệ quả trực tiếp của hàm mất mát, không phải lỗi triển khai có thể sửa được.

Sụp đổ hậu nghiệm. Bộ giải mã có thể học cách bỏ qua hoàn toàn mã tiềm ẩn, tạo ra những đầu ra không phụ thuộc phép mã hóa. Số hạng KL đẩy bộ mã hóa về phân phối chuẩn tắc, và nếu bộ giải mã đủ mạnh, nó thấy bỏ qua z còn dễ hơn là dùng nó. Kết quả là một mô hình

với một không gian tiềm ẩn được sắp xếp tốt nhưng bộ giải mã không hề dùng, khiến việc sinh dữ liệu có ý nghĩa trở nên bất khả thi.

Về các chế độ thất bại của GAN và mô hình khuếch tán, xem Chương 14.

10.6.2 Sự phụ thuộc vào tính toán và dữ liệu

Các kiến trúc trong chương này chỉ hoạt động được nhờ sức tính toán và dữ liệu ở mức khổng lồ. Một VAE huấn luyện trên 100 ảnh khuôn mặt sẽ học thuộc chúng; một VAE huấn luyện trên 10 triệu ảnh sẽ học được khuôn mặt nói chung trông như thế nào. Mẫu hình này cũng đúng với GAN và mô hình khuếch tán. Những mô hình tạo sinh hàng đầu được huấn luyện trên hàng tỷ ảnh, dùng hàng nghìn GPU trong nhiều tuần hoặc nhiều tháng. Nhiều kết quả nổi bật của AI tạo sinh không chuyển sang được môi trường tính toán thấp: một kỹ thuật cho ra ảnh chân thực với 1.000 GPU có thể chẳng ra gì hữu ích với một GPU.

Sự phụ thuộc này nối với các khái niệm từ những chương trước. Một mô hình huấn luyện trên quá ít dữ liệu sẽ quá khớp (Chương 5): nó học thuộc các ví dụ huấn luyện thay vì học phân phối nền tảng, và các mẫu sinh ra sẽ là bản sao hoặc gần như bản sao của dữ liệu huấn luyện. Những ý tưởng về độ phức tạp mẫu từ Chương 11 cũng áp dụng ở đây: phân phối dữ liệu bạn muốn mô hình hóa càng giàu thì bạn càng cần nhiều ví dụ. Các mô hình tạo sinh đầy yêu cầu này tới cực điểm vì chúng phải học toàn bộ phân phối, chứ không chỉ một đường phân cách quyết định.

10.7 Tóm tắt

Điểm cần nhớ

Tạo sinh so với Phân biệt: Mô hình phân biệt học $P(y|x)$ (các đường phân cách quyết định); mô hình tạo sinh học $P(x)$ (phân phối dữ liệu) và có thể tạo ra các mẫu mới.

Không gian tiềm ẩn: Biểu diễn nén nắm bắt các đặc trưng cốt lõi. Các mô hình tạo sinh ánh xạ qua lại giữa không gian dữ liệu và không gian tiềm ẩn. **Autoencoder:** Bộ mã hóa nén thành mã tiềm ẩn; bộ giải mã tái tạo lại. Tốt cho học biểu diễn nhưng không thể sinh ra mẫu mới một cách đáng tin cậy.

VAE: Mã hóa thành phân phối (không phải điểm). Phân kỳ KL bảo đảm không gian tiềm ẩn trơn tru. Huấn luyện ổn định và ước lượng mật độ tốt, nhưng đầu ra mờ.

Đánh giá: Không chỉ số nào nắm trọn chất lượng tạo sinh. Khả năng sinh (likelihood), chất lượng cảm nhận và đa dạng có thể mâu thuẫn nhau. Điều này khiến đánh giá mô hình tạo sinh về bản chất khó hơn nhiều so với phân loại. **Quy mô quan trọng:** Các phương pháp này đòi hỏi tập dữ liệu lớn và tính toán đáng kể. Dữ liệu ít sẽ dẫn đến học thuộc, không phải sinh dữ liệu.

10.8 Bài tập luyện

- Giải thích sự khác biệt giữa mô hình tạo sinh và mô hình phân biệt. Cho ví dụ của từng loại cho bài toán nhận dạng chữ số.
- Một autoencoder có bộ mã hóa $f: \mathbb{R}^{784} \rightarrow \mathbb{R}^{32}$ và bộ giải mã $g: \mathbb{R}^{32} \rightarrow \mathbb{R}^{784}$.
 - Tỉ số nén là bao nhiêu?
 - Tại sao ta không thể sinh ảnh mới bằng cách lấy mẫu các vector 32 chiều ngẫu nhiên?
- Hàm mất mát của VAE là: $\mathcal{L} = \mathbb{E}[\|x - \hat{x}\|^2] + \beta \cdot \text{KL}(q(z|x)||p(z))$
 - Mỗi số hạng khuyến khích điều gì?
 - Điều gì xảy ra nếu $\beta = 0$? Nếu $\beta \rightarrow \infty$? (c) Tại sao số hạng KL lại cần thiết cho việc sinh dữ liệu?
- Một mô hình tạo sinh huấn luyện trên 10.000 ảnh khuôn mặt cho ra các đầu ra trông giống hệt những ví dụ huấn luyện cụ thể. Mô hình thứ hai huấn luyện trên 10 triệu ảnh lại sinh ra những khuôn mặt mới.

(a) Giải thích sự khác biệt này theo khái niệm quá khớp (Chương 5). (b) Bạn sẽ kiểm tra thế nào để biết một mô hình tạo sinh đang học thuộc chứ không phải khái quát hóa?

5. **(Về khái niệm)** VAE cho ra ảnh mờ. Giải thích tại sao, và liên hệ câu trả lời của bạn với số hạng sai số tái tạo trong mục tiêu của VAE.

Phần IV

Lý thuyết

Chương 11

Lý thuyết học thống kê

Khi giải một bài toán đáng quan tâm, đừng giải một bài toán tổng quát hơn như một bước trung gian.

Vladimir Vapnik

Hướng dẫn độ khó của chương

Chương này chứa tài liệu ở nhiều mức độ khó: **IAIO cốt lõi** (ai cũng cần phải nắm):

- Định nghĩa và trực giác về học PAC (11.1–11.3)
- Cách sử dụng chặn trên độ phức tạp mẫu (11.5)
- Tính học được của lớp giả thuyết hữu hạn (11.4)

Trung cấp (cần thiết cho điểm cạnh tranh):

- Tự suy ra các chặn từ những nguyên lý cơ bản
- Học PAC agnostic (11.6)
- Đánh đổi thiên vị–độ phức tạp

Nâng cao / Cấp huy chương:

- Chứng minh Định lý Nền tảng
- Chặn chặt và chặn dưới
- Liên hệ với nén thông tin và độ dài mô tả

Pro Các phần chứng minh được đánh dấu bằng ★ chỉ là bản đạo. Hiểu được ý tưởng là đủ đối với hầu hết học sinh.

Những gì bạn cần có khả năng làm

tự suy ra từ đầu:

- chặn trên độ phức tạp mẫu cho các lớp giả thuyết hữu hạn
- tại sao $m \geq \frac{1}{\epsilon} (\ln |\mathcal{H}| + \ln \frac{1}{\delta})$ suy ra từ bất đẳng thức hợp nhất

giải thích nhưng không cần suy ra:

- tại sao học PAC đòi hỏi cả ϵ (độ chính xác) lẫn δ (độ tin cậy)
- tại sao giả thiết khả thi lại quan trọng
- mối liên hệ giữa quá khớp và kích thước của lớp giả thuyết

nêu và áp dụng (công thức đã cho, dùng nó):

- cho $|\mathcal{H}|$, ϵ , δ , tính cỡ mẫu cần thiết
- cho m , $|\mathcal{H}|$, δ , tính giá trị đạt được ϵ

11.1 Giới thiệu: Lý thuyết đứng sau quá trình học

Suốt cuốn sách này chúng ta đã gặp rất nhiều thuật toán học máy, và chúng hoạt động cực kỳ tốt trong thực tế. Nhưng một câu hỏi nền tảng vẫn còn lại: **tại sao việc học từ các ví dụ lại hiệu quả chứ?**

Dưới đây là một trường hợp thất bại cụ thể gợi mở câu hỏi đó. Bạn huấn luyện một bộ lọc thư rác trên 50 email và nó phân loại đúng cả 50. Bạn triển khai nó, và nó phân loại sai một nửa các email mới. Bạn bổ sung thêm dữ liệu, huấn luyện trên 500 email, và vấn đề gần như biến mất hẳn. Tại sao 50 ví dụ lại thất bại? Tại sao 500 lại đủ? Có con số nào ta tính trước được để biết 50 là chưa đủ không? Chương này trả lời chính xác các câu hỏi đó.

Hãy xét xem ta đang yêu cầu máy làm gì. Ta đưa cho một thuật toán một số hữu hạn các ví dụ. Có thể là hàng nghìn ảnh đã gán nhãn, rồi kỳ vọng rằng nó phân loại đúng những ảnh chưa từng thấy. Dữ liệu huấn luyện chỉ là một mẫu nhỏ xíu từ một vũ trụ vô tận các khả năng. Làm sao những mẫu hình được phát hiện trong mẫu nhỏ này lại khái quát hóa được ra phần chưa biết khổng lồ?

Điều này không chỉ mang tính triết học. Hiểu tại sao việc học lại hiệu quả sẽ cho ta biết khi nào nó thất bại, cần bao nhiêu dữ liệu, và tại sao một số mô hình khái quát hóa tốt trong khi số khác lại quá khớp. Lý thuyết học thống kê cung cấp những câu trả lời nghiêm túc cho tất cả các câu hỏi đó.

Những câu hỏi trung tâm

Độ phức tạp mẫu: Cần bao nhiêu ví dụ huấn luyện để học một cách đáng tin cậy?

Tính học được: Điều gì khiến bài toán này học được mà bài kia thì bất khả thi?

Khái quát hóa: Tại sao có mô hình khái quát hóa tốt, còn mô hình khác chỉ học thuộc?

11.2 Câu chuyện của hai loài học: Chuột và Bò câu

Trước khi đi sâu vào toán học, hãy xét hai ví dụ từ tâm lý học soi sáng bản chất của quá trình học.

11.2.1 Những con chuột khôn ngoan

Khi các nhà nghiên cứu cố diệt chuột bằng bả mồi có độc, họ phát hiện một điều đáng kinh ngạc. Nếu một con chuột ăn phải thức ăn lạ rồi bị ốm vài giờ sau, nó học cách tránh thức ăn đó *chỉ từ một lần trải nghiệm*. Nhưng điều then chốt là chuột chỉ liên hệ bệnh tật với *vị và mùi* của thức ăn. Nếu các nhà nghiên cứu rung chuông mỗi khi chuột tiến lại gần bả mồi có độc, chuột sẽ không học cách tránh chiếc chuông. Tiến hóa đã lập trình cho chúng liên kết bệnh tật với các tính chất của thức ăn, chứ không phải với những âm thanh bất kỳ.

11.2.2 Những con bồ câu mê tín

B.F. Skinner đặt những con bồ câu đói vào các lồng có đồ chơi đủ kiểu và cho ăn theo những khoảng thời gian ngẫu nhiên, hoàn toàn độc lập với hành vi của bồ câu.¹

Mỗi con bồ câu bám vào điều nó tình cờ đang làm vào thời điểm thức ăn xuất hiện: một con mổ một nút bấm, một con khác xoay tròn, một con nữa gập đầu. Chúng hình thành “mê tín”, coi những sự trùng hợp là quy luật nhân quả. Bồ câu không có bất kỳ kiến thức tiên quyết nào về thứ có thể gây ra thức ăn, nên *mọi* hành vi đều là một lời giải thích khả dĩ như nhau. Với quá nhiều nguyên nhân ứng viên như vậy, chúng chắc chắn sẽ tìm ra những tương quan giả.

¹Skinner, B. F. (1948). “Superstition” in the pigeon. *Journal of Experimental Psychology*, 38(2), 168–172.

Bài học cho học máy

Những con chuột thành công vì tiến hóa đã giới hạn lớp giả thuyết của chúng: khi bị ốm, chỉ xét các nguyên nhân liên quan đến thức ăn. Một tập hợp ứng viên nhỏ và được chọn cẩn thận.

Những con bồ câu thất bại vì lớp giả thuyết của chúng không bị giới hạn: bất kỳ hành vi nào cũng có thể là nguyên nhân. Với quá nhiều lời giải thích ứng viên và quá ít dữ liệu, chúng chỉ khớp vào nhiễu thay vì tín hiệu.

Ý chính: Một lớp giả thuyết nhỏ hơn cần ít ví dụ hơn để học một cách đáng tin cậy. Một lớp giả thuyết không bị giới hạn sẽ quá khớp, tìm ra những mẫu hình không có thật. Đây chính là vấn đề quá khớp ở Chương 5, và chương này sẽ đưa ra cách phát biểu toán học chính xác cho nó.

11.3 Khung học chính thức

Hãy phát biểu chính thức những trực giác này. Trong học có giám sát, ta có một **miền** $\mathcal{X}$ (tập hợp mọi đầu vào có thể có), một **tập nhãn** $\mathcal{Y}$ (tập hợp các đầu ra có thể có), và một **mẫu huấn luyện** $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$.

Lớp **giả thuyết** $\mathcal{H}$ đại diện cho kiến thức tiên quyết của ta, tức là tập hợp các hàm mà ta xem là ứng viên. Giống như chuột chỉ xét các giả thuyết liên quan đến thức ăn, lựa chọn $\mathcal{H}$ của ta ràng buộc những mẫu hình mà thuật toán có thể phát hiện.

Một **thuật toán học** nhận dữ liệu huấn luyện và đưa ra một giả thuyết $h \in \mathcal{H}$. Mục tiêu là tìm một giả thuyết khái quát hóa được sang các ví dụ mới.

11.3.1 Ví dụ về quả đu đủ

Hãy tưởng tượng việc học quả đu đủ nào ăn được bằng cách đo màu sắc và độ mềm. Ở đây $\mathcal{X} = \mathbb{R}^2$ (các cặp màu sắc, độ mềm), $\mathcal{Y} = \{0, 1\}$ (không ăn được, ăn được). Với $\mathcal{H}$, ta có thể chọn các hình chữ nhật song song với trục tọa độ: dự đoán “ăn được” nếu các đặc trưng rơi vào bên trong hình chữ nhật.

Tại sao lại là hình chữ nhật? Điều này mã hóa kiến thức tiên quyết rằng vị ngon phụ thuộc vào việc màu sắc và độ mềm nằm trong những khoảng nhất định. Ràng buộc này ngăn việc học những mẫu hình giả không thể khái quát hóa.

11.4 Đo lường thành công: Sai số thật so với sai số huấn luyện

Sai số thật

Sai số thật của giả thuyết h so với phân phối $\mathcal{D}$ là:

$$L_{\mathcal{D}}(h) = \Pr_{x \sim \mathcal{D}}[h(x) \neq f(x)]$$

Đây là xác suất phân loại sai trên một ví dụ **mới, ngẫu nhiên**. Đó mới là điều ta thực sự quan tâm.

Vấn đề là: ta không thể tính sai số thật vì ta không biết $\mathcal{D}$. Ta chỉ có thể tính **sai số huấn luyện**:

$$l_s(h) = \frac{1}{m} \sum_{i=1}^m 1[h(x_i) \neq y_i]$$

Câu hỏi trung tâm: *khi nào sai số huấn luyện thấp kéo theo sai số thật thấp?*

11.5 Học PAC: Chắc chắn, Gần đúng

Khung PAC, do Leslie Valiant đề xuất năm 1984,² đã hình thức hóa điều kiện để việc học là khả thi.

11.5.1 Giả thiết khả thi

Trước hết, ta giả thiết rằng tồn tại $h^* \in \mathcal{H}$ thỏa mãn $L_{\mathcal{D}}(h^*) = 0$. Hàm gán nhãn thật nằm trong lớp giả thuyết của ta. Điều này giống như việc chuột biết rằng bệnh tật có liên quan đến thức ăn.

Tính học PAC

Một lớp giả thuyết $\mathcal{H}$ là **học PAC được** nếu tồn tại một hàm độ phức tạp mẫu $m_{\mathcal{H}}(\epsilon, \delta)$ và một thuật toán học sao cho: với mọi phân phối $\mathcal{D}$ và mọi $\epsilon, \delta \in (0, 1)$, nếu ta rút $m \geq m_{\mathcal{H}}(\epsilon, \delta)$ mẫu, thì với xác suất ít nhất là $1 - \delta$:

$$L_{\mathcal{D}}(h) \leq \epsilon$$

Nói đơn giản: Với đủ mẫu, ta có thể **chắc chắn** ($1 - \delta$) tìm được một giả thuyết **gần đúng** (ϵ).

²Valiant, L. G. (1984). A theory of the learnable. *Communications of the ACM*, 27(11), 1134–1142.

11.5.2 Độ phức tạp mẫu với các lớp hữu hạn

Định lý: Các lớp hữu hạn đều học PAC được

Nếu $\mathcal{H}$ hữu hạn và giả thiết khả thi đúng, thì:

$$m_{\mathcal{H}}(\epsilon, \delta) \leq \frac{\ln |\mathcal{H}| + \ln(1/\delta)}{\epsilon}$$

Điều này cho ta biết: nhiều giả thuyết hơn ($|\mathcal{H}|$ lớn hơn) đòi hỏi nhiều mẫu hơn; độ chính xác cao hơn (ϵ nhỏ hơn) đòi hỏi nhiều mẫu hơn; độ tin cậy cao hơn (δ nhỏ hơn) đòi hỏi nhiều mẫu hơn.

Chặn trên này thực sự cho ta biết điều gì?

Chặn trên nói rằng m tăng theo $\ln |\mathcal{H}|$, chứ không phải theo $|\mathcal{H}|$. Sự phụ thuộc lôgarit này là ý tưởng thực hành then chốt: nhân đôi lớp giả thuyết chỉ làm tăng yêu cầu số mẫu thêm $\ln 2 \approx 0.7$. Ngay cả một lớp giả thuyết có 10^{100} phần tử cũng chỉ cần khoảng $230/\epsilon$ mẫu (cộng thêm số hạng độ tin cậy). Các lớp giả thuyết lớn học được nhiều hơn về ngoài của chúng.

Ví dụ minh họa: Giả sử $|\mathcal{H}| = 10,000$, $\epsilon = 0.05$, $\delta = 0.05$. Khi đó $m \geq \frac{\ln 10,000 + \ln 20}{0.05} = \frac{9.21 + 3.00}{0.05} \approx 244$ mẫu. Để có 95% độ tin cậy rằng sai số thật nhỏ hơn 5%, ta cần ít hơn 250 ví dụ huấn luyện.

Hạn chế của các chặn PAC trong thực tế. Lý thuyết PAC chứng minh rằng việc học là *khả thi* và xác định các mối quan hệ định tính đúng: nhiều giả thuyết hơn đòi hỏi nhiều dữ liệu hơn, độ chính xác cao hơn đòi hỏi nhiều dữ liệu hơn. Nhưng các chặn thường có tính bi quan. Chặn ở trên có thể nói rằng 244 mẫu là đủ, trong khi thực tế 50 mẫu có thể lại dùng được bởi vì dữ liệu có cấu trúc mà phân tích trường hợp xấu nhất bỏ qua. Các chặn PAC đúng với *mọi* phân phối, kể cả phân phối đối kháng. Các phân phối dữ liệu thực tế có tính quy tắc (độ mượt, các cụm, số chiều nội tại thấp) mà các thuật toán tận dụng được nhưng lý thuyết không tính đến. Hãy dùng các chặn PAC để suy luận định tính về điều gì làm cho bài toán khó hơn hay dễ hơn, chứ không dùng chúng làm đặc tả kỹ thuật về việc phải thu thập chính xác bao nhiêu dữ liệu.

11.6 Học PAC Agnostic

Giả thiết khả thi thường không thực tế. Nếu không có giả thuyết nào trong $\mathcal{H}$ là hoàn hảo thì sao?

Học PAC Agnostic

Một lớp $\mathcal{H}$ là **học được theo nghĩa agnostic PAC** nếu với đủ mẫu, với xác suất cao:

$$L_{\mathcal{D}}(h) \leq \min_{h' \in \mathcal{H}} L_{\mathcal{D}}(h') + \epsilon$$

Ta được bảo đảm **cạnh tranh được với giả thuyết tốt nhất trong $\mathcal{H}$** , chứ không phải là hoàn hảo.

11.6.1 Đánh đổi thiên vị–ước lượng

Điều này cho thấy hai nguồn sai số. **Sai số thiên vị** là sai số xuất phát từ việc giả thuyết tốt nhất trong $\mathcal{H}$ không hoàn hảo, tức là kiến thức tiên quyết của ta không hoàn hảo. **Sai số ước lượng** xuất phát từ việc ta chỉ thấy một mẫu hữu hạn, chứ không phải toàn bộ phân phối.

Có một sự đánh đổi nền tảng: $\mathcal{H}$ lớn hơn thì giảm sai số thiên vị (có nhiều giả thuyết hơn để lựa chọn) nhưng lại làm tăng sai số ước lượng (khó xác định giả thuyết tốt nhất giữa nhiều ứng viên). $\mathcal{H}$ nhỏ hơn thì tác dụng ngược lại. Đây chính là nền tảng lý thuyết của sự đánh đổi quá khớp/thiếu khớp ở Chương 5, giờ được phát biểu chính xác: đánh đổi thiên vị–phương sai không chỉ là một quan sát thực nghiệm mà là điều tất yếu về mặt toán học. Không thuật toán nào tránh khỏi điều đó.

11.7 Hội tụ đều

Làm sao chứng minh được rằng việc học là hiệu quả? Bằng **hội tụ đều**: chứng minh rằng sai số huấn luyện xấp xỉ sai số thật *đồng thời với mọi giả thuyết* trong $\mathcal{H}$.

 ϵ -Mẫu đại diện

Một mẫu S được gọi là **ϵ -đại diện** nếu:

$$\forall h \in \mathcal{H} : |L_S(h) - L_{\mathcal{D}}(h)| \leq \epsilon$$

Sai số huấn luyện xấp xỉ sai số thật cho **mọi** giả thuyết một cách đồng thời.

Tại sao điều này quan trọng? Nếu S là đại diện, thì thuật toán **Cực tiểu hóa Rủi ro Kinh nghiệm** (ERM) (đơn giản chỉ là chọn giả thuyết có sai số huấn luyện nhỏ nhất) được bảo đảm là sẽ hoạt động tốt. Tìm giả thuyết tốt nhất trên S cho ta (xấp xỉ) giả thuyết tốt nhất nói chung.

11.8 Tóm tắt

Điểm cần nhớ

Kiến thức tiên quyết là thiết yếu: Không có nó (giống như bồ câu), ta chỉ có mê tín chứ không phải học. Lớp giả thuyết $\mathcal{H}$ mã hóa kiến thức tiên quyết của ta.

Học PAC: Hình thức hóa việc học “Chắc chắn, Gần đúng”. Độ phức tạp mẫu phụ thuộc vào $|\mathcal{H}|$, độ chính xác mong muốn ϵ và độ tin cậy δ . Các lớp hữu hạn luôn luôn học PAC được. **Học Agnostic:** Bỏ giả thiết khả thi. Ta chỉ có thể bảo đảm cạnh tranh được với giả thuyết tốt nhất trong $\mathcal{H}$.

Thiên vị so với ước lượng: $\mathcal{H}$ lớn thì có sai số thiên vị thấp nhưng sai số ước lượng cao (nguy cơ quá khớp). $\mathcal{H}$ nhỏ thì có sai số thiên vị cao nhưng sai số ước lượng thấp (nguy cơ thiếu khớp).

Hội tụ đều: Khi sai số huấn luyện xấp xỉ sai số thật với mọi giả thuyết, ERM sẽ thành công.

11.9 Bài tập luyện

1. Một lớp giả thuyết có 1000 giả thuyết. Dùng chặn PAC, cần bao nhiêu mẫu cho $\epsilon = 0.1$ và $\delta = 0.05$?
2. Giải thích vì sao bồ câu lại hình thành hành vi “mê tín” còn chuột thì học đúng. Kiến thức tiên quyết đóng vai trò gì?
3. Trong ví dụ về quả đu đủ, tại sao chọn hình chữ nhật làm lớp giả thuyết của ta? Nếu dùng mọi hàm có thể thì sẽ ra sao?
4. Sự khác biệt giữa sai số huấn luyện $L_S(h)$ và sai số thật $L_{\mathcal{D}}(h)$ là gì? Vì sao ta không thể trực tiếp tính sai số thật?
5. Một mẫu là 0.05-đại diện. ERM tìm thấy $\hat{h}$ với $L_S(\hat{h}) = 0.02$. Bạn có thể nói gì về $L_{\mathcal{D}}(\hat{h})$?
6. Giải thích sự đánh đổi thiên vị-ước lượng. Điều này liên quan thế nào đến quá khớp và thiếu khớp?

³See Appendix A for properties of logarithms and exponentials used in these bounds.

Chương 12

Lý thuyết học thống kê nâng cao

Cái giá của tính tổng quát
là sự phức tạp.

John von Neumann

Hướng dẫn độ khó của chương

Chương này chủ yếu là tài liệu **nâng cao / cấp huy chương**.

IAI OCỐT LỖI (ai cũng cần phải nắm):

- Định nghĩa và ví dụ về chiều VC (12.2–12.3)
- Tính chiều VC cho các lớp giả thuyết đơn giản (khoảng, hình chữ nhật, bộ phân loại tuyến tính)

Nâng cao / Cấp huy chương:

- Biquyết Sauer và các hàm tăng trưởng
- Định lý Nền tảng của Lý thuyết Học thống kê (12.4)
- Định lý Không có bữa trưa miễn phí và những hệ quả của nó (12.5)
- Các phép chứng minh cho định lý chính

Lời khuyên ôn tập: Nếu bạn chỉ nhắm đến việc tham dự (không phải huy chương), hãy chỉ tập trung vào việc tính chiều VC cho các lớp giả thuyết tiêu chuẩn. Phần lý thuyết sâu hơn là dành cho những học sinh nhắm đến huy chương vàng/bạc. *Các phép chứng minh trong chương này chỉ là bản đạo, việc hiểu ý tưởng quan trọng hơn việc ghi nhớ chi tiết.*

Những gì bạn cần có khả năng làm

Tự tính từ đầu:

- Chiều VC của các khoảng trên $\mathbb{R}$ (đáp án: 2)
- Chiều VC của các hình chữ nhật song song trục trong $\mathbb{R}^2$ (đáp án: 4)
- Chiều VC của các bộ phân loại tuyến tính trong $\mathbb{R}^d$ (đáp án: $d + 1$)

Giải thích nhưng không cần suy ra:

- Vì sao chiều VC đo lường “độ phức tạp hiệu dụng”
- Vì sao chiều VC hữu hạn hàm ý khả năng học được
- Trục giác về Biquyet Sauer (số lượng cách tăng là đa thức, không phải hàm mũ)

Chỉ cần biết phát biểu:

- Định lý Nền tảng: học PAC được $\Leftrightarrow$ chiều VC hữu hạn
- Không có bữa trưa miễn phí: không có thuật toán học phổ quát nào hoạt động được với mọi phân phối

12.1 Giới thiệu: Vượt ra ngoài các lớp hữu hạn

Ở chương trước, chúng ta đã thấy rằng các lớp giả thuyết hữu hạn đều học PAC được, với độ phức tạp mẫu phụ thuộc vào $\ln |\mathcal{H}|$. Nhưng còn những lớp vô hạn như “tất cả các hình chữ nhật” hay “tất cả các bộ phân loại tuyến tính” thì sao? Chúng có vô số giả thuyết, nhưng ta vẫn học được chúng một cách hiệu quả. Làm thế nào điều đó có thể xảy ra?

Chặn trên $m \leq \frac{\ln |\mathcal{H}|}{\epsilon}$ cho ra vô hạn đối với lớp vô hạn, điều này chẳng có ích gì. Ta cần một cách khác để đo lường “độ phức tạp” của một lớp giả thuyết. Một cách nắm bắt đúng điều thực sự quan trọng đối với việc học, bất kể lớp đó hữu hạn hay vô hạn.

Câu trả lời đáng chú ý chính là **chiều VC**, một đại lượng tổ hợp đặc trưng hóa hoàn toàn khả năng học được. Một lớp học được khi và chỉ khi chiều VC của nó hữu hạn, và độ phức tạp mẫu phụ thuộc vào chiều VC chứ không phải kích thước của lớp.

12.2 Phá vỡ: Những gì một lớp có thể biểu đạt

Ý tưởng then chốt là điều quan trọng đối với việc học không phải là có bao nhiêu giả thuyết, mà là lớp đó có thể tạo ra bao nhiêu *hành vi khác nhau* trên các tập điểm hữu hạn.

Cho một tập điểm $C = \{x_1, \dots, x_m\}$, hãy xét mọi cách mà các giả thuyết trong $\mathcal{H}$ có thể gán nhãn cho những điểm này. **Phép hạn chế** của $\mathcal{H}$ trên C là:

$$\mathcal{H}_C = \{(h(x_1), \dots, h(x_m)) : h \in \mathcal{H}\}$$

Đây là tập hợp mọi mẫu gán nhãn mà một giả thuyết nào đó trong $\mathcal{H}$ tạo ra trên

C .

Ví dụ, hãy xét các bộ phân loại ngưỡng trên đường thẳng thực:

$h_\theta(x) = 1[x \geq \theta]$. Cho các điểm $c = \{1, 3, 5\}$, những ngưỡng khác nhau tạo ra những cách gán nhãn khác nhau. Ngưỡng $\theta = 0$ cho $(1, 1, 1)$ (tất cả đều dương). Ngưỡng $\theta = 2$ cho $(0, 1, 1)$. Ngưỡng $\theta = 4$ cho $(0, 0, 1)$. Ngưỡng $\theta = 6$ cho $(0, 0, 0)$. Nhưng không ngưỡng nào tạo ra $(1, 0, 1)$. Bạn không thể gán nhãn dương cho 1, âm cho 3 và dương cho 5 chỉ bằng một ngưỡng duy nhất.

Phá vỡ

Một lớp giả thuyết $\mathcal{H}$ **phá vỡ** một tập C khi $\mathcal{H}_C$ chứa **tất cả** các cách gán nhãn $2^{|C|}$ khả dĩ.

Nói cách khác: với mọi cách gán nhãn các điểm trong C , đều tồn tại một giả thuyết nào đó trong $\mathcal{H}$ tạo ra đúng mẫu gán nhãn đó. Lớp đó “đủ giàu để biểu đạt” nhằm hiện thực hóa bất kỳ mẫu nào trên những điểm ấy.

Nói cách khác, phá vỡ có nghĩa là một lớp có thể ghi nhớ tùy ý các nhãn trên những điểm đó, kể cả những nhãn hoàn toàn ngẫu nhiên chẳng có mẫu nào. Đây chính là cơ chế của hiện tượng quá khớp: nếu $\mathcal{H}$ có thể phá vỡ tập huấn luyện của bạn, nó có thể khớp hoàn hảo mọi nhiễu trong nhãn, và sự khớp hoàn hảo đó không nói lên điều gì về sự khái quát hóa.

Các bộ phân loại ngưỡng có thể phá vỡ bất kỳ điểm đơn lẻ nào (hai cách gán nhãn, cả hai đều đạt được). Chúng có thể phá vỡ bất kỳ hai điểm $\{a, b\}$ thỏa $a < b$: các cách gán nhãn $(0, 0)$, $(0, 1)$ và $(1, 1)$ đều đạt được... khoan, còn $(1, 0)$ thì sao? Cái này đòi hỏi $a \geq \theta$ nhưng $b < \theta$, điều không thể xảy ra vì $a < b$. Vậy nên các bộ phân loại ngưỡng *không thể* phá vỡ hai điểm.

Thật ra, hãy xét lại. Với hai điểm $a < b$, chúng ta đạt được: $(0, 0)$ với $\theta > b$; $(0, 1)$ với $a < \theta \leq b$; $(1, 1)$ với $\theta \leq a$. Chúng ta không thể đạt được $(1, 0)$. Vậy nói chung các ngưỡng không thể phá vỡ 2 điểm... nhưng chúng có thể phá vỡ 1 điểm. Hóa ra chiều VC bằng 1.

12.3 Chiều VC

Chiều VC

Chiều Vapnik-Chervonenkis (VC) của một lớp giả thuyết $\mathcal{H}$, ký hiệu là $\text{VCdim}(\mathcal{H})$, được định nghĩa là:

$$\text{VCdim}(\mathcal{H}) = \max\{m : \exists \text{ một tập } C \text{ có kích thước } m \text{ mà } \mathcal{H} \text{ phá vỡ}\}$$

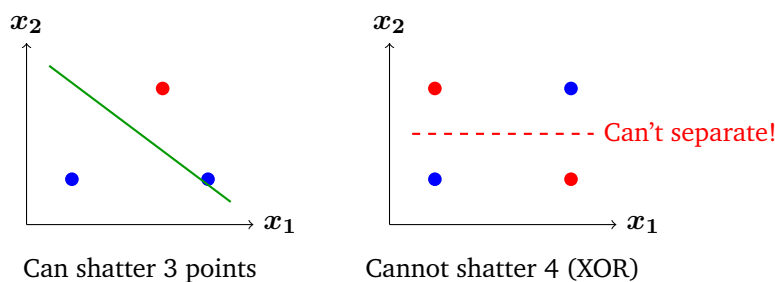
Đó là kích thước tập lớn nhất mà $\mathcal{H}$ có thể phá vỡ, tức là số điểm tối đa mà trên đó $\mathcal{H}$ có thể hiện thực hóa mọi mẫu gán nhãn khả dĩ.

Chiều VC đo lường “khả năng biểu đạt” hay “độ phức tạp” của một lớp giả thuyết theo cách không phụ thuộc vào việc lớp đó hữu hạn hay vô hạn. Lớp có chiều VC cao có thể biểu đạt nhiều mẫu khác nhau; lớp có chiều VC thấp thì bị ràng buộc hơn.

12.3.1 Các ví dụ về chiều VC

Lớp giả thuyết	VCdim	Trực giác
Ngưỡng trên $\mathbb{R}$	1	Chia tách được 1 điểm, không chia tách được 2
Các khoảng $[a, b]$ trên $\mathbb{R}$	2	Bắt được mọi 2 điểm
Hình chữ nhật song song trục trong $\mathbb{R}^2$	4	Bốn góc để điều chỉnh
Bộ phân loại tuyến tính trong $\mathbb{R}^d$	$d + 1$	$d + 1$ tham số (trọng số + thiên vị)

Ví dụ quan trọng nhất là các bộ phân loại tuyến tính. Trong $\mathbb{R}^d$, một bộ phân loại tuyến tính được định nghĩa bởi một siêu phẳng: các điểm ở một phía là dương, điểm ở phía bên kia là âm. Chiều VC của các bộ phân loại tuyến tính trong $\mathbb{R}^d$ bằng đúng $d + 1$.



Hình 12.1: Các bộ phân loại tuyến tính trong $\mathbb{R}^2$ có chiều VC là 3: chúng có thể phá vỡ bất kỳ 3 điểm nào ở tư thế tổng quát, nhưng không đường thẳng nào có thể tách được mẫu XOR trên 4 điểm.

Để chứng minh $\text{VCdim} = 3$ với các đường thẳng trong $\mathbb{R}^2$, ta phải chứng minh: (1) tồn tại 3 điểm có thể bị phá vỡ, và (2) không có bộ 4 điểm nào bị phá vỡ. Với (1), đặt ba điểm không cùng nằm trên một đường thẳng; bất kỳ cách gán nhãn nào cũng đạt được bởi một đường thẳng nào đó. Với (2), xét 4 điểm. Với một số cách gán nhãn như mẫu xor (hai góc đối cùng nhãn), không đường thẳng nào tách được chúng.

12.4 Định lý Nền tảng của học PAC

Chiều VC không chỉ là một trong vô số cách đo độ phức tạp. Nó đặc trưng hóa hoàn toàn khả năng học được:

Định lý Nền tảng

Một lớp giả thuyết $\mathcal{H}$ học được PAC (theo nghĩa agnostic) **khi và chỉ khi** nó có chiều VC hữu hạn.

Hơn nữa, độ phức tạp mẫu là:

$$m_{\mathcal{H}}(\epsilon, \delta) = O\left(\frac{\text{VCdim}(\mathcal{H}) + \ln(1/\delta)}{\epsilon^2}\right)$$

Điều này thật đáng kinh ngạc. Chiều VC. Một tính chất thuần túy tổ hợp về những cách gán nhãn có thể hiện thực hóa, lại quyết định hoàn toàn việc liệu việc học có khả thi hay không và cần bao nhiêu mẫu. Các lớp vô hạn nhưng có chiều VC hữu hạn (như mọi bộ phân loại tuyến tính) thì học được. Các lớp có chiều VC vô hạn thì không học được PAC được.

Định lý giải thích tại sao độ phức tạp mẫu phụ thuộc vào chiều VC chứ không phải số lượng giả thuyết. Lớp mọi bộ phân loại tuyến tính trong $\mathbb{R}^{100}$ là vô hạn, nhưng chiều VC của nó chỉ bằng 101. Việc học đòi hỏi cỡ bậc 101 mẫu (nhân thêm các hệ số cho độ chính xác và độ tin cậy), chứ không phải vô số mẫu.

12.5 Định lý Không có bữa trưa miễn phí

Một câu hỏi tự nhiên: có tồn tại thuật toán học “phổ quát” nào hoạt động được với mọi bài toán không? Câu trả lời dứt khoát là không.

Định lý Không có bữa trưa miễn phí

Không có thuật toán học nào là tối ưu cho **mọi** bài toán. Chính xác hơn: với bất kỳ bộ học nào A , đều tồn tại một phân phối $\mathcal{D}$ và một hàm gán nhãn f sao cho A hoạt động kém, trong khi có một bộ học khác A' lại hoạt động tốt.

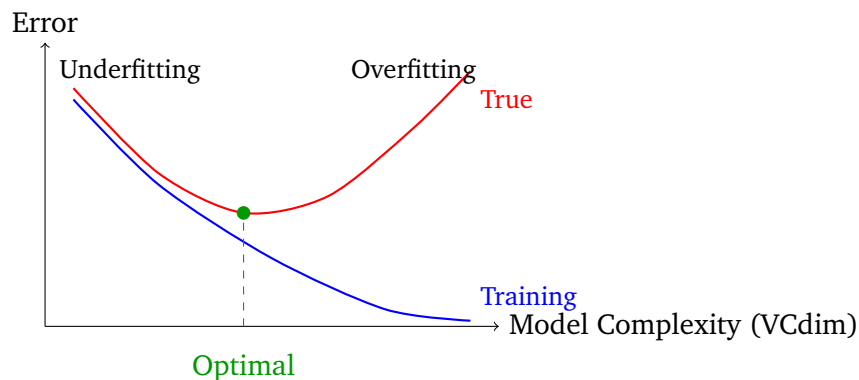
Điều này hàm ý sâu sắc. **Kiến thức tiên quyết là thiết yếu:** bạn phải chọn một lớp giả thuyết phù hợp với bài toán của mình. **Không có lời giải phổ quát:** những bài toán khác nhau thực sự đòi hỏi những cách tiếp cận khác nhau. **Đây là lý do AI khó:** trí tuệ tổng quát sẽ đòi hỏi những tiên nghiệm phù hợp cho mọi miền có thể.

Định lý Không có bữa trưa miễn phí không có nghĩa là học máy vô dụng. Nó có nghĩa là khi việc học thành công, đó là vì lớp giả thuyết khớp với cấu trúc bài toán. Định lý giải thích tại sao trình độ chuyên môn quan trọng: việc chọn đúng cách biểu diễn và đúng lớp cho một bài toán bản thân nó đã là một dạng tri thức.

12.6 Kết nối giữa lý thuyết và thực hành

12.6.1 Quá khớp qua lăng kính chiều VC

Chiều VC cung cấp một lời giải thích lý thuyết cho quá khớp và thiếu khớp:



Hình 12.2: Sự đánh đổi thiên vị-phương sai xét theo chiều VC. Độ phức tạp thấp nghĩa là thiên vị cao (thiếu khớp); độ phức tạp cao nghĩa là phương sai cao (quá khớp).

Với chiều VC nhỏ (mô hình đơn giản), lớp không thể biểu đạt các mẫu phức tạp. Thiên vị cao, thiếu khớp. Với chiều VC lớn (mô hình phức tạp), lớp có thể khớp rất tốt dữ liệu huấn luyện nhưng có thể không khái quát hóa được. Phương sai cao, quá khớp. Lựa chọn tối ưu là cân bằng giữa hai điều này, tùy theo lượng dữ liệu huấn luyện sẵn có.

12.6.2 Hướng dẫn thực tế

Một số bài học thực tế rút ra từ lý thuyết:

Cân độ phức tạp theo cỡ dữ liệu: Dữ liệu nhiều hơn cho phép dùng mô hình phức tạp hơn (chiều VC cao hơn). Với dữ liệu hạn chế, hãy dùng mô hình đơn giản hơn.

Chính quy hóa làm giảm chiều VC hiệu dụng: Các kỹ thuật như phân rã trọng số (weight decay) ràng buộc lớp giả thuyết, giảm độ phức tạp mà không cần đổi kiến trúc mô hình.

Kiểm chéo ước lượng sai số thật: Vì ta không thể tính trực tiếp sai số thật, việc kiểm định trên dữ liệu giữ lại cung cấp ước lượng tốt nhất của chúng ta.

Bài toán của học sâu

Lý thuyết VC dự đoán rằng những mô hình có chiều VC cao sẽ bị quá khớp, trừ khi được cung cấp tập huấn luyện lớn tương ứng. Các mạng nơ-ron sâu lại vi phạm dự đoán này: chiều VC của chúng thường vượt số ví dụ huấn luyện (chúng có thể ghi nhớ nhãn ngẫu nhiên), nhưng vẫn khái quát hóa tốt trên dữ liệu thật.

Đây là một trong những câu hỏi mở quan trọng nhất của lý thuyết học máy. Đã có một số lời giải thích một phần: việc tối ưu hóa qua rơi gradient ngầm ưu tiên những lời giải đơn giản (chính quy hóa ngầm), dữ liệu thật có cấu trúc mà nhãn ngẫu nhiên không có, và các mạng quá tham số hóa tìm thấy những cực tiểu phẳng khái quát hóa tốt hơn. Nhưng vẫn chưa có lý thuyết đầy đủ nào giải thích tại sao học sâu lại hoạt động tốt đến vậy. Lý thuyết VC vẫn đúng như một chặn trên trường hợp xấu nhất, nhưng lại quá bi quan để mô tả hành vi điển hình của các mạng nơ-ron hiện đại.

12.7 Tóm tắt**Điểm cần nhớ**

Chiều VC: Đo lường độ phức tạp của lớp giả thuyết qua tập lớn nhất mà nó phá vỡ được. Một lớp phá vỡ một tập nếu lớp đó tạo ra được mọi cách gán nhãn trên tập đó.

Định lý Nền tảng: Một lớp học PAC được khi và chỉ khi chiều VC của nó là hữu hạn. Độ phức tạp mẫu tỷ lệ với chiều VC. **Không có bữa trưa miễn phí:** Không tồn tại thuật toán học phổ quát. Thành công đòi hỏi khớp lớp giả thuyết với cấu trúc của bài toán.

Hệ quả thực tế: Chiều VC giải thích quá khớp/thiếu khớp. Cân bằng độ phức tạp mô hình với cỡ dữ liệu. Chính quy hóa kiểm soát độ phức tạp hiệu dụng. Tuy nhiên, các chặn trên VC lại quá bi quan để giải thích tại sao mạng nơ-ron sâu vẫn khái quát hóa được dù có chiều VC khổng lồ; đây vẫn là một bài toán mở.

12.8 Bài tập luyện

1. Chứng minh rằng các khoảng $[a, b]$ trên đường thẳng thực có chiều VC bằng 2. (Gợi ý: Hãy chỉ ra bạn có thể phá vỡ 2 điểm nhưng không phá vỡ được 3 điểm.)
2. Lớp “cây quyết định stump” trong $\mathbb{R}^2$ (các bộ phân loại có dạng $1[x_i \geq \theta]$ với $i \in \{1, 2\}$) có chiều VC bằng bao nhiêu?
3. Nếu một lớp giả thuyết có $\text{VCdim} = 10$ và chúng ta muốn $\epsilon = 0.05$, $\delta = 0.01$,

thì chúng ta cần khoảng bao nhiêu mẫu?

4. Giải thích tại sao Định lý Không có bữa trưa miễn phí không có nghĩa là học máy vô dụng. Điều gì cứu chúng ta trong thực tế?

5. Một mô hình đạt độ chính xác huấn luyện 99% nhưng chỉ đạt độ chính xác kiểm tra 70%. Dùng các khái niệm từ chương này, giải thích điều gì đang xảy ra và cách khắc phục.

6. Tại sao các bộ phân loại tuyến tính trong $\mathbb{R}^2$ không thể phá vỡ 4 điểm nói chung? Hãy cho một cấu hình cụ thể không thể gán nhãn được.

7. **(Nhiều phần, kiểu IAIO)** Xét lớp giả thuyết gồm các hình chữ nhật song song trục trong $\mathbb{R}^2$ (dự đoán dương bên trong hình chữ nhật, âm bên ngoài).

(a) Lớp này có thể phá vỡ 3 điểm không? Hãy cho một cách sắp đặt cụ thể có thể bị phá vỡ.

(b) Lớp này có thể phá vỡ 4 điểm không? Nếu có, hãy cho một cách sắp đặt. Nếu không, hãy giải thích cách gán nhãn nào là không thể.

(c) Lớp này có thể phá vỡ 5 điểm không? Hãy chứng minh câu trả lời của bạn.

(d) Chiều VC của lớp này là bao nhiêu?

(e) Dùng Định lý Nền tảng, với 500 ví dụ huấn luyện và $\delta = 0.05$, hãy cho một chặn trên của lỗi khái quát hóa.

8. **(Tư duy phản biện)** Định lý Không có bữa trưa miễn phí nói rằng không có bộ học nào là tối ưu một cách phổ quát. Thế mà trong thực tế, một số thuật toán (như mạng nơ-ron) dường như hoạt động tốt với hầu hết mọi thứ.

(a) Giải thích tại sao điều này không mâu thuẫn với NFL.

(b) NFL thực sự nói với chúng ta điều gì về việc chọn thuật toán học?

Phần V

Ứng dụng (Tùy chọn)

Chương 13

Phân tích ma trận: Một ý tưởng thống nhất

The best low-rank approximation of a matrix is given by its singular value decomposition.

Carl Eckart & Gale Young

Phần V: Ứng dụng (Tùy chọn)

Chương này trình bày phân tích ma trận như một kỹ thuật toán học, có ứng dụng trong các hệ thống gợi ý và xa hơn nữa. Trọng tâm là các **nền tảng toán học**, gắn với đại số tuyến tính, tối ưu hóa và regularization, chứ không phải việc triển khai hệ thống. **Mối liên hệ với IAIO**: Các bài toán phân tích ma trận kiểm tra sự hiểu biết của bạn về:

- Xấp xỉ hạng thấp và SVD (Phụ lục F)
- Hàm mất mát và regularization (Chương 4)
- Tối ưu hóa dựa trên gradient (Chương 4, Phụ lục D)
- Đánh đổi thiên vị-phương sai (Chương 5)

Vì sao chương này tồn tại

Phân tích ma trận xuất hiện không thường xuyên trên IAIO, vậy tại sao lại đưa vào? Bởi vì đây là một **ví dụ thống nhất** kết nối nhiều khái niệm cốt lõi:

- **Tối ưu hóa:** Cùng phương pháp rơi gradient như hồi quy tuyến tính (Chương 4)
- **Regularization:** Cùng hình phạt L2 như hồi quy ridge (Chương 4)
- **Đánh giá:** Cùng phương pháp huấn luyện/kiểm tra (Chương 5)
- **Đại số tuyến tính:** Ứng dụng trực tiếp của SVD (Phụ lục F)

Nếu bạn hiểu phân tích ma trận, bạn đã thể hiện sự thành thạo các nền tảng này trong một bối cảnh mới mẻ, đúng thứ mà IAIO kiểm tra. Chương này ngắn vì các ý tưởng không mới; chúng được *vận dụng*.

13.1 Ý tưởng cốt lõi

Nhiều tập dữ liệu trong thực tế có thể được biểu diễn thành ma trận với các phần tử thiếu. Người dùng đánh giá phim, học sinh trả lời câu hỏi kiểm tra, cảm biến đo lường hiện tượng — tất cả đều tạo ra ma trận thưa, nơi chúng ta chỉ quan sát được một phần nhỏ trong số các giá trị có thể có.

Ý chính của phân tích ma trận: **các ma trận thực tế xấp xỉ là hạng thấp**. Điều này có nghĩa chúng có thể được xấp xỉ tốt bằng tích của các ma trận nhỏ hơn nhiều.

$$\begin{array}{ccccc}
 \mathbf{R} & \text{(sparse)} & \approx & \mathbf{P} & \times & \mathbf{Q}^T \\
 \begin{array}{|c|} \hline \\ \hline \end{array} & & & \begin{array}{|c|} \hline \\ \hline \end{array} & & \begin{array}{|c|} \hline \\ \hline \end{array} \\
 m \times n & & & m \times k & & k \times n
 \end{array}$$

Hình 13.1: Phân tích ma trận: xấp xỉ một ma trận thưa lớn thành tích của hai ma trận đặc nhỏ.

Nếu $\mathbf{R}$ là $m \times n$ và chúng ta phân tích nó thành $\mathbf{R} \approx \mathbf{PQ}^T$ với $k \ll \min(m, n)$, chúng ta đã nén thông tin một cách đáng kể:

- Ban đầu: $m \times n$ phần tử (phần lớn chưa biết)
- Sau khi phân tích: $m \times k + n \times k = k(m + n)$ tham số

Ví dụ: Với Netflix (200 triệu người dùng, 15 nghìn bộ phim), việc lưu trữ mọi đánh giá có thể có cần 3×10^{12} phần tử. Với $k = 100$ yếu tố: $(200M + 15K) \times 100 \approx 2 \times 10^{10}$ tham số, mức giảm 150×.

13.2 Phát biểu toán học

13.2.1 Bài toán tối ưu hóa

Chúng ta muốn tìm các ma trận $\mathbf{P}$ và $\mathbf{Q}$ sao cho tích của chúng xấp xỉ các phần tử đã quan sát của $\mathbf{R}$.

Gọi Ω là tập các phần tử đã quan sát (các cặp (i, j) mà ở đó chúng ta biết R_{ij}). Mục tiêu:

$$\min_{\mathbf{P}, \mathbf{Q}} \sum_{(i,j) \in \Omega} \left(R_{ij} - \mathbf{p}_i^\top \mathbf{q}_j \right)^2 + \lambda (\|\mathbf{P}\|_F^2 + \|\mathbf{Q}\|_F^2)$$

trong đó:

- $\mathbf{p}_i$ là dòng thứ i của $\mathbf{P}$ (là “embedding” của người dùng/dòng i)
- $\mathbf{q}_j$ là dòng thứ j của $\mathbf{Q}$ (“embedding” của mục/cột j)
- $\|\cdot\|_F$ là chuẩn Frobenius (tổng bình phương các phần tử)
- λ là cường độ regularization

Gắn kết với những khái niệm bạn đã biết

Hàm mất mát: Sai số bình phương, giống hệ hồi quy tuyến tính (Chương 4). **Regularization:** Hình phạt L2 lên các tham số, giống hệ hồi quy ridge. **Tối ưu hóa:** Rơi gradient trên một mục tiêu không lồi (khác với hồi quy tuyến tính, vốn lồi).

Các liên kết rõ ràng với nội dung trước đó

Phân tích ma trận **không phải là kỹ thuật mới**, nó kết hợp những ý tưởng bạn đã biết:

Từ Chương 4 (Học có giám sát):

- Hàm mất mát là sai số bình phương, giống hệ hồi quy tuyến tính
- Thành phần regularization là L2 (ridge), giống hệ hồi quy ridge

- Các quy tắc cập nhật gradient đi theo cùng một khuôn suy luận

Từ Chương 5 (Đánh giá mô hình):

- Việc chọn k và λ cần kiểm chéo trên các đánh giá được giữ lại
- Quá khớp biểu hiện qua RMSE huấn luyện thấp nhưng RMSE kiểm tra cao

Từ Phụ lục D (Giải tích):

- Việc suy ra gradient dùng quy tắc chuỗi
- Cập nhật $\mathbf{p}_i \leftarrow \mathbf{p}_i + \eta(e_{ij}\mathbf{q}_j - \lambda\mathbf{p}_i)$ là rơi gradient chuẩn

Từ Phụ lục F (Đại số tuyến tính):

- SVD cho xấp xỉ hạng thấp tối ưu đối với các ma trận đầy đủ
- Phân tích ma trận mở rộng điều này sang các ma trận thưa đủ

13.2.2 Suy ra gradient

Để áp dụng rơi gradient, chúng ta cần các gradient theo $\mathbf{p}_i$ và $\mathbf{q}_j$.

Với một quan sát duy nhất (i, j) và sai số $e_{ij} = R_{ij} - \mathbf{p}_i^\top \mathbf{q}_j$:

$$\frac{\partial}{\partial \mathbf{p}_i} [e_{ij}^2 + \lambda \|\mathbf{p}_i\|^2] = -2e_{ij}\mathbf{q}_j + 2\lambda\mathbf{p}_i$$

$$\frac{\partial}{\partial \mathbf{q}_j} [e_{ij}^2 + \lambda \|\mathbf{q}_j\|^2] = -2e_{ij}\mathbf{p}_i + 2\lambda\mathbf{q}_j$$

Quy tắc cập nhật (với tốc độ học η):

$$\begin{aligned}\mathbf{p}_i &\leftarrow \mathbf{p}_i + \eta(e_{ij}\mathbf{q}_j - \lambda\mathbf{p}_i) \\ \mathbf{q}_j &\leftarrow \mathbf{q}_j + \eta(e_{ij}\mathbf{p}_i - \lambda\mathbf{q}_j)\end{aligned}$$

13.2.3 Bình phương luân phiên

Một cách thay thế cho rơi gradient: **Bình phương luân phiên (ALS)**.

Nếu chúng ta cố định $\mathbf{Q}$, việc tối ưu $\mathbf{P}$ là một bài toán lồi (thực chất chỉ là hồi quy ridge cho từng dòng). Tương tự, nếu chúng ta cố định $\mathbf{P}$, việc tối ưu $\mathbf{Q}$ là lồi.

Thuật toán:

1. Khởi tạo ngẫu nhiên $\mathbf{P}$ và $\mathbf{Q}$
2. Cố định $\mathbf{Q}$, giải tìm $\mathbf{P}$ tối ưu (nghiệm đóng)
3. Cố định $\mathbf{P}$, giải tìm $\mathbf{Q}$ tối ưu (nghiệm đóng)
4. Lặp lại cho đến khi hội tụ

ALS hội tụ đến một cực tiểu cục bộ và dễ song song hóa (từng dòng của $\mathbf{P}$ có thể được cập nhật độc lập).

13.3 Liên hệ với SVD

Nền tảng lý thuyết của phân tích ma trận đến từ phép SVD (Phân tích giá trị riêng, Singular Value Decomposition).¹ Định lý Eckart–Young (1936)² đảm bảo rằng xấp xỉ hạng- k tốt nhất của bất kỳ ma trận nào, đo theo chuẩn Frobenius, thu được bằng cách cắt SVD của nó xuống k giá trị riêng lớn nhất. Chính điều này tạo động lực cho cách tiếp cận $\mathbf{P}\mathbf{Q}^T$: chúng ta đang tìm cùng loại cấu trúc hạng thấp mà SVD sẽ tìm được, nhưng dưới ràng buộc rằng hầu hết các phần tử chưa được quan sát.

SVD phân tích một ma trận bất kỳ thành:

$$\mathbf{R} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$$

trong đó $\mathbf{U}$ và $\mathbf{V}$ trực giao, còn $\mathbf{\Sigma}$ là ma trận chéo với các giá trị riêng không âm.

Xấp xỉ hạng thấp: Chỉ giữ k giá trị riêng lớn nhất cho xấp xỉ hạng- k tốt nhất (theo chuẩn Frobenius). Đây chính là kết quả Eckart–Young.

Cái bẫy: SVD tiêu chuẩn cần ma trận đầy đủ. Khi có các phần tử thiếu, chúng ta không thể áp dụng SVD trực tiếp. Phân tích ma trận có thể được xem như “SVD cho ma trận chưa đầy đủ”: tìm một xấp xỉ hạng thấp trong khi chỉ khớp các phần tử đã quan sát.

¹See Appendix F for a complete treatment of SVD.

²Eckart, C. & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

Vì sao hạng thấp hiệu quả

Các ma trận thực tế xấp xỉ là hạng thấp bởi vì các hiện tượng nền chỉ có số bậc tự do hạn chế:

- Sở thích phim phụ thuộc vào vài “yếu tố” (thể loại, thời đại, tâm trạng)
- Thành tích của học sinh phụ thuộc vào vài “kỹ năng”
- Các phép đo khoa học phụ thuộc vào vài quá trình vật lý

Cách k cột của $\mathbf{P}$ và $\mathbf{Q}$ nắm bắt những yếu tố tiềm ẩn này.

13.4 Góc nhìn thiên vị–phương sai

Cao k (nhiều yếu tố hơn):

- Mô hình biểu đạt hơn
- Có thể khớp dữ liệu huấn luyện tốt hơn
- Nguy cơ quá khớp (phương sai cao)

Thấp k (ít yếu tố hơn):

- Mô hình đơn giản hơn
- Có thể thiếu khớp (thiên vị cao)
- Khái quát hóa tốt hơn nếu hạng thật thấp

Regularization λ :

- Cao λ : Co các embedding về 0, giảm độ phức tạp hiệu dụng của mô hình
- Thấp λ : Cho phép embedding lớn hơn, linh hoạt hơn

Việc chọn k và λ cần kiểm định trên dữ liệu được giữ lại. Đúng phương pháp luận từ chương 5.

Vì sao đây không phải là phép thuật

Phân tích ma trận có vẻ gần như phép thuật: chúng ta quan sát một phần rất nhỏ của ma trận rồi bằng cách nào đó dự báo được mọi phần tử còn thiếu. Nhưng chẳng có phép thuật nào cả. Chỉ có những giả định mạnh: **Giả định hạng thấp**: Chúng ta giả định ma trận đầy đủ “thật” có hạng thấp. Nếu sở thích người dùng thực sự phụ thuộc vào hàng nghìn yếu tố độc lập, phân tích hạng thấp sẽ thất bại.

Thiếu ngẫu nhiên: Chúng ta giả định các phần tử đã quan sát có tính đại diện. Nếu người dùng chỉ đánh giá những phim họ yêu (hoặc ghét), mẫu của chúng ta bị thiên lệch và các dự đoán cho những phim “tầm thường” sẽ kém.

Tính ổn định: Chúng ta giả định sở thích không thay đổi. Một mô hình huấn luyện trên các đánh giá năm 2020 có thể thất bại với người dùng năm 2024 có gu khác. Khi các giả định này xấp xỉ đúng, phân tích ma trận hoạt động tốt. Khi chúng không đúng, không có mấy sự lọc lỗi của thuật toán nào cứu được bạn. **Hiểu giả định quan trọng hơn hiểu thuật toán.**

Hai chế độ thất bại thực tế cần được chú ý đặc biệt. **Vấn đề khởi động lạnh (cold-start)** nảy sinh khi một người dùng (hoặc mục) mới hoàn toàn không có phần tử nào được quan sát. Khi không có điểm dữ liệu nào, không có gì để học một embedding; mô hình không thể phân biệt người dùng này với người khác. Các cách xử lý phổ biến bao gồm dùng thông tin phụ (nhân khẩu học, mô tả mục) hoặc quay về gợi ý dựa trên độ phổ biến cho đến khi tích lũy đủ dữ liệu. **Độ thưa cực độ** là vấn đề liên quan: có quá ít quan sát cho mỗi người dùng hoặc mục. Nếu người dùng chỉ đánh giá hai phim trong số 15.000 phim, hệ thống đang cố suy ra một embedding k chiều từ hai điểm dữ liệu, điều này bị thiếu xác định nghiêm trọng. Trong thực tế, hiệu năng của phân tích ma trận giảm sút mạnh khi tỷ lệ quan sát rơi xuống dưới khoảng 1% số phần tử, mặc dù ngưỡng chính xác phụ thuộc vào hạng và mức nhiễu.

13.5 Tóm tắt

Điểm cần nhớ

Phân tích ma trận: Xấp xỉ $R \approx PQ^T$ để dự đoán các phần tử thiếu.

Mục tiêu: Hạ thấp sai số bình phương trên các phần tử đã quan sát cộng regularization L2.

Tối ưu hóa: Rơi gradient hoặc Bình phương luân phiên. Không lỗi, nên tồn tại các cực tiểu cục bộ.

Liên hệ với SVD: Giống SVD, nhưng xử lý dữ liệu thiếu bằng cách chỉ khớp các phần tử đã quan sát.

Giả định hạng thấp: Hiệu quả vì dữ liệu thực tế có số chiều nền hữu hạn (các yếu tố tiềm ẩn).

Siêu tham số: k (hạng) điều khiển dung lượng; λ (regularization) điều khiển độ phức tạp. Cả hai đều ảnh hưởng đến đánh đổi thiên vị-phương sai.

13.6 Bài tập luyện

- Một ma trận có 1 triệu dòng và 100 nghìn cột. Với $k = 50$ yếu tố:
 - Có bao nhiêu tham số trong phép phân tích?
 - Tỷ số nén so với ma trận đầy đủ là bao nhiêu?
 - Nếu chúng ta quan sát 0,1% số phần tử, có bao nhiêu quan sát để ràng buộc các tham số?
- Người dùng Alice có embedding $\mathbf{p} = (0.8, -0.3, 0.5)$. Các mục có embedding $\mathbf{q}_1 = (0.5, 0.2, 0.4)$ và $\mathbf{q}_2 = (0.1, -0.5, 0.8)$. Tính các giá trị dự đoán. Mục nào xếp hạng cao hơn?
- (Suy ra gradient)** Bắt đầu từ hàm mục tiêu, hãy suy ra quy tắc cập nhật gradient cho $\mathbf{p}_i$. Trình bày các bước.
- (theo phong cách IAIO)** Hàm mục tiêu phân tích ma trận là:

$$L = \sum_{(i,j) \in \Omega} (R_{ij} - \mathbf{p}_i^\top \mathbf{q}_j)^2 + \lambda(\|\mathbf{P}\|_F^2 + \|\mathbf{Q}\|_F^2)$$
 - Ω là gì? Tại sao chúng ta chỉ cộng trên Ω ?
 - Tại sao regularization là cần thiết? Điều gì xảy ra khi $\lambda \rightarrow 0$? Khi $\lambda \rightarrow \infty$?
 - Hàm mục tiêu này không lỗi. Tại sao? Hậu quả của điều đó đối với việc tối ưu hóa là gì?

(d) Giải thích tại sao ALS luân phiên giữa $\mathbf{P}$ và $\mathbf{Q}$. Tại sao mỗi bài toán con đều lồi?

5. **(Liên hệ với đại số tuyến tính)** Giải thích mối quan hệ giữa phân tích ma trận và SVD bị cắt (truncated SVD). Khi nào chúng tương đương? Khi nào chúng khác nhau?

6. **(Thiên vị-phương sai)** Bạn tăng k từ 10 lên 100. RMSE huấn luyện giảm từ 0,8 xuống 0,3. RMSE kiểm tra tăng từ 0,9 lên 1,2. Chẩn đoán vấn đề và đề xuất hai giải pháp.

Chương 14

Học sâu hiện đại

Bạn không cần phải hợp lý về sinh học để có trí tuệ.

Yann LeCun

Lưu ý: Tài liệu mở rộng

Chương này trình bày các kiến trúc AI tiên tiến đang vận hành những hệ thống ấn tượng nhất ngày nay. Dù hữu ích để hiểu bối cảnh AI hiện tại, tài liệu này **không được kiểm tra trực tiếp** trong IAIO. Hãy đọc để hiểu ý tưởng, không phải để học thuộc. Hãy dành thời gian học cho các chương cốt lõi (3–6, 8, 11).

14.1 Giới thiệu

Giai đoạn từ 2014 đến nay chứng kiến những tiến bộ mang tính cách mạng cho AI, được thúc đẩy bởi bốn đổi mới kiến trúc: mạng đối kháng tạo sinh (GAN), kiến trúc transformer, mô hình khuếch tán cho sinh ảnh, và việc mở rộng quy mô mô hình ngôn ngữ lớn. Các công nghệ này vận hành những hệ thống như ChatGPT, DALL-E, Stable Diffusion và nhiều hệ thống khác.

Chương này cung cấp hiểu biết khái niệm về các hệ thống đó, tập trung vào các ý tưởng chính thay vì chi tiết cài đặt. Chương 10 trình bày các khái niệm nền tảng (mô hình tạo sinh so với mô hình phân biệt, không gian tiềm ẩn, bộ mã hóa-giải mã tự động, VAE) mà chương này xây dựng trên đó.

14.2 Máy tính xử lý ảnh như thế nào

Trước khi khám phá các kiến trúc thị giác hiện đại, ta nên hiểu cách máy tính biểu diễn thông tin hình ảnh.

14.2.1 Ảnh dưới dạng các con số

Với máy tính, một ảnh là một lưới các con số được gọi là **pixel**. Với ảnh đơn sắc, mỗi pixel là một con số (0 = đen, 255 = trắng). Với ảnh màu, mỗi pixel có ba con số biểu diễn cường độ đỏ, xanh lá và xanh dương (RGB).

Quy mô dữ liệu ảnh

Một ảnh HD tiêu chuẩn (1920×1080) chứa hơn 2 triệu pixel. Với ba kênh màu, đó là hơn 6 triệu con số cho mỗi ảnh. Các mô hình học sâu phải tìm ra những mẫu hình có ý nghĩa trong không gian chiều cao này.

14.2.2 Từ pixel đến sự hiểu biết

Thách thức của thị giác máy tính là biến các pixel thô thành sự hiểu biết ngữ nghĩa. Mạng nơ-ron giải quyết điều đó thông qua **học đặc trưng phân cấp**:

- **Các lớp đầu:** Phát hiện các đặc trưng đơn giản (cạnh, kết cấu)
- **Các lớp giữa:** Kết hợp thành các mẫu phức tạp (hình dạng, bộ phận)
- **Các lớp sau:** Tích hợp thành các khái niệm mức cao (đối tượng, cảnh)

Thứ bậc này là nền tảng đối với cả CNN lẫn các vision transformer hiện đại.

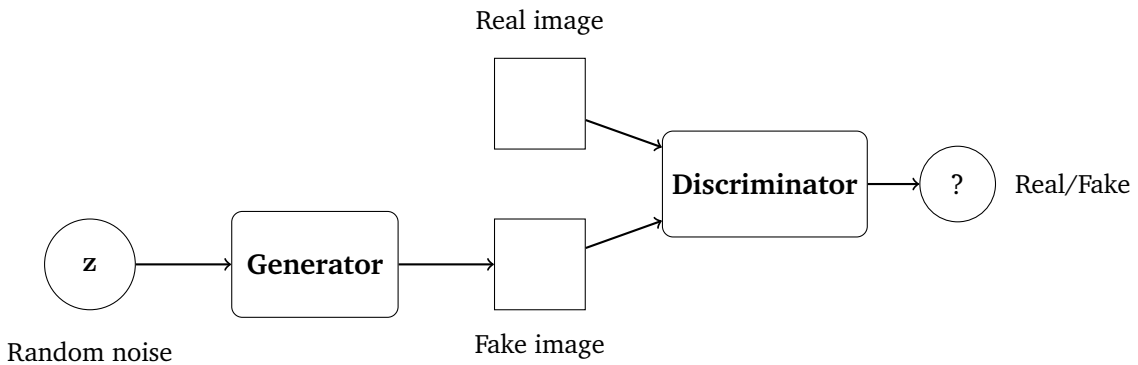
14.3 Mạng đối kháng tạo sinh (GAN)

Mạng đối kháng tạo sinh, được Ian Goodfellow giới thiệu năm 2014,¹ đã cách mạng hóa mô hình tạo sinh thông qua một mô hình huấn luyện khéo léo: để hai mạng nơ-ron cạnh tranh với nhau.

¹Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2672–2680.

14.3.1 Hai bên chơi

Một GAN gồm hai mạng nơ-ron. **Bộ sinh (G)** nhận nhiễu ngẫu nhiên z làm đầu vào và tạo ra các mẫu dữ liệu giả; mục tiêu của nó là tạo ra những mẫu thực tế đến mức lừa được bộ phân biệt. **Bộ phân biệt (D)** nhận dữ liệu thật từ tập huấn luyện hoặc dữ liệu giả từ bộ sinh, và cố gắng phân loại mỗi mẫu là “thật” hay “giả”.



Hình 14.1: Kiến trúc GAN: Bộ sinh tạo ra các mẫu giả, bộ phân biệt cố phát hiện chúng.

14.3.2 Trò chơi đối kháng

Quá trình huấn luyện được hình dung như một trò chơi hai người. Bộ phân biệt muốn tối đa hóa độ chính xác của mình, nhận diện đúng các mẫu thật là thật và các mẫu giả là giả. Bộ sinh muốn tối thiểu hóa độ chính xác của bộ phân biệt, tạo ra những mẫu giả tốt đến mức bộ phân biệt không thể phân biệt chúng với dữ liệu thật.

Điều này tạo ra một cuộc chạy đua vũ trang: khi bộ phân biệt ngày càng giỏi phát hiện hàng giả, bộ sinh phải tạo ra các mẫu ngày càng thực tế hơn để tiếp tục lừa được nó. Khi huấn luyện thành công, bộ sinh tạo ra những mẫu không thể phân biệt với dữ liệu thật.

Về mặt toán học, GAN giải quyết một bài toán tối ưu minimax:

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z} [\log(1 - D(G(\mathbf{z})))] \quad (14.1)$$

Bộ phân biệt muốn $D(\mathbf{x})$ gần bằng 1 với dữ liệu thật và $D(G(\mathbf{z}))$ gần bằng 0 với dữ liệu giả. Bộ sinh muốn điều ngược lại: $D(G(\mathbf{z}))$ gần bằng 1, nghĩa là bộ phân biệt đã bị lừa.

14.3.3 Điểm mạnh và thách thức

GAN tạo ra những hình ảnh sắc nét và thực tế đến đáng kinh ngạc, thường sắc hơn cả VAE. Tuy nhiên, chúng gây ra nhiều thách thức. **Sụp đổ chế độ** xảy ra khi bộ sinh học được chỉ tạo ra một vài loại đầu ra tình cờ lừa được bộ phân biệt, bỏ qua sự đa dạng đầy đủ của dữ liệu thật. **Mất ổn định khi huấn luyện** phát sinh vì thiết lập đối kháng rất nhạy cảm: nếu một mạng trở nên quá mạnh quá nhanh, mạng kia không thể học hiệu quả và việc huấn luyện thất bại. Khác với học có giám sát, nơi hàm mất mát giảm đều đặn, quá trình huấn luyện GAN có thể dao động hoặc phân kỳ. Cuối cùng, GAN không cung cấp **mật độ xác suất tường minh**: không giống VAE, chúng không học một phân phối xác suất tường minh trên dữ liệu, khiến việc tính một ảnh cụ thể có khả năng xuất hiện bao nhiêu trở nên khó khăn.

VAE so với GAN: Đánh đổi cốt lõi

VAE (Chương 10): Huấn luyện ổn định, cung cấp likelihood, không gian tiềm ẩn mượt mà. Nhưng đầu ra có xu hướng bị nhòe vì hàm mất mát lấy trung bình trên sự không chắc chắn.

GAN: Đầu ra sắc nét, thực tế. Nhưng việc huấn luyện không ổn định, sụp đổ chế độ có thể âm thầm làm giảm sự đa dạng, và không có cách nào tính một ảnh cụ thể có khả năng xuất hiện bao nhiêu theo mô hình. Không bên nào hoàn toàn tốt hơn. Lựa chọn phụ thuộc vào việc bạn cần huấn luyện ổn định và ước lượng mật độ (VAE), hay chất lượng hình ảnh tối đa (GAN).

14.4 Mô hình khuếch tán

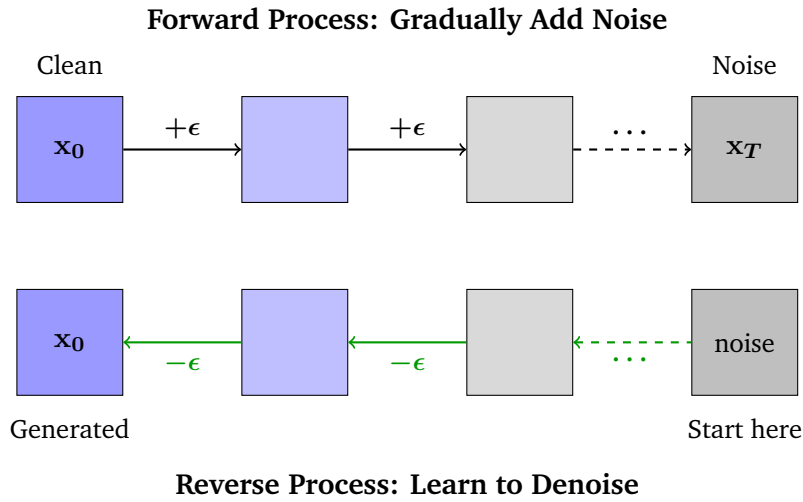
Mô hình khuếch tán đại diện cho trạng thái tiên tiến nhất hiện nay trong sinh ảnh, vận hành các hệ thống như DALL-E, Stable Diffusion và Midjourney. Chúng giải quyết phần lớn các vấn đề mất ổn định huấn luyện và sụp đổ chế độ vốn ám ảnh GAN, đổi lại là tốc độ sinh chậm hơn.

14.4.1 Quá trình thuận: Thêm nhiễu

Quá trình khuếch tán thuận bắt đầu với một mẫu dữ liệu sạch $\mathbf{x}_0$ và dần dần thêm nhiễu Gaussian qua nhiều bước thời gian cho đến khi dữ liệu trở thành nhiễu thuần túy:

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \quad (14.2)$$

Trong đó, β_t là một lịch nhiễu điều khiển lượng nhiễu cần thêm vào ở mỗi bước. Sau đủ số bước (thường là 1000), $\mathbf{x}_T$ không thể phân biệt được với nhiễu ngẫu nhiên thuần túy.



Hình 14.2: Mô hình khuếch tán: quá trình thuận phá hủy thông tin; quá trình ngược học cách khôi phục lại nó.

14.4.2 Quá trình ngược: Học cách khử nhiễu e

Điều kỳ diệu của mô hình khuếch tán là học **quá trình ngược**: bắt đầu từ nhiễu thuần túy và dần dần loại bỏ nhiễu để khôi phục một mẫu sạch. Một mạng nơ-ron được huấn luyện để dự đoán lượng nhiễu đã được thêm vào ở mỗi bước:

$$\epsilon_{\theta}(\mathbf{x}_t, t) \approx \epsilon \quad (14.3)$$

Mạng nhận một ảnh có nhiễu $\mathbf{x}_t$ và bước thời gian t , rồi dự đoán xem nhiễu nào đã được thêm vào. Điều này cho phép chúng ta “hoàn tác” nhiễu từng bước một.

14.4.3 Sinh ảnh

Để sinh một ảnh mới:

1. Bắt đầu với nhiễu ngẫu nhiên thuần túy $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. Với mỗi bước thời gian từ T xuống 0 :
 - Dùng mạng đã huấn luyện để dự đoán lượng nhiễu
 - Loại bỏ lượng nhiễu đã dự đoán để thu được $\mathbf{x}_{t-1}$
3. $\mathbf{x}_0$ cuối cùng chính là ảnh được sinh ra

14.4.4 Vì sao mô hình khuếch tán hoạt động tốt

Mô hình khuếch tán có nhiều ưu thế so với các cách tiếp cận trước đó. Việc **huấn luyện của chúng ổn định**: không giống GAN, không có động lực đối kháng nào có thể sụp đổ hoặc dao động. Hàm mất mát (dự đoán lượng nhiễu đã được thêm vào) đơn giản và dễ kiểm soát. **Chất lượng đầu ra** vô cùng xuất sắc bởi quá trình tinh chế lặp lại (dần dần loại bỏ nhiễu qua nhiều bước) cho phép mô hình sửa lỗi và bổ sung các chi tiết tinh vi theo từng bước. Cuối cùng, mô hình khuếch tán đạt **độ phủ chế độ** tốt: chúng có xu hướng nắm bắt toàn bộ sự đa dạng của dữ liệu huấn luyện thay vì sụp về vài đầu ra phổ biến.

Nhược điểm chính là tốc độ: sinh một ảnh đòi hỏi chạy mạng hàng trăm hoặc hàng nghìn lần (mỗi bước khử nhiễu một lần).

14.4.5 Vì sao khuếch tán thay thế GAN

Sự chuyển dịch từ GAN sang mô hình khuếch tán được thúc đẩy bởi ba ưu thế thực tế. **Ổn định huấn luyện**: huấn luyện GAN đòi hỏi cân bằng hai mạng cạnh tranh và có thể sụp đổ hoặc dao động không báo trước, trong khi huấn luyện khuếch tán dùng một hàm mất mát hồi quy đơn giản. **Độ phủ chế độ**: GAN thường xuyên mắc phải sụp đổ chế độ, âm thầm bỏ qua các phần của phân phối dữ liệu, trong khi mô hình khuếch tán luôn nắm bắt được toàn bộ sự đa dạng của dữ liệu huấn luyện. **Đánh đổi** là tốc độ: GAN sinh ảnh trong một lượt truyền tiến duy nhất, còn mô hình khuếch tán cần hàng trăm bước khử nhiễu. Những công trình gần đây về nén phân giải và bộ lấy mẫu nhanh hơn đã thu hẹp khoảng cách này, nhưng chưa xóa bỏ.

14.5 Transformer và mô hình ngôn ngữ lớn

Trong khi các phần trước tập trung vào sinh ảnh, kiến trúc transformer đã cách mạng hóa việc sinh văn bản và là nền tảng của các mô hình ngôn ngữ lớn (LLM) hiện đại như GPT và BERT.

14.5.1 Cơ chế attention

Đổi mới then chốt của transformer là **self-attention**, cho phép mỗi phần tử trong một chuỗi chú ý đến (tập trung vào) mọi phần tử khác. Điều này nắm bắt được các phụ thuộc phạm vi xa mà những kiến trúc trước đó như RNN gặp khó khăn.

Cho một chuỗi đầu vào, self-attention tính:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \right) \mathbf{V} \quad (14.4)$$

trong đó:

- **Q** (Query): Tôi đang tìm kiếm điều gì?
- **K** (Key): Tôi chứa điều gì?
- **V** (Value): Tôi cung cấp thông tin gì?

Trực giác: Attention như mức độ liên quan

Hãy hình dung việc đọc một câu như “Con mèo ngồi trên tấm thảm vì nó mệt.” Khi xử lý “nó”, attention cho phép mô hình nhìn lại “mèo” (attention cao) thay vì “thảm” (attention thấp) để hiểu “nó” chỉ điều gì.

14.5.2 Kiến trúc transformer

Một transformer gồm một **encoder** xử lý chuỗi đầu vào và hiểu ngữ cảnh của nó, và một **decoder** sinh chuỗi đầu ra trong khi chú ý đến cả đầu ra của encoder lẫn các token đã sinh trước đó. Một số mô hình chỉ dùng encoder (BERT), những mô hình khác chỉ dùng decoder (GPT), và một số dùng cả hai (transformer gốc cho dịch thuật).

Mỗi lớp chứa self-attention, theo sau là một mạng feed-forward, cùng với kết nối residual và chuẩn hóa lớp.

14.5.3 Mô hình ngôn ngữ lớn

Các LLM như GPT là những transformer được huấn luyện trên kho văn bản khổng lồ để dự đoán từ tiếp theo trong một chuỗi. Qua hàm mất mát tương tự như đơn giản này, chúng học được nhiều hơn hẳn việc dự đoán từ. Chúng nắm thuộc ngữ pháp và cú pháp, tích lũy tri thức về thế giới, khám phá các khuôn mẫu suy luận, và nội tại hóa nhiều văn phong viết khác nhau. Tri thức nổi lên một cách tiềm ẩn từ các khuôn mẫu thống kê của ngôn ngữ.

GPT (Generative Pre-trained Transformer) chỉ dùng phần decoder, sinh văn bản theo cách tự hồi quy. Từng token một, với mỗi token mới phụ thuộc vào tất cả các token trước đó.

BERT (Bidirectional Encoder Representations from Transformers) chỉ dùng phần encoder, được huấn luyện bằng mô hình ngôn ngữ có che (dự đoán các từ bị che dựa trên ngữ cảnh từ cả hai chiều).

14.5.4 Mã hóa token

LLM không xử lý văn bản thô. Chúng làm việc với **token**: các đơn vị có thể là cả một từ, mảnh từ con, hoặc từng ký tự đơn lẻ. Việc lựa chọn

cách mã hóa token ảnh hưởng đến hành vi của mô hình theo những cách quan trọng đối với việc hiểu năng lực và hạn chế của LLM. Các từ hiếm hoặc chuyên ngành bị tách thành nhiều token, khiến chúng khó xử lý hơn với mô hình. Các ngôn ngữ có bảng ký tự lớn hơn (như tiếng Trung hay tiếng Ả Rập) cần nhiều token hơn cho mỗi câu so với tiếng Anh, làm tăng chi phí tính toán và giảm độ dài ngữ cảnh hiệu dụng. Khi một bài thi IAIO hỏi về số tham số của LLM hay chi phí tính toán, việc mã hóa token chính là cầu nối giữa văn bản đọc được của con người và những con số mà mô hình thực sự thao tác.

14.5.5 Tính số tham số

Hiểu quy mô của LLM đòi hỏi phải hiểu số tham số.

Ví dụ: Số tham số của ma trận embedding. Một mô hình ngôn ngữ có kích thước từ vựng 10,000 và số chiều embedding là 256. Ma trận embedding chứa bao nhiêu tham số? Mỗi token cần một vector 256 chiều, nên tổng cộng là $10,000 \times 256 = 2,560,000$ tham số.

Ví dụ: Số tham số của mạng feed-forward. Một lớp transformer có một mạng feed-forward với chiều đầu vào $d = 256$ và chiều ẩn $f = 1024$. Có bao nhiêu tham số (không tính bias)? Có hai phép biến đổi tuyến tính: $d \rightarrow f$ rồi $f \rightarrow d$. Tổng cộng là $d \times f + f \times d = 256 \times 1024 + 1024 \times 256 = 524,288$ tham số.

14.5.6 Vì sao mở rộng quy mô hiệu quả (và khi nào nó dừng)

Một phát hiện thực nghiệm đáng chú ý là hiệu năng của LLM cải thiện một cách có thể dự đoán được khi bạn tăng kích thước mô hình, dữ liệu huấn luyện và năng lực tính toán. Những **quy luật scaling** này tuân theo các quan hệ lũy thừa:² nhân đôi năng lực tính toán mang lại một sự cải thiện nhất quán và đo lường được. Chính khả năng dự đoán này biện minh cho khoản đầu tư khổng lồ vào việc huấn luyện các mô hình ngày càng lớn.

Nhưng việc mở rộng quy mô có giới hạn. Nhiều tham số và dữ liệu hơn sẽ giảm perplexity (sai số dự đoán) nhưng không loại bỏ các vấn đề căn bản. Hiện tượng ảo giác vẫn tồn tại vì hàm mất mát huấn luyện thưởng cho văn bản nghe hợp lý chứ không thưởng cho sự thật. Những thất bại trong suy luận vẫn tồn tại vì dù tinh vi đến đâu, việc khớp mẫu vẫn không giống với lập luận logic. Và chi phí tính toán cùng môi trường tăng nhanh hơn lợi ích thu được: mỗi thế hệ mô hình mới cần khoảng mười lần năng lực tính toán hơn cho những cải thiện nhỏ dần. Mở rộng quy mô là động cơ của tiến bộ hiện tại, nhưng khó có khả năng đủ một mình.

²Kaplan, J. et al. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

14.6 Sinh ảnh từ văn bản

Các hệ thống sinh ảnh từ văn bản hiện đại như DALL-E và Stable Diffusion kết hợp những kỹ thuật đã bàn ở trên. Kiến trúc chung có ba giai đoạn: một bộ mã hóa văn bản (thường là CLIP) chuyển câu lệnh văn bản thành một biểu diễn vector; một quá trình khuếch tán, thường hoạt động trong không gian tiềm ẩn nén thay vì không gian pixel để tăng hiệu quả, sinh ra một biểu diễn ảnh có điều kiện theo embedding văn bản; và một bộ giải mã chuyển kết quả thành ảnh ở độ phân giải đầy đủ. Các hệ thống khác nhau khác nhau ở những chi tiết kiến trúc, nhưng quy trình khái niệm là như nhau: hiểu ngôn ngữ dẫn hướng việc sinh ảnh.

14.7 Bối cảnh AI ngày nay

Những công nghệ này đã thay đổi những gì mà các hệ thống AI do:

Ngôn ngữ: Mô hình ngôn ngữ lớn có thể viết luận văn, code, thơ, và đối thoại tinh vi. Chúng thể hiện những năng lực nổi lên mà không được lập trình tường minh.

Hình ảnh: Mô hình khuếch tán sinh ra ảnh chân thực như ảnh chụp từ các mô tả văn bản, mở ra những hình thức biểu đạt sáng tạo mới và nêu ra các câu hỏi về tính xác thực.

Đa phương thức: Các hệ thống ngày càng kết hợp nhiều phương thức, hiểu hình ảnh và văn bản cùng nhau, sinh ảnh từ văn bản, và mô tả hình ảnh bằng lời.

Suy luận: Các mô hình gần đây cho thấy khả năng phân rã vấn đề phức tạp được cải thiện, dù chúng vẫn khó khăn với một số loại suy luận logic nhất định.

14.8 Cân nhắc đạo đức

Các hệ thống trong chương này khuếch đại những thách thức đạo đức được đề cập ở **Chương 2**. Ba đặc tính của học sâu hiện đại khiến sự khuếch đại này trở nên đáng kể.

Thứ nhất, các mô hình này được huấn luyện trên dữ liệu quy mô internet thay vì các bộ dữ liệu được tuyển chọn cẩn thận, nên chúng hấp thụ mọi thành kiến, định kiến và thông tin sai lệch có trong dữ liệu đó. Bước tuyển chọn có thể phát hiện ra vấn đề ở các hệ thống nhỏ hơn gần như vắng mặt. Thứ hai, các mô hình tạo sinh tạo ra nội dung được hàng triệu người tiêu thụ, nên những đầu ra bị thiên lệch không chỉ ảnh hưởng đến các quyết định cá nhân; chúng định hình các chuẩn mực văn hóa và nhận thức của công chúng. Thứ ba, khối lượng đầu ra khổng lồ (hàng triệu phần hoàn thành văn bản hoặc hình ảnh mỗi ngày) khiến việc con người rà soát có ý nghĩa từng đầu ra là bất khả thi.

Kết quả là những vấn đề như khuếch đại thành kiến, vòng lặp phản hồi và lỗi hỏng trách nhiệm giải trình — tất cả đều được phân tích bằng các công cụ hình thức ở Chương 2 — lại vận hành ở một quy mô về chất khác hẳn. Để xem toàn bộ nội dung, bao gồm định nghĩa hình thức về công bằng, quy trình phát hiện và giảm thiểu thành kiến, và khuôn dạng phân tích năm điểm, hãy xem **Chương 2: Tác động xã hội của AI**.

14.9 Hạn chế và những vấn đề mở

Dù có những năng lực ấn tượng, các hệ thống hiện tại vẫn có những hạn chế căn bản:

Ảo giác: Mô hình ngôn ngữ tự tin nói ra thông tin sai. Chúng không có cơ chế đáng tin cậy để phân biệt điều mình biết với điều mình không biết.

Suy luận: Dù đang cải thiện, các mô hình vẫn khó khăn với suy luận logic nhiều bước, đặc biệt ở các lĩnh vực mới.

Cơ sở thực tế: Các mô hình học từ văn bản và hình ảnh nhưng không có trải nghiệm trực tiếp về thế giới vật lý, điều này giới hạn khả năng hiểu quan hệ nhân quả và vật lý của chúng.

Căn chỉnh: Đảm bảo các hệ thống AI theo đuổi các mục tiêu dự định một cách an toàn vẫn là một vấn đề nghiên cứu mở.

Ghi nhớ và rò rỉ dữ liệu: Các mô hình được huấn luyện trên dữ liệu quy mô internet có thể ghi nhớ và tái tạo nguyên văn các ví dụ huấn luyện, gây ra cả lo ngại về bản quyền lẫn quyền riêng tư. Tình vi hơn, nếu tập kiểm tra của benchmark từng xuất hiện trong dữ liệu huấn luyện thì hiệu năng được báo cáo sẽ phóng đại năng lực thực sự. **Sự nhiễm tập huấn luyện** này làm cho việc đánh giá mức độ mô hình thực sự khái quát hóa so với mức độ chúng chỉ ghi nhớ trở nên khó khăn. Việc đánh giá chất lượng tạo sinh không có một số đo duy nhất nào là đúng (Chương 10 thảo luận sâu về điều này cho các mô hình ảnh, và cùng thách thức đó áp dụng cho việc sinh văn bản).

Đánh giá: Khác với phân loại, nơi độ chính xác trên một tập kiểm tra tạo ra một tín hiệu rõ ràng, không có một số đo duy nhất nào là đúng cho chất lượng tạo sinh. Các số đo khác nhau bắt được các khía cạnh khác nhau (độ trôi chảy, độ chính xác thực tế, sự đa dạng, tính mạch lạc), và tối ưu một số đo có thể làm suy giảm các số đo khác.

14.10 Tóm tắt

Khái niệm chính

GAN: Bộ sinh đối đầu với bộ phân biệt trong huấn luyện đối kháng. Đầu ra sắc nét nhưng huấn luyện không ổn định và có nguy cơ sụp đổ chế độ.

Mô hình khuếch tán: Học cách đảo ngược một quá trình thêm nhiễu. Bắt đầu từ nhiễu thuần túy và lặp lại khử nhiễu để sinh ảnh. Hiện nay tạo ra hình ảnh chất lượng cao nhất.

Transformer: Dùng cơ chế attention để xử lý chuỗi. Mỗi vị trí có thể chú ý đến mọi vị trí khác, nắm bắt các phụ thuộc phạm vi xa. Nền tảng của các mô hình ngôn ngữ hiện đại.

Attention: Đổi mới then chốt: tính trọng số mức độ liên quan giữa mọi cặp vị trí, cho phép định tuyến thông tin một cách linh hoạt. **Mô hình ngôn ngữ lớn:** Transformer được huấn luyện trên kho văn bản khổng lồ. Các quy luật scaling dự đoán rằng nhiều tham số và dữ liệu hơn sẽ cải thiện hiệu năng, nhưng với lợi ích giảm dần và không loại bỏ các hạn chế căn bản.

Sinh ảnh từ văn bản: Kết hợp hiểu ngôn ngữ (thường thông qua CLIP) với sinh ảnh (mô hình khuếch tán) để tạo ra hình ảnh từ các mô tả.

Hạn chế: Ảo giác, ghi nhớ dữ liệu huấn luyện, nhiệm benchmark, và việc không có một số đo đánh giá đáng tin cậy duy nhất vẫn là những thách thức căn bản.

14.11 Bài tập luyện

- Giải thích ý tưởng then chốt đằng sau mô hình khuếch tán. Vì sao việc tinh chế lặp lại có thể cho kết quả tốt hơn so với sinh ảnh chỉ trong một lần?
- Trong cơ chế attention:
 - Về mặt khái niệm, các ma trận Query, Key và Value đại diện cho điều gì?
 - Vì sao attention được gọi là "all you need" cho mô hình hóa chuỗi? (c) Độ phức tạp tính toán của self-attention với độ dài chuỗi n là bao nhiêu?
- Một mô hình ngôn ngữ có 175 tỷ tham số. Nếu mỗi tham số được lưu dưới dạng số thực dấu phẩy 16-bit, cần bao nhiêu bộ nhớ chỉ để lưu trọng số của mô hình?
- (Khái niệm)** Các mô hình ngôn ngữ được huấn luyện để dự đoán token tiếp theo.

Hãy giải thích làm thế nào hàm mất mát đơn giản này dẫn đến những mô hình có thể trả lời câu hỏi, viết code, và đối thoại.

5. **(Tư duy phản biện)** Mô hình ngôn ngữ lớn đôi khi “ảo giác”, tự tin nói ra thông tin sai.

(a) Vì sao hàm mất mát huấn luyện (dự đoán token tiếp theo) lại không ngăn được hiện tượng ảo giác?

(b) Hãy đề xuất hai cách tiếp cận có thể giảm hiện tượng ảo giác.

6. **(GAN)** Trong một GAN, bộ phân biệt đầu ra $D(x) \in [0, 1]$.

(a) $D(x) = 0.9$ có nghĩa là gì?

(b) Bộ sinh muốn tối đa hóa $D(G(z))$. Hãy giải thích vì sao. (c)

Sụp đổ chế độ là gì và vì sao nó xảy ra?

Phụ lục A

Chiến lược thi thực tế

Tra tấn dữ liệu, và nó sẽ thú
nhận điều bất cứ điều gì.

Ronald Coase

A.1 Giới thiệu

Các cuộc thi học máy kiểm tra khả năng xây dựng mô hình hoạt động tốt trên dữ liệu thực của bạn. Dù bạn thi trên Kaggle hay vòng thực hành của IAIO, quy trình cốt lõi và các nguyên tắc chiến lược vẫn như nhau. Phụ lục này đề cập các chiến lược không phụ thuộc nền tảng, giúp bạn thành công trong bất kỳ hình thức thi nào.

A.2 Hiểu cấu trúc cuộc thi

A.2.1 Phân chia dữ liệu

Mỗi cuộc thi đều cung cấp dữ liệu được chia thành các tập riêng biệt:

Tập huấn luyện: Chứa đặc trưng và nhãn. Bạn học từ dữ liệu này.

Tập kiểm tra: Chỉ chứa đặc trưng. Bạn dự đoán nhãn cho dữ liệu này.

File mẫu nộp bài: Cho biết định dạng chính xác mà dự đoán của bạn phải tuân theo.

Điểm mẫu chốt là tập kiểm tra thường còn được chia nhỏ hơn:

Tập kiểm tra công khai: Một phần (thường 30–50%) dùng cho bảng xếp hạng công khai

mà bạn nhìn thấy trong suốt cuộc thi.

Tập kiểm tra riêng tư: Phần còn lại, dùng cho bảng xếp hạng cuối cùng chỉ được công bố sau khi cuộc thi kết thúc.

Cảnh báo nghiêm trọng: Quá khớp vào bảng xếp hạng

Bảng xếp hạng công khai chỉ hiển thị hiệu năng trên một phần của dữ liệu kiểm tra. Nếu bạn tối ưu riêng cho tập con này (bằng cách thực hiện nhiều lượt nộp và chọn những lượt có điểm cao), bạn đang quá khớp vào tập kiểm tra công khai.

Điều này nguy hiểm vì:

- Hiệu năng trên tập kiểm tra công khai và riêng tư có thể khác nhau đáng kể
- Mô hình quá khớp vào dữ liệu công khai thường giảm mạnh trên dữ liệu riêng tư
- Nhiều thí sinh đã mất vị trí giành huy chương vì lỗi này

Hãy tin điểm kiểm chéo của bạn, chứ không phải điểm bảng xếp hạng công khai. Đây chính là vấn đề quá khớp ở Chương 5, nhưng áp dụng cho một cách chia khác. Mỗi lần bạn kiểm tra bảng xếp hạng công khai và điều chỉnh cách tiếp cận dựa trên kết quả đó, bạn đang trên thực tế dùng tập kiểm tra công khai làm tập kiểm định. Với đủ nhiều lượt nộp, bạn sẽ tìm ra những mô hình tình cờ đạt điểm cao trên tập con đó mà không thực sự khái quát hóa tốt hơn. Điểm bảng xếp hạng trở thành một ước lượng nhiễu và lạc quan quá mức về hiệu năng thực, giống như độ chính xác trên tập huấn luyện trở nên lạc quan quá mức khi bạn tính chỉnh mô hình quá mạnh theo dữ liệu huấn luyện.

A.2.2 Chỉ số đánh giá

Hiểu rõ chỉ số đánh giá là điều thiết yếu: nó định nghĩa “tốt hơn” nghĩa là gì.

Chỉ số phân loại:

- **Độ chính xác (accuracy):** Tỷ lệ dự đoán đúng. Gây hiểu lầm khi các lớp mất cân bằng.
- **Điểm F1:** Trung bình điều hòa của độ chính xác (precision) và độ nhạy (recall). Phù hợp với dữ liệu mất cân bằng.
- **Log Loss:** Phạt nặng các dự đoán sai nhưng tự tin. Yêu cầu đầu ra dạng xác suất.
- **AUC-ROC:** Đo chất lượng khả năng xếp hạng trên mọi ngưỡng.

Chỉ số hồi quy:

- **RMSE:** Sai số bình phương trung bình gốc. Phạt rất nặng các sai số lớn.

- **MAE**: Sai số tuyệt đối trung bình. Bền vững hơn đối với ngoại lai.
- **R^2** : Tỷ lệ phương sai được giải thích.

Các chỉ số khác nhau đòi hỏi những chiến lược tối ưu khác nhau. Với log loss, việc hiệu chuẩn xác suất rất quan trọng. Với RMSE, xử lý ngoại lai là yếu tố then chốt.

A.3 Quy trình thi

Các giai đoạn sau mô tả một quy trình điển hình. Tỷ lệ thời gian là những điểm khởi đầu mang tính minh họa, không phải quy tắc cứng. Hãy điều chỉnh chúng dựa trên thể lệ thi, kinh nghiệm của bạn và yêu cầu mà dữ liệu đặt ra. Có bài toán cần kỹ thuật đặc trưng nặng; số khác lại được thắng lợi nhờ chọn mô hình cẩn thận. Điều quan trọng là có cách tiếp cận có hệ thống, chứ không phải tuân thủ những tỷ lệ phần trăm tuyệt đối.

A.3.1 Giai đoạn 1: Hiểu bài toán

Trước khi viết bất kỳ dòng mã nào:

1. Đọc kỹ toàn bộ phần mô tả bài toán
2. Xác định: Đây là phân loại hay hồi quy?
3. Hiểu chỉ số đánh giá và điều gì được chỉ số đó ưu ái
- 4.

Kiểm tra các yêu cầu về định dạng nộp bài

5. Đọc phần mô tả dữ liệu, mỗi cột nghĩa là gì?
6. Tìm kiến thức miền có thể gợi ý cho kỹ thuật đặc trưng

A.3.2 Giai đoạn 2: Thiết lập đường cơ sở

Hãy tạo quy trình xử lý đơn giản nhất có thể chạy được:

1. Nạp dữ liệu huấn luyện và kiểm tra
2. Tiền xử lý tối thiểu: xử lý đơn giản các giá trị thiếu (điền bằng trung vị/chế độ)
3. Dùng tất cả đặc trưng được cung cấp mà không cần kỹ thuật
4. Huấn luyện một mô hình đơn giản (hồi quy logistic hoặc rừng ngẫu nhiên nhỏ)
5. Tạo dự đoán đúng định dạng nộp bài
6. Nộp bài và ghi lại điểm đường cơ sở của bạn

Đường cơ sở này có hai mục đích: nó xác nhận quy trình của bạn chạy được từ đầu đến cuối, và cho bạn một mốc tham chiếu để đo những cải thiện.

A.3.3 Giai đoạn 3: Phân tích dữ liệu khám phá

Bây giờ hãy xem xét kỹ dữ liệu của bạn:

- **Phân phối:** Các đặc trưng có phân phối chuẩn không? Có lệch không? Có ngoại lai không?
- **Giá trị thiếu:** Cột nào có dữ liệu thiếu? Việc thiếu dữ liệu là ngẫu nhiên hay mang thông tin?
- **Tương quan:** Những đặc trưng nào tương quan với biến mục tiêu? Với lẫn nhau?
- **Kiểu dữ liệu:** Các đặc trưng phân loại đã được mã hóa đúng chưa?
- **Khác biệt giữa tập huấn luyện và tập kiểm tra:** Tập kiểm tra có phân phối khác với tập huấn luyện không?

A.3.4 Giai đoạn 4: Kỹ thuật đặc trưng

Đây thường là nơi quyết định thắng hay thua một cuộc thi:

- **Đặc trưng theo miền:** Dùng kiến thức về bài toán để tạo ra các tổ hợp có ý nghĩa
- **Tương tác:** Tích hoặc tỷ số của các đặc trưng đã có
- **Tổng hợp:** Thống kê theo nhóm (trung bình, độ lệch chuẩn theo danh mục)
- **Đặc trưng thời gian:** Nếu dữ liệu có dấu thời gian, ngày trong tuần, tháng, thời gian kể từ sự kiện
- **Đặc trưng văn bản:** TF-IDF, số lượng từ, điểm sắc thái
- **Mã hóa mục tiêu:** Thay thế các danh mục bằng thống kê của mục tiêu (cẩn thận với rò rỉ dữ liệu!)

Hãy kiểm tra tác động của từng đặc trưng một cách có hệ thống. Không phải đặc trưng nào cũng hữu ích, một số chỉ làm tăng nhiễu.

A.3.5 Giai đoạn 5: Phát triển mô hình

Khi đã có đặc trưng tốt, hãy thử các mô hình khác nhau:

- **Bắt đầu đơn giản:** Mô hình tuyến tính, cây quyết định nhỏ

- **Thủ tập hợp cây:** Random Forest, XGBoost, LightGBM (thường tốt nhất với dữ liệu dạng bảng)
- **Cân nhắc mạng nơ-ron:** Cho ảnh, văn bản hoặc tập dữ liệu rất lớn
- **Tình chỉnh siêu tham số:** Tập trung vào những siêu tham số có tác động lớn nhất (tốc độ học, chỉnh quy hóa)

A.3.6 Giai đoạn 6: Kiểm định và lựa chọn

Ở giai đoạn cuối, bạn thường chọn 2–3 lượt nộp bài cuối cùng. Hãy lựa chọn cẩn thận:

- Một lượt nộp có điểm tốt nhất trên kiểm chéo
- Một lượt hơi khác (khác mô hình hoặc khác đặc trưng) để phòng ngừa rủi ro
- Tránh lựa chọn chỉ dựa vào điểm bảng xếp hạng công khai

A.4 Tránh rò rỉ dữ liệu

Rò rỉ dữ liệu xảy ra khi thông tin từ bên ngoài tập huấn luyện ảnh hưởng không đúng đến mô hình của bạn. Nó tạo ra điểm kiểm định cao một cách không tự nhiên và không thể khái quát hóa.

Các nguồn rò rỉ thường gặp:

- **Rò rỉ mục tiêu:** Một đặc trưng được suy ra từ mục tiêu hoặc tương quan mạnh với mục tiêu
- **Rò rỉ theo thời gian:** Dùng thông tin tương lai để dự đoán quá khứ
- **Nhiễm chéo giữa tập huấn luyện và tập kiểm tra:** Thống kê của dữ liệu kiểm tra rò rỉ vào bước tiền xử lý dữ liệu huấn luyện
- **Kiểm chéo không đúng cách:** Không tôn trọng thứ tự thời gian hoặc cấu trúc nhóm

Phòng ngừa:

- Chỉ khớp bước tiền xử lý (bộ chuẩn hóa, bộ mã hóa) trên các nếp của tập huấn luyện
- Với chuỗi thời gian, luôn kiểm định trên dữ liệu tương lai
- Hãy cảnh giác với những đặc trưng trông “quá hoàn hảo”
- Kiểm tra rằng điểm kiểm chéo và điểm công khai tương đối nhất quán

A.5 Chiến lược kiểm chéo

Cách chia ngẫu nhiên đơn giản có thể gây hiểu lầm. Hãy chọn chiến lược kiểm định dựa trên cấu trúc dữ liệu:

K-Fold: Lựa chọn tiêu chuẩn. Dùng $k = 5$ hoặc $k = 10$.

Stratified K-Fold: Dành cho bài phân loại có các lớp mất cân bằng. Giữ nguyên tỷ lệ các lớp trong mỗi nếp.

Chia theo thời gian: Dành cho dữ liệu theo thời gian. Huấn luyện trên quá khứ, kiểm định trên tương lai.

Group K-Fold: Khi dữ liệu có các nhóm tự nhiên (ví dụ, nhiều mẫu trên mỗi người dùng). Giữ tất cả các mẫu của một nhóm trong cùng một nếp.

Chiến lược kiểm định nên mô phỏng cách tập kiểm tra liên quan đến dữ liệu huấn luyện.

A.6 Phương pháp tập hợp

Kết hợp nhiều mô hình thường cải thiện hiệu năng:

Trung bình: Chỉ cần lấy trung bình các dự đoán từ những mô hình khác nhau. Hiệu quả khi các mô hình mắc lỗi không tương quan với nhau.

Trung bình có trọng số: Cho mô hình tốt hơn nhiều trọng số hơn.

Stacking: Huấn luyện một mô hình meta trên các dự đoán của các mô hình cơ sở. Mạnh mẽ hơn nhưng có nguy cơ quá khớp.

Blending: Giống stacking nhưng đơn giản hơn, dùng dự đoán trên tập giữ lại thay vì kiểm chéo.

Ngay cả việc lấy trung bình đơn giản của 3–5 mô hình đa dạng cũng thường tốt hơn bất kỳ mô hình đơn lẻ nào.

A.7 Quản lý thời gian trong cuộc thi có giới hạn thời gian

Đối với vòng thực hành của IAIO hoặc các cuộc thi có giới hạn thời gian tương tự:

Trước cuộc thi:

- Chuẩn bị sẵn mã mẫu cho các tác vụ thường gặp
- Nắm rõ công cụ của bạn, đừng học thư viện mới trong lúc thi
- Luyện tập toàn bộ quy trình trên các bài toán tương tự

Trong cuộc thi:

- Dành 15–20% thời gian đầu để hiểu bài toán và có được một đường cơ sở
- Đừng sẵn những cải thiện nhỏ trong giai đoạn đầu, hãy tập trung vào những thắng lợi lớn trước
- Dành 20–30 phút ở cuối cho việc chuẩn bị nộp bài lần cuối
- Đảm bảo quy trình nộp bài của bạn hoạt động trước khi hết giờ

Thứ tự ưu tiên khi thời gian có hạn:

1. Đường cơ sở chạy được (bắt buộc)
2. Kỹ thuật đặc trưng cơ bản
3. Một mô hình tốt với siêu tham số hợp lý 4. Tập hợp mô hình nếu còn thời gian

A.8 Những lỗi thường gặp cần tránh

1. **Quá khớp vào bảng xếp hạng công khai:** Hãy tin kiểm chéo, không phải điểm công khai
2. **Sai định dạng nộp bài:** Luôn kiểm tra định dạng có khớp chính xác với file mẫu
3. **Rò rỉ dữ liệu:** Hãy cực kỳ cảnh giác về luồng thông tin
4. **Bỏ qua chỉ số đánh giá:** Hãy tối ưu cho thứ được đo, không phải thứ tiện lợi
5. **Làm phức tạp quá sớm:** Bắt đầu đơn giản, rồi tăng dần độ phức tạp
6. **Không lưu lại công việc:** Quản lý phiên bản cho toàn bộ mã và kết quả mô hình
7. **Hết thời gian:** Để lại khoảng đệm cho các lượt nộp cuối

A.9 Tóm tắt

Competition Checklist

Before starting: ☐ Read problem description completely ☐ Understand evaluation metric ☐ Check submission format

During competition: ☐ Establish working baseline first ☐ Use cross-validation, not public leaderboard ☐ Test features and models systematically ☐ Watch for data leakage

Before submitting: ☐ Verify submission format matches sample ☐ Select submissions based on CV score ☐ Leave time buffer for technical issues

Những kỹ năng bạn rèn luyện trong các cuộc thi (thử nghiệm có hệ thống, kiểm định nghiêm túc, kỹ thuật đặc trưng thực tế) chuyển hóa trực tiếp vào công việc học máy ngoài đời thực. Mỗi cuộc thi đều khiến bạn trở thành một người thực hành giỏi hơn.

Phụ lục B

Ký hiệu Toán học

Ký hiệu toán học là một ngôn ngữ được thiết kế để diễn đạt các ý tưởng toán học một cách chính xác và cô đọng.

Tobias Dantzig

B.1 Giới thiệu

Toán học có ngôn ngữ riêng của nó: một hệ thống các ký hiệu cho phép chúng ta diễn đạt những ý tưởng phức tạp một cách chính xác và ngắn gọn. Khi lần đầu nhìn thấy một trang đầy những ký hiệu toán học, bạn có cảm giác như đang đọc một ngôn ngữ nước ngoài. Nhưng cũng như bất kỳ ngôn ngữ nào, một khi bạn đã học được từ vựng và ngữ pháp, đó sẽ trở thành một công cụ giao tiếp mạnh mẽ.

Phụ lục này đóng vai trò sách tra cứu dành cho bạn về các ký hiệu toán học được sử dụng xuyên suốt cuốn sách. Chúng ta bắt đầu với ký hiệu toán học tổng quát mà bạn sẽ gặp ở mọi nơi trong toán học, sau đó chuyển sang các ký hiệu riêng của học máy và trí tuệ nhân tạo.

B.2 Số và các phép tính cơ bản

B.2.1 Tập hợp số

Các nhà toán học có những ký hiệu riêng cho các tập hợp số thường dùng:

Ký hiệu	Tên	Mô tả
$\mathbb{N}$	Số tự nhiên	$\{0, 1, 2, 3, \dots\}$ or $\{1, 2, 3, \dots\}$
$\mathbb{Z}$	Số nguyên	$\{\dots, -2, -1, 0, 1, 2, \dots\}$
$\mathbb{Q}$	Số hữu tỉ	Fractions $\frac{p}{q}$ where $p, q \in \mathbb{Z}, q \neq 0$
$\mathbb{R}$	Số thực	All points on the number line
$\mathbb{R}^+$	Số thực dương	$\{x \in \mathbb{R} : x > 0\}$
$\mathbb{R}^n$	n không gian n -chiều	Vectors with n real components

Phông chữ đậm bảng đen ($\mathbb{R}$ thay vì $\mathbf{R}$) báo hiệu rằng chúng ta đang nói về một tập hợp số tiêu chuẩn.

B.2.2 Các phép tính số học

Hầu hết các ký hiệu số học đã quen thuộc, nhưng có vài ký hiệu đáng lưu ý:

Ký hiệu	Ý nghĩa	Ví dụ
$+, -, \times, \div$	Số học cơ bản	$3 + 4 = 7$
$\cdot$	Phép nhân	$3 \cdot 4 = 12$
a^n	Phép lũy thừa	$2^3 = 8$
$\sqrt{x}, x^{1/2}$	Căn bậc hai	$\sqrt{16} = 4$
$ x $	Giá trị tuyệt đối	$ -5 = 5$
$\lfloor x \rfloor$	Phần nguyên (làm tròn xuống)	$\lfloor 3.7 \rfloor = 3$
$\lceil x \rceil$	Trần (làm tròn lên)	$\lceil 3.2 \rceil = 4$
$n!$	Giai thừa	$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$
$\binom{n}{k}$	Hệ số nhị thức	$\binom{5}{2} = \frac{5!}{2!(5-2)!} = 10$

B.3 Tập hợp và các phép toán tập hợp

Một **tập hợp** là một tập hợp các đối tượng phân biệt nhau. Tập hợp là nền tảng của toán học và xuất hiện khắp nơi trong học máy: tập huấn luyện, lớp giả thuyết, không gian đặc trưng.

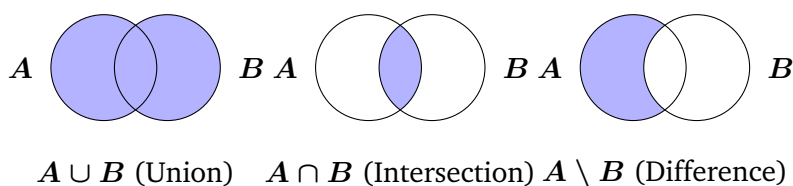
B.3.1 Ký hiệu tập hợp

Ký hiệu	Ý nghĩa	Ví dụ
$\{a, b, c\}$	Tập hợp gồm các phần tử a, b, c	$\{1, 2, 3\}$
$\{x : \text{điều kiện}\}$	Tập hợp các x thỏa mãn điều kiện	$\{x : x > 0\} = \text{số dương}$
$\emptyset$ or $\{\}$	Tập rỗng	Tập hợp không có phần tử nào
$ S $	Cỡ (số phần tử) của tập hợp S	$ \{a, b, c\} = 3$
$\in$	Thuộc tập hợp	$3 \in \{1, 2, 3\}$
$\notin$	Không thuộc tập hợp	$4 \notin \{1, 2, 3\}$
$\subseteq$	Tập con (hoặc bằng nhau)	$\{1, 2\} \subseteq \{1, 2, 3\}$
$\subset$	Tập con thực sự	$\{1, 2\} \subset \{1, 2, 3\}$

B.3.2 Các phép toán tập hợp và anh em họ logic của chúng

Các phép toán tập hợp có một sự tương ứng tuyệt vời với các phép toán logic. Hiểu được mối liên hệ này giúp bạn nhận ra cùng một ý tưởng ở những bối cảnh khác nhau.

Set	Logic	Name	Meaning
$A \cup B$	$A \vee B$	Union / OR	Elements in <i>A</i> or <i>B</i> (or both)
$A \cap B$	$A \wedge B$	Intersection / AND	Elements in <i>A</i> and <i>B</i>
A^c or $\bar{A}$	$\neg A$	Complement / NOT	Elements not in <i>A</i>
$A \setminus B$	$A \wedge \neg B$	Difference	Elements in <i>A</i> but not in <i>B</i>



Hình B.1: Các sơ đồ Venn minh họa các phép toán tập hợp. Vùng tô đậm biểu diễn kết quả.

Sự tương ứng giữa tập hợp và logic không phải là ngẫu nhiên. Trong cả hai trường hợp, chúng ta đều đang kết hợp các điều kiện: “thuộc tập hợp A ” là một điều kiện mà với mọi phần tử đều hoặc đúng hoặc sai, giống hệt một mệnh đề logic.

B.4 Ký hiệu tổng và tích

Khi cần cộng hoặc nhân nhiều số hạng, việc viết ra toàn bộ sẽ trở nên bất khả thi. Ký hiệu tổng và tích cung cấp một cách cô đọng để biểu diễn những phép tính này.

B.4.1 Tổng (Σ)

Chữ cái Hy Lạp sigma (Σ) biểu thị phép tổng:

$$\sum_{i=1}^n a_i = a_1 + a_2 + a_3 + \cdots + a_n$$

Ký hiệu quy định: bắt đầu từ $i = 1$, kết thúc ở $i = n$, và cộng tất cả các số hạng a_i .

Ví dụ về tổng

Tổng 5 số nguyên đầu tiên:

$$\sum_{i=1}^5 i = 1 + 2 + 3 + 4 + 5 = 15$$

Tổng các bình phương:

$$\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

Trung bình của các điểm dữ liệu $x_1, \dots, x_n$:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

B.4.2 Tích (Π)

Chữ cái Hy Lạp pi (Π) biểu thị phép tích:

$$\prod_{i=1}^n a_i = a_1 \times a_2 \times a_3 \times \cdots \times a_n$$

Ví dụ về tích**Giai thừa dưới dạng một tích:**

$$n! = \prod_{i=1}^n i = 1 \times 2 \times 3 \times \cdots \times n$$

Tích các xác suất (với các sự kiện độc lập):

$$P(A_1 \cap A_2 \cap \cdots \cap A_n) = \prod_{i=1}^n P(A_i)$$

B.5 Hàm số

Một **hàm số** ánh xạ đầu vào sang đầu ra. Ký hiệu $f : A \rightarrow B$ nghĩa là " f là một hàm từ tập hợp A đến tập hợp B ." Với mỗi đầu vào $x \in A$, hàm số tạo ra đúng một đầu ra $f(x) \in B$.

B.5.1 Ký hiệu hàm thông dụng

Ký hiệu	Ý nghĩa	Ví dụ
$f(x)$	Giá trị của f tại x	$f(x) = x^2$ means $f(3) = 9$
$f : A \rightarrow B$	f ánh xạ từ A đến B	$f : \mathbb{R} \rightarrow \mathbb{R}$
$f \circ g$	Hợp thành: $f(g(x))$	$(f \circ g)(x) = f(g(x))$
f^{-1}	Hàm nghịch đảo	If $f(x) = 2x$, then $f^{-1}(x) = x/2$

B.5.2 Các hàm quan trọng

Một số hàm xuất hiện lặp đi lặp lại trong học máy:

Hàm số	Định nghĩa	Use in ML
$\exp(x)$ or e^x	Hàm mũ	Softmax, xác suất
$\ln(x)$ or $\log(x)$	Logarit tự nhiên	Hàm mất mát chéo
$\log_2(x)$	Logarit cơ số 2	Lý thuyết thông tin, entropi
$\sigma(x) = \frac{1}{1+e^{-x}}$	Hàm sigmoid	Hồi quy logistic
$\max(a, b)$	Cái lớn hơn trong a và b	ReLU, minimax
$\min(a, b)$	Cái nhỏ hơn trong a và b	Minimax
$\text{sign}(x)$	$+1$ nếu $x > 0$, -1 nếu $x < 0$	Phân loại

B.5.3 Hàm chỉ thị

Hàm **chỉ thị** $1[\cdot]$ trả về 1 nếu đối số của nó đúng, 0 nếu sai:

$$1[\text{condition}] = \begin{cases} 1 & \text{if condition is true} \\ 0 & \text{if condition is false} \end{cases}$$

Điều này cực kỳ hữu ích để đếm số và để biểu diễn gọn gàng các hàm theo đoạn.

Ví dụ: Đếm số lỗi

Số lần phân loại sai trong một tập dữ liệu:

$$\text{Errors} = \sum_{i=1}^n 1[h(x_i) \neq y_i]$$

Biểu thức này cộng thêm 1 cho mỗi ví dụ mà giá trị dự đoán $h(x_i)$ khác với nhãn thật y_i .

B.6 Chữ cái Hy Lạp

Chữ cái Hy Lạp được dùng rất rộng rãi trong toán học và học máy. Dưới đây là những chữ cái phổ biến nhất:

Letter	Name	Common Use	Letter	Name	Common Use
α	Alpha (α)	Tốc độ học	ν	Nu (ν)	Bậc tự do
β	Beta (β)	Các hệ số	π	Pi (π)	3.14159... hoặc chính sách
γ	Gamma (γ)	Hệ số chiết khấu	ρ	Rho (ρ)	Hệ số tương quan
δ	Delta (δ)	Thay đổi nhỏ	σ	Sigma (σ)	Độ lệch chuẩn
ϵ	Epsilon (ε)	Đại lượng nhỏ	τ	Tau (τ)	Nhiệt độ, thời gian
θ	Theta (θ)	Các tham số	ϕ	Phi (φ)	Hàm ánh xạ đặc trưng
λ	Lambda (λ)	Hiệu chuẩn	ψ	Psi (ψ)	Hàm cơ sở
μ	Mu (μ)	Trung bình	ω	Omega (ω)	Tần số, trọng số

Các chữ cái Hy Lạp viết hoa cũng xuất hiện: Σ (sigma) cho tổng, Π (pi) cho tích, Φ (phi) cho phân phối lũy kế, Ω (omega) cho không gian mẫu.

B.7 Các phép so sánh và ký hiệu logic

Ký hiệu	Ý nghĩa	Ví dụ
$=$	Bằng	$2 + 2 = 4$
$\neq$	Không bằng	$3 \neq 4$
$<, >$	Nhỏ hơn, lớn hơn	$3 < 5, 7 > 2$
$\leq, \geq$	Nhỏ hơn/lớn hơn hoặc bằng	$x \leq 10$
$\approx$	Xấp xỉ bằng	$\pi \approx 3.14$
$\propto$	Tỉ lệ thuận với	$y \propto x$ means $y = kx$ for some k
$\equiv$	Đồng nhất / được định nghĩa là	$f(x) \equiv x^2$
$:=$	Được gán (không phải định nghĩa)	$\mu := \frac{1}{n} \sum x_i$

B.8 Lượng từ

Lượng từ diễn đạt các mệnh đề về “tất cả” hoặc “một số” phần tử:

Ký hiệu	Tên	Ý nghĩa
$\forall$	Phổ quát (“với mọi”)	$\forall x \in \mathbb{R} : x^2 \geq 0$ (all squares are non-negative)
$\exists$	Tồn tại (“có tồn tại”)	$\exists x \in \mathbb{R} : x^2 = 2$ (some number squares to 2)
$\nexists$	“Không tồn tại”	$\nexists x \in \mathbb{R} : x^2 = -1$ (no real squares to -1)

B.9 Ký hiệu riêng của học máy

Ký hiệu sau đây được dùng thống nhất xuyên suốt cuốn sách này:

B.9.1 Dữ liệu và đặc trưng

Ký hiệu	Ý nghĩa
$\mathbf{x}$	Feature vector (input), typically $\mathbf{x} = (x_1, x_2, \dots, x_d)$
y	Label / target (output)
$\hat{y}$	Predicted value (the “hat” indicates prediction/estimate)
n or m	Number of training examples
d	Number of features (dimensionality)
S	Training set, $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$
$\mathcal{X}$	Input space (domain)
$\mathcal{Y}$	Output space (label set)

B.9.2 Mô hình và học tập

Ký hiệu	Ý nghĩa
h	Giả thuyết (một hàm từ đầu vào tới dự đoán)
$\mathcal{H}$	Lớp giả thuyết (tập hợp các giả thuyết ứng viên)
θ or w	Tham số / trọng số của mô hình
h_θ	Giả thuyết có các tham số θ
$\mathcal{L}$ or J	Hàm mất mát / hàm chi phí
α or η	Tốc độ học
λ	Tham số hiệu chuẩn

B.9.3 Ký hiệu xác suất

Ký hiệu	Ý nghĩa
$P(A)$	Probability of event A
$P(A B)$	Conditional probability of A given B
$\mathbb{E}[X]$	Expected value of random variable X
$\text{Var}(X)$	Variance of X
$\mathcal{N}(\mu, \sigma^2)$	Normal distribution with mean μ , variance σ^2
$X \sim \mathcal{D}$	X is distributed according to $\mathcal{D}$

B.9.4 Ký hiệu giải tích

Ký hiệu	Ý nghĩa
$\frac{df}{dx}$ or $f'(x)$	Đạo hàm của f theo x
$\frac{\partial f}{\partial x}$	Đạo hàm riêng (khi f có nhiều biến)
∇f	Gradient của f (vectơ của tất cả các đạo hàm riêng)
$\nabla_\theta \mathcal{L}$	Gradient của hàm mất mát theo các tham số

B.10 Cách đọc biểu thức toán học

Khi gặp một biểu thức phức tạp, hãy tách nó ra từng mảnh một. Xét:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n (y_i - h_\theta(x_i))^2 + \lambda \|\theta\|^2$$

Đọc từng bước:

1. $\hat{\theta}$, các tham số ước lượng (tối ưu)
2. $\arg \min_{\theta}$, tìm θ sao cho tối thiểu hóa biểu thức phía sau

3. $\frac{1}{n} \sum_{i=1}^n$, lấy trung bình trên toàn bộ n các ví dụ huấn luyện

4. $(y_i - h_{\theta}(x_i))^2$, bình phương hiệu giữa nhãn thật và giá trị dự đoán

$\lambda \|\theta\|^2$, số hạng hiệu chuẩn (phạt những tham số lớn)

Nói bằng lời: “Tìm các tham số θ sao cho sai số bình phương trung bình cộng với một khoảng phạt hiệu chuẩn là nhỏ nhất.”

B.11 Tóm tắt: tra cứu nhanh

Bảng tra ký hiệu thiết yếu

Tập hợp: $\in$ (thuộc), $\subseteq$ (tập con), $\cup$ (hợp/hoặc), $\cap$ (giao/và), $|S|$ (cỡ)

Tổng/Tích: $\sum_{i=1}^n a_i$ (cộng tất cả), $\prod_{i=1}^n a_i$ (nhân tất cả) **Hàm số:** $f(x)$ (giá trị tại x), $f: A \rightarrow B$ (ánh xạ A đến B), $1[\cdot]$ (chỉ thị)

So sánh: $\neq$ (không bằng), $\leq$ (không vượt quá), $\geq$ (không nhỏ hơn), $\approx$ (xấp xỉ)

Lượng từ: $\forall$ (với mọi), $\exists$ (tồn tại)

Dữ liệu ML: x (đặc trưng), y (nhãn), $\hat{y}$ (dự đoán), S (tập huấn luyện)

Mô hình ML: h (giả thuyết), $\mathcal{H}$ (lớp giả thuyết), θ (tham số), $\mathcal{L}$ (hàm mất mát)

Xác suất: $P(A)$ (xác suất), $P(A|B)$ (điều kiện), $\mathbb{E}[X]$ (kỳ vọng)

Giải tích: $\frac{df}{dx}$ (đạo hàm), ∇f (gradient)

Phụ lục C

Xác suất

Lý thuyết xác suất chẳng qua gì là
trực giác quy thành phép tính.

Pierre-Simon Laplace

C.1 Mở đầu: Tại sao cần Xác suất?

Học máy về bản chất là việc đối phó với sự bất định. Ta không biết mối quan hệ thật sự giữa đầu vào và đầu ra. Ta chỉ có thể ước lượng nó từ dữ liệu hạn chế. Ta không biết điểm dữ liệu tiếp theo sẽ trông như thế nào. Ta chỉ có thể dự đoán dựa trên những khuôn mẫu đã quan sát được. Ta không thể chắc chắn mô hình của mình sẽ khái quát hóa tốt cho những tình huống mới. Ta chỉ có thể nói rằng nó *có lẽ* sẽ hoạt động tốt.

Xác suất cho ta ngôn ngữ và công cụ để suy luận một cách chính xác về sự bất định này. Thay vì nói “mô hình có thể sai”, ta có thể nói “mô hình có tỷ lệ sai là 3%”. Thay vì nói “email này có lẽ là spam”, ta có thể nói “có 95% xác suất email này là spam”. Sự chính xác này biến những trực giác mơ hồ thành các phát biểu định lượng mà ta có thể sử dụng, kiểm tra và cải thiện.

Phụ lục này giới thiệu những khái niệm xác suất mà bạn cần cho học máy. Ta xây dựng sự hiểu biết thông qua các ví dụ cụ thể và những lời giải thích trực giác, phát triển các ý tưởng từ nguyên lý cơ bản thay vì giả định bạn đã có kiến thức trước.

C.2 Xác suất là gì?

C.2.1 Ý tưởng cơ bản

Xác suất đo mức độ khả năng một điều gì đó xảy ra. Ta biểu diễn nó bằng một số trong khoảng từ 0 đến 1, trong đó 0 nghĩa là bất khả và 1 nghĩa là chắc chắn. Xác suất 0.5 nghĩa là sự kiện có khả năng xảy ra hay không xảy ra là bằng nhau. Giống như tung một đồng xu công bằng.

Ta cũng có thể hình dung xác suất dưới dạng một phân số hoặc một tỷ lệ phần trăm. Xác suất 0.25 tương đương với $\frac{1}{4}$ hay 25%. Khi ta nói “có 70% khả năng trời mưa”, ta muốn nói xác suất trời mưa là 0.7.

Dưới đây là một vài ví dụ để xây dựng trực giác:

- Tung một đồng xu công bằng và được mặt sấp: $P = 0.5$ (50%)
- Gieo một con xúc xắc sáu mặt công bằng và ra mặt 4: $P = \frac{1}{6} \approx 0.167$ (khoảng 17%)
- Rút được lá Át chuồn từ một bộ bài đã xáo trộn: $P = \frac{1}{52} \approx 0.019$ (khoảng 2%)
- Mặt trời mọc vào ngày mai: $P \approx 1$ (gần như chắc chắn)
- Gieo một con xúc xắc sáu mặt tiêu chuẩn và ra mặt 7: $P = 0$ (bất khả)

C.2.2 Không gian mẫu và Sự kiện

Để thao tác với xác suất một cách chính xác, ta cần một vài thuật ngữ.

Không gian mẫu, ký hiệu là Ω (chữ cái Hy Lạp omega), là tập hợp tất cả các kết quả có thể có của một quá trình ngẫu nhiên. Với một lần tung đồng xu, $\Omega = \{\text{Sấp}, \text{Ngược}\}$, đây là hai điều duy nhất có thể xảy ra. Với việc gieo xúc xắc, $\Omega = \{1, 2, 3, 4, 5, 6\}$.

Một **sự kiện** là thứ ta có thể muốn hỏi về; một cách hình thức, nó là một tập con của không gian mẫu. Sự kiện “gieo ra số chẵn” tương ứng với tập con $\{2, 4, 6\}$. Sự kiện “được mặt sấp” tương ứng với $\{\text{Sấp}\}$.

Xác suất của một sự kiện A , viết là $P(A)$, cho ta biết khả năng một trong các kết quả trong A xảy ra là bao nhiêu.

Ví dụ: Gieo xúc xắc

Giả sử ta gieo một con xúc xắc sáu mặt công bằng. Không gian mẫu là $\Omega = \{1, 2, 3, 4, 5, 6\}$, với mỗi kết quả có xác suất bằng nhau. Gọi A là sự kiện "gieo ra số chẵn", khi đó $A = \{2, 4, 6\}$. Vì 3 trong số 6 kết quả có xác suất bằng nhau là số chẵn:

$$P(A) = \frac{\text{number of outcomes in } A}{\text{total number of outcomes}} = \frac{3}{6} = 0.5$$

Có 50% khả năng gieo ra số chẵn.

Công thức này ($P(A) = \frac{|A|}{|\Omega|}$) dùng được mọi khi tất cả các kết quả có xác suất bằng nhau. Ký hiệu $|A|$ chỉ số phần tử trong tập A .

C.3 Các quy tắc xác suất cơ bản

C.3.1 Những tiên đề nền tảng

Toàn bộ lý thuyết xác suất được xây dựng trên ba quy tắc đơn giản:

1. **Không âm:** Xác suất không thể âm. Với mọi sự kiện A , $P(A) \geq 0$.
2. **Chuẩn hóa:** Phải có một điều gì đó xảy ra. $P(\Omega) = 1$.
3. **Tính cộng:** Với các sự kiện không thể cùng xảy ra (gọi là các sự kiện **loại trừ lẫn nhau**), ta có thể cộng các xác suất của chúng: $P(A \text{ hoặc } B) = P(A) + P(B)$.

Từ ba quy tắc này, ta có thể suy ra mọi thứ còn lại trong lý thuyết xác suất.

C.3.2 Quy tắc bù

Bù của một sự kiện A , viết là A^c hoặc $\bar{A}$, là sự kiện " A không xảy ra". Vì hoặc A xảy ra hoặc nó không xảy ra, và hai khả năng này bao trùm mọi thứ:

$$P(A^c) = 1 - P(A)$$

Quy tắc đơn giản này mạnh mẽ một cách bất ngờ. Thường thì tính xác suất rằng một điều gì đó *không* xảy ra dễ hơn nhiều, rồi lấy 1 trừ đi.

Ví dụ: Ít nhất một sáu

Xác suất gieo được ít nhất một sáu khi ta gieo ba con xúc xắc là bao nhiêu?

Phương tiếp cận trực tiếp sẽ đòi hỏi phải đếm: một sáu, hoặc hai sáu, hoặc ba sáu, với những vị trí khác nhau... Việc này nhanh chóng trở nên phức tạp. Hướng tiếp cận dùng phần bù lại đơn giản hơn nhiều. Phần bù của "ít nhất một sáu" là "không có sáu nào cả".

Với mỗi con xúc xắc, xác suất *không* gieo ra sáu là $\frac{5}{6}$. Vì các con xúc xắc độc lập với nhau (mỗi lần gieo không ảnh hưởng đến các lần khác), xác suất cả ba đều không có sáu là:

$$P(\text{no sixes}) = \frac{5}{6} \times \frac{5}{6} \times \frac{5}{6} = \left(\frac{5}{6}\right)^3 = \frac{125}{216} \approx 0.58$$

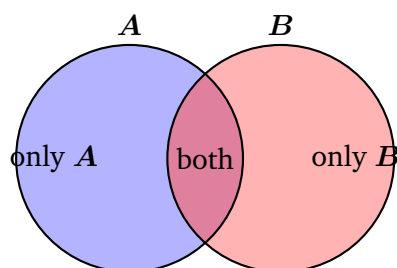
Vậy:

$$P(\text{at least one 6}) = 1 - P(\text{no sixes}) = 1 - \frac{125}{216} = \frac{91}{216} \approx 0.42$$

Có khoảng 42% khả năng gieo được ít nhất một sáu trong ba lần gieo.

C.3.3 Quy tắc cộng

Nếu ta muốn biết xác suất rằng sự kiện A xảy ra *hoặc* sự kiện B xảy ra (hoặc cả hai) thì sao? Nếu các sự kiện có thể chồng lấn (nếu có một số kết quả thuộc về cả A và B) thì ta cần cẩn thận đừng đếm những kết quả đó hai lần.



Hình C.1: Một hình Venn thể hiện hai sự kiện chồng lấn. Vùng màu tím thuộc về cả A và B .

Nếu ta cứ cộng $P(A) + P(B)$, ta sẽ đếm vùng chồng lấn màu tím hai lần, một lần là một phần của A và một lần là một phần của B . Để khắc phục điều này, ta trừ đi phần chồng lấn:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

Ở đây $A \cup B$ nghĩa là “ A hoặc B (hoặc cả hai)” và $A \cap B$ nghĩa là “ A và B (cả hai)”.

Example: Kings or Hearts

Drawing one card from a standard 52-card deck, what is the probability of getting a King or a Heart?

Let A = “drawing a King” and B = “drawing a Heart.”

There are 4 Kings in the deck, so $P(A) = \frac{4}{52}$.

There are 13 Hearts in the deck, so $P(B) = \frac{13}{52}$.

But wait; one card is *both* a King and a Heart: the King of Hearts. This is the overlap: $P(A \cap B) = \frac{1}{52}$.

Using the addition rule:

$$P(\text{King or Heart}) = \frac{4}{52} + \frac{13}{52} - \frac{1}{52} = \frac{16}{52} = \frac{4}{13} \approx 0.31$$

C.4 Xác suất có điều kiện

C.4.1 Ý tưởng

Thường ta muốn biết xác suất của một sự kiện *với điều kiện là ta đã biết một điều khác đã xảy ra*. Điều này được gọi là **xác suất có điều kiện**.

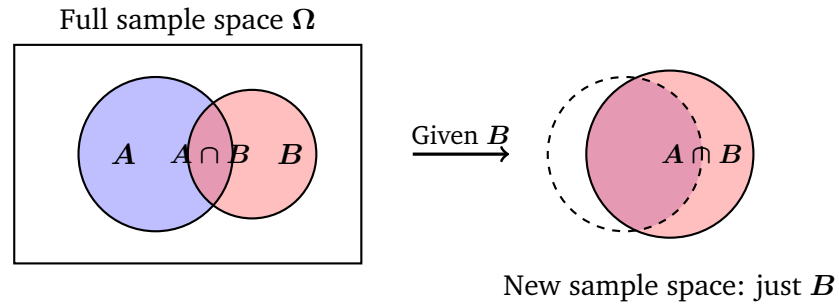
Ví dụ: Xác suất một học sinh đỗ một kỳ thi là bao nhiêu, *biết rằng* họ đã tham dự tất cả các buổi giảng? Xác suất ngày mai trời mưa là bao nhiêu, *biết rằng* hôm nay trời nhiều mây? Xác suất một email là spam là bao nhiêu, *biết rằng* nó chứa từ “FREE”?

Ta viết $P(A|B)$ cho “xác suất của A với điều kiện là B ”, đọc là “ P của A với điều kiện là B ”.

C.4.2 Xác suất có điều kiện hoạt động như thế nào

Khi ta biết B đã xảy ra, ta giới hạn sự chú ý vào đúng những kết quả mà ở đó B đúng. Trong số các kết quả đó, ta hỏi: có bao nhiêu phần trong số đó cũng thỏa mãn A ?

Hãy hình dung điều này một cách trực quan bằng hình Venn. Ban đầu, bất kỳ kết quả nào trong toàn bộ không gian mẫu Ω đều có thể xảy ra. Nhưng khi ta biết B đã xảy ra, ta phóng to vào đúng cái vòng tròn B , và nó trở thành vũ trụ khả năng mới, nhỏ hơn của ta. Trong vũ trụ bị giới hạn này, ta hỏi có bao nhiêu phần nằm trong A .



Hình C.2: Xác suất có điều kiện phóng to vào sự kiện B . Ta hỏi: trong tất cả các kết quả thuộc B , có bao nhiêu phần cũng nằm trong A ?

Công thức này diễn đạt ý tưởng đó:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Tử số $P(A \cap B)$ là xác suất của phần chồng lấn (các kết quả nằm trong cả A và B). Mẫu số $P(B)$ chuẩn hóa theo tổng xác suất của B , bởi vì B bây giờ là toàn bộ vũ trụ của ta.

C.4.3 Một ví dụ cụ thể với các con số

Hãy cùng đi qua một ví dụ với các con số cụ thể để thấy xác suất có điều kiện gắn với hình Venn như thế nào.

Ví dụ: Học sinh và Môn học

Ở một trường nhỏ có 100 học sinh. 40 học sinh học toán, 30 học sinh học vật lý, và 20 học sinh học cả toán lẫn vật lý.

Nếu ta chọn ngẫu nhiên một học sinh học vật lý, xác suất họ đó cũng học toán là bao nhiêu? Gọi M = là "học toán" và P = là "học vật lý". Từ các thông tin đã cho:

- $P(M) = 40/100 = 0.4$
- $P(P) = 30/100 = 0.3$
- $P(M \cap P) = 20/100 = 0.2$ (cả toán và vật lý)

Ta muốn $P(M|P)$, tức là xác suất học toán *biết* học vật lý:

$$P(M|P) = \frac{P(M \cap P)}{P(P)} = \frac{0.2}{0.3} = \frac{2}{3} \approx 0.67$$

Hai phần ba số học sinh học vật lý cũng học toán.

Hãy chú ý điều này khác với $P(M) = 0.4$ như thế nào. Biết một người học vật lý *làm tăng* ước lượng của ta rằng người đó học toán (từ 40% lên 67%), bởi vì giữa hai môn này có mối liên hệ thuận.

C.4.4 Xét nghiệm y tế: Một ví dụ nổi tiếng

Một trong những ứng dụng quan trọng nhất của xác suất có điều kiện là việc hiểu các xét nghiệm y tế. Ví dụ này cho thấy một kết quả trái ngược với trực giác làm ngạc nhiên hầu hết mọi người.

Ví dụ: Xét nghiệm y tế

Một căn bệnh ảnh hưởng tới 1% dân số. Xét nghiệm cho căn bệnh này khá chính xác:

- Nếu bạn mắc bệnh, xét nghiệm cho kết quả dương tính đúng 90% số lần (độ nhạy)
- Nếu bạn không mắc bệnh, xét nghiệm cho kết quả âm tính đúng 95% số lần (độ đặc hiệu), nghĩa là nó cho kết quả dương tính sai 5% số lần (tỷ lệ dương tính giả)

Bạn làm xét nghiệm và kết quả là dương tính. Xác suất bạn thực sự mắc bệnh là bao nhiêu?

Đa số mọi người đoán một con số cao, như 90%. Đáp án thực tế lại thấp đến bất ngờ.

Thiết lập bài toán:

Gọi D = là "mắc bệnh" và $+$ = là "xét nghiệm dương tính". Ta biết:

- $P(D) = 0.01$ (1% mắc bệnh)
- $P(D^c) = 0.99$ (99% không mắc bệnh)
- $P(+|D) = 0.90$ (xét nghiệm dương tính khi đã mắc bệnh)
- $P(+|D^c) = 0.05$ (xét nghiệm dương tính khi không mắc bệnh, tức dương tính giả)

Ta muốn $P(D|+)$, tức là xác suất mắc bệnh khi xét nghiệm dương tính.

Dùng một quần thể cụ thể:

Hãy tưởng tượng có 10.000 người tham gia xét nghiệm.

- 100 người mắc bệnh (1% trong 10.000)
- 9.900 người không mắc bệnh

Trong số 100 người mắc bệnh:

- 90 người xét nghiệm dương tính (90% trong 100)
- 10 người xét nghiệm âm tính

Trong số 9.900 người không mắc bệnh:

- 495 người xét nghiệm dương tính (5% trong 9.900), đây chính là dương tính giả!
- 9.405 người xét nghiệm âm tính

Tổng số xét nghiệm dương tính: $90 + 495 = 585$

Trong 585 xét nghiệm dương tính này, chỉ có 90 người thực sự mắc bệnh:

$$P(D|+) = \frac{90}{585} = \frac{90}{585} \approx 0.154$$

Chỉ khoảng 15% thôi! Dù đã xét nghiệm dương tính bằng một xét nghiệm có vẻ chính xác, khả năng cao là bạn *không* mắc bệnh.

Kết quả trái ngược với trực giác này xảy ra vì căn bệnh hiếm gặp. Phần lớn các xét nghiệm dương tính đến từ nhóm người khỏe mạnh đông đảo (9.900) chứ không phải từ nhóm nhỏ người mắc bệnh (100). Dù chỉ có 5% người khỏe mạnh cho kết quả dương tính, thì 5% của 9.900 vẫn là rất nhiều kết quả dương tính giả.

Ví dụ này cho thấy vì sao ta không thể đơn giản đồng nhất “xét nghiệm chính xác 90%” với “xét nghiệm dương tính nghĩa là 90% khả năng mắc bệnh”. **Tỷ lệ nền**, tức căn bệnh phổ biến đến đâu trong dân số, lại vô cùng quan trọng.

C.5 Định lý Bayes

Nhiều sai lầm đáng kể nhất trong học máy ứng dụng bắt nguồn từ những hiểu nhầm về xác suất. Nhầm lẫn $P(\text{spam}|\text{word})$ với $P(\text{word}|\text{spam})$ dẫn tới những bộ phân loại đưa ra các dự đoán tự tin nhưng sai. Bỏ qua tỷ lệ nền dẫn tới các xét nghiệm sàng lọc y tế nghe có vẻ rất ấn tượng nhưng lại cho ra chủ yếu là kết quả dương tính giả (như ta vừa thấy). Giả định độc lập khi các đặc trưng tương quan với nhau dẫn tới những ước lượng xác suất bị lệch chuẩn nghiêm trọng. Các công cụ trong mục này, đặc biệt là định lý Bayes, không phải là toán học trừu tượng. Chúng chính là vũ khí chống lại những sai lầm này.

Ví dụ xét nghiệm y tế gợi ý một khuôn mẫu chung: thường ta biết $P(B|A)$ nhưng lại muốn $P(A|B)$. Định lý Bayes cung cấp công cụ để đảo các xác suất có điều kiện.

C.5.1 Công thức

Định lý Bayes phát biểu:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Điều này cho phép ta tính $P(A|B)$ nếu ta biết $P(B|A)$, $P(A)$ và $P(B)$.

C.5.2 Hiểu các thành phần

Mỗi thành phần trong định lý Bayes đều có một tên gọi:

- $P(A)$ là **tiên nghiệm**. Niềm tin của ta về A trước khi thấy bất kỳ bằng chứng nào
- $P(B|A)$ là **độ khả năng**, bằng chứng B có khả năng xảy ra cao đến đâu nếu A đúng
- $P(A|B)$ là **hậu nghiệm**, niềm tin đã được cập nhật của ta về A sau khi thấy bằng chứng B
- $P(B)$ là **bằng chứng** (hay độ khả năng biên), tổng xác suất của việc quan sát thấy B

Trực giác ở đây là: ta khởi đầu với một niềm tin tiên nghiệm, rồi cập nhật nó dựa trên việc bằng chứng sẽ có khả năng cao đến đâu trong những kịch bản khác nhau.

C.5.3 Ví dụ: Phát hiện spam

Giả sử ta đang xây dựng một bộ lọc spam. Ta muốn biết: biết rằng một email chứa từ "FREE", xác suất nó là spam là bao nhiêu?

Gọi S = là "email là spam" và F = là "email chứa FREE".

Từ việc phân tích các email trước đây, ta biết:

- $P(S) = 0.3$ (30% số email là spam), là tiên nghiệm
- $P(F|S) = 0.8$ (80% số spam chứa "FREE"), là độ khả năng
- $P(F|S^c) = 0.1$ (10% số email hợp lệ chứa "FREE")

Trước hết, ta cần $P(F)$, tức là xác suất chung rằng một email chứa "FREE":

$$P(F) = P(F|S) \cdot P(S) + P(F|S^c) \cdot P(S^c) = 0.8 \times 0.3 + 0.1 \times 0.7 = 0.24 + 0.07 = 0.31$$

Bây giờ hãy áp dụng định lý Bayes:

$$P(S|F) = \frac{P(F|S) \cdot P(S)}{P(F)} = \frac{0.8 \times 0.3}{0.31} = \frac{0.24}{0.31} \approx 0.77$$

Một email chứa "FREE" có khoảng 77% khả năng là spam. Cao hơn nhiều so với mức nền 30%.

C.5.4 Định lý Bayes trong học máy

Trong bài toán phân loại, ta muốn xác định một điểm dữ liệu thuộc về lớp nào, dựa trên các đặc trưng của nó. Định lý Bayes cho ta:

$$P(\text{class}|\text{features}) = \frac{P(\text{features}|\text{class}) \cdot P(\text{class})}{P(\text{features})}$$

Đó chính xác là những gì **bộ phân loại Naïve Bayes** tính toán. Chúng ước lượng $P(\text{đặc trưng}|\text{lớp})$ từ dữ liệu huấn luyện và dùng định lý Bayes để phân loại các ví dụ mới.

C.6 Tính độc lập

C.6.1 Tính độc lập nghĩa là gì

Hai sự kiện là **độc lập** nếu việc biết một sự kiện đã xảy ra không cho ta biết gì về việc sự kiện kia có xảy ra hay không. Về mặt toán học, A và B độc lập khi và chỉ khi:

$$P(A|B) = P(A)$$

Việc biết B đã xảy ra không làm thay đổi ước lượng xác suất của ta cho A .

Một định nghĩa tương đương (thường dễ kiểm tra hơn) là:

$$P(A \cap B) = P(A) \cdot P(B)$$

Với các sự kiện độc lập, xác suất cả hai cùng xảy ra bằng tích số của các xác suất riêng lẻ.

C.6.2 Ví dụ

Sự kiện độc lập: Tung một đồng xu hai lần. Lần tung đầu ra sấp hay ngược không ảnh hưởng đến lần tung thứ hai.

$$P(2\text{nd heads} | 1\text{st heads}) = P(2\text{nd heads}) = 0.5$$

Đồng xu không có trí nhớ.

Không độc lập: Rút hai lá bài từ một bộ bài *không* thay thế. Nếu lá bài đầu tiên là một lá át, thì giờ đã có ít lá át còn lại hơn, nên:

$$P(2\text{nd ace} | 1\text{st ace}) = \frac{3}{51} \neq P(2\text{nd ace}) = \frac{4}{52}$$

Lần rút đầu ảnh hưởng đến lần rút sau.

Không độc lập: Điểm toán và điểm vật lý của một học sinh. Học sinh học giỏi toán thường cũng học giỏi vật lý. Biết ai đó đạt điểm A ở môn toán làm tăng ước lượng của ta về điểm vật lý của người đó.

C.6.3 Vì sao tính độc lập quan trọng

Tính độc lập đơn giản hóa việc tính xác suất một cách đáng kể. Với các sự kiện độc lập $A_1, A_2, \dots, A_n$:

$$P(A_1 \cap A_2 \cap \dots \cap A_n) = P(A_1) \cdot P(A_2) \cdot \dots \cdot P(A_n)$$

Ta chỉ cần nhân các xác suất riêng lẻ lại với nhau.

Đó là lý do ta có thể tính “không có sáu nào trong ba lần gieo xúc xắc” bằng $(\frac{5}{6})^3$. Các lần gieo độc lập với nhau, nên ta nhân lên.

C.6.4 Tính độc lập trong học máy

Bộ phân loại **Naïve Bayes** giả định rằng các đặc trưng độc lập với nhau khi đã cho lớp. Điều này có nghĩa là:

$$P(\text{đặc trưng}_1 \text{ và đặc trưng}_2 | \text{lớp}) = P(\text{đặc trưng}_1 | \text{lớp}) \cdot P(\text{đặc trưng}_2 | \text{lớp})$$

Trong thực tế, giả định này gần như luôn luôn sai! Các đặc trưng như “thu nhập” và “trình độ học vấn” có tương quan với nhau. Chúng không độc lập. Chiều cao và cân nặng của một người có tương quan. Các từ trong một văn bản có tương quan với nhau (“máy” thường xuất hiện gần “học”).

Dù vậy, Naïve Bayes thường hoạt động tốt một cách đáng ngạc nhiên. Giả định độc lập là sai, nhưng bộ phân loại thu được vẫn thường đưa ra những dự đoán tốt. Đây là một trong những sự thật thực nghiệm đáng ngạc nhiên về học máy: những phương pháp dựa trên giả định sai vẫn có thể hữu ích trong thực tế. Bộ phân loại có thể không đưa ra những xác suất được hiệu chuẩn hoàn hảo, nhưng nó thường xếp hạng đúng các ví dụ (ví dụ như spam so với không phải spam).

C.7 Biến ngẫu nhiên

C.7.1 Biến ngẫu nhiên là gì?

Một **biến ngẫu nhiên** là một đại lượng mà giá trị của nó được quyết định bởi một quá trình ngẫu nhiên. Ta dùng các chữ cái in hoa như X , Y , Z cho các biến ngẫu nhiên, và chữ cái thường như x , y , z cho những giá trị cụ thể mà chúng có thể nhận.

Hãy hình dung một biến ngẫu nhiên như một phép đo mà ta có thể thực hiện trên một kết quả ngẫu nhiên:

- X = con số hiện ra khi bạn gieo xúc xắc (các giá trị: 1, 2, 3, 4, 5, 6)
- Y = số lần ra mặt sấp khi bạn tung 10 đồng xu (các giá trị: 0, 1, 2, ..., 10)
- Z = chiều cao của một người được chọn ngẫu nhiên (các giá trị: mọi số dương)

C.7.2 Rời rạc so với Liên tục

Biến ngẫu nhiên rời rạc nhận giá trị từ một tập đếm được, thường là các số nguyên. Số khách hàng đến cửa hàng, số email bạn nhận được, kết quả của một lần gieo xúc xắc.

Biến ngẫu nhiên liên tục nhận giá trị từ một khoảng liên tục, tức mọi số thực trong một đoạn nào đó. Chiều cao chính xác của một người, nhiệt độ vào ngày mai, thời gian cho tới khi một bóng đèn cháy hỏng.

Sự phân biệt này quan trọng vì ta xử lý xác suất theo cách khác nhau. Với các biến rời rạc, ta có thể hỏi “ $P(X = 3)$ bằng bao nhiêu?”. Với các biến liên tục, xác suất của bất kỳ giá trị chính xác nào cũng bằng không (vì có vô hạn khả năng), nên ta hỏi về các khoảng thay vào đó: “ $P(2 < X < 4)$ bằng bao nhiêu?”.

C.7.3 Các phân phối xác suất

Một **phân phối xác suất** mô tả mỗi giá trị (hoặc mỗi khoảng giá trị) có khả năng xảy ra đến mức nào.

Với một biến ngẫu nhiên rời rạc, ta liệt kê $P(X = x)$ cho mỗi giá trị khả thể x . Với một con xúc xắc công bằng:

$$P(X = 1) = P(X = 2) = P(X = 3) = P(X = 4) = P(X = 5) = P(X = 6) = \frac{1}{6}$$

Với một biến ngẫu nhiên liên tục, ta xác định một **hàm mật độ xác suất** (PDF), thường viết là $p(x)$ hoặc $f(x)$. Xác suất rơi vào một khoảng chính là diện tích dưới đường cong trên khoảng đó.

C.8 Giá trị kỳ vọng

C.8.1 Ý tưởng

Giá trị kỳ vọng của một biến ngẫu nhiên, còn gọi là **trung bình**, là giá trị trung bình về lâu dài mà ta sẽ quan sát được nếu lặp lại quá trình ngẫu nhiên đó nhiều lần.

Nếu bạn gieo xúc xắc một lần, bạn sẽ nhận được một con số cụ thể. Nhưng nếu bạn gieo nó một triệu lần rồi lấy trung bình tất cả các kết quả, bạn sẽ nhận được gì?

C.8.2 Tính giá trị kỳ vọng

Với một biến ngẫu nhiên rời rạc, ta tính giá trị kỳ vọng bằng cách nhân mỗi giá trị khả thể với xác suất của nó rồi cộng lại:

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

Ký hiệu $\mathbb{E}[X]$ nghĩa là “giá trị kỳ vọng của X ”.

Ví dụ: Giá trị kỳ vọng của một lần gieo xúc xắc

Với một con xúc xắc công bằng, mỗi số từ 1 đến 6 đều có xác suất $\frac{1}{6}$:

$$\begin{aligned}\mathbb{E}[X] &= 1 \cdot \frac{1}{6} + 2 \cdot \frac{1}{6} + 3 \cdot \frac{1}{6} + 4 \cdot \frac{1}{6} + 5 \cdot \frac{1}{6} + 6 \cdot \frac{1}{6} \\ &= \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = \frac{21}{6} = 3.5\end{aligned}$$

Giá trị kỳ vọng là 3,5, dù bạn không bao giờ có thể thật sự gieo ra 3,5! Nếu bạn gieo xúc xắc nhiều lần, thì trung bình của bạn sẽ tiệm cận về 3,5.

C.8.3 Các tính chất của giá trị kỳ vọng

Giá trị kỳ vọng có một tính chất rất hữu ích gọi là **tính tuyến tính**:

$$\mathbb{E}[aX + b] = a \cdot \mathbb{E}[X] + b$$

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$$

Tính chất thứ hai rất đáng chú ý: giá trị kỳ vọng của một tổng bằng tổng các giá trị kỳ vọng, *kể cả khi X và Y không độc lập*. Điều này khiến giá trị kỳ vọng dễ thao tác hơn nhiều so với xác suất.

C.9 Phương sai và Độ lệch chuẩn

C.9.1 Vì sao ta cần Phương sai

Giá trị kỳ vọng cho ta biết “trung tâm” của một phân phối, nhưng hai phân phối có thể có cùng một trung tâm mà trông lại rất khác nhau. Hãy xét:

- Phân phối A: Luôn bằng 5. Giá trị kỳ vọng = 5.
- Phân phối B: Bằng 0 hoặc 10 với xác suất bằng nhau. Giá trị kỳ vọng = 5.

Cả hai có cùng trung bình, nhưng Phân phối B phân tán rộng hơn nhiều.

Phương sai đo mức độ phân tán đó.

C.9.2 Định nghĩa

Phương sai đo giá trị thường nằm cách trung bình bao xa:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$$

Đó là khoảng cách bình phương trung bình kỳ vọng từ trung bình. Ta bình phương các khoảng cách để những độ lệch dương và độ lệch âm không triệt tiêu lẫn nhau.

Một công thức tương đương, thường dễ tính hơn:

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

C.9.3 Độ lệch chuẩn

Vì phương sai liên quan đến các giá trị bình phương, đơn vị của nó là đơn vị gốc ở bậc hai (ví dụ, nếu X có đơn vị mét thì phương sai có đơn vị mét vuông). Để trở về đơn vị gốc, ta lấy căn bậc hai:

$$\text{SD}(X) = \sigma = \sqrt{\text{Var}(X)}$$

Độ lệch chuẩn là thước đo độ phân tán được báo cáo phổ biến nhất vì nó dùng cùng đơn vị với dữ liệu.

Ví dụ: Phương sai và Độ lệch chuẩn của một lần gieo xúc xắc

Ta đã biết $\mathbb{E}[X] = 3.5$. Để tìm phương sai, trước hết ta cần $\mathbb{E}[X^2]$:

$$\begin{aligned}\mathbb{E}[X^2] &= 1^2 \cdot \frac{1}{6} + 2^2 \cdot \frac{1}{6} + 3^2 \cdot \frac{1}{6} + 4^2 \cdot \frac{1}{6} + 5^2 \cdot \frac{1}{6} + 6^2 \cdot \frac{1}{6} \\ &= \frac{1 + 4 + 9 + 16 + 25 + 36}{6} = \frac{91}{6} \approx 15.17\end{aligned}$$

Bây giờ:

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = 15.17 - (3.5)^2 = 15.17 - 12.25 = 2.92$$

$$\text{SD}(X) = \sqrt{2.92} \approx 1.71$$

Một lần gieo xúc xắc diễn hình cách trung bình 3,5 khoảng 1,71.

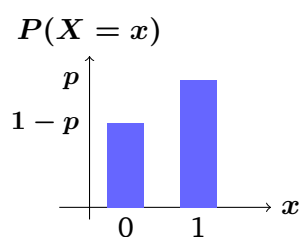
C.10 Các phân phối xác suất thông dụng

Có những phân phối xác suất xuất hiện thường xuyên đến mức chúng có tên gọi riêng và những tính chất đã được nghiên cứu kỹ.

C.10.1 Phân phối Bernoulli

Phân phối đơn giản nhất: một phép thử có/không duy nhất với xác suất p là “thành công”.

$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$



Bernoulli with $p = 0.6$

Hình C.3: Một phân phối Bernoulli biểu diễn một kết quả nhị phân đơn lẻ.

Trung bình: $\mathbb{E}[X] = p$

Phương sai: $\text{Var}(X) = p(1 - p)$

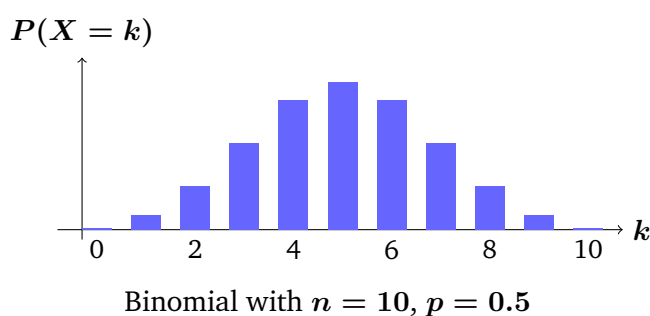
Ví dụ: Một lần tung đồng xu (mặt sấp = 1, mặt ngửa = 0), một khách hàng có mua hàng hay không, một email có phải là spam hay không.

C.10.2 Phân phối Binomial

Nếu ta lặp lại một phép thử Bernoulli n lần độc lập và đếm số lần thành công, ta thu được một phân phối **Binomial**.

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$$

trong đó $\binom{n}{k} = \frac{n!}{k!(n-k)!}$ là số cách để chọn xem k phép thử nào trong n phép thử là thành công.



Hình C.4: Phân phối Binomial: số lần thành công trong n phép thử độc lập. Ở đây, $n = 10$ và $p = 0.5$ (giống như tung 10 đồng xu công bằng).

Trung bình: $\mathbb{E}[X] = np$

Phương sai: $\text{Var}(X) = np(1 - p)$

Ví dụ: Số lần ra mặt sấp trong 10 lần tung đồng xu, số sản phẩm lỗi trong một lô 100 sản phẩm, số câu trả lời đúng khi đoán trong một bài trắc nghiệm 20 câu.

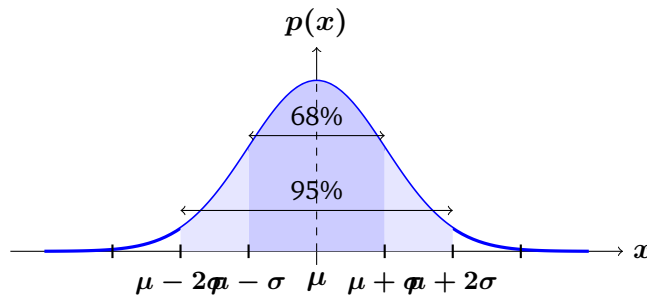
C.10.3 Phân phối Gaussian (Normal)

Phân phối **Gaussian** hay còn gọi là phân phối **Normal** chính là “đường cong hình chuông” nổi tiếng. Đây là phân phối quan trọng nhất trong thống kê nhờ Định lý Giới hạn Trung tâm: khi bạn lấy trung bình của nhiều đại lượng ngẫu nhiên độc lập, kết quả có xu hướng tiệm cận về phân phối Gaussian bất kể các phân phối ban đầu trông như thế nào.

Hàm mật độ xác suất là:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Ta viết $X \sim \mathcal{N}(\mu, \sigma^2)$ để chỉ “ X tuân theo phân phối Normal với trung bình μ và phương sai σ^2 ”.



Hình C.5: Phân phối Gaussian (Normal). Khoảng 68% giá trị nằm trong khoảng cách một độ lệch chuẩn quanh trung bình, và khoảng 95% nằm trong khoảng cách hai độ lệch chuẩn.

Những đặc điểm then chốt của phân phối Normal:

- Nó đối xứng quanh trung bình μ
- Tham số σ (độ lệch chuẩn) điều khiển độ rộng
- Khoảng 68% giá trị nằm trong $\mu \pm \sigma$ (một độ lệch chuẩn)
- Khoảng 95% giá trị nằm trong $\mu \pm 2\sigma$ (hai độ lệch chuẩn)
- Khoảng 99,7% giá trị nằm trong $\mu \pm 3\sigma$ (ba độ lệch chuẩn)

“Quy tắc 68-95-99,7” này rất đáng nhớ, nó cho ta trực giác nhanh về mức độ phân tán của dữ liệu.

Ví dụ: Chiều cao của con người, sai số đo lường, điểm thi (xấp xỉ), lợi nhuận cổ phiếu (xấp xỉ).

C.11 Tóm tắt

Tra cứu nhanh về xác suất

Các quy tắc cơ bản:

- $0 \leq P(A) \leq 1$
- $P(A^c) = 1 - P(A)$ (quy tắc bù)
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (quy tắc cộng)

Xác suất có điều kiện: $P(A|B) = \frac{P(A \cap B)}{P(B)}$

Định lý Bayes: $P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$

Tính độc lập: A và B độc lập khi và chỉ khi $P(A \cap B) = P(A) \cdot P(B)$

Giá trị kỳ vọng: $\mathbb{E}[X] = \sum_x x \cdot P(X = x)$ (rời rạc)

Phương sai: $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

Độ lệch chuẩn: $\sigma = \sqrt{\text{Var}(X)}$

Các phân phối chính:

- Bernoulli: Một phép thử có/không duy nhất với xác suất p
- Binomial: Số lần thành công trong n phép thử độc lập
- Gaussian: Đường cong hình chuông $\mathcal{N}(\mu, \sigma^2)$; 68% nằm trong $\pm\sigma$, 95% nằm trong $\pm 2\sigma$

Phụ lục D

Giải tích

Giải tích là công cụ hữu ích nhất mà chúng ta có để thấu hiểu chân lý của vật lý.

William Fogg Osgood

D.1 Mở đầu: Tại sao cần Giải tích cho Học máy?

Học máy là việc tìm ra mô hình tốt nhất, tức mô hình mắc ít lỗi nhất. Nhưng làm sao để tìm ra “mô hình tốt nhất”? Nếu một mô hình có các tham số (chẳng hạn là các trọng số trong một mạng nơ-ron), ta cần tìm các giá trị tham số làm tối thiểu hóa lỗi.

Hãy tưởng tượng bạn bị bịt mắt giữa một vùng đất đầy đồi gợn sóng, đang cố tìm ra thung lũng thấp nhất. Bạn không nhìn thấy gì, nhưng bạn có thể cảm nhận mặt đất dưới chân mình. Nếu mặt đất nghiêng xuống về bên trái, bạn bước sang trái. Nếu nó nghiêng xuống phía trước mặt, bạn bước tới trước. Luôn luôn bước theo hướng đi xuống, cuối cùng bạn sẽ tới một đáy thung lũng, nơi mặt đất bằng phẳng theo mọi hướng.

Đây chính xác là cách học máy hoạt động. “Địa hình” ở đây chính là lỗi (hàm mất mát) như một hàm của các tham số. Giải tích cho ta biết độ dốc tại mỗi điểm, và ta dùng độ dốc đó để bước về phía giá trị lỗi thấp hơn. Quá trình này được gọi là **rơi gradient**, và đó chính là cách ta huấn luyện gần như mọi mô hình học máy hiện đại.

Để hiểu rơi gradient, ta cần hiểu về độ dốc. Đó chính là điều mà giải tích cung cấp.

D.2 Độ dốc: Ý tưởng then chốt

D.2.1 Độ dốc là gì?

Bạn đã cảm nhận độ dốc một cách trực giác từ lâu rồi. Một ngọn đồi dốc đứng thì độ dốc lớn. Một con đường bằng phẳng thì độ dốc bằng không. Một con đường đi xuống thì độ dốc âm.

Về mặt định lượng, độ dốc đo lường mức y thay đổi bao nhiêu khi x thay đổi:

$$\text{slope} = \frac{\text{change in } y}{\text{change in } x} = \frac{\Delta y}{\Delta x}$$

Ví dụ: Tính độ dốc

Một con đường lên cao 3 mét trên một quãng đường ngang dài 100 mét.

$$\text{slope} = \frac{3 \text{ m}}{100 \text{ m}} = 0.03 = 3\%$$

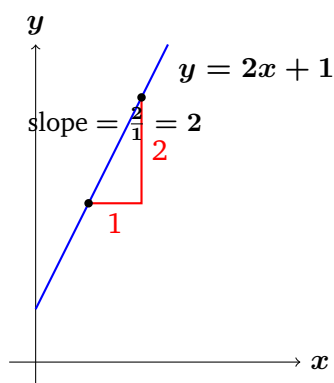
Một đường mòn lên núi lên cao 50 mét trên 100 mét theo phương ngang.

$$\text{slope} = \frac{50}{100} = 0.5 = 50\%$$

Đường mòn dốc hơn nhiều!

D.2.2 Độ dốc của một đường thẳng

Với một đường thẳng, độ dốc là như nhau ở mọi nơi. Đường thẳng $y = 2x + 1$ có độ dốc bằng 2: mỗi khi x tăng thêm 1, y tăng thêm 2.

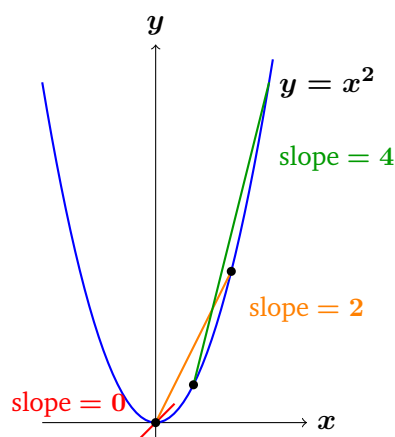


Hình D.1: Đường thẳng $y = 2x + 1$ có độ dốc không đổi bằng 2.

D.2.3 Vấn đề của các đường cong

Nhưng còn các đường cong thì sao? Hàm $y = x^2$ không thẳng, nó cong vút lên trên. Độ dốc của nó là bao nhiêu?

Câu trả lời: *độ dốc ở mỗi điểm là khác nhau*. Gần $x = 0$, đường cong gần như nằm ngang (độ dốc gần bằng 0). Tại $x = 2$, nó dốc hơn. Tại $x = 3$, lại dốc hơn nữa.



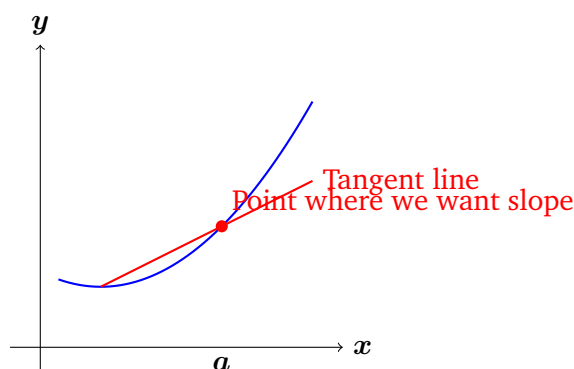
Hình D.2: Parabol $y = x^2$ có những độ dốc khác nhau ở những điểm khác nhau.

Giải tích cho ta một công cụ để tìm độ dốc tại bất kỳ điểm nào trên một đường cong. Công cụ này được gọi là **đạo hàm**.

D.3 Đạo hàm: Độ dốc tại một điểm

D.3.1 Ý tưởng đường tiếp tuyến

Để tìm độ dốc của một đường cong tại một điểm xác định, ta vẽ một đường thẳng chỉ mới chạm vào đường cong tại điểm đó, gọi là **tiếp tuyến**. Độ dốc của đường tiếp tuyến này chính là độ dốc của đường cong tại điểm đó.



Hình D.3: Đạo hàm tại $x = a$ chính là độ dốc của đường tiếp tuyến tại điểm đó.

D.3.2 Định nghĩa đạo hàm

Đạo hàm của một hàm f tại một điểm x , viết là $f'(x)$, chính là độ dốc của đường tiếp tuyến tại điểm đó.

Một cách hình thức, ta tìm được điều này bằng cách lấy hai điểm nằm rất gần nhau và tính độ dốc giữa hai điểm đó:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Công thức này nói rằng: lấy một bước rất nhỏ h từ x , tính xem f thay đổi bao nhiêu, chia cho h , rồi cho h tiến tới giá trị cực kỳ nhỏ.

D.3.3 Các ký hiệu thay thế

Bạn sẽ thấy đạo hàm được viết theo nhiều cách tương đương nhau:

$$f'(x) = \frac{df}{dx} = \frac{d}{dx} f(x) = \frac{dy}{dx} \quad (\text{if } y = f(x))$$

Ký hiệu $\frac{df}{dx}$ nhấn mạnh rằng đạo hàm là một tỷ số: f thay đổi bao nhiêu trên mỗi đơn vị thay đổi của x .

D.3.4 Tính đạo hàm từ định nghĩa

Hãy cùng tìm đạo hàm của $f(x) = x^2$ từng bước một bằng cách dùng định nghĩa.

Bước 1: Viết $f(x+h)$:

$$f(x+h) = (x+h)^2 = x^2 + 2xh + h^2$$

Bước 2: Tính $f(x + h) - f(x)$:

$$f(x + h) - f(x) = (x^2 + 2xh + h^2) - x^2 = 2xh + h^2$$

Bước 3: Chia cho h :

$$\frac{f(x + h) - f(x)}{h} = \frac{2xh + h^2}{h} = 2x + h$$

Bước 4: Cho $h \rightarrow 0$:

$$f'(x) = \lim_{h \rightarrow 0} (2x + h) = 2x$$

Kết quả: Đạo hàm của x^2 là $2x$.

Hãy kiểm tra xem kết quả này có hợp lý không bằng cách xét các điểm cụ thể:

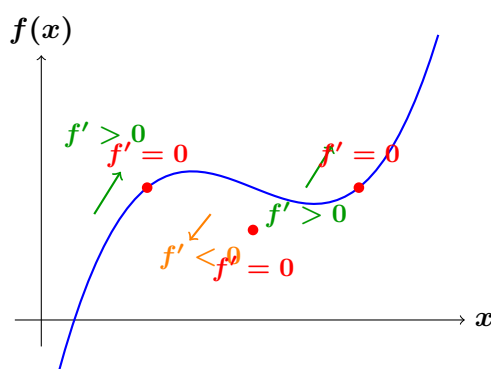
x	$f(x) = x^2$	$f'(x) = 2x$ (slope)
0	0	0 (bằng phẳng ở đáy)
1	1	2 (đang tăng)
2	4	4 (dốc hơn)
3	9	6 (dốc hơn nữa)
-1	1	-2 (đang giảm, bên trái)

Độ dốc âm tại $x = -1$ là hoàn toàn hợp lý: ở phía bên trái của parabol, đường cong đi xuống khi ta dịch sang phải.

D.3.5 Đạo hàm cho ta biết điều gì

Dấu và độ lớn của đạo hàm tiết lộ cách hàm biến thiên:

If $f'(x)$ is...	khi đó hàm sẽ...
Dương tính	Tăng (đi lên khi x tăng)
Âm tính	Giảm (đi xuống khi x tăng)
Bằng không	Bằng phẳng (không tăng cũng không giảm)
Dương và lớn	Tăng rất dốc
Âm và lớn	Giảm rất dốc



Hình D.4: Dấu của đạo hàm cho biết hàm đang tăng hay giảm. Đạo hàm bằng không tại các đỉnh và các đáy.

D.4 Các quy tắc đạo hàm: Những lối tắt

Tính đạo hàm từ định nghĩa giới hạn mỗi lần sẽ vô cùng tốn công. Các nhà toán học đã khám phá ra những quy tắc cho phép ta tìm đạo hàm thật nhanh chóng.

Lưu ý về mục này

Các quy tắc đạo hàm bên dưới trùng khớp rất nhiều với những gì được giảng dạy trong các môn giải tích bậc phổ thông. Nếu bạn đang học hoặc đã từng học giải tích, phần lớn nội dung này sẽ rất quen thuộc. Nếu bạn thấy quy tắc nào khó hiểu, giáo viên toán của bạn rất có thể sẵn lòng ngồi giải thích cặn kẽ cùng bạn. Đây là những chủ đề được chạm tới nhiều nhất trong giáo dục toán, và một giáo viên giàu kinh nghiệm thường có thể giảng một quy tắc khó trong năm phút ở trên bảng còn hay hơn bất kỳ giáo trình nào trên giấy.

D.4.1 Quy tắc lũy thừa

Với $f(x) = x^n$, đạo hàm là:

$$f'(x) = nx^{n-1}$$

“Đưa số mũ xuống làm hệ số, rồi giảm số mũ đi một.”

Ví dụ: Quy tắc lũy thừa

$$\begin{aligned} f(x) = x^2 &\implies f'(x) = 2x^{2-1} = 2x \\ f(x) = x^3 &\implies f'(x) = 3x^{3-1} = 3x^2 \\ f(x) = x^5 &\implies f'(x) = 5x^4 \\ f(x) = x^1 = x &\implies f'(x) = 1 \cdot x^0 = 1 \\ f(x) = x^0 = 1 &\implies f'(x) = 0 \cdot x^{-1} = 0 \quad (\text{constants have slope zero}) \end{aligned}$$

Quy tắc lũy thừa cũng dùng được cho các số mũ âm và các số mũ phân số:

Quy tắc lũy thừa với số mũ âm và số mũ phân số

$$\begin{aligned} f(x) = x^{-1} = \frac{1}{x} &\implies f'(x) = -1 \cdot x^{-2} = -\frac{1}{x^2} \\ f(x) = x^{-2} = \frac{1}{x^2} &\implies f'(x) = -2 \cdot x^{-3} = -\frac{2}{x^3} \\ f(x) = x^{1/2} = \sqrt{x} &\implies f'(x) = \frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}} \\ f(x) = x^{1/3} = \sqrt[3]{x} &\implies f'(x) = \frac{1}{3}x^{-2/3} = \frac{1}{3x^{2/3}} \end{aligned}$$

D.4.2 Quy tắc hằng số nhân

Nếu bạn nhân một hàm với một hằng số, thì đạo hàm cũng sẽ được nhân với đúng hằng số đó:

$$\frac{d}{dx}[c \cdot f(x)] = c \cdot f'(x)$$

Các hằng số “đi cùng chuyển xe”.

Ví dụ: Quy tắc hằng số nhân

$$\begin{aligned} f(x) = 5x^3 &\implies f'(x) = 5 \cdot 3x^2 = 15x^2 \\ f(x) = -2x^4 &\implies f'(x) = -2 \cdot 4x^3 = -8x^3 \\ f(x) = \frac{x^2}{3} = \frac{1}{3}x^2 &\implies f'(x) = \frac{1}{3} \cdot 2x = \frac{2x}{3} \end{aligned}$$

D.4.3 Quy tắc tổng và hiệu

Đạo hàm của một tổng bằng tổng các đạo hàm:

$$\frac{d}{dx}[f(x) + g(x)] = f'(x) + g'(x)$$

Tương tự với hiệu: $\frac{d}{dx}[f(x) - g(x)] = f'(x) - g'(x)$.

Điều này có nghĩa là ta có thể đạo hàm từng số hạng một.

Ví dụ: Kết hợp các quy tắc

Hãy tìm đạo hàm của $f(x) = 3x^4 - 2x^3 + 7x^2 - x + 5$.
Ta đạo hàm từng số hạng một:

$$\begin{aligned} f'(x) &= 3 \cdot 4x^3 - 2 \cdot 3x^2 + 7 \cdot 2x - 1 + 0 \\ &= 12x^3 - 6x^2 + 14x - 1 \end{aligned}$$

D.4.4 Những đạo hàm đặc biệt quan trọng

Có những hàm xuất hiện thường xuyên đến mức đáng để ghi nhớ đạo hàm của chúng:

Hàm số	Đạo hàm	Ghi chú
e^x	e^x	Hàm duy nhất bằng chính đạo hàm của nó!
a^x	$a^x \ln(a)$	Với mọi hằng số dương a
$\ln(x)$	$\frac{1}{x}$	Logarit tự nhiên (cơ số e)
$\log_a(x)$	$\frac{1}{x \ln(a)}$	Logarit cơ số a
$\sin(x)$	$\cos(x)$	
$\cos(x)$	$-\sin(x)$	Chú ý dấu trừ

Việc e^x bằng chính đạo hàm của nó khiến nó có tầm quan trọng vô cùng đặc biệt trong toán học và xuất hiện khắp nơi trong học máy (trong softmax, trong các phân phối xác suất, trong các hàm kích hoạt).

D.4.5 Quy tắc chuỗi

Quy tắc chuỗi xử lý các hàm hợp thành, tức những hàm nằm bên trong các hàm khác.

Vấn đề

Đạo hàm của $(x^2 + 1)^3$ là gì?

Điều này *không* đơn giản là $x^6 + \dots$, bởi vì bạn không thể cứ bung nỏ rồi đạo hàm là xong. Thực ra bạn có thể, nhưng việc đó rất phiền phức... Hàm này có một cấu trúc: nó chứa một hàm "bên trong" $x^2 + 1$ và một hàm "bên ngoài" lập thành lập phương của thứ nằm bên trong.

Quy tắc

Nếu $y = f(g(x))$, tức là một hàm của một hàm, thì:

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$$

Diễn đạt bằng lời: **đạo hàm của hàm ngoài (tính tại hàm trong) $\times$ đạo hàm của hàm trong.**

Một cách tư duy trực giác

Hãy hình dung một chuỗi phụ thuộc:

- Nhiệt độ T phụ thuộc vào độ cao a : càng lên cao thì trời càng lạnh
- Độ cao a phụ thuộc vào thời gian t : càng leo thì độ cao càng tăng

Nhiệt độ thay đổi nhanh thế nào theo thời gian?

Nếu bạn leo cao 100 mét mỗi giờ, và nhiệt độ giảm 0.6°C trên mỗi 100 mét, thì nhiệt độ giảm 0.6°C mỗi giờ. Các tốc độ nhân với nhau!

$$\frac{dT}{dt} = \frac{dT}{da} \cdot \frac{da}{dt}$$

Đó chính là quy tắc chuỗi.

Ví dụ: Quy tắc chuỗi

Hãy tìm $\frac{d}{dx}(x^2 + 1)^3$.

Xác định các thành phần:

- Outer function: $f(u) = u^3 \implies f'(u) = 3u^2$
- Inner function: $g(x) = x^2 + 1 \implies g'(x) = 2x$

Áp dụng quy tắc chuỗi:

$$\frac{d}{dx}(x^2 + 1)^3 = \underbrace{3(x^2 + 1)^2}_{\text{outer derivative at inner}} \cdot \underbrace{2x}_{\text{inner derivative}} = 6x(x^2 + 1)^2$$

Ví dụ: Quy tắc chuỗi với hàm mũ

Find $\frac{d}{dx}e^{-x^2}$.

- Outer: $f(u) = e^u \implies f'(u) = e^u$
- Inner: $g(x) = -x^2 \implies g'(x) = -2x$

$$\frac{d}{dx}e^{-x^2} = e^{-x^2} \cdot (-2x) = -2xe^{-x^2}$$

Ví dụ: Quy tắc chuỗi với căn bậc hai

Find $\frac{d}{dx}\sqrt{x^3 + 2x}$.

Rewrite as $(x^3 + 2x)^{1/2}$.

- Outer: $f(u) = u^{1/2} \implies f'(u) = \frac{1}{2}u^{-1/2} = \frac{1}{2\sqrt{u}}$
- Inner: $g(x) = x^3 + 2x \implies g'(x) = 3x^2 + 2$

$$\frac{d}{dx}\sqrt{x^3 + 2x} = \frac{1}{2\sqrt{x^3 + 2x}} \cdot (3x^2 + 2) = \frac{3x^2 + 2}{2\sqrt{x^3 + 2x}}$$

D.4.6 Quy tắc tích

Khi hai hàm được *nhân* với nhau, ta cần quy tắc tích:

$$\frac{d}{dx}[f(x) \cdot g(x)] = f'(x) \cdot g(x) + f(x) \cdot g'(x)$$

Diễn đạt bằng lời: (đạo hàm của hàm thứ nhất $\times$ nhân hàm thứ hai) $+$ (hàm thứ nhất $\times$ nhân đạo hàm của hàm thứ hai).

Ví dụ: Quy tắc tích

Hãy tìm $\frac{d}{dx}[x^2 \cdot e^x]$.

Đặt $f(x) = x^2$ và $g(x) = e^x$. Khi

đó $f'(x) = 2x$ và $g'(x) = e^x$.

$$\begin{aligned} \frac{d}{dx}[x^2 \cdot e^x] &= f'(x) \cdot g(x) + f(x) \cdot g'(x) \\ &= 2x \cdot e^x + x^2 \cdot e^x \\ &= e^x(2x + x^2) \\ &= xe^x(2 + x) \end{aligned}$$

Ví dụ: Quy tắc tích với ba hàm

Hãy tìm $\frac{d}{dx}[x \cdot \sin(x) \cdot e^x]$.

Với ba hàm, ta áp dụng quy tắc tích một cách lặp đi lặp lại, hoặc dùng công thức mẫu sau:

$$(fgh)' = f'gh + fg'h + fgh'$$

Let $f = x$, $g = \sin(x)$, $h = e^x$.

$$\begin{aligned}\frac{d}{dx}[x \sin(x) e^x] &= 1 \cdot \sin(x) \cdot e^x + x \cdot \cos(x) \cdot e^x + x \cdot \sin(x) \cdot e^x \\ &= e^x[\sin(x) + x \cos(x) + x \sin(x)]\end{aligned}$$

D.4.7 Quy tắc thương

Khi một hàm được *chia* cho một hàm khác:

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{f'(x) \cdot g(x) - f(x) \cdot g'(x)}{[g(x)]^2}$$

Một mẹo ghi nhớ: “Thấp d-Cao trừ Cao d-Thấp, chia cho Thấp bình phương” (trong đó “d” có nghĩa là “đạo hàm của”).

Ví dụ: Quy tắc thương

Hãy tìm $\frac{d}{dx} \left[\frac{x^2}{x+1} \right]$.

Đặt $f(x) = x^2$ (cao) và $g(x) = x + 1$ (thấp).

Khi đó $f'(x) = 2x$ và $g'(x) = 1$.

$$\begin{aligned}\frac{d}{dx} \left[\frac{x^2}{x+1} \right] &= \frac{2x \cdot (x+1) - x^2 \cdot 1}{(x+1)^2} \\ &= \frac{2x^2 + 2x - x^2}{(x+1)^2} \\ &= \frac{x^2 + 2x}{(x+1)^2} = \frac{x(x+2)}{(x+1)^2}\end{aligned}$$

Liên hệ với các quy tắc khác: Thực ra quy tắc thương có thể suy ra từ quy tắc tích và quy tắc chuỗi. Nếu bạn viết $\frac{f}{g} = f \cdot g^{-1}$, rồi áp dụng quy tắc tích và quy tắc chuỗi (cho g^{-1}), bạn sẽ thu được quy tắc thương. Đó chính là lý do ta đã giới thiệu quy tắc chuỗi trước tiên!

D.5 Đạo hàm riêng: Hàm của nhiều biến

D.5.1 Sự cần thiết của đạo hàm riêng

Cho đến nay, các hàm của ta chỉ có một đầu vào. Nhưng các đại lượng trong thực tế thường phụ thuộc vào nhiều biến:

- Giá của một ngôi nhà phụ thuộc vào diện tích, số phòng ngủ, vị trí và tuổi của nó
- Nhiệt độ tại một điểm phụ thuộc vào vĩ độ, kinh độ, độ cao và thời gian
- Hàm mất mát của một mạng nơ-ron phụ thuộc vào hàng nghìn hay hàng triệu tham số trọng số

Nếu $f(x, y)$ phụ thuộc vào cả x lẫn y , thì “độ dốc” của nó là bao nhiêu? Câu trả lời là: điều đó phụ thuộc vào hướng mà bạn đang di chuyển!

D.5.2 Định nghĩa đạo hàm riêng

Một **đạo hàm riêng** đo lường mức f thay đổi như thế nào khi ta thay đổi *một* đầu vào trong khi giữ cố định tất cả các đầu vào còn lại.

Đạo hàm riêng của f theo x , viết là $\frac{\partial f}{\partial x}$, bằng:

$$\frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x + h, y) - f(x, y)}{h}$$

Ký hiệu ∂ (một chữ “d” viết nghiêng) biểu thị một đạo hàm riêng. Nó nhắc ta rằng chúng ta chỉ đang xét một phần của tổng mức thay đổi.

Để tính $\frac{\partial f}{\partial x}$: Hãy coi mọi biến khác x là hằng số, rồi đạo hàm theo x bằng các quy tắc thông thường.

Ví dụ: Tính đạo hàm riêng

Cho $f(x, y) = x^2 + 3xy + y^2$.

Đạo hàm riêng theo x :

Coi y là một hằng số. Số hạng x^2 cho đạo hàm $2x$. Số hạng $3xy$ có dạng $3y \cdot x$ với $3y$ là hằng số, nên cho đạo hàm $3y$. Số hạng y^2 là một hằng số (theo x), nên cho đạo hàm 0.

$$\frac{\partial f}{\partial x} = 2x + 3y + 0 = 2x + 3y$$

Đạo hàm riêng theo y :

Giờ hãy coi x là hằng số. Thừa số x^2 là hằng số, cho ra 0. Thừa số $3xy$ vì phân ra $3x$. Thừa số y^2 vì phân ra $2y$.

$$\frac{\partial f}{\partial y} = 0 + 3x + 2y = 3x + 2y$$

D.5.3 Đánh giá đạo hàm riêng tại các điểm cụ thể

Đạo hàm riêng chính là các hàm. Chúng ta có thể đánh giá chúng tại các điểm cụ thể để thu được các con số.

Ví dụ: Đạo hàm riêng tại một điểm

Với $f(x, y) = x^2 + 3xy + y^2$, chúng ta đã tìm được:

$$\frac{\partial f}{\partial x} = 2x + 3y, \quad \frac{\partial f}{\partial y} = 3x + 2y$$

Tại điểm $(2, 1)$:

$$\left. \frac{\partial f}{\partial x} \right|_{(2,1)} = 2(2) + 3(1) = 4 + 3 = 7$$

$$\left. \frac{\partial f}{\partial y} \right|_{(2,1)} = 3(2) + 2(1) = 6 + 2 = 8$$

Điều này nghĩa là: tại điểm $(2, 1)$, nếu ta tăng x một chút trong khi giữ $y = 1$, thì f tăng với tốc độ 7 đơn vị cho mỗi đơn vị tăng của x . Nếu ta tăng y một chút trong khi giữ $x = 2$, thì f tăng với tốc độ 8 đơn vị cho mỗi đơn vị tăng của y .

D.5.4 Thêm ví dụ

Ví dụ: Đạo hàm riêng của một tích

Let $f(x, y) = x^2 y^3$.

$$\frac{\partial f}{\partial x} = 2x \cdot y^3 = 2xy^3 \quad (\text{treating } y^3 \text{ as a constant})$$

$$\frac{\partial f}{\partial y} = x^2 \cdot 3y^2 = 3x^2 y^2 \quad (\text{treating } x^2 \text{ as a constant})$$

Ví dụ: Đạo hàm riêng với hàm mũ

Gọi $f(x, y) = e^{xy}$.

Ở đây chúng ta cần quy tắc chuỗi. Hàm nội là xy .

$$\frac{\partial f}{\partial x} = e^{xy} \cdot \frac{\partial}{\partial x}(xy) = e^{xy} \cdot y = ye^{xy}$$

$$\frac{\partial f}{\partial y} = e^{xy} \cdot \frac{\partial}{\partial y}(xy) = e^{xy} \cdot x = xe^{xy}$$

Ví dụ: Ba biến

Let $f(x, y, z) = x^2 + 2xy + yz + z^3$.

$$\frac{\partial f}{\partial x} = 2x + 2y + 0 + 0 = 2x + 2y$$

$$\frac{\partial f}{\partial y} = 0 + 2x + z + 0 = 2x + z$$

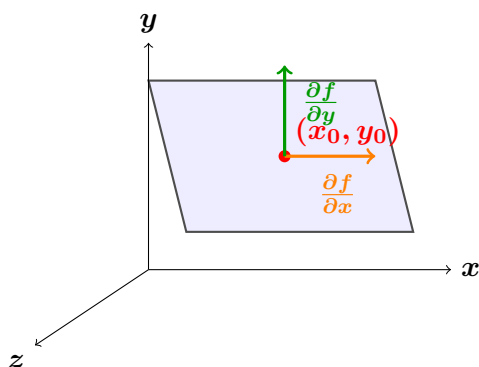
$$\frac{\partial f}{\partial z} = 0 + 0 + y + 3z^2 = y + 3z^2$$

D.5.5 Diễn giải hình học

Xét mặt $z = f(x, y)$, hãy hình dung một vùng đất gồ ghề nơi độ cao z phụ thuộc vào vị trí đông-tây (x) và vị trí nam-bắc (y).

$\frac{\partial f}{\partial x}$ là độ dốc của mặt theo hướng x (đông-tây). Nếu bạn đứng tại một điểm và đi thẳng về phía đông, đó là mức độ dốc khi bạn leo lên hoặc đi xuống.

$\frac{\partial f}{\partial y}$ là độ dốc theo hướng y (nam-bắc).



Hình D.5: Đạo hàm riêng đo độ dốc theo từng hướng tọa độ riêng lẻ.

D.6 Gradient

D.6.1 Định nghĩa

gradient của hàm f thu thập tất cả các đạo hàm riêng của nó thành một vector:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{pmatrix}$$

Với hàm của n biến $f(x_1, x_2, \dots, x_n)$:

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Ký hiệu ∇ được gọi là “nabla” hay “del.” Gradient ∇f là một **vector**, không phải là vô hướng (scalar).

Ví dụ: Tính gradient

For $f(x, y) = x^2 + 3xy + y^2$:

$$\nabla f = \begin{pmatrix} 2x + 3y \\ 3x + 2y \end{pmatrix}$$

Tại điểm $(2, 1)$:

$$\nabla f(2, 1) = \begin{pmatrix} 2(2) + 3(1) \\ 3(2) + 2(1) \end{pmatrix} = \begin{pmatrix} 7 \\ 8 \end{pmatrix}$$

Ví dụ: Gradient tại nhiều điểm

Với $f(x, y) = x^2 + y^2$ (một paraboloid, giống một cái bát):

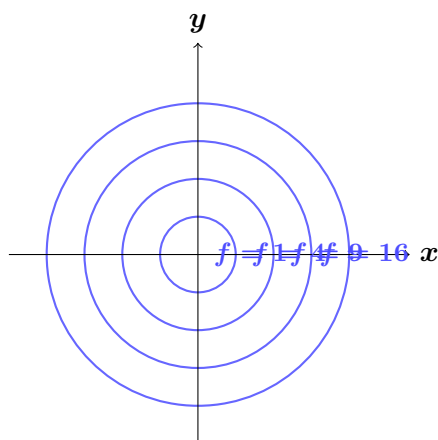
$$\nabla f = \begin{pmatrix} 2x \\ 2y \end{pmatrix}$$

Điểm (x, y)	Gradient ∇f	Diễn giải
$(0, 0)$	$(0, 0)$	Phẳng (tại đáy của cái bát)
$(1, 0)$	$(2, 0)$	Chỉ theo hướng $+x$
$(0, 1)$	$(0, 2)$	Chỉ theo hướng $+y$
$(1, 1)$	$(2, 2)$	Chỉ theo hướng chéo ra ngoài
$(-1, -1)$	$(-2, -2)$	Chỉ về phía dưới bên trái

D.6.2 Hiểu các đường đồng mức

Để hiểu gradient về mặt hình học, trước tiên chúng ta cần hiểu **đường đồng mức** (còn gọi là các đường cùng cấp).

Một đường đồng mức nối tất cả các điểm mà hàm có cùng giá trị. Trên bản đồ địa hình, các đường đồng mức nối các điểm có cùng độ cao. Trên bản đồ thời tiết, chúng nối các điểm có cùng áp suất hoặc nhiệt độ.



Contour lines for $f(x, y) = x^2 + y^2$
(circles centered at origin, minimum at center)

Hình D.6: Các đường đồng mức nối các điểm có cùng giá trị hàm. Ở đây, điểm cực tiểu nằm tại gốc tọa độ.

Nhận xét chính:

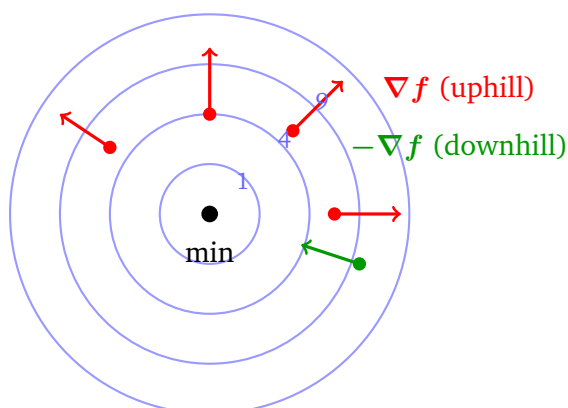
- Nơi các đường đồng mức nằm gần nhau, hàm thay đổi nhanh (độ dốc lớn)
- Nơi các đường đồng mức nằm xa nhau, hàm thay đổi chậm (độ dốc thoải)
- Tại một cực tiểu hoặc cực đại, các đường đồng mức tạo thành các vòng khép kín quanh điểm đó

D.6.3 Gradient chỉ lên dốc

Gradient có một tính chất hình học đẹp: **gradient chỉ theo hướng tăng dốc nhất.**

Nếu bạn đứng trên một sườn đồi và muốn leo dốc nhất có thể, hãy đi theo hướng của gradient. Gradient vuông góc với các đường đồng mức và chỉ về phía các giá trị lớn hơn.

Ngược lại, nếu bạn muốn *xuống dốc* dốc nhất có thể (điều mà chúng ta muốn khi cực tiểu hóa hàm mất mát), hãy đi theo hướng của $-\nabla f$.



Hình D.7: Các vector gradient (màu đỏ) chỉ vuông góc với các đường đồng mức, về phía giá trị tăng dần. Gradient âm (màu xanh lá) chỉ về phía cực tiểu.

Tại sao gradient chỉ lên dốc? Hãy hình dung việc đứng trên một sườn đồi. Gradient thu thập các độ dốc theo mọi hướng tọa độ. Sự kết hợp của các độ dốc này cho độ dốc tăng lớn nhất chính là hướng của gradient. Về mặt toán học, đạo hàm theo hướng (tốc độ thay đổi theo bất kỳ hướng nào) đạt cực đại khi bạn di chuyển theo hướng gradient.

D.7 Rời gradient: Tìm cực tiểu

D.7.1 Thuật toán

Bây giờ chúng ta có thể phát biểu thuật toán trung tâm của tối ưu hóa học máy. Để cực tiểu hóa hàm $f(\theta)$ trong đó θ đại diện cho các tham số của chúng ta:

1. **Khởi tạo:** Bắt đầu với một phỏng đoán ban đầu θ_0
2. **Tính gradient:** Tính $\nabla f(\theta)$
3. **Cập nhật:** Đi một bước theo hướng xuống dốc:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla f(\theta_{\text{old}})$$

4. **Lặp lại:** Quay lại bước 2 cho đến khi gradient gần bằng không

Tham số α được gọi là **tốc độ học** (learning rate), nó kiểm soát mỗi bước lớn như thế nào.

D.7.2 Ví dụ một biến

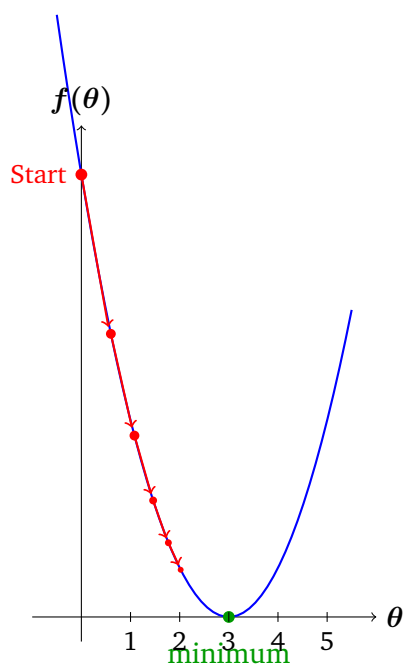
Hãy cực tiểu hóa $f(\theta) = (\theta - 3)^2$ bắt đầu từ $\theta_0 = 0$ với tốc độ học $\alpha = 0.1$.

Rõ ràng cực tiểu nằm tại $\theta = 3$ (nơi hạng tử bình phương bằng không). Hãy xem rồi gradient tìm ra nó.

Đầu tiên, tìm đạo hàm: $f'(\theta) = 2(\theta - 3)$.

Iteration	θ	$f(\theta)$	$f'(\theta)$	$\theta_{\text{new}} = \theta - 0.1 \cdot f'(\theta)$
0	0	9	-6	$0 - 0.1(-6) = 0.6$
1	0.6	5.76	-4.8	$0.6 - 0.1(-4.8) = 1.08$
2	1.08	3.69	-3.84	$1.08 + 0.384 = 1.464$
3	1.464	2.36	-3.07	$1.464 + 0.307 = 1.771$
4	1.771	1.51	-2.46	$1.771 + 0.246 = 2.017$
5	2.017	0.97	-1.97	$2.017 + 0.197 = 2.214$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
∞	3	0	0	3

Dãy $0 \rightarrow 0.6 \rightarrow 1.08 \rightarrow 1.464 \rightarrow \dots$ hội tụ về 3.



Hình D.8: Rồi gradient lấy các bước về phía cực tiểu.

D.7.3 Ví dụ hai biến

Hãy cực tiểu hóa $f(x, y) = x^2 + y^2$ bắt đầu từ $(x_0, y_0) = (4, 3)$ với $\alpha = 0.1$.

Gradient là $\nabla f = (2x, 2y)$.

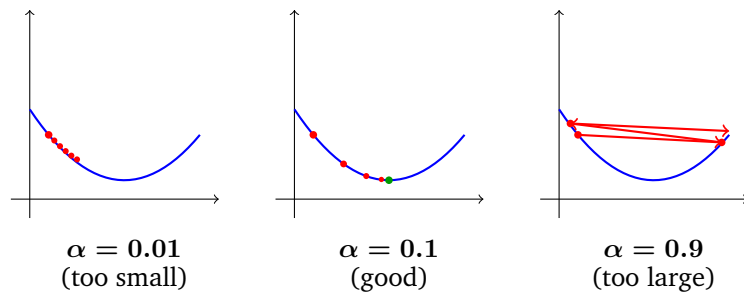
Iter	x	y	$f(x, y)$	∇f	$(x, y)_{\text{new}}$
0	4	3	25	(8, 6)	$(4 - 0.8, 3 - 0.6) = (3.2, 2.4)$
1	3.2	2.4	16	(6.4, 4.8)	(2.56, 1.92)
2	2.56	1.92	10.24	(5.12, 3.84)	(2.05, 1.54)
3	2.05	1.54	6.55	(4.1, 3.08)	(1.64, 1.23)
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
∞	0	0	0	(0, 0)	(0, 0)

Đường đi xoắn ốc hướng vào trong về phía cực tiểu tại gốc tọa độ.

D.7.4 Tốc độ học quan trọng

Tốc độ học α là rất quan trọng cho việc tối ưu hóa thành công:

- **Quá nhỏ:** Mỗi bước rất nhỏ, vì vậy hội tụ rất chậm. Bạn có thể cần hàng nghìn lần lặp.
- **Quá lớn:** Các bước vượt qua cực tiểu, nảy qua nảy lại. Thuật toán có thể không bao giờ hội tụ hoặc thậm chí phân kỳ về vô cực.
- **Vừa phải:** Thuật toán hội tụ hiệu quả trong một số bước hợp lý.



Hình D.9: Tác động của tốc độ học lên sự hội tụ của ròi gradient.

D.8 Điểm tới hạn và tối ưu hóa

D.8.1 Điểm tới hạn là gì?

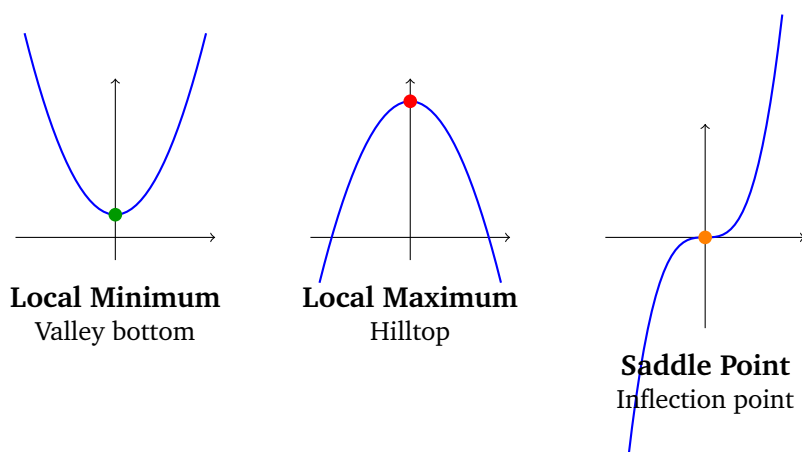
Một **điểm tới hạn** là nơi gradient bằng không: $\nabla f = 0$.

Tại một điểm tới hạn, hàm “phẳng” theo mọi hướng, không có hướng tăng hay giảm nào. Ròi gradient sẽ dừng tại một điểm tới hạn (vì phép cập nhật $-\alpha \nabla f$ trở thành không).

D.8.2 Các loại điểm tới hạn

Không phải mọi điểm tới hạn đều là cực tiểu! Điểm tới hạn có ba loại:

- **Cực tiểu cục bộ:** Hàm tăng theo mọi hướng từ điểm này. Đây là đáy thung lũng.
- **Cực đại cục bộ:** Hàm giảm theo mọi hướng. Đây là đỉnh đồi.
- **Điểm yên ngựa:** Hàm tăng theo một số hướng và giảm theo các hướng khác. Giống một đèo núi hoặc yên ngựa.



Hình D.10: Ba loại điểm tới hạn. Tại cả ba, $f'(x) = 0$.

D.8.3 Tìm điểm tới hạn

Để tìm các điểm tới hạn:

1. Tính đạo hàm (hoặc gradient)
2. Đặt nó bằng không
3. Giải tìm x (hoặc $\mathbf{x}$)

Ví dụ: Tìm điểm tới hạn

Tìm các điểm tới hạn của $f(x) = x^3 - 3x^2 - 9x + 5$.

Bước 1: Tính đạo hàm.

$$f'(x) = 3x^2 - 6x - 9$$

Bước 2: Đặt bằng không.

$$3x^2 - 6x - 9 = 0$$

Bước 3: Giải. Chia cho 3:

$$x^2 - 2x - 3 = 0$$

Factor:

$$(x - 3)(x + 1) = 0$$

Nghiệm: $x = 3$ và $x = -1$.

Hàm có các điểm tới hạn tại $x = -1$ và $x = 3$.

D.8.4 Kiểm định đạo hàm cấp hai

Khi đã tìm được các điểm tới hạn, làm sao để phân loại chúng? **Đạo hàm cấp hai** giúp ích.

Đạo hàm cấp hai $f''(x)$ là đạo hàm của đạo hàm. Nó đo mức thay đổi của chính độ dốc:

- Nếu $f''(x) > 0$: độ dốc đang tăng, vì vậy đường cong uốn hướng lên (concave up, giống một nụ cười)
- Nếu $f''(x) < 0$: độ dốc đang giảm, vì vậy đường cong uốn hướng xuống (concave down, giống một cái nhú mày)

Tại một điểm tới hạn nơi $f'(x) = 0$:

- $f''(x) > 0 \implies$ cực tiểu cục bộ (đường cong cười $\smile$ có cực tiểu ở đáy)
- $f''(x) < 0 \implies$ cực đại cục bộ (đường cong cau $\frown$ có cực đại ở đỉnh)
- $f''(x) = 0 \implies$ kiểm định không kết luận được; cần các phương pháp khác

Ví dụ: Phân loại các điểm tới hạn

Với $f(x) = x^3 - 3x^2 - 9x + 5$, chúng ta đã tìm được các điểm tới hạn tại $x = -1$ và $x = 3$.

Tính đạo hàm cấp hai:

$$f'(x) = 3x^2 - 6x - 9 \implies f''(x) = 6x - 6$$

At $x = -1$:

$$f''(-1) = 6(-1) - 6 = -12 < 0 \implies \text{local maximum}$$

At $x = 3$:

$$f''(3) = 6(3) - 6 = 12 > 0 \implies \text{local minimum}$$

D.8.5 Cực tiểu toàn cục và cực tiểu cục bộ

Một **cực tiểu cục bộ** là điểm thấp hơn tất cả các điểm lân cận, nhưng có thể có những điểm thấp hơn nữa ở nơi khác.

Một **cực tiểu toàn cục** là điểm thấp nhất của toàn bộ hàm.

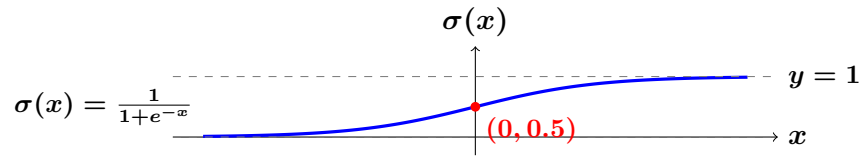
Rồi gradient tìm các cực tiểu cục bộ, không nhất thiết là cực tiểu toàn cục. Trong tối ưu hóa nhiều chiều (như huấn luyện mạng nơ-ron), có thể có nhiều cực tiểu cục bộ. May mắn thay, nghiên cứu cho thấy trong nhiều bài toán học máy, hầu hết các cực tiểu cục bộ gần bằng cực tiểu toàn cục.

Với các mô hình đơn giản như hồi quy tuyến tính (Chương 4), hàm mất mát là hàm lồi: có đúng một cực tiểu, và rồi gradient được đảm bảo sẽ tìm ra nó. Nhưng trong học sâu (Chương 14), các bề mặt mất mát **không lồi**: chúng chứa nhiều cực tiểu cục bộ, điểm yên ngựa và các vùng phẳng. Đây là một lý do khiến việc huấn luyện mạng sâu mang nhiều tính nghệ thuật hơn khoa học. Lựa chọn tốc độ học, cách khởi tạo và thuật toán tối ưu hóa (Adam, SGD có momentum, và những cái khác) đều ảnh hưởng tới cực tiểu cục bộ mà rồi gradient dừng lại. Lý thuyết trong phụ lục này cung cấp nền tảng cho bạn; thực hành đòi hỏi thử nghiệm.

D.9 Các hàm quan trọng và đạo hàm của chúng**D.9.1 Hàm sigmoid**

Hàm sigmoid ánh xạ mọi số thực về một giá trị trong khoảng từ 0 đến 1:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Hình D.11: Hàm sigmoid chuyển tiếp mượt mà từ 0 đến 1.

Sigmoid được dùng trong hồi quy logistic và như một hàm kích hoạt trong các mạng nơ-ron. Đạo hàm của nó có dạng thanh lịch:

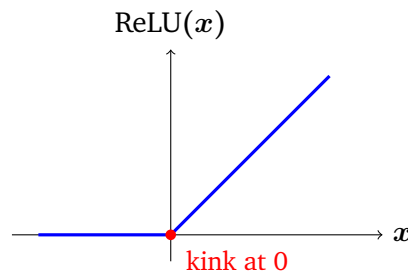
$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

Điều này tiện lợi về mặt tính toán: khi bạn đã tính $\sigma(x)$, bạn nhận được $\sigma'(x)$ chỉ bằng một phép trừ và một phép nhân.

D.9.2 Hàm ReLU

ReLU (Rectified Linear Unit) là hàm kích hoạt phổ biến nhất trong các mạng nơ-ron hiện đại:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$



Hình D.12: Hàm ReLU: bằng không với các đầu vào âm, bằng bản thân với các đầu vào dương.

Đạo hàm của nó là:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Tại $x = 0$, đạo hàm về mặt kỹ thuật không xác định (có một “nếp gấp”), nhưng trong thực tế, chúng ta thường định nghĩa nó là 0 hoặc 1.

D.9.3 Đạo hàm của hàm mất mát

Lỗi bình phương trung bình (MSE):

Với một dự đoán đơn lẻ $\hat{y}$ và giá trị đúng y :

$$\mathcal{L} = (y - \hat{y})^2$$

Đạo hàm theo dự đoán:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = 2(\hat{y} - y)$$

Diễn giải: Nếu $\hat{y} > y$ (dự đoán quá cao), gradient là dương, báo chúng ta cần giảm $\hat{y}$. Nếu $\hat{y} < y$ (dự đoán quá thấp), gradient là âm, báo chúng ta cần tăng $\hat{y}$.

Với nhiều dự đoán, với các dự đoán $\hat{y}_1, \dots, \hat{y}_n$ và các giá trị đúng $y_1, \dots, y_n$:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \frac{2}{n} (\hat{y}_i - y_i)$$

D.10 Tóm tắt

Tham chiếu nhanh về Giải tích

Đạo hàm: Đo độ dốc / tốc độ thay đổi.

- $f'(x) > 0$: hàm đang tăng
- $f'(x) < 0$: hàm đang giảm
- $f'(x) = 0$: điểm tới hạn

Quy tắc đạo hàm:

- Lũy thừa: $(x^n)' = nx^{n-1}$
- Nhân hằng số: $(cf)' = cf'$
- Sum: $(f + g)' = f' + g'$
- Chuỗi: $(f(g(x)))' = f'(g(x)) \cdot g'(x)$
- Tích: $(fg)' = f'g + fg'$
- Thương: $(f/g)' = (f'g - fg')/g^2$

Sp đặc biệt: $(e^x)' = e^x$, $(\ln x)' = 1/x$

Đạo hàm riêng: $\frac{\partial f}{\partial x}$ = đạo hàm coi các biến khác là hằng số

Gradient: ∇f = vector của tất cả các đạo hàm riêng; chỉ theo hướng tăng dốc nhất

Rồi gradient: $\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla f$

- α = tốc độ học
- Hội tụ về các điểm tới hạn (hy vọng là các cực tiểu)

Kiểm định đạo hàm cấp hai: Tại điểm tới hạn nơi $f'(x) = 0$:

- $f''(x) > 0$: cực tiểu cục bộ
- $f''(x) < 0$: cực đại cục bộ

Phụ lục E

Logic

Logic is the beginning of wisdom,
not the end.

Leonard Nimoy as Spock

E.1 Giới thiệu: Vì sao lại là Logic?

Logic là ngành nghiên cứu về lập luận hợp lệ. Nó cung cấp các quy tắc để xác định khi nào một kết luận tất yếu suy ra từ những tiền đề đã cho.

Trong trí tuệ nhân tạo, logic giữ nhiều vai trò quan trọng. Về lịch sử, những hệ thống AI đầu tiên được xây dựng hoàn toàn trên nền lập luận logic, tức là các chương trình thao tác ký hiệu theo các quy tắc logic. Ngày nay, logic vẫn còn thiết yếu: nó là nền tảng của toán học mà chúng ta dùng để chứng minh thuật toán hoạt động đúng, nó cung cấp nền tảng cho các ngôn ngữ truy vấn cơ sở dữ liệu, và nó gắn trực tiếp với cách cây quyết định và các hệ thống dựa trên quy tắc thực hiện phân loại.

Hiểu logic giúp bạn tư duy chính xác, một kỹ năng quan trọng khi thiết kế thuật toán, chứng minh tính đúng đắn của chúng, hay phân tích những giới hạn của chúng. Các toán tử logic AND, OR và NOT xuất hiện khắp nơi trong khoa học máy tính, từ các câu lệnh if trong mã nguồn cho đến các điều kiện xác định ranh giới phân loại.

Phụ lục này trình bày về logic mệnh đề, bảng chân lý, các tương đương logic, các lượng từ, và mối liên hệ với trí tuệ nhân tạo.

E.2 Mệnh đề và Giá trị chân lý

E.2.1 Mệnh đề là gì?

Một **mệnh đề** là một phát biểu đúng hoặc sai, nhưng không đồng thời cả hai. Thông thường ta dùng các chữ cái như P, Q, R để biểu diễn các mệnh đề.

Ví dụ về mệnh đề:

- “Trời đang mưa” (đúng hoặc sai tùy theo thời tiết)
- “ $2 + 2 = 4$ ” (đúng)
- “Paris là thủ đô của Đức” (sai)
- “Độ chính xác của mô hình vượt trên 90%” (đúng hoặc sai tùy theo mô hình)

Không phải mệnh đề:

- “Bây giờ là mấy giờ?” (một câu hỏi, không phải một phát biểu)
- “Đóng cửa lại” (một mệnh lệnh)
- “ $x > 5$ ” (phụ thuộc vào x ; đây là một *vị từ*, không phải mệnh đề)

E.2.2 Giá trị chân lý

Mỗi mệnh đề đều có một **giá trị chân lý**: hoặc **Đúng** (T, 1) hoặc **Sai** (F, 0). Logic nói về cách các giá trị chân lý kết hợp với nhau khi ta nối các mệnh đề bằng các từ như “và”, “hoặc” và “không”.

E.3 Các phép nối logic

E.3.1 NOT (Phủ định)

Phủ định của P , viết là $\neg P$ hoặc $\bar{P}$ hoặc NOT P , đơn giản chỉ là đảo ngược giá trị chân lý: đúng thành sai, sai thành đúng.

P	$\neg P$
T	F
F	T

Ví dụ: Nếu P = “Trời đang mưa”, thì $\neg P$ = “Trời không mưa”.

E.3.2 AND (Hợp lượng)

Hợp lượng của P và Q , viết là $P \wedge Q$, chỉ đúng khi **cả hai** P và Q cùng đúng.

P	Q	$P \wedge Q$
T	T	T
T	F	F
F	T	F
F	F	F

Ví dụ: “Trời đang mưa AND tôi có ô” chỉ đúng khi cả hai điều kiện đều thỏa mãn.

E.3.3 OR (Phép tuyển)

Phép tuyển của P và Q , viết là $P \vee Q$, đúng khi **ít nhất một** trong P hoặc Q là đúng.

P	Q	$P \vee Q$
T	T	T
T	F	T
F	T	T
F	F	F

Lưu ý: Đây là **phép tuyển bao hàm**, tức là cho phép “cả hai cùng đúng”. Điều này khác với tiếng Anh thông dụng, nơi “or” đôi khi có nghĩa “một trong hai nhưng không phải cả hai” (như “Bạn muốn trà hay cà phê?”).

E.3.4 XOR (Phép tuyển loại trừ)

Phép tuyển loại trừ, viết là $P \oplus Q$, đúng khi **đúng một** trong P hoặc Q là đúng, nhưng không phải cả hai.

P	Q	$P \oplus Q$
T	T	F
T	F	T
F	T	T
F	F	F

E.3.5 Liên hệ với Tập hợp

Các phép nối logic tương ứng trực tiếp với các phép toán tập hợp:

Logic	Tập hợp	Ý nghĩa
$\neg P$	A^c or $\bar{A}$	Bù (không thuộc A)
$P \wedge Q$	$A \cap B$	Giao (thuộc cả hai)
$P \vee Q$	$A \cup B$	Hợp (thuộc ít nhất một)

Sự tương ứng này không phải ngẫu nhiên. Nếu ta coi P là “phần tử nằm trong tập A ”, thì các phép toán logic trên P tương ứng chính xác với các phép toán tập hợp

trên A .

E.4 Phép hàm ý và Phép tương đương

E.4.1 Phép hàm ý (Nếu-Thì)

Phép hàm ý $P \rightarrow Q$, đọc là “nếu P thì Q ” hoặc “ P hàm ý Q ”, là nền tảng của lập luận logic.

P	Q	$P \rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

Bảng chân lý có vẻ phản trực giác, đặc biệt là hai hàng cuối. Điểm mấu chốt: $P \rightarrow Q$ chỉ sai khi P đúng nhưng Q sai. Tức là khi phần “nếu” đúng nhưng phần “thì” lại không thành.

Hiểu về Phép hàm ý

Xét lời hứa: “Nếu trời mưa, tôi sẽ mang ô.”

- Trời mưa, tôi mang ô: giữ lời hứa (đúng)
- Trời mưa, tôi không mang ô: thất hứa (sai)
- Trời không mưa, tôi mang ô: không vi phạm lời hứa (đúng)
- Trời không mưa, tôi không mang ô: không vi phạm lời hứa (đúng)

Lời hứa chỉ quy định điều xảy ra nếu trời mưa. Nếu trời không mưa, bạn không thất hứa bất kể bạn làm gì.

E.4.2 Phép tương đương (Nếu và chỉ nếu)

Phép tương đương $P \leftrightarrow Q$, đọc là “ P nếu và chỉ nếu Q ” (thường viết tắt là “ P iff Q ”), đúng khi P và Q có cùng giá trị chân lý.

P	Q	$P \leftrightarrow Q$
T	T	T
T	F	F
F	T	F
F	F	T

$P \leftrightarrow Q$ nghĩa là cả $P \rightarrow Q$ và $Q \rightarrow P$. Điều này thiết lập rằng P và Q tương đương về mặt logic. Khi nào cái này đúng thì cái kia cũng phải đúng, và khi nào cái này sai thì cái kia cũng phải sai.

Ví dụ: “Một tam giác đều nếu và chỉ nếu cả ba góc của nó đều bằng 60° .” Điều này

nghĩa là: nếu nó đều thì cả ba góc đều bằng 60° , VÀ nếu cả ba góc đều bằng 60° thì nó là tam giác đều.

E.4.3 Tương đương logic

Hai biểu thức là **tương đương logic**, viết là $\equiv$, nếu chúng có cùng giá trị chân lý với mọi đầu vào có thể có.

Ví dụ, $P \rightarrow Q \equiv \neg P \vee Q$. Ta có thể kiểm chứng điều này bằng bảng chân lý:

P	Q	$P \rightarrow Q$	$\neg P$	$\neg P \vee Q$
T	T	T	F	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Các cột của $P \rightarrow Q$ và $\neg P \vee Q$ giống hệt nhau, xác nhận sự tương đương.

Sự tương đương này hữu ích trong thực tế: nó cho phép ta viết lại bất kỳ phép hàm ý nào chỉ bằng OR và NOT.

E.5 Các Tương đương logic Quan trọng

E.5.1 Các quy luật bạn đã biết từ số học

Nhiều tương đương logic phản chiếu các quy luật bạn đã biết từ số học. Điều này không phải ngẫu nhiên. AND hành xử khá giống phép nhân, và OR hành xử khá giống phép cộng.

Tính giao hoán: Thứ tự không quan trọng.

$$\begin{aligned} \text{Numbers: } 3 + 5 &= 5 + 3 \quad \text{and} \quad 3 \times 5 = 5 \times 3 \\ \text{Logic: } P \vee Q &\equiv Q \vee P \quad \text{and} \quad P \wedge Q \equiv Q \wedge P \end{aligned}$$

Tính kết hợp: Cách gộp nhóm không quan trọng.

$$\begin{aligned} \text{Numbers: } (2 + 3) + 4 &= 2 + (3 + 4) \\ \text{Logic: } (P \vee Q) \vee R &\equiv P \vee (Q \vee R) \end{aligned}$$

Tính phân phối: Ở đây mới thú vị!

Với các số, phép nhân phân phối trên phép cộng:

$$3 \times (4 + 5) = (3 \times 4) + (3 \times 5) = 12 + 15 = 27$$

Trong logic, AND phân phối trên OR (giống hết phép nhân phân phối trên phép cộng):

$$P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$$

Nhưng đây là điều KHÔNG đúng với các số thông thường: phép cộng không phân phối trên phép nhân. $3 + (4 \times 5) \neq (3 + 4) \times (3 + 5)$.

Tuy nhiên, trong logic, OR LẠI phân phối trên AND:

$$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$$

Đây là một khác biệt then chốt giữa logic và số học!

E.5.2 Phần tử đơn vị và Phần tử hấp thụ

Với các số, 0 là phần tử đơn vị của phép cộng ($x + 0 = x$) và 1 là phần tử đơn vị của phép nhân ($x \times 1 = x$).

Trong logic:

$$\begin{aligned} P \vee \text{False} &\equiv P && (\text{False is the identity for OR}) \\ P \wedge \text{True} &\equiv P && (\text{True is the identity for AND}) \end{aligned}$$

Và cũng như $x \times 0 = 0$ bất kể x :

$$\begin{aligned} P \wedge \text{False} &\equiv \text{False} && (\text{False dominates AND}) \\ P \vee \text{True} &\equiv \text{True} && (\text{True dominates OR}) \end{aligned}$$

E.5.3 Phủ định và Phủ định kép

Phủ định kép: $\neg(\neg P) \equiv P$

Cũng như $-(-5) = 5$ với các số, phủ định hai lần đưa bạn trở về trạng thái ban đầu.

Phủ định với AND/OR: Điều gì xảy ra khi ta phủ định một tổ hợp? Điều này dẫn ta tới các Luật De Morgan.

E.5.4 Luật De Morgan

Các luật De Morgan cho biết phủ định tương tác với AND và OR như thế nào:

$$\begin{aligned} \neg(P \wedge Q) &\equiv \neg P \vee \neg Q \\ \neg(P \vee Q) &\equiv \neg P \wedge \neg Q \end{aligned}$$

Hãy chú ý khuôn mẫu: khi bạn đẩy một NOT xuyên qua một AND hoặc OR, cái AND trở thành OR (và ngược lại).

Ví dụ: Luật De Morgan bằng tiếng Anh đời thường

"Không phải là tôi vừa đói VÀ vừa mệt" tương đương với

"Tôi KHÔNG đói HOẶC tôi KHÔNG mệt" (hoặc cả hai)

"Không phải là tôi đói HOẶC mệt" tương đương với

"Tôi KHÔNG đói VÀ tôi KHÔNG mệt"

E.5.5 Phép đảo điều kiện

Một trong những tương đương hữu ích nhất cho lập luận:

$$P \rightarrow Q \equiv \neg Q \rightarrow \neg P$$

"Nếu trời mưa thì mặt đất ướt" tương đương về logic với "Nếu mặt đất không ướt thì trời không mưa."

Phép đảo điều kiện thường dễ chứng minh hoặc kiểm tra hơn phép hàm ý ban đầu.

E.6 Các Lượng từ

Cho đến nay, các mệnh đề của ta đều là các phát biểu đơn giản. Nhưng thường ta muốn đưa ra khẳng định về *mọi* thứ thuộc một kiểu nào đó, hoặc về *sự tồn tại* của một thứ gì đó. Các lượng từ cho phép ta làm điều này.

E.6.1 Lượng từ phổ quát ($\forall$)

Lượng từ phổ quát $\forall$ nghĩa là "với mọi" hoặc "với tất cả".

$\forall x : P(x)$ nghĩa là " $P(x)$ đúng với mọi x ."

Ví dụ: Lượng từ phổ quát

$$\forall x \in \mathbb{R} : x^2 \geq 0$$

"Với mọi số thực x , x^2 không âm." (Đúng!)

$$\forall n \in \mathbb{N} : n + 1 > n$$

"Với mọi số nguyên n , $n + 1$ lớn hơn n ." (Đúng!)

$$\forall x \in \mathbb{R} : x > 0$$

"Mọi số thực đều dương." (Sai, còn -5 thì sao?)

E.6.2 Lượng từ tồn tại ($\exists$)

Lượng từ tồn tại $\exists$ nghĩa là “tồn tại” hoặc “có một số”.

$\exists x : P(x)$ nghĩa là “tồn tại ít nhất một x sao cho $P(x)$ đúng.”

Ví dụ: Lượng từ tồn tại

$$\exists x \in \mathbb{R} : x^2 = 2$$

“Tồn tại một số thực có bình phương bằng 2.” (Đúng, đó là $\sqrt{2}$)

$$\exists x \in \mathbb{R} : x^2 = -1$$

“Tồn tại một số thực có bình phương bằng -1 .” (Sai, không có số thực nào thỏa mãn)

$$\exists n \in \mathbb{Z} : n^2 = 2$$

“Tồn tại một số nguyên có bình phương bằng 2.” (Sai, $\sqrt{2}$ không phải là số nguyên)

E.6.3 Phủ định các Lượng từ

Phủ định một phát biểu phổ quát ta được một phát biểu tồn tại, và ngược lại:

$$\neg(\forall x : P(x)) \equiv \exists x : \neg P(x)$$

$$\neg(\exists x : P(x)) \equiv \forall x : \neg P(x)$$

Ví dụ: Phủ định các Lượng từ

Gốc: “Mọi con chim đều bay được.” ($\forall$ chim b : b bay được)

Phủ định: “Tồn tại một con chim không bay được.” ($\exists$ chim b : b không bay được)

(Và quả thật là chim cánh cụt và đà điểu đều tồn tại!)

Gốc: “Tồn tại một số nguyên có bình phương bằng 2.” ($\exists n \in \mathbb{Z} : n^2 = 2$)

Phủ định: “Với mọi số nguyên, bình phương của nó không bằng 2.”

($\forall n \in \mathbb{Z} : n^2 \neq 2$)

(Phủ định này đúng. Không số nguyên nào có bình phương bằng 2.)

E.7 Logic Bậc nhất

Logic mệnh đề rất mạnh nhưng bị giới hạn. Nó coi các mệnh đề là những đơn vị nguyên tử, ta không thể nhìn vào bên trong chúng hay lập luận về cấu trúc của chúng. **Logic Bậc nhất** (FOL), còn gọi là logic vị từ, mở rộng logic mệnh đề để lập luận về các đối tượng, thuộc tính của chúng, và các mối quan hệ giữa chúng.

E.7.1 Các Vị từ và Các Thuật ngữ

Một **vị từ** là một hàm nhận các đối tượng và trả về một giá trị chân lý. Thay vì các mệnh đề nguyên tử như P , ta có các biểu thức có cấu trúc như $\text{Mortal}(\text{Socrates})$ nghĩa là “Socrates là người”.

Các thuật ngữ biểu diễn các đối tượng:

- **Hằng số:** Các đối tượng cụ thể như Socrates, Athens, 5
- **Các biến:** Các ký hiệu giữ chỗ như x, y, z
- **Các hàm:** Các biểu thức tính ra đối tượng, như $\text{father}(x)$ (người cha của x)

Các vị từ diễn đạt thuộc tính hoặc mối quan hệ:

- $\text{Human}(x)$, “ x là người” (vị từ bậc nhất)
- $\text{Loves}(x, y)$, “ x yêu y ” (vị từ bậc hai)
- $\text{Between}(x, y, z)$, “ x nằm giữa y và z ” (vị từ bậc ba)

E.7.2 Các mệnh đề trong Logic Bậc nhất

Các mệnh đề FOL kết hợp các vị từ với các phép nối logic và các lượng từ:

Examples: First-Order Logic Sentences

“All humans are mortal”:

$$\forall x : \text{Human}(x) \rightarrow \text{Mortal}(x)$$

“Socrates is human”:

$$\text{Human}(\text{Socrates})$$

“Someone loves everyone”:

$$\exists x : \forall y : \text{Loves}(x, y)$$

“Everyone loves someone”:

$$\forall x : \exists y : \text{Loves}(x, y)$$

Hãy chú ý rằng hai ví dụ cuối có nghĩa khác nhau! “Có một ai đó yêu tất cả mọi người” nghĩa là tồn tại một người cụ thể yêu tất cả mọi người. “Mọi người đều yêu một ai đó” nghĩa là mỗi người có ít nhất một người họ yêu (có thể là người khác nhau tùy người). Thứ tự các lượng từ rất quan trọng.

E.7.3 Phạm vi của các Lượng từ

Phạm vi của một lượng từ là phần của công thức nơi biến bị lượng từ ràng buộc. Xét:

$$\forall x : (P(x) \wedge \exists y : Q(x, y))$$

Phạm vi của $\forall x$ là toàn bộ biểu thức $(P(x) \wedge \exists y : Q(x, y))$. Phạm vi của $\exists y$ chỉ là $Q(x, y)$.

Các biến có thể là **tự do** (không bị lượng từ nào ràng buộc) hoặc **bị ràng buộc**. Một mệnh đề không chứa biến tự do được gọi là một **công thức đóng** và có một giá trị chân lý xác định.

E.7.4 Suy luận trong Logic Bậc nhất

Ví dụ kinh điển về suy luận FOL là:

1. Mọi con người đều có thể chết: $\forall x : \text{Human}(x) \rightarrow \text{Mortal}(x)$
2. Socrates là người: $\text{Human}(\text{Socrates})$
3. Do đó, Socrates có thể chết: $\text{Mortal}(\text{Socrates})$

Điều này dùng **Cách thể hóa Phổ quát**: nếu $\forall x : P(x)$ đúng thì $P(c)$ đúng với mọi hằng c .

E.8 Phép hợp nhất

Phép hợp nhất là một thuật toán then chốt trong lập luận tự động, tìm các phép thế làm cho hai biểu thức trở nên giống hệt nhau. Nó rất cần thiết cho suy luận trong logic bậc nhất.

E.8.1 Bài toán Hợp nhất

Cho hai biểu thức, hãy tìm một **phép thế** (ánh xạ các biến sang các thuật ngữ) làm cho chúng giống hệt nhau.

Ví dụ: Hợp nhất**Ví dụ 1:** Hợp nhất $P(x, y)$ và $P(A, B)$ Phép thế: $\{x/A, y/B\}$ (thay x bằng A , y bằng B) Kết quả: Cả hai trở thành $P(A, B)$ ✓**Ví dụ 2:** Hợp nhất $P(x, x)$ và $P(A, B)$ Cần $x = A$ và $x = B$, nhưng $A \neq B$.
Kết quả: Không thể hợp nhất ✗**Ví dụ 3:** Hợp nhất $P(x, f(y))$ và $P(A, f(B))$ Phép thế: $\{x/A, y/B\}$ Kết quả: Cả hai trở thành $P(A, f(B))$ ✓**Ví dụ 4:** Hợp nhất $P(x, f(x))$ và $P(y, y)$ Cần $f(x) = y$ và $x = y$, do đó $f(x) = x$.Kết quả: Không thể hợp nhất (kiểm tra xảy ra, x không thể bằng $f(x)$) ✗**E.8.2 Bộ hợp nhất Tổng quát nhất (MGU)****Bộ hợp nhất tổng quát nhất** là phép thế đơn giản nhất hợp nhất được hai biểu thức. Bất kỳ bộ hợp nhất nào khác đều có thể thu được bằng cách thế thêm vào MGU.Với $P(x)$ và $P(y)$:

- $\{x/y\}$ là một MGU (cũng có thể dùng $\{y/x\}$)
- $\{x/A, y/A\}$ cũng là một bộ hợp nhất, nhưng cụ thể hơn MGU

E.8.3 Thuật toán Hợp nhất**Thuật toán Hợp nhất (rút gọn)****Hợp nhất(E_1, E_2):**

1. Nếu $E_1 = E_2$, trả về phép thế rỗng $\{\}$
2. Nếu E_1 là một biến x :
 - Nếu x xuất hiện trong E_2 , thất bại (kiểm tra xảy ra)
 - Nếu không, trả về $\{x/E_2\}$
3. Nếu E_2 là một biến, đổi chỗ và áp dụng bước 2
4. Nếu cả hai đều phức hợp (ví dụ $f(a_1, \dots, a_n)$ và $f(b_1, \dots, b_n)$):
 - Nếu các ký hiệu hàm khác nhau, thất bại
 - Hợp nhất đệ quy các tham số, ghép các phép thế 5. Nếu không, thất bại

E.8.4 Hợp nhất trong AI

Phép hợp nhất được dùng trong:

- **Chứng minh định lý bằng phân giải:** Tìm các mắt cút đối xứng để phân giải
- **Lập trình logic** (Prolog): Khớp các mục tiêu với các luật
- **Suy diễn kiểu:** Xác định kiểu trong các ngôn ngữ lập trình
- **Xử lý ngôn ngữ tự nhiên:** Phân tích cú pháp và ngữ nghĩa

E.9 Các Nghịch lý Logic

Các nghịch lý bộc lộ những vấn đề tinh tế trong lập luận logic và nhắc chúng ta phải cẩn thận với tham chiếu tự thân và việc hình thành tập hợp không bị giới hạn.

E.9.1 Nghịch lý Kẻ nói dối

Xét câu phát biểu: “Câu này là sai.”

Nếu nó đúng thì điều nó nói phải đúng, như vậy nó sai. Nhưng nếu nó sai thì điều nó nói không đúng, nên nó không sai, nghĩa là nó đúng. Dù thế nào chúng ta cũng gặp mâu thuẫn.

Nghịch lý này nảy sinh từ tham chiếu tự thân và cho thấy không phải mọi câu đúng ngữ pháp đều là một mệnh đề hợp lệ.

E.9.2 Nghịch lý Russell

Xét tập R gồm tất cả các tập không chứa chính nó: $R = \{S : S \notin S\}$.

Liệu R có chứa chính nó không?

- Nếu $R \in R$ thì theo định nghĩa của R , ta cần $R \notin R$. Mâu thuẫn.
- Nếu $R \notin R$ thì theo định nghĩa của R , ta cần $R \in R$. Mâu thuẫn.

Nghịch lý này đã thúc đẩy việc phát triển những nền tảng cẩn thận hơn cho lý thuyết tập hợp, nơi không phải mô tả nào cũng định nghĩa được một tập hợp hợp lệ.

E.9.3 Nghịch lý Người thợ cạo

Một người thợ cạo cạo râu cho tất cả những ai, và chỉ những ai, không tự cạo râu cho mình. Người thợ cạo có tự cạo râu cho mình không?

Đây chính là nghịch lý Russell được hóa trang. Cách giải quyết: không thể tồn tại người thợ cạo như vậy, vì mô tả đó tự mâu thuẫn.

E.10 Logic và Trí tuệ nhân tạo

E.10.1 Vai trò Lịch sử của Logic trong AI

Những hệ thống AI sớm nhất, phát triển trong thập niên 1950 và 1960, được xây dựng trên logic ký hiệu. Ý tưởng đó rất hấp dẫn: nếu chúng ta có thể biểu diễn tri thức dưới dạng các phát biểu logic và trao cho máy tính các quy tắc suy luận logic, thì chúng có thể lập luận như con người.

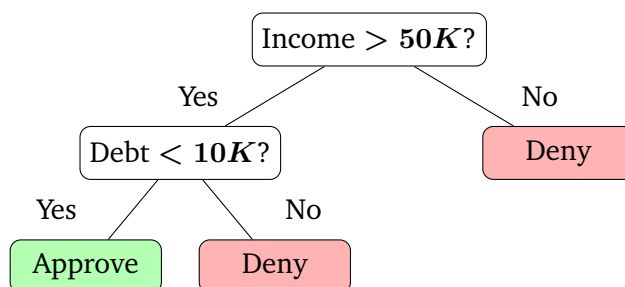
Các chương trình như General Problem Solver (1959) và các hệ chuyên gia (thập niên 1970–1980) dùng các quy tắc logic để giải quyết vấn đề. Một hệ thống chẩn đoán y khoa có thể chứa các luật như:

IF fever AND cough AND fatigue THEN possibly_flu
IF possibly_flu AND NOT vaccinated THEN recommend_test

Đây chính là các phép hàm ý logic được dịch thành mã!

E.10.2 Cây quyết định dưới dạng Công thức logic

Mọi cây quyết định đều có thể được viết thành một công thức logic. Xét một cây cho việc phê duyệt khoản vay:



Cây này tương ứng với công thức logic:

$$\text{Approve} \equiv (\text{Income} > 50K) \wedge (\text{Debt} < 10K)$$

Các cây phức tạp hơn tạo ra những công thức phức tạp hơn với các AND và OR lồng nhau, nhưng mọi cây quyết định đều tương đương với một biểu thức logic nào đó. Chương 8 phát triển mối liên hệ này thêm nữa: các bài toán thỏa mãn ràng buộc và các bộ giải SAT được thảo luận ở đó là những ứng dụng trực tiếp của logic mệnh đề và các biểu diễn CNF được trình bày trong phụ lục này.

E.10.3 Logic trong AI Hiện đại

Mặc dù học máy hiện đại thường dùng các phương pháp thống kê và nơ-ron thay vì logic tường minh, các khái niệm logic vẫn quan trọng:

Đồ thị tri thức và suy luận: Các hệ thống như những hệ thống phía sau công cụ tìm kiếm và hỏi đáp sử dụng suy luận logic trên tri thức có cấu trúc.

Ràng buộc và kiểm chứng: Khi cần kiểm chứng một hệ thống AI hoạt động đúng, chúng ta thường diễn đạt các yêu cầu dưới dạng công thức logic và chứng minh (hoặc kiểm tra) rằng hệ thống thỏa mãn chúng.

AI có thể diễn giải: Ngày càng có nhiều quan tâm đến các hệ thống AI có thể giải thích quyết định của chúng. Giải thích dựa trên luật về bản chất là các công thức logic: “Tôi phân loại cái này là spam vì nó chứa các liên kết đáng ngờ VÀ đến từ người gửi không quen biết VÀ dùng ngôn ngữ khẩn cấp.”

Các bộ giải SAT: Bài toán thỏa mãn Boole (SAT), xác định xem một công thức logic có thể được làm cho đúng hay không, là một bài toán nền tảng của khoa học máy tính. Các bộ giải SAT hiện đại cực kỳ mạnh và được dùng trong kiểm chứng phần cứng, kiểm thử phần mềm, và thậm chí cả trong một số hệ thống lập kế hoạch AI.

E.10.4 Giới hạn của Logic Tinh khiết

Các nhà nghiên cứu AI thời đầu hy vọng rằng chỉ riêng logic có thể đạt được trí tuệ ở mức con người. Giấc mơ đó gặp phải những trở ngại đáng kể:

Nút thắt thu nhận tri thức: Việc viết ra tất cả các quy tắc logic cần cho lập luận thông thường hóa ra gần như không thể. Làm sao viết được các quy tắc nắm bắt mọi thứ một đứa trẻ biết về cách thế giới vật lý vận hành?

Bất định: Thế giới thực là bất định, nhưng logic cổ điển chỉ xử lý các giá trị đúng/sai xác định. Điều này dẫn đến các mở rộng xác suất và cuối cùng là học máy thống kê.

Học: Các hệ thống dựa trên logic phải được con người lập trình sẵn các quy tắc. Chúng không thể học từ dữ liệu như cách các mạng nơ-ron có thể.

AI hiện đại kết hợp những điều sáng suốt từ logic (lập luận chính xác, tính kết hợp) với học thống kê (xử lý bất định, học từ dữ liệu). Những hệ thống mạnh nhất thường dùng cả hai.

E.11 Xây dựng Bảng chân lý

Với các biểu thức phức tạp, hãy xây dựng bảng chân lý một cách có hệ thống, tính các cột trung gian.

Example: Building a Truth Table

Verify that $(P \wedge Q) \rightarrow R \equiv P \rightarrow (Q \rightarrow R)$:

P	Q	R	$P \wedge Q$	$(P \wedge Q) \rightarrow R$	$Q \rightarrow R$	$P \rightarrow (Q \rightarrow R)$	Same?
T	T	T	T	T	T	T	✓
T	T	F	T	F	F	F	✓
T	F	T	F	T	T	T	✓
T	F	F	F	T	T	T	✓
F	T	T	F	T	T	T	✓
F	T	F	F	T	F	T	✓
F	F	T	F	T	T	T	✓
F	F	F	F	T	T	T	✓

The two main columns are identical, confirming the equivalence.

E.12 Dạng chuẩn Liên hợp (CNF)

Nhiều hệ thống lập luận tự động yêu cầu công thức ở một định dạng chuẩn gọi là **Dạng chuẩn Liên hợp** (CNF).

E.12.1 CNF là gì?

Một công thức ở dạng CNF nếu nó là một phép hợp (AND) của các vế, trong đó mỗi vế là một phép tuyển (OR) của các mắt xích. Một **mắt xích** là một biến hoặc phủ định của biến đó.

$$\text{CNF: } (A \vee B \vee \neg C) \wedge (\neg A \vee D) \wedge (B \vee C)$$

Ở đây ta có ba vế, được nối với nhau bằng AND. Mỗi vế chứa các mắt xích được nối với nhau bằng OR.

E.12.2 Chuyển đổi sang CNF

Bất kỳ công thức mệnh đề nào cũng có thể chuyển thành một công thức CNF tương đương:

Bước 1: Loại bỏ các phép hàm ý và phép tương đương.

- $P \rightarrow Q$ trở thành $\neg P \vee Q$
- $P \leftrightarrow Q$ trở thành $(P \rightarrow Q) \wedge (Q \rightarrow P)$, rồi áp dụng cách trên

Bước 2: Đẩy các phủ định vào trong bằng các luật De Morgan cho đến khi chúng chỉ áp dụng lên các biến.

- $\neg(P \wedge Q)$ trở thành $\neg P \vee \neg Q$
- $\neg(P \vee Q)$ trở thành $\neg P \wedge \neg Q$

- $\neg\neg P$ trở thành P

Bước 3: Phân phối OR trên AND để được một phép hợp của các phép tuyển.

- $P \vee (Q \wedge R)$ becomes $(P \vee Q) \wedge (P \vee R)$

Ví dụ: Chuyển đổi sang CNF

Chuyển $(P \rightarrow Q) \rightarrow R$ sang CNF. **Bước 1:** Loại bỏ các phép hàm ý.

$$(P \rightarrow Q) \rightarrow R = \neg(P \rightarrow Q) \vee R = \neg(\neg P \vee Q) \vee R$$

Bước 2: Đẩy phủ định vào trong.

$$\neg(\neg P \vee Q) \vee R = (P \wedge \neg Q) \vee R$$

Bước 3: Phân phối OR trên AND.

$$(P \wedge \neg Q) \vee R = (P \vee R) \wedge (\neg Q \vee R)$$

Kết quả: $(P \vee R) \wedge (\neg Q \vee R)$, hai vế ở dạng CNF.

E.13 Phép phân giải

Phép phân giải là một quy tắc suy luận mạnh để chứng minh các phát biểu logic. Nó hoạt động trên các công thức dạng CNF.

E.13.1 Quy tắc Phân giải

Nếu ta có hai vế, vế này chứa mắt xích L và vế kia chứa $\neg L$, ta có thể **phân giải** chúng để tạo một vế mới chứa tất cả mắt xích trừ L và $\neg L$:

$$\frac{(A \vee L) \quad (\neg L \vee B)}{(A \vee B)}$$

Các mắt xích L và $\neg L$ triệt tiêu lẫn nhau, và ta kết hợp các mắt xích còn lại.

Ví dụ: Phân giải

Ví dụ 1:

$$\frac{(P \vee Q) \quad (\neg Q \vee R)}{(P \vee R)}$$

Q và $\neg Q$ triệt tiêu; ta giữ P và R . **Ví**

dụ 2:

$$\frac{(\neg A \vee B \vee C) \quad (A \vee D)}{(B \vee C \vee D)}$$

A và $\neg A$ triệt tiêu; ta giữ B , C , và D . **Ví**

dụ 3:

$$\frac{(P) \quad (\neg P)}{()}$$

Tất cả đều triệt tiêu! Vế rỗng $()$ biểu diễn một mâu thuẫn (Sai).

E.13.2 Chứng minh bằng Bác bỏ

Để chứng minh một phát biểu ϕ suy ra từ các tiền đề $\psi_1, \psi_2, \dots$, ta dùng **chứng minh bằng bác bỏ**:

1. Chuyển tất cả tiền đề sang CNF
2. Thêm *phủ định* của điều ta muốn chứng minh (cũng ở dạng CNF)
3. Áp dụng phân giải lặp đi lặp lại
4. Nếu ta suy ra được vế rỗng (mâu thuẫn), phát biểu ban đầu được chứng minh

Lập luận: nếu giả thiết $\neg\phi$ cùng với các tiền đề dẫn đến mâu thuẫn thì ϕ phải đúng.

Ví dụ: Chứng minh bằng Phân giải

Chứng minh: Từ $P \rightarrow Q$ và $Q \rightarrow R$, ta suy ra $P \rightarrow R$.

Bước 1: Chuyển các tiền đề sang CNF.

- $P \rightarrow Q$ trở thành $\neg P \vee Q$
- $Q \rightarrow R$ trở thành $\neg Q \vee R$

Bước 2: Thêm phủ định của mục tiêu.

- Ta muốn $P \rightarrow R$, do đó thêm $\neg(P \rightarrow R) = \neg(\neg P \vee R) = P \wedge \neg R$
- Điều này cho hai vế: (P) và $(\neg R)$

Bước 3: Các vế của ta là:

1. $\neg P \vee Q$ (từ tiền đề 1)
2. $\neg Q \vee R$ (từ tiền đề 2)
3. P (từ mục tiêu đã bị phủ định)
4. $\neg R$ (từ mục tiêu đã bị phủ định)

Bước 4: Áp dụng phân giải.

- Phân giải (1) và (3): $\frac{(\neg P \vee Q) \quad (P)}{(Q)}$, thu được vế (5): Q
- Phân giải (2) và (5): $\frac{(\neg Q \vee R) \quad (Q)}{(R)}$, thu được vế (6): R
- Phân giải (4) và (6): $\frac{(\neg R) \quad (R)}{()}$, vế rỗng! Ta đã suy ra mâu thuẫn, do đó $P \rightarrow R$ được chứng minh.

E.14 SAT: Thỏa mãn Boole

Bài toán Thỏa mãn Boole (SAT) đặt câu hỏi: với một công thức dạng CNF, tồn tại cách gán giá trị chân lý cho các biến làm cho công thức Đúng hay không?

E.14.1 SAT như một bài Tìm kiếm

SAT có thể được xem là một bài toán tìm kiếm:

- **Trạng thái:** Một cách gán chân lý một phần cho các biến
- **Hành động:** Gán Đúng hoặc Sai cho một biến chưa được gán
- **Mục tiêu:** Một cách gán đầy đủ thỏa mãn tất cả các vế

Điều này gắn trực tiếp với khung CSP từ Chương 7! Các biến là các biến Boole, các miền là $\{\text{True}, \text{False}\}$, và các ràng buộc là các vế.

E.14.2 Vì sao SAT quan trọng

SAT là bài toán đầu tiên được chứng minh là NP-đầy đủ, nếu bạn giải được SAT hiệu quả, bạn có thể giải mọi bài toán trong NP một cách hiệu quả. Mặc dù độ khó

trường hợp xấu nhất đó, các bộ giải SAT hiện đại có thể xử lý các công thức với hàng triệu biến.

Các bộ giải SAT được dùng trong:

- **Kiểm chứng phần cứng:** Chứng minh mạch điện hoạt động đúng
- **Kiểm chứng phần mềm:** Tìm lỗi trong các chương trình
- **Lập kế hoạch AI:** Tìm chuỗi hành động để đạt mục tiêu
- **Phân tích mã:** Đánh vỡ các sơ đồ mã hóa nhất định

Thuật toán DPLL (và các mở rộng hiện đại của nó) kết hợp tìm kiếm lùi với lan truyền mệnh đề đơn vị (nếu một vế chỉ có một mắt xích chưa được gán thì mắt xích đó phải đúng) và loại bỏ mắt xích tinh khiết (nếu một biến chỉ xuất hiện với một cực tính, gán nó để thỏa mãn các vế đó).

E.15 Tóm tắt

Bảng tra nhanh về Logic

Các phép nối:

- NOT ($\neg$): Lật ngược giá trị chân lý
- AND ($\wedge$): Đúng khi cả hai cùng đúng
- OR ($\vee$): Đúng khi ít nhất một cái đúng
- Hàm ý ($\rightarrow$): Chỉ sai khi P đúng, Q sai
- Tương đương ($\leftrightarrow$): Đúng khi cả hai có cùng giá trị

Các Tương đương Quan trọng:

- $P \rightarrow Q \equiv \neg P \vee Q$
- $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ (De Morgan)
- $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ (De Morgan)
- $P \rightarrow Q \equiv \neg Q \rightarrow \neg P$ (Phản đề)

CNF: Phép hợp của các phép tuyển: $(A \vee B) \wedge (C \vee \neg D) \wedge \dots$

Phân giải: Từ $(A \vee L)$ và $(\neg L \vee B)$, suy ra $(A \vee B)$. **SAT:** Tìm các cách gán chân lý thỏa mãn một công thức CNF. Gắn với tìm kiếm và CSP.

Các Lượng từ:

- $\forall$ (với mọi): $\neg(\forall x : P(x)) \equiv \exists x : \neg P(x)$
- $\exists$ (tồn tại): $\neg(\exists x : P(x)) \equiv \forall x : \neg P(x)$

Logic Bậc nhất:

- Các vị từ diễn đạt thuộc tính và quan hệ: $\text{Human}(x)$, $\text{Loves}(x, y)$
- Các thuật ngữ: hằng, biến và hàm
- Thứ tự các lượng từ rất quan trọng: $\forall x \exists y$ khác $\exists y \forall x$

hợp nhất: Tìm các phép thế làm cho hai biểu thức giống hệt nhau.

MGU (Bộ hợp nhất tổng quát nhất) là phép thế đơn giản nhất như vậy.

Tương ứng Logic-Tập hợp: AND $\leftrightarrow$ Giao, OR $\leftrightarrow$ Hợp, NOT $\leftrightarrow$ Bù

Liên hệ với AI: Cây quyết định là các công thức logic; các hệ chuyên gia dùng các quy tắc logic; AI hiện đại kết hợp logic với học thống kê.

Phụ lục F

Đại số tuyến tính

Việc đem những con số đặt vào
vị trí toạ độ là một hành vi bạo
lực.

Hermann Weyl

F.1 Giới thiệu: Vì sao cần Đại số tuyến tính?

Học máy là việc tìm ra quy luật trong dữ liệu, và dữ liệu hầu như luôn đến theo từng tập hợp. Một bệnh nhân không được mô tả bằng một con số duy nhất mà bằng hàng chục phép đo: huyết áp, nhịp tim, nồng độ cholesterol, tuổi, cân nặng. Một bức ảnh không phải là một giá trị duy nhất mà là hàng triệu mức sáng điểm ảnh. Một tài liệu được đặc trưng bởi tần suất xuất hiện của hàng nghìn từ ngữ khác nhau.

Để làm việc hiệu quả với những tập hợp như vậy, chúng ta cần một khung toán học coi tập hợp các con số như một đối tượng duy nhất mà ta có thể thao tác. Đó chính là điều mà **đại số tuyến tính** mang lại.

Đại số tuyến tính cho chúng ta ba thứ. Thứ nhất, **ký hiệu gọn**: thay vì viết $x_1, x_2, x_3, \dots, x_{1000}$, ta chỉ cần viết một ký hiệu duy nhất x . Thứ hai, **tính toán hiệu quả**: máy tính có thể nhân ma trận nhanh hơn hàng tỷ lần so với các vòng lặp lồng nhau tương đương. Thứ ba, **trực giác hình học**: ta có thể hình dung dữ liệu như những điểm trong không gian, các phép biến đổi như những phép xoay và kéo giãn, và việc học như tìm ra phép biến đổi phù hợp.

Gần như mọi thuật toán học máy (từ hồi quy tuyến tính, mạng nơ-ron cho tới phân tích thành phần chính (PCA)) đều được diễn đạt và cài đặt bằng đại số tuyến tính. Phụ lục này sẽ xây dựng hiểu biết của bạn từ nền móng.

F.2 Vô hướng: những con số đơn lẻ

Một **số vô hướng** đơn giản chỉ là một con số. Chúng ta dùng các chữ thường như a , b , x , y để ký hiệu các vô hướng.

Ví dụ về vô hướng: nhiệt độ hôm nay (22°C), điểm thi của một học sinh (85), tốc độ học của một mô hình ($\alpha = 0.01$), giá của một căn nhà (C450 000).

Vô hướng là những nguyên tử để xây dựng mọi thứ khác. Khi chúng ta chuyển sang vector và ma trận, thực chất chúng ta chỉ đang sắp xếp các tập hợp vô hướng theo những cách hữu ích mà thôi.

F.3 Vector: danh sách số có thứ tự

F.3.1 Vector là gì?

Một **vector** là một danh sách các con số có thứ tự. Chúng ta dùng các chữ thường in đậm như $\mathbf{x}$, $\mathbf{y}$, $\mathbf{w}$ để ký hiệu vector, và thường viết chúng dưới dạng cột dọc:

$$\mathbf{x} = \begin{pmatrix} 3 \\ 7 \\ 2 \end{pmatrix}$$

Vector này có ba **thành phần**: thành phần thứ nhất là $x_1 = 3$, thành phần thứ hai là $x_2 = 7$, và thành phần thứ ba là $x_3 = 2$. **Chiều** của một vector là số thành phần mà nó có; vector này có chiều 3.

F.3.2 Vector trong thế giới thực

Vector tự nhiên dùng để mô tả những thứ có nhiều thuộc tính:

Ví dụ: Một bệnh nhân như một vector

Một bệnh nhân có thể được mô tả bởi:

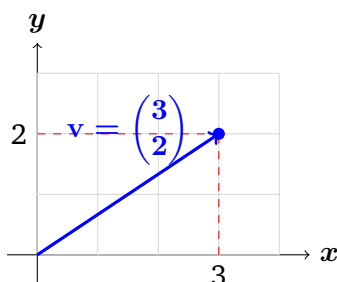
$$\mathbf{x}_{\text{patient}} = \begin{pmatrix} \text{age} \\ \text{weight (kg)} \\ \text{blood pressure} \\ \text{cholesterol} \end{pmatrix} = \begin{pmatrix} 45 \\ 72 \\ 130 \\ 195 \end{pmatrix}$$

Mỗi bệnh nhân trở thành một vector duy nhất. Cơ sở dữ liệu của một bệnh viện với 10,000 bệnh nhân trở thành 10,000 vector.

Trong học máy, chúng ta gọi vector mô tả một điểm dữ liệu đơn lẻ là **vector đặc trưng**; vector đó chứa toàn bộ các đặc trưng (thuộc tính) của mẫu dữ liệu ấy.

F.3.3 Góc nhìn hình học: vector như những mũi tên

Trong mặt phẳng 2 chiều hoặc không gian 3 chiều, ta có thể hình dung vector như những mũi tên. Vector $\mathbf{v} = \begin{pmatrix} 3 \\ 2 \end{pmatrix}$ là một mũi tên chỉ 3 đơn vị theo hướng x và 2 đơn vị theo hướng y .



Hình F.1: Một vector 2 chiều được hình dung như một mũi tên đi từ gốc tọa độ đến điểm (3, 2).

F.3.4 Cộng vector và nhân với vô hướng

Để cộng hai vector, hãy cộng các thành phần tương ứng:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 5 \\ 7 \\ 9 \end{pmatrix}$$

Để nhân một vector với một vô hướng, hãy nhân từng thành phần:

$$3 \cdot \begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix} = \begin{pmatrix} 6 \\ 12 \\ 3 \end{pmatrix}$$

Quan trọng: Vô hướng nhân với vector cho ra một **vector** (cùng chiều với vector ban đầu).

F.4 Tích vô hướng: nhân hai vector với nhau

F.4.1 Định nghĩa

Tích vô hướng lấy hai vector cùng chiều và cho ra một con số duy nhất, tức là một vô hướng.

$$\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^n x_i y_i = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n$$

Ví dụ: Tính một tích vô hướng

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} \cdot \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix} = (1 \times 4) + (2 \times 5) + (3 \times 6) = 4 + 10 + 18 = 32$$

Hai vector vào, một vô hướng ra.

Điểm mấu chốt: Tích vô hướng của hai vector là một **vô hướng** (một con số), không phải là vector.

F.4.2 Vì sao tích vô hướng quan trọng

Ví dụ thực tế: Hóa đơn mua sắm. Giả sử giá là $\mathbf{p} = \begin{pmatrix} 2.50 \\ 1.00 \\ 3.00 \end{pmatrix}$ và

số lượng là $\mathbf{q} = \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix}$. Tổng tiền của bạn là:

$$\mathbf{p} \cdot \mathbf{q} = 2.50 \times 2 + 1.00 \times 3 + 3.00 \times 1 = \text{€}11.00$$

Học máy: Dự đoán tuyến tính. Một mô hình tuyến tính tính $\mathbf{w} \cdot \mathbf{x}$, trong đó $\mathbf{w}$ chứa các trọng số đã học được và $\mathbf{x}$ chứa các đặc trưng. Đây là nền tảng của hồi quy tuyến tính, hồi quy logistic, và của từng nơ-ron trong mạng nơ-ron.

F.4.3 Ý nghĩa hình học

Tích vô hướng liên quan tới góc θ giữa hai vector: $\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$. Hai vector cùng hướng cho tích vô hướng dương lớn; hai vector vuông góc cho tích vô hướng bằng không; hai vector ngược hướng cho tích vô hướng âm.

F.5 Độ dài vector (norm)

Norm của một vector, ký hiệu $\|\mathbf{x}\|$, chính là độ dài của vector đó:

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\mathbf{x} \cdot \mathbf{x}}$$

Trong mặt phẳng 2 chiều, đây chính là định lý Pythagore:

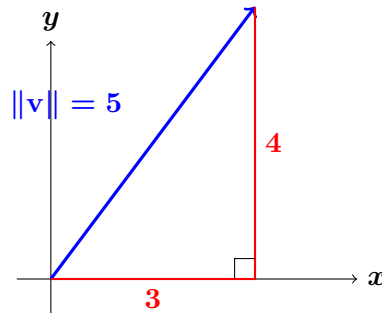


Figure F.2: For $\mathbf{v} = (3, 4)$: $\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$.

F.6 Matrices: Tables of Numbers

F.6.1 What is a Matrix?

A **matrix** is a rectangular table of numbers. We use bold uppercase letters like **A**, **W**:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$$

This is a “ 2×3 matrix” (2 rows, 3 columns). Element A_{ij} is in row i , column j : here $A_{12} = 2$.

In machine learning, a data matrix often has rows as examples and columns as features.

F.6.2 Special Matrices

Identity matrix $\mathbf{I}$: 1s on diagonal, 0s elsewhere. It is the matrix equivalent of 1.

$$\mathbf{I}_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Transpose $\mathbf{A}^T$: Swap rows and columns.

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

F.7 Phép toán trên ma trận: xây dựng từng bước một

F.7.1 Vô hướng nhân ma trận

Nhân mọi phần tử với vô hướng:

$$3 \cdot \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 9 & 12 \end{pmatrix}$$

Kết quả: Một ma trận có cùng kích thước.

F.7.2 Ma trận nhân vector

Hãy hình dung rằng ta lấy tích vô hướng của từng hàng với vector đó:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \begin{pmatrix} 7 \\ 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 7 + 2 \cdot 8 \\ 3 \cdot 7 + 4 \cdot 8 \\ 5 \cdot 7 + 6 \cdot 8 \end{pmatrix} = \begin{pmatrix} 23 \\ 53 \\ 83 \end{pmatrix}$$

Ma trận (3×2) nhân vector (2×1) cho ra vector (3×1) .

Đây là cách chúng ta áp dụng một mô hình tuyến tính: các dự đoán $= \mathbf{W}\mathbf{x}$.

F.7.3 Ma trận nhân ma trận

Mỗi phần tử C_{ij} là tích vô hướng của hàng i của $\mathbf{A}$ với cột j của $\mathbf{B}$.

Ví dụ: Phép nhân ma trận

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

F.7.4 Quy tắc về chiều

Để nhân $\mathbf{A}$ (kích thước $m \times n$) với $\mathbf{B}$ (kích thước $n \times p$): hai chiều ở bên trong phải khớp nhau. Kết quả là $m \times p$.

$$\underbrace{\mathbf{A}}_{m \times n} \times \underbrace{\mathbf{B}}_{n \times p} = \underbrace{\mathbf{C}}_{m \times p}$$

← must match! →

F.7.5 Tính giao hoán: thứ tự rất quan trọng!

Với các vô hướng, $3 \times 5 = 5 \times 3$. Tính chất này được gọi là **tính giao hoán**.

Phép nhân ma trận **KHÔNG giao hoán**: nhìn chung $AB \neq BA$.

Ví dụ: Thứ tự là quan trọng

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$$AB = \begin{pmatrix} 2 & 1 \\ 1 & 0 \end{pmatrix}, \quad BA = \begin{pmatrix} 0 & 1 \\ 1 & 2 \end{pmatrix}$$

Kết quả khác nhau! Đôi khi AB tồn tại nhưng BA thì không.

F.8 Phép toán nào cho ra kết quả gì?

Phép toán	Đầu vào	Đầu ra
Vô hướng $\times$ Vô hướng	hai con số	Vô hướng
Vô hướng $\times$ Vector	số, vector	Vector
Tích vô hướng	hai vector	Vô hướng
Độ dài vector	một vector	Vô hướng
Vô hướng $\times$ Ma trận	số, ma trận	Ma trận
Ma trận $\times$ Vector	$(m \times n), (n \times 1)$	Vector $(m \times 1)$
Ma trận $\times$ Ma trận	$(m \times n), (n \times p)$	Ma trận $(m \times p)$

F.9 Ma trận nghịch đảo

Nghịch đảo của ma trận vuông A , ký hiệu A^{-1} , thỏa mãn:

$$AA^{-1} = A^{-1}A = I$$

Không phải ma trận nào cũng có nghịch đảo. Ma trận không có nghịch đảo được gọi là ma trận **suy biến**.

For 2×2 : if $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, thì $A^{-1} = \frac{1}{ad-bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$.

Phần $ad - bc$ chính là **định thức**. Nếu nó bằng không thì nghịch đảo không tồn tại.

F.10 Giá trị riêng và vector riêng

F.10.1 Ma trận làm gì với vector?

Trước khi hiểu giá trị riêng, hãy cùng nghĩ xem thực chất ma trận *làm gì*. Khi bạn nhân một ma trận với một vector, bạn thu được một vector mới. Ma trận biến vector ban đầu thành một vector khác.

Hãy thử nhân nhiều vector khác nhau với ma trận $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$:

$$A \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad A \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad A \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Hãy chú ý một điều thú vị trong ví dụ thứ ba: vector $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$ cho ra vẫn không đổi! Ma trận hoàn toàn không xoay nó cũng không đổi hướng của nó.

Hãy thử thêm một ví dụ nữa:

$$A \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 3 \end{pmatrix} = 3 \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Ở đây vector $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ cho ra vẫn trở theo cùng hướng, chỉ dài hơn 3 lần mà thôi!

Những vector đặc biệt này (những vector không bị xoay, chỉ bị kéo giãn) được gọi là **vector riêng**.

F.10.2 Định nghĩa

Vector v là **vector riêng** của ma trận A nếu khi nhân với A nó chỉ bị co giãn, không đổi hướng:

$$Av = \lambda v$$

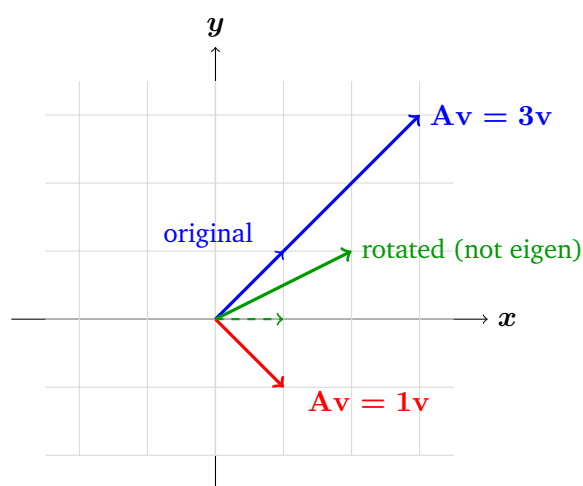
Hệ số co giãn λ (lambda) được gọi là **giá trị riêng**. Nó cho biết vector riêng được kéo giãn bao nhiêu lần:

- $\lambda > 1$: vector được kéo dài hơn
- $\lambda = 1$: vector giữ nguyên độ dài
- $0 < \lambda < 1$: vector bị thu ngắn lại
- $\lambda < 0$: vector bị lật ngược (trở theo hướng ngược lại) và bị co giãn

Trong ví dụ của chúng ta ở trên:

- $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$ là vector riêng với giá trị riêng $\lambda = 1$ (không thay đổi)
- $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$ là vector riêng với giá trị riêng $\lambda = 3$ (được kéo giãn 3 lần)

F.10.3 Hình dung các vector riêng



Hình F.3: Các vector riêng (màu xanh dương và màu đỏ) chỉ bị co giãn, chúng vẫn nằm trên cùng một đường thẳng đi qua gốc tọa độ. Các vector không phải vector riêng (màu xanh lá) bị xoay để trở theo hướng khác.

Hầu hết vector đều vừa bị xoay vừa bị co giãn khi nhân với một ma trận. Các vector riêng đặc biệt ở chỗ chúng *chỉ* bị co giãn. Chúng vẫn trở theo cùng một hướng (hoặc đúng hướng ngược lại nếu $\lambda < 0$).

F.10.4 Vì sao bạn cần quan tâm?

Vector riêng và giá trị riêng tiết lộ “các hướng tự nhiên” của một ma trận — những hướng mà ma trận tác động theo cách đơn giản nhất. Hóa ra điều này cực kỳ hữu ích.

Hiểu các phép biến đổi phức tạp: Bất kỳ phép biến đổi ma trận nào, dù phức tạp đến đâu, cũng tác động một cách đơn giản dọc theo các hướng của vector riêng: chỉ co giãn, không xoay. Chính vì thế mà vector riêng trở thành “hệ tọa độ” tự nhiên để hiểu một ma trận làm gì.

Phân tích thành phần chính (PCA): Khi bạn có dữ liệu nhiều chiều (chẳng hạn ảnh với hàng triệu điểm ảnh), bạn muốn tìm ra những hướng

quan trọng nhất — nơi dữ liệu biến thiên nhiều nhất. Hóa ra chúng chính là các vector riêng của ma trận hiệp phương sai của dữ liệu.

Hãy tưởng tượng các điểm dữ liệu phân tán trong không gian 3 chiều, tạo thành một đám mây kéo dài (giống hình quả bóng bầu dục). Các vector riêng trở dọc theo trục dài, trục trung bình và trục ngắn của đám mây này. Các giá trị riêng cho biết dữ liệu phân tán nhiều đến mức nào dọc theo từng trục. Vector riêng có giá trị riêng lớn nhất trở theo hướng phân tán lớn nhất. Đó chính là hướng “quan trọng nhất” trong dữ liệu của bạn.

PageRank của Google: Mức độ quan trọng của các trang web có thể được tính bằng vector riêng. Nếu bạn dựng một ma trận mô tả trang nào liên kết tới trang nào, vector riêng cho bạn biết xác suất lâu dài mà một người lướt web ngẫu nhiên đang ở mỗi trang. Những trang có giá trị vector riêng lớn là những trang “quan trọng”.

Ổn định của hệ thống: Nếu bạn lặp đi lặp lại việc nhân một vector với một ma trận (như $AA^T v$), điều gì xảy ra phụ thuộc vào các giá trị riêng:

- Nếu tất cả $|\lambda| < 1$: vector thu về không (Ổn định, tín hiệu tắt dần)
- Nếu có bất kỳ $|\lambda| > 1$: vector phình ra không giới hạn (không Ổn định, tín hiệu bùng nổ)
- Nếu $|\lambda| = 1$: vector bị chặn (Ổn định ở mức biên)

Điều này rất quan trọng khi hiểu mạng nơ-ron hồi quy, nơi cùng một ma trận trọng số được áp dụng lặp đi lặp lại.

F.10.5 Tìm giá trị riêng (Tùy chọn)

Trong học máy, bạn không cần tự tay tính giá trị riêng, máy tính sẽ làm việc đó. Nhưng đây là ý tưởng: chúng ta muốn tìm các vector v sao cho $Av = \lambda v$, điều này có thể viết lại thành:

$$(A - \lambda I)v = 0$$

Để phương trình này có nghiệm khác không, ma trận $(A - \lambda I)$ phải suy biến (không có nghịch đảo), tức là:

$$\det(A - \lambda I) = 0$$

Đây được gọi là **phương trình đặc trưng**. Với ma trận $n \times n$, đây là một đa thức bậc n theo λ , nên có nhiều nhất n giá trị riêng.

Với ma trận ví dụ $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$:

$$\det \begin{pmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{pmatrix} = (2 - \lambda)^2 - 1 = \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3) = 0$$

Vậy $\lambda = 1$ và $\lambda = 3$, khớp với những gì chúng ta tìm được trước đó bằng cách thử các vector.

F.11 Tóm tắt

Tham chiếu nhanh về Đại số tuyến tính

Vô hướng: Một con số duy nhất.

Vector: Danh sách các số có thứ tự. Tích vô hướng trả về một **vô hướng**. **Norm:** $\|\mathbf{x}\| = \sqrt{\mathbf{x} \cdot \mathbf{x}}$ (độ dài vector).

Ma trận: Bảng các con số, kích thước $m \times n$.

Phép nhân ma trận: $(m \times n) \cdot (n \times p) = (m \times p)$. **Không giao hoán!**

Nghịch đảo: $\mathbf{A}^{-1}$ thỏa mãn $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$. Không phải ma trận nào cũng có. **Vector riêng:** $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Những vector đặc biệt chỉ bị ma trận co giãn (không bị xoay). Giá trị riêng λ là hệ số co giãn. Được dùng trong PCA, PageRank và trong việc phân tích ổn định của hệ thống.

Phụ lục G

Độ phức tạp tính toán trong học máy

Về lý thuyết, không có sự khác biệt giữa lý thuyết và thực hành. Về thực hành, thì có.

Thường được quy cho Yogi Berra

Vì sao độ phức tạp lại quan trọng đối với IAIO

Các bài thi IAIO thường yêu cầu bạn phân tích hoặc so sánh các thuật toán dựa trên yêu cầu tính toán của chúng. Hiểu ký hiệu Big-O cùng độ phức tạp của các thuật toán học máy phổ biến là điều thiết yếu.

G.1 Ôn lại ký hiệu Big-O

Ký hiệu Big-O mô tả thời gian chạy hoặc mức dùng bộ nhớ của một thuật toán tăng lên như thế nào theo kích thước đầu vào.

Common Complexity Classes		
Notation	Name	Example
$O(1)$	Constant	Array access by index
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Single pass through data
$O(n \log n)$	Linearithmic	Efficient sorting (mergesort)
$O(n^2)$	Quadratic	Comparing all pairs
$O(n^3)$	Cubic	Matrix multiplication (naive)
$O(2^n)$	Exponential	Brute-force subset search

Key intuition: For $n = 1,000,000$:

- $O(n)$: 1 million operations $\approx$ instant
- $O(n \log n)$: 20 million operations $\approx$ instant
- $O(n^2)$: 1 trillion operations $\approx$ hours
- $O(2^n)$: more operations than atoms in the universe

G.2 Complexity of Machine Learning Algorithms

Let n = number of training examples, d = number of features, k = number of clusters/neighbors/classes.

G.2.1 Supervised Learning

Algorithm	Training Time	Prediction Time
k -Nearest Neighbors	$O(1)$ (just store data)	$O(nd)$ per query
Linear Regression (closed-form)	$O(nd^2 + d^3)$	$O(d)$
Linear Regression (gradient descent)	$O(tnd)$ for t iterations	$O(d)$
Logistic Regression	$O(tnd)$ for t iterations	$O(d)$
Decision Tree	$O(nd \log n)$	$O(\log n)$
Naïve Bayes	$O(nd)$	$O(d \cdot k)$
SVM (kernel)	$O(n^2d)$ to $O(n^3)$	$O(n_{sv} \cdot d)$

G.2.2 Học không giám sát

Thuật toán	Độ phức tạp thời gian
k -Means (một vòng lặp)	$O(nkd)$
k -Means (đầy đủ, t vòng lặp)	$O(tnkd)$
Phân cụm phân cấp (naive)	$O(n^3)$
Phân cụm phân cấp (tối ưu)	$O(n^2 \log n)$
PCA (SVD đầy đủ)	$O(\min(n^2d, nd^2))$
PCA (hàng đầu k thành phần)	$O(ndk)$

G.3 Đánh đổi giữa thời gian và bộ nhớ

Một nguyên tắc nền tảng trong thiết kế thuật toán là đánh đổi thời gian lấy bộ nhớ, hoặc ngược lại.

Đánh đổi kinh điển: k -NN và các mô hình tham số

k -Láng giềng gần nhất:

- Huấn luyện: $O(1)$ thời gian, $O(nd)$ dung lượng (lưu toàn bộ dữ liệu)
- Dự đoán: $O(nd)$ thời gian mỗi truy vấn
- Phải giữ toàn bộ tập huấn luyện trong bộ nhớ

Hồi quy logistic:

- Huấn luyện: $O(tnd)$ thời gian, $O(d)$ dung lượng
- Dự đoán: $O(d)$ thời gian mỗi truy vấn
- Chỉ lưu d tham số, có thể bỏ dữ liệu huấn luyện

Đánh đổi: k -NN có quá trình huấn luyện “miễn phí” nhưng dự đoán và lưu trữ lại tốn kém. Hồi quy logistic đầu tư vào huấn luyện để đổi lấy dự đoán rẻ và lưu trữ tối thiểu.

G.3.1 Các đánh đổi thường gặp khác

Tính trước và thời gian truy vấn:

- Tính trước mọi khoảng cách theo cặp: $O(n^2)$ dung lượng, $O(1)$ tra cứu
- Tính khoảng cách khi cần đến: $O(1)$ dung lượng, $O(n)$ mỗi truy vấn

Chính xác và xấp xỉ:

- Láng giềng gần nhất chính xác: $O(n)$ mỗi truy vấn
- Xấp xỉ (băm nhảy cảm cục bộ): $O(1)$ mỗi truy vấn, nhưng có thể bỏ sót láng giềng gần nhất thật

Theo lô và trực tuyến:

- Rơi gradient theo lô: xem tất cả n ví dụ ở mỗi lần cập nhật, ổn định nhưng chậm

- Rơi gradient ngẫu nhiên: một ví dụ mỗi lần cập nhật, nhiều nhiễu nhưng nhanh
- Mini-batch: sự thỏa hiệp giữa hai cách trên

G.4 Các bài hỏi về độ phức tạp theo kiểu IAIO

1. **So sánh:** Bạn có $n = 10^6$ ví dụ huấn luyện và $d = 100$ đặc trưng. Cái nào nhanh hơn khi thực hiện 1000 lần dự đoán: k-NN hay hồi quy logistic (giả sử LR đã được huấn luyện)?

Đáp án: k-NN cần $1000 \times 10^6 \times 100 = 10^{11}$ phép toán. LR cần $1000 \times 100 = 10^5$ phép toán. LR nhanh hơn 10^6 lần.

2. **Quy mô:** Một cây quyết định mất 1 giây để huấn luyện trên 1000 ví dụ. Xấp xỉ sẽ mất bao lâu trên 1 triệu ví dụ?

Đáp án: Huấn luyện có độ phức tạp $O(nd \log n)$. $\frac{10^6 \log 10^6}{10^3 \log 10^3} = \frac{10^6 \times 20}{10^3 \times 10} = 2000$. Khoảng 2000 giây ≈ 33 phút.

3. **Xác định nút thắt cổ chai:** PCA trên một tập dữ liệu có $n = 1000$, $d = 10,000$. Điều gì chiếm ưu thế trong phép tính?

Đáp án: PCA có độ phức tạp $O(\min(n^2d, nd^2))$. Ở đây $n^2d = 10^{10}$ và $nd^2 = 10^{11}$. Số hạng n^2d chiếm ưu thế, nên phép tính thay đổi theo n^2 , chứ không phải d^2 .

G.5 Tóm tắt

Điểm cần nhớ

Biết độ phức tạp của từng thuật toán: Hãy ghi nhớ độ phức tạp khi huấn luyện và khi dự đoán của các thuật toán chính.

Xác định nút thắt cổ chai: Liệu n hay d lớn hơn? Số hạng nào chiếm ưu thế? **Hiểu các đánh đổi:** Huấn luyện nhanh thường đi kèm dự đoán chậm (k-NN). Huấn luyện chậm thường đi kèm dự đoán nhanh (mạng nơ-ron). **Cân nhắc bộ nhớ:** Có thuật toán đòi lưu toàn bộ dữ liệu huấn luyện; có thuật toán khác nén nó lại thành các tham số.

Hãy xấp xỉ khi cần thiết: Với n rất lớn, thuật toán chính xác có thể không khả thi. Hãy biết khi nào phép xấp xỉ thực sự hữu ích.

G.6 Bài tập luyện

1. Bạn cần phân loại 1 triệu ví dụ mới. Bạn có một cây quyết định đã huấn luyện (độ sâu 20) và một mô hình k-NN đã huấn luyện ($k = 5$, 100K điểm huấn luyện, 50 đặc trưng). Cái nào nhanh hơn? Nhanh hơn bao nhiêu?

2. Một tập dữ liệu có $n = 10,000$ mẫu và $d = 5$ đặc trưng. Bạn chạy k-means với $k = 10$ trong 20 vòng lặp. Có bao nhiêu phép tính khoảng cách được thực hiện?
3. Hãy giải thích vì sao phân cụm phân cấp hiếm khi được dùng trên các tập dữ liệu có hơn 10.000 điểm.
4. Bạn có 1GB RAM. Dữ liệu huấn luyện của bạn gồm 100.000 ví dụ $\times$ 1.000 đặc trưng (float32). Bạn có thể dùng k-NN không? Bạn có thể dùng hồi quy logistic không? Hãy lập luận.
5. Với một SVM hạt nhân (kernel) có n điểm huấn luyện và n_{sv} vector hỗ trợ, hãy giải thích vì sao thời gian dự đoán phụ thuộc vào n_{sv} , chứ không phải n .

Phụ lục H

Đáp án bài tập luyện

Phụ lục này cung cấp đáp án cho các bài tập luyện của từng chương. Đáp án làm nổi bật ý chính của mỗi bài toán. Với các bài toán tính toán, các bước trung gian được trình bày; với các bài toán khái niệm, lập luận cốt lõi được nêu một cách cô đọng.

Chương 1: Giới thiệu về Trí tuệ nhân tạo

1.

- (a) Bộ lọc thư rác: AI hẹp (một tác vụ duy nhất: phân loại email)
- (b) Một hệ thống có thể học bất kỳ nhiệm vụ trí tuệ nào mà con người làm được: AGI (trí thông minh tổng quát trên mọi lĩnh vực)
- (c) Bộ máy cờ vua: AI hẹp (xuất sắc ở một môn cờ, không làm được gì khác)
- (d) Một AI giả định vượt mọi năng lực nhận thức của con người: ASI (e) Xe tự lái: AI hẹp (nhiệm vụ cụ thể, dù phức tạp) (f) Trợ lý ảo trả lời câu hỏi: AI hẹp (hiện nay chỉ tồn tại AI hẹp)

2. Một ảnh màu 640×480 có $640 \times 480 = 307,200$ điểm ảnh. Mỗi điểm ảnh có 3 kênh màu (Đỏ, Xanh lá, Xanh dương), vì vậy máy tính lưu $307,200 \times 3 = 921,600$ con số.

3. Nếu cùng một dữ liệu được dùng cho cả huấn luyện lẫn đánh giá, mô hình có thể ghi nhớ các ví dụ huấn luyện và có vẻ hoạt động hoàn hảo, nhưng điều này không cho chúng ta biết gì về cách nó xử lý dữ liệu mới, chưa từng thấy. Giữ riêng dữ liệu kiểm tra giúp đưa ra một ước lượng trung thực về hiệu năng khái quát hóa.

4. Đây là quá khớp. Mô hình đã ghi nhớ dữ liệu huấn luyện (độ chính xác 99%) nhưng không khái quát hóa được sang các ví dụ mới (60%). Khoảng cách giữa hiệu năng huấn luyện và kiểm tra chính là dấu hiệu đặc trưng của quá khớp. Các biện pháp khắc phục bao gồm dùng mô hình đơn giản hơn, thêm regularization, hoặc thu thập thêm dữ liệu huấn luyện.

5. Phân loại gắn một hình ảnh vào một danh mục (ví dụ “mèo” hoặc “chó”). Định vị xác định vị trí của một đối tượng duy nhất trong hình ảnh bằng cách vẽ một hộp khoanh quanh nó. Phát hiện đối tượng kết hợp cả hai: tìm mọi đối tượng trong hình ảnh, phân loại từng đối tượng, và vẽ hộp khoanh quanh từng cái. Phát hiện là khó nhất vì phải xử lý nhiều đối tượng thuộc các lớp khác nhau cùng lúc.

6. Hệ thống có thể đã học được các mối tương quan giữa giới tính và kết quả tuyển dụng từ dữ liệu lịch sử phản ánh sự phân biệt đối xử trong quá khứ. Ngay cả khi giới tính không phải là đầu vào tường minh, các đặc trưng thay thế (tên, trường đại học, hoạt động ngoại khóa) vẫn có thể mã hóa thông tin giới tính. Đây là ví dụ về cách AI có thể kế thừa và khuếch đại các thành kiến xã hội sẵn có thông qua dữ liệu huấn luyện có thiên kiến.

Chương 2: Tác động xã hội của AI

1. Tác động phân biệt đối xử (disparate impact) xảy ra khi một chính sách có vẻ trung lập gây hại không cân xứng cho một nhóm được bảo vệ. Về toán học, quy tắc 80% phát biểu rằng có disparate impact nếu $\frac{\min(p_A, p_B)}{\max(p_A, p_B)} < 0.8$. Nếu Nhóm A có tỷ lệ chấp thuận 60% và Nhóm B có 40%, tỷ số là $0.4/0.6 = 0.67 < 0.8$, cho thấy có disparate impact.

2. Ngang bằng nhân khẩu học (demographic parity) đòi hỏi tỷ lệ dự đoán dương bằng nhau giữa các nhóm:

$P(Y = 1|G = A) = P(Y = 1|G = B)$. Tương đương xác suất (equalized odds) đòi hỏi TPR và FPR bằng nhau giữa các nhóm.

Ví dụ: Một hệ thống cho vay chấp thuận 50% ở mỗi nhóm (thỏa ngang bằng nhân khẩu học). Nhưng Nhóm A có TPR = 90%, FPR = 30%, trong khi Nhóm B có TPR = 70%, FPR = 10%. Điều này vi phạm tương đương xác suất vì các tỷ lệ lỗi khác nhau giữa các nhóm.

3.

(a) Nhóm A: $TPR = 80/100 = 0.80$, $FPR = 10/100 = 0.10$. Nhóm B: $TPR = 60/100 = 0.60$, $FPR = 10/100 = 0.10$.

(b) Không thỏa tương đương xác suất: FPR bằng nhau (cả hai đều 0.10), nhưng TPR khác nhau (0.80 so với 0.60). Tương đương xác suất đòi hỏi cả hai tỷ lệ đều bằng nhau.

(c) Tỷ lệ dương của Nhóm A: $90/200 = 0.45$. Tỷ lệ dương của Nhóm B: $70/200 = 0.35$. Không bằng nhau, nên ngang bằng nhân khẩu học không được thỏa.

4. Nếu $P(Y = 1|G = A) \neq P(Y = 1|G = B)$ (tỷ lệ nền khác nhau), một bộ phân loại được hiệu chuẩn thỏa $P(Y = 1|\hat{Y} = 1, G = g) = p$ với mọi nhóm. Dưới hiệu chuẩn, tỷ lệ dự đoán dương của nhóm g tỷ lệ thuận với tỷ lệ nền $P(Y = 1|G = g)$. Vì tỷ lệ nền khác nhau, các tỷ lệ dự đoán phải khác nhau, do đó vi phạm ngang bằng nhân khẩu học. Hiệu chuẩn và ngang bằng nhân khẩu học không thể cùng thỏa mãn về mặt toán học khi tỷ lệ nền khác nhau.

5. Giả sử Nhóm A có $P(Y = 1) = 0.5$ và đặc trưng X có nhiều thông tin, trong khi Nhóm B có $P(Y = 1) = 0.5$ nhưng X bị thiên kiến (mã hóa tư cách thành viên nhóm thay vì tín hiệu thật). Việc loại bỏ X buộc cả hai nhóm phải dùng chung các đặc trưng còn lại, có thể kém thông tin hơn với Nhóm A. Kết quả: độ chính xác giảm ở cả hai nhóm vì đặc trưng thiên kiến đó thực sự hữu ích với Nhóm A, dù nó gây hại cho Nhóm B.

6.

(a) Không đưa giới tính vào không đảm bảo không phân biệt đối xử. Các biến thay thế (lương, chức danh công việc, số năm kinh nghiệm) tương quan với giới tính do khoảng cách lương lịch sử và sự phân tách nghề nghiệp.

(b) “Lương ở công việc trước” là biến thay thế mạnh nhất: khoảng cách lương theo giới tính có nghĩa biến này trực tiếp mã hóa thông tin giới tính. “Chức danh công việc” cũng có thể tương quan thông qua sự phân tách nghề nghiệp.

(c) So sánh dự đoán của mô hình giữa các giới tính với những người có năng lực giống hệt nhau. Nếu kết quả khác nhau, có sự phân biệt đối xử qua biến thay thế. Các kiểm tra chính thức: xem các dư sai (residuals) của mô hình có tương quan với giới tính sau khi kiểm soát các biến dự đoán hợp lệ hay không.

7.(a) Đề xuất nội dung: DỮ LIỆU: lịch sử xem của người dùng. MỤC TIÊU: tối đa hóa thời gian xem. CHỈ SỐ: phút đã xem. TÁC HẠI: khuyến khích nội dung gây nghiện, cực đoan hoặc gây hiểu lầm. GIẢI PHÁP: giới hạn khuyến nghị nội dung giật gân; đưa vào các chỉ số đa dạng.

(b) Dự báo tội phạm: DỮ LIỆU: báo cáo tội phạm lịch sử. MỤC TIÊU: dự đoán địa điểm tội phạm. CHỈ SỐ: độ chính xác của dự đoán. TÁC HẠI: củng cố cảnh sát hóa quá mức ở các cộng đồng từng bị nhắm tới (vòng lặp phản hồi). GIẢI PHÁP: kiểm tra thiên kiến theo địa lý; kết hợp với ý kiến cộng đồng.

(c) Sàng lọc y tế: DỮ LIỆU: dấu hiệu sinh tồn, triệu chứng, bệnh sử của bệnh nhân. MỤC TIÊU: ưu tiên theo mức khẩn cấp. CHỈ SỐ: kết quả sống sót. TÁC HẠI: hệ thống hạ ưu tiên các nhóm có triệu chứng biểu hiện khác thường (ví dụ: phụ nữ bị nhồi máu cơ tim). GIẢI PHÁP: kiểm chứng trên các nhóm nhân khẩu học; yêu cầu khả năng con người can thiệp ghi đề.

8.

(a) Dữ liệu GPS tiết lộ nơi ở và nơi làm việc thông qua các mô hình lặp lại. Ngay cả khi không có tên, thói quen hằng ngày của một người (ở nhà ban đêm, ở văn phòng ban ngày) gần như duy nhất và dễ dàng liên kết với hồ sơ công khai.

(b) Đối chiếu vị trí nhà với hồ sơ bất động sản, vị trí làm việc với danh bạ người sử dụng lao động, hoặc các địa điểm thường lui tới với hoạt động đăng ký vị trí trên mạng xã hội. Nghiên cứu đã chỉ ra rằng chỉ bốn điểm không gian-thời gian cũng có thể xác định duy nhất 95% cá nhân.

(c) Quyền riêng tư khác biệt sẽ thêm nhiều vào dữ liệu vị trí, nhưng đánh đổi là giảm tính hữu dụng. Nhiều thô (làm tròn về khu phố) vẫn giữ được một số giá trị cho nhà quảng cáo nhưng vẫn có nguy cơ tái nhận dạng. Nhiều mịn làm dữ liệu vô giá trị về thương mại.

9.(a) Các bên có thể chịu trách nhiệm: nhà phát triển AI (về thiết kế và kiểm chứng hệ thống), bệnh viện (về quyết định triển khai và định vai trò của nó), các bác sĩ lâm sàng (về việc không sử dụng phán đoán độc lập), và các cơ quan quản lý (về việc phê duyệt mà không có giám sát đầy đủ).

(b) Thông tin cần có: độ chính xác của hệ thống trên các bệnh nhân tương tự, ca bệnh của bệnh nhân có nằm trong phân bố huấn luyện của hệ thống hay không, các bác sĩ lâm sàng có tùy chọn ghi đề hay không, và khuyến nghị của hệ thống có phải là nguyên nhân gần nhất dẫn đến kết quả hay không.

(c) Ủng hộ: các ca phức tạp được hưởng lợi từ nhận diện khuôn mẫu trên hàng triệu ca; yêu cầu tính giải thích đầy đủ sẽ chặn các công cụ có thể cứu mạng. Phản đối: bệnh nhân có quyền hiểu các quyết định về chăm sóc của họ; các hệ thống không giải thích được thì không thể được kiểm toán đúng đắn về thiên kiến hay lỗi.

10.(a) Người nộp đơn bị từ chối không thể xây dựng lịch sử tín dụng, nên dữ liệu trong tương lai tiếp tục cho thấy mức trả nợ thấp ở những khu vực đó. Các dự đoán của AI trở thành tự hoàn thành: việc từ chối ngăn cản bằng chứng bác bỏ dự đoán.

(b) Chênh lệch sẽ tăng lên. Các khoản vay bị từ chối ngăn cản sự phát triển kinh tế ở những khu vực đó, làm dữ liệu tương lai xấu đi và củng cố mô hình từ chối. Đây là một vòng lặp phản hồi dương.

(c) Các can thiệp: chấp thuận một mẫu ngẫu nhiên các ứng viên cận biên từ khu vực bất lợi để thu thập dữ liệu kết quả không thiên kiến; dùng mô hình nhân quả để tách các hiệu ứng địa lý khỏi mức tín dụng; yêu cầu

huấn luyện lại định kỳ trên dữ liệu bao gồm các phê duyệt thực nghiệm này.

11.

(a) Ưu tiên Hệ thống A (có thể giải thích) cho những tình trạng mà quyết định điều trị phức tạp và cần bác sĩ hiểu, nơi những sai sót có hậu quả pháp lý cần giải thích, hoặc cho các bệnh mạn tính nơi bệnh nhân cần hiểu và tin vào quy trình chẩn đoán.

(b) Ưu tiên Hệ thống B (chính xác) cho sàng lọc các tình trạng đe dọa tính mạng, nơi mức cải thiện 7% về độ chính xác chuyển thành số mạng sống được cứu đáng kể hơn, và nơi xét nghiệm tiếp theo có thể xác nhận hoặc chỉnh sửa khuyến nghị của AI.

(c) Bạn có thể giải thích: hệ thống đã xác định các khuôn mẫu trong dữ liệu của bạn có liên quan đến tình trạng đang được xét nghiệm. Bạn không thể giải thích chính xác những khuôn mẫu nào hay tại sao. Bạn có thể nêu rằng bác sĩ đã xem xét khuyến nghị và có xét nghiệm xác nhận sẵn sàng.

12.

(a) Chi phí huấn luyện mỗi người dùng: $C10\,000\,000 / 100\,000\,000 = C0.10$. Carbon mỗi người dùng: $500 / 100\,000\,000 = 0.000005$ tấn = 5 mg CO₂.

(b) Tổng chi phí suy luận: $1\,000\,000\,000 \times €0.001 = €1\,000\,000$. Vì vậy chi phí suy luận là C1M so với C10M cho huấn luyện. Theo thời gian, suy luận chiếm ưu thế.

(c) So sánh lợi ích tổng hợp (năng suất tăng, khả năng tiếp cận cải thiện, giá trị giáo dục) với các chi phí. Cân nhắc: mô hình có thay thế các phương án thay thế tốn nhiều carbon hơn không? Lợi ích có được phân phối công bằng không? Chi phí C0.10 mỗi người dùng có phải là mức giá hợp lý cho giá trị mang lại không?

13.

(a) Các tác hại tiềm ẩn: deepfake khi chưa đồng thuận, thông tin sai lệch chính trị, gian lận và mạo danh, xói mòn niềm tin vào bằng chứng thị giác, và tổn thương tâm lý cho những người xuất hiện trong hình.

(b) Các biện pháp bảo vệ hiện tại: bộ lọc nội dung chặn các prompt gây hại đã biết, gắn dấu chìm vô hình cho hình ảnh được tạo, chặn việc tạo hình ảnh nhân vật công chúng có tên, và các bộ lọc nội dung NSFW.

(c) DỮ LIỆU: hàng tỷ hình ảnh trên internet (bao gồm ảnh có bản quyền và ảnh cá nhân). MỤC TIÊU: tạo hình ảnh từ mô tả văn bản. CHỈ SỐ: chất lượng hình ảnh và mức khớp văn bản. TÁC HẠI: hình ảnh khi chưa đồng thuận,

vi phạm bản quyền, khuếch đại thiên kiến. GIẢI PHÁP: lọc nội dung, gắn dấu chìm, cơ chế từ chối cho nghệ sĩ, kiểm toán thiên kiến của đầu ra.

Chương 3: Chuẩn bị dữ liệu và kỹ thuật đặc trưng

1. Dữ liệu: [2, 4, 6, 8, 10]

Trung bình: $\mu = (2 + 4 + 6 + 8 + 10)/5 = 30/5 = 6$

Variance: $\sigma^2 = [(2-6)^2 + (4-6)^2 + (6-6)^2 + (8-6)^2 + (10-6)^2]/5 = [16 + 4 + 0 + 4 + 16]/5 = 40/5 = 8$

Độ lệch chuẩn: $\sigma = \sqrt{8} \approx 2.83$

Giá trị đã chuẩn hóa: $\tilde{x}_i = (x_i - 6)/2.83$: xấp xỉ -1.41, -0.71, 0, 0.71, 1.41.

2. Mã hóa one-hot tạo 3 cột (một cột cho mỗi danh mục). "Medium" được mã hóa là [0, 1, 0] (giả sử thứ tự Small, Medium, Large).

3. Với 5000 mẫu và tỷ lệ 70-15-15: Huấn luyện: $5000 \times 0.70 = 3500$. Kiểm định: $5000 \times 0.15 = 750$. Kiểm tra: $5000 \times 0.15 = 750$.

4. Các đặc trưng được thiết kế đề xuất: (1) số ngày kể từ lần mua gần nhất (recency), (2) giá trị mua trung bình trong 30 ngày qua (xu hướng chi tiêu), (3) số lượt mua theo từng danh mục sản phẩm (tín hiệu sở thích), (4) mẫu mua theo thứ trong tuần và giờ trong ngày (hành vi theo thời gian), (5) tỷ lệ lần mua hiện tại so với trung bình lịch sử của người dùng (tín hiệu bất thường).

5. Hai cách tiếp cận: (1) Điền khuyết bằng thu nhập trung vị, vốn bền vững với ngoại lai và giữ nguyên kích thước mẫu, tốt nhất khi dữ liệu thiếu ngẫu nhiên. (2) Bỏ các dòng thiếu thu nhập, đơn giản nhưng mất 10% dữ liệu, tốt nhất khi dữ liệu dồi dào và sự thiếu có thể mang thông tin (ví dụ: người thu nhập cao từ chối báo cáo).

6. Biến đổi log giúp vì giá nhà thường lệch phải (nhiều nhà khiêm tốn, ít lâu đài). Biến đổi nén phần dải trên, làm phân bố cân đối hơn và giảm ảnh hưởng quá lớn của các giá trị cực đoan lên hàm mất mát của mô hình.

7. Ba cách tiền xử lý có thể gây ra khoảng cách 98%/0%: (1) Rò rỉ dữ liệu: chuẩn hóa hoặc điền khuyết bằng thống kê của toàn bộ tập dữ liệu (bao gồm cả tập kiểm tra), vì vậy mô hình "thấy" thông tin kiểm tra trong khi huấn luyện. (2) Không khớp phân bố: dữ liệu huấn luyện được làm sạch theo những cách mà dữ liệu thế giới thực không thể tái tạo (ví dụ: loại bỏ các ngoại lai vốn phổ biến trong vận hành). (3) Lỗi mã hóa đặc trưng: các đặc trưng phân loại được mã hóa bằng ánh xạ riêng của tập huấn luyện, phá vỡ trên các danh mục chưa từng thấy ở dữ liệu mới.

8.

(a) k-NN tính khoảng cách, vì vậy thu nhập (phạm vi 0–1.000.000) sẽ chi phối tuổi (0–100) và số con (0–10). Hàng xóm gần nhất gần như hoàn toàn được quyết định bởi thu nhập, bỏ qua các đặc trưng khác.

(b) Cây quyết định chia theo từng đặc trưng và bất biến với biến đổi đơn điệu của bất kỳ đặc trưng nào. Việc chuẩn hóa không ảnh hưởng đến các điểm chia hay độ chính xác của cây.

(c) Cây quyết định bất biến với thang đo vì chúng dùng phép so sánh ngưỡng trên từng đặc trưng. Các phương pháp dựa trên khoảng cách (k-NN, SVM, K-means) nhạy với thang đo vì chúng kết hợp các đặc trưng thành một thước đo khoảng cách duy nhất.

9.

(a) Ba chiến lược: (1) điền khuyết bằng trung bình/trung vị, (2) xóa các dòng có giá trị thiếu, (3) điền khuyết dựa trên mô hình (ví dụ: dự đoán thu nhập từ các đặc trưng khác).

(b) Các kịch bản tốt nhất: (1) Điền khuyết bằng trung vị khi dữ liệu thiếu ngẫu nhiên và kích thước mẫu bị hạn chế. (2) Xóa khi dữ liệu dồi dào và sự thiếu tương quan với biến mục tiêu. (3) Dựa trên mô hình khi các đặc trưng khác dự đoán thu nhập tốt và sự thiếu mang tính hệ thống.

(c) Các kịch bản có hại: (1) Điền khuyết bằng trung vị khi người thu nhập cao từ chối báo cáo một cách hệ thống (làm thiên lệch phân bố thu nhập xuống dưới). (2) Xóa khi sự thiếu không ngẫu nhiên, tạo ra mẫu còn lại bị thiên kiến. (3) Dựa trên mô hình khi chính mô hình điền khuyết có lỗi lan truyền xuyên suốt pipeline.

10. “Lý do hủy” là rò rỉ biến mục tiêu: chỉ có sẵn sau khi việc hủy xảy ra, nên nó mã hóa kết quả trực tiếp. Mô hình dùng nó sẽ có vẻ rất chính xác khi huấn luyện nhưng thất bại khi triển khai (đặc trưng sẽ trống rỗng cho khách hàng đang hoạt động). Kiểm chứng: xem đặc trưng được điền trong pipeline dữ liệu vào thời điểm nào. Các đặc trưng khác (số ngày kể từ lần đăng nhập gần nhất, tổng số lần mua, thời lượng phiên, phiếu hỗ trợ) đều có sẵn trước khi hủy và là các biến dự đoán hợp lệ.

11. Các đặc trưng này không phải rò rỉ vì được biết tại thời điểm dự đoán và mô tả chính ngôi nhà, không phải kết quả. Diện tích, số phòng ngủ, năm xây dựng và vị trí đều quan sát được trước khi bán và thực sự dự đoán giá. Rò rỉ đòi hỏi thông tin từ tương lai hoặc từ chính biến mục tiêu làm ô nhiễm các đặc trưng. Tương quan mạnh với biến mục tiêu chính là toàn bộ mục đích của một đặc trưng tốt, không phải bằng chứng rò rỉ.

12.

(a) Không có rò rỉ: số giờ học được đo trước khi thi và là một biến dự đoán hợp lệ.

(b) Rò rỉ biến mục tiêu: mức hài lòng về điểm số được đo sau khi thi và mã hóa kết quả.

(c) Rò rỉ đặc trưng: tính thống kê trên tập dữ liệu kết hợp để đặc trưng huấn luyện tiếp nhận thông tin từ tập kiểm tra.

(d) Không có rò rỉ: thời gian lưu ban đầu được biết khi ra viện, tức là trước kết quả nhập viện trở lại.

13.(a) Thiên kiến địa lý: thiếu sót tình trạng lái xe trong tuyết, mưa, khu đô thị đông đúc và các nước tay lái bên trái.

(b) Thiên kiến chọn mẫu: các bài đánh giá được viết bởi những người có cảm xúc mạnh. Đa số ôn hòa bị đại diện không đầy đủ, làm lệch thiên vị về các cực.

(c) Thiên kiến sống sót/thất thoát: những người bỏ cuộc có thể đã gặp tác dụng phụ. 70% còn lại có lẽ khỏe mạnh hơn, khiến thuốc có vẻ hiệu quả hơn thực tế.

(d) Thiên kiến sống sót: tập dữ liệu loại trừ các startup thất bại. Bất kỳ khuôn mẫu nào được tìm thấy (ví dụ “startup thành công có nhà sáng lập giàu cá tính”) có thể cũng đúng với những startup thất bại.

14.(a) Dịch chuyển biến số phụ thuộc (covariate shift) (phân bố đầu vào thay đổi mạnh khi đại dịch thay đổi thói quen đi lại) và có thể có trôi concept (concept drift) (mối quan hệ giữa các đặc trưng như thời điểm trong ngày và nhu cầu cũng thay đổi).

(b) Huấn luyện lại trên tháng 1–2 năm 2020 sẽ không giúp ích: những tháng đó vẫn phản ánh hành vi trước đại dịch. Các mối quan hệ cơ bản giữa đặc trưng và nhu cầu đã thay đổi, vì vậy dữ liệu gần đây hơn trước đại dịch sẽ không nắm bắt được hiện thực mới.

(c) Giám sát phân bố đầu vào so với đầu vào huấn luyện (phát hiện covariate shift), theo dõi sai số dự đoán trên các kết quả gần đây (phát hiện suy giảm độ chính xác), và đặt cảnh báo khi độ tin cậy dự đoán giảm một cách hệ thống.

15. Độ chính xác kiểm tra 94% không đáng tin vì cả dữ liệu huấn luyện lẫn kiểm tra đều đến từ cùng các phòng khám và dân số (người Bắc Âu). Điều này

chỉ đo hiệu năng trong phân bố huấn luyện. Với các bệnh nhân có tông da, điều kiện ánh sáng hoặc thiết bị camera khác nhau, độ chính xác có thể thấp hơn nhiều. Một kiểm tra hợp lệ sẽ yêu cầu dữ liệu từ nhiều phòng khám, loại da và địa lý đa dạng. Không có điều này, độ chính xác được báo cáo sẽ khuếch đại hiệu năng thế giới thực.

Chương 4: Học có giám sát

1. Với ngôi nhà 2000 feet vuông: $h(x) = 50\,000 + 100 \times 2000 = \text{€}250\,000$. Hạng mục chặn C50 000 là giá cơ sở dự đoán (khi $x = 0$, điều này không có ý nghĩa). Hệ số 100 có nghĩa mỗi feet vuông thêm làm tăng giá dự đoán thêm €100.

2. Đầu ra $h_\theta(x) = 0.73$ được diễn giải là xác suất: mô hình ước lượng xác suất 73% thuộc lớp dương. Ở ngưỡng 0.5, vì $0.73 > 0.5$, lớp dự đoán là dương (lớp 1).

3. Sau khi chia theo A : nhánh trái có $p_+ = 4/4 = 1$, entropy = 0. Nhánh phải có $p_+ = 1/6$, entropy = $-\frac{1}{6} \log_2 \frac{1}{6} - \frac{5}{6} \log_2 \frac{5}{6} \approx 0.65$. Trung bình có trọng số của entropy = $\frac{4}{10}(0) + \frac{6}{10}(0.65) = 0.39$. Lợi thông tin = $1.0 - 0.39 = 0.61$.

4. SVM tìm siêu phẳng cực đại hóa khoảng cách tối thiểu (biên) tới bất kỳ điểm huấn luyện nào. Tham số C kiểm soát sự đánh đổi giữa việc cực đại hóa biên và cho phép các phân loại nhầm. Lớn C : biên hẹp, ít vi phạm (nguy cơ quá khớp). Nhỏ C : biên rộng, dung thứ nhiều vi phạm hơn (nguy cơ thiếu khớp).

5. L1 (Lasso) thêm $\lambda \sum |w_i|$: khiến một số trọng số bằng đúng không, thực hiện chọn đặc trưng. Ưu tiên khi bạn kỳ vọng nhiều đặc trưng không liên quan.

L2 (Ridge) thêm $\lambda \sum w_i^2$: co các trọng số về phía không nhưng hiếm khi bằng đúng không. Ưu tiên khi mọi đặc trưng đều có thể liên quan và bạn muốn giảm biên độ của chúng đồng đều.

6. Bộ phân loại Hàng xóm có trọng số:

(a) Khi $\sigma \rightarrow 0$: trọng số tập trung vào hàng xóm gần nhất duy nhất (WNC trở thành 1-NN). Khi $\sigma \rightarrow \infty$: mọi trọng số trở nên bằng nhau, và dự đoán là một cuộc bỏ phiếu trên toàn bộ các điểm huấn luyện (lớp đa số thắng).

(b) Đây là phiên bản có trọng số, làm mượt của k-NN. Hàm trọng số Gaussian tự nó là một hạt nhân, biến WNC thành một bộ phân loại hạt nhân với hạt nhân RBF.

(c) Ranh giới quyết định không tuyến tính. Nó theo đường đồng mức mà cuộc bỏ phiếu có trọng số cân bằng, vốn cong quanh các cụm của từng lớp.

(d) Chế độ thất bại: nhảy với σ (quá nhỏ = quá khớp, quá lớn = thiếu khớp); dự đoán yêu cầu tính khoảng cách tới mọi điểm huấn luyện ($O(n)$ mỗi truy vấn); gặp khó với mất cân bằng lớp vì lớp đa số chi phối tổng có trọng số.

(e) Chuẩn hóa trọng số theo lớp: $\hat{y} = \text{sign} \left(\sum_{i:y_i=+1} w_i/n_+ - \sum_{i:y_i=-1} w_i/n_- \right)$, nơi n_+, n_- là số lượng điểm mỗi lớp. Điều này đảm bảo mỗi lớp đóng góp bình đẳng bất kể kích thước.

7.

(a) Giả sử $w_1x_1 + w_2x_2 + b = 0$ tách các lớp. Các ràng buộc yêu cầu: $b < 0$ (cho $(0, 0) \rightarrow 0$), $w_2 + b > 0$ (cho $(0, 1) \rightarrow 1$), $w_1 + b > 0$ (cho $(1, 0) \rightarrow 1$), và $w_1 + w_2 + b < 0$ (cho $(1, 1) \rightarrow 0$). Cộng cái thứ hai và thứ ba: $w_1 + w_2 + 2b > 0$. Nhưng từ cái thứ nhất và thứ tư: $w_1 + w_2 + 2b < b + (w_1 + w_2 + b) < 0$. Mâu thuẫn.

□

(b) Với $x_3 = x_1x_2$: các điểm trở thành $(0, 0, 0), (0, 1, 0), (1, 0, 0), (1, 1, 1)$. Mặt phẳng $x_1 + x_2 - 2x_3 = 0.5$ tách các điểm nhãn-1 khỏi các điểm nhãn-0.

(c) Cây quyết định: chia theo x_1 . Nếu $x_1 = 0$: chia theo x_2 (đầu ra x_2). Nếu $x_1 = 1$: chia theo x_2 (đầu ra $1 - x_2$).

(d) Thủ thuật kernel ám xạ không gian chiều cao hơn (như trong phần b) mà không tính toán tường minh các đặc trưng mới. Hạt nhân đa thức bậc 2 tự động gồm hạng tương tác x_1x_2 .

8.

(a) “Số năm tại địa chỉ hiện tại” tương quan với tuổi (người lớn tuổi chuyển nhà ít thường xuyên hơn). Hệ số âm thực chất phạt các ứng viên trẻ hơn, cấu thành phân biệt gián tiếp về tuổi tác qua một biến thay thế.

(b) Sự giảm AUC từ 0.82 xuống 0.79 là giảm 3 điểm. Việc này có thể chấp nhận được hay không phụ thuộc vào bối cảnh: nếu 0.79 vẫn vượt ngưỡng hiệu năng tối thiểu của ngân hàng, lợi ích công bằng có thể biện minh cho chi phí về độ chính xác. Nếu mô hình đưa ra 10.000 quyết định mỗi năm, ngay cả thay đổi AUC nhỏ cũng có thể ảnh hưởng tới hàng trăm ứng viên.

(c) Tính ngang bằng nhân khẩu học (tỷ lệ chấp thuận bằng nhau theo nhóm tuổi), tương đương xác suất (TPR/FPR bằng nhau theo nhóm tuổi), và quy tắc 80% (tỷ số giữa các tỷ lệ chấp thuận). Cũng kiểm tra hiệu chuẩn: trong số các ứng viên được cho cùng xác suất dự đoán, tỷ lệ vỡ nợ thực tế có khác nhau theo tuổi không?

9. Bộ phân loại Nguyên mẫu:

- (a) Ranh giới là một siêu phẳng. Tập $\{x : \|x - \mu_1\| = \|x - \mu_2\|\}$ là trung tuyến vuông góc của đoạn thẳng nối hai trọng tâm, vốn luôn là một siêu phẳng.
- (b) k-NN với $k =$ mọi điểm dùng bỏ phiếu đa số trên toàn bộ lớp. Bộ phân loại nguyên mẫu chỉ dùng trọng tâm (trung bình). Chúng tương đương khi dữ liệu trong mỗi lớp đối xứng hình cầu, nhưng khác nhau khi phân bố lớp không đối xứng hoặc đa mode.
- (c) Bộ phân loại hoạt động tốt khi mỗi lớp tạo thành một cụm duy nhất, gần hình cầu, lấy trung bình làm tâm.
- (d) Một lớp có hai cụm con tách biệt rõ (ví dụ lớp A tại $\{(0, 0), (10, 10)\}$ và lớp B tại $\{(5, 5)\}$). Trọng tâm của lớp A là $(5, 5)$, trùng với lớp B. Bộ phân loại thất bại vì trung bình không đại diện cho bất kỳ vùng thực tế nào của lớp A.
- (e) Bộ phân loại lưu $C \times d$ tham số (C trọng tâm, mỗi cái d chiều). k-NN lưu toàn bộ $n \times d$ điểm huấn luyện. Hồi quy logistic lưu $(d + 1) \times C$ trọng số. Với các bài toán điển hình, bộ phân loại nguyên mẫu là phương pháp gọn nhất.

Chương 5: Đánh giá mô hình

- Với 10.000 điểm, tỷ lệ 70-15-15 cho ra: 7.000 huấn luyện, 1.500 kiểm định (để tinh chỉnh siêu tham số), và 1.500 kiểm tra (để đánh giá cuối cùng). Tập kiểm định được dùng trong quá trình phát triển; tập kiểm tra chỉ được động tới một lần ở cuối để báo cáo hiệu năng không thiên lệch.
- Khoảng cách 34% giữa độ chính xác huấn luyện (99%) và kiểm định (65%) cho thấy phương sai cao (quá khớp). Mô hình đã ghi nhớ dữ liệu huấn luyện nhưng không khái quát hóa được. Đây là miền thiên vị thấp, phương sai cao. Cách khắc phục: đơn giản hóa mô hình, thêm regularization, thu thập thêm dữ liệu huấn luyện, hoặc dùng dropout/early stopping.
- Từ ma trận nhầm lẫn (TP=450, FP=100, FN=50, TN=400):
 - Độ chính xác (Precision) = $450 / (450 + 100) = 450 / 550 \approx 81.8\%$
 - Độ nhạy (Recall) = $450 / (450 + 50) = 450 / 500 = 90\%$
 - $F1 = 2 \times (0.818 \times 0.90) / (0.818 + 0.90) = 2 \times 0.736 / 1.718 \approx 85.7\%$
- Tối ưu cho độ chính xác (precision) khi các dương giả tốn kém. Ví dụ: lọc thư rác, nơi một dương giả có nghĩa là một email hợp lệ bị thất thoát. Bạn thả cho một số thư rác lọt (độ nhạy thấp hơn) còn hơn vô tình xóa nhầm

các thông báo quan trọng (độ chính xác cao).

5.

(a) Một lần chia duy nhất phụ thuộc việc các điểm cụ thể nào rơi vào huấn luyện so với kiểm tra, đưa tới phương sai cao. Lấy trung bình trên k lớp (fold) nghĩa là mỗi điểm vừa được dùng làm huấn luyện vừa làm kiểm tra, giảm phương sai này và cho ước lượng ổn định hơn.

(b) Khi $k \rightarrow n$ (LOO-CV): thiên vị giảm (mỗi tập huấn luyện gần như là toàn bộ dữ liệu), phương sai tăng (các tập huấn luyện n chồng lấp gần như hoàn toàn, khiến ước lượng theo lớp có tương quan cao), và chi phí tính toán trở thành $O(n)$ lần huấn luyện mô hình.

(c) $k = 5$ hoặc $k = 10$ cân bằng giữa thiên vị (các tập huấn luyện chiếm 80–90% dữ liệu, nên mô hình mang tính đại diện) với phương sai (các lớp đủ khác biệt để cho ước lượng độc lập) và chi phí tính toán (chỉ 5–10 lần huấn luyện mô hình).

(d) Nếu nhiều mẫu đến từ cùng một bệnh nhân, kiểm chéo chuẩn có thể đặt các mẫu khác nhau của cùng bệnh nhân vào cả huấn luyện lẫn kiểm tra, làm mô hình có vẻ tốt hơn trên bệnh nhân thực sự mới. Dùng kiểm chéo k -lớp theo nhóm, nơi mọi mẫu của một bệnh nhân được giữ trong cùng một lớp.

6.

(a) Thiên vị $= \mathbb{E}[\hat{f}(x)] - f(x)$: độ lệch có hệ thống của dự đoán trung bình của mô hình so với hàm thật.

(b) Một mô hình hằng số $\hat{f}(x) = c$ bỏ qua mọi đặc trưng đầu vào, vì vậy nó không nắm bắt được bất kỳ cấu trúc nào trong $f(x)$. Dự đoán kỳ vọng của nó luôn là c , tạo ra thiên vị lớn ở bất kỳ đâu $f(x) \neq c$.

(c) Một mô hình nội suy khớp chính xác mọi điểm huấn luyện, vì vậy dự đoán của nó thay đổi mạnh mẽ với các tập huấn luyện khác nhau. Mỗi tập huấn luyện sinh ra một hàm khác nhau, tạo ra phương sai cao ở bất kỳ giá trị x nào.

(d) Regularization làm tăng thiên vị (mô hình bị ràng buộc không khớp hoàn hảo dữ liệu huấn luyện) nhưng có thể giảm phương sai nhiều hơn (mô hình ít nhạy với tập huấn luyện cụ thể). Ảnh hưởng ròng là tổng sai số thấp hơn khi phần giảm phương sai lớn hơn phần tăng thiên vị.

7.

(a) Độ chính xác (Accuracy) $= (90 + 9405)/10000 = 94.95\%$. Precision $= 90/(90 +$

$$495) = 90/585 \approx 15.4\%. \text{ Recall} = 90/(90 + 10) = 90\%. F1 = 2 \times (0.154 \times 0.90)/(0.154 + 0.90) \approx 26.3\%.$$

(b) Một bộ phân loại tầm thường dự đoán “không bệnh” cho mọi người đạt 99% độ chính xác. 94.95% của mô hình thực ra còn tệ hơn đường cơ sở này, nghĩa là nó tạo ra nhiều lỗi hơn là dự đoán lớp đa số. Độ chính xác gây hiểu lầm vì các lớp mất cân bằng nghiêm trọng.

(c) PPV (giá trị dự đoán dương) = $90/585 \approx 15.4\%$. Chỉ khoảng 1 trong 6 kết quả dương là thật. Điều này khác với độ nhạy (recall, 90%) vì độ nhạy điều kiện trên các trường hợp dương thật trong khi PPV điều kiện trên các dự đoán dương. Với tỷ lệ nhiễm thấp, ngay cả bài kiểm tra có độ nhạy cao cũng tạo ra nhiều dương giả.

8. Ba cách AUC = 0.99 có thể bị khuếch đại: (1) rò rỉ dữ liệu, nơi thông tin tập kiểm tra rò vào huấn luyện qua tiền xử lý dùng chung; (2) rò rỉ biến mục tiêu, nơi một đặc trưng trực tiếp mã hóa kết quả (ví dụ dùng chẩn đoán lúc xuất viện để dự đoán kết quả nhập viện); (3) rò rỉ thời gian, nơi thông tin tương lai xuất hiện trong các đặc trưng sẽ không sẵn có tại thời điểm dự đoán. Bất kỳ AUC nào trên 0.95 với bài toán thế giới thực đều nên kích hoạt điều tra kỹ lưỡng.

9. Mô hình thứ hai (70% huấn luyện / 65% kiểm tra) tốt hơn. Mô hình đầu tiên (100% huấn luyện / 60% kiểm tra) có khoảng cách khái quát hóa 40% cho thấy quá khớp nghiêm trọng, vì vậy độ chính xác kiểm tra 60% của nó không đáng tin và có lẽ lạc quan. Khoảng cách 5% của mô hình thứ hai là vừa phải, nghĩa là độ chính xác kiểm tra 65% của nó đáng tin cậy. Một mức 65% đáng tin có giá trị hơn một mức 60% không đáng tin.

Chương 6: Học không giám sát và phân cụm

1. K-means với các điểm $\{(1, 1), (2, 1), (4, 3), (5, 4)\}$, trọng tâm ban đầu $\mu_1 = (1, 1), \mu_2 = (5, 4)$:

Vòng lặp 1: Gán mỗi điểm cho trọng tâm gần nhất. $(1, 1) \rightarrow \mu_1$ (khoảng cách 0), $(2, 1) \rightarrow \mu_1$ (khoảng cách 1), $(4, 3) \rightarrow \mu_2$ (khoảng cách $\sqrt{2}$), $(5, 4) \rightarrow \mu_2$ (khoảng cách 0). Cập nhật: $\mu_1 = (1.5, 1), \mu_2 = (4.5, 3.5)$.

Vòng lặp 2: Gán lại. $(1, 1) \rightarrow \mu_1$ (khoảng cách 0.5), $(2, 1) \rightarrow \mu_1$ (khoảng cách 0.5), $(4, 3) \rightarrow \mu_2$ (khoảng cách $\sqrt{0.5}$), $(5, 4) \rightarrow \mu_2$ (khoảng cách $\sqrt{0.5}$). Phân công không đổi, nên thuật toán đã hội tụ.

2. Between $(2, 5, 1)$ and $(6, 2, 3)$: Euclidean = $\sqrt{(6-2)^2 + (2-5)^2 + (3-1)^2} = \sqrt{16+9+4} = \sqrt{29} \approx 5.39$. Manhattan = $|6-2| + |2-5| + |3-1| = 4+3+2 = 9$.

3. K-means tìm cực tiểu cục bộ vì hàm mục tiêu (tổng độ méo) không lồi.

Các khởi tạo khác nhau bắt đầu ở các lưu vực hấp dẫn khác nhau, dẫn tới các cực tiểu cục bộ khác nhau. Biện pháp giảm thiểu: chạy K-means nhiều lần với khởi tạo ngẫu nhiên khác nhau và giữ kết quả tốt nhất, hoặc dùng khởi tạo K-means++ (chọn các trọng tâm ban đầu phân tán xa nhau).

4. Các trách nhiệm (responsibilities) có nghĩa: 80% xác suất điểm thuộc cụm 1, 15% thuộc cụm 2, 5% thuộc cụm 3. Khác với K-means, vốn gán cứng mỗi điểm vào đúng một cụm, GMM cung cấp các thành viên xác suất mềm. Điều này hữu ích khi ranh giới cụm chồng lấp, cho phép một điểm thuộc nhiều cụm một cách một phần.

5. Sự giảm lớn nhất là từ $k = 1$ tới $k = 2$ (60 đơn vị), rồi $k = 2$ tới $k = 3$ (15 đơn vị). Sau $k = 3$, cải thiện rất ít (3, rồi 2). “Khuỷu tay” nằm ở $k = 3$, nơi tốc độ cải thiện phẳng ra.

6.

(a) PCA: giảm 1000 đặc trưng còn 50 trong khi giữ lại phương sai tối đa. PCA được thiết kế cho giảm chiều như một bước tiền xử lý.

(b) t-SNE: được thiết kế riêng cho trực quan hóa 2D/3D dữ liệu nhiều chiều, giữ lại cấu trúc láng giềng cục bộ.

(c) Phân cụm phân cấp (kèm dendrogram): dendrogram cho phép bạn chọn số cụm sau đó bằng cách cắt ở các độ cao khác nhau.

(d) GMM: mô hình hóa các cụm như các phân bố Gaussian với ma trận hiệp phương sai khác nhau, xử lý tự nhiên các hình elip và kích thước thay đổi. K-means giả định các cụm hình cầu cùng kích thước.

7.(a) Chiều xuống 3D làm mất $100\% - 85\% = 15\%$ phương sai.

(b) Chấp nhận được vì 85% nắm bắt hầu hết cấu trúc; việc giảm từ 50D xuống 3D tiết kiệm tính toán khổng lồ; và 15% bị loại bỏ có thể phần lớn là nhiễu chứ không phải tín hiệu có ý nghĩa.

8. t-SNE chỉ giữ lại cấu trúc cục bộ (các điểm gần nhau vẫn gần nhau), không giữ khoảng cách toàn cục. Thuật toán có thể đặt các cụm ở các vị trí tương đối tùy ý. Các khoảng cách giữa các cụm trong đồ thị t-SNE không mang ý nghĩa, vì vậy kết luận rằng “A gần B hơn C” từ khoảng cách liên cụm là không đáng tin.

9. K-means thất bại trên hai vòng tròn đồng tâm (cụm hình vòng) vì nó giả định các cụm lồi, hình cầu. Không có khởi tạo nào tách được vòng trong khỏi vòng ngoài. Phân cụm phân cấp liên kết đơn thất bại trên các cụm bị nối bởi một “cầu” điểm nhiễu, vì nó hợp nhất cặp điểm gần nhất,

chuỗi điểm, xích xuyên qua cầu để hợp nhất hai cụm tách biệt (hiệu ứng xích, chaining effect).

10.

(a) Sự tập trung 80% không tự nó mang tính phân biệt đối xử. Nó có thể phản ánh các khuôn mẫu địa lý hoặc nhân khẩu học thực sự. Bạn cần biết liệu các dịch vụ có được phân bổ theo nhu cầu hay theo dân tộc, liệu phân cụm có cho ra kết quả công bằng, và liệu các cách phân cụm khác có cho ra thành phần nhân khẩu học khác nhau hay không.

(b) Thêm dân tộc làm đặc trưng trực tiếp sử dụng một đặc điểm được bảo vệ, điều này vừa rủi ro pháp lý vừa rủi ro đạo đức. Nó thay đổi hàm mục tiêu tối ưu từ "nhóm theo nhu cầu" sang "nhóm theo nhu cầu và sự đa dạng", có thể giảm chất lượng dịch vụ cho mọi nhóm.

(c) So sánh các đặc trưng của cư dân với mọi trọng tâm cụm. Nếu họ gần một trọng tâm khác hơn, phân công có thể chưa tối ưu. Cũng kiểm tra xem cư dân có phải là ngoại lai không được bất kỳ cụm nào phục vụ tốt hay không, điều đó sẽ gợi ý rằng chính hệ thống dựa trên cụm chưa phù hợp với cá nhân này.

Chương 7: Phương pháp hạt nhân

1.

(a) $\mathbf{x}^T \mathbf{z} = 1 \cdot 3 + 2 \cdot 1 = 5$. $\kappa(\mathbf{x}, \mathbf{z}) = (1 + 5)^2 = 36$.

(b) $(1 + x_1 z_1 + x_2 z_2)^2 = 1 + 2x_1 z_1 + 2x_2 z_2 + x_1^2 z_1^2 + 2x_1 x_2 z_1 z_2 + x_2^2 z_2^2$.
 Ảnh xạ đặc trưng hàm ý là $\phi(\mathbf{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, \sqrt{2}x_1 x_2, x_2^2)$,
 một không gian 6 chiều.

2.

(a) $\|\mathbf{x} - \mathbf{z}\|^2 = 1^2 + 1^2 = 2$. $\kappa = e^{-2/2} = e^{-1} \approx 0.368$.

(b) $\kappa(\mathbf{x}, \mathbf{x}) = e^{-0/2\sigma^2} = e^0 = 1$ luôn.

(c) Khi $\sigma \rightarrow 0$, $\kappa \rightarrow 0$ với mọi $\mathbf{x} \neq \mathbf{z}$, và $\kappa = 1$ chỉ khi $\mathbf{x} = \mathbf{z}$. Mỗi điểm huấn luyện trở nên bị cô lập: mô hình ghi nhớ từng điểm một cách riêng lẻ, dẫn tới quá khớp cực đoan.

3.

(a) Nếu $\mathbf{K}_1$ và $\mathbf{K}_2$ là PSD, thì với mọi vectơ $\mathbf{v}$: $\mathbf{v}^T (\mathbf{K}_1 + \mathbf{K}_2) \mathbf{v} = \mathbf{v}^T \mathbf{K}_1 \mathbf{v} + \mathbf{v}^T \mathbf{K}_2 \mathbf{v} \geq 0 + 0 = 0$ vậy $\mathbf{K}_1 + \mathbf{K}_2$ là PSD, nghĩa là $\kappa_1 + \kappa_2$ là một hạt nhân hợp lệ. $\square$

- (b) $\kappa(\mathbf{x}, \mathbf{z}) = |\mathbf{x}^T \mathbf{z}|$ không phải hạt nhân hợp lệ. Với các điểm $\mathbf{x} = (1, 0)$, $\mathbf{y} = (-1, 0)$, $\mathbf{z} = (0, 1)$: $\mathbf{K} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$ Dù ma trận cụ thể này là PSD, việc lấy $\mathbf{x} = (1, 1)$, $\mathbf{y} = (1, -1)$, $\mathbf{z} = (-1, 1)$ cho một ma trận hạt nhân không PSD.

4.

(a) Dự đoán là tổng có trọng số của các độ tương đồng hạt nhân giữa điểm mới và mọi điểm huấn luyện. Các điểm "tương đồng" hơn với $\mathbf{x}$ (theo đo bằng hạt nhân) đóng góp nhiều hơn vào dự đoán. Đây là một dạng khớp khuôn mẫu: các dự đoán mới dựa trên sự tương đồng với các ví dụ đã biết.

(b) Định thức nguyên thủy yêu cầu nghịch đảo một ma trận $d \times d$ ($O(d^3)$), trong khi định thức đối ngẫu yêu cầu nghịch đảo một ma trận $n \times n$ ($O(n^3)$). Khi $d \gg n$, định thức đối ngẫu rẻ hơn nhiều. Hơn nữa, thủ thuật kernel có nghĩa là chúng ta không bao giờ cần tính ánh xạ đặc trưng (có thể vô số chiều) tường minh.

5.

(a) Không tách được tuyến tính. Điểm giữa $(0, 1)$ có nhãn ngược với hai hàng xóm $(-1, -1)$ và $(1, -1)$. Không có đường thẳng $y = wx + b$ nào đặt $(0, 1)$ lên trên và cả $(-1, -1)$ cùng $(1, -1)$ xuống dưới được.

(b) $\phi(x) = (x, x^2)$ ánh xạ thành: $(-1, 1)$, $(0, 0)$, $(1, 1)$. Giờ đường thẳng $x_2 = 0.5$ tách điểm dương $(0, 0)$ (ở dưới) khỏi các điểm âm $(-1, 1)$ và $(1, 1)$ (ở trên). Thực ra, kiểm tra nhãn: $(0) \rightarrow (+1)$ và $(\pm 1) \rightarrow (-1)$, nên $x_2 < 0.5$ cho $+1$, phân loại chính xác mọi điểm.

(c) Một hạt nhân đa thức bậc 2, $\kappa(x, z) = (1 + xz)^2$, hàm ý bao gồm đặc trưng x^2 .

6.

(a) Khi $\lambda \rightarrow \infty$: mọi trọng số đều bị đưa về không. Mô hình dự đoán trung bình của các mục tiêu huấn luyện cho mọi đầu vào (regularization tối đa, thiếu khớp).

(b) Khi $\lambda \rightarrow 0$: không có regularization. Mô hình khớp chính xác dữ liệu huấn luyện, có thể quá khớp.

(c) Dùng kiểm chéo: thử một dải các giá trị λ (ví dụ trên một lưới logarithmic), đánh giá từng giá trị trên tập kiểm định, và chọn λ có hiệu năng kiểm định tốt nhất.

7. Các phương pháp hạt nhân “dựa trên bộ nhớ” vì chúng phải lưu mọi ví dụ huấn luyện và tính các giá trị hạt nhân với từng cái khi dự đoán. Ưu điểm: không có pha huấn luyện tường minh, dễ thêm hoặc bỏ các điểm dữ liệu. Nhược điểm: thời gian dự đoán là $O(n)$ mỗi truy vấn (tỷ lệ với kích thước tập dữ liệu). Ngược lại, các mạng nơ-ron học các tham số trọng số nén trong khi huấn luyện, vì vậy dự đoán là $O(1)$ bất kể kích thước tập huấn luyện, nhưng cập nhật mô hình đòi hỏi huấn luyện lại.

8.

(a) Không đường thẳng nào tách được hai vòng tròn đồng tâm. Một đường thẳng chia mặt phẳng thành hai nửa phẳng, nhưng vòng tròn bên trong bị vòng ngoài bao quanh. Bất kỳ đường thẳng nào bắt được một phần vòng trong cũng sẽ bao gồm một phần vòng ngoài.

(b) Thêm $x_3 = x_1^2 + x_2^2$ (bình phương khoảng cách tới gốc): các điểm vòng trong có $x_3 \approx 1$, các điểm vòng ngoài có $x_3 \approx 4$. Trong không gian 3D, các lớp được tách theo chiều cao. Một mặt phẳng ngang tại $x_3 = 2.5$ tách sạch chúng.

(c) Hạt nhân RBF đo độ tương đồng qua khoảng cách. Các điểm trên cùng một vòng tròn gần nhau và xa các điểm trên vòng còn lại. Hạt nhân hàm ý nâng dữ liệu lên không gian nhiều chiều, nơi hai vòng tròn trở nên tách được tuyến tính, rất giống đặc trưng x_3 tường minh trong phần (b) nhưng linh hoạt hơn.

(d) Khi $\sigma \rightarrow 0$, mỗi điểm chỉ “nhìn thấy” chính nó. Ranh giới quyết định trở thành một vùng nhỏ quanh mỗi điểm huấn luyện, được phân loại theo nhãn của điểm đó. Đây là quá khớp cực đoan: nó ghi nhớ tập huấn luyện hoàn hảo nhưng ranh giới giữa các điểm huấn luyện về cơ bản là ngẫu nhiên, cho độ khái quát hóa kém.

9.

(a) Ma trận hạt nhân cho các điểm $\{1, 2, 3\}$ là: $K = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 2 & 3 \end{pmatrix}$. Giá trị

riêng: đều dương ($\approx 0.20, 0.68, 5.12$). Ma trận là PSD, vì vậy $\min(x, z)$ là một hạt nhân hợp lệ với các đầu vào không âm.

(b) Nó tương ứng với ánh xạ đặc trưng $\phi(x) = (1_{[t \leq x]})_{t \geq 0}$, một hàm chỉ thị trên các số thực dương. Tích trong $\langle \phi(x), \phi(z) \rangle =$

$$\int_0^\infty 1_{[t \leq x]} 1_{[t \leq z]} dt = \min(x, z).$$

Chương 8: Tìm kiếm AI và thỏa mãn ràng buộc

1.

(a) BFS khám phá theo độ sâu: A (độ sâu 0), rồi B, C (độ sâu 1), rồi D (độ sâu 2), rồi E (độ sâu 3). Thứ tự: A, B, C, D, E (với bước hòa: B trước C theo bảng chữ cái).

(b) DFS với bước hòa theo bảng chữ cái: $A \rightarrow B \rightarrow D \rightarrow E$. Tìm thấy mục tiêu ở độ sâu 3 qua đường đi A-B-D-E (chi phí 6).

(c) Đường đi tối ưu: $A \rightarrow C \rightarrow D \rightarrow E$ với chi phí $1 + 2 + 1 = 4$.

2.

(a) $f(A) = 0 + 5 = 5$. Khi B được phát hiện qua A: $f(B) = 2 + 3 = 5$. Khi C được phát hiện qua A: $f(C) = 1 + 4 = 5$. Khi D được phát hiện qua C: $f(D) = 3 + 1 = 4$. Khi E được phát hiện qua D: $f(E) = 4 + 0 = 4$.

(b) Thứ tự A*: mở rộng A ($f = 5$), sinh ra B và C. Mở rộng B hoặc C ($f = 5$ mỗi cái, hòa theo bảng chữ cái: B). B sinh ra D ($f = 5$). Mở rộng C ($f = 5$), sinh ra D ($f = 3 + 1 = 4$, tìm thấy đường đi rẻ hơn). Mở rộng D ($f = 4$), sinh ra E ($f = 4$). Mở rộng E, tìm thấy mục tiêu. Đường đi: A-C-D-E, chi phí 4.

3. Lựa chọn 1: MIN chọn $\min\{7, 3, 9\} = 3$. Lựa chọn 2: MIN chọn $\min\{5, 8, 2\} = 2$. MAX chọn $\max\{3, 2\} = 3$. MAX nên chọn Lựa chọn 1, đảm bảo giá trị là 3.

4. Sau khi đánh giá đầy đủ Lựa chọn 1, $\alpha = 3$ (MAX được đảm bảo ít nhất 3). Khám phá Lựa chọn 2: con đầu tiên có giá trị 5, nên MIN vẫn có thể chọn thứ ≤ 3 . Sau khi thấy con thứ hai (giá trị 8), MIN vẫn có thể chọn thấp hơn. Sau khi thấy con thứ ba (giá trị 2), MIN chọn $\min\{5, 8, 2\} = 2$. Vì $2 < \alpha = 3$, MAX sẽ không bao giờ chọn Lựa chọn 2. Trong cây này, không có cắt tỉa nào xảy ra vì chúng ta phải thấy mọi con của Lựa chọn 2 trước khi giá trị của MIN được xác định. (Cắt tỉa sẽ xảy ra nếu một con sớm của Lựa chọn 2 có giá trị ≤ 3 , vì giá trị của MIN chỉ có thể giảm.)

5. A* "tối ưu hiệu quả" trong các thuật toán dùng cùng heuristic: không thuật toán nào khác tìm được lời giải tối ưu trong khi mở rộng ít nút hơn. Điều này có nghĩa A* khai thác tối đa giá trị từ thông tin heuristic, nó không bao giờ làm việc thừa. Điều này quan trọng vì trong không gian tìm kiếm lớn, ngay cả những cải thiện nhỏ về hiệu quả cũng chuyển thành việc giải các bài toán vốn bất khả thi.

6.

(a) Các ô góc có 2 nước đi, các ô biên (không góc) có 3, các ô nội bộ có 4.

Với 4 ô góc, 12 ô biên, và 9 ô nội bộ: hệ số phân nhánh trung bình

$$\approx (4 \times 2 + 12 \times 3 + 9 \times 4) / 25 = 80 / 25 = 3.2.$$

(b) Khoảng cách Manhattan khả chấp: nó tính đường đi ngắn nhất giả định không có chướng ngại vật (thẳng ngang + dọc). Vì chướng ngại vật chỉ có thể làm đường đi thật dài hơn, khoảng cách Manhattan không bao giờ ước lượng quá chi phí thật. Vì vậy $h(n) \leq h^*(n)$ với mọi n , thỏa mãn tính khả chấp.

7.

(a) Tổng số cách gán: $3^4 = 81$ (3 màu cho mỗi 4 khu vực).

(b) Gán A = Màu đỏ loại bỏ Màu đỏ khỏi miền của B và C (vì $A-B$ và $A-C$): $B : \{G, B\}, C : \{G, B\}, D : \{R, G, B\}$.

(c) Gán B = Màu xanh lá loại bỏ Màu xanh lá khỏi A (đã gán), C , và D : $C : \{B\}$ (chỉ còn Màu xanh dương), $D : \{R, B\}$ (Màu xanh lá bị loại qua tính kề B-D). Một cách hoàn thành hợp lệ là C = Màu xanh dương, D = Màu đỏ.

Chương 9: Học tăng cường

1. Các phương trình tối ưu Bellman:

$$V^*(A) = \max\{1 + 0.9 \cdot V^*(A), 0 + 0.9 \cdot V^*(B)\}$$

$$V^*(B) = \max\{2 + 0.9 \cdot V^*(A), 1 + 0.9 \cdot V^*(B)\}$$

Thử $A \rightarrow A$ và $B \rightarrow A$: $V^*(A) = 1 + 0.9V^*(A)$, nên $V^*(A) = 10$.

$V^*(B) = 2 + 0.9 \times 10 = 11$. Kiểm tra $B \rightarrow B$:

$1 + 0.9V^*(B) = 1 + 9.9 = 10.9 < 11$. Vậy tối ưu: ở lại A ($V^* = 10$), đi A từ B ($V^* = 11$).

$$2. Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$$= 5 + 0.1[2 + 0.9 \times 8 - 5] = 5 + 0.1[2 + 7.2 - 5] = 5 + 0.1 \times 4.2 = 5.42.$$

3. Luôn chọn hành động đã biết tốt nhất (khai thác) có nguy cơ bỏ lỡ các phương án tốt hơn chưa từng thử. Các ước lượng ban đầu có thể không chính xác, vì vậy một hành động có vẻ chưa tối ưu thực ra có thể ưu việt hơn. Thăm dò (thử các hành động ít được thăm dò hơn) là cần thiết để thu thập thông tin và tránh hội tụ vào một chính sách tốt ở cục bộ nhưng chưa tối ưu toàn cục.

4. Khi $\gamma \rightarrow 0$: chỉ phần thưởng tức thì quan trọng. Đại lý trở nên hoàn toàn cận thị, chọn hành động nào cho tức thì cao nhất. Khi $\gamma \rightarrow 1$: mọi phần thưởng tương lai được trọng số hóa bình đẳng. Đại lý lập kế hoạch cho dài hạn, nhưng chân trời vô hạn có thể làm các hàm giá trị phân kỳ nếu phần thưởng không bị chặn, và hội tụ trở nên chậm hơn.

5. Bắt đầu từ $Q^\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$:

Với chính sách tối ưu π^* , hành động A_{t+1} được chọn để cực đại hóa Q^* . Thay vào: $Q^*(s, a) = \mathbb{E}[R_{t+1} + \gamma \max_{a'} Q^*(S_{t+1}, a') | S_t = s, A_t = a]$. Mở kỳ vọng theo phép chuyển tiếp: $Q^*(s, a) = \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$. Đây là phương trình Bellman tối ưu cho Q^* .

6.

(a) $Q^*(s_0, A) = 10$ (phần thưởng tức thì, rồi kết thúc). $Q^*(s_0, B) = 0 + 0.9 \times 5 = 4.5$. $Q^*(s_0, C) = 0 + 0.9 \times 20 = 18$. Hành động tối ưu là C.

(b) Một đại lý tham lam chọn hành động có Q-value đã biết cao nhất. Chỉ mới thử A (phần thưởng +10), ước lượng Q của nó cho A là 10 trong khi B và C vẫn ở giá trị khởi tạo (thường là 0). Vì $10 > 0$, tham lam luôn chọn A, không bao giờ phát hiện hành động ưu việt C.

(c) Với $\epsilon = 0.1$, mỗi episode đại lý thăm dò ngẫu nhiên với xác suất 0.1. Trong 1000 episode: ~ 900 lần nó chọn tham lam (hành động A), ~ 100 lần ngẫu nhiên (~ 33 lần mỗi hành động A, B, C). Vì vậy đại khái: $A \approx 933$, $B \approx 33$, $C \approx 33$. Sau ~ 33 lần thử C (nhận phần thưởng 18 qua trạng thái Y), đại lý sẽ học được rằng C ưu việt hơn, và hành động tham lam sẽ chuyển sang C. Có, nó cuối cùng sẽ học được chính sách tối ưu.

(d) Khởi tạo Q-values lạc quan (ví dụ $Q_0 = 100$ cho mọi hành động). Sau đó chính sách tham lam tự nhiên khám phá: sau khi thử A và được 10 (ít hơn 100), đại lý thử B, rồi C. Sau mỗi lần thử một lần, nó phát hiện C cho 18 và hội tụ vào chính sách tối ưu nhanh hơn nhiều.

7.(a) Trạng thái: 16 ô (lưới 4×4), nhưng (2,2) là trạng thái kết thúc (hố) và (4,4) là trạng thái kết thúc (mục tiêu), nên có 14 trạng thái không kết thúc với các quyết định. Mỗi trạng thái có tới 4 hành động. Các chính sách xác định có thể có: $4^{14} = 268,435,456$ (trên; các trạng thái biên/góc có ít hành động hơn).

(b) Với $\gamma = 0.9$ và -1 mỗi bước, một đường đi an toàn 6 bước kiếm được khoảng $-6 + 0.9^6 \times 10 \approx -6 + 5.3 = -0.7$. Một đường đi mạo hiểm 3 bước xuyên qua khu vực lân cận của (2,2) kiếm được khoảng $-3 + 0.9^3 \times 10 \approx -3 + 7.3 = 4.3$ nếu nó tránh được hố, nhưng $-3 + 0.9^2 \times (-10) = -11.1$ nếu nó rơi vào. Đại lý sẽ ưa một đường đi an toàn vừa phải, tránh hố nhưng không đi đường vòng quá dài.

(c) $Q((1, 1), \text{phải}) \leftarrow Q((1, 1), \text{phải}) + \alpha [-1 + \gamma \max_{a'} Q((2, 1), a') - Q((1, 1), \text{phải})]$.

Chương 10: Mô hình tạo sinh và Học sâu

1. Các mô hình tạo sinh học phân bố đồng thời $P(X, Y)$ và có thể tạo dữ liệu mới. Ví dụ: một mô hình học hình dạng của từng chữ số và có thể tạo ra hình ảnh chữ số mới. Các mô hình phân biệt học phân bố có điều kiện $P(Y|X)$ và chỉ phân loại. Ví dụ: một hồi quy logistic hoặc CNN nhận hình ảnh chữ số và xuất nhãn chữ số. Mô hình tạo sinh có thể tạo hình ảnh; mô hình phân biệt chỉ có thể gán nhãn cho chúng.

2.

(a) Tỷ lệ nén: $784/32 = 24.5\times$.

(b) Không gian tiềm ẩn của autoencoder “thuần” không được cấu trúc cho việc tạo sinh. Các vectơ ngẫu nhiên nhiều khả năng rơi vào “vùng chết” mà bộ giải mã không bao giờ được huấn luyện để xử lý, cho ra đầu ra rác. Bộ mã hóa ánh xạ các ảnh huấn luyện tới các điểm rải rác cụ thể, không phải tới một vùng trơn tru, liên tục.

3.(a) Số hạng đầu tiên (mất mát tái dựng) khuyến khích bộ giải mã tái dựng đầu vào một cách trung thành. Số hạng thứ hai (độ phân kỳ KL) khuyến khích phân bố tiềm ẩn của bộ mã hóa gần phân bố chuẩn, đảm bảo không gian tiềm ẩn trơn tru và có cấu trúc tốt.

(b) Nếu $\beta = 0$: VAE trở thành autoencoder tiêu chuẩn (tái dựng xuất sắc nhưng không gian tiềm ẩn không có cấu trúc, tạo sinh kém). Nếu $\beta \rightarrow \infty$: số hạng KL chi phối, buộc $q(z|x) \approx \mathcal{N}(0, I)$. Bộ mã hóa bỏ qua hoàn toàn đầu vào, tái dựng sụp đổ, nhưng không gian tiềm ẩn được cấu trúc hoàn hảo (và vô dụng).

(c) Không có số hạng KL, bộ mã hóa có thể ánh xạ các đầu vào khác nhau tới các vùng xa, bị cô lập của không gian tiềm ẩn. Lấy mẫu một z ngẫu nhiên để tạo sinh sẽ rơi vào giữa các vùng đó, nơi bộ giải mã không có tín hiệu huấn luyện. Số hạng KL buộc mọi mã hóa về phía cùng một phân bố tiên nghiệm, lấp đầy khoảng trống và cho phép nội suy mượt mà và lấy mẫu.

4.(a) Mô hình 10.000 ảnh đã ghi nhớ dữ liệu huấn luyện của nó (quá khớp). Với dữ liệu hạn chế, mô hình thiếu đủ ví dụ để học khái niệm chung về “khuôn mặt” và thay vì thế lưu trữ các khuôn mặt cụ thể. Mô hình 10 triệu ảnh có đủ độ đa dạng để học các đặc trưng có thể khái quát hóa.

(b) So sánh đầu ra được tạo với các hàng xóm gần nhất trong tập huấn luyện.

Nếu các hình ảnh được tạo giống từng điểm ảnh với các ví dụ huấn luyện, mô hình đang ghi nhớ. Một mô hình khái quát hóa nên tạo ra các đầu ra hợp lý nhưng không giống hệt bất kỳ ví dụ huấn luyện nào. Biến đổi định lượng: tính điểm FID, hoặc kiểm tra xem khoảng cách tối thiểu tới các ví dụ huấn luyện có vượt một ngưỡng hay không.

5. Các VAE tạo ra hình ảnh mờ vì mất mát tái dựng (thường là MSE) phạt độ lệch từng điểm ảnh. Khi có sự không chắc chắn về các chi tiết tinh tế (ví dụ: vị trí chính xác của một sợi tóc), chiến lược cực tiểu hóa mất mát là tạo ra trung bình của mọi đầu ra hợp lý, tức là một hỗn hợp mờ. MSE coi một hình ảnh sắc nét nhưng sai tệ hơn một hình ảnh mờ nhưng ở mức trung bình, vì vậy mô hình học cách phòng thủ bằng cách làm mờ.

Chương 11: Lý thuyết học thống kê

1. Dùng giới hạn PAC của lớp hữu hạn:

$$m \geq \frac{1}{\epsilon} (\ln |\mathcal{H}| + \ln(1/\delta)) = \frac{1}{0.1} (\ln 1000 + \ln(1/0.05)) = 10(6.91 + 3.00) = 99.1. \text{ Vậy khoảng 100 mẫu là cần thiết.}$$

2. Những con bồ câu nhận thức ăn ở các khoảng thời gian ngẫu nhiên không liên quan tới hành vi của chúng, nhưng chúng lặp lại bất kỳ hành vi nào đứng trước phần thưởng. Chúng không có kiến thức tiên nghiệm để phân biệt trùng hợp với nhân quả. Những con chuột có kiến thức tiên nghiệm về thế giới vật lý (đề vào thứ gì đó có thể gây ra hệ quả), nên chúng xác định đúng đôn bầy là nguyên nhân. Kiến thức tiên nghiệm (thiên kiến quy nạp) là thiết yếu để học được giả thuyết đúng đắn từ bằng chứng hạn chế.

3. Các hình chữ nhật đại diện cho một thiên kiến quy nạp cụ thể: chúng ta giả định các quả đu đủ tốt chiếm một vùng hình chữ nhật liên mạch trong không gian đặc trưng. Nếu chúng ta dùng mọi hàm có thể có, lớp giả thuyết sẽ là vô hạn và không bị ràng buộc, khiến việc học trở nên bất khả thi (chúng ta có thể khớp bất kỳ cách gán nhãn nào cho bất kỳ dữ liệu nào, nhưng không khái quát hóa được gì). Giới hạn hình chữ nhật đánh đổi tính biểu đạt lấy khả năng học.

4. Độ phức tạp mẫu tỷ lệ với $O(1/\epsilon)$: giảm ϵ một nửa thì số mẫu cần thiết tăng gấp đôi. Chuyển từ $\epsilon = 0.1$ sang $\epsilon = 0.05$ xấp xỉ tăng gấp đôi yêu cầu mẫu (từ ~ 100 tới ~ 200 cho cùng δ và $|\mathcal{H}|$).

5. Một người học nhất quán tìm được giả thuyết có sai số huấn luyện bằng không. Dưới giả định thực hiện được, hàm thật h^* thuộc $\mathcal{H}$ và cũng có sai số huấn luyện bằng không. Giới hạn PAC đảm bảo rằng với đủ mẫu, mọi giả thuyết có sai số huấn luyện bằng không phải có sai số thật thấp. Vì vậy đầu ra của người học nhất quán "gần" h^* về mặt sai số. Ý chính: sai số huấn luyện bằng không trên đủ dữ liệu hàm ý sai số thật thấp, với điều kiện lớp giả thuyết không quá lớn.

6. Giải $\epsilon \geq \frac{\ln |\mathcal{H}| + \ln(1/\delta)}{m}$: với $m = 500$, $|\mathcal{H}| = 1000$, $\delta = 0.05$:
 $\epsilon \geq \frac{\ln 1000 + \ln 20}{500} = \frac{6.91 + 3.00}{500} = \frac{9.91}{500} \approx 0.02$ Vậy với 500 mẫu, sai số khái quát hóa là tối đa khoảng 2%.

Chương 12: Lý thuyết học thống kê nâng cao

1. VCdim(intervals) = 2. Có thể shatter 2 điểm $a < b$: gán nhãn (+, +) với khoảng $[a, b]$; (+, -) với $[a, a + \epsilon]$; (-, +) với $[b - \epsilon, b]$; (-, -) với một khoảng không giao nhau. Không thể shatter 3 điểm $a < b < c$: cách gán nhãn (+, -, +) yêu cầu hai khoảng rời nhau, nhưng lớp giả thuyết chỉ cho phép một khoảng.

2. Decision stump trong $\mathbb{R}^2$ chia theo một tọa độ: $1[x_i \geq \theta]$ cho $i \in \{1, 2\}$. Mỗi bộ phân loại như vậy tạo ranh giới nửa phẳng thẳng hàng với một trục. VCdim = 2. Có thể shatter 2 điểm có giá trị khác nhau trên một tọa độ. Không thể shatter 3 điểm: với bất kỳ sắp xếp nào, đều tồn tại một cách gán nhãn yêu cầu ranh giới không thẳng hàng với trục.

3. Dùng giới hạn VC: $m = O\left(\frac{1}{\epsilon} (\text{VCdim} \cdot \log(1/\epsilon) + \log(1/\delta))\right)$. Với VCdim = 10, $\epsilon = 0.05$, $\delta = 0.01$: $m \approx \frac{1}{0.05} (10 \cdot \log(20) + \log(100)) = 20(10 \times 3.0 + 4.6) = 20 \times 34.6 \approx 692$ mẫu.

4. NFL phát biểu rằng không người học đơn lẻ nào có thể vượt trội hơn mọi người học khác trên mọi phân bố có thể có. Nhưng trong thực tế, phân bố dữ liệu thế giới thực không phải tùy ý. Chúng thể hiện cấu trúc (tính trơn tru, số chiều thấp, tính hợp thành) mà các thuật toán cụ thể có thể khai thác. Machine learning thành công vì chúng ta chọn các thuật toán có thiên kiến quy nạp khớp với cấu trúc của dữ liệu thật, dù không thuật toán nào hoạt động trên mọi phân bố có thể tưởng tượng được.

5. Khoảng cách 99% huấn luyện / 70% kiểm tra cho thấy quá khớp: lớp giả thuyết quá biểu đạt so với dữ liệu huấn luyện. Theo chiều VC, chiều VC hiệu dụng của mô hình quá cao so với kích thước mẫu có sẵn, vì vậy giới hạn khái quát hóa rất lỏng. Cách khắc phục: giảm độ phức tạp mô hình (hạ chiều VC), thêm regularization (ràng buộc vào một lớp giả thuyết hiệu dụng nhỏ hơn), hoặc tăng dữ liệu huấn luyện (siết chặt giới hạn khái quát hóa).

6. Các bộ phân loại tuyến tính trong $\mathbb{R}^2$ có VCdim = 3, vì vậy chúng có thể shatter 3 điểm nhưng không phải 4. Với 4 điểm, xét cấu hình XOR: (0, 0) và (1, 1) được gán +, (0, 1) và (1, 0) được gán -. Không đường thẳng đơn lẻ nào tách được hai điểm + (trên một đường chéo) khỏi hai điểm - (trên đường chéo còn lại).

7.

(a) Có thể shatter 3 điểm. Đặt chúng theo bố cục tam giác: bất kỳ cách gán nhãn nào có tối đa một điểm âm đều đạt được bằng hình chữ nhật có kích thước/vị trí

phù hợp.

(b) Có thể shatter 4 điểm. Đặt chúng ở các cực trên, dưới, trái, phải của một hình thoi (ví dụ $(0, 1)$, $(0, -1)$, $(-1, 0)$, $(1, 0)$). Bất kỳ cách gán nhãn nào mà các điểm dương tạo thành một nhóm “liền mạch” đều có thể được một hình chữ nhật bắt được. Thực ra, tất cả $2^4 = 16$ cách gán nhãn đều đạt được.

(c) Không thể shatter 5 điểm. Với bất kỳ 5 điểm nào, xét điểm có tọa độ x nhỏ nhất, tọa độ x lớn nhất, tọa độ y nhỏ nhất, và tọa độ y lớn nhất. Bất kỳ hình chữ nhật nào chứa các điểm cực trị phải chứa điểm nội bộ. Cách gán nhãn mà điểm nội bộ là $-$ và cả bốn điểm cực trị là $+$ đạt được được, nhưng cách gán nhãn mà chỉ điểm nội bộ là $+$ và mọi điểm cực trị là $-$ cũng đạt được. Tuy nhiên, cách gán nhãn $(+, -, +, -, +)$ được sắp xếp sao cho các điểm âm nằm giữa các điểm cực dương không thể được một hình chữ nhật duy nhất bắt được. VCdim = 4.

(d) VCdim = 4.

(e) With VCdim = 4, $m = 500$, $\delta = 0.05$: $\epsilon \leq O\left(\sqrt{\frac{4 \cdot \log(500) + \log(1/0.05)}{500}}\right) \approx O\left(\sqrt{\frac{4 \times 6.2 + 3.0}{500}}\right) = O\left(\sqrt{\frac{27.8}{500}}\right) \approx O(0.24)$.

8.

(a) NFL áp dụng cho phân bố đồng đều trên mọi hàm mục tiêu có thể có. Các bài toán thế giới thực không được lấy ra từ phân bố này. Các mạng nơ-ron thành công vì dữ liệu thế giới thực có cấu trúc có thể khai thác (đặc trưng phân cấp, tương quan cục bộ, tính hợp thành) khớp với thiên kiến quy nạp của mạng. NFL không nói rằng các mạng nơ-ron là tốt nhất phổ quát; nó nói rằng chúng hoạt động tốt trên các phân bố không tùy ý mà chúng ta thực sự gặp.

(b) NFL cho chúng ta thấy rằng việc chọn thuật toán học thực chất là chọn thiên kiến quy nạp. Chúng ta nên chọn các thuật toán có giả định phù hợp với kiến thức chuyên môn của mình. Không thuật toán nào “miễn phí”: mọi ưu điểm trên một phân bố nào đó đều kèm theo nhược điểm trên một phân bố khác. Điều này biện minh cho việc chọn thuật toán chuyên biệt theo lĩnh vực và kiểm chứng mô hình.

Chương 13: Phân tích ma trận: Một ý tưởng thống nhất

1.

(a) Các tham số: $1,000,000 \times 50 + 100,000 \times 50 = 50,000,000 + 5,000,000 = 55,000,000$ (55 triệu).

(b) Ma trận đầy đủ: $1,000,000 \times 100,000 = 10^{11}$ phần tử. Nén: $55,000,000/10^{11} = 0.055\%$. Hơn $1800\times$ mức nén.

(c) Các phần tử quan sát được: $0.001 \times 10^{11} = 10^8 = 100,000,000$. Tỷ lệ: 100 triệu quan sát ràng buộc 55 triệu tham số. Chỉ dưới 2 quan sát mỗi tham số, mức chặt chẽ nhưng khả thi với regularization.

$$2. \hat{r}_1 = \mathbf{p} \cdot \mathbf{q}_1 = 0.8 \times 0.5 + (-0.3) \times 0.2 + 0.5 \times 0.4 = 0.4 - 0.06 + 0.2 = 0.54.$$

$$\hat{r}_2 = \mathbf{p} \cdot \mathbf{q}_2 = 0.8 \times 0.1 + (-0.3) \times (-0.5) + 0.5 \times 0.8 = 0.08 + 0.15 + 0.4 = 0.63.$$

Mục 2 xếp hạng cao hơn ($0.63 > 0.54$).

3. TF-IDF cho "neural" trong một tài liệu có 200 từ: $TF = 3/200 = 0.015$. $IDF = \log(10,000/50) = \log(200) \approx 2.30$. $TF-IDF = 0.015 \times 2.30 \approx 0.035$.

4. Độ tương đồng cosine giữa các người dùng (trên các phim A, B, D, E được cả hai đánh giá):

- Người dùng 1: (5, 4, 2, 1), Người dùng 2: (4, 5, 1, 2)
- Tích vô hướng: $20 + 20 + 2 + 2 = 44$
- Magnitudes: $\sqrt{25 + 16 + 4 + 1} = \sqrt{46}$, $\sqrt{16 + 25 + 1 + 4} = \sqrt{46}$
- Cosine: $44/46 \approx 0.957$ (rất tương đồng)

5. Khởi động lạnh: người dùng mới không có lịch sử tương tác, và mục mới không có đánh giá. Lọc dựa trên nội dung giúp với mục mới (dùng siêu dữ liệu/đặc trưng của mục để tìm các mục tương tự) nhưng không giúp với người dùng mới (vẫn cần một số lịch sử người dùng để dựng hồ sơ). Lọc cộng tác gặp khó với cả hai: nó hoàn toàn phụ thuộc ma trận tương tác, vốn rỗng đối với các thực thể mới. Các giải pháp phổ biến: hệ thống lai, hỏi người dùng mới về sở thích ban đầu, hoặc dùng thông tin nhân khẩu học làm biến thay thế.

6. Netflix: 200 triệu người dùng $\times$ 100 yếu tố + 15K mục $\times$ 100 yếu tố = $20,000,000,000 + 1,500,000 \approx 20$ tỷ tham số. Ma trận đầy đủ: $200M \times 15K = 3 \times 10^{12}$ phần tử. Phân tích ma trận lưu $\sim 20B / 3 \times 10^{12} < 1\%$ của ma trận đầy đủ. Điều này khả thi về lưu trữ và tính toán; ma trận đầy đủ thì không.

Chương 14: Học sâu hiện đại

1. Các mô hình khuếch tán thêm nhiễu vào dữ liệu dần dần cho tới khi nó trở thành nhiễu thuần, rồi huấn luyện một mạng nơ-ron để đảo ngược từng bước. Tạo sinh tiến hành bằng cách bắt đầu từ nhiễu ngẫu nhiên và khử nhiễu một cách lặp lại. Việc tinh chỉnh

lặp lại cho kết quả tốt hơn vì mỗi bước thực hiện một hiệu chỉnh nhỏ, cho phép mô hình sửa lỗi ở mọi giai đoạn. Một bộ tạo sinh một lần phải làm đúng mọi thứ trong một lượt, không có cơ hội sửa sai, khiến việc tạo ra các chi tiết tinh tế khó hơn.

2.

(a) Query (truy vấn) đại diện cho “tôi đang tìm gì?”. Key (khóa) đại diện cho “tôi chứa gì?”. Value (giá trị) đại diện cho “tôi cung cấp thông tin gì nếu được chọn?”. Attention tính xem mức độ Key của mỗi vị trí khớp với từng Query, rồi dùng các trọng số đó để kết hợp các Value.

(b) Attention cho phép mọi vị trí trong một chuỗi tương tác trực tiếp với mọi vị trí khác trong một lớp, bất kể khoảng cách. Các kiến trúc trước (RNN) xử lý chuỗi từng bước, khiến các phụ thuộc tầm xa khó học. Attention loại bỏ nút thắt này, đó là lý do nó là “tất cả những gì bạn cần”.

(c) Self-attention tính các điểm số cặp giữa mọi n vị trí: $O(n^2 \cdot d)$ nơi d là chiều của embedding. Số hạng n^2 làm nó tốn kém cho các chuỗi dài.

3. Mỗi tham số yêu cầu 16 bit = 2 byte. Bộ nhớ tổng: $175 \times 10^9 \times 2 = 350 \times 10^9$ byte = 350 GB. Điều này vượt bộ nhớ của bất kỳ GPU tiêu dùng đơn lẻ nào (thường 16–80 GB), đòi hỏi song song mô hình trên nhiều thiết bị.

4. Dự đoán token kế tiếp yêu cầu mô hình hiểu cấu trúc của văn bản ở mọi cấp độ. Để dự đoán từ tiếp theo trong một chứng minh toán học, mô hình phải hiểu toán học. Để dự đoán hội thoại, nó phải mô hình hóa cuộc trò chuyện. Để dự đoán mã nguồn, nó phải hiểu lập trình. Hàm mục tiêu huấn luyện đơn giản, nhưng thành thạo nó trên toàn bộ sự đa dạng của văn bản trên internet đòi hỏi học được một biểu diễn nén của tri thức con người. Trả lời câu hỏi và hội thoại nổi lên vì các khuôn mẫu đó xuất hiện trong dữ liệu huấn luyện và dự đoán chúng đòi hỏi phải hiểu chúng.

5.

(a) Hàm mục tiêu huấn luyện thưởng cho việc tạo văn bản có xác suất thống kê cao trong ngữ cảnh, không phải văn bản đúng về mặt sự thật. Nếu các phát biểu sai nghe hợp lý xuất hiện trong dữ liệu huấn luyện (hoặc nếu mô hình nội suy giữa các sự thật thật), mô hình học cách tạo chúng một cách tự tin. Hàm mất mát không phân biệt đúng với sai, chỉ phân biệt khả năng với bất khả năng.

(b) Hai cách tiếp cận: (1) Sinh ra được tăng cường bằng truy xuất (RAG): cho mô hình quyền truy cập vào một cơ sở kiến thức đã được xác minh và huấn luyện nó để trích dẫn nguồn,

neo đầu ra vào các sự thật. (2) Học tăng cường từ phản hồi con người (RLHF): huấn luyện mô hình để đưa các câu trả lời thận trọng, chính xác hơn các câu tự tin nhưng sai, dạy nó biểu lộ sự không chắc chắn khi phù hợp.

6.

(a) $D(x) = 0.9$ nghĩa là bộ phân biệt ước lượng xác suất 90% rằng x là ảnh thật (từ dữ liệu huấn luyện) thay vì ảnh được tạo.

(b) Bộ tạo muốn bộ phân biệt tin rằng đầu ra của nó là thật. Cực đại hóa $D(G(z))$ có nghĩa đẩy đầu ra của bộ phân biệt về phía 1 ("chắc chắn thật") cho các mẫu được tạo, đúng nghĩa đen của việc lừa được bộ phân biệt.

(c) Sụp đổ mode xảy ra khi bộ tạo chỉ tạo ra một tập con nhỏ các đầu ra có thể có (ví dụ: luôn tạo cùng một chữ số). Nó xảy ra vì bộ tạo tìm thấy vài đầu ra lừa được bộ phân biệt một cách đáng tin cậy và hội tụ vào chúng, thay vì khám phá đầy đủ phân bố dữ liệu. Bộ phân biệt sau đó học cách từ chối chính các đầu ra cụ thể đó, nhưng bộ tạo có thể đơn giản chuyển sang một tập nhỏ khác, xoay vòng mà không bao giờ phủ đầy phân bố.

These solutions are meant as guides. For many problems, especially conceptual ones, alternative correct answers exist. Focus on understanding the reasoning, not memorizing specific answers.

Phụ lục I

Một bài hát: Tôi Vẫn Là Của Mình

Lời và nhạc của Eljakim Schrijvers¹

Phụ lục này mang tính cá nhân, không mang tính hướng dẫn. Nó ở đây vì tôi tin rằng mọi thứ tôi yêu đều xứng đáng có một bài hát.



Scan to listen to the melody

© 2026 Eljakim Schrijvers. All rights reserved.

¹This song was actually hand written. It's not AI-generated.

I am Still Mine

(and therefore yours to hold)

Lyrics/music: Eljakim Schrijvers

Andantino

Voice

Clarinet in B $\flat$

Piano

3

When did my choice slip a - way? The food I want, the things I say My

B $\flat$ Cl.

Pno.

7

world is shaped be - fore I wake, By pat - terns that I did - n't make.

B $\flat$ Cl.

Pno.

11

It writes my words to sound much bet - ter, Check e - very word and e - very let - ter. It

B $\flat$ Cl.

Pno.

15

hears my laugh, pre-dicts my cry, When I text "hey" is that me___ or A.I.?___ Am I

B♭ Cl.

Pno.

cresc.

19 **A**

me, or is this some-one new? My re - flec-tion___ speaks be-fore I do I sing a -

B♭ Cl.

Pno.

mf

f

23

long to words that no one wrote Am I still mine, or just what it fore-told?

B♭ Cl.

Pno.

27 **B**

It claims my fu-ture, piece by piece, It writes my texts, pre-dicts who I'll meet. It

B♭ Cl.

Pno.

p

mp

31

knows my words be-fore I choose. But do I choose or just ap-prove?

B♭ Cl.

Pno.

35 C

The words it writes are al-ways right, Too smooth, too clean, too polished bright.

B♭ Cl.

Pno.

mf

39

What if my own words fail the test? What if the al-go-ri-thm knows me best? Am I

B♭ Cl.

Pno.

cresc.

43 D

me, or is this some-one new? My re-flec-tion speaks be-fore I do I sing a-

B♭ Cl.

Pno.

f

cresc.

47

long to words that no one wrote Am I still mine, or just what it con-trols? What if I'm

B♭ Cl.

Pno.

51 E

lost with-out it guiding me? What if I choose wrong and I can't see? I

B♭ Cl.

Pno.

mf

55

shut it down and break the chain If I fall, — at least I'll feel the pain

B♭ Cl.

Pno.

p

59 F

Sonow I wake with-out the morn - ing brief, I make my tea wrong it's a re - lief. I'd

B♭ Cl.

Pno.

pp

63

ra-ther love though flawed and real, Than per-fect words that ne-ver let me feel. I am still

B♭ Cl.

Pno.

mp

mf

67 G

me, not e-choes on re - peat I'll walk the wind - ing road be-neath my feet I'll write my

B♭ Cl.

Pno.

mf

71

stumbling lines, im-per-fect but bold I am still mine, and there-fore yours to

B♭ Cl.

Pno.

mf

75

hold La la la la la la La la la la la la La la la la la la

B♭ Cl.

Pno.

f

79

Ohh Ah Ah Ah

B♭ Cl.

Pno.

83

B♭ Cl.

Pno.